



IBM PowerPC 970MP RISC Microprocessor

User's Manual

Version 2.3

March 7, 2008



© Copyright International Business Machines Corporation 2005, 2008

All Rights Reserved

Printed in the United States of America March 2008

The following are trademarks of International Business Machines Corporation in the United States, or other countries, or both:

IBM	POWER	PowerPC
IBM Logo	Power Architecture	PowerPC Architecture
ibm.com		

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both.

Other company, product, and service names may be trademarks or service marks of others.

All information contained in this document is subject to change without notice. The products described in this document are NOT intended for use in applications such as implantation, life support, or other hazardous uses where malfunction could result in death, bodily injury, or catastrophic property damage. The information contained in this document does not affect or change IBM product specifications or warranties. Nothing in this document shall operate as an express or implied license or indemnity under the intellectual property rights of IBM or third parties. All information contained in this document was obtained in specific environments, and is presented as an illustration. The results obtained in other operating environments may vary.

THE INFORMATION CONTAINED IN THIS DOCUMENT IS PROVIDED ON AN "AS IS" BASIS. In no event will IBM be liable for damages arising directly or indirectly from any use of the information contained in this document.

IBM Systems and Technology Group
2070 Route 52, Bldg. 330
Hopewell Junction, NY 12533-6351

The IBM home page can be found at ibm.com®

The IBM Semiconductor solutions home page can be found at ibm.com/chips

Version 2.3
March 7, 2008

Contents

List of Figures	13
List of Tables	15
Revision Log	19
About This Book	21
Audience	21
Organization	21
Related Documents	22
Companion Manuals	22
Additional Documentation	23
General PowerPC Documentation	24
Conventions	25
Acronyms and Abbreviations	26
Terminology Conventions	31
1. PowerPC 970MP Overview	33
1.1 PowerPC 970MP Microprocessor Overview	33
1.2 PowerPC 970MP Functional Units	36
1.2.1 Introduction	36
1.2.1.1 Key Design Fundamentals of the Microprocessor Core	36
1.2.1.2 Detailed Features of the Microprocessor Core	37
1.3 970MP Dual-Core Module	41
2. Programming Model	43
2.1 970MP Processor Register Set	43
2.1.1 Architected Registers in the 970MP Implementation	49
2.1.1.1 MSR Register (MSR)	49
2.1.1.2 Machine Status Save/Restore Register (SRR1)	49
2.1.1.3 Time Base and Decrementer (TB, DEC)	50
2.1.1.4 Processor ID Register (PIR)	50
2.1.2 PowerPC 970MP-Specific Registers	50
2.1.2.1 Move To and Move From System Register Instructions	50
2.1.2.2 HID Registers (HID0, HID1, HID4, and HID5)	54
2.1.2.3 Data Address Breakpoint Register (DABR)	61
2.1.2.4 Instruction Address Breakpoint Register (IABR)	62
2.1.2.5 Instruction Match CAM Array Access Register (IMC)	62
2.1.2.6 Performance Monitor Registers (MMCR0, MMCR1, MMCR2, PMC1-8)	63
2.1.2.7 Sampled Instruction Address and Sampled Data Address Registers (SIAR, SDAR)	64
2.1.2.8 Scan Communication Registers (SCOMC and SCOMD)	64
2.1.2.9 Hypervisor Decrementer Interrupt Register (HDEC)	65
2.1.2.10 Hypervisor Save/Restore Register (HSRR0, HSRR1)	65
2.1.2.11 Hypervisor SPRGs (HSPRG0, HSPRG1)	65

IBM PowerPC 970MP RISC Microprocessor

2.1.2.12 Trigger Registers (TRIG0, TRIG1, TRIG2)	65
2.1.2.13 Hardware Interrupt Offset Register (HIOR)	66
2.2 Instruction Set Summary	67
2.2.1 Classes of Instructions	67
2.2.1.1 Definition of Boundedly Undefined	68
2.2.1.2 Defined Instructions	68
2.2.1.3 Illegal Instructions	69
2.2.1.4 Reserved Instructions	69
2.2.2 Instruction Set Overview	69
2.2.3 Fixed-Point Processor	70
2.2.3.1 Fixed-Point Arithmetic and Compare Instructions	70
2.2.3.2 Fixed-Point Logical, Rotate, and Shift Instructions	70
2.2.3.3 Move to and Move from System Register Instructions	70
2.2.3.4 Move to and Move from Machine State Register	70
2.2.3.5 Fixed-Point Invalid Forms and Undefined Conditions	71
2.2.4 Floating-Point Processor	72
2.2.4.1 Floating-Point Arithmetic Instructions	72
2.2.4.2 Floating-Point Invalid Forms and Undefined Conditions	72
2.2.5 Vector Processor	72
2.2.6 Load Store Processor	73
2.2.6.1 Floating-Point Load-and-Store Instructions	73
2.2.6.2 Fixed-Point Load Instructions	73
2.2.6.3 Fixed-Point Store Instructions	73
2.2.6.4 Fixed-Point Load-and-Store Multiple Instructions	73
2.2.6.5 Fixed-Point Load-and-Store String Instructions	74
2.2.6.6 Load/Store Invalid Forms and Undefined Conditions	75
2.2.7 Branch Processor	76
2.2.7.1 Branch Processor Instructions	76
2.2.7.2 Branch Processor Instructions with Undefined Results	76
2.2.7.3 Move To Condition Register Fields Instruction	77
2.2.8 Storage Control Instructions	77
2.2.8.1 Key Aspects of Storage Control Instructions	77
2.2.8.2 Instruction Cache Block Invalidate (icbi)	78
2.2.8.3 Instruction Cache Synchronize (isync)	78
2.2.8.4 Data Cache Block Touch (dcbt and dcbst)	78
2.2.8.5 Data Cache Block Zero (dcbz)	79
2.2.8.6 Data Cache Block Store (dcbst)	79
2.2.8.7 Data Cache Block Flush (dcbf)	80
2.2.8.8 Load and Reserve and Store Conditional Instructions (lwarx/ldarx, stwcx/stdcx)	80
2.2.9 Memory Synchronization Instructions	80
2.2.10 Recommended Simplified Mnemonics	81
3. Storage Subsystem	83
3.1 Storage Hierarchy	83
3.2 Caches	84
3.2.1 Store Gathering	84
3.3 Storage Model	85
3.3.1 Atomicity	85
3.3.2 Storage Access Ordering	85
3.3.2.1 Storage Access Alignment Support	85

3.3.3 Atomic Updates and Reservations	86
3.4 Cache Management	87
3.4.1 Flushing the L1 I-Cache	87
3.4.2 Flushing the L1 D-Cache	87
3.4.3 L2 Cache Disabling and Enabling	87
3.4.4 L2 Cache Flushing	87
3.4.4.1 L2 Cache Flush in Direct-Mapped Mode	87
3.4.5 L2 Cache Flush Algorithm	88
3.5 Functional Units	90
3.5.1 Core Interface Unit	91
3.5.2 L2 Cache Controller	91
3.5.2.1 Cache Coherency	93
3.5.2.2 Cache-Coherency Paradoxes	93
3.5.2.3 Cache State Transition Tables	93
3.5.3 Data Prefetch	96
3.5.3.1 Optional dcbt Variant	96
3.5.3.2 Enhanced dcbt Variant	97
4. Exceptions	99
4.1 970MP Microprocessor Exceptions	100
4.2 Exception Recognition and Priorities	103
4.2.1 Exception Priorities	103
4.3 Exception Processing	105
4.3.1 Machine Status Save/Restore Register 0 (SRR0)	105
4.3.2 Machine Status Save/Restore Register 1 (SRR1)	105
4.3.3 Machine State Register (MSR)	106
4.3.4 Enabling and Disabling Exceptions	108
4.3.5 Exception Processing Steps	108
4.3.6 Setting the Recoverable Exception in the MSR	109
4.3.7 Returning from an Exception Handler	109
4.4 Process Switching	110
4.5 Exception Definitions	110
4.5.1 System Reset Exception	110
4.5.2 Machine Check Exceptions	111
4.5.3 Data Storage Exception	113
4.5.4 Data Segment Exception	113
4.5.5 Instruction Storage Exception	113
4.5.6 Instruction Segment Exception	113
4.5.7 External Interrupt Exception	114
4.5.8 Alignment Exception	114
4.5.9 Program Exception	114
4.5.10 Floating-Point Unavailable Exception	115
4.5.11 Decrementer Exception	115
4.5.12 System Call Exception	115
4.5.13 Trace Exception	115
4.5.14 Performance Monitor Exception	116
4.5.15 VPU Unavailable Exception	117
4.5.16 Instruction Address Breakpoint Exception	117

IBM PowerPC 970MP RISC Microprocessor

4.5.17 Maintenance Exception	117
4.5.18 VPU Assist Exception	118
5. Memory Management	119
5.1 MMU Overview	119
5.1.1 Speculative Storage Accesses	120
5.1.2 Storage Protection	121
5.1.3 Storage Access Modes	121
5.1.4 Support for 32-Bit Operating Systems	121
5.2 Real Addressing Mode	122
6. Software Optimization Guidelines	123
6.1 Design Characteristics	123
6.2 Software Considerations for the 970MP Microprocessor	126
7. Signal Description	129
7.1 Signal Configuration	130
7.2 Signal Descriptions	131
7.2.1 Processor Interface	131
7.2.1.1 Address/Data In (ADIN[0:43])—Input	131
7.2.1.2 Snoop Response In (SRIN[0:1], $\overline{\text{SRIN}}[0:1]$)—Input	132
7.2.1.3 Clock In (CLKIN/ $\overline{\text{CLKIN}}$)—Input	132
7.2.1.4 Address Data Out (ADOUT[0:43])—Output	133
7.2.1.5 Snoop Response Out (SROUT[0:1], $\overline{\text{SROUT}}[0:1]$)—Output	133
7.2.1.6 Clock Out (CLKOUT/ $\overline{\text{CLKOUT}}$)—Output	133
7.2.2 Processor Status and Control	133
7.2.2.1 Quiescent Request ($\overline{\text{CP0_QREQ}}$ and $\overline{\text{CP1_QREQ}}$)—Output	133
7.2.2.2 Quiescent Acknowledgment ($\overline{\text{CP0_QACK}}$ and $\overline{\text{CP1_QACK}}$)—Input	134
7.2.2.3 Time-Base Enable (TBEN)—Input	134
7.2.2.4 Processor ID (PROCID[0:1])—Input	134
7.2.2.5 Bus Configuration Select (BUSCFG[0:2])—Input	134
7.2.2.6 PLL Locked (PLL_LOCK)—Output	135
7.2.2.7 Clock Receiver Termination (CKTERM_DIS)—Input	135
7.2.3 Clock Control	135
7.2.3.1 System Clock (SYSCLK/ $\overline{\text{SYSCLK}}$)—Input	135
7.2.3.2 Phase Synchronization (psync)—Input	135
7.2.3.3 PLL Bypass ($\overline{\text{BYPASS}}$)—Input	135
7.2.3.4 PLL Multiplier (PLL_MULT)—Input	136
7.2.3.5 PLL Range Select (PLL_RANGE[0:1])—Input	136
7.2.4 Interrupts and Resets	136
7.2.4.1 Interrupt ($\overline{\text{CP0_INT}}$ and $\overline{\text{CP1_INT}}$)—Input	136
7.2.4.2 Machine Check Interrupt ($\overline{\text{MCP}}$)—Input	136
7.2.4.3 Checkstop ($\overline{\text{CHKSTOP}}$)—Bidirectional	136
7.2.4.4 Hard Reset ($\overline{\text{CP0_HRESET}}$ and $\overline{\text{CP1_HRESET}}$)—Input	137
7.2.4.5 Soft Reset ($\overline{\text{CP0_SRESET}}$ and $\overline{\text{CP1_SRESET}}$)—Input	137
7.2.5 Debug/Test Interface	137
7.2.5.1 Attention (ATTENTION)—Output	137
7.2.5.2 Processor Interface Disable (EI_DISABLE)—Input	137
7.2.5.3 Trigger Out (TRIGGEROUT)—Output	137

7.2.5.4 JTAG Signals	137
7.2.5.5 I ² C Signals	138
7.2.6 Voltage and Ground	138
8. Processor Interconnect Bus	139
8.1 Overview	140
8.1.1 Packets	141
8.1.2 Bus Segments	141
8.1.2.1 Address/Data Bus Segment	141
8.1.2.2 Transfer-Handshake Bus Segment	142
8.1.2.3 Snoop-Response Bus Segment	142
8.1.3 Transactions	142
8.1.3.1 Read Transaction	143
8.1.3.2 Write Transaction	144
8.1.3.3 Command-Only Transaction	145
8.1.4 Memory and Cache Coherency	146
8.1.4.1 Physical Memory Size	146
8.1.4.2 Coherency Protocol	146
8.1.4.3 Coherency Block Size	146
8.2 Packet Transfer Protocol	147
8.2.1 Command Packet Definition	147
8.2.1.1 Address Modifiers	147
8.2.1.2 Transfer Type Field	149
8.2.1.3 Tag Definition	151
8.2.1.4 Command Pacing	151
8.2.2 Data Packet Definition	152
8.2.2.1 Two-Beat Transfers	153
8.2.2.2 Multi-Beat Transfers	153
8.2.3 Transfer-Handshake Packets	155
8.2.3.1 Null Transfer Handshake	156
8.2.3.2 Transfer-Handshake Acknowledgment	156
8.2.3.3 Transfer-Handshake Retry	157
8.2.3.4 Transfer-Handshake Parity Error	158
8.3 Snoop Responses	158
8.3.1 Snoop-Response Bus Implementation	159
8.3.2 Snoop-Response Descriptions	160
8.3.2.1 SResp Retry Response Code (Priority 1 - highest)	160
8.3.2.2 SResp Modified-Intervention Response Code (Priority 2)	161
8.3.2.3 SResp Shared-Intervention Response Code (Priority 3)	161
8.3.2.4 SResp Modified Response Code (Priority 4)	162
8.3.2.5 SResp Shared Response Code (Priority 5)	162
8.3.2.6 SResp Null or Clean Response Code (Priority 6 - lowest)	162
8.4 Bus Transactions	163
8.4.1 Terms	163
8.4.2 Memory Read Transactions (General)	164
8.4.2.1 Read Transaction	164
8.4.2.2 Read with No Intent to Cache Transaction	165
8.4.2.3 Read with Intent to Modify Burst Transaction	166
8.4.2.4 LARX-Reserve Transaction	166

IBM PowerPC 970MP RISC Microprocessor

8.4.3 Memory Write Transactions (General)	167
8.4.3.1 Write-With-Kill Transaction	167
8.4.3.2 Write-With-Clean Transaction	168
8.4.3.3 Write-With-Flush Transaction	168
8.4.4 Command-Only Transactions	168
8.4.4.1 DCLAIM Transaction (Invalidate Others)	168
8.4.4.2 Flush Transaction	169
8.4.4.3 Clean Transaction	169
8.4.4.4 IKill Transaction	169
8.4.4.5 TLBIE Transaction	170
8.4.4.6 TLBSYNC Transaction	170
8.4.4.7 SYNC Transaction	170
8.4.4.8 EIEIO Transaction	171
8.4.4.9 Null Transaction	171
9. Power and Thermal Management	173
9.1 Definitions	173
9.1.1 Full Power Mode	173
9.1.2 Doze Mode	173
9.1.3 Nap Mode	173
9.1.4 Deep Nap Mode	174
9.1.5 Dynamic Power Management	174
9.2 Power-Management Support	174
9.2.1 Power-Management Control Bits	174
9.2.2 Interrupts	175
9.2.3 Bus Snooping	175
9.2.3.1 Delay Calculation	178
9.2.4 Thermal Diodes	179
9.2.5 Bus States while in Power Saving Modes	179
9.3 Software Considerations for Power Management	180
9.3.1 Entering Power Saving Mode	180
9.3.2 External Interrupt Enable	180
9.4 Power Tuning Facility Overview	181
9.4.1 Power Tuning Facility Definitions	181
9.4.2 Power Modes	183
9.4.3 Power Transition Latencies	186
9.4.3.1 Idle to Run Transitions	187
9.4.3.2 Exiting Deep Nap Using a Decrementer Interrupt	188
9.4.3.3 Frequency Transitions in the Power Tuning Facility	188
9.5 PLL Design	189
9.6 Time-Base and Decrementer Registers	191
9.7 I ² C Bus Interface	191
9.8 Frequency and Voltage Scaling	191
9.8.1 Frequency Scaling	191
9.8.1.1 Initiating a Frequency Change	191
9.8.1.2 Power Control Register	194
9.8.1.3 Power Control Register High (PCRH)	196
9.8.1.4 Power Status Register	197
9.8.2 Power Adjustment Bus Transaction	198

9.8.3 Clock Dithering	200
9.8.4 Voltage Scaling	201
9.8.5 Frequency and Voltage Scaling Latencies	202
9.9 Reducing Clock Mesh Power	203
9.9.1 Power Saving in Deep Nap	203
9.10 Additional Dynamic Power Management	204
10. 970MP Performance Monitor	205
10.1 Instrumentation Facilities Overview	205
10.1.1 Performance Monitor Facilities	206
10.1.2 Performance Monitor Event Selection	206
10.1.3 Machine States and Enabling the Performance Monitor Counters	206
10.1.4 Trigger Events and Enabling the Performance Monitor Counters	206
10.1.5 Performance Monitor Exceptions	206
10.1.6 Sampling	207
10.1.7 Thresholding	207
10.1.8 Trace Support Facilities	207
10.2 Instruction Sampling Facilities	207
10.2.1 Special Purpose Registers and Fields Associated with Instrumentation	207
10.3 Performance Monitor Components	210
10.4 Performance Monitor Control Registers	211
10.4.1 Performance Monitor Control Register MMCR0	211
10.4.2 Performance Monitor Control Register MMCR1	214
10.4.3 Performance Monitor Control Register MMCRA	217
10.4.4 Performance Monitor Count Registers PMC1 - 8	219
10.4.5 Performance Monitor and Trace Related Bits in the Machine State Register (MSR)	220
10.4.6 Performance Monitor Related Bits in Hardware Implementation-Dependent Register 0 (HID0)	221
10.4.7 Performance Monitor Related Bits in the Control Register (CTRL)	221
10.4.8 Performance Monitor Related Bits in the SCOM0240, 1240 Register (SCOM x'240')	222
10.4.9 Performance Monitor Related Bits in the SCOM0360, 1360 Register (SCOM x'360')	223
10.4.10 Performance Monitor Related Bits in the IMC Array (IMC)	224
10.4.11 Performance Monitor Related Bits in the Sampled Instruction Address Register (SIAR)	224
10.4.12 Performance Monitor Related Bits in the Sampled Data Address Register (SDAR)	224
10.4.13 Performance Monitor Related Bits in the SRR1 (SRR1)	225
10.4.14 Performance Monitor Related Bits in the Time-Base Register (TB)	226
10.5 Performance Monitor Event Selection	227
10.5.1 Direct Events	228
10.5.1.1 Combined Events	228
10.5.1.2 Source-Encoded Events	228
10.5.1.3 Instruction Counts	229
10.5.2 Over 32-Bit Count	232
10.5.2.1 Examples of Over Bit Count	232
10.5.3 Speculative Count	232
10.6 Configuring the Performance Monitor Bus	233
10.7 Enabling the Performance Monitor Counters	243
10.7.1 Machine States	243

IBM PowerPC 970MP RISC Microprocessor

10.7.2 Trigger Events	244
10.7.2.1 Time-Base Transition Events	245
10.7.2.2 PMC1 Counter Negative Condition Events	245
10.7.2.3 PMCj ($2 \leq j \leq 8$) Counter Negative Condition Events	246
10.7.3 Method for Enabling or Disabling Performance Monitor Counting	246
10.8 Performance Monitor Exceptions	247
10.9 Instruction Matching and Sampling	248
10.9.1 Stage 1 Eligibility	248
10.9.2 Stage 2 Eligibility	248
10.9.3 Stage 3 Eligibility	248
10.10 IFU Instruction Matching Facility	249
10.10.1 Overview of the IFU Instruction Matching Facility	249
10.10.2 IMC Array	250
10.10.3 Reading the IMC SPR with the mfimc Instruction	251
10.10.4 Writing the IMC SPR With the mtime Instruction	252
10.10.5 The v0 and v1 Mask Criteria	253
10.10.6 Instruction Matching Examples	254
10.11 IDU Instruction Sampling Facility	254
10.11.1 Overview of the IDU Instruction Sampling Facility	254
10.11.2 Stage 1 Eligibility	255
10.11.3 Stage 2 Eligibility	257
10.11.4 Stage 3 Mark/No Mark	258
10.11.5 Complete Masking, Matching, and Marking Cycle	260
10.11.6 Examples of Instruction Sampling Scenarios	261
10.11.7 Enabling and Disabling Marking	264
10.12 SIAR and SDAR Registers	265
10.12.1 Instruction Sampling	265
10.12.1.1 Performance Monitor Exceptions	265
10.12.2 Single Step and Branch Trace Marking Mode	266
10.12.2.1 Single Step Trace Mode	266
10.12.2.2 Branch Trace Mode	267
10.12.3 Comparison to Previous PowerPC Processors	267
10.13 Thresholding	267
10.14 Detailed Event Information	270
11. System Design	279
11.1 I ² C Interface	279
11.2 Bus Initialization, Configuration, Power Management, and Test	279
11.2.1 Bus Initialization	279
11.2.2 Configurable Parameters	279
11.2.3 Configuration Interface	282
11.2.3.1 Processor Configurable Timing Delay Parameter Register (BUSCONF)	283
11.2.3.2 North Bridge Configurable Timing Delay Parameter Register	284
11.2.4 Power Management	285
11.2.5 Reliability, Availability, and Serviceability (RAS) Requirement	287
11.3 Processor Interconnect Electrical Interface	288
11.3.1 Initialization at Power-On Reset	289
11.3.2 Target Cycle	289

11.4 Processor Interconnect Bus Error Detection and Correction	291
11.4.1 Error Detection for Balanced Encoding	291
11.4.2 Error Detection for Alternative Encodings	291
11.4.2.1 Single-Error and Double-Error Detection	292
11.4.2.2 Single-Error Correct, Double-Error Detection	292
12. SCOM Interface and Registers	295
12.1 Processor Core SCOM SPR Access	295
12.1.1 Operating System Protocol to Access SCOM SPRs	295
12.1.2 SCOMD Format	296
12.1.3 SCOMC Format	297
12.2 SCOM Address Allocation	299
12.2.1 Register Description Conventions	303
12.2.2 SCOM Error Handling	303
12.2.3 Access Status Register	304
12.3 Core Pervasive SCOM Register Definitions	305
12.3.1 Processor CoreRAS Facilities (x'02[1:4]XXX')	305
12.3.2 Processor Core SPR SCOM Access (x'023XXX')	320
12.3.3 Processor Core Performance Monitor Sampling Control (x'02400X')	327
12.3.4 Processor Core FIR Facilities (x'03[0:5]XXX')	328
12.3.5 Instruction Mark Configuration (x'03600X')	335
12.4 Storage Subsystem SCOM Register Definition	337
12.4.1 L2 SCOM Register Definition	337
12.4.2 BIU SCOM Register Definition	340
12.4.3 Processor Interconnect Registers	347
12.5 Chip Pervasive SCOM Register Definition	357
12.5.1 Power-On Reset Registers (x'40XXXX')	357
12.5.2 Chip Free-Running Clock Section Control/Status (x'50[0:4]XXX')	367
12.5.3 Chip Parallel SCOM Control (x'6XXXXX')	375
12.5.4 Chip Clock/Scan Control (x'8[0:4]XXXX')	381
13. Vector Processing Unit	401
13.1 970MP Vector and SIMD Multimedia Overview	401
13.1.1 VPU Implementation	401
13.1.2 Vector ALU	402
13.2 Vector Registers	403
13.2.1 VRSAVE Register	403
13.2.2 Vector Status and Control Register (VSCR)	403
13.3 Effects on Existing PowerPC Facilities	405
13.3.1 Control Flow	405
13.3.1.1 Condition Register	405
13.3.1.2 Machine State Register	406
13.3.1.3 Machine Status Save/Restore Registers (SRR0, SRR1)	407
13.4 Exceptions	407
13.4.1 VPU Unavailable Exception	407
13.4.2 VPU Assist Exception	407
13.4.3 Data Storage Exception	407

IBM PowerPC 970MP RISC Microprocessor

13.5 Optional Instructions	408
13.5.1 Java Mode Instruction Handling Implementation	408
13.5.2 Least Recently Used Instructions	408
13.5.3 Data Stream Instructions	408
13.6 Vector Instruction Set	409

List of Figures

Figure 1-1.	970MP Block Diagram	34
Figure 1-2.	970MP Dual Core with Common Arbitration Logic	35
Figure 2-1.	970MP Programming Model—Registers	44
Figure 2-2.	Processor Attention Instruction	76
Figure 3-1.	970MP Storage Subsystem	90
Figure 3-2.	Data Flow in the 1 MB L2 Cache	92
Figure 3-3.	Data Cache Block Touch X-Form (Optional Variant)	96
Figure 3-4.	Data Cache Block Touch X-Form (Enhanced Variant)	97
Figure 7-1.	970MP Microprocessor Signal Groups	130
Figure 7-2.	Encoding and Selection Logic for the Drive Side of a 970MP Interconnect SSB	132
Figure 8-1.	Processor Interconnect Bus Configuration with Two 970MP Microprocessors	139
Figure 8-2.	Two Microprocessors Connected to a North Bridge	140
Figure 8-3.	Read Transaction Timing Diagram	143
Figure 8-4.	Write Transaction Timing Diagram	144
Figure 8-5.	Command-Only Transaction Timing Diagram	145
Figure 9-1.	Processor QREQ/QACK Signalling	176
Figure 9-2.	North Bridge QREQ/QACK Signalling	177
Figure 9-3.	Using a 970MP Microprocessor with a Single QREQ/QACK Pair	178
Figure 9-4.	970MP Power Mode States	183
Figure 9-5.	PLL Design	190
Figure 9-6.	Frequency Scaling Event Ordering	193
Figure 9-7.	Clock Dithering Block Diagram	200
Figure 9-8.	Sample Shift Pattern	201
Figure 10-1.	Performance Monitor Architecture	210
Figure 10-2.	Event Selection	227
Figure 10-3.	970MP Performance Monitor Bus Configuration	234
Figure 10-4.	Patch Map	251
Figure 10-5.	IFU and IDU Instruction Sampling Flow	259
Figure 10-6.	Performance Monitor Threshold Logic	268
Figure 11-1.	Configurable Timing Parameters	280
Figure 11-2.	North Bridge Configurable Timing Parameters	280
Figure 11-3.	Processor Configurable Timing Parameters	281
Figure 11-4.	Processor QREQ and QACK Signalling	286
Figure 11-5.	North Bridge QREQ and QACK Signalling	287
Figure 11-6.	Bus Diagram of a Dual-Processor 970MP Processor Interconnect-Based System	288
Figure 11-7.	Receive-Side FIFO Circuit	290
Figure 11-8.	Timing Diagram Showing Relationship Between Bclk and the Four Gate Signals	290
Figure 12-1.	Processor Unit SCOM Topology	295

IBM PowerPC 970MP RISC Microprocessor

Figure 12-2. SCOMC SPR Format	297
Figure 12-3. Format of an SCOM Address	299
Figure 12-4. Format of an SCOM Address within the BIU	299
Figure 12-5. Format of an SCOM Data Bus	299
Figure 12-6. Common Clock Commands	382
Figure 12-7. Example of LBIST Commands using the EPS Engine	390
Figure 13-1. VPU Block Diagram	402
Figure 13-2. VSCR Format	403
Figure 13-3. VSCR Moved to a Vector Register	404
Figure 13-4. Condition Register (CR)	405

List of Tables

Table i.	Acronyms and Abbreviated Terms	26
Table ii.	Terminology Conventions	31
Table iii.	Instruction Field Conventions	31
Table 2-1.	MSR Bits	49
Table 2-2.	Additional SRR1 Bit	49
Table 2-3.	Implementation-Specific SPRs	51
Table 2-4.	Move To/Move From SPR Behavior	53
Table 2-5.	Storage Control Instructions	77
Table 2-6.	dcbz Actions	79
Table 3-1.	Storage Hierarchy Characteristics	83
Table 3-2.	Simple Address Decode	87
Table 3-3.	Storage Subsystem Functional Units	90
Table 3-4.	Cache-Coherency Protocol	93
Table 3-5.	970MP L2 Cache State Transitions Due to Processor Instructions	93
Table 3-6.	970MP L2 Cache State Transitions Due to Bus Operations	94
Table 4-1.	970MP Microprocessor Exception Classifications	100
Table 4-2.	Exceptions and Conditions	101
Table 4-3.	IEEE Floating-Point Exception Mode Bits	108
Table 4-4.	Register Settings for Machine Check Exception	112
Table 4-5.	Register Settings for Alignment Exception	114
Table 4-6.	Register Settings for Trace Exception	115
Table 4-7.	Register Settings for the Performance Monitor Exception	116
Table 4-8.	Register Settings for VPU Unavailable Interrupt	117
Table 4-9.	Register Settings for Maintenance Exception	117
Table 4-10.	Register Settings for VPU Assist Exception	118
Table 5-1.	MMU Feature Summary	120
Table 5-2.	Treatment of WIMG Bits in the 970MP	121
Table 8-1.	Processor Interconnect Signal Description	141
Table 8-2.	Command Packet Description	147
Table 8-3.	Transfer Type Encoding	149
Table 8-4.	Transfer Size Encoding	150
Table 8-5.	Tag Definition	151
Table 8-6.	Read-Data Packet Header Description	152
Table 8-7.	Data Beat Description	152
Table 8-8.	Two-Beat Data Transfers	153
Table 8-9.	Packet Ordering for 128-Byte Interleaved Packets on 32-Byte Boundaries	154
Table 8-10.	Packet Ordering for 32-Byte Interleaved Packets	154
Table 8-11.	Transfer-Handshake Definition	155

IBM PowerPC 970MP RISC Microprocessor

Table 8-12.	Snoop-Response Bit Definition	158
Table 8-13.	Allowed Snoop Responses	159
Table 8-14.	Write-With-Kill Types Supported	167
Table 9-1.	Power-Management Control Bits	174
Table 9-2.	Minimum QAckIdleDelay requirement in bus clocks for 970MP	179
Table 9-3.	Minimum (QAckIdleDelay + QAckMinLowTime) requirement in bus clocks for 970MP	179
Table 9-4.	Power-Management Modes	181
Table 9-5.	Power Mode States	184
Table 9-6.	Transitions between Power Modes	185
Table 9-7.	Valid Combinations of Power Modes	186
Table 9-8.	Latency of Deep-Nap-to-Run Transitions in Full Frequency Cycles	188
Table 9-9.	Power Adjustment Transaction	198
Table 10-1.	970MP Performance Monitor and Trace-Related Special Purpose Registers	209
Table 10-2.	Performance Monitor Internal Multiplexer PMCxSEL[0:4] Bit Values	227
Table 10-3.	Event Data Source Encodings	228
Table 10-4.	Event Instruction Source Encodings	229
Table 10-5.	Direct Events	230
Table 10-6.	Speculative Count Events	233
Table 10-7.	Performance Monitor Bus Assignments	235
Table 10-8.	Examples of Event Counter Enabling States	244
Table 10-9.	Partial Match Rows in the IMC Array	250
Table 10-10.	Complete Match Rows in the IMC Array	250
Table 10-11.	IMC SPR Patch Map Sample Results	252
Table 10-12.	IMC SPR for a 17-Bit Match	253
Table 10-13.	IMC SPR Used when Writing the Second mtimc Instruction for a 32-Bit Match	253
Table 10-14.	Encoding Bits v0 and v1 of the IMC Array Mask	254
Table 10-15.	IFU BSFL Predecode Bit Definitions	256
Table 10-16.	Start and End Event Select Bits and the Performance Monitor Threshold Logic	269
Table 10-17.	Detailed Event Descriptions	270
Table 11-1.	Programmable Delay Parameters	281
Table 11-2.	I ² C Interface Signals	282
Table 11-3.	I ² C Registers Used by the 970MP Processor Interconnect	282
Table 11-4.	Bit Error Position Identifier	293
Table 12-1.	Operating System Code to Access SCOM	296
Table 12-2.	SCOM Base Addresses	300
Table 12-3.	SCOM Modifier Addresses	300
Table 12-4.	EPS Engine Description	389
Table 13-1.	VSCR Field Descriptions	404
Table 13-2.	CR6 Field Bit Settings for Vector Compare Instructions	405



Table 13-3. MSR Bit Settings Affecting the VPU	406
Table 13-4. Supported Vector Instructions	409



Revision Log

Each release of this document supersedes all previously released versions. The revision log lists all significant changes made to the document since its initial release. In the rest of the document, change bars in the margin indicate that the adjacent text was modified from the previous release of this document.

Revision Date	Page	Description
March 7, 2008	90	Version 2.3
	123	- Revised description in <i>Section 3.5 Functional Units</i>. - Added a new section on software optimization (see <i>Section 6 Software Optimization Guidelines</i>). - Removed <i>Section 2.2.1.3 Invalid Forms</i>. - Removed <i>Section 3.5.3 Non-Cacheable Unit</i>. - Removed <i>Section 3.5.4.1 Overview of the Hardware-Controlled Data Prefetch</i>. - Removed <i>Section 3.5.4.2 Hardware Prefetch Engine Implementation</i>. - Removed <i>Section 3.5.4.4 Vector Prefetch Instruction Support</i>. - Removed <i>Section 3.5.4.5 Programmability</i>. - Removed <i>Section 5.3 Memory Segment Model</i>.
March 21, 2007	61	Version 2.2
	112	Added DABRX bit description table to <i>Section 2.1.2.3 Data Address Breakpoint Register (DABR)</i>.
	114	Modified DSISR[6] bit setting in <i>Table 4-4 Register Settings for Machine Check Exception</i>.
	228	Modified DSISR[6] bit setting in <i>Table 4-5 Register Settings for Alignment Exception</i>.
	284	Edited <i>Table 10-3 Event Data Source Encodings</i>.
	345	Defined bit [31] as BCM in <i>Table 11.2.3.2 North Bridge Configurable Timing Delay Parameter Register</i>.
	297	Defined "g" as guarded access in the <i>BIU Mode Register</i> section.
June 2, 2006	44	Version 2.1
	194, 196	Added SPRG3 to User Model — USIA block in <i>Figure 2-1 970MP Programming Model—Registers</i> and added a description.
	295	Corrected the SCOM address of the Power Control Register (PCR).
	178	Added commonly used SCOM registers and their descriptions (<i>Section 12 SCOM Interface and Registers</i>).
		Added <i>Section 9.2.3.1 Delay Calculation</i>

IBM PowerPC 970MP RISC Microprocessor

Revision Date	Page	Description
June 28, 2005		Version 2.0
	50	• Clarified the explanation of illegal instructions.
	75	• Rewrote the description of the Load with Update instructions.
	153	• Updated <i>Table 8-8 Two-Beat Data Transfers</i>, the description of the data transfer format, and <i>Table 8-9 Packet Ordering for 128-Byte Interleaved Packets on 32-Byte Boundaries</i>.
	173 174	• Expanded the description of Doze mode and how it relates to power management.
	281	• Corrected the programmable delay parameters for SNOOPLAT and SNOOPACC.
January 17, 2005		Version 1.0 (Initial release)

About This Book

The primary objective of the *IBM PowerPC® 970MP RISC Microprocessor User's Manual* is to define the functionality of the PowerPC 970MP microprocessor for software and hardware developers.

The information in this book is subject to change without notice, as described in the disclaimers on the title page. As with any technical documentation, it is the readers' responsibility to be sure they are using the most recent version of the documentation. To locate any published errata or updates for this document, go to ibm.com/chips/techlib.

Note: Soft copies of many of the latest versions of the manuals and documents referred to in this manual that are produced by IBM can be accessed on the Web at ibm.com/chips/techlib.

Audience

This manual is intended for system software and hardware developers and application programmers who want to develop products for the 970MP microprocessor. It is assumed that the reader understands operating systems, microprocessor system design, basic principles of reduced instruction set computer (RISC) processing, and details of the PowerPC Architecture™.

Organization

For ease in reference, the arrangement of topics in this book is similar to that of the *PowerPC Microprocessor Family: The Programming Environments Manual for 64-Bit Microprocessors* and the *PowerPC Microprocessor Family: Vector/SIMD Multimedia Extension Technology Programming Environments Manual* (see *Related Documents* on page 22). Topics build upon one another, beginning with a description and summary of 970MP-specific registers and instructions and progressing to more specialized topics such as 970MP-specific details regarding the cache, exception, memory management models, and power management. Thus, chapters might include information from multiple levels of the architecture. For example, the discussion of the cache model uses information from both the virtual environment architecture (VEA) and the operating environment architecture (OEA).

A summary and a brief description of the major sections of this manual follows:

- *Chapter 1 PowerPC 970MP Overview* is useful for readers who want a general understanding of the features and functions of the PowerPC Architecture and the 970MP processor. This chapter describes the flexible nature of the PowerPC Architecture definition, and provides an overview of how the PowerPC Architecture defines the register set, operand conventions, addressing modes, instruction set, cache model, exception model, and memory management model.
- *Chapter 2 Programming Model* is useful for software engineers who need to understand the 970MP-specific registers, operand conventions, and details regarding how the PowerPC instructions are implemented on the 970MP microprocessor. Instructions are organized by function.
- *Chapter 3 Storage Subsystem* discusses the storage subsystem as implemented on the 970MP microprocessor. The storage subsystem includes the core interface logic, the non-cacheable unit, the L2 cache and controls, and the bus interface unit.
- *Chapter 4 Exceptions* describes the exception model defined in the PowerPC OEA and the specific exception model implemented on the 970MP microprocessor.

IBM PowerPC 970MP RISC Microprocessor

- *Chapter 5 Memory Management* describes the 970MP implementation of the memory management unit specifications provided by the PowerPC OEA for PowerPC processors.
- *Chapter 6 Software Optimization Guidelines* describes key design characteristics of the 970MP microprocessor.
- *Chapter 7 Signal Description* describes the individual signals of the 970MP microprocessor.
- *Chapter 8 Processor Interconnect Bus* describes the processor interface (PI), which is a bus architecture providing high-speed, high-performance interconnections for processors, I/O devices, memory subsystems, and bridge chips.
- *Chapter 9 Power and Thermal Management* provides information about power saving and thermal management modes for the 970MP microprocessor.
- *Chapter 10 970MP Performance Monitor* describes the operation of the performance monitor diagnostic tool incorporated in the 970MP microprocessor and provides detailed event information.
- *Chapter 11 System Design* describes system-related features such as power-on reset and reliability, availability, and serviceability (RAS) considerations.
- *Chapter 12 SCOM Interface and Registers* describes the scan communication (SCOM) facility that is used to access processor debug and diagnostic facilities.
- *Chapter 13 Vector Processing Unit* provides a general understanding of the features and functions of the vector processing unit (VPU) used on the 970MP microprocessor.

Related Documents

Companion Manuals

This manual is intended as a companion to the following reference manuals:

- PowerPC Architecture¹ books:

Note: The PowerPC Architecture books supersede the *PowerPC Programming Environments Manual* for the 970MP implementation. However, not all features available in the PowerPC Architecture are supported in the 970MP microprocessor (such as, logical partitioning).

- *PowerPC User Set Architecture (Book I, Version 2.01)*. Covers the base user instruction set architecture (UISA), user-level registers, data types, memory conventions, memory and programming models, and related facilities available to the application programmer.
- *PowerPC Virtual Environment Architecture (Book II, Version 2.01)*. Defines the storage model and related instructions and facilities available to the programmer, and the time-keeping facilities available to the application programmer. The VEA, which is the smallest component of the PowerPC Architecture, defines additional user-level functionality that falls outside typical user-level software requirements. The VEA describes the memory model for an environment in which multiple processors or other devices can access external memory and define aspects of the cache model and cache control instructions from a user-level perspective. The resources defined by the VEA are particularly useful for optimizing memory accesses and for managing resources in an environment in which other processors and other devices can access external memory.

1. PowerPC Architecture refers to the instructions and facilities described in Books I, II, and III.

Implementations that conform to the PowerPC VEA also conform to the PowerPC UISA, but might not necessarily adhere to the operating environment architecture (OEA).

- *PowerPC Operating Environment Architecture (Book III, Version 2.01)*. Defines the system (privileged) instructions and related facilities. The OEA defines supervisor-level resources typically required by an operating system. The OEA defines the PowerPC memory management model, supervisor-level registers, and the exception model.

Implementations that conform to the PowerPC OEA also conform to the PowerPC UISA and VEA.

- *PowerPC Microprocessor Family: Programming Environments Manual for 64-Bit Microprocessors* (referred to as the *Programming Environments Manual*). Provides information about resources defined by the PowerPC Architecture that are common to PowerPC processors. This manual describes the functionality of the 64-bit architecture model.
- *PowerPC Microprocessor Family: Vector/SIMD Multimedia Extension Technology Programming Environments Manual*. Describes how the vector/SIMD technology relates to both the 64-bit and the 32-bit portions of the PowerPC Architecture.
- *The PowerPC Architecture: A Specification for a New Family of RISC Processors* by C. May, E. Silha, R. Simpson, and H. Warren, Morgan Kaufman, May 1994. Defines the architecture from the perspective of the three programming environments and remains the defining document for the PowerPC Architecture.

Because the PowerPC Architecture is designed to be flexible in order to support a broad range of processors, these documents provide a general description of features that are common to PowerPC processors and indicate those features that are optional or that might be implemented differently in the design of each processor.

It is important to note that some resources are defined more generally at one level in the architecture and more specifically at another. For example, conditions that cause a floating-point unavailable exception are defined by the UISA, while the exception mechanism itself is defined by the OEA.

Additional Documentation

Some additional PowerPC documentation is available at **ibm.com/chips/techlib** through IBM Customer Connect at <http://ibm.com/technologyconnect>.

- *IBM PowerPC 970MP RISC Microprocessor Datasheet*. This datasheet provides specific data about bus timing, signal behavior, and ac, dc, and thermal characteristics, as well as other design considerations for the 970MP implementation.
- *PowerPC 970MP Power On Reset Application Note*. This document contains information about required power-on-reset design and initialization.
- *PowerPC Microprocessor Family: The Programmer's Reference Guide (MPRPPCPRG-01)*. This is a concise reference that includes the register summary, memory control model, exception vectors, and the PowerPC instruction set.
- Application notes. These short documents contain information about specific design issues useful to programmers and engineers working with PowerPC processors.

IBM PowerPC 970MP RISC Microprocessor

General PowerPC Documentation

The following documentation provides useful information about the PowerPC Architecture and computer architecture in general:

Ferraiolo, F., E. Cordero, D. Dreps, M. Floyd, "Power4: Synchronous Wave-Pipelined Interface." *Hot Chips 1999*, Stanford, CA.

Hennessy, John L. and David A. Patterson. *Computer Architecture: A Quantitative Approach*. 2nd ed.

I²C-Bus Specification. Version 2.1. Philips Semiconductors, 2000.

IEEE Standard Test Access Port and Boundary-Scan Architecture, *IEEE Std. 1149.1a-1993*.

McClanahan, Kip. *PowerPC Programming for Intel Programmers*. Foster City, CA: IDG Books Worldwide, Inc.

Shanley, Tom. *PowerPC System Architecture*. Richardson, TX: Mindshare, Inc.

Conventions

This document uses the following notational conventions:

&	AND logical operator.
	OR logical operator.
'0'	Binary values in text are either spelled out (zero and one) or appear in single quotation marks. For example: '10101'. In tables, these quotation marks are omitted.
¬	NOT logical operator.
crfD	Instruction syntax used to identify a destination CR field.
crfS	Instruction syntax used to identify a source Condition Register (CR) field.
frA, frB, frC	Instruction syntax used to identify a source Floating-Point Register (FPR).
frD	Instruction syntax used to identify a destination FPR.
<i>italics</i>	Italics indicate variable command parameters. For example, bcctrx . Book titles in text are set in italics.
mnemonics	Instruction mnemonics are shown in lowercase bold.
<i>n</i>	Used to express an undefined numeric value.
overbar	Overbars designate active-low (non-differential) signals.
rA, rB	Instruction syntax used to identify a source General-Purpose Register (GPR).
rD	Instruction syntax used to identify a destination GPR.
REG[FIELD]	Abbreviations or acronyms for registers are shown in uppercase text. Specific bits, fields, or ranges appear in brackets. For example, MSR[POW] refers to the Power-Management bit in the Machine State Register.
Reserved	Indicates reserved bits or bit fields in a register. Although these bits can be written to as either ones or zeros, they are always read as zeros.
vA, vB, vC	Instruction syntax used to identify a source Vector Register (VR).
vD	Instruction syntax used to identify a destination VR.
x	In certain contexts, such as a signal encoding, this indicates a don't care.
x'0'	A lowercase x precedes hexadecimal values. For example, x'0B00'.

IBM PowerPC 970MP RISC Microprocessor
Acronyms and Abbreviations

Table i contains acronyms and abbreviations that are used in this document.

Table i. *Acronyms and Abbreviated Terms* (Page 1 of 5)

Term	Meaning
ALU	arithmetic logic unit
AS	application system
ASR	Address Space Register
BAT	block address translation
BCM	balanced coding method
BHT	branch history table
BIST	built-in self test
BIU	bus interface unit
BPU	branch processing unit
BSDL	boundary-scan description language
CAM	content-addressable memory
CDF	critical data forward
CIU	core interface unit
CMOS	complementary metal-oxide semiconductor
COP	common on-chip processor
CQ	completion queue
CR	Condition Register
CRA	custom register array
CTR	Count Register
DABR	Data Address Breakpoint Register
DAR	Data Address Register
D-cache	data cache
DCMP	data translation lookaside buffer (TLB) compare
DEC	Decrementer Register
DMISS	data TLB miss address
DPM	dynamic power management
DSI	data storage interrupt
DSISR	Data Storage Interrupt Status Register. Register used to determine the source of a DSI exception.
DTLB	data translation lookaside buffer
EA	effective address
EAR	External Access Register
ECC	error checking and correction
eCR	emulation CR
eFPR	emulation FPR

Table i. Acronyms and Abbreviated Terms (Page 2 of 5)

Term	Meaning
eGPR	emulation GPR
ERAT	effective-to-real-address translation
FIFO	first-in-first-out
FPECR	Floating-Point Exception Cause Register
FPR	Floating-Point Register
FPSCR	Floating-Point Status and Control Register
FPU	floating-point unit
GCT	global completion table
GPIO	general-purpose I/O pins
GPR	General-Purpose Register
HID _n	Hardware Implementation-Dependent Register
I ² C	inter-integrated circuit
IABR	Instruction Address Breakpoint Register
IAP	initial alignment pattern
I-cache	instruction cache
IEEE	Institute for Electrical and Electronics Engineers
IFU	instruction fetch unit
IMC	Instruction Match CAM Register
IQ	instruction queue
ISI	instruction storage interrupt
ISU	instruction sequencer unit
ITLB	instruction translation lookaside buffer
JTAG	Joint Test Action Group
L2	secondary cache (level 2 cache)
L2C	L2 cache controller
LHR	load-hit-reload. A load presented through a load/store port to the LMQ matches an existing entry that has already initiated a request to the L2.
LHS	load-hit-store. A load presented through a load/store port to the store reorder queue (SRQ) matches an existing entry. Store forwarding may be attempted. If the store contains all the data required by the load, store forwarding can occur. If the store does not contain all the data required by the load, store forwarding cannot occur and the load is rejected or flushed.
LIFO	last-in-first-out
LMQ	load miss queue. An 8-entry queue, which tracks loads that miss the L1 and are awaiting data from the 970MP storage subsystem (STS). Each entry can handle two loads associated with a cache line.
LR	Link Register
LRU	least recently used
LSb	least-significant bit
LSB	least-significant byte
LSU	load/store unit. The unit in the microprocessor that executes load-and-store instructions.

IBM PowerPC 970MP RISC Microprocessor
Table i. Acronyms and Abbreviated Terms (Page 3 of 5)

Term	Meaning
MERSI	modified/exclusive/recent/shared/invalid cache-coherency protocol
MMCR n	Monitor Mode Control Registers
MMU	memory management unit
MRU	most recently used
MSb	most-significant bit
MSB	most-significant byte
MSR	Machine State Register
NaN	not a number
NCU	non-cacheable unit
NIA	next instruction address
no-op	no operation
NSA	next sequential address
NTC	next to complete
OEA	operating environment architecture
PFQ	data prefetch filter queue. Filter queue of 12 entries, which can detect data streams for prefetching.
PI	processor interface
PID	processor identification tag
PLL	phase-locked loop
PMC n	Performance Monitor Counter Registers
POR	power-on reset
POWER™	Performance Optimized with Enhanced Reduced Instruction Set Computing (RISC) Architecture
PRQ	data prefetch request queue. A prefetch queue of eight streams, which will be prefetched.
PTE	page table entry
PTEG	page table entry group
PVR	Processor Version Register
RAS	reliability, availability, and serviceability
RAW	read-after-write
RCQ	read/claim queue
RISC	reduced instruction set computing
RLM	random logic macro
RMCI	real mode cache inhibited
RTL	register transfer language
RWITM	read with intent to modify
RWNITM	read with no intent to modify
SCOM	scan communications
SCOMC	SCOM control
SCOMD	SCOM data

Table i. Acronyms and Abbreviated Terms (Page 4 of 5)

Term	Meaning
SDA	sampled data address register
SDQ	store data queue
SDR1	Register that specifies the page table base address for virtual-to-physical address translation.
SHL	store-hit-load
SHR	store-hit-reload. A committed store ready to write to the L1 data cache (D-cache) line that matches an existing LMQ entry. The store is stalled until the reload is complete.
SIAR	Sampled Instruction Address Register
SIMD	single-instruction, multiple-data
SIMM	signed immediate value
SLB	segment lookaside buffer
SMP	symmetric multiprocessor
SPR	Special Purpose Register
SR _n	Segment Register
SRQ	store reorder queue. A 32-entry queue that tracks all stores active in the LSU.
SRR0	Machine Status Save/Restore Register 0
SRR1	Machine Status Save/Restore Register 1
SSB	source-synchronous bus
STE	segment table entry
STQ	store queue
STS	970MP storage subsystem, which includes core interface logic, a noncacheable unit, the L2 cache and controls, and the bus interface unit.
TB	timebase facility
TBL	Timebase Lower Register
TBU	Timebase Upper Register
TLB	translation lookaside buffer
TType	transfer type
UIMM	unsigned immediate value
UISA	user instruction set architecture
UMMCR _n	User Monitor Mode Control Registers
UPMC _n	User Performance Monitor Counter Registers
USIA	User Sampled Instruction Address Register
VA	virtual address
VALU	vector unit arithmetic logic unit (ALU)
VEA	virtual environment architecture
VPERM	vector permute unit
VPU, vector units	vector processing units within the core.
VR	Vector Register

IBM PowerPC 970MP RISC Microprocessor*Table i. Acronyms and Abbreviated Terms* (Page 5 of 5)

Term	Meaning
VRF	Vector Register file
VRSAVE	Vector Save/Restore Register
VSCR	Vector Status and Control Register
WAR	write-after-read
WAW	write-after-write
WIMG	write-through/caching-inhibited/memory-coherency enforced/guarded bits
XER	Integer Exception Register, used to indicate conditions such as carries and overflows for integer operations.

Terminology Conventions

Table ii describes terminology conventions used in this manual and the equivalent terminology used in the PowerPC Architecture specification.

Table ii. Terminology Conventions

Architecture Specification	Current Manual
Data storage interrupt (DSI)	DSI exception
Extended mnemonics	Simplified mnemonics
Instruction storage interrupt (ISI)	ISI exception
Interrupt	Exception
Privileged mode (or privileged state)	Supervisor-level privilege
Problem mode (or problem state)	User-level privilege
Real address	Physical address
Relocation	Translation
Storage (locations)	Memory
Storage (the act of)	Access
Store in	Write back
Store through	Write through
Swizzling	Double-word swap

Table iii describes instruction field notation used in this manual.

Table iii. Instruction Field Conventions

Architecture Specification	Equivalent to:
BA, BB, BT	crbA, crbB, crbD
BF, BFA	crfD, crfS
D	d
DS	ds
FLM	FM
FRA, FRB, FRC, FRT, FRS	frA, frB, frC, frD, frS
FXM	CRM
RA, RB, RT, RS	rA, rB, rD, rS
SI	SIMM
U	IMM
UI	UIMM
VA, VB, VT, VS	vA, vB, vD, vS
VEC	Vector/SIMD multimedia extension technology



1. PowerPC 970MP Overview

The IBM PowerPC 970MP reduced instruction set computer (RISC) microprocessor is an implementation of the PowerPC Architecture. This chapter provides an overview of the features of the 970MP microprocessor and includes two block diagrams showing the major functional components.

Note: The 970MP microprocessor incorporates two complete microprocessors on a single chip, along with some common logic to connect these microprocessors to a system. The terms microprocessor, processor, and processing unit are used interchangeably to describe each of the two individual processors. The term core refers to the instruction fetch and execution logic, including the L1 cache, but excluding the storage subsystem, of each processor. The term PowerPC 970MP or 970MP refers to the single chip module comprising the two processing units and the common logic.

1.1 PowerPC 970MP Microprocessor Overview

The 970MP microprocessor is a dual core, 64-bit PowerPC RISC microprocessor with vector processing unit (VPU) extensions—the single-instruction, multiple-data (SIMD) operations that accelerate data intensive processing tasks. This processor is designed to support multiple system configurations ranging from desktop and low-end server applications, up through 4-way symmetric multiprocessor (SMP) configurations.

Each processing unit of the IBM PowerPC 970MP RISC Microprocessor consists of three main components:

- The core, which includes the VPUs
- The storage subsystem (STS), which includes the core interface logic, noncacheable unit, L2 cache and controls, and bus interface unit
- Pervasive functions

The block diagram in *Figure 1-1* on page 34 shows the major functional units comprising the core and storage subsystem. In the core, these units include instruction fetch, decode and dispatch units, plus the register files and execution units. The storage subsystem includes the second level (L2) cache and interface units.

The block diagram in *Figure 1-2* on page 35 shows how the two processing units (PU0 and PU1) are connected through the common logic to the processor interface.



Figure 1-1. 970MP Block Diagram

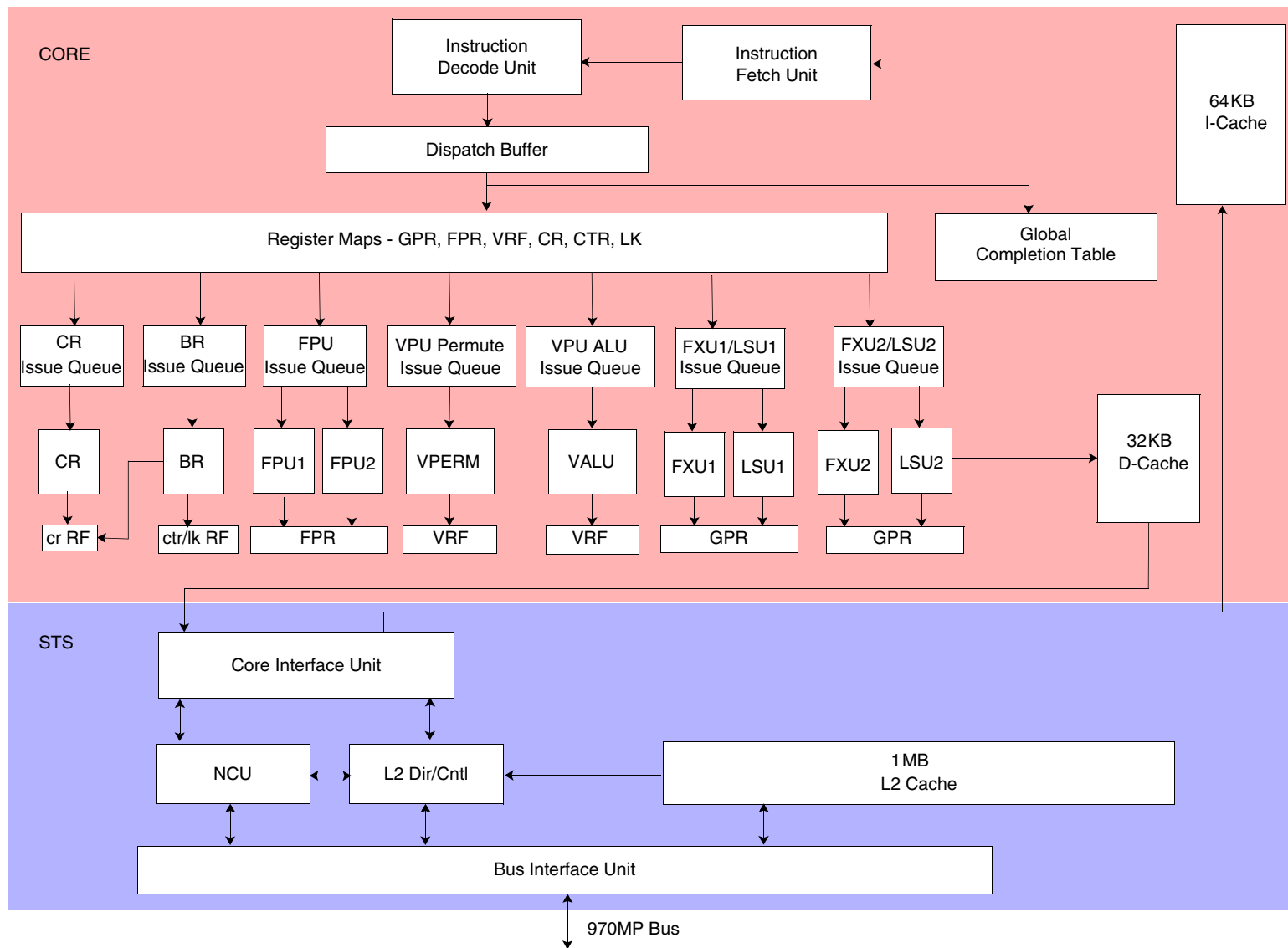
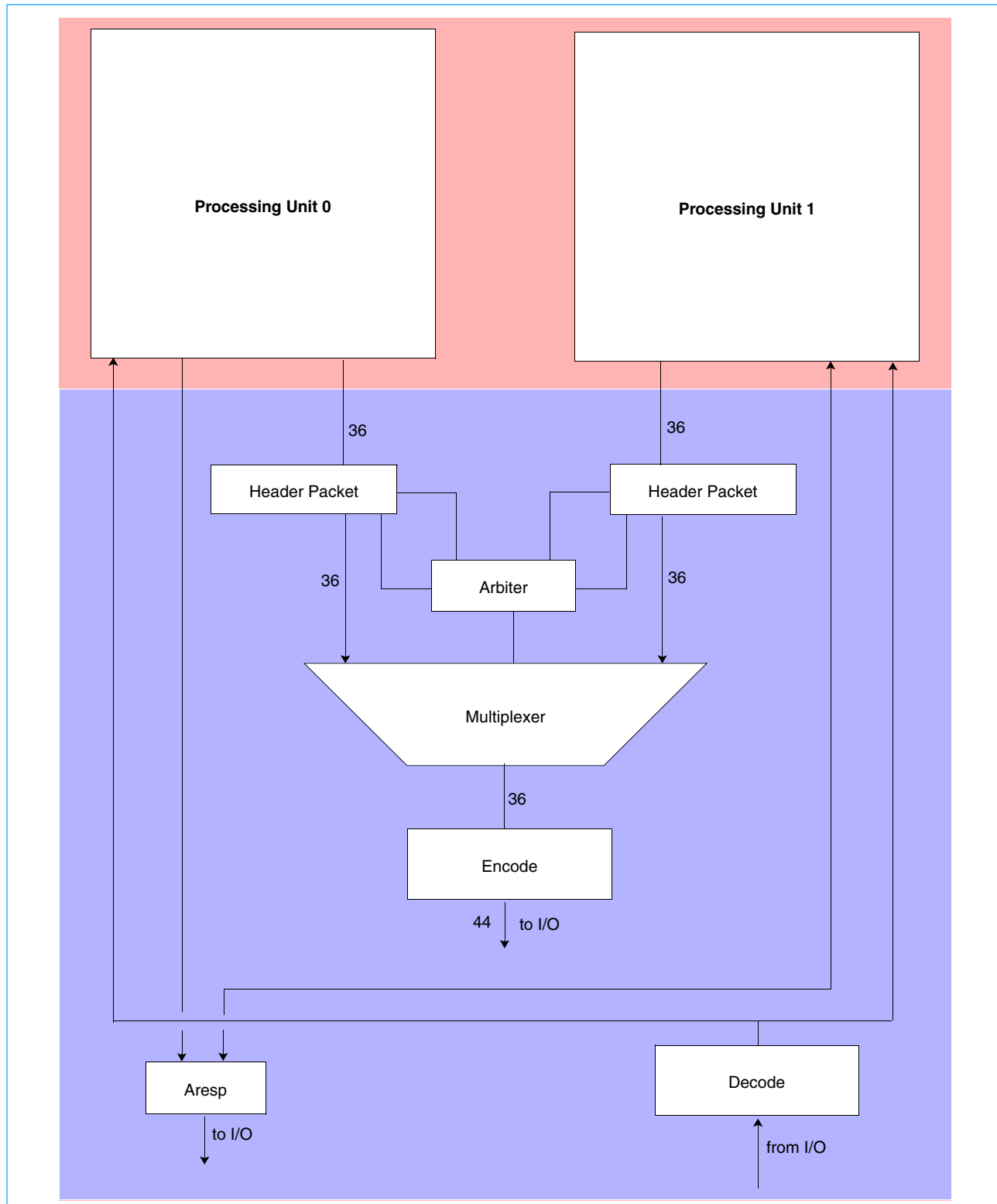


Figure 1-2. 970MP Dual Core with Common Arbitration Logic



1.2 PowerPC 970MP Functional Units

1.2.1 Introduction

This section provides an overview of the 970MP microprocessor core, VPU, storage, and bus interface units of each processing unit. It includes a summary and details of key design fundamentals.

1.2.1.1 Key Design Fundamentals of the Microprocessor Core

- 64-bit implementation of the PowerPC Architecture (version 2.01)
 - Binary compatibility with all PowerPC Architecture application level code (user [problem] state)
 - Support for the 32-bit operating system bridge facility
 - Vector/SIMD multimedia extension
- Layered implementation strategy for very high-frequency operation
 - Deeply pipelined design
 - 16 stages for most fixed-point register-to-register operations
 - 18 stages for most load-and-store operations (assuming an L1 D-cache hit)
 - 21 stages for most floating-point operations
 - 19 stages for fixed-point, 22 stages for complex-fixed, and 25 stages for floating-point operations in the vector arithmetic logic unit (VALU)
 - 19 stages for VPU permute operations
 - Dynamic instruction cracking¹ for some instructions allows for simpler inner core dataflow
 - Dedicated dataflow for cracking one instruction into two internal operations
 - Microcoded templates for longer emulation sequences
- Speculative superscalar inner core organization
 - Aggressive branch prediction
 - Prediction for up to two branches per cycle
 - Support for up to 16 predicted branches in flight
 - Prediction support for branch direction and branch addresses
 - In-order dispatch of up to five operations into the distributed issue queue structure
 - Out-of-order issue of up to 10 operations into 10 execution pipelines
 - Two load or store operations
 - Two fixed-point register-to-register operations
 - Two floating-point operations
 - One branch operation
 - One Condition Register operation
 - One VPU permute operation
 - One VPU arithmetic logic unit (ALU) operation
 - Register renaming on General Purpose Registers (GPRs), Floating-Point Registers (FPRs), Vector Registers (VRs), Condition Register (CR) fields, two bits of the Integer Exception Register (XER), Floating-Point Status and Control Register (FPSCR), the Vector Save/Restore Register (VRSAVE), Vector Status and Control Register (VSCR), Link Register (LR), and Count Register (CTR)

1. Process by which some complex instructions are broken into two simpler, more RISC-like instructions.

- Large number of instructions in flight (theoretical maximum of 215 instructions)
 - Up to 16 instructions in the instruction fetch unit (fetch buffer and overflow buffer)
 - Up to 32 instructions in the instruction fetch buffer in the instruction decode unit
 - Up to 35 instructions in three decode pipe stages and four dispatch buffers
 - Up to 100 instructions in the inner core (after dispatch)
 - Up to 32 stores queued in the store queue (STQ) (available for forwarding)
 - Fast, selective flush of incorrect speculative instructions and results
- Specific focus on storage latency management
 - Out-of-order and speculative issue of load operations
 - Support for up to eight outstanding L1 cache line misses
 - Hardware-initiated instruction prefetching from the L2 cache
 - Software-initiated data stream prefetching with support for up to eight active streams
 - Critical word forwarding—critical sector first
 - New branch processing—Prediction hints on branch instructions
- Power management
 - Static power management
 - Software initiated Doze, Nap, and Deep Nap low-power modes
 - Dynamic power management
 - Parts of the design stop their clocks when not in use under hardware control
 - Power tuning through frequency scaling
 - Software initiated slow down of the processor; selectable to a half or quarter of the nominal operating frequency
 - Programmable latency for power mode transitions to control current spikes

1.2.1.2 Detailed Features of the Microprocessor Core

- Instruction fetching and branch prediction
 - 64KB, direct-mapped instruction cache (I-cache)
 - 128-byte lines (broken into four 32-byte sectors)
 - Dedicated 32-byte read/write interface from the L2 cache with a critical-sector-first reload policy
 - Effective-address index, real address tags
 - Cache supports one read or one write per cycle
 - Five additional predecode bits per word to aid in fast decoding and group formation
 - Parity protected with a force invalidate and reload on parity error
 - 128 total entries in the effective-to-real-address translation (ERAT) cache; 2-way, set-associative
 - Organization is 64 entries by two ways
 - Each entry translates 4 KB (no large page support; large pages take multiple entries)
 - 4-entry, 128-byte, instruction prefetch queue above the I-cache; hardware-initiated prefetches
 - Fetch a 32-byte aligned block of eight instructions per cycle
 - Branch prediction
 - Scan all eight fetched instructions for branches each cycle
 - Predict up to two branches per cycle
 - 3-table prediction structure: global, local, and selector (16 K entries x 1 bit each)
 - 16-entry link stack for address prediction (with stack recovery)
 - 32-entry count cache for address prediction (indexed by the address of the Branch Conditional to Count Register [**bcctr**] instructions)

IBM PowerPC 970MP RISC Microprocessor

- Instruction decode and preprocessing
 - 3-cycle pipeline to decode and preprocess instructions
 - Dedicated dataflow for cracking one instruction into two internal operations
 - Microcoded templates for longer emulation sequences of internal operations
 - All internal operations expanded into 86-bit internal form to simplify subsequent processing and explicitly expose register dependencies for all register pools
 - Dispatch groups (up to five instructions) formulated along with inter-instruction dependence masks
 - Cracked and microcoded instructions have access to four renamed emulation GPRs (eGPRs), one renamed emulation FPR (eFPR), and one renamed emulation CR (eCR) field (in addition to architected facilities)
 - 8-entry (16 bytes per entry) instruction fetch buffer (up to eight instructions in and five instructions out during each cycle)
 - Microcode patch facility allows most instructions other than branches to trap to microcode, which can be programmed to either emulate the effects of the instruction or cause an interrupt.
- Instruction dispatch, sequencing, and completion control
 - Four dispatch buffers, which can hold up to four dispatch groups when the global completion table (GCT) is full
 - 20-entry global completion table
 - Group-oriented tracking associates a 5-operation dispatch group with a single GCT entry
 - Tracks internal operations from dispatch to completion for up to 100 operations
 - Capable of restoring the machine state for any of the instructions in flight
 - Very fast restoration for instructions on group boundaries (that is, branches)
 - Slower for instructions contained within a group
 - Supports precise exceptions (including machine check interrupt)
 - Register renaming resources
 - 80-entry GPR rename mapper (32 architected GPRs plus four eGPRs and VRSAVE)
 - 80-entry FPR rename mapper (32 architected FPRs plus one eFPR)
 - 80-entry Vector Register file (VRF) rename mapper (32 architected VRFs)
 - 24-entry XER rename mapper (the XER is broken into mappable and nonmappable fields)
 - Two mappable fields: ov and ca
 - Nonmappable field: string-count
 - 16-entry LR/CTR rename mapper (one architected LR and one architected CTR)
 - 32-entry CR rename mapper (eight architected CR fields plus one eCR field)
 - 20-entry FPSCR rename mapper
 - VRSAVE
 - VSCR
 - Instruction queuing resources:
 - Two 18-entry issue queues for fixed-point and load/store instructions
 - Two 10-entry issue queues for floating-point instructions
 - 12-entry issue queue for branch instructions
 - 10-entry issue queue for CR-logical instructions
 - 16-entry issue queue for vector permute instructions
 - 20-entry issue queue for vector ALU instructions and VPU stores

- Two fixed-point execution pipelines
 - Both capable of basic arithmetic, logical, and shifting operations
 - Both capable of multiplies
 - One capable of divides; the other capable of SPR operations
- Out-of-order issue with bias towards oldest operations first
- Symmetric forwarding between fixed-point and load/store execution pipelines
- Load/store execution pipelines
 - Two 6-stage load/store execution pipelines
 - Out-of-order issue with bias towards oldest operations first
 - Stores issue twice—an address generation operation (load/store), and a data steering operation (FXU/FPU/VPU)
 - 32KB, 2-way, set-associative D-cache
 - Triple ported to support two reads and one write every cycle (no banking)
 - 2-cycle load-use penalty for FXU loads
 - 4-cycle load-use penalty for FPU loads
 - 3-cycle load-use penalty for loads to vector permute unit (VPERM)
 - 4-cycle load-use penalty for loads to VALU
 - Store-through policy; no allocation on store misses
 - 128-byte cache line
 - Least recently used (LRU) replacement policy
 - Dedicated 32-byte reload interface from the L2 cache
 - Effective-address index, real address tags (hardware fix up on alias cases)
 - Parity protected; precise machine check interrupt on parity error; software fix if HID5[50] equals '1'. Otherwise, recovery is done by hardware (default).
 - 128-entry (total) ERAT cache, 2-way, set-associative
 - Organization is 64 entries by two ways
 - Each entry translates 4 KB (no large page support; large pages take multiple entries)
 - 32-entry store queue logically above the D-cache (real address based; content-addressable memory [CAM] structure)
 - Store addresses and store data can be supplied on different cycles
 - Stores wait in this queue until they are completed; then they write the cache
 - Supports store forwarding to inclusive subsequent loads (even if both are speculative)
 - 32-entry load reorder queue (real address based; CAM structure)
 - Keeps track of out-of-order loads and watches for hazards
 - Previous store to the same address that gets executed after the load causes a flush
 - Previous load from the same address when a cross-invalidate has occurred causes a flush
 - 8-entry load miss queue (LMQ) (real address based)
 - Keeps track of loads that have missed in the L1 D-cache
 - Allows a second load from the same cache line to merge onto a single entry

IBM PowerPC 970MP RISC Microprocessor

- Branch and Condition Register execution pipelines
 - One branch execution pipeline
 - Computes actual branch address and branch direction for comparison with prediction
 - Redirects instruction fetching if either prediction was incorrect
 - Assists in training and maintaining the branch table predictors, the link stack, and the count cache
 - One Condition Register logical pipeline
 - Executes CR logical instructions and the CR movement operations
 - Executes some Move To Special Purpose Register (**mtspr**) and Move From Special Purpose Register (**mfspr**) instructions also
 - Out-of-order issue with a bias towards oldest operations first
- Floating-point execution pipelines
 - Two 9-stage floating-point execution pipelines (6-stage execution)
 - Both capable of the full set of floating-point instructions
 - All data formats supported in hardware (no floating-point assist interrupts)
 - Out-of-order issue with bias towards oldest operations first
 - Symmetric forwarding between the floating-point pipelines
 - No support for the non-IEEE mode
- VPU execution pipelines
 - Two dispatchable units:
 - VALU contains three subunits:
 - Vector simple fixed: 1-stage execution
 - Vector complex fixed: 4-stage execution
 - Vector floating point: 7-stage execution
 - VPERM: 1-stage execution
 - Out-of-order issue with a bias towards oldest operations first
 - Symmetric forwarding between the permute and VALU pipelines
- Unified second-level memory management (address translation)
 - 1024-entry, 4-way, set-associative translation lookaside buffer (TLB)
 - Supports new large page architecture (16MB large pages supported)
 - Hardware-based reload (from the L2 cache interface; no L1 D-cache impact)
 - Hardware-based update of the referenced (R) and changed (C) bits in a page table entry (PTE)
 - Parity protected; precise machine check interrupt on parity error (software fix up)
 - 64-entry fully associative segment lookaside buffer (SLB)
 - SLB miss results in an interrupt; software reload of the SLB
 - SLB can also be loaded by the 32-bit PowerPC Segment Register instructions
 - Supports a 65-bit virtual address and a 42-bit real address
- Data stream prefetch
 - Eight data prefetch streams supported in hardware. Eight hardware streams are only available if VPU prefetch instructions are disabled.
 - Four VPU prefetch streams supported using four of the eight hardware streams. The VPU prefetch mapping algorithm supports most commonly used forms of vector prefetch instructions.

1.3 970MP Dual-Core Module

The 970MP chip consists of two processing units, each containing an execution core with L1 caches, a storage subsystem including an L2 cache, and pervasive functions. In addition, a small amount of common logic that is outside either processing unit is included to connect each processing unit to the single bus interface.

Generally, the two processing units function as would two processing units on separate chips. For example, they maintain memory coherence through the North Bridge; they are able to Doze independently; they have private access to most processor resources, including their own L2 cache. Also, like processors on separate chips, they scale frequency together, using the power tuning facility.

However, sharing the same chip constrains the behavior of the two processing units in several ways. First, the two processing units have separate voltage planes for power, but the processing unit voltages will always be the same when the two processors are running. Similarly, the processing unit frequencies will always be the same. The two 970MP processing units must go into and come out of Deep Nap together.

The other difference between having dual processing units on a single chip, versus two separate chips, is that they share a single processor interface (PI) to the North Bridge. This requires that the interface between the bus interface unit (BIU) and the PI logic be enhanced with buffers and multiplexers to support the sharing of the PI between the two processing units. *Figure 1-2* on page 35 shows the relationship among the two processing units and the common logic.

For incoming PI data and commands, the output of the PI decoder is passed directly to both processors. For outgoing PI data and commands, an arbiter and multiplexer are introduced in front of the PI encoder to give one or the other processor access to the outgoing PI bus at any given time. The arbiter implements a round robin scheme, with provisions for adjusting priorities when one processing unit receives repeated serial retries. Logic in the BIU of each processing unit is modified to allow the arbiter to prevent that processing unit from sending data to the PI bus when a transaction from the other processor is in progress. The PI bus configuration parameters apply to a single bus, not to the individual processors. The arbiter enforces those parameters, such as the command pipeline delay (COMPACT) timing. To minimize dead time on the bus, header packets for each processor are queued at the arbiter. Finally, snoop responses from the two processors are combined on chip, and sent as a single response over the PI bus to the North Bridge, as indicated in the lower left corner of *Figure 1-2* on page 35.

The additional logic at the PI/BIU interface might require different values for the programmable bus delay parameters than those used for the previous design. The range of some of these parameters has therefore been increased (see *Table 11-1* on page 281).

Intercommunication between the processors on chip occurs just as if they were on separate chips, through the North Bridge. In particular, on-chip L2-to-L2 intervention is not supported.



2. Programming Model

This chapter describes the 970MP programming model, emphasizing those features specific to the 970MP microprocessor and summarizing those that are common to PowerPC processors. It consists of two major sections, which describe the following:

- Registers implemented in each 970MP processing unit
- 970MP instruction set

2.1 970MP Processor Register Set

This section describes the registers implemented in each 970MP processing unit. It includes an overview of registers defined by the PowerPC Architecture, highlighting differences in how these registers are implemented in the 970MP processing units. It also includes a detailed description of 970MP-specific registers.

Registers are defined at all three levels of the PowerPC Architecture—user instruction set architecture (UISA), virtual environment architecture (VEA), and operating environment architecture (OEA). The PowerPC Architecture defines register-to-register operations for all computational instructions. Source data for these instructions is accessed from the on-chip registers or is provided as immediate values embedded in the opcode. The 3-register instruction format allows specification of a target register distinct from the two source registers, thus preserving the original data for use by other instructions and reducing the number of instructions required for certain operations. Data is transferred between memory and registers with explicit load-and-store instructions only.

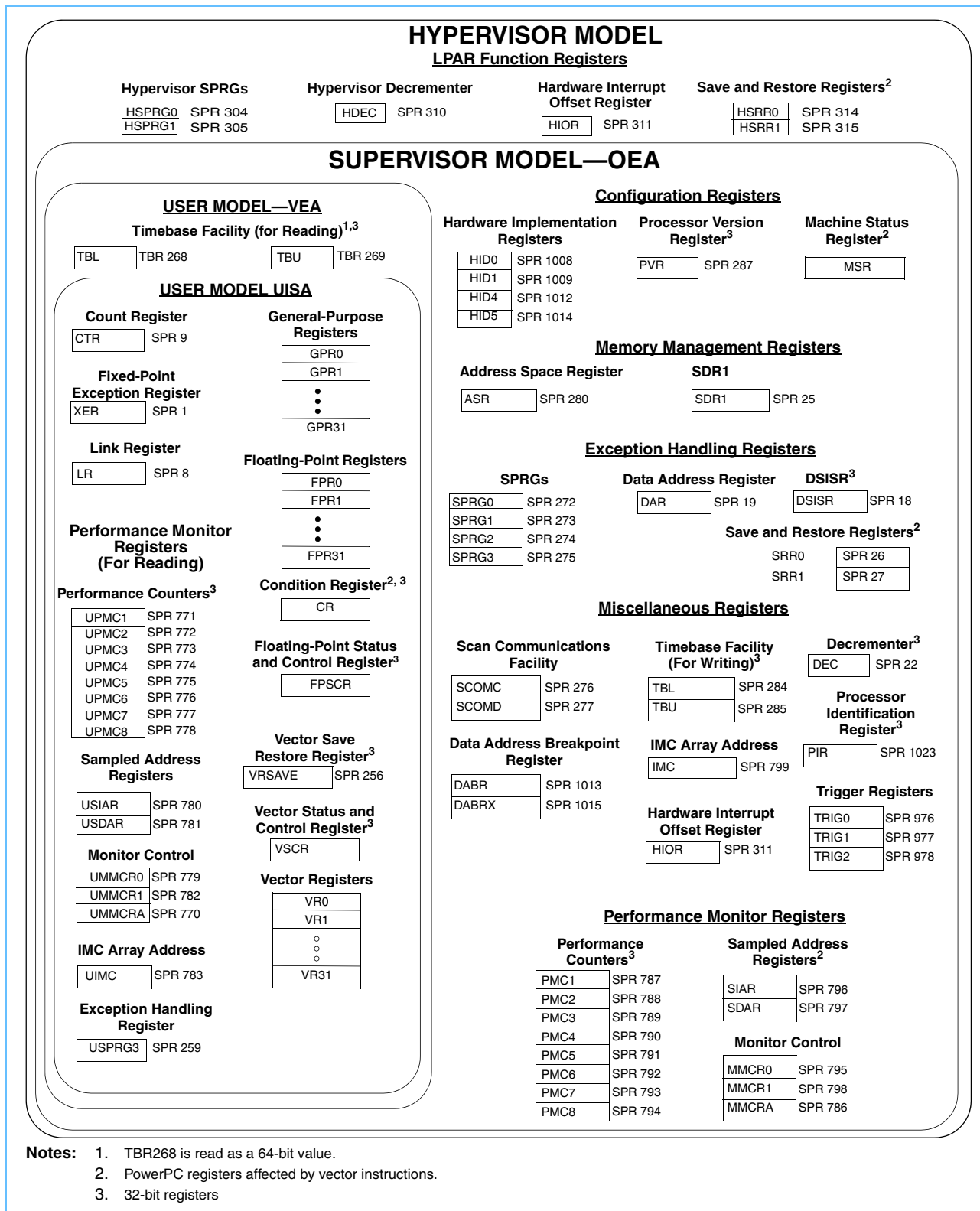
PowerPC processors have two levels of privilege—supervisor mode of operation (typically used by the operating system) and user mode of operation (used by the application software; also called problem state). The programming models incorporate 32 General Purpose Registers (GPRs), 32 Floating-Point Registers (FPRs), 32 Vector Registers (VRs), Special-Purpose Registers (SPRs), and several miscellaneous registers. Each PowerPC microprocessor also has its own unique set of Hardware Implementation-Dependent (HID) Registers.

While running in supervisor mode, the operating system is able to execute all instructions and access all registers defined in the PowerPC Architecture. In this mode, the operating system establishes all address translations and protection mechanisms, loads all Processor State Registers, and sets up all other control mechanisms defined on the 970MP microprocessor. While running in user mode (problem state), many of these registers and facilities are not accessible. Any attempt to read or write to these registers in user mode results in a program exception.

The registers implemented on each of the 970MP processing units are shown in *Figure 2-1 970MP Programming Model—Registers* on page 44. The number to the right of the SPRs indicates the number that is used in the syntax of the instruction operands to access the register (for example, the number used to access the Integer Exception Register (XER) is SPR 1). These registers can be accessed using the Move To Special Purpose Register (**mtspr**) and Move From Special Purpose Register (**mfspr**) instructions. The inclusion of the vector processing unit (VPU) involves additional registers, and affects bit settings in some of the PowerPC registers (including the Machine State Register [MSR], Machine Status Save/Restore Register 1 [SRR1], and Condition Register [CR]) when the VPU facility is in use.

IBM PowerPC 970MP RISC Microprocessor

Figure 2-1. 970MP Programming Model—Registers



The PowerPC UISA registers are user-level. GPRs, FPRs, and VRs are accessed through instruction operands. Access to registers can be explicit (by using instructions for that purpose such as **mtspr** and **mfspr**) or implicit as part of the execution of an instruction. Some registers are accessed both explicitly and implicitly.

Implementation Note: The 970MP microprocessor fully decodes the special purpose register (SPR) field of the instruction. If the SPR specified is undefined, an illegal instruction program exception occurs.

- **User-level registers (UISA)**—The user-level registers can be accessed by all software with either user or supervisor privileges. They include the following registers:
 - General-Purpose Registers (GPRs). The 32 GPRs (GPR0–GPR31) serve as data source or destination registers for fixed-point instructions and provide data for generating addresses.
 - Floating-Point Registers (FPRs). The 32 FPRs (FPR0–FPR31) serve as the data source or destination for all floating-point instructions.
 - Condition Register (CR). The 32-bit CR consists of eight 4-bit fields, CR0–CR7, that reflect results of certain arithmetic operations and provide a mechanism for testing and branching.
 - Floating-Point Status and Control Register (FPSCR). The FPSCR contains all floating-point exception signal bits, exception summary bits, exception enable bits, and rounding control bits needed for compliance with the IEEE 754 standard.
 - Vector Registers (VRs). The vector register file consists of 32 VRs (VR0–VR31). The VRs serve as vector source and vector destination registers for all vector instructions.
 - Vector Status and Control Register (VSCR). The VSCR contains the non-Java™ control bit and the saturation status bit associated with vector operations.

The remaining user-level registers are SPRs. Note that the PowerPC Architecture provides a separate mechanism for accessing SPRs (the **mtspr** and **mfspr** instructions). These instructions are commonly used to explicitly access certain registers, while other SPRs might be more typically accessed as a side effect of executing other instructions.

- Link Register (LR). The LR provides the branch target address for the Branch Conditional to Link Register (**bclrx**) instruction. It can be used to hold the logical address of the instruction that follows a branch and link instruction, typically used for linking to subroutines.
- Count Register (CTR). The CTR holds a loop count that can be decremented during execution of appropriately coded branch instructions. The CTR can also provide the branch target address for the Branch Conditional to Count Register (**bcctrx**) instruction.
- Vector Save/Restore Register (VRSAVE). The VRSAVE assists the application and operating system software in saving and restoring the Vector Register architectural state across context-switching events.
- User Performance Monitor Counter Registers (UPMC1–UPMC8). UPMC1–UPMC8 provide user-level read access to the Performance Monitor Counter Registers (PMC1–PMC8).
- User Monitor Mode Control Registers (UMMCR0, UMMCR1, UMMCRA). These registers provide user-level read access to the Monitor Mode Control Registers (MMCR0, MMCR1, MMCRA).
- User Instruction Match Content-Addressable Memory (CAM) Register (UIMC). The UIMC provides user-level read access to the Instruction Match CAM Register (IMC).
- User Sampled Instruction Address Register (USIAR). The USIA provides user-level read access to the Sampled Instruction Address Register (SIAR).

IBM PowerPC 970MP RISC Microprocessor

- User Sampled Data Address Register (USDAR). The USDA provides user-level read access to the Sampled Data Address Register (SDAR).
- Integer Exception Register (XER). The XER indicates overflow and carries for integer operations and the number of bytes to be transferred by the indexed instructions of the load/store string.

Implementation Note: The architecture defines XER[44:56] as reserved.

- Software Use Special Purpose Register 3 (SPRG3). SPRG3 can be read in problem state using SPR 259.
- **User-level registers (VEA)**—The PowerPC VEA defines the time-base facility (TB), which consists of two 32-bit registers—Time-Base Upper (TBU) and Time-Base Lower (TBL). The Time-Base Registers can be written to only by supervisor-level instructions, but can be read by both user and supervisor-level software.
- **Supervisor-level registers (OEA)**—The OEA defines the registers that an operating system uses for memory management, configuration, exception handling, and other operating system functions. The OEA defines the following supervisor-level registers:
 - Configuration registers
 - Machine State Register (MSR). The MSR defines the state of the processor. The MSR can be modified by the Move to Machine State Register (**mtmsr**), Move to Machine State Register Doubleword (**mtmsrd**), System Call (**sc**), and Return from Exception Doubleword (**rfid**) instructions. It can be read by the Move from Machine State Register (**mfmsr**) instruction. When an exception is taken, the contents of the MSR are saved to the Machine Status Save/Restore Register 1 (SRR1). See *Section 2.1.1.1 MSR Register (MSR)* on page 49 for more information.
 - Processor Version Register (PVR). This is a read-only register that identifies the version (model) and revision level of the PowerPC processor. See the *IBM PowerPC 970MP RISC Microprocessor Datasheet* for details of the PVR.
 - Memory management registers
 - Address Space Register (ASR). In the 970MP microprocessor, the Address Space Register is supported. Due to the software reload of the segment lookaside buffers (SLBs) on the 970MP microprocessor, this register does not actually participate in any other specific hardware functions on the chip. It has been included as a convenience (and performance enhancement) for the SLB reload software.
 - Storage Description Register (SDR1). SDR1 specifies the page-table base address used in virtual-to-physical address translation.
 - Exception-handling registers
 - Data Address Register (DAR). After a data storage interrupt (DSI) exception or an alignment exception, the DAR is set to the effective address (EA) generated by the faulting instruction.
 - Software Use Special Purpose Registers 0 - 3 (SPRG0–SPRG3). SPRG0–SPRG3 are provided for operating system use.
 - Data Storage Interrupt Status Register (DSISR). DSISR defines the cause of DSI and alignment exceptions.
 - Machine Status Save and Restore Register 0 (SRR0). SRR0 is used to save the address of the instruction at which execution continues when **rfid** executes at the end of an exception handler routine. See *Section 2.1.1.2 Machine Status Save/Restore Register (SRR1)* on page 49 for more information.

- Machine Status Save and Restore Register 1 (SRR1). SRR1 is a 64-bit register used to save machine status on exceptions and restore the Machine Status Register when an **rfid** instruction is executed. See *Section 2.1.1.2 Machine Status Save/Restore Register (SRR1)* on page 49 for more information.

Note: For information about how specific exceptions affect SRR1, see the individual exception in *Section 4.5 Exception Definitions* on page 110.

– Miscellaneous registers

- Time Base (TB). This register is a 64-bit structure provided for maintaining the time of day and operating interval timers. The TB consists of two 32-bit registers—Time-Base Upper (TBU) and Time-Base Lower (TBL). The Time-Base Registers can be written to only by supervisor-level software, but can be read by both user- and supervisor-level software. See *Section 2.1.1.3 Time Base and Decrementer (TB, DEC)* on page 50 for more information.

Implementation Note: In the 970MP microprocessor, the Time-Base Register is incremented once every sixteen full frequency processor clocks. Alternatively, when HID0[19] is set to '1', the Time-Base Register is incremented at the input frequency of the timebase_enable input pin (TBEN).

- Decrementer Register (DEC). This register is a 32-bit decrementing counter that provides a mechanism for causing a decrementer exception after a programmable delay. See *Section 2.1.1.3 Time Base and Decrementer (TB, DEC)* on page 50 for more information.

Implementation Note: In the 970MP microprocessor, the Decrementer Register is decremented once every 16 full frequency processor clocks. Alternatively, when HID0[19] is set to '1', the Decrementer Register is decremented at the TBEN input frequency.

- Processor ID Register (PIR). The PIR Register is used to differentiate between individual processors in a multiprocessor environment. See *Section 2.1.1.4 Processor ID Register (PIR)* on page 50 for more information.

– Performance Monitor Registers. The following registers are used to define and count events for use by the performance monitor:

- The Performance Monitor Counter Registers (PMC1–PMC8) are used to record the number of times a certain event has occurred. See *Section 2.1.2.6 Performance Monitor Registers (MMCR0, MMCR1, MMCR2, PMC1-8)* on page 63 for more information.
- The Monitor Mode Control Registers (MMCR0, MMCR1, MMCR2) are used to identify what events will be monitored and to enable various performance monitor interrupt functions. See *Section 2.1.2.6 Performance Monitor Registers (MMCR0, MMCR1, MMCR2, PMC1-8)* on page 63 for more information.
- The Sampled Instruction Address Register (SIAR) contains the effective address of an instruction executing at or around the time that the processor signals the performance-monitor interrupt condition. See *Section 2.1.2.7 Sampled Instruction Address and Sampled Data Address Registers (SIAR, SDAR)* on page 64 for more information.
- The Sampled Data Address Register (SDAR) contains the effective address of the storage access instruction executing at or around the time that the processor signals the performance monitor interrupt condition. See *Section 2.1.2.7 Sampled Instruction Address and Sampled Data Address Registers (SIAR, SDAR)* on page 64 for more information.

IBM PowerPC 970MP RISC Microprocessor

- **970MP-specific registers**—The PowerPC Architecture allows implementation- specific SPRs. The following registers are incorporated in each 970MP processing unit:

Note: In the 970MP microprocessor, these registers are all supervisor-level registers.

- Hardware Implementation-Dependent Register 0 (HID0). This register controls various functions, such as enabling checkstop¹ conditions, locking, enabling, invalidating the instruction and data caches, and power modes. See *Section 2.1.2.2 HID Registers (HID0, HID1, HID4, and HID5)* on page 54 for more information.
- Hardware Implementation-Dependent Register 1 (HID1). HID1 contains additional mode bits that are related to the instruction fetch and instruction decode functions in the 970MP microprocessor. See *Section 2.1.2.2 HID Registers (HID0, HID1, HID4, and HID5)* on page 54 for more information.
- Hardware Implementation-Dependent Register 4 (HID4). HID4 contains bits related to the load/store function in the 970MP microprocessor. See *Section 2.1.2.2 HID Registers (HID0, HID1, HID4, and HID5)* on page 54 for more information.
- Hardware Implementation-Dependent Register 5 (HID5). HID5 contains bits related to the load/store function in the 970MP microprocessor. See *Section 2.1.2.2 HID Registers (HID0, HID1, HID4, and HID5)* on page 54 for more information.
- Data Address Breakpoint Register (DABR) and Data Address Breakpoint Register Extension (DABRX). The DABR controls the data address breakpoint mechanism, which provides a means of detecting load-and-store accesses to a designated double word. See *Section 2.1.2.3 Data Address Breakpoint Register (DABR)* on page 61 for more information.
- Scan Communications Register (SCOMC). SCOMC is a control register that includes a command field, a destination field, and a set of status bits. See *Section 2.1.2.8 Scan Communication Registers (SCOMC and SCOMD)* on page 64 for more information.
- Scan Communications Register (SCOMD). SCOMD is an associated data register that acts as either a source of data or as a destination for data depending on the command placed into the SCOMC. See *Section 2.1.2.8 Scan Communication Registers (SCOMC and SCOMD)* on page 64 for more information.
- Instruction Match CAM Registers (IMCs). The IMC SPRs are used to access the IMC array, which contains the mask values used for instruction matching. The **mtimc** and **mfimc** instructions can be executed only in supervisor mode. See *Section 2.1.2.5 Instruction Match CAM Array Access Register (IMC)* on page 62 for more information.
- Trigger Registers (TRIG0-TRIG2). Writes to the Trigger Registers, named TRIG0, TRIG1, and TRIG2, can be inserted in the instruction stream to cause triggers to the on-chip trace array debug logic. These are intended to be used for debug and bring-up only and architecturally behave as no-ops. See *Section 2.1.2.12 Trigger Registers (TRIG0, TRIG1, TRIG2)* on page 65 for more information.
- Hardware Interrupt Offset Register (HIOR). The HIOR is used for interrupt vector relocation. See *Section 2.1.2.13 Hardware Interrupt Offset Register (HIOR)* on page 66 for more information.

Note: While it is not guaranteed that the implementation of 970MP-specific registers is consistent among PowerPC processors, other processors might implement similar or identical registers.

1. Hardware has detected a condition that it cannot resolve, and which prevents normal operation. It stops executing instructions, responding to interrupts, and so on.

2.1.1 Architected Registers in the 970MP Implementation

Several architected registers are implemented in each 970MP processing unit in a way that varies from, or extends, the definition in the PowerPC Architecture.

2.1.1.1 MSR Register (MSR)

The PowerPC Architecture describes the MSR bits 2, 4:47, 57, 60, and 63 as either optional or reserved. In the 970MP microprocessor, bit 38 is used as the vector processor available (VP) enable and bit 45 is used as the power management (POW) enable. The other bits are not implemented and will return the value '0' when read.

Note: Little-endian mode is not supported (that is, MSR[LE] and MSR[ILE] are treated as reserved).

Implementation Note: *Table 2-1* describes MSR bits that the 970MP microprocessor implements that deviate from the PowerPC Architecture.

Table 2-1. MSR Bits

Bit	Name	Description
3	—	Reserved; returns a value of '1' when read.
38	VP	VP available. 0 The processor prevents execution of all vector instructions including loads, stores, and moves. If such execution is attempted, a VP unavailable exception is raised. 1 The processor can execute all vector instructions. The VRSAVE Register is not protected by MSR [VP]. The data streaming family of instructions (dst , dstt , dstst , dststt , dss , and dssall) are not affected by the MSR[VP].
45	POW	Activates power management. The 970MP microprocessor will clear the POW bit when it leaves a power saving mode. See <i>Chapter 9 Power and Thermal Management</i> for more information.
47	—	Reserved. The ILE bit is not implemented in the 970MP microprocessor.
48	EE	External interrupt enable 0 The processor delays recognition of external interrupts and decremter exception conditions. 1 The processor is enabled to take an external interrupt or the decremter exception. Note: Resetting MSR[EE] masks not only the architecture-defined external interrupt and decremter exceptions, but also the 970MP-specific instrumentation, debug, and performance monitor exceptions.
63	—	Reserved. The LE bit is not implemented in the 970MP microprocessor.

2.1.1.2 Machine Status Save/Restore Register (SRR1)

This register is used to save machine status during interrupts. In the 970MP microprocessor, SRR1 bits 1:2, 4:32, 37, 39:41, 47, 56:57, 60, and 63 are treated as reserved. These bits are not implemented and will return the value '0' when read. See *Section 4.3.2 Machine Status Save/Restore Register 1 (SRR1)* on page 105 for additional information.

Table 2-2. Additional SRR1 Bit

Bit	Function
33	SIAR and SDAR contents synchronized.

IBM PowerPC 970MP RISC Microprocessor

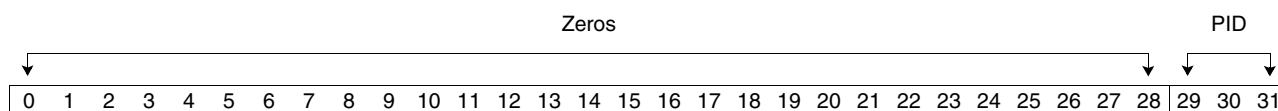
2.1.1.3 Time Base and Decrementer (TB, DEC)

The time-base counter and the decrementer are clocked at 1/16 (one-sixteenth) of the full frequency processor. The 970MP microprocessor supports two modes of operation (controlled by HID0[19] and the time-base enable input pin) for updating the time base and decrementer. When HID0[19] is zero, then the counters constantly update as long as the TBEN is high (traditional mode of operation). When HID0[19] is one, the counters update only on the rising edge of the TBEN input pin.

When the processor is stopped due to various breakpoint, debug and support processor functions, an additional mode bit, HID0[18], determines whether the time base and the decrementer continue counting. Note that some support processor operations require the use of an alternate clocking mode for scan, and in these cases, the time base and the decrementer will not continue counting.

2.1.1.4 Processor ID Register (PIR)

The Processor Identification Register (PIR) is a 32-bit register that holds a processor identification tag. In the 970MP processing unit, this tag is in the three least-significant bits (29:31). The least-significant bit of the processor identification tag (PID) is hardwired to '0' for PU0 and to '1' for PU1. This tag is used to tag bus transactions and to differentiate processors in multiprocessor systems. The PIR is a read-only register. The format of the register is as follows:



Bits	Field Name	Description
0:28	—	Reserved (read as zeros)
29:31	PID	3-bit processor ID value (least-significant bit hardwired to differentiate PU0 and PU1)

During power-on reset, PID is set to a unique value for each processor in a multi processor system. For more information about the power-on reset configuration process, see *Section 11.3.1 Initialization at Power-On Reset* on page 289.

2.1.2 PowerPC 970MP-Specific Registers

This section describes registers that are defined for the 970MP microprocessor, but are not included in the PowerPC Architecture.

2.1.2.1 Move To and Move From System Register Instructions

The 970MP architecture defines several new implementation-specific system registers. Note that some of these registers are also readable in user mode through a second set of SPR encodings, and that some of these registers have special software synchronization requirements.

The encoded SPR values for these implementation-specific registers are shown in *Table 2-3*. Note that the SPR is encoded in the **mf spr** and **mt spr** instructions. Bits 5:9 of the SPR field represent the 5 high-order bits of the SPR number, and bits 0:4 of the SPR field represent the 5 low-order bits of the SPR number.

Table 2-3. Implementation-Specific SPRs

SPR			Register Name	R/W	Synchronization Requirements		
Decimal (privileged)	Decimal (user)	SPR(5:9) SPR(0:4)			Before Reads	After Writes	Before Writes
1023		11111 11111	PIR	R	none	N/A	N/A
1013		11111 10101	DABR	R/W	none	context synchronizing instruction (CSI)	sync
1015		11111 10111	DABRX	R/W			
1008		11111 10000	HID0	R/W	none	Note 1	Note 1
1009		11111 10001	HID1	R/W	none	Note 2	Note 2
1012		11111 10100	HID4	R/W	none	Note 3	Note 3
1014		11111 10110	HID5	R/W	none	Note 4	Note 4
795	779	11000 n1011	MMCR0	R/W	none	Note 5	Note 5
798	782	11000 n1110	MMCR1	R/W	none	Note 5	Note 5
786	770	11000 n0010	MMCR4	R/W	none	Note 5	Note 5
787	771	11000 n0011	PMC1	R/W	sync	none	none
788	772	11000 n0100	PMC2	R/W	sync	none	none
789	773	11000 n0101	PMC3	R/W	sync	none	none
790	774	11000 n0110	PMC4	R/W	sync	none	none
791	775	11000 n0111	PMC5	R/W	sync	none	none
792	776	11000 n1000	PMC6	R/W	sync	none	none
793	777	11000 n1001	PMC7	R/W	sync	none	none
794	778	11000 n1010	PMC8	R/W	sync	none	none
276		01000 10100	SCOMC	R/W	none	CSI	none
277		01000 10101	SCOMD	R/W	none	CSI	none
796	780	11000 n1100	SIAR	R/W	sync	none	none
797	781	11000 n1101	SDAR	R/W	sync	none	none
799	783	11000 n1111	IMC	R/W	none	CSI	none
976		11110 10000	TRIG0	W	N/A	none	none
977		11110 10001	TRIG1	W	N/A	none	none
978		11110 10010	TRIG2	W	N/A	none	none
256		01000 00000	VRSAVE	R/W	N/A	none	none
311		01001 10111	HIOR	R/W			

Table 2-3. Implementation-Specific SPRs (Continued)

For **mtspr**, n must be '1'. For **mfspir**, reading the SPR is privileged if and only if n equals '1'.

Notes:

1. The following sequence must be used when modifying HID0:

```
sync
mtspr HID0,Rx
mfspir Rx,HID0
mfspir Rx,HID0
mfspir Rx,HID0
mfspir Rx,HID0
mfspir Rx,HID0
mfspir Rx,HID0
```

After modifying HID0, executing six **mfspir** instructions specifying HID0 as the source and specifying the same target General Purpose Register (GPR) (Rx) in all six instructions is necessary to ensure that the modification is effective and that the processor is in a valid state to continue executing subsequent instructions.

2. The following sequence must be used when modifying HID1:

```
mtspr HID1,Rx
mtspr HID1,Rx
isync
```

Executing two **mtspir** instructions is necessary to ensure that updates to all portions of HID1 will be complete before the Instruction Cache Synchronize (**isync**) instruction completes.

3. The following sequence must be used when modifying HID4:

```
sync
mtspr HID4,Rx
isync
```

When HID4[23] is changed, the previous sequence should be preceded by a Move to Segment Register (**mtsr**) and Synchronize (**sync**) instruction, which will cause the effective-to-real-address translations (ERATs) to be flushed.

4. The following sequence must be used when modifying HID5:

```
sync
mtspr HID5,Rx
isync
```

Whenever HID5[56] or HID5[57] is changed, the entire instruction cache must be flushed to ensure that any succeeding Data Cache Block Set to Zero (**dcbz**) instruction is executed in the context of the new HID5 bit settings.

5. Although it is not necessary to use synchronizing instructions when modifying the MMCR(0,1,A) registers, it is recommended that the following sequence be used:

```
sync
mtspir MMCRz,Rx
isync
```

IBM PowerPC 970MP RISC Microprocessor

Table 2-4 describes the behavior of the 970MP microprocessor for the **mtspr** and **mfspir** instructions.

Table 2-4. Move To/Move From SPR Behavior

Condition				Resulting Action
SPR		MSR[PR]	R/W	
SPR(0)	Register			
1	Any invalid SPR encoding	0	mfspir	No-op (target register is unchanged)
1	Any invalid SPR encoding	0	mtspr	No action (write is inhibited)
1	Address Compare Control Register (ACCR), ASR, Control Register (CTRL), DABR, DAR, DEC, DSISR, HID0, HID1, HID4, HID5, IMC, SCOMC, SCOMD, SDR1, SDAR, SIAR, SRR0, SRR1, SPRG0, SPRG1, SPRG2, SPRG3, TBL, TBU, Performance Monitor Registers	0	mfspir	Returns a value to a GPR.
		0	mtspr	Target SPR is updated.
1	TRIG0, TRIG1, TRIG2	0	mfspir	Causes an illegal instruction type of program interrupt.
		0	mtspr	Causes a trigger to the trace array debug logic.
1	PIR	0	mfspir	Returns a value to a GPR.
		0	mtspr	Causes an illegal instruction type of program interrupt.
1	Any SPR encoding (with SPR(0) equal to '1')	1	mtspr mfspir	Causes a privileged instruction type of program interrupt.
0	Any invalid SPR encoding except: spr(0:9) = '00000 00000' spr(0:9) = '00100 00000' spr(0:9) = '00101 00000' spr(0:9) = '00110 00000'	X	mfspir	No-op (target register is unchanged)
		X	mtspr	No action (write is inhibited)
0	spr(0:9) = '00000 00000' spr(0:9) = '00100 00000' spr(0:9) = '00101 00000' spr(0:9) = '00110 00000'	X	mtspr mfspir	Causes an illegal instruction type of program interrupt.

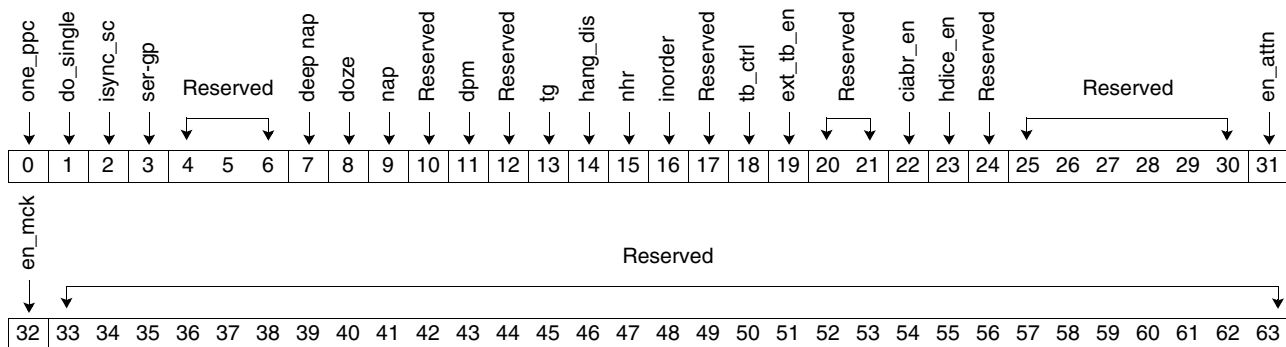
IBM PowerPC 970MP RISC Microprocessor

2.1.2.2 HID Registers (HID0, HID1, HID4, and HID5)

The 970MP microprocessor includes many implementation-dependent mode bits that allow various features of the chip to be enabled and disabled. These bits are included in the Hardware Implementation-Dependent Registers (HID0, HID1, HID4, and HID5). In general, HID0 attempts to line up the 970MP microprocessor modes with the relevant ones from earlier PowerPC implementations and then adds a few new ones. HID1 contains additional mode bits that are related to the instruction fetch and instruction decode functions in the 970MP microprocessor. HID4 and HID5 contain bits related to the load/store function in the 970MP microprocessor. All of these registers are supervisor resources.

The state of each of the HID Registers after a normal scan-based POR is all zeros. The preferred state of these registers for optimal performance and function is also all zeros, except where indicated.

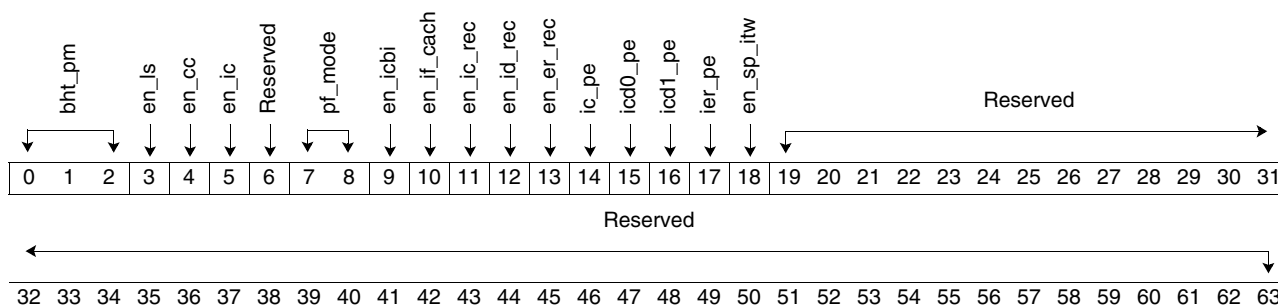
HID0 Bit Functions



Bits	Field Name	Description
0	one_ppc	One PowerPC Architecture instruction per dispatch group mode. An instruction might span more than one group.
1	do_single	Single group completion mode. Flush and refetch after the completion of each group or the completion of each microcoded instruction, if the instruction spans multiple groups.
2	isync_sc	Disable isync scoreboard optimization.
3	ser_gp	Serialize group dispatch. The next group is not dispatched until the previous group completes.
4:6	—	Reserved
7	deep_nap	Deep nap
8	doze	Doze
9	nap	Nap
10	—	Reserved
11	dpm	Enable dynamic power management.
12	—	Reserved
13	tg	Performance monitor threshold granularity control.
14	hang_dis	Disable processor hang-detection mechanism.
15	nhr	Not hard reset. Check after snoop response in (SRI) to see if hard or soft.
16	inorder	Serialized group issue mode. The next group is not issued until the previous group completes. Does not include branch or CR-logical instructions.

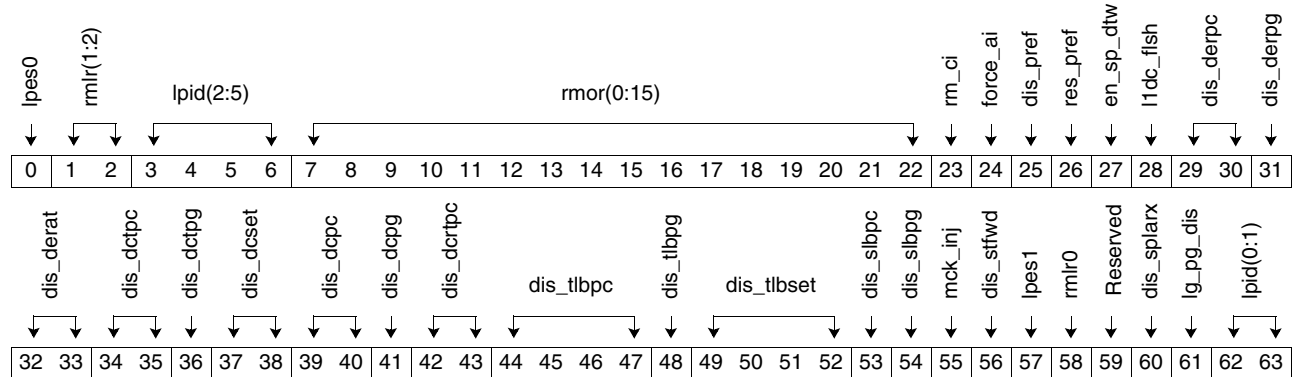
IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
17	—	Reserved
18	tb_ctrl	Enable time-base counting when the processor is stopped.
19	ext_tb_en	External time-base enable. 0 Use TBEN input as enable. TB is clocked at 1/8 of the full processor frequency. 1 Use TBEN input to clock time base (external clock).
20:21	—	Reserved
22	ciabr_en	Enable Completion Instruction Address Breakpoint Register (CIABR).
23	hdice_en	Enable hypervisor decremter interrupt conditionally (HDICE). The initial reset value must be '0' and disables hypervisor interrupts.
24:30	—	Reserved
31	en_attn	Enable support processor attention instruction.
32	en_mck	Enable external machine check interrupts (preferred state equals '1').
33:63	—	Reserved

IBM PowerPC 970MP RISC Microprocessor
HID1 Bit Functions


Bits	Field Name	Description
0:2	bht_pm	Branch history table (BHT) prediction mode. 000 Static prediction 001 Unused (same as 000) 010 Global BHT prediction only 011 Global prediction with history compression 100 Local BHT prediction only 101 Unused (same as 100) 110 Full global/local prediction with global selection (gsel) 111 Full global/local prediction with gsel and history compression (preferred state)
3	en_ls	Enable link stack (preferred state equals '1').
4	en_cc	Enable count cache (preferred state equals '1').
5	en_ic	Enable instruction cache (must be '1' for proper functioning).
6	—	Reserved
7:8	pf_mode	Prefetch mode. 00 No instruction prefetch. 01 Select next sequential address (NSA) instruction prefetch. 10 Select NSA and NSA + 1 instruction prefetch (preferred state). 11 Disable prefetch buffer.
9	en_icbi	Enable forced Instruction Cache Block Invalidate (icbi) match mode.
10	en_if_cach	Enable instruction fetch cacheability control. 0 All instruction fetch accesses are treated as cache inhibited regardless of the state of the I bit in the page table. 1 Instruction fetch cacheability is controlled by the state of the I bit in the page table (preferred state).
11	en_ic_rec	Enable I-cache parity error recovery (preferred state equals '1').
12	en_id_rec	Enable I-directory parity error recovery (preferred state equals '1').
13	en_er_rec	Enable instruction ERAT (I-ERAT) parity error recovery (preferred state equals '1').
14	ic_pe	Force instruction cache parity error (error inject).
15	icd0_pe	Force instruction cache directory 0 parity error (error inject).
16	icd1_pe	Force instruction cache directory 1 parity error (error inject).
17	ier_pe	Force I-ERAT parity error (error inject).
18	en_sp_itw	Enable speculative tablewalks. The ERAT is never loaded using a page table entry (PTE) if PTE[G] is set to '1' (preferred state equals '1').
19:63	—	Reserved

HID4 Bit Functions

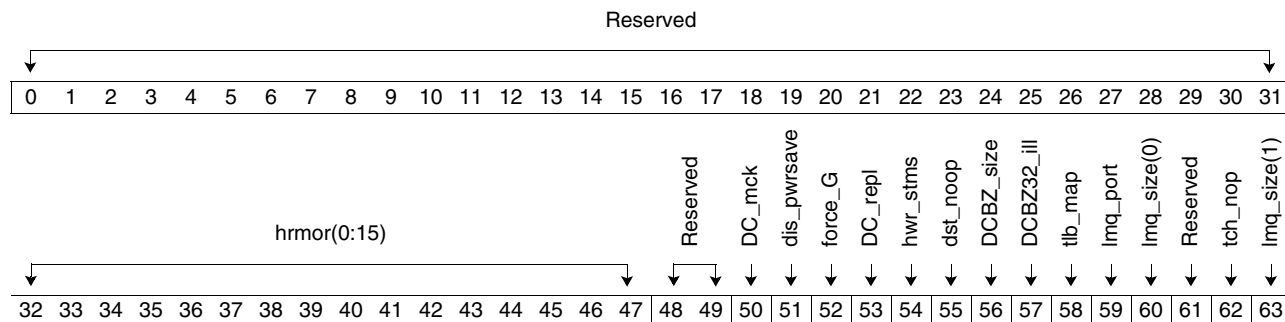


Bits	Field Name	Description
0	lpes0	LPAR environment selector bit [0]. LPES[0:1] are located in HID4[0, 57]. LPES[0:1] determine how MSR[HV] is set using interrupts and how memory access is performed when not in hypervisor mode. This is described in the PowerPC Architecture version 2.01.
1:2	rmlr(1:2)	LPAR real mode limit register (see HID4[58] for bit [0]).
3:6	lpid(2:5)	LPAR partition identity bits [2:5] (see also bits [62:63] for lpid(0:1)).
7:22	rmor(0:15)	LPAR real mode offset register [0:15].
23	rm_ci	Data accesses in real mode are treated as cache-inhibited.
24	force_ai	Force alignment interrupt instead of microcoding unaligned operations (that is instead of breaking unaligned operations into multiple smaller operations).
25	dis_pref	Disables data prefetching.
26	res_pref	Setting HID4[26] to '1' resets the data prefetch mechanism, suppressing subsequent prefetch requests and clearing the stream detection logic, therefore stream detection is not affected by accesses performed before setting the bit back to '0'.
27	en_sp_dtw	Enable speculative load tablewalk.
28	l1dc_flash	L1 data cache flash invalidate. 0 Normal operation 1 All sectors set to invalid and held invalid
29:30	dis_derpc	Disable data ERAT (D-ERAT) parity checking (one bit for each set).
31	dis_derpg	Disable D-ERAT parity generation (force parity to '0' on EA[0:45] only).
32:33	dis_derat	Disable one or more ways of the 4-way set associative D-ERAT (one bit per set); valid states are 00, 01, and 10.
34:35	dis_dctpc	Disable data cache tag parity checking (one bit for each set).
36	dis_dctpg	Disable data cache tag parity generation.
37:38	dis_dcset	Disable data cache set (one bit for each set).
39:40	dis_dcpc	Disable parity checking in one or more ways of the 4-way set-associative data cache (one bit per set).
41	dis_dcpq	Disable data cache parity generation.
42:43	dis_dcrtpc	Disable parity checking on the physical address tag of the data cache (one bit per set).
44:47	dis_tlbpc	Disable parity checking in one or more ways of the 4-way set-associative translation lookaside buffer (TLB) (one bit per set).
48	dis_tlbpg	Disable TLB parity generation.

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
49:52	dis_tlbset	Disable set in one or more ways of the 4-way set associative TLB (one bit per set); valid states are x'0', x'7', x'B', x'D', and x'E'.
53	dis_slbpc	Disable SLB parity checking.
54	dis_slbpg	Disable SLB parity generation.
55	mck_inj	Enable machine-check error injection.
56	dis_stfwd	Disable store forwarding (cause reject).
57	lpes1	LPAR environment selector bit [1]. LPES[0:1] are located in HID4[0, 57]. LPES[0:1] determine how MSR[HV] is set using interrupts and how memory access is performed when not in hypervisor mode. This is described in the PowerPC Architecture version 2.01.
58	rmlr0	HID4 bits [58, 1:2] are real mode limit register bits [0:2]. 011 64 MB 111 128 MB 100 256 MB x10 1 GB x01 16 GB 000 256 GB
59	—	Reserved
60	dis_splarx	Disable speculative Load Word and Reserve Indexed (lwarx) and Load Double Word and Reserve Indexed (ldarx) instructions.
61	lg_pg_dis	Disable large page support. The large page (L) bit input to SLB will be forced to zero (software will read a zero L bit).
62:63	lpid(0:1)	LPAR partition identity bits [0:1]. HID4[62:63, 3:6] are LPID[0:5] respectively.

HID5 Bit Functions



Bits	Field Name	Description
0:31	—	Reserved
32:47	hrmor(0:15)	LPAR hypervisor real mode offset register.
48:49	—	Reserved
50	DC_mck	Machine check enabled for data cache and data cache tag parity errors (software recovery enabled).
51	dis_pwrsave	L1 data cache (D-cache), L1 D-cache tag, D-ERAT power savings disable.
52	force_G	Force guarded (G equals '1') load.
53	DC_repl	Data cache replacement algorithm. 0 Least recently used (LRU) (default) 1 First-in-first-out (FIFO)
54	hwr_stms	Number of available hardware prefetch streams. 0 Four hardware streams and four VPU streams 1 Eight hardware streams (HID5[55] must also be '1')
55	dst_noop	Data Stream Touch (DST) instructions no-op. 0 DSTs are enabled. 1 DSTs are a no-ops and discarded in the load/store unit (LSU).
56	DCBZ_size	Makes dcbz a 32-byte store when bit 10 of the dcbz instruction is set to '0'.
57	DCBZ32_ill	Makes a dcbz instruction with bit 10 equal to '0' an illegal instruction.
58	tlb_map	TLB mapping. 0 4-way set associative 1 Direct mapped Note: When setting HID5[58] to make the TLB direct mapped, the TLB set disable bits, HID4[49:52], must be cleared; otherwise, translation will not work.
59	lmq_port	Demand miss (load miss queue [LMQ] to 970MP storage subsystem [STS]). 0 Permit two per cycle. 1 Permit only one per cycle (this setting is not currently supported).
60	lmq_size(0)	Number of outstanding requests to STS. Maximum outstanding requests HID5 [60, 63] 00 8 01 1 (this setting is not currently supported) 10 2 11 4
61	—	Reserved

IBM PowerPC 970MP RISC Microprocessor

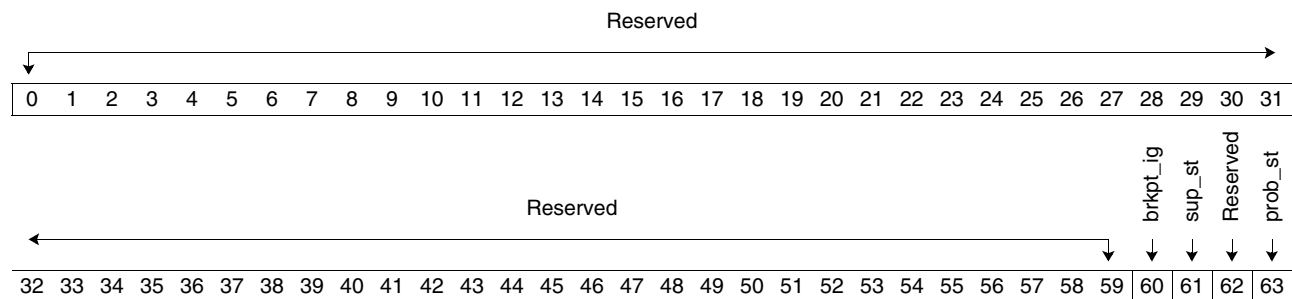
Bits	Field Name	Description
62	tch_nop	Make the Data Cache Block Touch (dcbt) and Data Cache Block Touch for Store (dcbtst) instructions act like no-ops.
63	lmq_size(1)	See description of HID5[60].

2.1.2.3 Data Address Breakpoint Register (DABR)

The data address breakpoint facility provides a means of detecting load-and-store accesses to a designated double word. The address comparison is done on an effective address. The data address breakpoint facility is controlled by the architected Data Address Breakpoint Register (DABR) and the 970MP microprocessor-specific Data Address Breakpoint Register Extension (DABRX).

Data Address Breakpoint Register Extension (DABRX)

The DABRX register is only active in hypervisor mode.



Bits	Name	Description
0:59	—	Reserved
60	BTI	Breakpoint translation ignore
61	HYP	Hypervisor state
62	PNH	Privileged but non-hypervisor state
63	PRO	Problem state (user mode)

Note: Bits [61:63] are termed the privileged mask (PRIVM).

Data Address Compare

The 970MP microprocessor supports the address compare control facility and the Address Compare Control Register (ACCR) as defined in the architecture. In addition, the 970MP microprocessor supports the optional data address breakpoint facility and associated Data Address Breakpoint Register (DABR) described in the architecture. In either case, upon taking a data storage interrupt, the 970MP processing unit sets the DAR correctly.

The architecture allows some flexibility on whether an ACCR match, a DABR match, or both actually occurs for certain conditions. More specifically, in the 970MP processing unit, store conditional instructions that are executed but not successful (that is, the store does not actually occur) will cause either an ACCR match or a DABR match if the appropriate match conditions are met. String instructions with zero length will not cause ACCR or DABR matches. The **dcbz** instruction will cause a DABR match if the appropriate match conditions are met.

As an alternative to causing an interrupt, a DABR match can be made to cause various forms of hard stops or soft stops for use as a debug aid (these controls are available through special SCOM commands). In general, this capability is not recommended for use in normal system operation because it might require the presence of an engineering support processor to restart the processing unit.

IBM PowerPC 970MP RISC Microprocessor

2.1.2.4 Instruction Address Breakpoint Register (IABR)

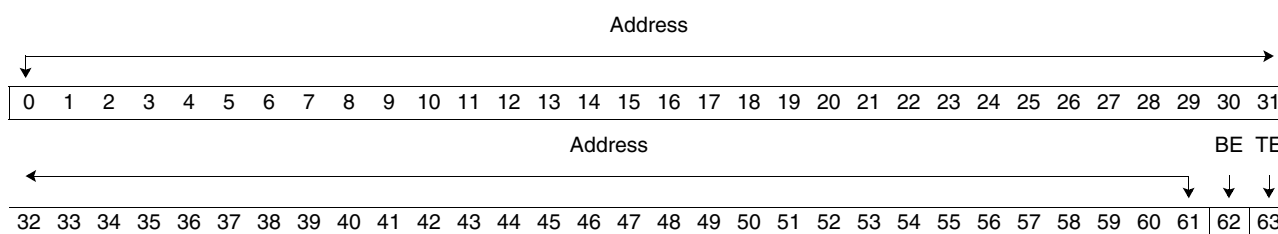
The Instruction Address Breakpoint Register can be used as a debug tool to trigger an event upon the fetch of a particular instruction address. The address in the IABR is compared to the Instruction Fetch Address Register, which will also contain addresses of speculative instruction fetches. The IABR is set up as described in the *PowerPC Microprocessor Family: The Programming Environments* manual, except, in the 970MP microprocessor, the IABR is only available as a trigger to the debug logic. This trigger can be programmed to perform functions such as quiesce or checkstop. If the word specified in the IABR is fetched, the instruction breakpoint handler is invoked. The instruction that triggers the breakpoint does not execute before the handler is invoked.

CIABR can be enabled by either HID0[22] (software accessible) or scan/SCOM override.

The IABR uses the IFU FETCH address, not the current instruction address (CIA) that is executing. An IABR match occurs on the fetch of any instruction, even a speculative instruction.

Note: There can be multiple IABR matches for a single instruction before it is actually executed (or completed).

During power-on reset, all bits are reset to '0'.



Bits	Field Name	Description
0:61	Address	Word address to be compared
62	BE	Breakpoint enabled. An address match causes a trigger to the debug logic.
63	TE	Translation enabled. An IABR match is signaled if this bit matches MSR[IR].

2.1.2.5 Instruction Match CAM Array Access Register (IMC)

The instruction match CAM (IMC) array facility is used for performance monitoring instrumentation. This latter use is restricted for the support processor and is not available through SPR access to this register array. The array has privileged write access and user-level read access through this SPR. Writes to the register array are used to configure the IMC, and reads return information about the availability of registers within the facility. See *Instruction Match CAM (IMC) Register* on page 323 for additional details on the IMC register.

2.1.2.6 Performance Monitor Registers (MMCR0, MMCR1, MMCRA, PMC1-8)

The Performance Monitor Counter Registers (PMC1-PMC8) and the Performance Monitor Control Registers (MMCR0, MMCR1, MMCRA) are supported in the 970MP microprocessor.

The Performance Monitor Control Registers, MMCR0, MMCR1, and MMCRA, are used with the MSR and other SPRs to set up the performance monitor enable states, interrupt conditions, threshold values, match criteria, and selection of the events counted in each of the Performance Monitor Counter Registers, PMC1-PMC8.

The MMCRx Register bit assignments are shown in *Section 10.4.1* on page 211; *Section 10.4.2* on page 214; and *Section 10.4.3* on page 217. All of the MMCRx and PMCx Registers flush to zero unless otherwise noted in the MMCRx and PMCx tables.

The MSR bits that relate to performance monitor functions are shown in *Section 4.3.3* on page 106. The value of the SRR1 Registers when a performance monitor interrupt is taken is shown in *Chapter 10 970MP Performance Monitor*.

Performance Monitor Counter Registers (PMC1-8)

CTR_NEG

CTRDATA

0

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

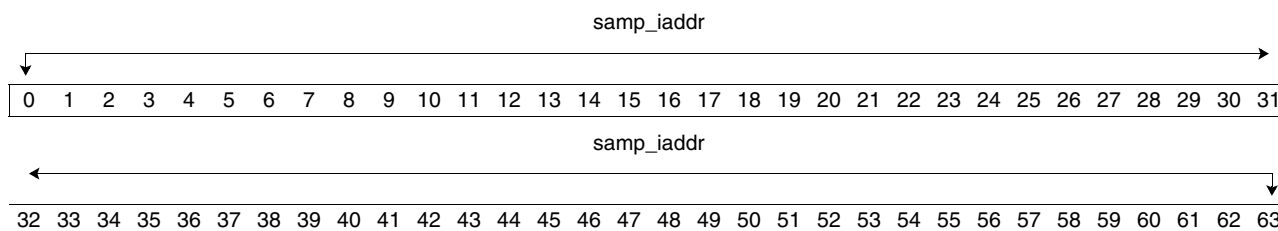
30

31

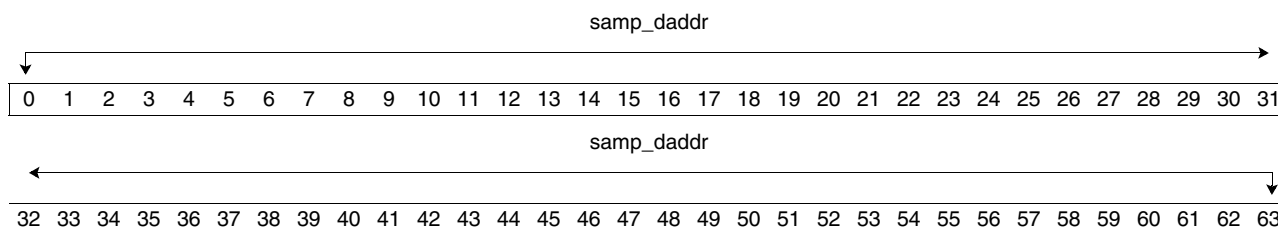
Bits	Field Name	Description
0	CTR_NEG	Counter negative bit
1:31	CTRDATA	Count data

IBM PowerPC 970MP RISC Microprocessor
2.1.2.7 Sampled Instruction Address and Sampled Data Address Registers (SIAR, SDAR)

The Sampled Instruction Address Register (SIAR) and the Sampled Data Address Register (SDAR) are supported in the 970MP microprocessor. The SIAR is used to save the effective address of a sampled instruction and the SDAR is used the effective address of a storage operand for a sampled instruction, when the processor is in either trace-marking mode or performance-marking mode. The terms 'sampled' and 'marked' are used interchangeably in this document.

Sampled Instruction Address Register (SIAR)


Bits	Field Name	Description
0:63	samp_iaddr	Sampled Instruction Address

Sampled Data Address Register (SDAR)


Bits	Field Name	Description
0:63	samp_daddr	Sampled Data Address

2.1.2.8 Scan Communication Registers (SCOMC and SCOMD)

Each 970MP processing unit includes a pair of registers to aid in communicating with the Scan Communications facility (SCOM). The SCOMC Register is a control register that includes a command field, a destination field, and a set of status bits. The SCOMD Register is an associated data register that acts as either a source of data or as a destination for data depending on the command placed into the SCOMC Register.

The SCOM facility contains an arbiter, which serializes use of the facility among the bus masters (processor cores and core service processor). However, there are very specific programming conventions associated with the use of this facility. See *Chapter 12 SCOM Interface and Registers on 295* for a detailed description of the SCOM facility.

2.1.2.9 Hypervisor Decrementer Interrupt Register (HDEC)

The Hypervisor Decrementer Interrupt Register (HDEC) is a 32-bit decrementing counter that provides a mechanism for causing a hypervisor decrementer interrupt after a programmable delay.

The HDEC is driven by the same frequency as the Time Base Register, and in the same manner as the Decrementer Register. The Hypervisor Decrementer Register counts down, causing an interrupt and is implemented in SPR 310.

2.1.2.10 Hypervisor Save/Restore Register (HSRR0, HSRR1)

The Hypervisor Machine Status Save/Restore Register 0 (HSRR0) is located in SPR 314 and HSRR1 is located in SPR 315. When a hypervisor decrementer interrupt occurs, the state of the machine is saved in the Hypervisor Machine Status Save/Restore Registers (HSRR0 and HSRR1). The effective address is stored in HSRR0 and the MSR in HSRR1. The contents of these registers is used to restore machine state when a **hrfid** instruction is executed.

2.1.2.11 Hypervisor SPRGs (HSPRG0, HSPRG1)

HSPRG0 and HSPRG1 are 64-bit registers provided for use by hypervisor programs. HSPRG0 is located at SPR 304 and HSPRG1 is located at SPR 305.

Note: Neither the contents of the HSPRGs, nor accessing them using `mtspr` or `mfspir`, has a side effect on the operation of the processor. One or more of the registers is likely to be needed by hypervisor interrupt handler programs (for example, as scratch registers, pointers, or both to processor save areas).

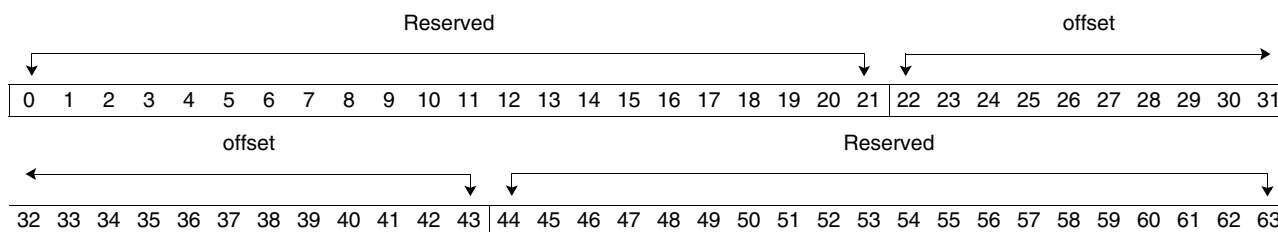
2.1.2.12 Trigger Registers (TRIG0, TRIG1, TRIG2)

Writes to the Trigger Registers, named TRIG0, TRIG1, and TRIG2, can be inserted in the instruction stream to cause triggers to the on-chip debug logic of the trace array. These are intended to be used for lab debug and bring-up only and architecturally behave as a no-op.

IBM PowerPC 970MP RISC Microprocessor
2.1.2.13 Hardware Interrupt Offset Register (HIOR)

The Hardware Interrupt Offset Register (HIOR) should be scanned (the HIOR is on the mode ring) to the system's starting address during initialization. Subsequently, the HIOR should be set to zero.

The physical address of the interrupt vector is found using HIOR[22:43] combined with the 20-bit vector offset for the particular exception.



Bits	Field Name	Description
0:21	—	Reserved
22:43	offset	Offset
44:63	—	Reserved

2.2 Instruction Set Summary

This section describes instructions and addressing modes defined for the 970MP microprocessor. These instructions are divided into the following execution unit categories:

- Fixed-Point Processor
- Floating-Point Processor
- Vector Processor
- Load-and-Store Processor
- Branch and Flow Control
- Storage Control
- Memory Synchronization

Fixed-point instructions operate on byte, half-word, word, and double-word operands. Floating-point instructions operate on single-precision and double-precision floating-point operands. The PowerPC Architecture uses instructions that are 4 bytes long and word-aligned. It provides for byte, half-word, word, and double-word operand loads and stores between memory and a set of 32 General-Purpose Registers (GPRs). It provides for word and double-word operand loads and stores between memory and a set of 32 Floating-Point Registers (FPRs). The VPU extension to the PowerPC Architecture provides for quadword operand loads and stores between memory and a set of 32 Vector Registers.

Arithmetic and logical instructions do not read or modify memory. To use the contents of a memory location in a computation and then modify the same or another memory location, the memory contents must be loaded into a register, modified, and then written to the target location using load-and-store instructions.

2.2.1 Classes of Instructions

The 970MP microprocessor instructions belong to one of the following three classes:

- Defined
- Illegal
- Reserved

Note: While the definitions of these terms are consistent among the PowerPC processors, the assignment of these classifications is not.

The class is determined by examining the primary opcode and the extended opcode, if any. If the opcode or the combination of opcode and extended opcode, is not that of a defined instruction or of a reserved instruction, then the instruction is illegal. Instruction encodings that are now illegal might become assigned to instructions in the architecture or might be reserved by being assigned to processor-specific instructions.

IBM PowerPC 970MP RISC Microprocessor

2.2.1.1 Definition of Boundedly Undefined

If instructions are encoded with incorrectly set bits in the reserved fields, the results of execution can be said to be boundedly undefined. If a user-level program executes the incorrectly coded instruction, the resulting undefined results are bounded in that a spurious change from user to supervisor state is not allowed, and the level of privilege exercised by the program in relation to memory access and other system resources cannot be exceeded. Boundedly-undefined results for a given instruction can vary between implementations, and between execution attempts in the same implementation.

2.2.1.2 Defined Instructions

The 970MP microprocessor provides support for the following optional instructions:

fsqrt	Floating-Point Square Root
fsqrts	Floating-Point Square Root Single
fres	Floating-Point Reciprocal Estimate Single
frsqrte	Floating-Point Reciprocal Square Root Estimate A-Form
fsel	Floating-Point Select
mfsr	Move from Segment Register
mfsrin	Move from Segment Register Indirect
mtmsr	Move to Machine State Register (32-bit)
mtsr	Move to Segment Register
mtsrin	Move to Segment Register Indirect
slbie	SLB Invalidate Entry
slbia	SLB Invalidate All
tlbie	TLB Invalidate Entry
tlbsync	TLB Synchronize

The 970MP microprocessor does *not* provide support for the following optional or obsolete instructions (or instruction forms). Attempted use of these will result in an illegal instruction type of program interrupt.

bccbr	Branch Conditional to CBR (obsolete)
dcba	Data Cache Block Allocate (obsolete)
dcbi	Data Cache Block Invalidate (obsolete)
eciwx	External Control In Word Indexed
ecowx	External Control Out Word Indexed
mcrxr	Move to Condition Register from XER Register (obsolete)
mtsrdr	Move to Segment Register Double Word (obsolete)
mtsrdrin	Move to Segment Register Double Word Indirect (obsolete)
rfi	Return from Interrupt (obsolete)
tlbia	TLB Invalidate All
tlbiex	TLB Invalidate Entry by Index (obsolete)
slbiex	SLB Invalidate Entry by Index (obsolete)

2.2.1.3 Illegal Instructions

Illegal instructions can be grouped into the following categories:

- Instructions not defined in the PowerPC Architecture. The following primary opcodes are defined as illegal, but might be used in future extensions to the architecture: 1, 5, 6, 56, 57, 60, 61.

Note: Primary opcode 4 is used in the 970MP microprocessor to implement the vector extensions that are described in *Chapter 10 970MP Performance Monitor*.

- Instructions defined in the PowerPC Architecture but not implemented in a specific PowerPC implementation. For example, the following primary opcodes that are legal on 64-bit PowerPC processors are considered illegal by 32-bit processors: 30, 62.

Note: On the 970MP microprocessor, these instructions are executed in 32-bit mode.

- All unused extended opcodes for instructions. Notice that extended opcodes for instructions defined only for 64-bit implementations are illegal in 32-bit implementations. The following primary opcodes have unused extended opcodes: 19, 30, 31, 56, 57, 59, 60, 61, 62, 63. (Primary opcodes 30 and 62 are illegal for 32-bit implementations, but as 64-bit opcodes they have some unused extended opcodes.)
- An instruction consisting entirely of zeros is guaranteed to be an illegal instruction. This increases the probability that an attempt to execute data or uninitialized memory invokes the system illegal instruction error handler (a program exception).

See *Section 4.5.9 Program Exception* on page 114 for additional information about illegal and invalid instruction exceptions. Except for an instruction consisting of binary zeros, illegal instructions are available for additions to the PowerPC Architecture.

2.2.1.4 Reserved Instructions

The PowerPC Architecture breaks the reserved instruction class down into several categories. The 970MP microprocessor behaves as described below for each category of reserved instructions:

- Primary opcode equals zero. The 970MP processing unit will take an illegal instruction type of program interrupt for all cases except the Support Processor Attention (**attn**) instruction when HID0[31] is set to '1'.
- Power Architecture® instructions not in the PowerPC Architecture. The 970MP processing unit will take an illegal instruction type of program interrupt.
- Implementation-specific instructions used to conform to the architecture. No action taken.
- Other instructions. The 970MP processing unit supports the implementation-specific instruction, **tlbiel** (the processor local form of the TLB Invalidate entry used for managing TLB parity errors).

In addition, several implementation-specific registers are available for access through the **mtspr** and **mfspr** instructions. These are described in *Section 2.1.2.1 Move To and Move From System Register Instructions* on page 50.

2.2.2 Instruction Set Overview

The following sections provide a brief overview of the PowerPC instructions implemented in the 970MP microprocessor and highlight how a 970MP microprocessor implements a particular instruction. Note that the categories used in this section correspond to those used in *“Addressing Modes and Instruction Set Summary”*

IBM PowerPC 970MP RISC Microprocessor

in the *PowerPC Microprocessor Family: The Programming Environments* manual. These categorizations are somewhat arbitrary and are provided for the convenience of the programmer. They do not necessarily reflect the PowerPC Architecture specification.

Note: Some instructions have the following optional features:

- CR Update. The dot (.) suffix on the mnemonic enables the update of the CR.
- Overflow option. The **o** suffix indicates that the overflow bit in the XER is enabled.

2.2.3 Fixed-Point Processor

2.2.3.1 Fixed-Point Arithmetic and Compare Instructions

The architecture states that instructions that have the overflow exception (OE) bit set, or instructions that can set the carry (CA) bit, might execute more slowly than instructions that do not. In the 970MP microprocessor, the summary overflow (SO) bit in the XER is not renamed. For instructions with the OE set, it is initially assumed that no overflow will occur and that the SO bit does not need to be changed. If the instruction does cause an overflow and the SO bit was not set before the instruction executed (and therefore needs to be set), the machine will flush this instruction and those beyond this instruction, set the non-renamed SO bit, and then refetch and re-execute the instructions that follow. In general, if no overflow occurs or the SO bit has already been set, this strategy will not have an adverse effect on performance.

Alternatively most instructions that set and use the CA bit do not have any particular performance considerations. This field of the XER is renamed, and many of the common dependence hazards are minimized.

2.2.3.2 Fixed-Point Logical, Rotate, and Shift Instructions

The architecture defines the preferred no-op to be OR Immediate (**ori**) 0,0,0. In the 970MP microprocessor, this no-op form is recognized by the hardware and allowed to complete without taking any execution resources. This makes the instruction valuable for padding other instructions to achieve better alignment or better separation.

2.2.3.3 Move to and Move from System Register Instructions

The **mtspr** instruction provide access to system registers using a GPR as the source register. The **mfspr** instruction provides access to the system registers using a GPR as the destination register. *Table 2-3 Implementation-Specific SPRs* on page 51 lists the SPR numbers for both user-level and supervisor-level access to 970MP-specific registers.

2.2.3.4 Move to and Move from Machine State Register

The 970MP microprocessor supports both the 32-bit **mtmsr** instruction and the 64-bit **mtmsrd** instruction. The 970MP microprocessor works to optimize the **mtmsr** instruction to help speed up cases where little or no synchronization is required (that is, updates to the MSR[EE] bit).

2.2.3.5 Fixed-Point Invalid Forms and Undefined Conditions

The results of executing an invalid form of a fixed-point instruction or an instance of a fixed-point instruction for which the architecture specifies that some results are undefined are described below for the cases in which executing an instruction does not cause an exception. Only results that differ from those specified by the architecture are described in the following list.

- **Instruction with Reserved Fields**
Bits in reserved fields are ignored; the results of executing an instruction in which one or more reserved bits are '1' is the same as if the bits were '0'.
- **Divide Word (divw), Divide Word Overflow (divwo), Divide Word Unsigned (divwu), and Divide Word Unsigned Overflow (divwuo) Instructions**
Bits 0:31 of the rD^1 are set to x'0000 0000'.
- **Multiply High Word (mulhw) and Multiply High Word Unsigned (mulhwu) Instructions**
 rD bits 0:31 contain the same result as $rD[32:63]$.
- **Divide Instructions (divide by zero)**
If the divisor is zero, rD is set to zero. If, in addition, the record bit (RC) in the vector instruction field equals '1', CR0 is set to '0010'.
- **Move To and Move From Special Purpose Register Instructions**
Table 2-4 on page 53 describes the results of specifying an SPR value that is not defined for the implementation.
- **Move From Time-Base Instruction**
The **mftb** instruction is treated as an alias for the **mfspr** instruction; the results are the same as for executing an **mfspr** instruction.
- **Move From Condition Register Instruction** (bit 11 is set to '1')
One CR field is copied into the associated bits of the rD , and the remaining bits of the rD are set to zeros.
- **Move From Condition Register Instruction** (bit 11 is set to '1', and multiple bits of the FXM² field are set to '1')
The source is CR(n), where n is the CR field specified by the bit in the FXM that is set and has the smallest index value. If no bit in FXM is set to '1', the results will be the same as if the FXM was set to '00000001'.
- **Move To Condition Register Fields Instruction** (bit 11 is set to '1', and multiple bits of the FXM field are set to '1')
CR(n) is updated where n is the CR field specified by the bit in FXM that is set and has the smallest index value. If no bit in the FXM is set to '1', executing the instruction does not modify the CR.

1. Field used to specify a General Purpose Register (GPR) to be used as a target.
2. Field mask used to identify the CR fields to be updated by the **mtcrf** instructions.

IBM PowerPC 970MP RISC Microprocessor

2.2.4 Floating-Point Processor

Each 970MP processing unit contains two double-precision floating-point units. Each of these units is optimized for fully pipelined double-precision multiply-add functionality. In addition, each unit is capable of performing floating-point divide and square root instructions. For more information about the performance of floating-point operations.

The optional floating-point instructions (**fsqrt**, **fsqrts**, **fres**, **frsqrite**, and **fsel**) defined in the *PowerPC Microprocessor Family: The Programming Environments* manual are implemented.

Note: The 970MP microprocessor does *not* support the non-IEEE mode that was defined in earlier versions of the architecture.

2.2.4.1 Floating-Point Arithmetic Instructions

The architecture requires operands for single-precision floating-point arithmetic instructions to be representable in single-precision format. If they are not, then the results of the single-precision arithmetic instructions are undefined. For the single-precision divide and square-root instructions, **fdivs** and **fsqrts**, single-precision algorithms are executed on the double-precision data with the final results rounded to single-precision.

2.2.4.2 Floating-Point Invalid Forms and Undefined Conditions

The results of executing an invalid form of a floating-point instruction or an instance of a floating-point instruction for which the architecture specifies that some results are undefined are described below for the cases in which executing an instruction does not cause an exception. Only results that differ from those specified by the architecture are described.

- **Instruction with Reserved Fields**
Bits in reserved fields are ignored; the results of executing an instruction in which one or more reserved bits are '1' is the same as if the bits were '0'.
- **Floating-Point Convert to Integer Word Instructions: fctiw or fctiwz**
The Instruction target register (**frD**[0:31]) is set to x'FFF8 0000'.
- **Floating-Point Convert to Fixed-Point Instructions (fctiw, fctiwz, fctid, and fctidz)**
The contents of **FPSCR**(**FPRF**) are set to '00000'.
- **Move from FPSCR Instruction**
frD[0:31] is set to x'FFF8 0000'.

2.2.5 Vector Processor

Each 970MP processing unit contains two vector units: the vector arithmetic logical unit (VALU) and the vector permute unit (VPERM). The vector instructions and their implementation are described in *Chapter 13 Vector Processing Unit*.

2.2.6 Load Store Processor

2.2.6.1 Floating-Point Load-and-Store Instructions

Most forms of unaligned floating-point storage accesses are executed entirely in hardware (see *Section 3.3.2.1 Storage Access Alignment Support* on page 85).

2.2.6.2 Fixed-Point Load Instructions

Most forms of unaligned load operations are executed entirely in hardware. If a basic load operation crosses a page boundary, and either page translation signals an exception condition, then when the interrupt occurs it will appear as though none of the load instructions have executed. This is not always the case for load multiple or load string instructions. For more information, see *Section 2.2.6.4 Fixed-Point Load-and-Store Multiple Instructions* and *Section 2.2.6.5 Fixed-Point Load-and-Store String Instructions* on page 74.

The Load Algebraic, Load with Byte Reversal, and Load with Update instructions might have greater latency than other load instructions. These instructions are implemented as a sequence of internal operations. Due to the dynamic scheduling and out-of-order execution capability of the processor, these effects are somewhat minimized. It should also be noted that, although these instructions are broken up in this manner, the effects are never visible from a programming model perspective. For more information about the performance of these instructions.

Any load from storage marked cache-inhibited that is not aligned will cause an alignment interrupt.

2.2.6.3 Fixed-Point Store Instructions

Most forms of unaligned store operations are executed entirely in hardware. If a store operation crosses a page boundary, and the second page translation signals an exception condition, then after the interrupt is taken it will appear as though none of the storage updates have occurred to either page. (This is not always the case for store multiple or store string instructions. See *Section 2.2.6.4 Fixed-Point Load-and-Store Multiple Instructions* and *Section 2.2.6.5 Fixed-Point Load-and-Store String Instructions* on page 74 for more information.)

Any store to storage marked cache-inhibited that is not aligned will cause an alignment interrupt.

2.2.6.4 Fixed-Point Load-and-Store Multiple Instructions

The Load Multiple Word (**lmw**) instruction is executed so that up to two registers are loaded each cycle. Similarly, the Store Multiple Word (**stmw**) instruction is executed so that up to two registers are stored each cycle. The 32-entry store queue can accept up to two 8-byte stores per cycle; the cache can accept one 8-byte store per cycle. Because these instructions are emulated through the use of microcoded templates, after a small start-up penalty, they are processed at a rate of up to two registers per cycle.

Most forms of **lmw** and **stmw** instructions, even those that cross page and segment boundaries, are executed entirely in hardware. These instructions and the individual storage accesses associated with the instructions are not atomic. If an **stmw** crosses a page boundary, and the second page translation signals an exception condition, then after the interrupt is taken it will appear as though none, some, or all of the accesses to the first page have occurred. It will also appear as though none of the accesses to the second page have occurred. However, for an **lmw** instruction that crosses a page boundary where the second page translation signals an exception condition, all of the target registers will have an undefined value.

IBM PowerPC 970MP RISC Microprocessor

An attempt to execute a non-word aligned **lmw** or **stmw** will cause an alignment interrupt. An attempt to execute an **lmw** or **stmw** to storage marked cache-inhibited will also cause an alignment interrupt.

The architecture allows these instructions to be interrupted by certain types of asynchronous interrupts (external interrupts, decremter interrupts, machine check interrupts, and system reset interrupts). In these cases, for the load multiple instructions, all of the registers that were to be updated will have an undefined value. The instruction must be completely restarted to achieve the full effect (that is, no partial restart capability is supported). For the store multiple instructions, some of the storage locations referred to by the instruction might have been updated. However, to guarantee full completion of the store multiple instructions, they must also be completely restarted.

2.2.6.5 Fixed-Point Load-and-Store String Instructions

The Load String Word (**lsw**) instruction is executed so that up to two registers are loaded each cycle. Similarly, the Store String Word (**stsw**) instruction is executed so that up to two registers are stored each cycle. The 32-entry store queue can accept up to two 8-byte stores per cycle; the cache itself can only accept one 8-byte store per cycle.

Because the immediate forms of these instructions are implemented using microcoded templates they incur a small start-up penalty. The X-form of the instructions contains a dependency on bits in the fixed-point XER Register. Therefore, depending on when the last update to these bits occurred, the instruction might be subject to a more expensive runtime flush and emulate sequence. For more information about the performance of these instructions.

Most Load String and Store String instructions that cross page or segment boundaries are executed entirely in hardware. If a Store String crosses a page boundary, and the second page translation signals an exception condition, then after the interrupt is taken it will appear as though none, some, or all of the accesses to the first page have completed. It will also appear as though none of the accesses to the second page have occurred. However, for Load String instructions that cross a page boundary where the second page translation signals an exception condition, all of the target registers will have an undefined value.

If the storage operand of a Load String Word Immediate (**lswi**) instruction is word aligned, then the accesses are performed in an optimal manner. If the operands are so aligned, the accesses are performed in an optimal manner if the operand resides entirely within a 64-byte block that is resident in the L1 D-cache or resides entirely within a 32-byte block. Although other unaligned string operations are supported in hardware, they might cause machine flushes and require long sequences of microcode. As a result, these types of unaligned string instructions can have significantly longer latencies.

An attempt to execute an **lswi**, Load String Word Indexed (**lswx**), Store Sting Word Immediate (**stswi**), or Store String Word Indexed (**stswx**) instruction to storage marked cache-inhibited will cause an alignment interrupt.

The architecture allows these instructions to be interrupted by certain types of asynchronous interrupts (external interrupts, decremter interrupts, machine check interrupts, and system reset interrupts). In these cases, for the Load String instructions, all of the registers that were to be updated will have an undefined value. The instruction must be completely restarted to achieve the full effect (that is, no partial restart capability is supported). For the store string instructions, some of the storage locations referred to by the instruction might have been updated.

The architecture describes some preferred forms for the use of load-and-store string instructions. In the 970MP microprocessor, these preferred forms have no effect on the performance of the instructions.

2.2.6.6 Load/Store Invalid Forms and Undefined Conditions

The results of executing an invalid form of a load/store instruction for which the architecture specifies that some results are undefined are described below for the cases in which executing an instruction does not cause an exception. Only results that differ from those specified by the architecture are described.

- **Load with Update Instructions** (rA^1 is set to '0')
The storage operand addressed by the EA is placed into rD . The sum of that storage operand and rB is placed in rA .
- **Load with Update Instructions** (rA equals rD)
The EA is placed into the rD . The storage operand addressed by the EA is accessed, but the data returned by the load is discarded.
- **Load Multiple Instructions** (rA is in the range of registers to be loaded)
If an exception (for example, a data storage or external exception) causes the execution of the instruction to be interrupted, the instruction is restarted, the rA has been altered by the previous partial execution of the instruction, and the rA does not equal '0', the new contents of the rA are used to compute the EA.
- **Load Multiple Instructions** (causing a misaligned access)
For a Load Multiple Word instruction, if the storage operand specified by the EA is not a multiple of four, an alignment exception is taken. For a Load Multiple Double Word instruction, if the storage operand specified by the EA is not a multiple of eight, an alignment exception is taken.
- **Load String Instructions** (zero length string)
The rD is not altered.
- **Load String Instructions** (rA , or rB^2 , or both are in the range of registers to be loaded)
If rA , or rB , or both are in the range of registers to be loaded, the results are as follows:

Indexed Form: If rA is set to '0', let R_x be rB ; otherwise let R_x be the register specified by the smaller of the two values in instruction fields rA and rB . If the rD equals R_x , no registers are loaded. Otherwise, registers rD through R_x-1 are loaded as specified in the architecture (that is, only part of the storage operand is loaded).

Immediate Form: If rA is set to '0', the instruction is executed as if it were a valid form. If rA equals rD , no registers are loaded; otherwise, registers rD through $rA-1$ are loaded as if the instruction was a valid form but specifying a shorter operand length.

- **Store with Update Instructions** (rA is set to '0')
EA is placed into R_0 .
- **Load or Store Floating-Point with Update Instructions** (rA is set to '0')
EA is placed into R_0 .
- **Floating-Point Store Single Instructions** (exponent less than 874 and FRS[09:31] not equal to '0')
The value placed in storage is a '0' with the same sign as the value in the register.

1. Field used to specify a GPR to be used as a source or as a target.
2. Field used to specify a GPR to be used as a source.

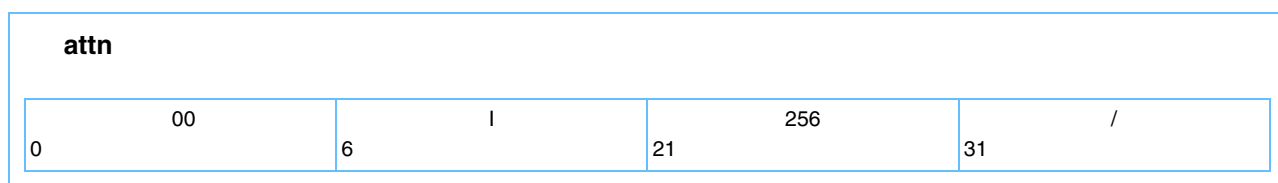
2.2.7 Branch Processor

2.2.7.1 Branch Processor Instructions

Support Processor Attention Instruction

The 970MP microprocessor supports a special, implementation-dependent instruction for signalling an attention to the support processor.

Figure 2-2. Processor Attention Instruction



The immediate field (I) has no effect on the operation of this instruction in the 970MP microprocessor. If the support processor attention enable bit is set (HID0[31] = '1'), this instruction will cause all preceding instructions to run to completion, the machine to quiesce, and the assertion of the support processor attention signal. If the support processor attention enable is not set (HID0[31] = '0'), this instruction will cause an illegal instruction type of program interrupt.

2.2.7.2 Branch Processor Instructions with Undefined Results

The results of executing an invalid form of a branch instruction or an instance of a branch instruction for which the architecture specifies that some results are undefined are described below. Only results that differ from those specified by the architecture are described.

- **Instructions with Reserved Fields**

Bits in reserved fields are ignored. The results of executing an instruction in which one or more of these bits is '1' is the same as if the bits were '0'.

- **bcctr and bcctrl Instructions**

If branch-options (BO)[2] is set to '0', the contents of the CTR before any update are used as the target address and to test the contents of the CTR to resolve the branch. The contents of the CTR are then decremented and written back to the CTR.

- **System Call (sc) Instructions** (opcode 17)

Bits 30:31 Description

'00' **sc** instruction

'01' illegal instruction exception

'10' **sc** instruction

'11' **sc** instruction

2.2.7.3 Move To Condition Register Fields Instruction

The architecture warns that updating a subset of the CR fields on a Move to Condition Register Fields (**mtcrf**) instruction can have worse performance than updating all of the fields. In the 970MP microprocessor, both the **mtcrf** instruction and the Move from Condition Register (**mfcrr**) instruction are emulated through the microcode templates. For best performance, software should use the new, single-field variants of these instructions as described in the architecture. For more information about the performance of these instructions.

The 970MP microprocessor supports the optional architecture extension that defines slight variations to the **mtcrf** and **mfcrr** instructions to indicate that the movement of a single field of the Condition Register is preferred. Because the performance of these instructions is better than their multiple field counterparts, use of these instructions is encouraged.

2.2.8 Storage Control Instructions

2.2.8.1 Key Aspects of Storage Control Instructions

In each 970MP processing unit, all cache control instructions operate on aligned 128-byte sections of storage. *Table 2-5* summarizes many of the key aspects of the storage control instructions.

Table 2-5. Storage Control Instructions

Aspect	Cache Instructions					
	icbi	dcbt	dcbtst	dcbz	dcbst	dcbf
Granularity	128 bytes	128 bytes	128 bytes	32 or 128 bytes	128 bytes	128 bytes
Semantic checking	Load (DSI on storage exception)	Load (no-op on storage exception)	Load (no-op on storage exception)	Store (DSI on storage exception)	Load (DSI on storage exception)	Load (DSI on storage exception)
"i" bit update	Yes	Yes	Yes	Yes	Yes	Yes
"c" bit update	No	No	No	Yes	No	No
L1 I-cache effect	L1 I-cache and prefetch buffer	None	None	None	None	None
L1 D-cache effect	None	See Section 2.2.8.4	See Section 2.2.8.4	Invalidate	No-op	Invalidate
L2 Cache effect	None	See Section 2.2.8.4	See Section 2.2.8.4	See Section 2.2.8.5	See Section 2.2.8.6	See Section 2.2.8.7
TLB effect	Reload as required	Reload as required	Reload as required	Reload as required	Reload as required	Reload as required
SLB effect	Reload as required	None (no-op if miss)	None (no-op if miss)	Reload as required	Reload as required	Reload as required

2.2.8.2 Instruction Cache Block Invalidate (icbi)

The instruction cache block size for **icbi** on the 970MP processing unit is 128 bytes.

Execution of this instruction occurs in multiple phases. First, the effective address is computed and translated by the load/store execution pipeline. Next, the resulting real address is passed to the 970MP STS logic which broadcasts it onto the system bus. When the 970MP STS snoops this type of command on the system bus, it presents the command to the upstream instruction caches. As these invalidates are presented to the instruction cache, the associated real addresses are checked against all 16 possible locations in the effective-addressed I-cache that could contain the particular real address. Only entries that actually match the real address will be invalidated. In addition, all entries in the instruction prefetch queue will be invalidated (independent of the address). As an aid for quickly flushing the entire contents of the I-cache, a special mode bit is provided (HID1[9]) that forces each of these 16 entries to be invalidated on an **icbi** (even if their address does not match the invalidate address). For more information about this instruction.

The **icbi** instruction has no effect on the L2 cache.

To ensure that the storage access caused by an **icbi** instruction has been performed with respect to the processor executing the **icbi** instruction, an **isync** instruction must be executed on that processor.

2.2.8.3 Instruction Cache Synchronize (isync)

As a performance optimization, the 970MP microprocessor internally tracks and updates a scoreboard bit for instructions that change instruction-cache-oriented context that are required to be synchronized by the **isync** instruction. When the **isync** instruction is executed, this scoreboard bit is checked to see whether the machine must flush and refetch the instructions following the **isync**. In addition, the **isync** instruction is often used as a load barrier to prevent any subsequent load (or store) instructions from executing before previous load instructions have been completed. In these cases, the scoreboard bit will typically not be set, and **isync** can complete without causing a flush.

2.2.8.4 Data Cache Block Touch (dcbt and dcbtst)

The data cache block size for **dcbt** and **dcbtst** on the 970MP processing unit is 128 bytes.

These instructions act as a touch for the D-cache hierarchy and the TLB. If data translation is enabled (MSR[DR] is set to '1'), and an SLB miss results, then the instruction will be treated as a no-op. If a TLB miss results, then the instruction will reload the TLB (and set the reference bit). Once past translation, if the page protection attributes prohibit access, or the page is marked cache-inhibited, or the page is marked guarded, or the processors' D-cache is disabled (using the bits in the HID4 Register), then the instruction will be finished as a no-op and will not reload the cache. Otherwise, the instruction will check the state of the L1 D-cache, and, if the block is not present, it will then initiate a reload. Note that this might also reload the L2 cache with the addressed block if it is not already present. If the cache block is already present in the L1 D-cache, the cache content is not altered. Note that if the **dcbt** or **dcbtst** instruction does reload cache blocks, it will affect the state of the cache replacement algorithm bits.

The 970MP microprocessor does implement the optional extension to the **dcbt** instruction that allows software to directly engage a data stream prefetch from a particular address. For more information about data stream prefetch, see *Section 3.5.3 Data Prefetch* on page 96.

2.2.8.5 Data Cache Block Zero (dcbz)

The data cache block size for **dcbz** on the 970MP processing unit is 128 bytes. Support is also provided for a **dcbz** of 32 bytes in order to accommodate coding that assumes a 32-byte block size. The **dcbz** actions are listed in *Table 2-6*.

Note: The entire instruction cache must be flushed whenever HID5[56] or HID5[57] are changed.

Table 2-6. dcbz Actions

HID5[57]	HID5[56]	dcbz Instruction Bit 10	Action
1	X	0	Illegal instruction
X	X	1	Cache block (128 bytes) zeroed
0	0	0	Cache block (128 bytes) zeroed
0	1	0	32-byte block zeroed

The function of **dcbz** is performed in the L2 cache. As a result, if the block addressed by the **dcbz** is present in the L1 D-cache, then the block will be invalidated before the operation is sent to the L2 cache logic for execution. The L2 cache will gain exclusive access to the block (without actually reading the old data), and will perform the zeroing function. For the 32-byte **dcbz**, the L2 cache might be required to read the line and then zero the 32 bytes.

If the cache block specified by the **dcbz** instruction contains an error, even one that is not correctable with error checking and correction (ECC), the contents of all locations within the block are set to zeros in the L2 cache. If the specified block in the L2 cache does not contain a hard fault, a subsequent load from any location within the cache block will return zeros and not cause a machine check interrupt.

If the block addressed by the **dcbz** instruction is in a memory region marked cache-inhibited, or if the L1 D-cache or L2 cache is disabled (using the bits in HID registers), then the instruction will cause an alignment interrupt to occur.

Implementation Note: In order to emulate the behavior of the obsolete **dcba** instruction, a mode bit is provided that changes the behavior of **dcbz** as follows. When the mode bit is set to '1', if the block addressed by the **dcbz** instruction is in a memory region marked cache-inhibited, the instruction is treated as a no-op. The **dcba** instruction was defined such that the referenced and changed bits need not be updated in this case. However, the 970MP microprocessor will update these bits.

2.2.8.6 Data Cache Block Store (dcbst)

The data cache block size for **dcbst** on the 970MP processing unit is 128 bytes.

The **dcbst** instruction forces all preceding stores to the referenced block to become committed to the cache hierarchy, and then forces a clean operation in the L2 cache.

The **dcbst** instruction has no direct effect on the L1 D-cache (because it is store-through, it never contains modified data). The L2 cache updates and processor interconnect bus operations are performed as shown in *Table 3-5 970MP L2 Cache State Transitions Due to Processor Instructions* on page 93 and *Table 3-6 970MP L2 Cache State Transitions Due to Bus Operations* on page 94.

2.2.8.7 Data Cache Block Flush (dcbf)

The data cache block size for **dcbf** on the 970MP processing unit is 128 bytes.

The **dcbf** instruction forces all preceding stores to the referenced block to become committed to the cache hierarchy. It then acts like an invalidate to the L1 D-cache (because it is store-through, it never contains modified data). The L2 cache updates and processor interconnect bus operations are performed as shown in *Table 3-5 970MP L2 Cache State Transitions Due to Processor Instructions* on page 93 and *Table 3-6 970MP L2 Cache State Transitions Due to Bus Operations* on page 94.

2.2.8.8 Load and Reserve and Store Conditional Instructions (*lwarx/ldarx, stwcx/stdcx*)

The reservation granularity for the 970MP processing unit is 128 bytes. The **lwarx** and **ldarx** instructions are sometimes executed speculatively.

An attempt to execute a non-word aligned **lwarx** or **stwcx**, or a non-double-word aligned **ldarx** or **stdcx** will cause an alignment interrupt. An attempt to execute an **lwarx**, **ldarx**, **stwcx**, or **stdcx** instruction to storage marked cache-inhibited will cause a data storage interrupt.

2.2.9 Memory Synchronization Instructions

The 970MP design achieves high performance by exploiting speculative out-of-order instruction execution. The **sync** instruction, as defined in the architecture, acts as a serious barrier to this type of aggressive execution and therefore can have a dramatic effect on performance. Although the 970MP microprocessor has optimized the performance of **sync** to some degree, care should be exercised in the use of this instruction. As a performance consideration, software should attempt to use the lightweight version of **sync** (**lwsync**) whenever possible.

The 970MP microprocessor also supports the architected Page Table Entry Synchronization (**ptesync**) instruction for use in synchronizing page table updates. The 970MP microprocessor implements the Enforce In-Order Execution of I/O (**eieio**) instruction as described in the *PowerPC Virtual Environment Architecture (Book II)*.

In the 970MP storage subsystem logic, the store queues above the L2 cache attempt to gather both cacheable and cache-inhibited store operations sequentially to improve bandwidth. A mode bit exists in the BIU Mode Register (at SCOM address x'043000') to disable store gathering of cache-inhibited stores. Alternatively, if store gathering is not wanted, software must insert between successive stores either an **eieio** (preferable for performance) or a **sync** to prevent it. The **eieio** instruction is broadcast onto the system bus to allow ordering to be properly enforced throughout the cache hierarchy and memory system (when detected on the system bus, these transactions have no direct effect on the processor).

2.2.10 Recommended Simplified Mnemonics

To simplify assembly language coding, a set of alternative mnemonics is provided for some frequently used operations (such as no-op, load immediate, load address, move register, and complement register). Programs written to be portable across the various assemblers for the PowerPC Architecture should not assume the existence of mnemonics not described in this document.

For a complete list of simplified mnemonics, see the *PowerPC Architecture books*.



3. Storage Subsystem

The storage subsystem (STS) of the 970MP processing unit encompasses the core interface unit (CIU), the non-cacheable unit (NCU), the L2 cache control unit and the L2 cache, and the bus interface unit (BIU).

This section provides an overview and a high-level block diagram of the storage subsystem. It summarizes key design fundamentals and the storage hierarchy. The functional units are described in detail.

The following key features are fundamental to the design:

- Store-through L1 data cache (D-cache)
- No castouts or snoop pushes by the core
- Non-blocking snoop invalidates to the core (both instruction and data invalidates)
- Integrated L2 controller
- L2 controller handling of cacheable instruction fetches, loads and stores, and **dcbz** instructions.
- Non-cacheable unit handling of other storage type instructions.

3.1 Storage Hierarchy

Table 3-1. Storage Hierarchy Characteristics

Characteristic	L1 Instruction Cache	L1 Data Cache	L2 Cache
Data type	Instructions only	Data only	Instructions and data
Size	64 KB	32 KB	1 MB
Associativity (replacement policy)	Direct map	2-way (least recently used [LRU])	8-way set associate (LRU)
Line size (sector)	128 bytes (4 × 32 bytes)	128 bytes	128 bytes
Operation granularity	128 bytes	128 bytes	128 bytes
Index	Effective address	Effective address	Physical address
Tags	Physical address	Physical address	Physical address
Number of ports	1 read or 1 write (directory has 2 reads or 1 write)	2 reads and 1 write	1 read or 1 write
Inclusiveness	N/A	N/A	Inclusive of L1 D-cache; Not inclusive of L1 instruction cache (I-cache).
Hardware coherency	No	Yes	Yes; separate snoop ports
Store policy	N/A	Store-through; no allocate on store miss	Store back; allocate on store miss
Enable bit	Yes	Yes	No
Reliability, availability, serviceability (RAS)	Parity with invalidate on error for data and tags	Parity with invalidate on error for data and tags	Error checking and correction (ECC) on data; parity on tags (recoverable with redundant tags)

3.2 Caches

The 970MP processing unit contains two levels of cache hierarchy: L1 and L2. The coherence block size for the 970MP processing unit is 128 bytes. For more information about cache characteristics of the 970MP processor, see *Table 3-1 Storage Hierarchy Characteristics* on page 83.

The 970MP processing unit automatically maintains the coherency of all data cached in these caches. Because some levels of the cache hierarchy contain both instructions and data, when the L2 cache services an instruction cache reload request, it does this in a coherent manner. This avoids the scenario where a line is reloaded into the L2 cache on behalf of a non-coherent instruction fetch, but then accessed by a load or store instruction with an aliased address that calls for correct coherency. However, the processor does not maintain instruction storage consistent with data storage and, as described in PowerPC Architecture, synchronization code is required to make the two consistent.

The L1 I-cache is indexed with an effective address. As a result, multiple copies of a particular physical block of memory can reside in multiple positions in the L1 I-cache (up to sixteen because four bits of the effective address are used in indexing the cache). The tag comparison associated with lookups in this cache is done using real addresses, so there are no 'synonym' or 'alias' hazards that must be explicitly handled by the system software.

The L1 D-cache is indexed with an effective address. Only one copy of a particular real address block is kept in the cache at a time. On each access, a tag comparison is done with the real address. On a cache miss, the cache reload mechanism searches the other three related sets to determine if the required real address block is located elsewhere in the cache. If so it will appropriately eliminate these copies.

In addition to maintaining caches, each 970MP processing unit also includes several types of queues that act as logical predecessors and extensions to the caches. In particular, the machine contains store queues for holding store data "above the caches," cast-out queues for holding modified data that has been pushed out of the caches (by the replacement algorithm, cache control instructions, and/or snoop requests), and others. Hardware keeps all of these queues coherent, and in general neither software nor system hardware should be able to observe their presence.

3.2.1 Store Gathering

The 970MP microprocessor performs gathering of cacheable stores in order to reduce the store traffic into the L2 cache. The gathering occurs in L2 store queues that sit above the L2 cache. The store queue consists of eight, 64-byte wide, fully-associative entries. Stores can be gathered while architecturally permitted (that is, there is no intervening barrier operation) and the matching address is valid in the store queue. The conditions for pushing the store queue data into the L2 cache are not visible to the programmer.

Gathering of cache-inhibited stores is also supported and can be disabled with a mode bit in the Non-Cacheable Unit (NCU) Configuration Register.

3.3 Storage Model

3.3.1 Atomicity

The 970MP processing unit is fully compliant with the architectural requirement for single-copy atomicity on naturally aligned storage accesses.

3.3.2 Storage Access Ordering

The architecture defines a weakly ordered storage model for most types of storage access scenarios. For these cases, the 970MP microprocessor takes advantage of this relaxed requirement to achieve better performance through out-of-order instruction execution and out-of-order bus transactions. As a result, if strongly ordered storage accesses are required, software must use the appropriate synchronizing instruction (Synchronize [**sync**], Page Table Entry Synchronize [**ptesyn**], Enforce In-Order Execution of I/O [**eiio**], or Lightweight Synchronize [**lwsync**]) to enforce order explicitly, or perform these accesses to regions marked with attributes that require the hardware to enforce strong ordering (that is, stores to storage marked cache-inhibited and guarded must occur in-order).

The 970MP processing unit performs load operations out-of-order internally to the processor; however, it also keeps track of these loads in a way that lets it know when an external request for exclusivity might lead to the appearance of non-sequential execution. For these cases, the 970MP processing unit can flush potentially bad results, and re-execute the code starting from the suspect load instruction.

3.3.2.1 Storage Access Alignment Support

Most storage accesses are performed without software intervention (that is, without an alignment interrupt). The relative performance of these accesses depends to some degree on their alignment. In many cases, unaligned storage accesses are handled with a performance equivalent to aligned accesses. However, in some cases the 970MP processing unit is forced to break unaligned accesses into multiple internal operations. Further, because effective-address alignment for storage references cannot be determined until execution time, and dataflow-oriented execution pipelines of the 970MP microprocessor do not support iteration, some unaligned storage accesses actually cause a pipeline flush to allow a microcoded emulation of the instruction.

The following list summarizes the cases in which the 970MP processing unit will engage a microcoded emulation of unaligned storage references:

- Any fixed-point load operation that crosses a 64-byte boundary (note 1)
- Any fixed-point load operation that misses in the L1 D-cache and crosses a 32-byte boundary (note 1)
- Any fixed-point store operation that crosses a 4 KB boundary (note 2)
- Any floating-point load double operation that is word aligned and crosses a 64-byte boundary (note 1)
- Any floating-point load double operation that is word aligned, misses in the L1 D-cache, and crosses a 32-byte boundary (note 1)
- Any floating-point store operation that is word aligned and crosses a 4 KB boundary (note 1)

Notes:

1. If the instruction is not a multiple or string instruction, the access crosses a page boundary, and the access to either page causes an exception, appearing as though the load instruction has not been executed (that is, neither the **frD**¹ or **rD**² is modified).

IBM PowerPC 970MP RISC Microprocessor

2. If the access to the first page causes an exception, storage is not modified. Otherwise, storage in the first page is updated even if the access to the second page causes an exception.

As an aid for software identification of these cases, the 970MP microprocessor supports a debug-only mode, controlled by bit 24 in Hardware Implementation Dependent Register 4 (HID4[24]), that will force an alignment interrupt in these scenarios. See *Section 4.5.8 Alignment Exception* on page 114 for a summary of cases in which the 970MP processing unit will take an alignment interrupt.

3.3.3 Atomic Updates and Reservations

The coherency granule size in the 970MP processing unit is 128 bytes. The following events will affect the state of the Reservation Register:

- Execution of a Load Word and Reserve Indexed (**lwarx**) or Load Doubleword and Reserve Indexed (**ldarx**) instruction (sets new reservation)
- Execution of a Store Word Conditional Indexed (**stwcx**) or Store Doubleword Conditional Indexed (**stdcx**) instruction (successful or not, address match or not, the reservation is cleared)
- Snooped Read with Intent to Modify (RWITM) bus operation that matches the reservation address (clears the reservation)
- Snooped Data Line Claim (DCLAIM) bus operation that matches the reservation address (clears the reservation)
- Snooped Write with FLUSH bus operation that matches the reservation address (clears the reservation)
- Snooped Write with KILL bus operation that matches the reservation address (clears the reservation)

When performing bus snooping, the 970MP processing unit checks the state of the internal caches *and* the state of the Reservation Register to formulate a snoop response. If a particular coherency block is not in any of the caches but the address is valid in the Reservation Register, then the processor and STS act as though the coherency block is in the shared state for the snoop response (this prevents another processor from taking the block as exclusive on a simple READ bus transaction).

1. Field used to specify a Floating-Point Register (FPR) to be used as a target.
2. Field used to specify a General Purpose Register (GPR) to be used as a target.

3.4 Cache Management

3.4.1 Flushing the L1 I-Cache

To help flush the entire contents of the I-cache efficiently, the 970MP microprocessor implements a special mode of operation for the Instruction Cache Block Invalidate (**icbi**) instruction. This mode can be selected using a bit in the HID1 register. In this mode, all directory lookups on behalf of an **icbi** act as though there was a real address match. Therefore, all lines looked at by the **icbi** will in fact be invalidated. As a result, the entire L1 I-cache can be invalidated by issuing a series of **icbi** instruction that step through each congruence class of the I-cache

Note: Another way to clear the I-cache is to actually fill it with a set of known values by executing a piece of code that effectively touches each line of the cache. One way to write this code is to have a series of 512 branches to branches whose effective addresses are sequentially separated by 128 bytes (the line size of the I-cache). Many other possible code sequences can achieve the same effect.

3.4.2 Flushing the L1 D-Cache

The L1 D-cache is a store-through design, so it never holds modified data. As a result, to perform a flush of the L1 D-cache, the only instruction required is a **sync**. The **sync** instruction forces any pending stores in the store queue above the L1 cache to become globally coherent before the **sync** is allowed to complete.

To completely invalidate the L1 D-cache, use the l1dc_flnh mode bit located in the HID4 to cause a flash invalidate of the D-cache. Software needs to set this bit and follow it with a **sync** instruction.

3.4.3 L2 Cache Disabling and Enabling

The L2 cache cannot be disabled.

3.4.4 L2 Cache Flushing

3.4.4.1 L2 Cache Flush in Direct-Mapped Mode

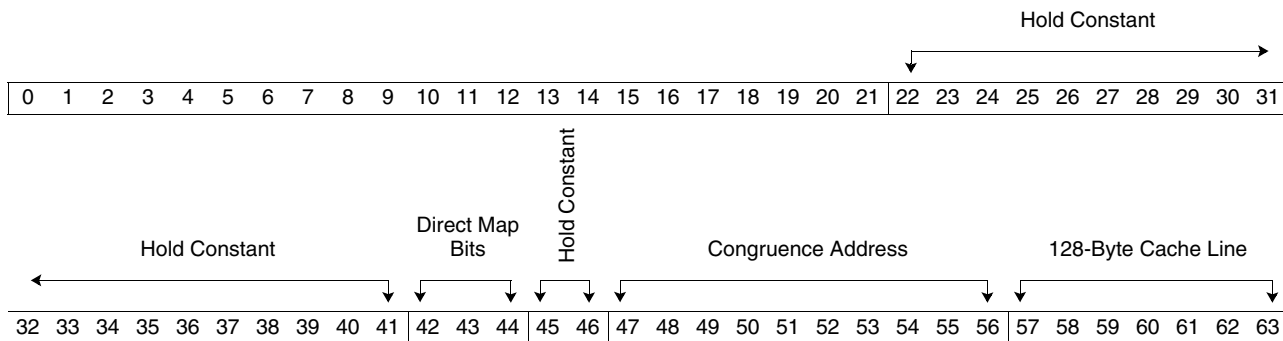
The BIU Mode Register (at SCOM address x'043000') is set to x'0000 0000 0000 8000' to enter direct-mapped mode. In direct-mapped mode, victims are selected based on a simple address decode. *Table 3-2* shows the decode. The three tag address bits used for the mapping are real address bits 42 - 44.

Table 3-2. Simple Address Decode

Real Address (Bits 42:44)	Selected Victim
000	A
001	B
010	C
011	D
100	E

IBM PowerPC 970MP RISC Microprocessor
Table 3-2. Simple Address Decode

Real Address (Bits 42:44)	Selected Victim
101	F
110	G
111	H

3.4.5 L2 Cache Flush Algorithm
L2 Address Map


Before the L2 cache of one processing unit is flushed, the other processing unit must be quiesced. For example, before flushing the L2 cache of processing unit 1, quiesce processing unit 0. Use an ATTN instruction or the service processor to quiesce the processing unit that is not being flushed (this step is not required if only one of the processing units is functional). Then, load the cache flush routine into the processing unit that is being flushed.

The following sequence will flush the entire L2 cache to memory via software:

1. Disable interrupts.
2. Disable data address translation by setting MSR[DR] to '0'.
3. Disable instruction cache (I-cache) prefetch by setting HID1[7:8] to '00'.
4. Disable data cache (D-cache) prefetch by setting HID4[25] to '1'.
5. Flash invalidate the D-cache by setting HID4[28] to '1'.
6. Execute a **sync** instruction.
7. Disable the D-cache (set HID4[37:38] to '11'). This will guarantee that all loads are visible to the L2.
8. Set the L2 to direct-mapped mode. This can be done by the service processor or through the SCOM control (SCOMC) and SCOM data (SCOMD) special purpose register (SPR) interface.
9. Execute a **sync** instruction.
10. Initialize a register with the starting address of a 4MB cacheable region of memory that is aligned on a 4-MB boundary (that is, bits 42 - 63 are all zeros).
11. Execute eight load instructions, incrementing the direct map field (bits 42 - 44) of the load address between each load (see the *L2 Address Map*).
12. Increment the congruence address field (bits 47 - 56) of the load address, and repeat step 11 (see the *L2 Address Map*).

13. Repeat step 12 for all 1024 congruence address values.

To power down after performing this sequence, the processor executes an ATTN instruction to enter the quiescent state. Once a processing unit has been flushed, it should be fenced if the intent is to power down that processing unit. This helps avoid snooping and hang problems.

To return to normal processing after performing this sequence, set the L2 cache to set-associative mode. Enable the D-cache, prefetching, data address translation, and interrupts, as required.

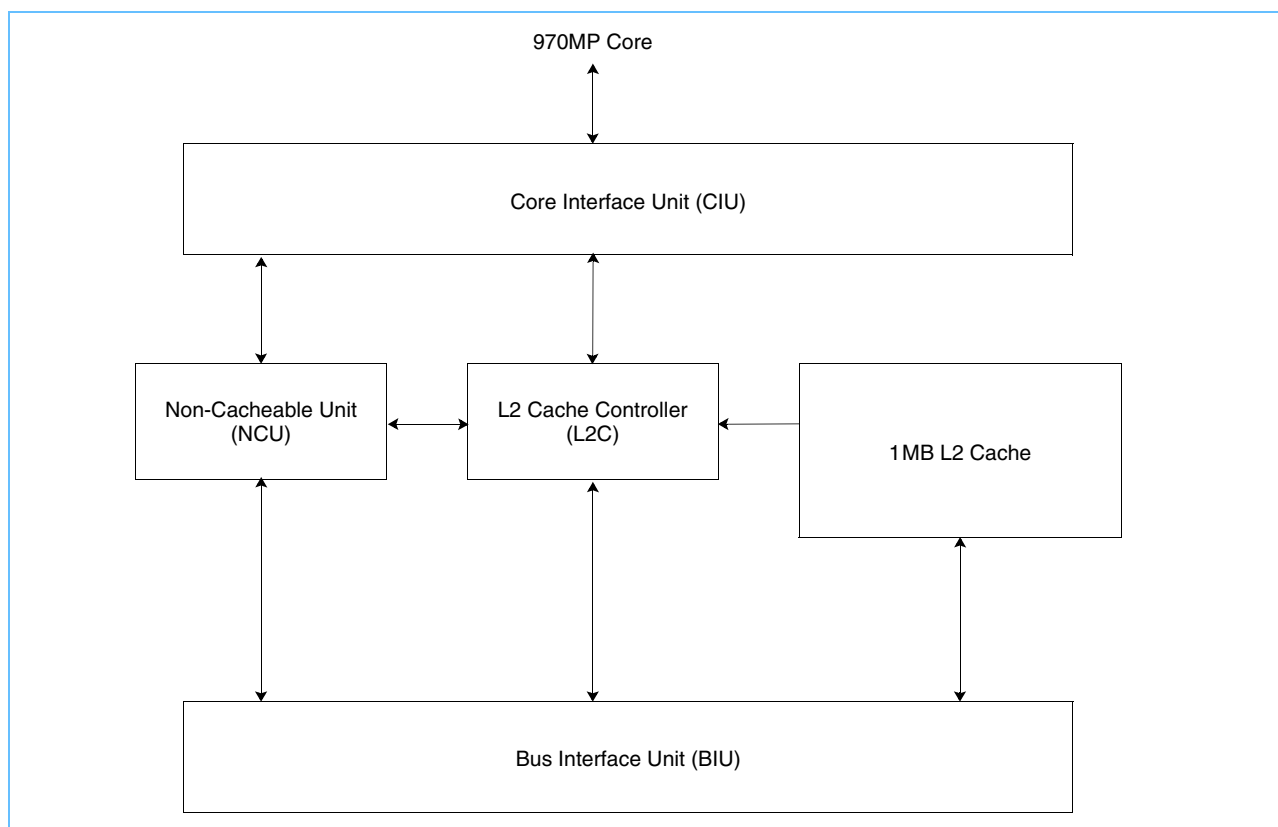
3.5 Functional Units

Table 3-3 lists the functional units within the storage subsystem, and Figure 3-1 shows how they interact. The non-cacheable unit (NCU) handles all communications to and from the core that are not handled by the L2 cache. The core interface unit (CIU) and L2 cache controller are described in detail in the following sections.

Table 3-3. Storage Subsystem Functional Units

Unit	Mnemonic
Core Interface Unit	CIU
L2 Cache Controller	L2C
Non-Cacheable Unit	NCU
Bus Interface Unit	BIU

Figure 3-1. 970MP Storage Subsystem



3.5.1 Core Interface Unit

The core interface unit (CIU) is the interface block between the 970MP core and the rest of the storage subsystem. It contains the necessary pipeline buffers and queues to maintain the required transfer rates to and from the 970MP core. The interface block consists of interfaces to the 970MP core, L2 cache interfaces, NCU interfaces, and reload interfaces.

The interfaces to the 970MP core include one instruction fetch unit (IFU) port, two load/store unit (LSU) ports, and one data prefetch and translate port. The CIU performs request arbitration, queueing, and flow control. It also maintains load/store ordering and provides prefetch support. In addition, the 970MP core interfaces include one store interface with the LSU. The CIU performs request queueing and flow control for this interface. It maintains store ordering and supports a 16-byte data path.

The CIU provides request flow control for the L2 cache interface. It dispatches operations to the L2 cache interface based on storage mode and operation type. The CIU also provides request flow control for the NCU interface. It dispatches operations to the NCU based on storage mode and operation type. It maintains cache-inhibited store ordering.

The reload/invalidate address interfaces include one IFU port, one LSU port, and one translate port. The CIU provides support for L1 cache invalidates. It also requests arbitration and flow control. The reload data bus is a 32-byte data path running at the CPU speed (1:1).

3.5.2 L2 Cache Controller

As shown in *Figure 3-1* on page 90, the L2 cache controller (L2C) resides between the CIU and the BIU and also interfaces with the NCU. See *Table 3-1 Storage Hierarchy Characteristics* on page 83 for additional details of the L2 cache features.

L2 Cache Features

- 1 MB size, 8-way set associative
- Fully inclusive of the L1 data cache
- Unified L2 cache controller (combines entities such as instructions, data, and PTEs)
- Store-in L2 cache (store-through L1 cache)
- Fully integrated cache, tags, and controller
- Five-state modified/exclusive/recent/shared/invalid (MERSI) coherency protocol

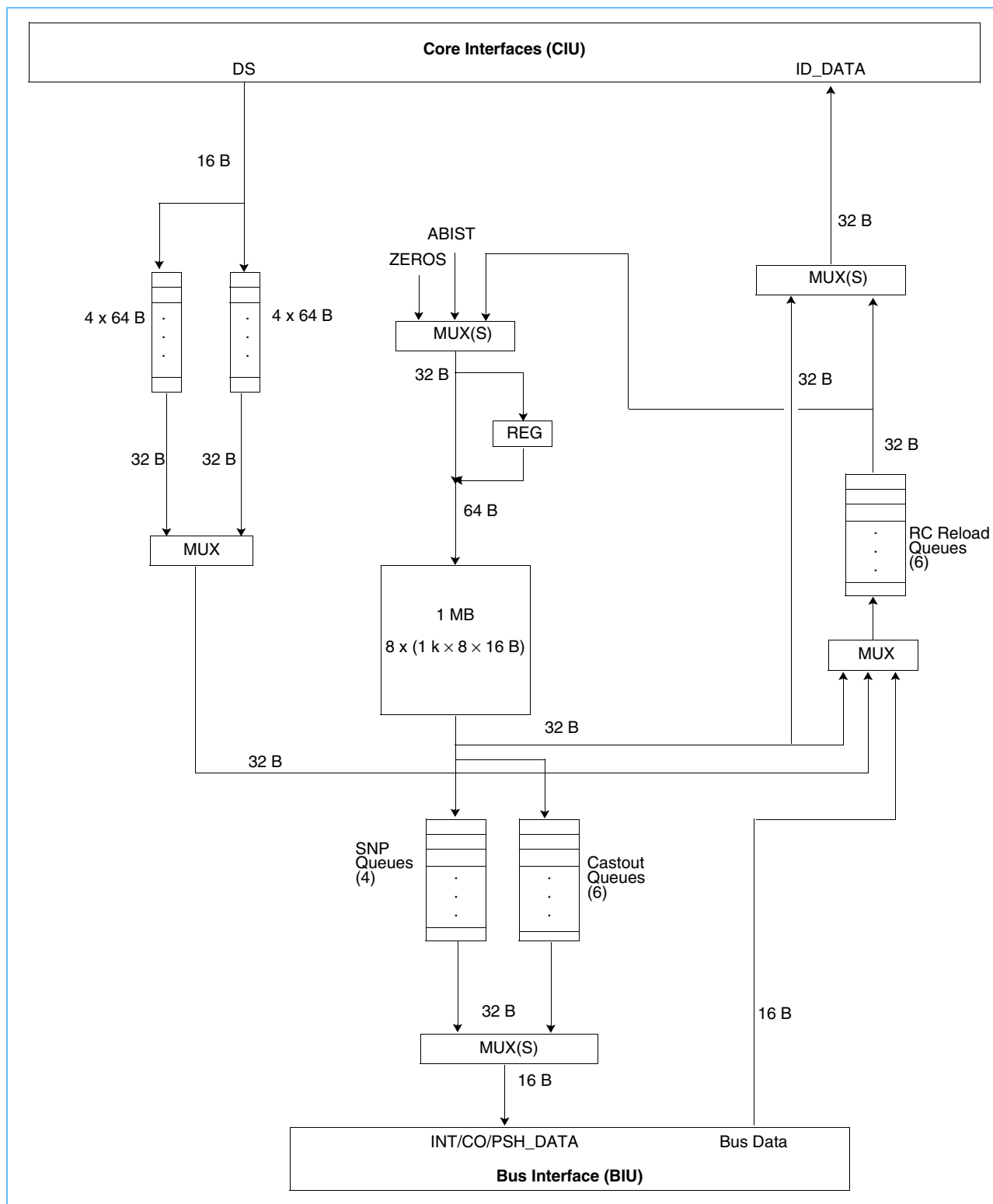
L2 Cache Controller Features

- Runs at core frequency (1:1)
- Handles all cacheable loads/stores (including **lwarx/stcwx**)
- Critical 32-byte forwarding on data loads
- Critical 32-byte forwarding in instruction fetches
- Six read/claim queues (RCQs)
- Eight 64-byte wide store queues
- Store gathering supported
- Non-blocking L1 D-cache invalidates
- Recoverable single-bit directory errors (through redundant directory)

L2 Cache Snooper Features

- Separate directory for all system bus snoops
- Four snoop/intervention/push queues

Figure 3-2 on page 92 shows the data flow of the L2 cache controller, including the data queues.

IBM PowerPC 970MP RISC Microprocessor
Figure 3-2. Data Flow in the 1MB L2 Cache


3.5.2.1 Cache Coherency

The cache-coherency protocol used in the L2 cache is standard MERSI as defined in *Table 3-4*.

Table 3-4. Cache-Coherency Protocol

Status Bit	Name	Meaning
M	Modified	The cache block is modified with respect to the rest of the memory subsystem.
E	Exclusive	The cache block is not cached in any other cache.
R ¹	Recent	The cache block is shared and this processor is the most recent reader of the cache block.
S	Shared	The cache block was (and still might be) cached by multiple processors.
I	Invalid	The cache block is invalid.

1. **Implementation Note:** The 970MP microprocessor supports a cache-coherency mode in which the R state is not used. R is replaced with shared-last (SL).

3.5.2.2 Cache-Coherency Paradoxes

In the 970MP processing unit, some parts of cache-inhibited operations are handled by a special section of logic that does not access the caches as part of its normal operation. As a result, if data associated with cache-inhibited operations is present in the caches (causing a cache-coherency paradox), the 970MP processing unit will bypass some of the caches. This introduces the possibility of observing stale data (more specifically, the 970MP processing unit will read from and write to the L1 D-cache if it hits, but it will bypass the L2 cache completely).

3.5.2.3 Cache State Transition Tables

Table 3-5 and *Table 3-6* on page 94 show the cache state transitions that occur as a result of processor instructions and snooped bus operations.

Table 3-5. 970MP L2 Cache State Transitions Due to Processor Instructions (Page 1 of 2)

Number	Instruction	Storage Mode	Coherency State	Bus Operation	AResp In	Comment
1	ld, dcb	Ca	M, E, S, R			
2	larx	Ca	M, E, S, R			
3	ld, dcbt, larx	Ca	I → S*, E	Read Cache Line	S, Null	Atomic if LARX
4	ld, larx,	NonCa		Read Noncache Line		Atomic if LARX
5	dcbt	NonCa				No-op
6	st, dcbtst, stcx	Ca	M → M			
7	st, stcx	Ca	E → M			
8	dcbtst	Ca	E → E			
9	st, dcbtst, stcx	Ca	S, R → M	DClaim		Atomic if STCX
10	st, dcbtst, stcx	Ca	I → M	RWITM	RTY	Atomic if STCX
11	st, stcx	NonCa		Write with Flush		Atomic if STCX
12	Deallocate	Ca	M → I	Write with Kill		Copyback, W = '0', M = '0'

Note: Ca: cacheable; I = '0'. NonCa: noncacheable; I = '1'. S* means R if enabled.

IBM PowerPC 970MP RISC Microprocessor

Table 3-5. 970MP L2 Cache State Transitions Due to Processor Instructions (Page 2 of 2)

Number	Instruction	Storage Mode	Coherency State	Bus Operation	AResp In	Comment
13	Deallocate	Ca	E, S, I → I			
14	dcbf	Ca	M → I	Write with Kill		W = '1', M = '0'
15	dcbf	Ca	E → I			
16	dcbf	Ca	I, S, R → I	Flush Block		
17	dcbst	Ca	M → S, E	Write with Clean		Cache > E
18	dcbst	Ca	E, R, S → E, R, S			→ E, R
19	dcbst	Ca	I	Clean		
20	dcbz	Ca	E, M → M			
21	dcbz	Ca	I, S → M	DClaim		
22	dcbz-32byte	Ca				32 bytes, treated as store
23	icbi			IKill		

Note: Ca: cacheable; I = '0'. NonCa: noncacheable; I = '1'. S* means R if enabled.

Table 3-6. 970MP L2 Cache State Transitions Due to Bus Operations (Page 1 of 2)

Number	Bus Operation		Snooper State	Rsrv State	AResp Out	AResp In	Comments
1	Read Burst	N = '1', S = '0'	M → S		M	M	Causes C → M → C data-only operation (Intervention).
2		N = '1', S = '1'	M → E		M	M	
3		N = '1', S = '0'	E, R → S		Shrl	Shrl	Causes C → C intervention.
4		N = '1', S = '1'	E, R → E, R		Shrl	Shrl	Causes C → C intervention.
5	Read Non Burst	N = '0', S = '0'	M → S		Retry	Retry	Causes Write with Clean (Push).
6		N = '0', S = '1'	M → E		Retry	Retry	Causes Write with Clean (Push).
7		N = '0', S = '0'	E, R → S		S	S	Reader can go to R state.
8		N = '0', S = '1'	E, R → E, R		S	S	Reader will go to S state.
9	Any Read		S		S	S	
10			I	R = '0' R = '1'	 S		
11	RWITM	N = '1'	M → I		M	M	Causes C → C intervention.
		N = '1'	E, R → I		Shdl	Shdl	Causes C → C intervention.
12		N = '0'	M → I		Retry	Retry	Causes Write with Kill (Push).
13		N = '0'	E, R → I		Null		N says do not intervene.
14			IS → I				
15	Write-with-Kill, DKill, DClaim		IESM → I				
16	Write-with-Flush		M → I		Retry	Retry	Causes Write with Kill (Push).
17			ISE → I				

IBM PowerPC 970MP RISC Microprocessor
Table 3-6. 970MP L2 Cache State Transitions Due to Bus Operations (Page 2 of 2)

Number	Bus Operation		Snooper State	Rsrv State	AResp Out	AResp In	Comments
18	Flush		M → I		M		Causes Write with Kill (Push).
19			ISER → I				
20	Clean		M → S, E		M		Causes Write with Clean. M = '0' Cache → E
21	Clean		E, R, S → E, R, S		S		→ E, R
22	Clean		I → I		Null		
23	SYNC, TLBSYNC				Retry Null		Retry until done. Null when done.

3.5.3 Data Prefetch

Software can manage the data prefetch hardware by using special forms of the **dcbt** instruction. Two forms of **dcbt** variants are implemented in each 970MP processing unit.

3.5.3.1 Optional *dcbt* Variant

The architecture describes the first **dcbt** variant as optional. This version of the instruction includes a 2-bit Touch Hint (TH) field (instruction bits 9 - 10), which permits a program to provide a hint regarding a sequence of data cache blocks. Such a sequence is called a "data stream." A **dcbt** instruction in which TH does not equal '00' is called a "data stream variant" of **dcbt**.

Figure 3-3 shows the instruction format and interpretation of the TH field for this **dcbt** variant.

Figure 3-3. Data Cache Block Touch X-Form (Optional Variant)

dcbt RA, RB, TH						
0	31	6	TH	11	16	278
		///		RA	RB	/

Let the effective address (EA) be the sum $(RA \mid 0) + (RB)$.

TH Description

00	The program will probably soon load from the block containing the byte addressed by the EA.
01	The program will probably soon load from the data stream consisting of the block containing the byte addressed by the EA and an unlimited number of sequentially following blocks (that is, consisting of the blocks containing the bytes addressed by $EA + n \times block_size$; where n equals 0, 1, 2,...).
10	Reserved
11	The program will probably soon load from the data stream consisting of the block containing the byte addressed by the EA and an unlimited number of sequentially preceding blocks (that is, consisting of the blocks containing the bytes addressed by $EA - n \times block_size$; where n equals 0, 1, 2,...).

Restrictions

For the data stream variant cases (TH equals '01' or TH equals '11'), prefetching the stream starts even if the first block of the stream is already in the L1 data cache.

3.5.3.2 Enhanced dcbt Variant

An additional variant of the **dcbt** instruction is implemented in each 970MP processing unit. In this version, the TH field is extended to four bits (instruction bits 7 - 10) to provide the additional variant of **dcbt**. Note that the 2-bit optional variant of the software touch is a subset of the 4-bit extended version.

Figure 3-4 on page 97 provides a brief description of this variant.

Figure 3-4. Data Cache Block Touch X-Form (Enhanced Variant) (Page 1 of 2)

dcbt RA, RB, TH						
0	31	/	7	TH	11	RA
6			16	RB	21	278
			31	/		

Let the effective address (EA) be the sum (RA | 0) + (RB).

TH Description

0000 The program will probably soon load from the block containing the byte addressed by the EA.

0001 The program will probably soon load from the data stream consisting of the block containing the byte addressed by the EA and an unlimited number of sequentially following blocks (that is, consisting of the blocks containing the bytes addressed by $EA + n \times block_size$, where n equals 0, 1, 2,...).

0011 The program will probably soon load from the data stream consisting of the block containing the byte addressed by EA and an unlimited number of sequentially preceding blocks (that is, consisting of the blocks containing the bytes addressed by $EA - n \times block_size$, where n equals 0, 1, 2,...).

1000 The **dcbt** instruction provides a hint that describes certain attributes of a data stream, and optionally indicates that the program will probably soon load from the stream. The EA, in this case, is interpreted as follows:

EA_TRUNC				D	UG	/	ID
0				56	57	58	59 60 63

Bits	Field Name	Description
0:56	EA_TRUNC	High-order 57 bits of the effective address of the first unit of the data stream. The low-order seven bits of that effective address are zero.
57	D	Direction 0 Subsequent units are the sequentially following units. 1 Subsequent units are the sequentially preceding units.
58	UG	0 No information is provided by the UG field. 1 The number of units in the data stream is unlimited, the program's need for each block of the stream is not likely to be transient, and the program will probably soon load from the stream.
59	—	Reserved
60:63	ID	Stream ID to use for this data stream.

Figure 3-4. Data Cache Block Touch X-Form (Enhanced Variant) (Page 2 of 2)

1010 The **dcbt** instruction provides a hint that describes certain attributes of a data stream, or indicates that the program will probably soon load from data streams that have been described using **dctb** instructions in which TH[0] equals '1', or will probably no longer load from such data streams.

The EA is interpreted as follows. If GO equals '1' and S $\neq$ '00' the hint provided by the instruction is undefined; the remainder of this instruction description assumes that this combination is not used. A completely described stream is one that has been described with both a '1000' TH values (specifying starting address and direction of the stream) and a '1010' TH value (specifying the length and transience of the stream).

Bits	Field Name	Description
0:31	—	Reserved.
32	GO	0 No information is provided by this field.
		1 The program will probably soon load from all completely described streams, and will probably no longer load from any partially defined streams. All other fields of the EA are ignored.
33:34	S	00 No information is provided by this field.
		01 Reserved
		10 The program will probably no longer load from the data stream (if any) associated with the specified stream ID. All other fields of the EA except ID are ignored.
		11 The program will probably no longer load from the data streams associated with all stream IDs. All other fields of the EA are ignored.
35:46	—	Reserved.
47:56	Unit_cnt	Number of (aligned 128 B) units in the data stream.
57	T	0 No information is provided by this field.
		1 The program's need for each unit of the data stream is likely to be transient.
58	U	0 No information is provided by this field.
		1 The number of units in the data stream is unlimited. The unit_cnt field is ignored.
59	—	Reserved.
60:63	ID	Stream ID.

All other TH decodes are reserved.

Restrictions

The TH equals '1000' version of the **dcbt** instruction is not recognized when MSR[DR] equals '0'.

4. Exceptions

The operating environment architecture (OEA) portion of the PowerPC Architecture defines the mechanism by which PowerPC processors implement exceptions (referred to as interrupts in the architecture specification). Exception conditions can be defined at other levels of the architecture. For example, the user instruction set architecture (UISA) defines conditions that can cause floating-point exceptions; the OEA defines the mechanism by which the exception is taken.

The PowerPC exception mechanism allows the processor to change to supervisor state as a result of unusual conditions arising in the execution of instructions and from external signals, bus errors, or various internal conditions. When exceptions occur, information about the state of the processor is saved to certain registers and the processor begins execution at an address (exception vector) predetermined for each exception. Processing of exceptions begins in supervisor mode.

Although multiple exception conditions can map to a single exception vector, often a more specific condition can be determined by examining a register associated with the exception—for example, the Data Storage Interrupt Status Register (DSISR) and the Floating-Point Status and Control Register (FPSCR). The high-order bits of the Machine State Register (MSR) are also set for some exceptions. Software can explicitly enable or disable some exception conditions.

The PowerPC Architecture requires that exceptions be taken in program order. Therefore, although a particular implementation can recognize exception conditions out-of-order, they are handled strictly in-order with respect to the instruction stream. When an instruction-caused exception is recognized, any unexecuted instructions that appear earlier in the instruction stream, including any that have not yet entered the execute state, are required to complete before the exception is taken. For example, if a single instruction encounters multiple exception conditions, those exceptions are taken and handled based on the priority of the exception. Likewise, exceptions that are asynchronous and precise are recognized when they occur, but are not handled until all instructions currently in the execute stage successfully complete execution and report their results.

To prevent loss of state information, exception handlers must save the information stored in the Machine Status Save/Restore Registers, SRR0 and SRR1, soon after the exception is taken to prevent this information from being lost due to another exception being taken. Because exceptions can occur while an exception handler routine is executing, multiple exceptions can become nested. It is up to the exception handler to save the necessary state information if control is to return to the excepting program.

In many cases, after the exception handler returns, there is an attempt to execute the instruction that caused the exception (such as a page fault). Instruction execution continues until the next exception condition is encountered. Recognizing and handling exception conditions sequentially guarantees that the machine state is recoverable and processing can resume without losing instruction results.

In this book, the following terms are used to describe the stages of exception processing.

Recognition	Exception recognition occurs when the condition that can cause an exception is identified by the processor.
Taken	An exception is said to be taken when control of instruction execution is passed to the exception handler. That is, the context is saved and the instruction at the appropriate vector offset is fetched and the exception handler routine is begun in supervisor mode.
Handling	Exception handling is performed by the software linked to the appropriate vector offset. Exception handling is begun in supervisor mode (referred to as privileged state in the architecture specification).

IBM PowerPC 970MP RISC Microprocessor

Note: The PowerPC Architecture documentation refers to exceptions as interrupts. In this book, the term “interrupt” is reserved to refer to asynchronous exceptions and sometimes to the event that causes the exception. The PowerPC Architecture also uses the word “exception” to refer to IEEE-defined floating-point exception conditions that might cause a program exception to be taken (see the *PowerPC Microprocessor Family: The Programming Environments* manual for more information). The occurrence of these IEEE exceptions might not cause an exception to be taken. IEEE-defined exceptions are referred to as IEEE floating-point exceptions or floating-point exceptions.

Note: Previous PowerPC microprocessors supported specifying the base real address by using the exception prefix field, MSR[IP]. The 970MP microprocessor does not support this.

4.1 970MP Microprocessor Exceptions

As specified by the PowerPC Architecture, exceptions can be either precise or imprecise and either synchronous or asynchronous. Asynchronous exceptions are caused by events external to the processor's execution; synchronous exceptions are caused by instructions. The types of exceptions are shown in *Table 4-1*.

Note: All exceptions except for the maintenance exception and performance monitor exception are defined, at least to some extent, by the PowerPC Architecture.

Table 4-1. 970MP Microprocessor Exception Classifications

Synchronous/Asynchronous	Precise/Imprecise	Exception Types
Asynchronous, nonmaskable	Imprecise	Machine check, system reset
Asynchronous, maskable	Precise	External interrupt, decrementer, maintenance exception, performance monitor exception
Synchronous	Precise	Instruction-caused exceptions

These classifications are discussed in greater detail in *Section 4.2* on page 103. For a better understanding of precise exceptions, see Chapter 6, “Exceptions” of the *PowerPC Microprocessor Family: The Programming Environments* manual. Exceptions implemented in the 970MP microprocessor, and conditions that cause them, are listed in *Table 4-2 Exceptions and Conditions* on page 101.

Table 4-2. Exceptions and Conditions (Page 1 of 2)

Exception Type	Vector Offset (hexadecimal)	Causing Conditions
System reset	00100	Either the assertion of the soft reset input pin or an SCOM command sequence for "soft reset." See <i>Section 4.5.1 System Reset Exception</i> on page 110.
Machine check	00200	There are many causes of a machine check exception. See <i>Section 4.5.2 Machine Check Exceptions</i> on page 111.
Data storage	00300	Page fault, as defined in the PowerPC Architecture. See <i>Section 4.5.3 Data Storage Exception</i> on page 113.
Data segment	00380	Data segment fault, as defined in the PowerPC Architecture. See <i>Section 4.5.4 Data Segment Exception</i> on page 113.
Instruction storage	00400	Page fault, as defined in the PowerPC Architecture. See <i>Section 4.5.5 Instruction Storage Exception</i> on page 113.
Instruction segment	00480	Instruction segment fault, as defined in the PowerPC Architecture. See <i>Section 4.5.6 Instruction Segment Exception</i> on page 113.
External interrupt	00500	Assertion of the external interrupt input signal. See <i>Section 4.5.7 External Interrupt Exception</i> on page 114.
Alignment	00600	There are many causes of an alignment exception. See <i>Section 4.5.8 Alignment Exception</i> on page 114.
Program	00700	As defined by the PowerPC Architecture (for example, an instruction opcode error). See <i>Section 4.5.9 Program Exception</i> on page 114.
Floating-point unavailable	00800	As defined by the PowerPC Architecture. See <i>Section 4.5.10 Floating-Point Unavailable Exception</i> on page 115.
Decrementer	00900	As defined by the PowerPC Architecture. When the most-significant bit of the Decrementer Register (DEC) changes to '1' and MSR[EE] equals '1', it is the responsibility of the service routine for the decrementer exception to clear DEC[0]. See <i>Section 4.5.11 Decrementer Exception</i> on page 115.
Hypervisor decrementer	00980	The Hypervisor Decrementer is similar to the decrementer and is used to return control to the hypervisor. This interrupt is activated when no higher priority interrupt is active and MSR[EE]='1' or MSR[HV]='0' and the Hypervisor Decrementer is negative (HDEC[0]='1'). This is a level sensitive interrupt and as such it is the responsibility of the interrupt service routine to clear HDEC[0].
System call	00C00	Execution of the System Call (sc) instruction. See <i>Section 4.5.12 System Call Exception</i> on page 115.
Trace	00D00	MSR[SE] equals '1' or MSR[BE] equals '1', and a trace-marked instruction successfully completes. See <i>Section 4.5.13 Trace Exception</i> on page 115.
Performance monitor	00F00	The MSR[EE] bit is set, the MMCR0[PMXE] bit is set, and any of the performance monitor counters overflow. The performance monitor exception can also be triggered by the '0' to '1' transition of a particular time-base bit. See <i>Section 4.5.14 Performance Monitor Exception</i> on page 116.
VPU unavailable	00F20	No higher priority exception exists, an attempt is made to execute a vector instruction, and MSR[VP] equals '0'. See <i>Section 4.5.15 VPU Unavailable Exception</i> on page 117.

IBM PowerPC 970MP RISC Microprocessor*Table 4-2. Exceptions and Conditions (Page 2 of 2)*

Exception Type	Vector Offset (hexadecimal)	Causing Conditions
Instruction address breakpoint	01300	PowerPC 970MP microprocessor does not support a visible form of the instruction address breakpoint facility. The instruction address breakpoint feature is accessible through the support processor interface. See <i>Section 4.5.16 Instruction Address Breakpoint Exception</i> on page 117.
Maintenance	01600	This exception can be signaled by a number of internal events, as well as by explicit commands from the support processor. See <i>Section 4.5.17 Maintenance Exception</i> on page 117.
VPU assist	01700	This exception occurs when operating in Java mode and the input operands or the result of an operation are denormalized. See <i>Section 4.5.18 VPU Assist Exception</i> on page 118.

4.2 Exception Recognition and Priorities

Exceptions are roughly prioritized by exception class, as follows.

- Nonmaskable, asynchronous exceptions have priority over all other exceptions. These are system reset and machine check exceptions. These exceptions cannot be delayed and do not wait for completion of any precise exception handling. (However, the machine check exception condition can be disabled so the condition causes the processor to go directly into the checkstop¹ state).
- Synchronous, precise exceptions are caused by instructions and are taken in strict program order.
- Imprecise exceptions (imprecise mode floating-point enabled exceptions) are caused by instructions, and they are delayed until higher priority exceptions are taken.

Note: The 970MP microprocessor does not implement an exception of this type.

- Maskable asynchronous exceptions (external, decrementer, maintenance, performance monitor, and exceptions) are delayed if higher priority exceptions are taken.

Section 4.3 Exception Processing on page 105 describes how the 970MP microprocessor handles exceptions up to the point of signalling the appropriate interrupt to occur. Note that a recoverable state is reached if the completed store queue is empty (drained, not cancelled) and any instruction that is next in program order and has been signaled to complete has completed. If MSR[RI] equals '0', the 970MP processing unit is in a nonrecoverable state. Also, instruction completion is defined as updating all architectural registers associated with that instruction, and then removing that instruction from the completion buffer.

4.2.1 Exception Priorities

The following list is a summary of the exception priorities for the 970MP microprocessor:

1. System reset exception
2. Machine check exception
3. Instruction dependent (as follows)
 - Fixed-point loads and stores
 - Mode dependent loads and stores
 - (1) Illegal instruction type of program exception
 - (2) Privileged type of program exception (for example, MSR[PR] set to '1')
 - Data segment exception
 - Data storage exception
 - Alignment exception
 - Trace exception
 - Floating-point loads and stores
 - Floating-point unavailable exception
 - Data segment exception
 - Data storage exception (DSI)
 - Alignment exception
 - Trace exception
 - Other floating-point instructions
 - Floating-point unavailable exception

1. Hardware has detected a condition that it cannot resolve, and which prevents normal operation. It stops executing instructions, responding to interrupts, and so on.

IBM PowerPC 970MP RISC Microprocessor

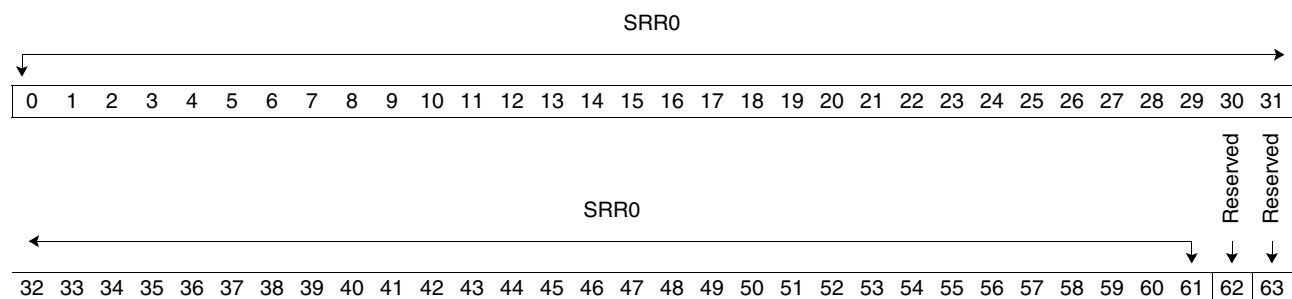
- Precise-mode, floating-point-enabled, exceptions type of program exception
 - Trace exception
 - Vector instructions
 - VPU unavailable exception
 - Trace exception
 - Return from Exception Doubleword (**rfid**) instruction, Move to Machine State Register (**mtmsr**), Move to Machine State Register Doubleword (**mtmsrd**)
 - Precise-mode, floating-point-enabled, exceptions type of program exception
 - Trace exception (for **mtmsr** or **mtmsrd** only)
 - Other instructions
 - Exceptions that are mutually exclusive and the same priority:
 - (1) Trap type of program exception
 - (2) System call
 - (3) Privileged instruction type of program exception
 - (4) Illegal instruction type of program exception
 - Trace exception
 - VPU assist exception
 - Instruction segment exception
 - Instruction storage exception
4. Maintenance exception
 5. External interrupt
 6. Performance monitor exception
 7. Decrementer exception

4.3 Exception Processing

When an exception is taken, the processor uses SRR0 and SRR1 to save the contents of the MSR for the current context, and to identify where instruction execution should resume after the exception is handled.

4.3.1 Machine Status Save/Restore Register 0 (SRR0)

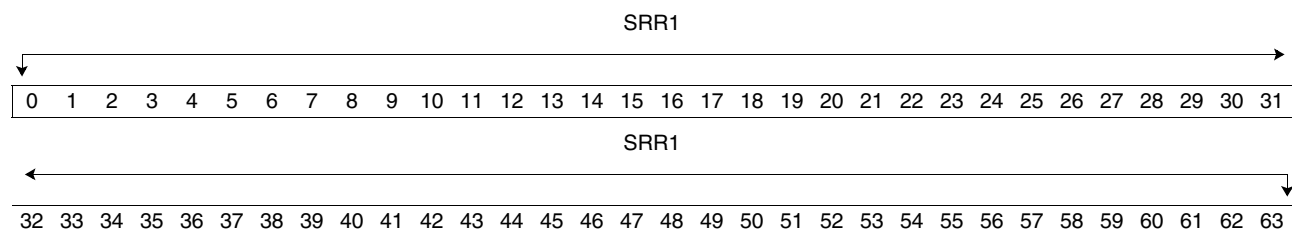
When an exception occurs, the address saved in SRR0 determines where instruction processing should resume when the exception handler returns control to the interrupted process. Depending on the exception, this might be the address in SRR0 or at the next address in the program flow. All instructions in the program flow preceding this one will have completed execution and no subsequent instruction will have begun execution. This might be the address of the instruction that caused the exception or the next one (as in the case of a system call, trace, or trap exception). The SRR0 Register is shown below.



Bits	Field Name	Description
0:61	SRR0	Holds the effective address (EA) for the instruction in the interrupted program flow.
62	—	Reserved. Returns a zero when read.
63	—	Reserved. Returns a zero when read.

4.3.2 Machine Status Save/Restore Register 1 (SRR1)

SRR1 is used to save machine status (selected MSR bits and possibly other status bits as well) on exceptions and to restore those values when an **rfid** instruction is executed. SRR1 is shown below.



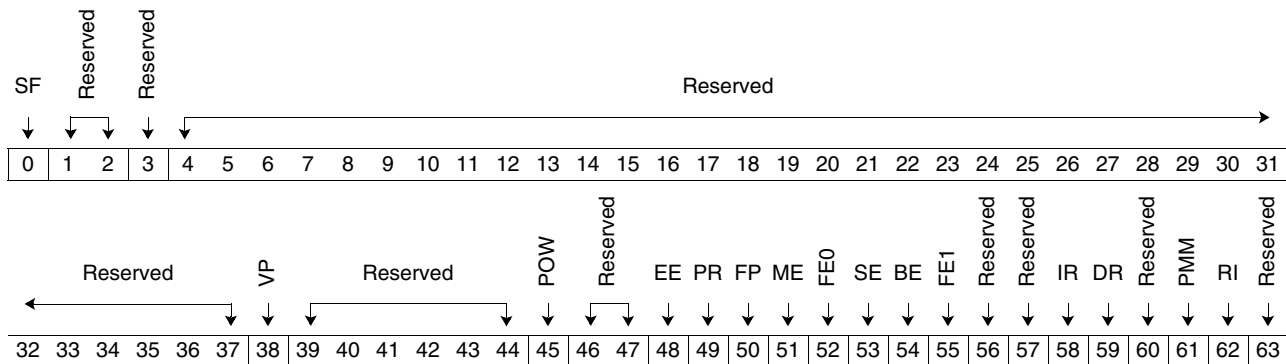
Bits	Field Name	Description
0:63	SRR1	Exception-Specific Information and MSR Bit Values For most exceptions, bits 33 - 36 and 42 - 47 of SRR1 are loaded with exception-specific information. Bits 0 - 32, 37 - 41, and 48 - 63 of SRR1 are loaded with a copy of the corresponding bits of the MSR (before taking the exception).

Note: The function of the SRR1 is to save the current state of the machine (that is, the MSR) before a temporary state is invoked to service exceptions. After the servicing of the exception, the contents of SRR1 are returned to the MSR and the code stream can continue.

IBM PowerPC 970MP RISC Microprocessor

4.3.3 Machine State Register (MSR)

The format of the 970MP processing unit's MSR is below.



Bits	Name	Description
0	SF	64-bit mode. 0 Processor runs in 32-bit mode. 1 Default mode. Processor runs in 64-bit mode.
1:2	—	Reserved. Returns zeros when read.
3	HV	Hypervisor mode. Set when running on a non-partitioned system or when "hypervisor code" is executing on a partitioned system. MSR[HV] can be set to '1' only by the system call instruction and some interrupts. It can be set to '0' only by the rfd and hrfd instructions.
4:37	—	Reserved. Returns zeros when read.
38	VP	Vector processor available. 0 The processor prevents execution of all vector instructions, including loads, stores, and moves. If such execution is attempted, a VPU unavailable exception is raised. 1 The processor can execute all vector instructions. The Vector Save/Restore Register (VRSAVE) is not protected by MSR[VP]. None of the data streaming family of instructions (dst , dstt , dstst , dststt , dss , and dssall) are affected by MSR[VP].
39:44	—	Reserved. Returns zeros when read.
45	POW	Power-management enable 0 Power management disabled (normal operation mode). 1 Power management enabled (reduced power mode).
46:47	—	Reserved. Returns zeros when read.
48	EE	External exception enable. 0 The processor delays recognition of external exceptions and decremter exception conditions. 1 The processor is enabled to take an external exception or the decremter exception. Note: Setting MSR[EE] masks not only the architecture-defined external exception and decremter exceptions, but also the 970MP-specific debug and performance monitor exceptions.
49	PR	Problem state (user mode). 0 The processor is privileged to execute any instruction. 1 The processor can only execute nonprivileged instructions.
50	FP	Floating-point available. 0 The processor prevents dispatch of floating-point instructions, including floating-point loads, stores, and moves. 1 The processor can execute floating-point instructions and can take floating-point enabled program exceptions.

IBM PowerPC 970MP RISC Microprocessor

Bits	Name	Description
51	ME	Machine check enable. 0 Machine check exceptions are disabled. If one occurs, the system enters checkstop. 1 Machine check exceptions are enabled. Only rfid instructions can alter MSR[ME].
52	FE0	IEEE floating-point exception mode 0.
53	SE	Single-step trace enable. 0 The processor executes instructions normally. 1 The processor generates a single-step trace exception upon the successful execution of every instruction except rfid , Instruction Cache Synchronize (isync), and sc . Successful execution means that the instruction caused no other exception.
54	BE	Branch trace enable. 0 The processor executes branch instructions normally. 1 The processor generates a branch type of trace exception when a branch instruction executes successfully.
55	FE1	IEEE floating-point exception mode 1.
56	—	Reserved. Returns a zero when read.
57	—	Reserved. Returns a zero when read.
58	IR	Instruction address translation. 0 Instruction address translation is disabled. 1 Instruction address translation is enabled.
59	DR	Data address translation 0 Data address translation is disabled. If data stream touch (dst) and data stream touch for store (dstst) instructions are executed when DR equals '0', the results are boundedly undefined. 1 Data address translation is enabled. Data stream touch (dst) and data stream touch for store (dstst) instructions are supported when DR equals '1'.
60	—	Reserved. Returns a zero when read.
61	PMM	Performance monitor mode. This register bit is used to enable and disable performance monitor activity controlled by the process mark bit.
62	RI	Indicates whether a system reset or machine check exception is recoverable. 0 Exception is not recoverable. 1 Exception is recoverable. The RI bit indicates whether, from the perspective of the processor, it is safe to continue (that is, processor state data such as that saved to SRR0 is valid), but it does not guarantee that the interrupted process is recoverable. Exception handlers must look at SRR1[RI] to determine this.
63	—	Reserved. Returns a zero when read.

The 970MP microprocessor provides precise floating-point exceptions whenever either of the floating-point enabled exception mode bits (MSR[FE0] and MSR[FE1]) are set. *Table 4-3 IEEE Floating-Point Exception Mode Bits* on page 108 explains how the bits are used to set the mode. In all cases, the 970MP processing unit aggressively executes the floating-point instructions (even out-of-order as required), and sorts out any resulting exceptions at completion time. In some cases, due to the group-oriented instruction tracking scheme used by the 970MP microprocessor, when an exception is detected, the hardware will flush the pipeline and re-dispatch the instructions individually in order to provide the precise exception. Since this only happens if an exception is to be taken, it does not represent a measurable decrease in performance.

IBM PowerPC 970MP RISC Microprocessor
Table 4-3. IEEE Floating-Point Exception Mode Bits

FE0	FE1	Mode
0	0	Floating-point exceptions disabled.
0	1	Imprecise nonrecoverable. For this setting, the 970MP microprocessor operates in floating-point precise mode.
1	0	Imprecise recoverable. For this setting, the 970MP microprocessor operates in floating-point precise mode.
1	1	Floating-point precise mode.

4.3.4 Enabling and Disabling Exceptions

When a condition exists that might cause an exception to be generated, it must be determined whether the exception is enabled for that condition.

- IEEE floating-point enabled exceptions (a type of program exception) are ignored when both MSR[FE0] and MSR[FE1] are cleared. If either bit is set, all IEEE enabled floating-point exceptions are taken and cause a program exception.
- Asynchronous, maskable exceptions (external, decremter, performance monitor, and maintenance exceptions) are enabled by setting MSR[EE]. When MSR[EE] equals '0', recognition of these exception conditions is delayed. MSR[EE] is cleared automatically when an exception is taken to delay recognition of conditions causing those exceptions.
- A machine check exception can occur only if the machine check enable bit, MSR[ME], is set. If MSR[ME] is cleared, the processor goes directly into checkstop state when a machine check exception condition occurs.
- System reset exceptions cannot be masked.

4.3.5 Exception Processing Steps

After it is determined that the exception can be taken (by confirming that any instruction-caused exceptions occurring earlier in the instruction stream have been handled, and by confirming that the exception is enabled for the exception condition), the processor does the following steps:

1. Loads SRR0 with an instruction address that depends on the type of exception. Normally, this is the instruction that would have completed next had the exception not been taken. See the individual exception description (*Section 4.5* beginning on page 110) for details about how this register is used for specific exceptions.
2. Loads SRR1[33:36, 42:47] with information specific to the exception type.
3. Loads SRR1[0:32, 37:41, 48:63] with a copy of the corresponding MSR bits (prior to the exception).
4. Sets the MSR as described in *Section 4.5 Exception Definitions* on page 110. The new values take effect as the first instruction of the exception-handler routine is fetched.

Note: MSR[IR] and MSR[DR] are cleared for all exception types. Therefore, address translation is disabled for both instruction fetches and data accesses beginning with the first instruction of the exception-handler routine.

Instruction fetch and execution resumes, using the new MSR value, at a location specific to the exception type. The location is determined by adding the exception's vector offset (see *Table 4-2* on page 101) to the value in the Hardware Interrupt Offset Register (HIOR). For a machine check exception that occurs when MSR[ME] equals '0' (machine check exceptions are disabled), the checkstop state is entered (the machine stops executing instructions).

4.3.6 Setting the Recoverable Exception in the MSR

The recoverable exception (RI) bit in the MSR was designed to indicate to the exception handler whether the exception is recoverable. When an exception occurs, the RI bit is copied from the MSR to SRR1 and cleared in the MSR. All exceptions are disabled except machine check. If a machine check exception occurs while MSR[RI] is clear, a '0' value is found in SRR1[RI] to indicate that the machine state is definitely not recoverable. When MSR[RI] equals '1', the exception is recoverable as far as the current state of the machine and all programs concerned including noncritical machine checks. Thus, in all exceptions, if SRR1[RI] is cleared, the machine state is not recoverable. If it is set, the exception is recoverable with respect to the processor and all programs. An operating system can handle MSR[RI] as follows:

- Use the Special Purpose Registers (SPRG0-SPRG3) to aid in saving the machine state. IBM suggests pointing SPRG0 to a stack save area in memory and saving three General Purpose Registers (GPRs) in SPRG1-3. Move SPRG0 into one of the GPRs that was saved. This GPR now points to the save area in memory. Move the GPRs, SRR0, SRR1, SPRG1-3, and other registers to be used by the exception routine into the stack save area. Update SPRG0 to point to a new save area. Set MSR[RI] to indicate that machine state has been saved. Also set MSR[EE] if you want to re-enable external exceptions.
- When exception processing is complete, clear MSR[EE] and MSR[RI]. Adjust SPRG0 to point to the stack saved area, restore the GPRs, SRR0 and SRR1, and any other register that you might have saved, execute **rfid**. This returns the processor to the interrupted program.

4.3.7 Returning from an Exception Handler

The **rfid** instruction performs context synchronization by allowing previously-issued instructions to complete before returning to the interrupted process. In general, execution of the **rfid** instruction ensures the following:

- All previous instructions have completed to a point where they can no longer cause an exception.
- Previous instructions complete execution in the context (privilege, protection, and address translation) under which they were issued.
- The **rfid** instruction copies SRR1 bits back into the MSR, and resets the MSR[POW] bit.
- Instructions fetched after this instruction execute in the context established by this instruction.
- Program execution resumes at the instruction indicated by SRR0.

For a complete description of context synchronization, see Chapter 6, "Exceptions" of the *PowerPC Microprocessor Family: The Programming Environments* manual.

4.4 Process Switching

The following instructions are useful for restoring proper context during process switching:

- The Synchronize (**sync**) instruction orders the effects of instruction execution. All instructions previously initiated appear to have completed before the **sync** instruction completes, and no subsequent instructions appear to be initiated until the **sync** instruction completes.
- The Instruction Cache Synchronize (**isync**) instruction waits for all previous instructions to complete and then discards any fetched instructions, causing subsequent instructions to be fetched (or refetched) from memory and to execute in the context (privilege, translation, and protection) established by the previous instructions.
- The Store Word Conditional Indexed/Store Doubleword Conditional Indexed (**stwcx./stdcx.**) instruction clears any outstanding reservations, ensuring that a Load Word and Reserve Indexed/Load Double Word and Reserve Indexed (**lwarx/ldarx**) instruction in an old process is not paired with an **stwcx./stdcx.** instruction in a new one.

The operating system should set MSR[RI] as described in *Section 4.3.6 Setting the Recoverable Exception in the MSR* on page 109.

4.5 Exception Definitions

When an exception/interrupt is taken, all bits in the MSR are set to '0', with the following exceptions:

- Exceptions always set MSR[SF] to '1'.
- Only the machine check exception sets MSR[ME] to '0'. All other exceptions leave MSR[ME] unchanged.

The following sections describe the implementation-dependent aspects of the exceptions.

Note: If a description is not provided, the 970MP microprocessor behaves as described in the *PowerPC Architecture books*.

4.5.1 System Reset Exception

The system reset exception is a non-maskable, asynchronous exception that is caused by the assertion of either the soft reset input pin, or by the SCOM command sequence for soft reset.

The Not Hard Reset bit in HID0[15] can be used to help software distinguish between a hard reset and a soft reset. To use this capability, software should initially set this bit to a '1'. Later, when a system reset exception is taken, software can check the state of this bit to determine which type of reset occurred. If the bit is still set, then the reset was a soft reset, and if the bit is a zero, the reset was a hard reset.

4.5.2 Machine Check Exceptions

There are several possible causes of machine check exceptions in the 970MP microprocessor, some of which are generally recoverable, and some of which are non-recoverable.

The following causes of machine check exceptions are precise and synchronous with the instruction that caused the operation that encountered the error (that is, SRR0 contains the address of the instruction that caused the operation).

- The detection of a parity error in the L1 data cache (D-cache), the L1 D-cache tag, the data effective-to-real-address translation (D-ERAT), the translation lookaside buffer (TLB), or the segment lookaside buffer (SLB) during the execution of a load or store instruction. If the exception is caused by a soft error, then executing the appropriate sequence of instructions in the machine check handler program will clear the error condition without causing any loss of state, permitting the interrupted program to resume if MSR[RI] was a '1' when the instruction that encountered the error was executed.

Note: The L1 D-cache and the L1 D-cache tag parity errors are recovered by hardware in the 970MP processing unit (default mode), without a machine check interrupt.

- The detection of an uncorrectable error checking and correction (ECC) error in the L2 cache when a load instruction is executed.
- The detection of an uncorrectable ECC error in the L2 cache while the page table is being searched in the process of translating an address.
- The detection of erroneous data that is being returned to satisfy a load instruction for which the effective address specified a location in caching inhibited memory.

For hard errors, these characteristics cannot be reliably provided on a machine check, because it is likely that the failure will prevent reliable execution. Additionally, a machine check exception that occurs when MSR[ME] equals '0' results in a checkstop.

In addition, there are a few possible sources for asynchronous machine check exceptions. A machine check exception is taken when the machine check input pin is asserted, if enabled by setting HID0[32] to '1'. The Fault Isolation Register (FIR), debug logic, and hang recovery logic can also be programmed to induce machine check exceptions for various error conditions. Since these signals are asynchronous with respect to the executing program, asynchronous machine checks might or might not be recoverable. Software can use the MSR[RI] bit to help identify the cases where the machine check exception is recoverable.

Information about the suspected source of the error condition is logged into either the SRR1 Register, the DSISR Register, or both as defined in *Table 4-4* on page 112 for synchronous machine checks.

IBM PowerPC 970MP RISC Microprocessor

Table 4-4. Register Settings for Machine Check Exception

Register	Bits	Setting
SRR0	0:63	Effective address of the next instruction that would have executed if the machine check exception was not taken. When this is a recoverable machine check due to a load that has surfaced an error, this will be the address of the load instruction itself (the 970MP microprocessor allows the instruction to execute to surface the error, but inhibits the commitment of the results). When this is a recoverable machine check due to an instruction fetch surfacing an error, this will be the address of an instruction that initiated the memory/cache access.
SRR1	0:41	Loaded from MSR.
	42	Exception caused by instruction fetch unit (IFU) detection of a hardware uncorrectable error (UE).
	43	Exception caused by load or store detection of error (see DSISR below).
	44:45	Exception cause indicated by the following encoding: 00 No error encoded. 01 Exception caused by an SLB parity error detected while translating an instruction fetch address. 10 Exception caused by a TLB parity error detected while translating an instruction fetch address. 11 Exception caused by a hardware uncorrectable error (UE) detected while doing a reload of an instruction-fetch TLB tablewalk.
	46:61	Loaded from MSR.
	62	Loaded from MSR[62] if recoverable. Otherwise, set to zero.
	63	Loaded from MSR.
DSISR	0:5	All zeros.
	6	Set to '1' for a store or dcbz instruction; otherwise, set to '0'.
	7:15	All zeros.
	16	Exception caused by a UE deferred error (the Data Address Register [DAR] is undefined).
	17	Exception caused by a UE deferred error during a tablewalk (D-side).
	18	Exception was caused by a software-recoverable parity error in the L1 D-cache.
	19	Exception was caused by a software-recoverable parity error in the L1 D-cache tag.
	20	Exception was caused by a software-recoverable parity error in the D-ERAT.
	21	Exception was caused by a software-recoverable parity error in the TLB.
	22	Zero.
	23	Exception was caused by an SLB parity error (might not be recoverable). This condition could occur if the effective segment ID (ESID) fields of two or more SLB entries contain the same value.
	24:31	All zeros.
DAR	0:63	Effective address computed by a load or store instruction that caused the operation that encountered a parity error in the D-ERAT, TLB, or SLB, or that encountered an uncorrectable error while attempting to reload a TLB entry. Effective address computed by the load instruction that caused the operation that encountered a parity error in the L1 D-cache or L1 D-cache tag arrays For all other types of machine check exceptions, the DAR is undefined (including when the operand of the load instruction contains a UE).

Note: As mentioned previously, the machine check exception handler is expected to help hardware recover from certain types of D-cache, D-cache directory, D-ERAT, and TLB errors detected by the hardware. In general terms, the exception handler should:

- Check whether the machine check exception is recoverable by looking at the state of the RI bit in SRR1.
- Determine the type of error that caused the machine check by looking at the state of the SRR1 and DSISR Registers.
- Flush the contents of the array that reported the detected error (this process is slightly different for each of the possible arrays).
- Return to the interrupted process.

If no error is encoded in SRR1[44:45], then the exception is likely caused by an asynchronous machine check, in which case the exception handler should access the Asynchronous Machine Check Register through the SCOMC facility.

4.5.3 Data Storage Exception

The 970MP microprocessor implements the data storage exception as described in the PowerPC Architecture (OEA). A DSI exception occurs when no higher priority exception exists and an error condition related to a data memory access occurs. In case of a TLB miss for a load, store, or cache operation, a DSI exception is taken if the resulting hardware table search causes a page fault.

When this exception is taken, execution resumes at effective address x'00300'.

4.5.4 Data Segment Exception

The 970MP microprocessor implements the data segment exception as described in the PowerPC Architecture (OEA). A data segment exception occurs when no higher priority exception exists and a data access cannot be performed because data address translation is enabled (MSR[DR] is '1') and the effective address of any byte of the storage location specified by a Load, Store, Instruction Cache Block Invalidate (**icbi**), Data Cache Block Set to Zero (**dcbz**), Data Cache Block Store (**dcbst**), Data Cache Block Flush (**dcbf**), External Control In Word Indexed (**eciwx**), or External Control Out Word Indexed (**ecowx**) instruction cannot be translated to a virtual address.

When this exception is taken, execution resumes at effective address x'00380'.

4.5.5 Instruction Storage Exception

The 970MP microprocessor implements the instruction storage exception as described in the PowerPC Architecture (OEA). An instruction storage interrupt (ISI) exception occurs when no higher priority exception exists and an attempt to fetch the next instruction fails.

When this exception is taken, execution resumes at effective address x'00400'.

4.5.6 Instruction Segment Exception

The 970MP microprocessor implements the instruction segment exception as described in the PowerPC Architecture (OEA). An instruction segment exception occurs when no higher priority exception exists and next instruction to be executed cannot be fetched because instruction address translation is enabled (MSR[IR] is '1') and the effective address cannot be translated to a virtual address.

When this exception is taken, execution resumes at effective address x'00480'.

4.5.7 External Interrupt Exception

In the 970MP microprocessor, an external interrupt is signaled by the assertion of the external interrupt input signal. The external interrupt signal is expected to remain asserted until the processor has actually taken the interrupt (failure to meet this requirement might lead the processor to not recognize the interrupt request).

4.5.8 Alignment Exception

An alignment exception is taken if any of the following conditions are detected:

- **lwarx**, **stwcx**, Load Multiple Word (**lmw**), Store Multiple Word (**stmw**) instructions with non-word aligned addresses
- **ldarx** and **stdcx** instructions with non-double word aligned addresses
- **lmw** and **stmw** instructions to storage marked cache-inhibited
- Load String Word Immediate (**lswi**), Load String Word Indexed (**lswx**), Store Sting Word Immediate (**stswi**), and Store String Word Indexed (**stswx**) instructions to storage marked cache-inhibited
- **dcbz** to storage marked cache-inhibited (a **dcbz** to cache-inhibited space is treated as a no-op instead of causing an alignment interrupt if the `dcbz_ieq1_align` bit in the mode ring is set to a '0')
- Any load or store to storage marked cache-inhibited that is not naturally aligned
- Floating-point load single instructions that are not word aligned and cross a 32-byte boundary
- Floating-point store instructions that are not word aligned and cross a 4 KB boundary
- When `HID4[24]` is set, some forms of unaligned storage accesses that are normally handled by the hardware are forced to take an alignment exception (to assist in debugging).

Table 4-5. Register Settings for Alignment Exception

Register	Bits	Setting
DSISR	0:5	Unchanged.
	6	Set to '1' for a store or dcbz instruction; otherwise, set to '0'.
	7:31	Unchanged.
DAR	0:63	Set to the effective address computed by the load or store instruction that caused the alignment exception. When the exception is caused by an unsupported access to cache-inhibited space, the DAR will be set to the effective address of the first access into the cache-inhibited space.

4.5.9 Program Exception

The 970MP microprocessor implements the program exception as it is defined by the PowerPC Architecture (OEA). A program exception occurs when no higher priority exception exists and one or more of the exception conditions defined in the OEA occur.

The 970MP microprocessor invokes the program exception for a system illegal instruction when it detects any instruction from the illegal instruction class. The 970MP processing unit fully decodes the special purpose register (SPR) field of the instruction. If an undefined SPR is specified, a program exception is taken.

When this exception is taken, execution resumes at effective address `x'00700'`.

4.5.10 Floating-Point Unavailable Exception

The floating-point unavailable exception is implemented as defined in the PowerPC Architecture. When a floating-point unavailable exception is taken, instruction fetching resumes at the location determined by adding the offset x'00800' to the HIOR value.

4.5.11 Decrementer Exception

The decrementer exception is implemented as defined in the PowerPC Architecture. A decrementer exception occurs when no higher priority exception exists, the decrementer is negative (DEC[0] equals '1'), and MSR[EE] equals '1'. The decrementer exception is level sensitive. It is the responsibility of the interrupt service routine to clear DEC[0].

When this exception is taken, execution resumes at effective address x'0000_0000_0000_0900'.

4.5.12 System Call Exception

The 970MP microprocessor implements the system call exception as described in the PowerPC Architecture (OEA). A system call exception occurs when a system call (**sc**) instruction is executed.

When this exception is taken, execution resumes at effective address x'00C00'.

4.5.13 Trace Exception

The trace exception is taken when the single-step trace enable bit (MSR[SE]) or the branch trace enable bit (MSR[BE]) is set and an instruction successfully completes. After a trace exception is taken, SRR0, SRR1, Sampled Instruction Address Register (SIAR), and Sampled Data Address Register (SDAR) are set as shown in *Table 4-6*.

Table 4-6. Register Settings for Trace Exception

Register	Bits	Setting
SRR0	0:63	Set as specified in the architecture.
SRR1	0:32	Loaded from the MSR.
	33:34	'10'
	35	Set for a load instruction; otherwise, cleared. Not set for a zero-length lswx instruction.
	36	Set for a store instruction; otherwise, cleared. Not set for a zero-length stswx instruction.
	37:41	Loaded from the MSR.
	42	Set for a lwarx/ldarx or stwcx/stdcx instruction; otherwise, cleared.
	43	Set to '1'.
	44	Set to '0'.
	45:47	Set to '0'.
	48:63	Loaded from the MSR.
SIAR	0:63	Set to the effective address of the traced instruction.
SDAR	0:63	If the instruction that took the trace interrupt was a storage access instruction, the SDAR is set to the effective address of the storage access. SDAR is not set if an X-form Load String or Store String instruction specifies an operand length of zero.

IBM PowerPC 970MP RISC Microprocessor

If either MSR bits SE or BE is set to '1' by a Return from Interrupt or Move to MSR instruction, the contents of SIAR and SDAR are undefined until a trace interrupt occurs.

4.5.14 Performance Monitor Exception

The performance monitor exception is signalled when the MSR[EE] bit is set, and a performance monitor exception condition occurs. See *Chapter 10 970MP Performance Monitor* for a description of performance monitor exception conditions.

The following registers are set when a performance monitor exception occurs.

Table 4-7. Register Settings for the Performance Monitor Exception

Register	Bits	Setting
SRR0	0:63	Set to the effective address of the instruction that the processor would have attempted to execute next if no exception conditions were present.
SRR1	0:32	Loaded from the MSR.
	33	Set to '1' if the contents of the SDAR and the SIAR are associated with the same instruction.
	34:63	Loaded from the MSR.
SIAR	0:63	Set to the effective address of the marked instruction, where the marked instruction is an instruction that was executing, possibly out-of-order, at or around the time that the performance monitor exception occurred. The contents of the SIAR can be altered by the processor if and only if MMCR0[PMEE] equals '1'. Thus, after a performance monitor exception occurs, the contents of SIAR are not altered by the processor until software sets MMCR0[PMEE] to '1'. After software sets MMCR0[PMEE] to '1', the contents of SIAR are undefined until the next performance monitor exception occurs.
SDAR	0:63	Set to the effective address of the storage operand of an instruction that was executing, possibly out-of-order, at or around the time that the performance monitor exception occurred. This storage operand is called the marked data and might be, but need not be, the storage operand (if any) of the marked instruction. If the performance monitor exception causes a performance monitor interrupt, SRR1 indicates whether the marked data is in fact the storage operand of the marked instruction. The contents of the SDAR can be altered by the processor if and only if MMCR0[PMEE] equals '1'. Thus, after a performance monitor exception occurs, the contents of SDAR are not altered by the processor until software sets MMCR0[PMEE] to '1'. After software sets MMCR0[PMEE] to '1', the contents of SDAR are undefined until the next performance monitor exception occurs.

4.5.15 VPU Unavailable Exception

This exception occurs if there is an attempt to execute any vector instruction, including a vector load or store, with MSR[VP] negated. After this interrupt, execution resumes at offset x'0000_0000_0000_0F20'. The register settings for this interrupt are shown in *Table 4-8*.

Note: A **mtspr** or **mfspir** instruction that references the VRSAVE Register will not cause this interrupt.

Table 4-8. Register Settings for VPU Unavailable Interrupt

Register	Bits	Setting
SRR0	0:63	Set to the effective address of the instruction that caused the interrupt.
SRR1	0:32	Loaded from the MSR.
	33:36	Set to zeros.
	37:41	Loaded from the MSR.
	42:47	Set to zeros.
	48:63	Loaded from the MSR.

4.5.16 Instruction Address Breakpoint Exception

The 970MP microprocessor does not support a visible form of the instruction address breakpoint facility. The instruction address breakpoint feature is accessible through the support processor interface.

When this exception is taken, execution resumes at effective address x'01300'.

4.5.17 Maintenance Exception

The 970MP microprocessor provides support for an implementation-dependent maintenance exception. This exception can be signaled by a number of internal events, as well as by explicit commands from the support processor.

When this exception is taken, execution resumes at effective address x'0000_0000_0000_1600'.

This exception is controlled by the MSR[EE] bit in a manner similar to external interrupts. The register settings for this exception are shown in *Table 4-9* on page 117.

Table 4-9. Register Settings for Maintenance Exception

Register	Bits	Setting
SRR0	0:63	Set to the effective address of the next instruction that would have executed had the exception not been taken.
SRR1	0:32	Loaded from the MSR.
	33:36	Set to zeros.
	37:41	Loaded from the MSR.
	42:47	Set to zeros (can be used later to distinguish various causes of exception).
	48:63	Loaded from the MSR.

IBM PowerPC 970MP RISC Microprocessor

4.5.18 VPU Assist Exception

This exception occurs when operating in Java mode and the input operands or the result of an operation are denormalized.

When this exception is taken, execution resumes at offset x'0000_0000_0000_1700'.

The register settings for this exception are shown in *Table 4-10*.

Table 4-10. Register Settings for VPU Assist Exception

Register	Bits	Setting
SRR0	0:63	Set to the effective address of the instruction that caused the exception.
SRR1	0:32	Loaded from the MSR.
	33:36	Set to zeros.
	37:41	Loaded from the MSR.
	42:47	Set to zeros.
	48:63	Loaded from the MSR.

5. Memory Management

This chapter describes the 970MP implementation of the memory management unit (MMU) specifications provided by the operating environment architecture (OEA) for PowerPC processors. The primary function of the MMU in a PowerPC processor is the translation of logical (effective) addresses to physical addresses (referred to as real addresses in the architecture specification) for memory accesses and I/O accesses (I/O accesses are assumed to be memory-mapped). In addition, the MMU provides access protection on a segment or page basis. This chapter describes the specific hardware used to implement the MMU model of the OEA in each of the 970MP processing units. See the *PowerPC Operating Environment Architecture (Book III)* for a conceptual overview of the memory management model.

Two general types of memory accesses generated by PowerPC processors require address translation—instruction accesses and data accesses that are generated by load-and-store instructions. Generally, the address translation mechanism is defined in terms of the segment descriptors and page tables that the PowerPC processors use to locate the effective-to-physical address mapping for memory accesses. The segment information translates the effective address to an interim virtual address, and the page table information translates the interim virtual address to a physical address.

The segment descriptors, used to generate the interim virtual addresses, reside as segment table entries (STEs) in memory. Each 970MP processing unit uses a segment lookaside buffer (SLB) on-chip that caches recently used segment table entries. In addition, a translation lookaside buffer (TLB) is implemented on each 970MP processing unit to keep recently-used page address translations on-chip.

The MMU, together with the exception processing mechanism, provides the necessary support for the operating system to implement a paged virtual memory environment and to enforce protection of designated memory areas. Exception processing is described in *Chapter 4 Exceptions*. Specifically, *Section 4.3 Exception Processing* on page 105 describes the Machine State Register (MSR), which controls some of the critical functions of the MMUs.

5.1 MMU Overview

The 970MP microprocessor implements the memory management specification of the PowerPC operating environment architecture for 64-bit implementations. The 970MP microprocessor supports a 65-bit virtual address and a 42-bit physical (real) address.

Basic features of the MMU implementation in the 970MP processing unit as defined by the OEA are:

- Support for real addressing mode—Effective-to-physical address translation can be disabled separately for data and instruction accesses.
- Segmented address translation—The 64-bit effective address is translated to a 65-bit virtual address. This 65-bit virtual address space is divided into 4KB or 16MB pages, each of which can be mapped to a physical page.

The 970MP microprocessor also provides the following features that are not required by the PowerPC Architecture:

- Unified translation lookaside buffer (TLB)—The 1024-entry, 4-way, set-associative TLB supports:
 - A new large page architecture (16MB large pages supported).
 - Hardware-based reload (from the L2 cache interface in order to ensure no L1 D-cache impact).
 - Hardware-based update of the reference (R) and change (C) bits in a page table entry (PTE).
 - Parity protection; precise machine-check interrupt on parity error (software fix-up).

IBM PowerPC 970MP RISC Microprocessor

- Recently-used page address translations cached on-chip.
- Segment lookaside buffer (SLB)—The 64-entry, fully associative SLB supports:
 - Software reload of the SLB. An SLB miss results in an interrupt.
 - Loaded by the 32-bit PowerPC Segment Register instructions.
- TLB invalidation—The 970MP microprocessor implements the optional TLB Invalidate Entry (**tlbie**) and TLB Synchronize (**tlbsync**) instructions, which can be used to invalidate TLB entries. For more information about the **tlbie** and **tlbsync** instructions.
- **Little-endian mode is not supported.**

Table 5-1 summarizes the MMU features of the 970MP microprocessor, including those defined by the PowerPC Architecture (OEA) for 64-bit processors and those specific to the 970MP microprocessor.

Table 5-1. MMU Feature Summary

Feature Category	Architecturally Defined/ 970MP-Specific	Feature
Address ranges	Architecturally defined	2 ⁶⁴ bytes of effective address
	970MP-specific	2 ⁶⁵ bytes of virtual address
		2 ⁴² bytes of physical address
Page size	Architecturally defined	4 KB
	970MP-specific	16 MB
Segment size	Architecturally defined	256 MB
Memory protection	Architecturally defined	Segments selectable as no-execute
		Pages selectable as user or supervisor and read-only or guarded
Page history	Architecturally defined	Referenced and changed bits defined and maintained
Page address translation	Architecturally defined	Translations stored as PTEs in hashed page tables in memory
		Page table size determined by a mask in SDR1
TLB	Architecturally defined	Instructions for maintaining TLBs (tlbie and tlbsync instructions in the 970MP microprocessor)
	970MP-specific	1024-entry, 4-way, set-associative TLB (combined for both instruction and data).
Page table search support	970MP-specific	The 970MP microprocessor performs the table search operation in hardware.
Segment descriptors	Architecturally defined	Stored as STEs in hashed segment tables in memory
	970MP-specific	64-entry fully associative SLB
Segment table search support	970MP-specific	The 970MP microprocessor provides support for software reload of the SLB.

5.1.1 Speculative Storage Accesses

The 970MP processing unit is capable of speculatively executing load instructions to non-guarded, cacheable storage. This can occur when a load instruction is encountered on a predicted branch path, or when a logically preceding instruction causes an interrupt. As a result, it is possible for a speculative load that misses in the on-chip cache hierarchy to initiate an external storage request, even if that load instruction is not actually executed as part of the true instruction stream.

5.1.2 Storage Protection

When address translation is enabled, the protection mechanism is controlled by the following bits:

- MSR[PR], which distinguishes between supervisor (privileged) state and user (problem) state
- K_S and K_P , which are the supervisor (privileged) state and user (problem) state storage key bits in the SLB entry, used to translate the effective address
- For instruction fetches only:
 - the N (no-execute) value used for the access
 - the G (guarded) bit in the page table entry used to translate the effective address.

Thus, for an instruction fetch, access is not permitted if the N value is '1' or if G equals '1'.

5.1.3 Storage Access Modes

Storage access modes are controlled by the write-through/caching-inhibited/memory-coherency enforced/guarded bits (WIMG) bits. The 970MP microprocessor does not support the optional W bit or the optional M bit. All accesses are treated as though W equals '0' and M equals '1' independent of the value of these bits in the page table. Furthermore, when the hardware is performing a change bit update, it will write the W bit as '0' and the M bit as '1'.

Table 5-2 summarizes the treatment of the WIMG bits in the 970MP processing unit:

Table 5-2. Treatment of WIMG Bits in the 970MP Microprocessor

WIMG	Description
x1xx	Treated as WIMG equals '0111', for loads
	Treated as WIMG equals '011x', for stores
x0x1	Treated as WIMG equals '0011'
x0x0	Treated as WIMG equals '0010'

5.1.4 Support for 32-Bit Operating Systems

The 970MP microprocessor supports most of the optional bridge facilities and instructions for 64-bit implementations.

The bridge facility can be used to ease the transition to the PowerPC software-managed segment lookaside buffer (SLB) architecture, from either the Segment Register architecture provided by the 32-bit PowerPC implementation or the hardware-accessed segment table architecture provided by the 64-bit PowerPC implementations. The bridge facility permits the operating system to continue to use the 32-bit PowerPC implementation's Segment Register manipulation instructions and to continue to use the Address Space Register (ASR).

Associated with this support, the following optional instructions are supported:

- **mtsr** - Move to Segment Register
- **mtsrin** - Move to Segment Register Indirect
- **mfsr** - Move from Segment Register
- **mfsrin** - Move from Segment Register Indirect
- **mtmsr** - Move to Machine State Register (32-bit)

IBM PowerPC 970MP RISC Microprocessor

These instructions allow software to associate effective segments 0 through 15 with any of the virtual segments 0 through $2^{37}-1$. SLB entries 0 - 15 serve as virtual Segment Registers, with SLB entry i used to emulate Segment Register i . The **mtsr** and **mtsrin** instructions move 32 bits from a selected general purpose register (GPR) to a selected SLB entry. The **mfsr** and **mfsrin** instructions move 32 bits from a selected SLB entry to a selected GPR.

5.2 Real Addressing Mode

If address translation is disabled (MSR[IR] equals '0' or MSR[DR] equals '0') for a particular access, the effective address is treated as the physical address and is passed directly to the memory subsystem. These MSR bits are forced to '1' when running in user mode.

The WIMG bits for storage access in real addressing mode are determined as follows. The W and M bits are not supported in the 970MP microprocessor, and are considered to always have values of W equals '0' and M equals '1'. The G bit is always asserted in real addressing mode. For data accesses, bit 23 of Hardware Implementation-Dependent Register 4 (HID4[23]) determines the value of the I bit in real addressing mode. For instruction accesses, HID1[10] can be used to force the value of the I bit to '1', although this value applies to address translation mode as well as to real addressing mode.

6. Software Optimization Guidelines

This section highlights some 970MP microprocessor characteristics and conditions that should be considered when developing software.

6.1 Design Characteristics

The 970MP microprocessor has long pipelines with the following characteristics:

- There are six cycles from the instruction fetch to dispatch (dispatch is the sixth cycle).
- Complex instructions are broken down into sequences of simple internal operations.
- Some instructions stall in dispatch until certain interlocks are released.
 - The primary interlock is called the “non-rename scoreboard” bit.
 - Only one scoreboard bit exists for all scoreboarded resources.
 - Instructions that write a non-renamed resource set the non-rename scoreboard bit when dispatched and reset this bit when complete.
 - All SPRs are scoreboarded except: LR, CTR, and the following bits in XER: CA, OV.
 - Instructions that use or read from the non-renamed registers stall in the dispatch unit until the flag clears.
- Instructions that set the scoreboard also typically end a dispatch group and are completion serialized (wait until next-to-complete before eligible for execution).
- Dispatch receives groups, which are a unit of tracking.
 - Up to 20 groups active after dispatch (80 - 100 PowerPC instructions).
 - Four to seven cycles from dispatch to finish.

The 970MP microprocessor has multiple execution units:

- Two load/store units (LSU)
- Two floating-point units (FPU)
- Two fixed-point units (FXU) (that are symmetric except that FX1 does divides and FX0 does SPR access)
- One branch unit (BRU)
- One condition register unit (CRU)

The 970MP microprocessor utilizes out-of-order execution:

- Execution is in-order until dispatch has placed instructions into issue queues.
- Instructions issued from queues to execution units are out-of-order.
- Instructions complete in order.

IBM PowerPC 970MP RISC Microprocessor

The 970MP microprocessor has the following load/store unit characteristics:

- Complicated loads and stores are broken up by decode unit.
 - **lmw** and **stmw** are converted to a stream of single-register loads and stores. String instructions generate a similar stream, except that X-form string instructions cause generation of internal operations to read the byte count field from the XER, causing a dispatch stall if the XER setting instruction has not executed.
- Problems are handled by flush, refetch group, or dispatch as single-instruction groups.
- Loads that are dependent on a store in the same group cause a flush if forwarding is not possible. This is because the load must wait until the store has updated the cache, but the cache update must be non-speculative and can only be done after the store completes. Completion is done on a group basis, and can only be done when all internal operations (IOPs) in the group have finished. Therefore, the entire group is flushed. When decoded, the load is forced into a separate group.
- Any load with data that crosses a 64-byte boundary (32-byte boundary if a load misses in the L1 cache) causes flush and microcode expansion. If the offending load is an IOP generated by the microcode expansion of a string instruction, the entire PowerPC instruction is flushed and re-expanded such that each register's data is processed by two loads/stores and a merge.
- Loads dependent upon a store, but executed early (load executes before store), cause a flush.
- Flush and refetch costs about 20 cycles. Misaligned loads usually are flushed twice; once to get the load isolated in a dispatch group, and the second time to generate the microcoded sequence of IOPs to fetch the data and splice it together.
- The data prefetch engine can prefetch eight active streams.

The 970MP microprocessor uses the following memory hierarchy for data:

- The L1 data cache is a 32 Kb, 128-byte line with a 2-cycle latency.
 - The L1 D-cache is store-through.
 - A store miss in the L1 data cache does not establish a line in the L1 D-cache.
 - Cache reloads are 32 bytes per cycle.
- The L2 cache is a 1 Mb, 128-byte line.

The 970MP microprocessor decode unit has the following features:

- Processes a stream of PowerPC instructions and forms dispatch groups.
 - Branches always force an end of current group.
 - Some instructions are forced to be first in a group. For example: **divw**, CR logical.
- Cracking generates two IOPs from one PowerPC instruction. For example:
 - All update forms (load/store + add(i) to update register)
 - X-form fixed-point stores (add + store)
 - Load algebraic (load + extend sign)
 - Many record forms (basic arithmetic + compare immediate)
 - Fixed-point divides
 - All CR-logicals except destructive forms (**rD = rB**)

- Both IOPs of the cracked instruction must be in the same group. This forces the cracked pair to start a new group if the original instruction was last in the previous group (and there was no room for the second IOP).
- Microcode: generate three or more IOPs from a single PowerPC instruction.
 - Microcoded instructions generate one or more groups, thus forcing an end of the previous group. For example:
 - **lmw** and **lswi** (all multiples and string instructions)
 - **mtcrf** (more than one target field)
 - **mtxer** and **mfxer**
- Some instructions are forced to be first in a group. For example:
 - Fixed-point divide (also cracked)
 - **addc/subfc** (also cracked)
 - **mtspr/mfspr** (to satisfy FX0/LSU0/CRU execution requirement)
 - CR-logicals (can also be cracked)

The instruction fetch unit (IFU) has the following characteristics:

- Fetches are aligned on 8-word blocks
- It takes three cycles to redirect a fetch from Next-Sequential. For example, there are two dead cycles between the last fetch of a block containing a branch and fetching the branch target.
- The fetcher cannot handle a new fetch block until all branches in the current block have been recorded in the branch instruction queue (BIQ) for future resolution. Only branches between the branch target address and the end of the block are significant. These branches are recorded two per cycle, so the maximum time required is four cycles.

Branch prediction has the following characteristics:

- Predicts both direction (conditional) and address (to Link or Count).
- Highly accurate (95%) for most codes.
- Accuracy can be improved with hint bits.
- About 11 cycles are needed to correct a wrong guess.
- Replacing conditional branches with alternative code is likely to be a win (some fixed-point maximum, minimum, select).

Dispatch, issue, and issue queues have the following characteristics:

- Dispatch performs register renaming (mapping), scoreboard dependency checking, and distribution to correct the issue queue.
- Six instruction queues
 - FPQ0 (10 IOPs) feeding FPU0
 - FPQ1 (10 IOPs) feeding FPU1
 - FXQ0 (18 IOPs) feeding FXU0 and LSU0
 - FXQ1 (18 IOPs) feeding FXU1 and LSU1
 - BRQ (12 IOPs)
 - CRQ (10 IOPs)

IBM PowerPC 970MP RISC Microprocessor

- There is a fixed relationship between the dispatch group slot and the target instruction queue.
 - Slot 0: FPQ0, FXQ0, CRQ
 - Slot 1: FPQ1, FXQ1, CRQ
 - Slot 2: FPQ1, FXQ1
 - Slot 3: FPQ0, FXQ0
 - Slot 4: BRQ
- In addition, the FX, FP, and CR queues are subdivided into even and odd subqueues. The attached execution units can obtain IOPs from either subqueue, but IOPs always stay in the subqueue to which they were initially dispatched. Each subqueue has half the total capacity of the queue. Thus:
 - Slot 0: FXQ0-O or FPQ0-O or CRQ-O
 - Slot 1: FXQ1-O or FPQ1-O or CRQ-E
 - Slot 2: FXQ1-E or FPQ1-E
 - Slot 3: FXQ0-E or FPQ0-E
- IOPs are issued from the queues when all operands are ready, and there is an execution unit available; IOPs can be issued the next cycle after dispatch.
- Dependent IOPs cannot be issued back-to-back. That is, dependent instructions can be issued only every other cycle (assuming that they execute in one cycle)
- IOPs can be artificially serialized by being dispatched to the same FX queue. Thus, suboptimal scheduling might cause underutilization of one of two symmetric execution units.

6.2 Software Considerations for the 970MP Microprocessor

Software for the 970MP microprocessor needs to consider the following conditions:

- XER has non-renamed fields.
- X-form string instructions are slowed down; therefore, it is best to avoid these instructions.
- **mtxer** drains the functional units.
- SPRs are not renamed except for CTR, LR, and some XER fields. Referencing non-renamed SPRs causes pipeline drain.
- There is a scoreboard interlock between an **mtspr** and the next subsequent **mfspir** such that the **mfspir** is held in the dispatch until the scoreboard goes off (when the last **mtspr** completes).
- The **mtsr** instruction is not recommended, because it is scoreboarded and forces execution serialization.
- The L1 data cache is write-through, and stores the miss in the L1 cache that does not establish the line in the L1 cache, but establishes only the line in the L2 cache.
- Loads dependent upon previous stores can be slow, and can trigger a flush and refetch. They should be scheduled, so that they are dispatched in separate groups.
- Store forwarding: If the store data is in the store-reorder queue (SRQ), then the data can be forwarded to the load (as if the load hit the L1 cache).

This is possible only when the data loaded is completely contained in the data from the store.
For example:

- **lw** following an **stw** to the same address
- **lh** following an **stw** to the same address

- **lbz** from any byte in the word stored by an **stw**
- **lw** from one of the words stored by an **stfd**

If the bytes loaded overlap the bytes stored, then no forwarding can be done, and the load appears to stall until the store data has been written to the cache. For example:

- **lfd** following an **stw**
- **lw** following an **sth** to the same address

If the store and load are in the same dispatch group, then a flush and refetch is done so that they will be in different groups to permit completion of the store.

If the load executes before the store address is computed, a flush and refetch occurs. The first re-executed instruction is the “load/next” after the store. To prevent this, schedule the dependent load four instructions (or more) after the store.

- Because instructions are tracked internally in groups, dependent instructions must be arranged so that they are in separate groups. This minimizes the length of time the individual instructions are in the execution section of the machine.
- Use instructions that minimize cracking or microcode expansion. This maximizes utilization of the dispatch buffer. For example:
 - Use update forms, which are always cracked, if the cracked pair does not cause early group termination. Using update forms helps to reduce the code footprint in the instruction cache.
 - Do not use X-form fixed-point stores (always cracked and sometimes microcoded)
- The granularity of reservations (**lwarx/stwcx.**) is the data cache line, which is 128 bytes.
 - Any store by another processor to the same cache line causes the reservation to be lost.
 - Atomically updated variables should be carefully placed, because the atomic-update sequences treat the variable as a reservation cell.
 - Lock cells and atomically updated variables must be the sole occupant of a cache line. Read-only data in same line is refetched from other L2 if any datum has been modified.
- Instructions are fetched from the I-cache in aligned 8-word blocks.
 - Branch targets must be aligned on 8-word (32-byte) boundaries, where feasible. At a minimum, they must be aligned on a 4-word (16-byte) boundary, to maximize fetch and decode efficiency.
- Use **mfspr(sprg0)** as a high performance method to validate privileged mode.



7. Signal Description

This chapter describes the external signals of the 970MP microprocessor. It contains a concise description of individual signals, showing behavior when the signal is asserted and negated and when the signal is an input and an output.

Note: A bar over a signal name indicates that the signal is active low. For example, $\overline{\text{CHKSTOP}}$ (checkstop in/out) and $\overline{\text{BYPASS}}$ (PLL bypass). Active-low signals are referred to as asserted (active) when they are low and negated when they are high. Signals that are not active low, such as ADIN[0:43] (address bus signals) are referred to as asserted when they are high and negated when they are low.

The 970MP microprocessor signals are grouped as follows:

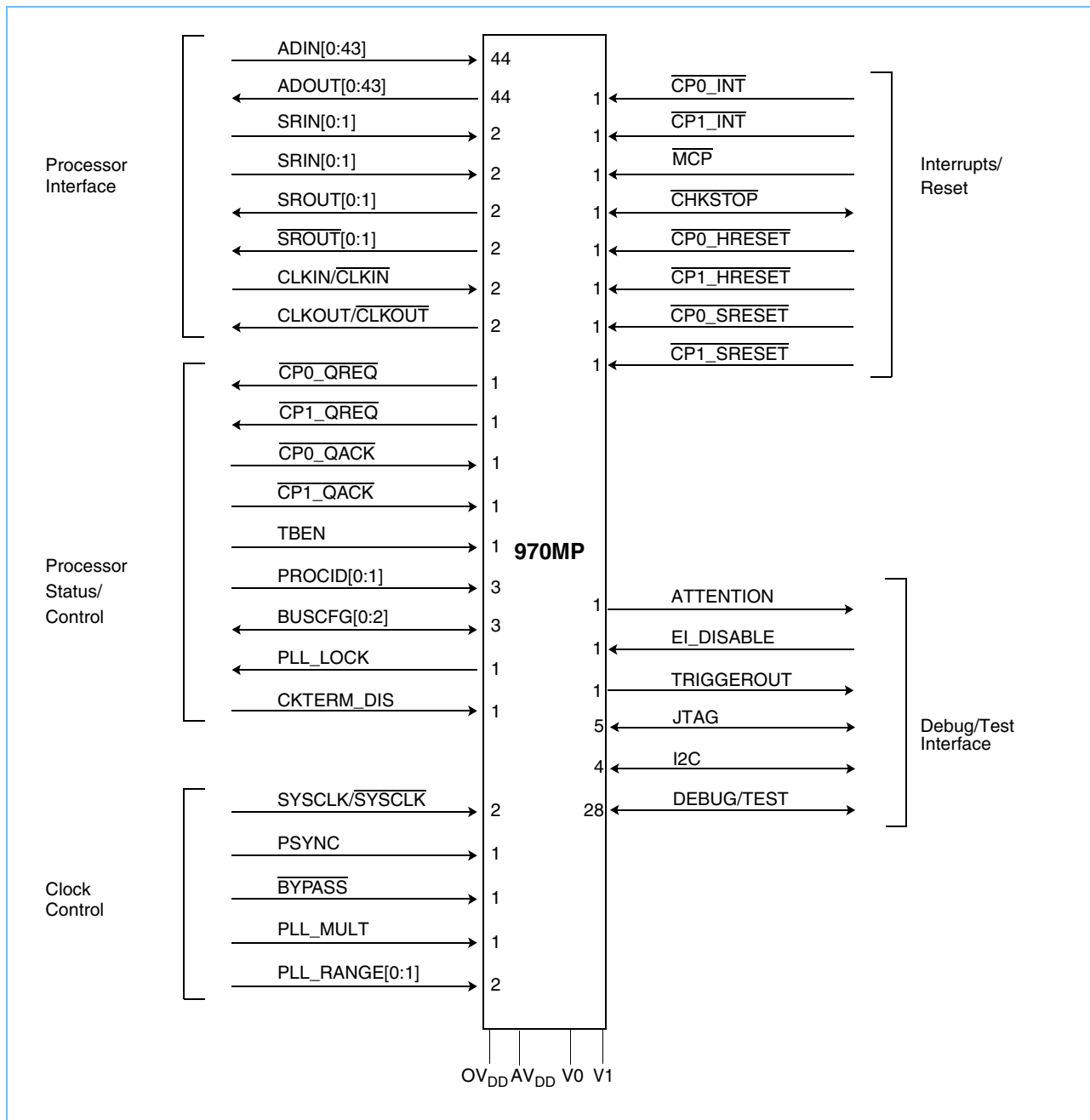
- Processor interface—These signals are used to transfer address, data, and control information between the 970MP microprocessor and a companion chip to provide coherent access to memory and access to memory-mapped I/O.
- Processor status and control—These signals are used to monitor and provide external control of various processor facilities, including the external bus and power management.
- Clock control—These signals determine the system clock frequency. They can also be used to synchronize multiprocessor systems.
- Interrupts/resets—These signals include the external interrupt signal, checkstop¹ signals, and both soft reset and hard reset signals. They are used to interrupt and to reset the processor under various conditions.
- Debug/test interface—The debug/test interface provides a serial interface to the system for performing debug, bring-up, and manufacturing tests. The JTAG (IEEE 1149.1a-1993) interface and the inter-integrated circuit (I²C) interface provide a serial interface to the system for performing board-level boundary-scan interconnect tests.

1. Hardware has detected a condition that it cannot resolve and which prevents normal operation. It stops executing instructions, responding to interrupts, and so on.

IBM PowerPC 970MP RISC Microprocessor
7.1 Signal Configuration

Figure 7-1 illustrates the configuration of the 970MP microprocessor signals, showing how the signals are grouped. A pinout showing pin numbers is included in the *IBM PowerPC 970MP RISC Microprocessor Datasheet*.

Figure 7-1. 970MP Microprocessor Signal Groups



7.2 Signal Descriptions

This section describes individual signals on the 970MP microprocessor, which are grouped as shown in *Figure 7-1 970MP Microprocessor Signal Groups* on page 130. In the following section, “cycle” or “clock” refers to a single bus clock period, which can correspond to one or more internal processor clocks depending on the clock mode programmed for the 970MP microprocessor.

Note: In PLL-bypass mode, the SYSCLK input signal clocks the internal processor directly, the PLL is disabled, and the bus mode is set to whatever bus mode is selected. This mode is intended for factory use only.

7.2.1 Processor Interface

The processor interface provides a high-speed, source-synchronous, point-to-point connection between the 970MP microprocessor and a companion chip. It consists of two unidirectional sets of signals, one to carry outgoing information from the 970MP microprocessor, the other to carry incoming information to the 970MP microprocessor. Each of these two sets of signals consists of a 44-bit bus to transfer logical data with redundancy, a differential clock (two signals), and a 2-bit differential snoop response (four signals).

Chapter 8 provides detailed information about the format and timing of these signals as they are used in the processor interconnect protocol implemented in the 970MP microprocessor.

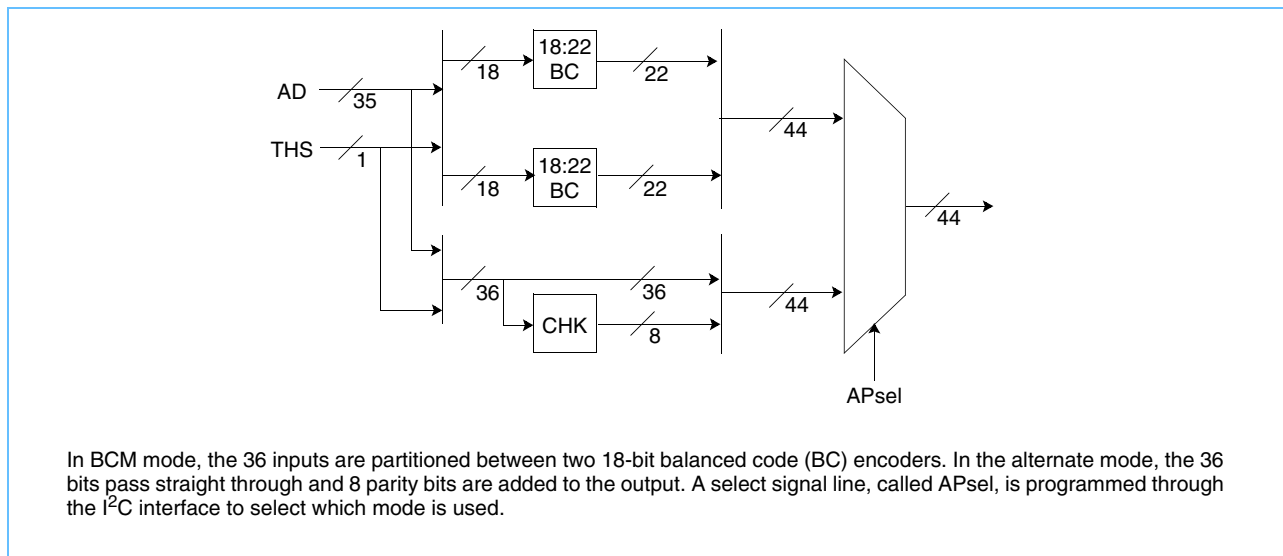
7.2.1.1 Address/Data In (ADIN[0:43])—Input

The address/data input signals carry address, data, and control information from the companion chip to the 970MP microprocessor. The 44 bits of ADIN carry 36 bits of address/data (AD) and transfer-handshake (TH) information plus 8 bits of redundancy.

There are two defined formats for encoding the 36 AD and TH signal lines onto the 44 source-synchronous bus (SSB) signal lines (see *Figure 7-2 Encoding and Selection Logic for the Drive Side of a 970MP Interconnect SSB* on page 132). The first format exploits a balanced coding method (BCM) to maintain an equal number of zeros and ones on the signal lines. During any valid state of the bus, exactly 22 of the signals lines are high and 22 are low. The BCM advantage is that it dramatically improves the signal-to-noise robustness of the bus for high-speed operation at the cost of a few extra signal lines. The BCM can inherently detect a single bit error from any of the 44 signal lines.

The second mode uses 36 of the 44 SSB signal lines for the data transfer. The remaining eight SSB signal lines are used to encode an 8-bit parity value that has sufficient redundancy to detect up to two bit errors across any of the 44 SSB signal lines and correctly identify the bit position of any single bit error.

Figure 7-2. Encoding and Selection Logic for the Drive Side of a 970MP Interconnect SSB



Timing: The processor interface is source synchronous, meaning that the same clock that launches data on the sending end is transferred with the data and used at the receiving end to capture the data. The interface is run in double data rate (DDR) fashion, with a data transfer on every rising and falling edge of the clock. Because there is no arbitration on this interface, valid data can be transferred on any clock edge. ADIN uses CLKIN as its reference.

7.2.1.2 Snoo Response In ($SRIN[0:1]$, $\overline{SRIN}[0:1]$)—Input

The snoop-response input signals carry a 2-bit code from the companion chip to the 970MP microprocessor, indicating the coherency response of the system to an earlier command sent on the ADOUT bus. $SRIN$ and $\overline{SRIN}$ represent a differential pair, such that $SRIN$ carries the snoop response in an asserted high signal level at the same time that $\overline{SRIN}$ carries the same snoop response in an asserted low signal level.

Timing: Same as ADIN.

7.2.1.3 Clock In ($CLKIN/\overline{CLKIN}$)—Input

The CLKIN signal originates in the companion chip and is sent synchronously with the data ($ADIN$ and $SRIN$) for use in data capture at the receivers in the 970MP microprocessor. This clock is transmitted as a differential pair.

Timing: The clock in signal is derived from the on-chip PLL on the companion chip and synchronized to the $psync$ signal, which provides a periodic global reference event. During the initial alignment procedure (IAP) for the processor interface, a rising edge of the clock in signal is identified as corresponding to time zero. Every other rising edge thereafter is a time zero, delimiting the basic unit of time on the bus, in which four beats of data can be transferred.

7.2.1.4 Address Data Out (ADOUT[0:43])—Output

The address/data output signals carry address, data, and control information from the 970MP microprocessor to the companion chip. The 44 bits of ADOUT carry 36 bits of address/data (AD) and transfer-handshake (TH) information plus 8 bits of redundancy, similarly to ADIN.

Timing: Same as ADIN, except that ADOUT uses CLKOUT as its reference.

7.2.1.5 Snoop Response Out (SROUT[0:1], $\overline{\text{SROUT}}[0:1]$)—Output

The snoop-response output signals carry a 2-bit code from the 970MP microprocessor to the companion chip, indicating the coherency response of the processor to an earlier reflected command sent on the ADIN bus. SROUT and $\overline{\text{SROUT}}$ represent a differential pair, such that SROUT carries the snoop response in an asserted high signal level at the same time that $\overline{\text{SROUT}}$ carries the same snoop response in an asserted low signal level.

Timing: Same as ADOUT.

7.2.1.6 Clock Out (CLKOUT/ $\overline{\text{CLKOUT}}$)—Output

The clock out signal originates in the 970MP microprocessor and is sent synchronously with the data (ADOUT and SROUT) for use in data capture at the receivers in the companion chip. This clock is transmitted as a differential pair.

Timing: The clock out signal is derived from the on-chip PLL on the 970MP microprocessor and synchronized to the *psync* signal, which provides a periodic global reference event. During the IAP for the processor interface, a rising edge of the clock out signal is identified as corresponding to time zero. Every other rising edge thereafter is a time zero, delimiting the basic unit of time on the bus, in which four beats of data can be transferred.

7.2.2 Processor Status and Control

7.2.2.1 Quiescent Request ($\overline{\text{CP0_QREQ}}$ and $\overline{\text{CP1_QREQ}}$)—Output

The $\overline{\text{CP0_QREQ}}$ and $\overline{\text{CP1_QREQ}}$ signals, along with $\overline{\text{CP0_QACK}}$ and $\overline{\text{CP1_QACK}}$, are used for power management on the 970MP microprocessor. The $\overline{\text{QREQ}}$ signals have two distinct uses. When a frequency shift procedure in the power tuning facility is not in progress, assertion of $\overline{\text{CP0_QREQ}}$ for PU0 ($\overline{\text{CP1_QREQ}}$ for PU1) indicates that the 970MP processing unit has entered Doze mode, and is prepared to go into Nap (or Deep Nap) mode. This signal remains asserted until the 970MP processing unit returns to Run mode.

When a frequency shift procedure in the power tuning facility is in progress, assertion of $\overline{\text{CP0_QREQ}}$ for PU0 ($\overline{\text{CP1_QREQ}}$ for PU1) indicates that the 970MP processing unit is prepared to perform the frequency shift itself. This signal remains asserted until the 970MP processing unit has completed the frequency shift procedure. See *Chapter 9* for more information about frequency shifting in the power tuning facility.

Timing: The $\overline{\text{CP0_QREQ}}$ and $\overline{\text{CP1_QREQ}}$ signals can be asserted or negated by the processor at any time.

7.2.2.2 Quiescent Acknowledgment ($\overline{CP0_QACK}$ and $\overline{CP1_QACK}$)—Input

The $\overline{CP0_QACK}$ and $\overline{CP1_QACK}$ signals, along with $\overline{CP0_QREQ}$ and $\overline{CP1_QREQ}$, are used for power management on the 970MP microprocessor. The $\overline{QACK}$ signals have two distinct uses. When a frequency shift procedure in the power tuning facility is not in progress, assertion of $\overline{CP0_QACK}$ for PU0 ($\overline{CP1_QACK}$ for PU1) indicates that all bus activity that requires snooping has stopped, and that the 970MP processing unit can enter Nap (or Deep Nap) mode. This signal must be negated whenever bus activity requiring snooping is resumed, or the 970MP processing unit negates $\overline{QREQ}$.

When a frequency shift procedure in the power tuning facility is in progress, assertion of $\overline{CP0_QACK}$ and $\overline{CP1_QACK}$ indicates that the rest of the system is prepared to perform the frequency shift itself. This signal remains asserted until the companion chip has completed the frequency shift procedure. See *Chapter 9* for more information about frequency shifting in the power tuning facility.

Timing: The $\overline{CP0_QACK}$ signal for PU0 ($\overline{CP1_QACK}$ signal for PU1) is asserted in response to assertion of the $\overline{CP0_QREQ}$ signal by PU0 ($\overline{CP1_QREQ}$ signal by PU1). It can be asserted any time $\overline{QREQ}$ is asserted, and can be negated at any time.

7.2.2.3 Time-Base Enable (TBEN)—Input

The TBEN input signal can be used in one of two ways, as determined by the value of HID0[19]. When HID0[19] equals '0', the Time-Base Register is incremented and the Decrementer Register is decremented at 1/16th of the full processor frequency whenever TBEN is asserted. These two timer registers maintain their value when TBEN is negated in this mode.

When HID0[19] equals '1', the Time-Base Register is incremented and the Decrementer Register decremented on every rising edge of the TBEN input signal. In this externally clocked mode, the TBEN frequency must not exceed 1/16th the full processor frequency in order to guarantee sufficient sampling of this external signal.

Timing: The TBEN input is asynchronous to the SYSCLK and processor clocks, and can change at any time, subject to the previously stated frequency restriction.

7.2.2.4 Processor ID (PROCID[0:1])—Input

The 2-bit processor ID is used to assign unique IDs to the two 970MP processing units in a system that can have up to eight processors. The PROCID signals are sampled during power-on reset, and the 2-bit value is placed in the second and third lowest-order bits of the Processor ID Register (PIR) of each processing unit. The lowest-order PIR bit is hardwired to a '0' for PU0 and to '1' for PU1.

Timing: These signals should be permanently tied to V_{DD} or GND, as appropriate for the required ID value.

7.2.2.5 Bus Configuration Select (BUSCFG[0:2])—Input

The 3-bit BUSCFG input encodes the processor clock to bus clock ratio. It is used to select the appropriate clock dividers in the 970MP microprocessor in order to generate the required bus clock frequency. Note that not all encodes work with the power tuning facility (see *Chapter 9* for more information). The interpretation of the BUSCFG values can be found in the *IBM PowerPC 970MP RISC Microprocessor Datasheet*.

Timing: These signals should be permanently tied to V_{DD} or GND, as appropriate for the required bus configuration value.

7.2.2.6 PLL Locked (PLL_LOCK)–Output

The PLL_LOCK signal is asserted when the PLL has achieved lock, or when running in bypass mode. The signal is negated otherwise.

Timing: The PLL_LOCK signal can change at any time. The initial maximum latency for the PLL to achieve lock is specified in the *IBM PowerPC 970MP RISC Microprocessor Datasheet*.

7.2.2.7 Clock Receiver Termination (CKTERM_DIS)–Input

The CKTERM_DIS signal allows the internal termination on the SYSCLK and $\overline{\text{SYSCLK}}$ signals to be disabled. When CKTERM_DIS is negated, the clock in signals are terminated. When the CKTERM_DIS signal is asserted, the termination of the clock in signals is removed from the receiver circuit.

Timing: This signal should be permanently tied to V_{DD} or GND, as appropriate for the required clock configuration.

7.2.3 Clock Control

7.2.3.1 System Clock (SYSCLK/ $\overline{\text{SYSCLK}}$)–Input

The SYSCLK inputs provide the reference clock from which the on-chip PLL develops the processor mesh clock, as well as the bus clock. The system clock is provided to the processor as a differential pair. The mesh clock frequency is determined by this reference clock and the value of the PLL_MULT input. The bus clock frequency is determined by the mesh clock frequency and the value of the BUSCFG input. See the *IBM PowerPC 970MP RISC Microprocessor Datasheet* for the correspondence between these inputs and the clock frequency ratios.

Timing: See the *IBM PowerPC 970MP RISC Microprocessor Datasheet* for clock specifications.

7.2.3.2 Phase Synchronization (psync)–Input

The *psync* signal provides a synchronization pulse to all processors and companion chips in the system, providing the basis for identifying a periodic time zero event in each chip.

Timing: See the *IBM PowerPC 970MP RISC Microprocessor Datasheet* for clock specifications.

7.2.3.3 PLL Bypass ($\overline{\text{BYPASS}}$)–Input

The $\overline{\text{BYPASS}}$ signal indicates to the processor that the system clock input should be fed directly to the PLL output, bypassing the PLL. This mode of clocking the processor can be used for debugging.

Timing: To bypass during debug, this signal should be tied to GND.

IBM PowerPC 970MP RISC Microprocessor

7.2.3.4 PLL Multiplier (PLL_MULT)–Input

The PLL_MULT signal is used to specify the ratio of the full processor mesh frequency to the system clock frequency. See the *IBM PowerPC 970MP RISC Microprocessor Datasheet* for the correspondence between the value of this signal and the clock ratio.

Timing: This signal should be permanently tied to V_{DD} or GND, as appropriate to the required clock configuration.

7.2.3.5 PLL Range Select (PLL_RANGE[0:1])–Input

The PLL_RANGE signal is used to identify the required frequency range of the processor mesh clock. See the *IBM PowerPC 970MP RISC Microprocessor Datasheet* for the correspondence between the value of this signal and the required frequency range.

Timing: This signal should be permanently tied to V_{DD} or GND, as appropriate to the required clock configuration.

7.2.4 Interrupts and Resets

Most system status signals are input signals that indicate when exceptions are received, when checkstop conditions have occurred, and when the 970MP microprocessor must be reset.

7.2.4.1 Interrupt ($\overline{CP0_INT}$ and $\overline{CP1_INT}$)–Input

The $\overline{CP0_INT}$ and $\overline{CP1_INT}$ signals provide a means for raising an external interrupt. This exception can be masked by the MSR[EE] bit. When MSR[EE] equals '0', the processing unit will not respond to the assertion of $\overline{INT}$.

7.2.4.2 Machine Check Interrupt ($\overline{MCP}$)–Input

The $\overline{MCP}$ signal provides a means for raising a machine check exception. This exception can be masked by two control bits. If HID0[32] equals '0', the assertion of $\overline{MCP}$ is ignored. If HID0[32] equals '1', and MSR[ME] equals '1', machine checks are enabled, and the assertion of $\overline{MCP}$ will result in a machine check exception being taken. If HID0[32] equals '1', and MSR[ME] equals '0', machine checks are disabled, and the assertion of $\overline{MCP}$ will cause the processor to enter the checkstop state.

Timing: This signal can be asserted at any time, asynchronously to the system clock. Once asserted, the $\overline{MCP}$ signal must remain asserted for at least two bus clock cycles to ensure that it is recognized.

7.2.4.3 Checkstop ($\overline{CHKSTOP}$) –Bidirectional

The checkstop signal is both an input and an output signal on the 970MP microprocessor.

Checkstop ($\overline{CHKSTOP}$) –Input

The checkstop input signal provides a means for external initiation of a checkstop.

Timing: This signal can be asserted at any time, asynchronously to the system clock.

Checkstop ($\overline{CHKSTOP}$) –Output

The checkstop output signal indicates that the processor has entered the checkstop state.

Timing: This signal can be asserted at any time.

7.2.4.4 Hard Reset ($\overline{CP0_HRESET}$ and $\overline{CP1_HRESET}$)–Input

The $\overline{CP0_HRESET}$ signal provides a means for resetting PU0 and initiating the power-on-reset sequence for PU0. The $\overline{CP1_HRESET}$ signal provides a means for resetting PU1 and initiating the power-on-reset sequence for PU1.

Timing: This signal can be asserted at any time, asynchronously to the system clock.

7.2.4.5 Soft Reset ($\overline{CP0_SRESET}$ and $\overline{CP1_SRESET}$)–Input

The $\overline{CP0_SRESET}$ and $\overline{CP1_SRESET}$ signals provide a means for external initiation of the soft (or warm) reset. When $\overline{CP0_SRESET}$ is asserted, the PU0 responds by taking a system reset exception. When $\overline{CP1_SRESET}$ is asserted, the PU1 responds by taking a system reset exception.

Timing: This signal can be asserted at any time, asynchronously to the system clock.

7.2.5 Debug/Test Interface

7.2.5.1 Attention ($\overline{ATTENTION}$)–Output

ATTENTION is an output signal from the 970MP microprocessor to the JTAG debugger, used in debug mode. I²C SCOM commands are sent directly to PSCOM and do not go through the JTAG TAP engine. Therefore, when Attention is active, a SCOM read/write command will not be acknowledged with the standard I²C acknowledgment (ACK) pulse because it is not a primitive test access port (TAP) command.¹

7.2.5.2 Processor Interface Disable ($\overline{EI_DISABLE}$)–Input

Turns off elasticity in the processor interface bus.

7.2.5.3 Trigger Out ($\overline{TRIGGEROUT}$)–Output

TRIGGEROUT is an output signal used to indicate that internal trace collection has begun.

7.2.5.4 JTAG Signals

The IEEE 1149.1 defines a five-wire interface called a test access port (TAP) for communicating with the boundary scan architecture. The five JTAG signals are: TDI, TDO, TMS, TCK, and TRST.

1. Primitive TAP commands are those that scan the IR or DR in the JTAG engine.

IBM PowerPC 970MP RISC Microprocessor

Test Clock (TCK)—Input

TCK is a JTAG test clock, which is separate from the system mesh clock. The TCK only controls the test access port functions (20 or 30 latches). SYSCLK must always be active to control the interfaces. The rising edge causes TMS and TDI to be sampled by the Access macro.

Test Data In (TDI)—Input

TDI is a JTAG serial input used to feed test data and test access port instructions.

Test Data Out (TDO)—Output

TDO is a JTAG serial output used to extract data from the chip under test control.

Test Mode Select (TMS)—Input

TMS is a JTAG select signal used to control the operation of the JTAG state machine. The value of TMS during a rising edge of TCK causes a state transition in the TAP controller.

Test Logic Reset ($\overline{\text{TRST}}$)—Input

$\overline{\text{TRST}}$ is an asynchronous JTAG signal used to reset the JTAG state machine. The $\overline{\text{TRST}}$ signal ensures that the JTAG logic does not interfere with the normal operation of the chip. The $\overline{\text{HRESET}}$ signal performs the function of $\overline{\text{TRST}}$ internally.

7.2.5.5 I²C Signals

The 970MP I²C bus conforms to the standard-mode timing specification and does not support high-speed or fast-mode timing. The 970MP microprocessor has the following I²C signals:

- I²C Signal Clock ($\overline{\text{I2CCK}}$)—I²C signal clock is both an input and output signal pin.
- I²C Interface Data ($\overline{\text{I2CDT}}$)—I²C interface data is both an input and output signal pin.
- I²C Interface Go (I2CGO)—I2CGO is an asynchronous, open-drain output signal used to prevent access collisions between JTAG and I²C. If the level of the interface is low, only JTAG should access the 970MP. I²C can use the interface if the level is high.
- I²C Select (I2CSEL)—I2CSEL controls the use of the mutually exclusive I²C or JTAG bus. When asserted, the I²C bus can be used. Otherwise, the JTAG bus can be used.

7.2.6 Voltage and Ground

The 970MP microprocessor provides the following connections for power and ground:

- OV_{DD}—The OV_{DD} signal provides the supply voltage connection for the drivers and receivers.
- AV_{DD}—AV_{DD} is a power signal that drives the analog sections of the PLL. See the *IBM PowerPC 970MP RISC Microprocessor Datasheet* for information about how to use this signal.
- V0—The V0 (V_{DD}) signal provides the supply voltage connection for processor core 0 and the common logic.
- V1—The V1 (V_{DD}) signal provides the supply voltage connection for processor core 1.

8. Processor Interconnect Bus

The IBM PowerPC 970MP RISC Microprocessor Processor Interconnect is a bus architecture providing high-speed, high-performance interconnections for processors, I/O devices, memory subsystems, and bridge chips. This bus architecture provides a forward-looking, general use, yet cost-effective solution for designing high-performance IBM PowerPC systems.

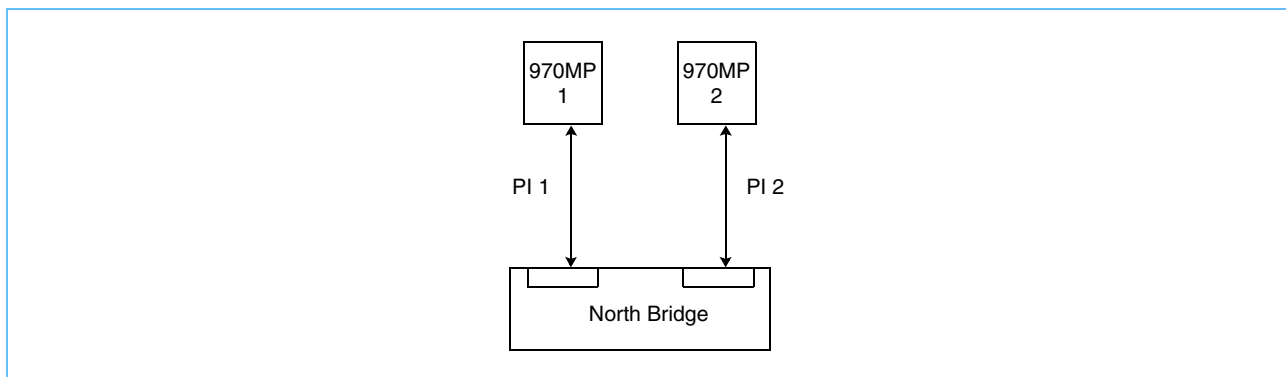
At the heart of the processor interconnect bus is a set of unidirectional, point-to-point bus segments, a new design selected to achieve maximum data transfer rates. The bus segments include two 35-bit address/data segments (one in each direction), two 1-bit transfer-handshake segments, and two 2-bit snoop-response segments. New features include:

- Pipelined transactions for reading and writing data and maintaining cache coherency
- Packet protocols for data sharing, data synchronization, and cache snooping
- True split transactions, enabling the master and slave to simultaneously conduct different transactions with each other
- Wave pipelining to exploit maximum data bandwidth at the electrical interface

The unidirectional segments are the basis for supporting the features previously listed. These buses are point-to-point connections, carry their own local clock signal (source synchronous), and require no arbitration. Error detection mechanisms exist for all bus segments.

There are many possible configurations that incorporate different numbers of processors, I/O interfaces, and memory bandwidth, and meet different speed, cost, and power requirements. *Figure 8-1* shows an example of a configuration with two 970MP microprocessors.

Figure 8-1. Processor Interconnect Bus Configuration with Two 970MP Microprocessors



The remainder of this section specifies the processor interconnect architecture targeting a dual processor, dual-ported North Bridge configuration, as shown in *Figure 8-1*. Using two processor interconnect ports on the North Bridge enables direct connection of two 970MP microprocessors.

IBM PowerPC 970MP RISC Microprocessor

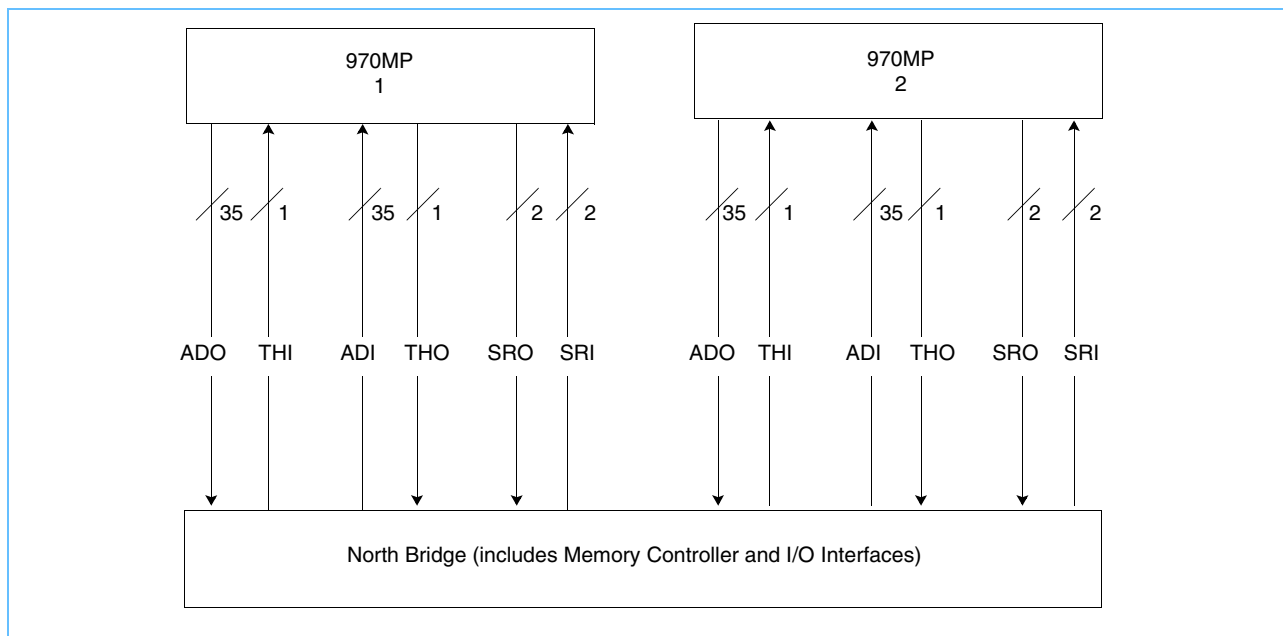
8.1 Overview

The processor interconnect bus consists of a set of unidirectional, point-to-point bus segments for maximum data transfer rates. No bus-level arbitration is required. An address/data (AD) bus segment, a transfer-handshake (TH) bus segment, and a snoop-response (SR) bus segment exist in each direction, outbound and inbound. *Figure 8-2* shows two 970MP microprocessors connected to a North Bridge using two processor interconnect buses.

This section frequently uses the terms “packet,” “beat,” “master,” and “slave.” The usage conventions of these terms are as follows:

- Data is transferred across a bus in beats from master to slave. A beat is a timing event relative to the rising or falling edge of the clock signal. Nominally there are two beats per clock cycle (one for the rising edge and one for the falling edge).
- A packet is the fundamental protocol data unit for the processor interconnect bus. A non-null packet consists of an even number of data elements that are sequentially transferred across a source-synchronous bus at the rate of one element per bus beat. The number of bits in each data element equals the width of the bus. Packets are used for sending commands, reading and writing data, maintaining distributed cache coherency, and transfer-protocol handshaking.
- A sender or source of packets for a bus segment is called a master and a receiver or recipient is called a slave. For example, on an outbound processor bus segment, the North Bridge is the slave and the processor is the master. On an inbound processor bus segment, the North Bridge is the master and the processor is the slave.

Figure 8-2. Two Microprocessors Connected to a North Bridge



8.1.1 Packets

Four basic packet types are defined: null packets, command packets, data packets, and transfer-handshake packets. Non-null packet lengths are always an even number of beats.

Null packets are sent across the address/data bus. For the null packet, all bits are zero. Null packets are ignored by slave devices.

Command packets are sent across the address/data bus. There are three types of command packets: read-command packets, write-command packets, and coherency-control packets.

Data packets are also sent across the address/data bus. There are two types of data packets: read-data packets and write-data packets. A write-data packet immediately follows a write-command packet. A read-data packet is sent in response to a read-command packet or a cache-coherency snoop operation. A data read header contains the address of the command, the command type, and transfer details.

Transfer-handshake packets are sent across the transfer-handshake bus. This packet is issued to confirm receipt and indicate the condition of the received command packet or data packet. Condition encoding includes Acknowledgment, Retry, Parity Error, or Null/Idle. A transfer-handshake packet is two beats in length.

See *Section 8.2 Packet Transfer Protocol* on page 147 for a detailed description of these four packet types.

8.1.2 Bus Segments

An AD bus segment, a TH bus segment, and an SR bus segment exist in each direction, outbound and inbound. *Table 8-1* and the following subsections further describe these signals.

Table 8-1. Processor Interconnect Signal Description

Signal Names	Signal Lines	Mnemonic	Description
Address/Data Out	35	ADO	Address or data and control information
Transfer Handshake Out	1	THO	Acknowledgment packet for command and data packets received on the address/data in bus
Snoop Response Out	2	SRO	Snoop coherency response from the processor
Address/Data In	35	ADI	Address or data, and control information
Transfer Handshake In	1	THI	Acknowledgment packet for command and data packets received on the address/data out bus
Snoop Response In	2	SRI	Accumulated snoop coherency response from the North Bridge

8.1.2.1 Address/Data Bus Segment

The address/data bus is used to transfer both command packets (containing control information) and data packets (containing the data to be transferred). The address/data bus consists of one 35-bit outbound address/data (ADO) bus segment and one 35-bit inbound address/data (ADI) bus segment.

Commands are issued to the bus as 2-beat packets. A read-data packet consists of a 2-beat header followed by the data payload. The number of beats issued with a data transfer depends on the size of the total transfer. Data payload is issued to the bus in even multiples of 4-byte wide data beats. Included in the packet is a bit for special system support and a data error bit.

8.1.2.2 Transfer-Handshake Bus Segment

The transfer-handshake bus sends transfer-handshake packets, which confirm that command or data packets were received on the address/data bus. The transfer-handshake bus consists of one 1-bit outbound transfer-handshake (THO) bus segment and one 1-bit inbound transfer-handshake (THI) bus segment. Every device issuing a command packet, data packet, or reflected command packet to the address/data bus receives a transfer-handshake packet through the transfer-handshake bus some fixed number of beats after issuing the command or data packet.

Each transfer-handshake bus segment sends transfer packets for command and data packets transferred in the opposite direction. That is, the outbound transfer-handshake bus sends acknowledgment packets for the command and data packets received on the inbound AD bus. There is no dependency or relationship between packets on the outbound address/data bus and the outbound transfer-handshake bus.

A transfer-handshake packet might result in a command packet being reissued to the bus because a data buffer in the command queue is full. IBM suggests that the North Bridge implement queues that are deep enough to minimize the impact of command packet retries on system performance.

A transaction remains active until it has passed all response windows. For write transactions, this includes the last beat of the data payload. Since commands might be retried for queue or buffer full conditions, transactions that must be ordered cannot be simultaneously in the active state.

A write transaction issued by the processor can be retried. The slave issues two transfer-handshake packets for a write transaction. The first packet is for the write-command packet and the second for the write-data packet.

For read transactions, the processor will not retry inbound (memory to processor) transfers. Reflected commands (that is, snoop requests inbound from the North Bridge to the processor) cannot be retried. This is necessary to ensure a fixed snoop window is maintained.

8.1.2.3 Snoop-Response Bus Segment

The snoop-response bus supports global snooping activities to maintain cache coherency. A processor uses this bus to respond to a reflected command packet received on the ADI bus. The snoop-response bus consists of one 2-bit, outbound snoop-response (SRO) bus segment and one 2-bit, inbound snoop-response (SRI) bus segment. The bus segments can detect single bit errors.

A snoop response begins when a processor receives a reflected command packet on the ADI bus. The processor provides a snoop response reporting the coherency status of the request received on the ADI bus segment. The North Bridge gathers snoop responses from all processors and sends the accumulated snoop response on the SRI bus segments concurrently to all processors.

8.1.3 Transactions

Three transaction types are defined: read, write, and command-only. *Section 8.4 Bus Transactions* on page 163 describes the transactions in detail. The following subsections show the sequence of operations for these transaction types.

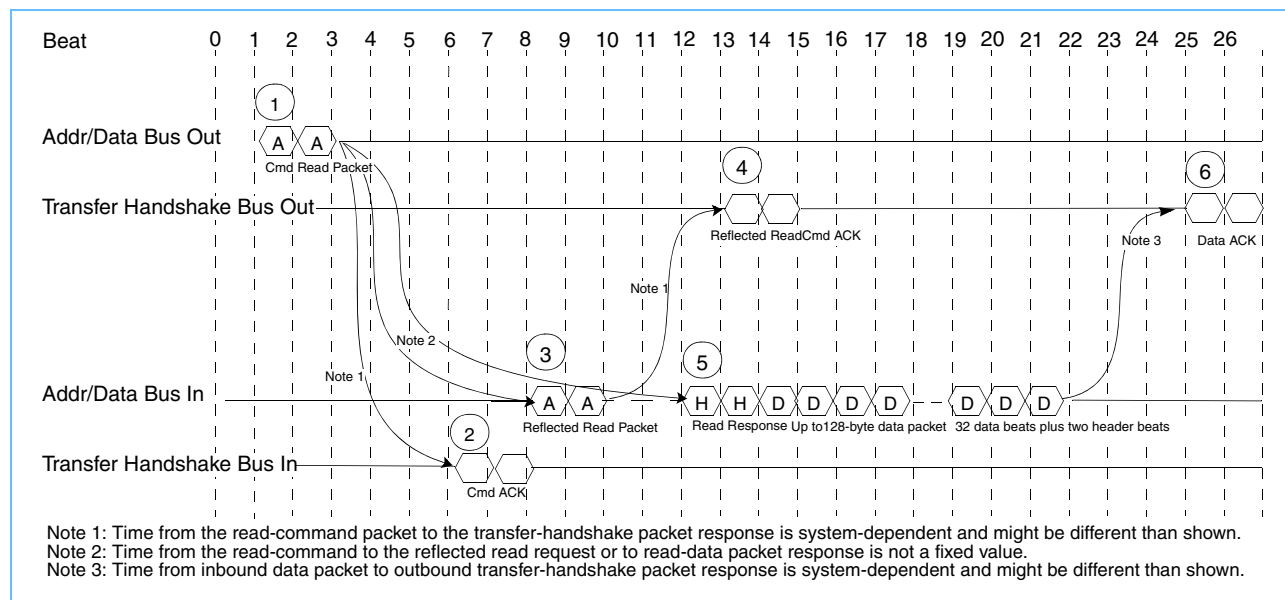
8.1.3.1 Read Transaction

Figure 8-3 shows the sequence of operations for a read transaction.

1. The master (requesting processor) issues a read-command packet on the ADO bus segment to request a full or partial cache line of data from the slave (North Bridge).
2. The slave sends a transfer-handshake packet to the master on the THI bus segment.
3. For cache-coherency purposes, the slave reflects the read-command packet on the ADI bus segment to all processors.
4. Each processor sends a transfer-handshake packet on the THO bus segment to the slave in response to the reflected read-command packet.
5. The slave sends the read-data packet on the ADI bus segment to the master.
6. The master sends a transfer-handshake packet on the THO bus segment to the North Bridge in response to the read-data packet.

The read-data packet transfer ranges from 4 to 34 beats. The first two beats transferred are a header containing the master's tag and data packet size. The data payload portion must be transmitted in sequence with the critical word first. A command packet might then be interjected into the data payload portion on an even-beat boundary.

Figure 8-3. Read Transaction Timing Diagram



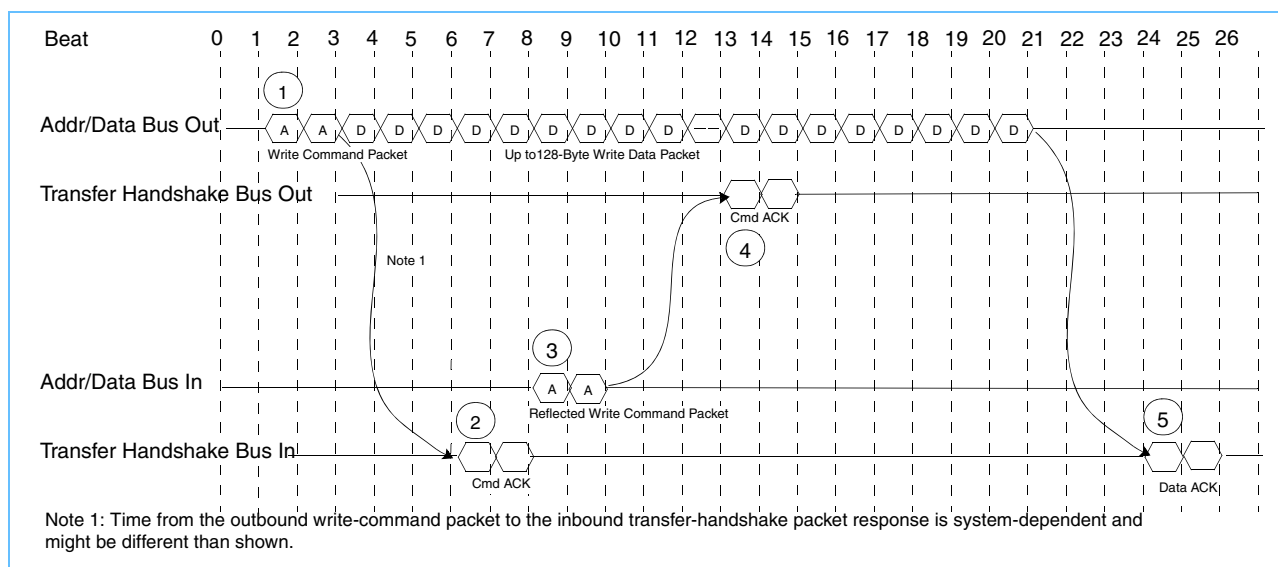
IBM PowerPC 970MP RISC Microprocessor

8.1.3.2 Write Transaction

A processor initiates a write transaction to store either a full or partial cache line of data to memory or to an I/O device. A write transaction consists of a command packet immediately followed by a data packet on the master's ADO bus segment. The data must be issued to the address/data bus segment in consecutive beats, but can be paused on an even beat to issue a command packet for a read operation. A write-command packet cannot be interjected into a write-data packet transfer. *Figure 8-4* shows the sequence of operations for a write transaction.

1. The master (requesting processor) issues a write-command packet on the ADO bus segment to write a full or partial cache line of data. The write-command packet is immediately followed by a write-data packet.
2. The slave (North Bridge) sends a transfer-handshake packet on the THI bus segment in response to the write-command packet.
3. For cache-coherency purposes, the slave reflects the write-command packet on the ADI bus segment to all processors.
4. Each processor sends a transfer-handshake packet on the THO bus segment to the slave in response to the reflected write-command packet.
5. The slave sends an acknowledgment packet on the THI bus segment to the master in response to the write-data packet.

Figure 8-4. Write Transaction Timing Diagram

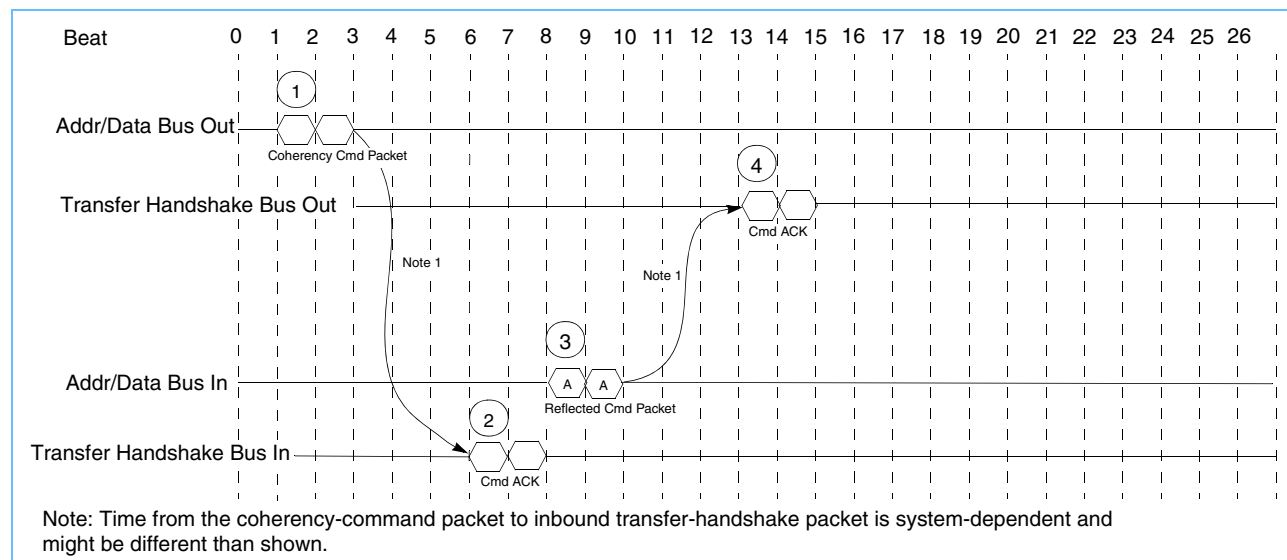


8.1.3.3 Command-Only Transaction

Figure 8-5 shows the sequence of operations for a command-only transaction.

1. The master (requesting processor) issues a command packet to the slave (North Bridge) on the ADO bus segment.
2. The slave sends a transfer-handshake packet to the master on the THI bus segment in response to the command packet.
3. For cache-coherency purposes, the slave reflects the command packet on the THI bus segment to all processors.
4. Each processor sends a transfer-handshake packet on the THO bus segment in response to the reflected command packet.

Figure 8-5. Command-Only Transaction Timing Diagram



8.1.4 Memory and Cache Coherency

8.1.4.1 Physical Memory Size

The PowerPC Architecture supports a maximum physical address bus of 64 bits. The processor interconnect specification limits the memory addressing to 42 bits. This allows for a maximum address space of 4 terabytes (TBytes).

8.1.4.2 Coherency Protocol

Coherency is maintained using global snoops of all command packets by reflecting command packets from the North Bridge to the processor. The snoop-response bus is used exclusively for this purpose. This bus consists of two unidirectional 2-bit bus segments per processor port, and is used to source response out and receive response in. Responses are sourced at a configurable time after the global snoop. The response in is sampled at a later time, also configurable. The snooping protocol is detailed in *Section 8.3 Snoop Responses* on page 158.

8.1.4.3 Coherency Block Size

The cache line is the smallest increment of memory over which coherency information is maintained. This bus can support 32-byte, 64-byte, and 128-byte coherency block sizes. The coherency block size is determined by the target processor. All bus attachments must support this coherency block size for uniform operation. The I/O must be capable of transferring less than or equal to, but not greater than, the coherency block size during direct memory access (DMA) transfers to and from coherent memory.

8.2 Packet Transfer Protocol

This section defines packet protocols for data sharing, data synchronization, and cache snooping. The processor interconnect defines four basic packet types: null packets, command packets, data packets, and transfer-handshake packets.

8.2.1 Command Packet Definition

The command packet transfer protocol specifies how addresses are passed between bus devices. Due to the narrow width of the bus, this transfer takes two bus beats to complete, thereby allowing one command packet every two beats.

The command packet consists of a memory address, command type, command size, and command tag. The command packet is identified on the address/data bus by the detection of the packet start signal and a packet-type encoding for a command packet. *Table 8-2* shows the bit definitions for the address/data bus during a command-packet transfer.

Table 8-2. Command Packet Description

Beat	Bits	Description
1	0:1	'10' (Address valid decode).
1	2:6	Transfer Type (0:4).
1	7:15	Transfer Tag (0:8).
1	16:17	Address Modifiers I/S, M (1:2).
1	18:34	Address (42:58). 17 bits of the 42-bit address.
2	0:1	'10' (Address valid decode).
2	2	Address Modifier W/N (0).
2	3:6	Transfer Size (0:3).
2	7:26	Address (22:41). Most-significant 20 bits of the 42 address bits.
2	27:29	Address Modifiers G, R, P (3:5).
2	30:34	Address (59:63). Least-significant 5 bits of the 42 address bits.
Note: W: write through, M: memory coherent, N: intervention, A: atomic, R: rerunning, I: cache inhibited, S: noncacheing coherent read, P: pipelined snoops, G: guarded read.		

8.2.1.1 Address Modifiers

Bits 16:17 of beat one, and bit 2 and bits 27:29 of beat two of a command packet contain the address modifier bits. These bits further describe the type of command packet. In some cases, they must be decoded along with the Transfer Type bits to determine the operation.

Table 8-3 Transfer Type Encoding on page 149 shows when these bits are used to modify transactions, what the modification is, and what the values are when they are hard coded. Under certain conditions, some bits might be sourced from the page table WIM bits ("W" stands for write through, "I" for cache inhibit, "M" for memory coherence).

IBM PowerPC 970MP RISC Microprocessor

Address Modifier[0]

The AM[0] bit, when indicated as a W, means that write through is wanted. When the bit is '1', it means that the data for a write transaction is to be forwarded all the way to system memory or a memory-mapped device. When the bit is '0', the data must be forwarded at least one cache level toward memory. This bit is normally, but not always, sourced from the page table.

On a read operation, when indicated as an N, this bit defines whether the master can support intervention on this request. If intervention is enabled (N equals '1'), then the transfer size must be the coherency block size. (A snooper might not intervene if this bit is reset, and might intervene if it is asserted.)

Address Modifier[1]

The AM[1] bit, when indicated as an I, indicates Cache-Inhibit status. If the bit is '1' in a write-command packet, it means that the data should not be cached downstream from this processor. When indicated as an S and the bit is a '1' in a read-command packet, it means that the requesting processor will not cache the data when received, and memory (or an intervening cache) might still retain the current coherency status.

Address Modifier[2]

The AM[2] bit, when indicated as an M, is always used as the memory coherent indicator or snoop request signal. If this bit is '0', the horizontal coherency snoopers ignore this transaction, meaning memory is not coherent or this is a transaction that snoopers do not need to look at (vertical caches need to snoop all snoop-response [SResp] enabled transactions regardless of the M bit).

Note: This bit should be defined consistently for future transactions that might be architected, as snoopers will not see any transaction where M equals '0'. This bit is frequently sourced from the page table WIM bits when indicated as an M, but at other times it is hard coded so snoopers see the transaction. For example, it might be set by an I/O adapter for coherent I/O.

Address Modifier[3]

The AM[3] bit is used to further define operations. For example, it is used to indicate a write-with-kill versus a write-with-clean. When indicated as a G, it is used to indicate a guarded read.

Address Modifier[4]

The AM[4] bit, when indicated as an R, means that this transaction has already been issued to the bus once, and is now being reissued.

Implementation Note: The bit should be set to zero in current implementations of the architecture, to remain compatible with potential architecture extensions.

Address Modifier[5]

The AM[5] bit, when indicated as a P, means that this transaction can be pipelined for snoop requests and responses. If P is '0', then command packets are reflected one at a time after the snoop response for previous command packets are seen by all processors.

8.2.1.2 Transfer Type Field

The transfer type (TType) field indicates the type of command packet that was issued to the bus. The valid transfer types defined by the processor interconnect bus are shown in *Table 8-3*. Both the processor and the North Bridge must support all commands listed in *Table 8-3*. I/O devices support only a limited subset of the commands.

Table 8-3. Transfer Type Encoding

Address Modifiers (WIMGRP)	TType Binary	TType Hex	Bus Operation	Code	Address Format	Data Payload	Comments
XXMXRP	00000	00	Clean	CL	Mem	N	M = '1' normally
WIMXRP	00010	02	Write with Flush	WNB	Mem	Y	M = '1' normally
XXMXRP	00100	04	Flush	FL	Mem	N	M = '1' normally
WXMORP	00110	06	Write with Kill	WBK	Mem	Y	W = 'X' if from a I/O bridge
WXM1RP	00110	06	Write with Clean	WBC	Mem	Y	W = '1', I = 'X', M = '0'
XXMXRP	01000	08	SYNC	SY	Tag	N	
NSMGRP	A1010	0A,1A	Read	RD	Mem	N	S = '1' means RWNITC
XXMXRP	01100	0C	DKill	DK	Mem	N	M = '1'
NXMXRP	A1110	0E,1E	RWITM	RWITM	Mem	N	I = 'X', normally M = '1'
XXMX0P	10000	10	EIEIO	EI	Tag	N	M = '0'
XXMXXX	10100	14	Reserved				M = '0'
XXMX0P	11000	18	TLBIE	TI	Tag	N	M = '0', P = '0'
XXMXXX	11100	1C	Reserved				M = '0'
XXMXRP	00001	01	LARX-Reserve	LR	Mem	N	M = '0'
XXMXRP	A0011	03,13	DClaim	DC	Mem	N	M = '1'
XXMXXX	001X1	05,07	Reserved				M = '0'
XXMXRP	01001	09	TLBSYNC	TS	Tag	N	M = '0'
XXMXXX	01X11	0B,0F	Reserved				M = '0'
XXMX0P	01101	0D	IKill	IK	Mem	N	M = '1' normally, P = '0'
XXMXXX	10001	11	Reserved			A	M = '0'
XXMXXX	10010	12	Reserved			N	M = '0'
XXMXRP	10101	15	Deallocate Dir Tag	DDT	Mem	A	M = '0'
XXMXXX	1011X	16,17	Reserved for customers				M = '0'
XXMXXX	110X1	19,1B	Reserved				M = '0'
XXMX0P	11111	1F	Null	NUL	None	N	M = '0'

Note:

W: write through, M: memory coherent, N: intervention, A: atomic, R: rerunning, I: cache inhibited, S: noncaching coherent read, P: pipelined snoops, G: guarded read, X: drive '0' when driving signal and don't care when receiving the signal.

Address Field

The address field contains the address associated with the command packet. This field is defined to be 42 bits wide.

IBM PowerPC 970MP RISC Microprocessor
Transfer Size Field

The transfer size field indicates the size of the data packet associated with the command packet. For command packets that do not have a data packet associated with them, this field is undefined. *Table 8-4* defines the encoding for the transfer size field for the commands that require a data packet.

Table 8-4. Transfer Size Encoding

Transfer Size	Description	Number of Data Beats
0000	8 Bytes	2
0001	1 Byte	2
0010	2 Bytes	2
0011	3 Bytes	2
0100	4 Bytes	2
0101	5 Bytes	2
0110	6 Bytes	2
0111	7 Bytes	2
1000	128 Bytes	32
1001	16 Bytes	4
1010	32 Bytes	8
1011	Reserved	
1100	64 Bytes	16
1101	Reserved	
1110	Reserved	
1111	Reserved	

Transfer Tag

Command packets contain a 9-bit transfer tag used to link a command with data. This field is valid for all transactions to the bus and contains a number (generated by the processor) to identify the read-data packet on a read transaction and the write-data packet for a write transaction. Explicit tagging of command and data packets allows a bus device to have multiple concurrent outstanding transactions that require a data packet. This means that read-data packets can appear out-of-order on the bus so that transactions can complete when data is available as opposed to returning all data packets in the order the commands were issued. In addition, the tag can be used to reference the response back to a command in an internal queue of a bus device. There must only be one outstanding transaction referred to by a tag at any time.

Tag Deallocation For Read Operations

Read transactions use the tag field to identify incoming read-data packets that are associated with the transaction. Once a tag is assigned to a read transaction, it cannot be reissued until all the read data has been received.

Tag Deallocation For Store, Castout, and Push Operations

Address/data command tags remain active until a clean global snoop response is received.

8.2.1.3 Tag Definition

Table 8-5 defines the 9-bit tag that is sent with a command or read-data packet.

Table 8-5. Tag Definition

Bits	Field Name	Description
0:3	Master number	Master number (one must be reserved for the North Bridge)
4:8	Master tag	Tag (one of 32) assigned to the master's requesting resource

Interjecting Command Packets

Data transfers on the bus are either write-data packets issued with a write-command packet, or read-data packets. These transfers consist of multiple data beats. When a transfer contains multiple beats of data payload transfer, a command packet might be interjected on an even-beat boundary. This feature allows new transactions to be started without having to wait for a long multi-beat data transfer to complete. This protocol allows read-command packets and coherency-control packets to be interjected. Write, castout, push, partial write operations, or other data packets cannot be interjected into a multiple-beat data transfer.

8.2.1.4 Command Pacing

It is possible for the processor to issue command packets at a rate faster than the slave can accept. The slave must then retry the packets so the commands are not lost. This is undesirable because of the additional bus bandwidth consumed for the retried commands. The North Bridge should implement queues that are sufficiently deep to minimize the impact of command packet retries on system performance. This scenario assumes the slaves can handle consecutive data packets, which requires the data buffering to be run at least at the bus clock speed. To avoid this situation, a command pipeline delay parameter, COMPACE, is defined for the bus.

The command pipeline delay parameter is a 4-bit field that is programmed into each bus master to indicate the number of bus beats of delay that must be placed between each command packet on the bus. The delay is in bus beats (assumed to be even). The allowable range of values for COMPACE and related processor delay parameters can be found in *Table 11-1 Programmable Delay Parameters* on page 281. *Table 11-1* also lists North Bridge delay parameters and typical values that these parameters might take on. See *Section 11.2.2 Configurable Parameters* on page 279 for additional information about these configurable delay parameters.

Note: This does not restrict the use of intervening bus beats for data packets.

IBM PowerPC 970MP RISC Microprocessor

8.2.2 Data Packet Definition

The data packet transfer protocol specifies how data is passed between bus devices. A data packet is defined as an even-numbered beat transfer on the address/data bus. A write-data packet immediately follows a write-command packet. It is identified on the bus by the data valid decode. A read-data packet has a 2-beat header that includes the tag and the data size. Typically, read-data packets are sent from the North Bridge to a processor. However, during intervention, a processor can send a read-data packet to the North Bridge.

A data packet of the minimum size consists of 8 bytes of data and the data error signal (DERR) to validate the data. Up to 16 pairs of data beats are used to transfer a cache line. *Table 8-6* shows the bit definitions for the read-data packet header on the address/data bus. *Table 8-7* shows the bit definitions for the address/data bus during a data transfer.

Table 8-6. Read-Data Packet Header Description

Beat	Bits	Description
1	0:1	'11' (Data and Address valid decode).
1	2:6	Reserved.
1	7:15	Transfer Tag (0:8).
1	16:18	Reserved.
1	19:22	Responder or Intervener ID.
1	23:34	Reserved.
2	0:1	'11' (Data and Address valid decode).
2	2	Reserved.
2	3:6	Transfer Size (0:3).
2	7:34	Reserved.

Table 8-7. Data Beat Description

Beat	Bits	Description
1	0:1	'01' (Data valid decode).
1	2:33	Next consecutive four bytes of the data packet.
1	34	Data error signal (DERR) indicates an off-bus data error; full data transfer is invalid.
2	0:1	'01' (Data valid decode).
2	2:33	Next consecutive four bytes of the data packet.
2	34	Reserved.

8.2.2.1 Two-Beat Transfers

The processor interconnect supports data transfers of varying lengths. Since the payload portion of all data packets must be at least two beats, a transfer of less than 8 bytes must be padded with additional data to fill the 8-byte minimum transfer size. The data on the bus must be address aligned so a request must be separated into two requests if an 8-byte address boundary is crossed. A master can transfer from 1 to 8 bytes of data during this operation. Data is returned in the original memory order. *Table 8-8* shows the address restrictions for transfers of 1 to 8 bytes.

Table 8-8. Two-Beat Data Transfers

Starting Address[61:63]	Byte Lanes																Data Size
	00	01	02	03	04	05	06	07									
000 – 111	x	x	x	x	x	x	x	x									1 Byte
000, 010, 100, 110	x		x		x		x										2 Byte
000	x																3 Byte
000, 100	x				x												4 Byte
000	x																8 Byte

Note:

1. 'x' is a valid starting position.
2. The operand may not cross a double word boundary.

8.2.2.2 Multi-Beat Transfers

The processor interconnect supports multiple-beat data transfers that are 16, 32, 64, and 128 bytes in length. All such requests for writes and reads that are less than a full coherency block (128 bytes) must be aligned to an address boundary equal to the size of the transfer. For read-data transfers that are a full coherency block, data is returned with the critical 16 bytes first, followed by the remaining data in an interleaved burst order. The resulting data transfer is a block of data that is aligned to the size of the request.

Data Transfer Format

On read data packet transfers that are a full coherency block, the order of the returned data words depends on the address that was specified inside the command packet. Each block of the read data packet is transferred in a sequence of 32-byte data beats. Data ordering is based on the block size. Within a word, data is always transferred in-order starting with the most-significant byte and ending with the least-significant byte.

Partial write commands with transfer sizes less than 8 bytes cannot cross an 8-byte boundary. All write commands (including write, castout, push, and partial write) with transfer sizes of 8 bytes or more must be aligned on an address boundary equal to the size of the transfer.

IBM PowerPC 970MP RISC Microprocessor
Table 8-9. Packet Ordering for 128-Byte Interleaved Packets on 32-Byte Boundaries

Address (57:59) 128-Byte	Packet Order Viewed at 16-byte Read Data Transfer	Packet Order Viewed at 32-byte Read Data Transfer
000	0 1 2 3 4 5 6 7	0 1 2 3
001 (Not Valid)	1 0 3 2 5 4 7 6	
010	2 3 0 1 6 7 4 5	1 0 3 2
011 (Not Valid)	3 2 1 0 7 6 5 4	
100	4 5 6 7 0 1 2 3	2 3 0 1
101 (Not Valid)	5 4 7 6 1 0 3 2	
110	6 7 4 5 2 3 0 1	3 2 1 0
111 (Not Valid)	7 6 5 4 3 2 1 0	

Table 8-10. Packet Ordering for 32-Byte Interleaved Packets

Address (59:60)	Packet Order for 4-Word Read-Data Transfer
00	0 1 2 3
01	1 0 2 3
10	2 3 0 1
11	3 2 1 0

Interjecting Command Packets

Data transfers on the bus are either write-data packets issued with a write-command packet, or read-data packets. These transfers consist of multiple data beats. When a transfer contains multiple beats of data payload, a command packet can be interjected on an even-beat boundary. This feature allows new transactions to be started without having to wait for a long multi-beat data transfer to complete. This specification allows read-command packets and coherency-control packets to be interjected. Write, castout, push, and partial write operations, or other data packets cannot be interjected into a multiple-beat data transfer.

8.2.3 Transfer-Handshake Packets

The transfer-handshake bus is used to acknowledge command or data packets that were received on the inbound bus. Every command and data packet that is received on the inbound bus is acknowledged by a transfer-handshake packet on the associated outbound transfer-handshake bus. The transfer-handshake packet occurs a fixed number of beats later. Each transfer-handshake packet is two beats in length.

Table 8-11 shows the handshake encoding for the bus.

Table 8-11. Transfer-Handshake Definition

Response Beat 0, Beat 1	Description
0 0	Null/Idle
1 0	Acknowledge (command/data accepted)
0 1	Retry (command/data rejected, reissue command)
1 1	Parity error (parity error detected on bus)

The slave sends this acknowledgment packet to the bus n beats after receipt of the last beat of the command or data packet (n is the minimum number of beats necessary for the slave to receive the data from the bus, check the command and address, and generate the response). This time is implementation-dependent and can vary from one device to the next. The master samples the response STATLAT beats after the last beat of the command or data packet. STATLAT is the number of bus beats *between* the last beat of the command or data packet and the first beat of the acknowledgment packet. For example, if the last beat of a command packet was on beat j and the first beat of the acknowledgment packet occurred on beat k , then the value for STATLAT would be $k-j-1$ (see Figure 11-1 on page 280). The STATLAT beat count includes the time required by the slave to generate the response plus the time that it takes for the packet to be sent and the acknowledgment to be returned. For consistency in design of the processors that attach to this bus, an upper limit is defined for the time between the master issuing the last beat of the command or data packet to the bus to when it receives the first beat of the acknowledgment packet (see Table 11-1 on page 281 and the *IBM PowerPC 970MP RISC Microprocessor Datasheet*). This time should be minimized to eliminate unnecessary delays on commands in the pipeline that have ordering requirements with the current command. The STATLAT parameter is configured by the inter-integrated circuit (I²C) interface during the bus initialization phase.

8.2.3.1 Null Transfer Handshake

The null transfer handshake is the default response from a slave device. If the slave does not drive the transfer handshake with either an acknowledgment, retry, or parity error, then the response is, by default, null. The null response can occur due to a slave device timeout or a terminated transaction under certain, special conditions defined below.

Master:

For the master, this transfer-handshake response from the slave indicates that the command or data packet that the master sent was not accepted by the slave. Based on the address/data packet type, the master actions are as follows:

Command packet: The processor responds by going into checkstop.¹ If the command was a write command and the master detects this response before it has completed the full data transfer of the write-data packet, it can either complete the full data transfer or discard the remaining even-numbered data beats for the transfer.

Read-data packet: The processor responds by going into checkstop. A master always transmits full packets on the bus. The handshake is received after the end of the read-data packet (see *Figure 8-3* on page 143).

Note: The error might also result from a timeout while waiting for data or from an incorrect transfer size by the slave.

Write-data packet: The processor responds by going into checkstop, if the write-command packet associated with this packet received an acknowledgment transfer handshake from the slave. Otherwise, the null response is ignored. If the write command is retried by the slave, then the null response is the correct response for the data packet associated with that write command.

Slave:

The slave issues the null transfer handshake response for the following non-error conditional data packet for a write command that was retried. The command or data packet is discarded, and status is logged in the slave for the error case.

8.2.3.2 Transfer-Handshake Acknowledgment

The acknowledgment response indicates that the addressed slave accepted this command or data packet. If a bus agent² accepts (acknowledges) a command packet to send to a remote bus, it is responsible for completing the transaction back to the bus master if the remote bus does not accept the command packet. For a read transaction, this implies returning data to the master with the data error signal activated. The data error is signalled by asserting the 35th bit (DERR signal) of the even data beats. For writes, the command and data packets are discarded. The device must also have a mechanism to signal a machine check indicating that the error occurred.

Master:

For the master, the acknowledgment response indicates that the command or data packet was accepted and that it might complete execution of the packet transfer. Based on the packet type, the master acts as follows:

-
1. Hardware has detected a condition that it cannot resolve, and which prevents normal operation. It stops executing instructions, responding to interrupts, and so on.
 2. Bus agents are devices such as the North Bridge, but not switches that might be used to relay command and data packets.

Command packet: For a command packet that results in data being returned by the slave, the acknowledgment response indicates that the command has been accepted and need not be reissued to the bus. Inbound data packets to complete the transaction can be received starting in the beat following the response. For a write-data packet, the acknowledgment response indicates that the command has been accepted. The slave cannot retry the data packet after it accepts the command. That is, an acknowledgment response for the command packet indicates that the slave has set aside buffer space for the write-data packet. For command packets, this response indicates that the command is complete.

Data packets: This response indicates to the master that the data being sent was accepted by the slave without errors.

Slave:

The slave issues this acknowledgment response when:

- The slave received the command packet with a valid transfer type, transfer size, and address.
- For write transactions, there is queue space for the command and data.

The slave stores command packets in a command queue and stores data packets in data buffers.

8.2.3.3 Transfer-Handshake Retry

A handshake retry can be issued to flow control the command packets when the slave does not have space for the command packet or the data packet associated with the command. Any command packet can be retried by the slave, except for reflected command packets. Data packets may not be retried.

Master:

For the master, the retry response indicates that the command was rejected by the slave for lack of space in the command queue or the data buffers. Based on the packet type, the master acts as follows:

Command packet: When the master receives a retry response for a command packet, it reissues the packet to the AD bus. If the command was a write command and the master detects this response before it has completed the full data transfer, then it can either complete the full data transfer, or discard the remaining even-numbered data beats for the transfer before reissuing the command packet.

Data packet: Retry responses are not valid for write-data packets and read-data packets.

Slave:

The slave issues this response when:

- The slave received the command packet but the command queue was full.
- If the packet was a write-command packet, there is no space for the command or the data.

To properly detect termination of a partial write-data packet, the slave must examine the Address Valid decode bits (see *Table 8-2 Command Packet Description* on page 147) on a per even-beat basis.

Note: The retry transfer handshake cannot be issued for write-data packets.

IBM PowerPC 970MP RISC Microprocessor

8.2.3.4 Transfer-Handshake Parity Error

This response is optionally issued whenever a single bit error is detected during any bus beat. It is an unrecoverable error that results in a machine check to the processor with all command and data packets in the pipeline being discarded.

Master:

For the master, the response is a hard error indicating that the bus is no longer functional. The processor responds by going into checkstop.

Command packet: When the master receives a parity error response for a command packet, it reports the failure back to the system. The bus must be reinitialized before it can be used again.

Data packet: Same as command packet errors.

Slave:

If the slave issues this response (optional), it should be within the normal packet response timings. (This packet error might make this timing determination imprecise.) For the slave, this condition is a hard error and the bus is no longer functional. The slave logs the error and reports it to the system. The error reporting mechanism is system-dependent.

8.3 Snoop Responses

Cache coherency is maintained using a global snoop method, where a memory controller device (the North Bridge) reflects command packets to all processors at the same time. Snooping is supported by dedicated snoop-response bus segments, consisting of one 2-bit SRO and one 2-bit SRI.

A snoop response begins when a processor receives a reflected command packet on the ADI bus. The processor starts a programmable timing chain that determines when the processor's SRO is driven and when the processor's SRI will be sampled.

The snoop response from each processor is transmitted on the SRO response bus in two beats (see *Table 8-12*). The North Bridge gathers the snoop responses from all processors and performs a logical OR operation on the accumulated responses. The North Bridge sends the logical OR of the snoop responses back to all processors on the SRI bus.

Table 8-12. Snoop-Response Bit Definition

Beat	Bits	Description
1	SR[0]	Intervention
1	SR[1]	Modified
2	SR[0]	Retry
2	SR[1]	Shared

Table 8-13. Allowed Snoop Responses

Retry	Intervention	Modified	Shared	Description
0	0	0	0	Null (exclusive for reads)
0	0	0	1	Shared
0	0	1	x	Modified
0	1	0	0	Invalid
0	1	0	1	Shared intervention
0	1	1	x	Modified intervention
1	x	x	x	Retry

8.3.1 Snoop-Response Bus Implementation

Each snoop-response bus is controlled by two configurable parameters: SNOOPLAT and SNOOPACC. For all parameters, time is measured in bus beats from the final locally clocked flip-flop or latch output to the first locally clocked input.

The processor SNOOPLAT parameter defines the number of bus beats between receiving the last beat of the reflected command packet and driving the first beat of the snoop response. SNOOPLAT does not need to be programmed for the processors, since the processors are assumed to be identical. The North Bridge SNOOPLAT value is the sum of the transfer time of the reflected command packet from the North Bridge to the processor, the processor SNOOPLAT value, and the transfer time of the snoop-response bus from the processor to the North Bridge (see *Figure 11-2 North Bridge Configurable Timing Parameters* on page 280).

On the North Bridge, the SNOOPACC parameter defines the delay between the time a processor sends the last beat of an individual snoop response to the time it receives the first beat of the accumulated snoop response from the North Bridge (see *Figure 11-3 Processor Configurable Timing Parameters* on page 281). SNOOPACC includes the time required by the North Bridge to gather the responses from all of the processors. The North Bridge reflects all incoming command packets at a pace determined by the SNOOPWIN parameter. SNOOPWIN sets the snoop window, which is the minimum distance between two consecutive snoop requests (see *Figure 11-3 Processor Configurable Timing Parameters* on page 281).

An address collision occurs if the current address is the same as a requested address for a previously received snoop. If this occurs, the current snoop request is delayed until the conflicting previous request is concluded. This condition is called previous adjacent address match (PAAM). The PAAMWIN parameter indicates the number of bus beats a request is active during which a conflicting snoop request cannot be issued. An unrelated snoop request can be sent during the PAAM window. *Figure 11-2 North Bridge Configurable Timing Parameters* on page 280 shows the timing of the PAAMWIN parameter.

For a snoop request to be issued, the following conditions must be satisfied:

1. At least SNOOPWIN beats have transpired since the previous snoop request was issued.
2. There is at least one non-active PAAM address slot available.
3. No active PAAM address conflicts with the request.

The number of PAAM address slots on the North Bridge is implementation-dependent, but ranges from two to four. A snoop request activates a PAAM address slot when it is issued. After PAAMWIN beats, the slot is deactivated and can be reassigned to another request. The number of address bits used to detect conflict is also implementation-dependent.

IBM PowerPC 970MP RISC Microprocessor

There is no requirement that all snoop requests fall in exact modulo SNOOPWIN beats. Even-numbered idle bus beats can be used beyond SNOOPWIN between two subsequent snoop requests. The PAAMWIN value is not required to be a multiple of the SNOOPWIN value.

The I²C interface is used to program all programmable delay parameters (see *Section 11.1 I2C Interface* on page 279).

8.3.2 Snoop-Response Descriptions

8.3.2.1 SResp Retry Response Code (Priority 1 - highest)

SResp Retry is issued for the following reasons:

- **Lost reservation:** A master that has a reservation will retry an atomic write/flush itself if the reservation has been lost since the write was issued.
- **Push condition:** A snooper will retry a transaction if a push is needed for a read or write-with-flush.
- **Resource conflict:** A snooper will retry a transaction due to collision with a resource that has ownership of the line.
- **Memory and intervention buffer full:** A North Bridge can retry a read transaction that might cause intervention, if it determines it temporarily cannot receive the intervention data. It is typically more efficient to use the transfer-handshake retry on the intervening data packet for this case.

SResp Retry ramifications:

- **Master:** Can reissue this operation and use a different tag, or can reissue a different operation instead of or before this operation is reissued. Any data transfer aborted by this retry can be terminated prior to the data packet completion.
- **Target:** Any operation that has completed an SResp Retry can take a variable amount of time to clean up resources and, therefore, can cause future retries due to resources being tied up by this operation. Guarded cache-inhibited write operations need to be ordered with respect to each other. The processor cannot proceed and cannot issue the next operation until the SResp window with the null response has passed.
- **Snooper:** Any operation that has completed an SResp Retry is aborted by the snooper and leaves the cache state unmodified, except when Intervention is disabled on a read request and the snooper has modified data. The snooper will then push the data back to memory and clean or invalidate the line.

8.3.2.2 SResp Modified-Intervention Response Code (Priority 2)

The Modified coding is activated when a snoopers detects the address of a cache line on a read operation that is contained in its own cache and is modified (dirty). The snoopers then provides the data by using intervention.

SResp Modified-Intervention is asserted if a snoopers asserts SResp Modified-Intervention on a Read or Read with Intent to Modify (RWITM) when bus intervention is enabled (N equals '1'), snooping is enabled (M equals '1') and a cache line is snooped modified. If SResp Retry is sampled instead of SResp Modified-Intervention, then the snoopers can either push the block to memory or leave the cache state unmodified.

The ramifications of an SResp Modified - Intervention for bus devices are:

- Master and Read or RWITM:
This tells the master that its request is satisfied by the cache holding the modified data.
- Memory and Read or RWITM:
This tells the North Bridge to cancel its read request. If the command was read, the North Bridge looks for the tagged data and copies the block to memory.

8.3.2.3 SResp Shared-Intervention Response Code (Priority 3)

The Shared-Intervention coding is activated when a snoopers detects the address of a cache line on a reflected read-command packet that is contained in the snoopers's own cache and is the owner (most recent recipient) of the data. This signal can only be asserted by one bus device, since there is only one owner of data. Since SResp Retry is higher priority than SResp Shared, the snoopers must wait until the snoop response is received before beginning the intervention push.

A snoopers using this code must accommodate the option on burst reads whereby the requester indicates intervention is not wanted. In these cases, the response must be SResp Shared.

The ramifications of an SResp Shared-Intervention are:

- A master receiving this SResp code looks for intervention data.
- The North Bridge treats SResp Shared - Intervention as SRespRetry.

8.3.2.4 SResp Modified Response Code (Priority 4)

The Shared-Intervention coding is activated when a snoopers detects the address of a cache line on a reflected read-command packet that is contained in the snoopers own cache and is the owner (most recent recipient) of the data. This signal can only be asserted by one bus device, since there is only one owner of data. Since SResp Retry is higher priority than SResp Shared, the snoopers must wait until the snoop response is received before beginning the intervention push.

A snoopers using this code must accommodate the option on burst reads whereby the requester indicates intervention is not wanted. In these cases, the response must be SResp Shared.

SResp Modified is asserted for the following reasons:

- A snoopers asserts SResp Modified on a Read or RWITM when bus intervention is not enabled (N equals '0'), snooping is enabled (M equals '1'), and a cache line is snooped modified. If SResp Retry is sampled instead of SResp Modified, then the snoopers can either push the block to memory or leave the cache state unmodified.
- A snoopers asserts SResp Modified for flush or clean bus operations if the addressed block is modified. If SResp Modified is sampled in this case, the snoopers pushes the block to memory and marks the cache Invalid (flush), or Shared/Exclusive (clean). If SResp Retry is sampled instead of SResp Modified, the snoopers can either push the block to memory or leave the cache state unchanged.

8.3.2.5 SResp Shared Response Code (Priority 5)

Snoopers:

The Shared response is encoded when a snoopers inspects the address of a cache line on a read transaction that is contained in its own cache and has not been modified, marking the block shared if the block was marked exclusive. This signal can be asserted by more than one snoopers, and the snoopers will retain a copy of the block.

I/O Snoopers:

I/O devices that do not cache data Exclusive or Modified (shared only) are allowed to assert without having the block cached (for example, they might snoop at a larger granularity than the block address).

Master:

This tells the bus master that the data on a read, when returned, must be marked shared and not exclusive.

8.3.2.6 SResp Null or Clean Response Code (Priority 6 - lowest)

The null or clean response is encoded to indicate one of the following:

- There is no local (or remote) device presently caching this line.
- A synchronize type of transaction (for example, **sync** or translation lookaside buffer sync [**tlbsync**]) has been completed by all snoopers.
- The line is cached, but the null response is allowed (for example, the null for a clean transaction that hits on an exclusive line).

8.4 Bus Transactions

This section provides details of the following processor interconnect bus transactions:

- Memory read transactions (general)
- Memory write transactions (general)
- Command only transactions

8.4.1 Terms

Each of the transactions in this section uses the following terms to define the parameters of the transaction:

Reservation	A reservation is an address location held by the processor. It is used to emulate atomic operations using the PowerPC load reserve indexed (larx) and store conditional (stcx) types of instructions. A processor has at most one reservation at any time. A reservation is established by executing a Load Word and Reserve Indexed (lwarx) or Load Double Word and Reserve Indexed (ldarx) instruction. It is normally lost when the corresponding Store Word Conditional Indexed (stwcx .) or Store Double Word Conditional Indexed (stdcx .) instruction is performed. A reservation might also be lost if the data at the address is modified by another processor or bus device.
Snooper	A bus device that inspects inbound reflected command packets and uses the snoop-response bus to keep cached data coherent with other system caches. A bus adapter or I/O bridge might contain a cache and, if so, will act like a snooper.
Memory	The bus device that responds to a memory read or write and handles positive acknowledgment for coherent operations. If some portion of memory is attached to a remote bus, the bus adapter also acts like memory for memory accesses to that remote memory space.
I/O Bridge	An I/O bridge device is a gateway to an I/O bus that cannot cache data in the Exclusive or Modified state. The bridge does not forward snoops to the I/O bus. If an I/O device has shared cache data, it is necessary to implement a directory for the cached data in the shared state.

8.4.2 Memory Read Transactions (General)

A master (processor) reads I/O or memory data by sending a read command to the memory controller of the North Bridge. The processor drives the ADO bus, provided it was not in the midst of sending another command packet, and there was no higher priority transaction ready to be sent. After a programmable number of beats (STATLAT), the master reads the transfer handshake from the THI bus to ascertain the status of the transfer. The slave (North Bridge) sends a positive acknowledgment on the THI bus if no parity error was detected and there was a slot to queue the read request. If no queuing space is available, a retry status is returned.

The North Bridge dequeues the request after internal arbitration and decodes the command packet. It issues a read request to the North Bridge for the indicated block size and reflects the command packet to all processors for snooping purposes. The North Bridge paces new snoop requests based on the programmable parameter, SNOOPWIN. The North Bridge will detect address collisions (transactions to the same cache line) and will delay the second conflicting transaction until PAAMWIN bus beats have transpired since the original conflicting transaction was issued for snooping. In addition, processors can request that transactions be handled one at a time, by setting the pipelined snoop (P) address modifier bit low.

Each processor drives their SRO bus during the snoop window that is seen by all processors and by the North Bridge at the same time. The processor can request that the transaction be retried with a retry snoop response. Otherwise, if a processor has a clean copy in its cache, the shared response code is returned. If the requested cache line is modified inside a processor cache, that processor signals the intervention snoop response, which is a promise to send to the North Bridge the modified copy in the form of a processor-to-memory read-data packet. The North Bridge accumulates the combined (logical-OR) snoop responses from all of the attached processors. Depending upon the combined response, the North Bridge might abort, delay, or send the memory data or the intervened data to the original requester. The intervened data is also written to memory for regular read transactions (no intention to modify).

When the North Bridge responds with the read data, it sends a read-data packet, which consists of a 2-beat header and 2 to 32 beats of payload data. The header contains the original tag and the data size. The payload data is sent immediately after the header. The DERR bit is asserted if the data contains an error.

8.4.2.1 Read Transaction

A read command is issued to get data that is not immediately going to be modified. The modifier bits that are valid are N (intervention) and S (non-caching). The M and I modifiers are sourced from the page table entry, hardwired, or set by the I/O.

Master:

A read burst is issued by the processor to satisfy a load, tablewalk access, Data Cache Block Touch (**dcbt**) or other data prefetch, or instruction fetch (I-fetch) to a cacheable page that misses the cache. A read non-burst is caused by a non-cacheable load or I-fetch.

Atomic:

The Atomic modifier (TType [0]) is set along with the M bit when the read is to satisfy a **lwarx** or **ldarx**.

S Bit:

The S bit is set along with the M bit when the master will not cache the data but wants the latest copy. If S is set, a snoop is allowed to clean up dirty data in its cache by pushing it to memory, but keeping it marked exclusive afterwards.

N Bit:

The N bit is set when the master and memory are capable of intervention, intervention is wanted, and the read-data size is the coherency block size for the system. All non-block size reads must have the N bit set to zero. In addition, the processor is capable of setting N to '0' for all reads in case it is attached to memory that does not support intervention.

G Bit:

The G bit is set when the read is the result of a load to cache-inhibited guarded storage. When set, the system implementation knows this read might only complete once.

P Bit:

The P bit is set when snoop pipelining is allowed (default for reads). This bit can be cleared for reads if the processor requires the transaction snoop response to be resolved before another independent transaction is issued. When an address collision is detected, the North Bridge automatically delays the colliding transaction until the previous transaction is resolved.

Snooper:

If the address contained in the reflected command packet is in the cache and marked Modified, the snooper performs a push or intervention.

Memory:

Memory can provide the addressed data no earlier than the end of the snoop window for that transaction. The North Bridge examines the snoop-response bus, and, if it was SResp Retry or SResp Intervention, the North Bridge will terminate the operation and deallocate the tag. If the SResp response is Modified, because the North Bridge supports intervention, the North Bridge captures the line as it is transferred to the requester and stores the line to memory.

I/O Bridge:

If the G bit is set, an I/O bridge can not issue the read to any memory-mapped I/O devices more than once. This means waiting until the previous guarded read is committed (no retry from the transfer handshake) before sending the next request.

8.4.2.2 Read with No Intent to Cache Transaction

Read with no Intent to Cache (RWNITC) is another name for a read transaction with the S bit and M bit set (see above). It is a coherent read; that is, the master wants the latest data, but does not cache it. Therefore, the snooper can keep caching the data as Exclusive after it provides the data by a push or an intervention.

8.4.2.3 Read with Intent to Modify Burst Transaction

Master:

The RWITM transaction is issued by a master to bring an entire block into a cache for the purpose of writing to it. It is always a block-sized read. It is triggered by a store, **stwcx.**, **stdcx.**, or Data Cache Block Touch for Store (**dcbtst**) to a cacheable page that misses in the cache. The master should mark its cache Exclusive if the SResp is not Retry.

Snooper:

The snooper invalidates any line cached at the same physical block address and asserts SResp Null if marked Invalid, Shared, or Exclusive. If the request hits its cache, and it is marked Modified, the snooper performs either a push or an intervention. If the system supports Shared-Intervention and the request was marked with N set to '1', then the snooper can respond Shared-Intervention and push the data.

Memory:

The memory can provide the addressed data no earlier than after SNOOPACC. The North Bridge must examine the snoop-response code, and if it was Retry or Intervention, the North Bridge should terminate the operation and deallocate the tag.

Atomic:

The Atomic modifier (TType [0]) is set along with the M bit when the read is to satisfy a cacheable copy-back **stwcx.** or **stdcx.** The master SResp retries its own RWITM-A if the reservation is subsequently cleared after issuing the RWITM but before SResp and does not reissue the RWITM-A. If a processor does not support any cache levels below it (that is, it sees all the system coherency traffic), then the A bit need not be set on RWITM.

N Bit:

The N bit is set when the master and memory are capable of intervention, intervention is wanted, and the read-data size is the coherency block size for the system. All non-block size reads must have the N bit set to zero. In addition, the processor is capable of setting N to '0' for all reads, in case the memory does not support intervention.

G and S Bits:

These bits are not defined for RWITM.

8.4.2.4 LARX-Reserve Transaction

Master:

The LARX-Reserve transaction is an address-only transaction that sets the reservation for every cache level below the level serviced by a read atomic operation. If the reservation at one level is already set to the same address as the LARX or the LARX-Reserve being propagated, then it should not be propagated further because this causes a bus operation each time the LARX is executed and might be part of a program loop.

Snooper:

Does not see the LARX-Reserve for M equals '0'.

Memory:

Ignores this operation.

8.4.3 Memory Write Transactions (General)

A master sends a write command to write data to memory or to an I/O device. The write-command packet is immediately followed by a write-data packet. The slave (North Bridge) checks the command to see if there is buffer space to store the write-data packet. The slave responds with a retry transfer-handshake packet if there is insufficient buffer space. The master can then terminate sending the write-data packet on an even beat. It can then try again to send the write-command packet and write-data packet at a later time.

The North Bridge reflects every command packet to all processors. The snoopers ignore the reflected command packet if M equals '0'. Only the original processor needs to see the address inside the command packet to deallocate the tag after the transaction is completed. At that time, the North Bridge takes responsibility for snooping for the pushed (castout) data. The transaction must be propagated all the way to memory if the W bit is asserted.

8.4.3.1 Write-With-Kill Transaction

Master:

The write-with-kill transaction is a burst operation used to tell all snoopers to invalidate any copies of this line in their caches, while also storing the line to memory.

Table 8-14. Write-With-Kill Types Supported

WIM Bits for Write-With-Kill	W Bit	M Bit
Copy back due to load, store, or Data Cache Block Set to Zero (dcbz)	0	0
I/O Write ¹	0 or 1	1
Flush due to Data Cache Block Flush (dcbf)	1	0
Push due to snoop	1	0

1. An I/O write is a full cache line write from a memory address.

Snooper:

If M equals '0', the snooper ignores this operation. If M equals '1', the snooper treats this operation as a Data Line Kill (DKILL) to the same address block, marking it Invalid (this includes any store buffers) and the operation is passed to any higher level cache.

Memory:

Memory must not update storage if the transfer-handshake packet indicates Retry or the SResp value (if applicable) is Retry.

IBM PowerPC 970MP RISC Microprocessor

8.4.3.2 Write-With-Clean Transaction**Master:**

A write-with-clean transaction is a burst operation caused by a processor executing a Data Cache Block Store (**dcbst**) instruction or a bus snoop read or clean to a modified block. It is used to tell all lower level caches that a copy still remains in this level, while updating memory (or I/O). Since no snooper has the line, it sets M to '0' so horizontal snooping is avoided. The block is written all the way to memory.

Snooper:

Snoopers should not see this operation since M equals '0'.

Memory:

Memory must not update storage if the SResp is Retry.

8.4.3.3 Write-With-Flush Transaction**Master:**

A write-with-flush transaction is a partial-block write to memory and might be a sub-block burst operation from the I/O. It is used for cache-inhibited or write-through writes from a processor (sub-block writes). The processor sources the M bit from the page table entry. I/O masters can also use this for DMA writes to a cache block without getting ownership first. The processor will set M to '1' for this transaction.

Snooper:

If M equals '1' and the line is cached Modified, this operation is SResp Retried. The line is pushed back to memory with a write-with-kill, then invalidated (this includes any store back buffers [SBBs]). The only appropriate SResp response is Retry by a snooper (other than SResp Null, which is the default response).

Memory:

Memory must not update storage if the transfer handshake indicates retry or the SResp value (if applicable) is Retry. A bus agent cannot pass a write-with-flush to an I/O bus that might contain memory-mapped devices or memory that can be reserved without first successfully passing the response window on the processor interconnect bus.

8.4.4 Command-Only Transactions**8.4.4.1 DCLAIM Transaction (Invalidate Others)****Master:**

A master issues a data line claim (DCCLAIM) transaction to service a **dcbtst**, **dcbz**, or store instruction. The DCLAIM is used to attempt to take a coherent block from the shared (or, with **dcbz**, invalid) state to the modified state and all other horizontal caches to the invalid state. It differs from DKILL in that the DClaim does not invalidate the master's copy in a lower level (higher number) cache.

Snooper:

Snoopers must invalidate their data cache blocks if there is a hit on this address.

Memory:

Ignores this operation.

8.4.4.2 Flush Transaction

Master:

A flush transaction is caused by a **dcbf** that hits on a memory coherent cache block and is marked shared or invalid. It is sent to other snoopers that might have a copy of the line.

Snooper:

If M equals '1', snoopers snoop their caches, and, if the line is cached, it is marked invalid. If the line was marked as modified, it is pushed back to memory. A snooper might respond modified, or might respond Null. An SResp Retry response should only be used if the command cannot be accepted or a pipeline address collision occurs.

Memory:

Memory might ignore this operation, even if a snooper responds SResp Modified, since intervention is not supported on the flush operation itself. The flushed data is sent to memory on a separate write-with-kill operation.

8.4.4.3 Clean Transaction

Master:

A clean transaction is caused by a **dcbst** that hits on a memory coherent cache block and is marked shared or invalid. It is sent to other snoopers that might have a modified copy of the line.

Snooper:

If the line is cached, then it is marked shared (or exclusive if it is the lowest cache in the hierarchy). If it was marked modified, the line is pushed back to memory. A snooper might respond modified, or might respond Null. An SResp Retry response should only be used if the command cannot be accepted or a pipeline address collision occurs.

Memory:

Memory might ignore this operation, even if a snooper responds SResp Modified, since intervention is not supported on the clean operation itself. The cleaned data is sent to memory on a separate write-with-clean operation.

8.4.4.4 IKill Transaction

Master:

The intent of the Instruction Line Kill (IKILL) transaction is to invalidate entries in any instruction-only caches in the system. Data only or combined caches are invalidated with other coherency operations. An IKILL block is caused by an ICBI instruction that hits on an instruction cache block that is marked as memory coherent. In order to prevent bus livelocks, this command should be issued with the P bit set to '0'.

Snooper:

Snoopers must invalidate their instruction cache blocks if there is a hit on this address. Any unified data or data only cache does not need to be snooped. A snooper might respond Retry, or might respond Null. An SResp Retry response should only be used if the command cannot be accepted due to resource conflicts.

Memory:

Memory can ignore this operation.

8.4.4.5 TLBIE Transaction

Master:

TLBIE is caused by a processor executing a TLB Invalidate Entry (**tlbie**) instruction.

Snooper:

Snoopers accept this transaction regardless of the M bit and invalidate any TLBs in the congruence class.

Memory:

Memory can ignore this operation.

8.4.4.6 TLBSYNC Transaction

The intent of the TLBSYNC transaction is to act as a barrier that forces all previous operations using invalidated TLBs to complete before the TLBSYNC completes.

Master:

The master issues the TLBSYNC transaction in response to a processor **tlbsync** instruction.

Snooper:

Snoopers must SResp Retry the TLBSYNC until all previous loads or stores and I-fetches that used any TLBs have been flushed or performed and any snooped TLBIEs are completed. A snooped TLBSYNC has the same effect on a processor that a **sync** would have if it were executed on that processor.

Memory:

Memory can ignore this operation.

8.4.4.7 SYNC Transaction

Master

A master issues a SYNC when a processor executes a **sync** instruction. The master stops processing all future instructions until all previous instructions have been completed. Then the SYNC transaction is issued to the bus, and the **sync** instruction is not completed until the SYNC transaction completes on the bus. SResp Retry will cause the operation to be repeated; SResp Null signals completion. To prevent bus live-locks, this command should be issued with the P bit set to '0', if the snooper implementation would cause resource conflict retries.

Snooper:

A snooper drives SResp Retry if there are any snoop operations pending, or cache pushes or snoop operations pending from previously snooped bus operations. Otherwise, it responds SResp Null.

Memory:

Memory signals SResp Retry until stores are performed if they can be reordered within the memory unit. Otherwise, it responds SResp Null. Memory can also respond SResp Null. The SYNC is used as a store barrier.

8.4.4.8 EIEIO Transaction

Master:

The intent of the Enforce In-Order Execution of I/O (EIEIO) transaction is to act as a barrier for all non-cacheable loads or stores that follow it. It forces all previous non-cacheable operations to complete before any non-cacheable operation issued after the EIEIO. EIEIO is caused by a processor executing an **eieio** instruction.

Snooper or Memory:

Ignore this operation.

I/O Bridge or Bus Adapter:

Accept and propagate toward memory-mapped I/O storage and do not allow any cache-inhibited storage access to bypass (if they can be reordered).

8.4.4.9 Null Transaction

Master:

A Null transaction is used by the processor to break cyclic deadlocks or prescheduled transactions that are no longer needed.

Snooper, Bus Adapter, or Memory:

Ignore.



9. Power and Thermal Management

The power-management design of the 970MP microprocessor has two primary objectives: to achieve high performance whenever it is needed and to minimize the operating power during both active and idle periods. Power-management support includes frequency and voltage scaling, dynamic power management, and power down of one core while the second core continues to operate.

9.1 Definitions

9.1.1 Full Power Mode

Full Power (Full Run mode) is the default power mode of the processor. After initialization or reset, the processor will always be in this mode. All internal units are clocked at full clock speed and are fully operational.

9.1.2 Doze Mode

This mode is entered from Full Power mode after the processing core has been quiesced, and instruction fetch and data prefetch have ceased. This mode is a power saving mode, because only the circuitry needed to provide bus snooping capability and maintain memory coherency is active.

To enter Doze mode, set `HID0[DOZE]` to '1', and then set `MSR[POW]` to '1'. When Doze mode is entered this way, it will stay in this mode until interrupted out, rather than try to transition further into Nap mode. An interrupt condition such as an external interrupt, decrementer, hard reset (*hreset*), soft reset (*sreset*), or machine check is required to return to full power.

9.1.3 Nap Mode

Nap mode provides additional power savings beyond Doze mode. In general, clocks to all internal units are switched off. Only the timer/decrementer facility, the I/O circuitry, and part of the pervasive unit are clocked and operating. The phase-locked loop (PLL) is running and stays locked to the global system clock (`SYSCLK`). The clock mesh is operating, as is the bus clock.

To enter Nap mode, first the Nap bit in Hardware Implementation Dependent Register 0 (`HID0[9]`) must be set. Then the power-management bit in the Machine State Register (`MSR[45]`) must be set. The processor will then gate its core clocks and enter Doze mode. In Doze mode, the processor will continue to snoop. However, it asserts its quiescent request (`QREQ`) signal to indicate to the chip set that it is prepared to go into Nap mode if snooping is not required. If the chip set determines that no memory activity requires the processor to snoop, it asserts a quiescent acknowledgment (`QACK`). Once the processor detects the assertion of `QACK`, it transitions to Nap mode. While in Nap mode, `QACK` is monitored constantly. If it is dropped, the processor transitions back to Doze mode.

If the processor has to act upon an incoming snoop, the bus interface unit (BIU) becomes active, and `QREQ` is deasserted. However, the processor stays in Doze mode and waits for the BIU to become idle again. As soon as the BIU is idle, `QREQ` is issued again. `QACK` can be reactivated when the snoop is completed (after the snoop-response time). The processor switches back to Nap mode after `QACK` is received.

IBM PowerPC 970MP RISC Microprocessor

If QACK is received without QREQ being sent (for example, the BIU is not idle), the processor will enter an error state. If QACK is deactivated while the processor is switching to Nap mode, the transfer to Nap mode completes before the processor is brought back to Doze mode. Any external interrupt, reset, or check condition transfers from Nap mode back to Full Power mode.

9.1.4 Deep Nap Mode

In Deep Nap mode, the chip is in the same state as Nap, except that the clock frequency in the chip is reduced to 1/64th of the nominal frequency. If this state is enabled, it is entered immediately after entering Nap mode and exited immediately before exiting Nap mode.

9.1.5 Dynamic Power Management

Dynamic power management (DPM) refers to the cycle-by-cycle control of clocks, as hardware facilities are used for computation and then go idle for some cycles. This gating of clocks while circuits are idle saves power with no reduction in performance. On a cycle-by-cycle basis, DPM enables stopping a logical function based on the need for the function. DPM also enables stopping a pipeline stage in a unit based on a change in the content of the pipeline stage.

9.2 Power-Management Support

System software manages power dissipation in a number of ways, using a number of hardware facilities.

9.2.1 Power-Management Control Bits

Dynamic power management (DPM) refers to the cycle-by-cycle control of clocks as hardware facilities are used for computation, and then go idle for some cycles. This gating of clocks while circuits are idle saves power with no reduction in performance. In normal operation, DPM should be enabled. It can be disabled, however, by negating HID0[11]. To enter an idle state, software must first set a bit in HID0 to identify which idle mode is wanted, and then set MSR[45] to trigger the transition to that mode. Setting HID0[9] selects Nap mode; setting HID0[8] selects Doze mode; setting HID0[7] selects Deep Nap mode. *Table 9-1* summarizes these power-management control bits.

Table 9-1. Power-Management Control Bits

Bit	Bit Name	Power Saving Mode
HID0[7]	deep nap	Deep nap
HID0[8]	doze	Doze
HID0[9]	nap	Nap
HID0[11]	dpm	Dynamic power management enable
MSR[45]	POW	Power management enable
MSR[48]	EE	Enable exception (interrupt)

9.2.2 Interrupts

The only way to get from a power saving mode back into the Full Power mode is by asserting one of the following interrupts:

- External interrupt
- Decrementer interrupt

Before entering a power saving mode, the MSR[EE] bit must be set to enable these interrupts. After an interrupt is taken, the return from interrupt (**rfid**) or the hypervisor return from interrupt (**hrfid**) automatically resets the MSR[POW] bit, therefore, software must set it once again to reenter a power-saving mode.

9.2.3 Bus Snooping

The processor interconnect participates in the system power management through two asynchronous control signals called QREQ and QACK. QREQ is a processor output signal that is asynchronously sampled by the local clock of the North Bridge. QACK is a North Bridge output signal that is asynchronously sampled by the local clock of the processor and other bus masters.

Figure 9-1 on page 176 is a flowchart of the sequence of steps for the processor to enter Doze or Nap mode. *Figure 9-2* on page 177 is a flowchart of the sequence of complementary steps taken by the North Bridge in response to the assertion or negation of QREQ by a processor. In Doze mode, the processor must be capable of snooping all reflected command packets from the North Bridge. In Nap mode, the processor is not required to snoop transactions, although it must be capable of returning to Doze mode for the purpose of snooping if QACK is negated.

In the normal (or preferred) sequence of events, the processor and North Bridge observe a 4-phase handshake for QREQ and QACK. The processor first asserts QREQ after the processor has quiesced, the snoopers are idle, and all outstanding processor interconnect bus transactions have completed. The processor then waits for the North Bridge to assert QACK. While the processor is waiting for the assertion of QACK, it is in an intermediate mode called Doze. Once the North Bridge asserts QACK, the processor enters Nap mode. To exit Nap mode, the processor negates QREQ and then waits for the North Bridge to negate QACK before returning to the Run state.

There are a few scenarios in which the 4-phase handshake is preempted.

1. While in Doze mode, the North Bridge reflects command packet snooping. The action taken by the processor is to negate QREQ while snooping the reflected command packet and while staying in Doze mode.
2. While in Doze mode, the processor receives an interrupt. The action taken by the processor is to negate QREQ and return to the Run state.
3. While in Nap mode, the North Bridge negates QACK while the processor has QREQ asserted. The processor must then return to Doze mode within 64 bus clocks so that it can return to snooping reflected command packets from the North Bridge.

As shown in *Figure 9-1* on page 176, the North Bridge normally negates QACK when QREQ is negated by any of the attached processors. However, it might also negate QACK if there is bus activity from any of the other attached bus devices that can be a bus master.

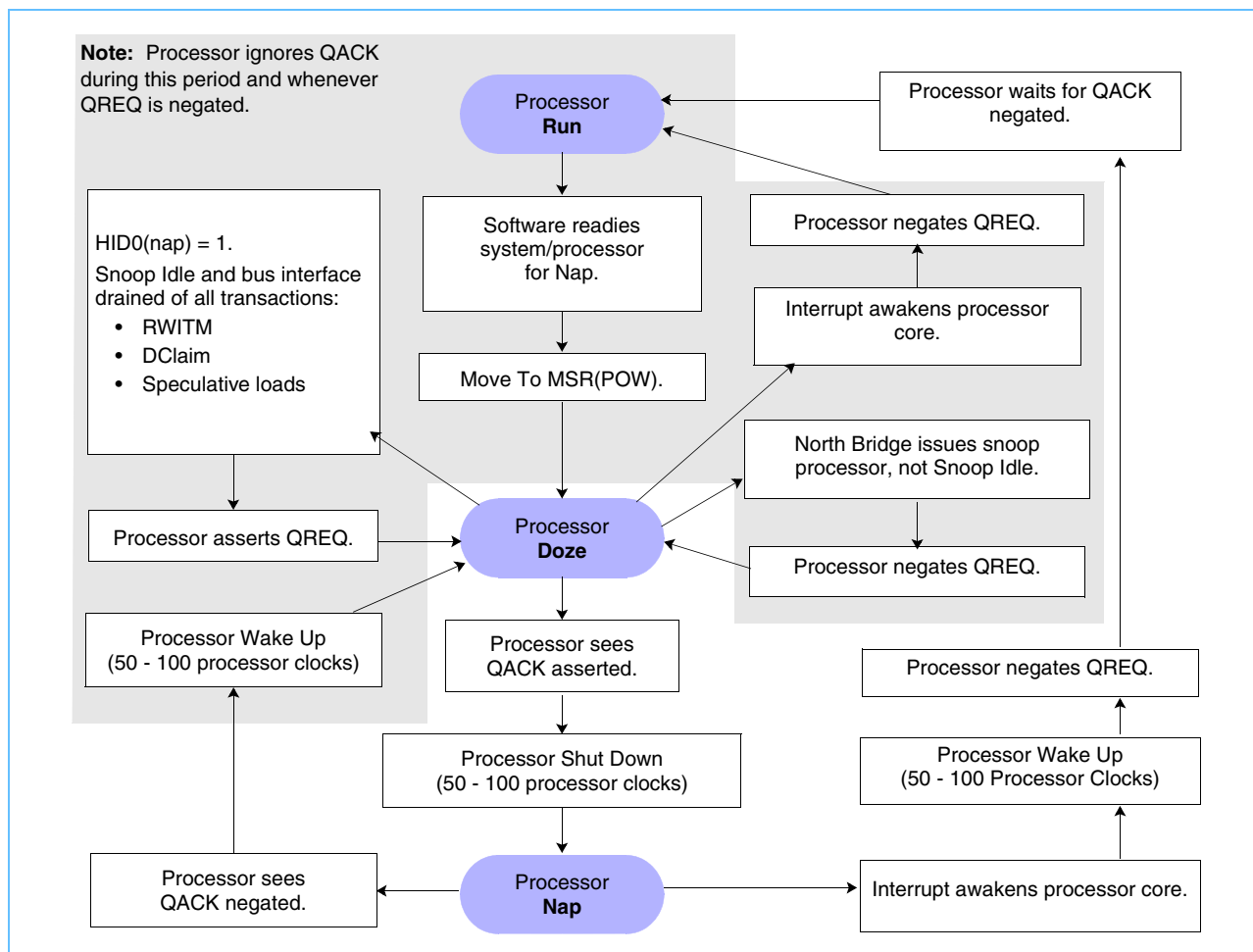
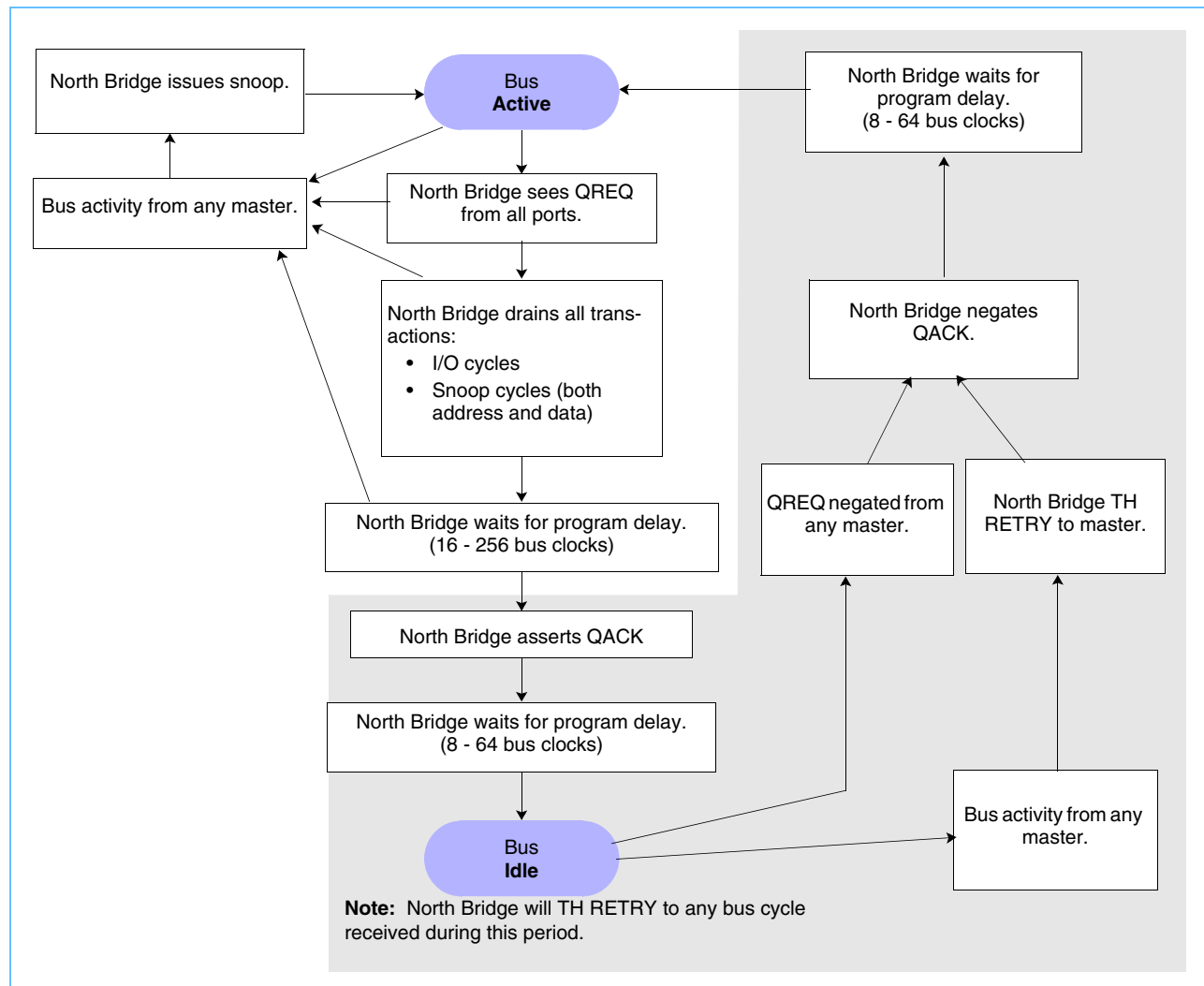
IBM PowerPC 970MP RISC Microprocessor
Figure 9-1. Processor QREQ/QACK Signalling


Figure 9-2. North Bridge QREQ/QACK Signalling



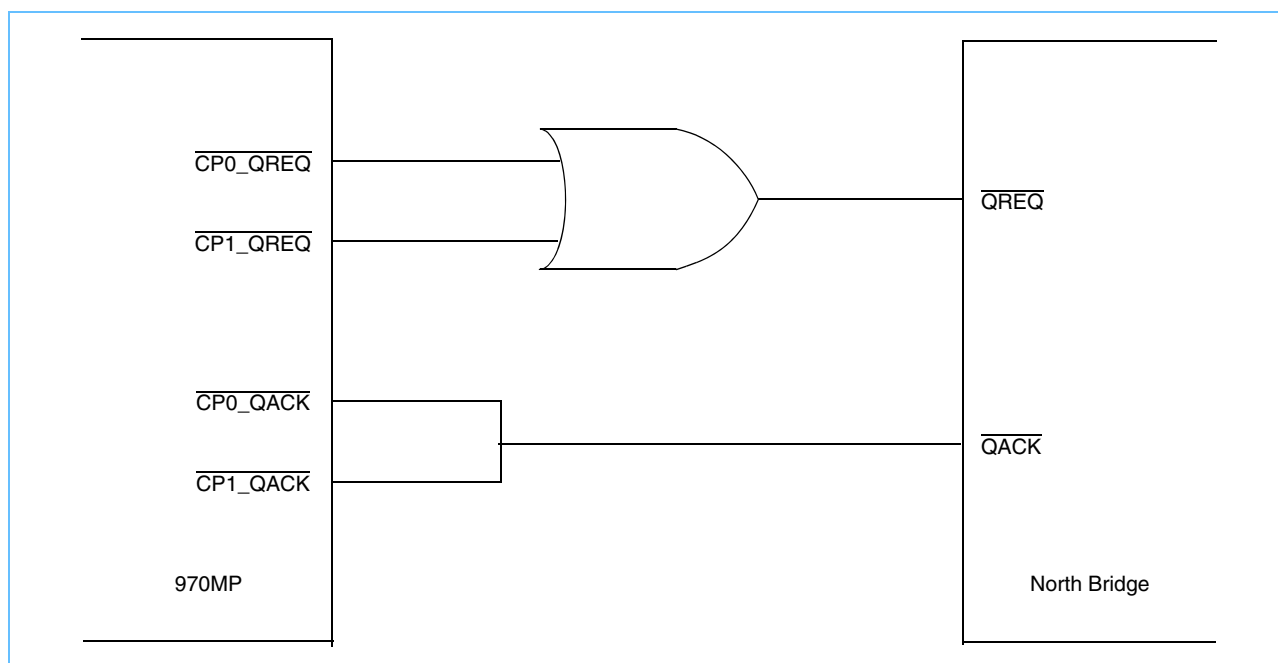
After the HID0[nap] bit is set and then the MSR[POW] bit is set, the processor enters Doze mode and asserts its QREQ signal. In this mode, core clocks are gated to reduce power, but clocks in the storage subsystem (STS) are still active to support bus snooping. The PLL, timers, and interrupt logic are also active in all the idle modes. The processor must remain in this Doze mode for as long as the system determines that snooping is required. The assertion of QREQ indicates to the system the processor's readiness to go into Nap mode. Once the system determines that snooping is not currently required, it can assert QACK. When the processor receives this signal, it completes the transition to Nap mode.

If snooping is required again, the system can negate QACK, signalling to the processor that it must transition back to Doze mode and begin snooping the bus. The general requirement is that the system must deassert QACK at least 64 bus cycles before it initiates bus activity to allow the processor to complete the transition back to Doze mode. However, the calculations below can be used to fine tune this delay. If this bus activity once again ceases, the system can assert QACK and the processor will go back into Nap mode.

IBM PowerPC 970MP RISC Microprocessor

The two pairs of QREQ/QACK signals in the 970MP design allow the two processors to independently Nap. It is possible to use the 970MP microprocessor with a North Bridge chip that supplies only a single QREQ/QACK pair. In this case, the two processors will be forced to go into and come out of Nap mode together. *Figure 9-3* shows the external logic and connectivity required to support the 970MP microprocessor with such a North Bridge.

Figure 9-3. Using a 970MP Microprocessor with a Single QREQ/QACK Pair



Because the QREQ signal is active low, an external OR gate is used to combine the two QREQ signals from the two processing units. Thus, QREQ is asserted to the North Bridge only if both processing units are asserting their QREQ signals. Once the North Bridge receives the asserted QREQ signal, its QACK signal is broadcast to both processing units by driving both QACK inputs from the single QACK on the North Bridge.

9.2.3.1 Delay Calculation

The requirement for the worst case QAckMinLowTime does not account for the case where the processor has not yet reached Nap or Deep Nap before QACK is negated. This additional delay can be accounted for by either increasing the required QAckIdleDelay or by imposing a requirement on QAckMinLowTime. Through *Table 9-2* and *Table 9-3*, below, we present the requirement as a minimum QAckIdleDelay, and a minimum combined QAckIdleDelay and QAckMinLowTime.

Table 9-2 provides the minimum QAckIdleDelay required for three different bus ratios and three different mesh clock frequencies. The required delay is 24 full frequency (f) processor clocks plus 195 mesh frequency (f, f/2 or f/4) processor clocks. Since the bus clock frequency scales with the mesh clock, the relation between these is a function of the bus ratio, but independent of frequency scaling. At 2:1, there are 4 bus clocks per mesh clock, at 3:1 there are 6 bus clocks per mesh clock, and at 4:1 there are 8 bus clocks per mesh clock. Since the 24 full frequency clocks do not scale with the mesh and bus clocks, the relation between these full frequency clocks and the bus clocks depends on both the bus ratio and the scaled frequency. At 2:1, there are 4 bus clocks per full frequency clock at f, 8 at f/2, and 16 at f/4. At 3:1, these values are 6 bus clocks per full frequency clock at f, 12 at f/2, and 24 at f/4. At 4:1, these values are 8 at f, 16 at f/2, and 32 at f/4.

Table 9-2. Minimum QAckIdleDelay requirement in bus clocks for 970MP

	2:1	3:1	4:1
f	55	37	28
f/2	52	35	26
f/4	51	34	26

Table 9-3 provides the minimum (QAckIdleDelay + QAckMinLowTime) required for three different bus ratios and three different mesh clock frequencies. The required delay is 48 full frequency (f) processor clocks plus 309 mesh frequency (f, f/2, or f/4) processor clocks.

Table 9-3. Minimum (QAckIdleDelay + QAckMinLowTime) requirement in bus clocks for 970MP

	2:1	3:1	4:1
f	90	60	45
f/2	84	56	42
f/4	81	42	41

Note: The default values for these two parameters are QAckIdleDelay = 50, and QAckMinLowTime = 6. This combination satisfies both of the requirements in the two tables shown for 3:1 mode at f/2, and therefore this combination is appropriate for use at this bus ratio and frequency configuration.

9.2.4 Thermal Diodes

Thermal diodes are placed near the hot spot on each core and brought off chip separately. External logic can be used to monitor the diode temperature of the two cores independently for managing power based on thermal limits.

9.2.5 Bus States while in Power Saving Modes

When serving snoops, the BIU is active and drives the outputs as required. When in Nap mode, there is no snooping.

- Data Out Bus, Transfer Handshake Out, and Coherence Response are driven to an idle mode.
- Clock Out is always driven with the proper clock signal.
- Clock In expects to receive a clock signal.

9.3 Software Considerations for Power Management

9.3.1 Entering Power Saving Mode

The following code sequence should be used to enter a power save mode

```
.....  
.....  
mthid0 (NAP)  
.....  
.....  
.....  
.....  
loop: dssall (VPU prefetching stop)  
sync  
mtmsr (POW)  
isync  
br    loop  
.....  
.....
```

The Data Stream Stop All (**dssall**) instruction is needed to stop those prefetch engines started in behalf of the vector processing unit (VPU) prefetches. Only the previous sequence will bring the processor into the power save mode. Switching the Move To HID0 (**mthid0**) instruction and the Move To Machine Status Register (**mtmsr**) instruction in the previous sequence does not result in a switch to a power saving mode. When an interrupt is taken, it resets the MSR[POW] bit.

9.3.2 External Interrupt Enable

Only an external interrupt or timer interrupt will bring the processor back from the power save mode. Therefore, note that MSR[EE] must be set before entering the above loop. Failing to set MSR[EE] and applying an external interrupt or a timer interrupt will result in unpredictable behavior by the processor.

9.4 Power Tuning Facility Overview

The power tuning facility is the heart of the power management for the 970MP microprocessor. It controls the power-management modes, on-chip and off-chip clock frequency, and supports voltage scaling for the 970MP microprocessor. *Table 9-4* lists the Power tuning modes supported by the 970MP microprocessor.

Table 9-4. Power-Management Modes

Static Power-Management Modes	Frequency Scaling
Full, Doze, Nap	f
Full, Doze, Nap	f/2
Full, Doze, Nap	f/4
Deep Nap	f/64
Note: 1. See the <i>IBM PowerPC 970MP RISC Microprocessor Datasheet</i> for actual power dissipation specifications.	

In the system, all processing units and the processing unit interfaces in the North Bridge change the power tuning mode (except for Deep Nap mode) concurrently. Any processing unit can request the mode change. This information is then transmitted to the North Bridge through the processor interface bus as a special request. The North Bridge grants the requests and mirrors this special request to all processing units. The North Bridge then waits for all processing units to signal that they have quiesced the bus and are ready to switch modes. The North Bridge then triggers the mode switch. It completes within 200 ns for bus ratios of 2:1, 3:1, 4:1, 6:1; and within 300 ns for a bus ratio of 12:1.

Frequency scaling on the processor interface bus requires changing the RoundTripDelay and TargetTime parameters in all the processing units and in the North Bridge. Because the I/O voltage is not changed, an initial alignment pattern (IAP) procedure is not required. The new parameters are sent along with the power tuning command; they overwrite the old parameters when the frequency switch occurs. No parameter change is required for the deep nap frequency, because there is no bus activity in this mode.

When switching power tuning modes, consideration must be given to the following items:

- Switching from high to low frequency will result in loss of accuracy and resolution in the decremter counter and will slow reaction on interrupts. The operating system has to set the decremter counter in order to prevent event and interrupt misses or queue overflow on external devices.
- Some interfaces must be running at a constant speed and voltage independent of the internal frequency and voltage (for example, the inter-integrated circuit [I²C] interface, SDRAM interface, and PCI interface).

9.4.1 Power Tuning Facility Definitions

Bus clock (Bclk)	The external bus clock has half the frequency of the data clock because of the double data rate transmission mode on the processor interconnect.
Data clock (Dclk)	The bus data clock has a frequency of 1/ <i>n</i> th of the mesh clock, where <i>n</i> is the bus ratio. Valid values for the bus ratio are 2, 3, 4, 6, and 12; with 8 and 16 supported only for test purposes.
Local clock (lclk)	The full frequency clock as delivered by the PLL, but with the same analog delay as the mesh. Every rising edge on the mesh clock has a concurrent rising edge on lclk with a small skew.

IBM PowerPC 970MP RISC Microprocessor

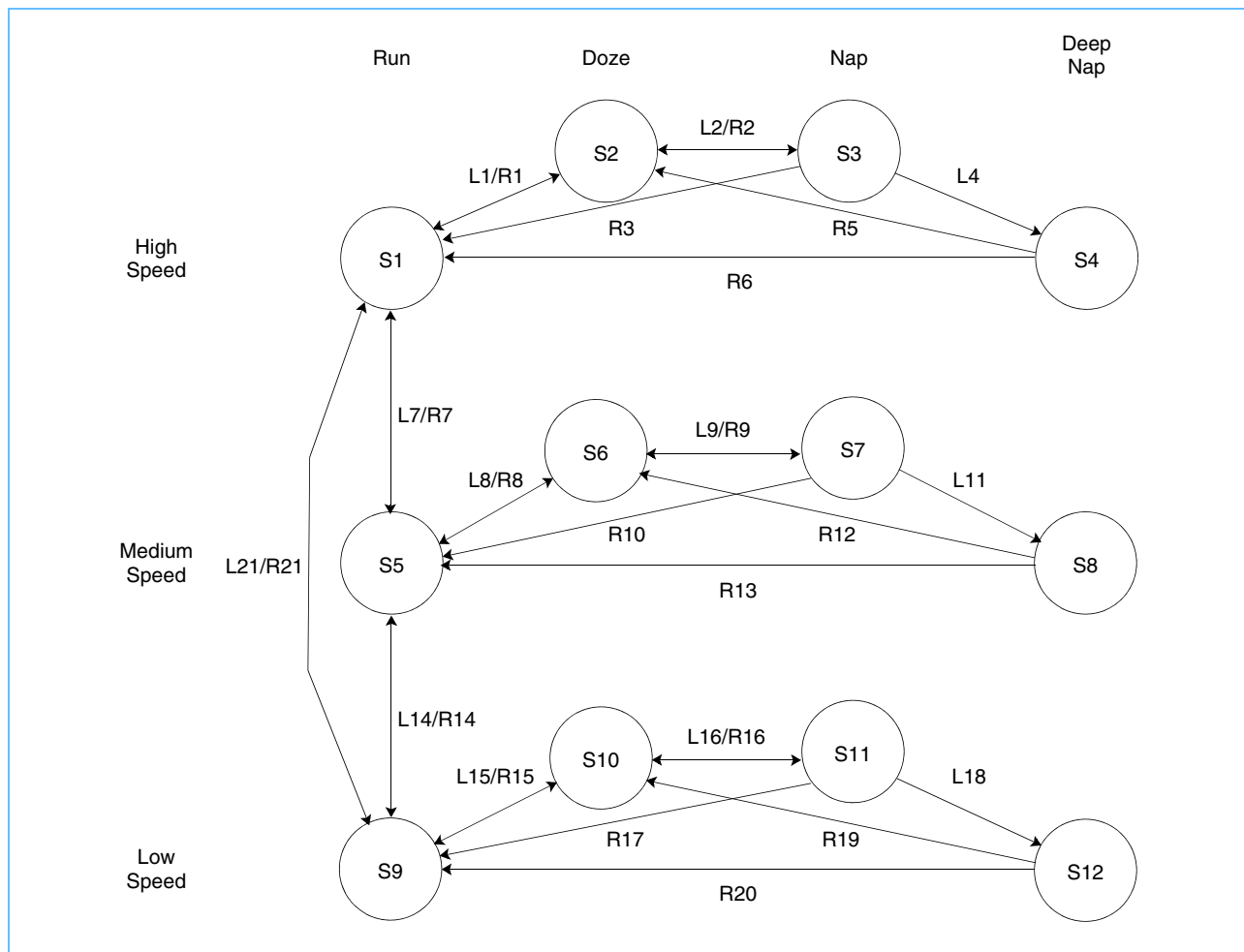
Mesh clock (mclk)	The logic behind the PLL generates full, half, or quarter frequency of the PLL clock and sends it on the mesh. The PLL also guarantees that some rising edge of the mesh clock at a latch is aligned to some rising edge of the SYSCLK when using full frequency.
PI	Processor interconnect bus (processor interface).
PLL/full frequency clock	Frequency is either 8 or 12 times the SYSCLK.
psync	A signal provided by the North Bridge that is active for one rising edge of SYSCLK every 24 SYSCLK cycles.
psync edge	A special mesh clock rising edge, aligned with a rising edge of the SYSCLK while the external <i>psync</i> is active.
SYSCLK	This is the system clock as provided on the board.
Time0	Time0 mark. A special rising edge on the bus clock, which is either concurrent to the <i>psync</i> edge or removed by two external bus clock phases (4 * x bus clocks edges). Note: Not all bus ratios are valid for all frequency modes (full, half, and quarter), considering that the external <i>psync</i> is a 24:1 SYSCLK signal, and taking the PLL multiplication factors and the Time0 definition into account.

9.4.2 Power Modes

Figure 9-4 is a state diagram showing the various power modes supported in the 970MP microprocessor and the transitions between them. Table 9-5 on page 184 identifies the power states in that diagram, and Table 9-6 on page 185 identifies the transitions labeled in the diagram.

Note: While the frequency associated with Deep Nap is 1/64 of full frequency in all cases, the three Deep Nap states are distinguished by the frequency of the processor that they transition from and to. When the processor goes into Deep Nap from full frequency, it will return to full frequency when it leaves Deep Nap. Similarly, it returns to half frequency if it came from half frequency or returns to quarter frequency if it came from quarter frequency. In order for these transitions between Run and Deep Nap to be fast, the voltage applied to the processor in Deep Nap will be the frequency required by the state it will return to. The power dissipation associated with the three different Deep Nap states will therefore not be the same, in general.

Figure 9-4. 970MP Power Mode States



There are 12 transitions that *lower* power dissipation, indicated as Lx, corresponding to left-to-right or top-to-bottom transitions in the diagram. Corresponding to nine of these is a reciprocal transition that *raises* power dissipation, indicated as Rx, and corresponding to the right-to-left or bottom-to-top direction in the diagram. In addition, there are three lower (L4, L11, and L18) and nine raise transitions (R3, R5, R6, R10, R12, R13, R17, R19 and R20) that do not have corresponding reciprocal transitions.

IBM PowerPC 970MP RISC Microprocessor

Table 9-5 describes the 12 power mode states.

Table 9-5. Power Mode States

State	Description
S1	Full Run, high speed
S2	Doze, high speed
S3	Nap, high speed
S4	Deep Nap, high speed
S5	Full Run, medium speed
S6	Doze, medium speed
S7	Nap, medium speed
S8	Deep Nap, medium speed
S9	Full Run, low speed
S10	Doze, low speed
S11	Nap, low speed
S12	Deep Nap, low speed

The three Full Run modes (S1, S5, S9), one each for high, medium, and low speed, correspond to all operating processor functions. The three Doze modes (S2, S6, S10) involve limited functionality, which include bus snooping, but not instruction execution. The timers, both decrementer and time base, continue to run during Doze modes, as does the logic for responding to interrupts. The three Nap modes (S3, S7, S11) correspond to a level of functionality below Doze, in which snooping is not supported. However, timer and interrupt logic are still active. The Deep Nap modes (S4, S8, S12) corresponds to the same functionality as Nap mode, but with the clocks running at 1/64 of full speed.

The state transitions between Run, Doze, and Nap at a given frequency are triggered as in the PowerPC 970MP microprocessor. Transitions associated with scaling the power tuning frequency are L7, R7, L14, R14, L21, and R21. These are triggered by the execution of a power tuning command, which is initiated by a write to the Power Control Register. These transitions are all summarized in *Table 9-6* on page 185 and briefly described below.

Table 9-6. Transitions between Power Modes

Transition	From	To	Trigger
L1	Run, High	Doze, High	MSR[POW] with HIO0[nap] = '1'
R1	Doze, High	Run, High	Interrupt
L2	Doze, High	Nap, High	QACK asserted
R2	Nap, High	Doze, High	QACK negated
R3	Nap, High	Run, High	Interrupt
L4	Nap, High	Deep Nap, High	HIO0[deep nap] = '1'
R5	Deep Nap, High	Doze, High	QACK negated
R6	Deep Nap, High	Run, High	Interrupt
L7	Run, High	Run, Medium	Power tuning command
R7	Run, Medium	Run, High	Power tuning command
L8	Run, Medium	Doze, Medium	MSR[POW] with HIO0[nap] equal to '1'
R8	Doze, Medium	Run, Medium	Interrupt
L9	Doze, Medium	Nap, Medium	QACK asserted
R9	Nap, Medium	Doze, Medium	QACK negated
R10	Nap, Medium	Run, Medium	Interrupt
L11	Nap, Medium	Deep Nap, Medium	HIO0[deep nap] equal to '1'
R12	Deep Nap, Medium	Doze, Medium	QACK negated
R13	Deep Nap, Medium	Run, Medium	Interrupt
L14	Run, Medium	Run, Low	Power tuning command
R14	Run, Low	Run, Medium	Power tuning command
L15	Run, Low	Doze, Low	MSR[POW] with HIO0[nap] equal to '1'
R15	Doze, Low	Run, Low	Interrupt
L16	Doze, Low	Nap, Low	QACK asserted
R16	Nap, Low	Doze, Low	QACK negated
R17	Nap, Low	Run, Low	Interrupt
L18	Nap, Low	Deep Nap	HIO0[deep nap] equal to '1'
R19	Deep Nap	Doze, Low	QACK negated
R20	Deep Nap	Run, Low	Interrupt
L21	Run, High	Run, Low	Power tuning command
R21	Run, Low	Run, High	Power tuning command

Software initiates the transition from a Full Run mode to a corresponding Doze mode by setting the MSR[POW] bit to a '1', when the HIO0[nap] bit is a '1'. This triggers the normal idle mode sequence:

- Instruction fetch quiesces.
- The BIU quiesces.
- The clocks to the core are gated.
- QREQ is asserted.

At this point, the processor is in Doze mode. If or when QACK is asserted, the clocks driving the snoop logic are gated, and the processor enters Nap mode. Once in Nap mode, if QACK is negated, the snoop logic is reactivated and the processor returns to Doze mode. From either Doze mode or Nap mode, an interrupt

IBM PowerPC 970MP RISC Microprocessor

(External, Decrementer, System Management, Performance Monitor, or Reset) will reactivate all the clocks. This returns the processor to Full Run mode, where it will execute instructions starting at the corresponding interrupt vector. This brief description of the transitions among corresponding Full Run, Doze, and Nap modes applies to all three processing speeds (high, medium, and low).

The transition to Deep Nap (L4, L11, and L18) occurs immediately after the transition to Nap, if the Deep Nap enable bit (HID0[deep nap]) is set. In this state, the processor frequency is lowered to 1/64 of its full-speed frequency. Otherwise, the processor behavior is the same as in Nap state. In particular, if QACK is negated during Deep Nap, the processor transitions into Doze mode. If an interrupt occurs while in Deep Nap mode, the processor transitions to Run mode at the previous frequency.

Under normal operation, in which both cores are powered and participating (or prepared to participate) in program execution, system software controls the power mode of each processing unit. *Table 9-7* lists valid combinations of power modes for the two processors.

Table 9-7. Valid Combinations of Power Modes

Processor 0	Processor 1
Run	Run
Run	Doze
Doze	Run
Doze	Doze
Nap	Nap
Deep Nap	Deep Nap

In all these cases, both processors are clocked at the same frequency (there is a single PLL on chip) and are powered at the same voltage. *Table 9-7* shows that both processing units can be in the same power mode, or one can be dozing while the other is running. Because the two processing units have separate QREQ and QACK signals, it is also possible for one processor to nap while the other is running (or dozing). However, this would require the napping processor to first flush its L2 cache so that it would no longer have to snoop the bus.

9.4.3 Power Transition Latencies

Each of the transitions in *Table 9-6* on page 185 has a change in power dissipation associated with it, as well as a latency for the state change. The 970MP design incorporates several mechanisms to control these transition latencies in order to reduce the induced voltage spike that would otherwise occur.

There are three situations in which the power requirements of the processor can change drastically. One occurs during Run mode, when the instruction stream being executed changes from a low activity application to a high activity application. Because the hardware cannot detect this case, the system must be designed with sufficient decoupling capacitance to control the rate of current change (di/dt) associated with this type of power transition.

The second situation is that in which the processor transitions between Run and idle (Doze, Nap, Deep Nap) modes, at a given voltage and at constant (or Deep Nap) frequency. For this case, the 970MP microprocessor implements a programmable delay, which is inserted at several points in the transition sequence when coming out of idle modes back to the Run mode. This facility is described in detail in *Section 9.4.3.1*.

The third type of power transition is that associated with frequency changes in the power tuning facility. In this case, the dithering mechanism introduced in the 970FX microprocessor is expanded to handle the higher frequency design points of the 970MP microprocessor, as well as the quarter to full frequency transition in the power tuning facility. Clock dithering in the 970MP microprocessor is described in *Section 9.8.3 Clock Dithering* on page 200.

9.4.3.1 Idle to Run Transitions

In order to reduce di/dt, the transition from Deep Nap to Run mode is partitioned into several phases, with a programmable delay incorporated within phases. In the Clock Ramping Configuration Register, a 6-bit value (ranging from 0 to 63) specifies the programmable delay. It specifies the number of cycles the processor will spend at six different stages in the transition from Deep Nap to Run mode. To make these delays nearly equal for full, half, and quarter frequency transitions, the following approach is used:

- The full six bits are used to specify the number of full frequency delay cycles.
- The high-order five bits are used to specify the number of half-frequency delay cycles.
- The high-order four bits are used to specify the number of quarter-frequency delay cycles.

For example, a value of 12 placed in the register would result in 12 full frequency, 6 half frequency or 3 quarter frequency cycles of delay. All three correspond to the same 4 ns delay on a 3.0 GHz processor. A value of 27 placed in the register would result in 27 full frequency, 13 half frequency, or 6 quarter frequency cycles of delay. These correspond to 9 ns (full frequency), 8.7 ns (half frequency), and 8 ns (quarter frequency) delays on a 3.0 GHz processor.

Note: The delay is specified in core clocks, so the absolute delay stays constant when scaling the frequency.

Table 9-8 on page 188 provides the number of full frequency clock cycles of latency in the four phases in the Deep-Nap-to-Run mode transition for each of the three mesh frequency settings.

Note that the transition from Deep-Nap-to-Run passes through the Nap and Doze states as power is gradually increased to support Run mode. The four phases are:

- Phase 1: Interrupt during Deep Nap to Nap
- Phase 2: Nap to Doze
- Phase 3: Doze to Run
- Phase 4: Interrupt presented to running processor

The parameters C_f , C_h , and C_q represent the number of full frequency cycles in the programmable delay when in full, half, and quarter frequency, respectively. These delays occur in both the Nap-to-Doze and the Doze-to-Run phases of the transition, just after (i) $C2^1$ clocks are issued every other cycle, (ii) $C2$ clocks are fully enabled, and (iii) $C1^2$ clocks are (fully) enabled.

1. $C2$ is the clock for the slave latch.
2. $C1$ is the clock for the master latch.

Table 9-8. Latency of Deep-Nap-to-Run Transitions in Full Frequency Cycles

	Full Frequency	Half Frequency	Quarter Frequency
Phase 1	44	70	122
Phase 2	$40 + 3C_f$	$80 + 3C_h$	$160 + 3C_q$
Phase 3	$24 + 3C_f$	$48 + 3C_h$	$96 + 3C_q$
Phase 4	15	30	60
Total	$123 + 6C_f$	$228 + 6C_h$	$438 + 6C_q$

For example, if the 6-bit programmable delay value is set to 12, then $C_f = C_h = C_q = 12$. The latency for the Deep-Nap-to-Run transition for full, half, or quarter frequency would be 195, 300, or 510 full frequency cycles, corresponding to 65, 100, or 170 ns at 3.0 GHz, respectively. For a programmable delay value of 27, $C_f = 27$, $C_h = 26$, and $C_q = 24$. This yields latencies for full, half, or quarter frequency of 285, 384 or 582 full frequency cycles, corresponding to 95, 128 or 194 ns at 3.0 GHz, respectively.

9.4.3.2 Exiting Deep Nap Using a Decrementer Interrupt

The timer registers (Time Base and Decrementer) are updated at a rate that depends on the mesh frequency. When the mesh frequency is sufficiently low, the timers are adjusted by more than one tick on each update in order to maintain accuracy. At such a frequency, the precision of the timers is reduced. In particular, during Deep Nap the mesh frequency will fall below that required to maintain the precision of the timers set by the time-base enable (*tben*) frequency (or the PLL frequency, in the case of internally clocked timers). The resulting loss of precision has no effect on the accuracy of the timers. It also has no visible effect on the time-base precision, because the Time-Base Register is not accessed during Deep Nap. This loss of precision can affect the latency of the processor in detecting the interrupt signal when the decrementer value goes negative.

However, the incremental latency resulting from this loss of precision in the decrementer is quite low in the 970MP microprocessor. To reduce the latency associated with exiting Deep Nap because of a decrementer interrupt, the timers are updated once every mesh cycle on the 970MP microprocessor. The additional latency for exiting Deep Nap because of the lower precision of the decrementer is the mesh clock period minus the target timer period. For example, if the *tben* input is driven at 66 MHz to clock the timers, and the full frequency processor clock is running at 1.5 GHz, the added latency is computed as follows. The mesh clock frequency in Deep Nap is 1.5 GHz / 64 or 23.4 MHz, so the mesh clock period is 43 ns. The period of the *tben* input is 15 ns. So, the added latency to exit Deep Nap because of a decrementer interrupt is $43 - 15 = 28$ ns. For higher frequency processors, this latency is less.

9.4.3.3 Frequency Transitions in the Power Tuning Facility

Transitions in the power tuning facility involve changing the mesh frequency while in Run mode. The frequency is changed from frequency one (F1), either full, half or quarter mode, to frequency two (F2), either full, half or quarter mode (but not the same as frequency one). These transitions are initiated by software changing the Power Control Register (PCR) in one of the processors, which causes a special bus transaction to the North Bridge. This transaction is reflected by the North Bridge to all processors in the system, which causes those processors to begin the transition process. Processors indicate their readiness to make the frequency switch itself by asserting QREQ to the North Bridge, and the North Bridge responds when it is ready and after it has received all the QREQs from the processors by asserting QACK to all processors. The

time it takes to get to this point in the procedure varies. It depends on activity levels in the processors and North Bridge. A typical implementation might achieve a 200 ns to 300 ns latency for this sequence when transitioning from full to half frequency on a 2.0 GHz processor.

From the time the processor receives QACK, it takes 20 additional F1 mesh clocks, plus eight full frequency clocks, to prepare for the frequency change. The frequency change itself occurs over the course of 24 or 48 cycles (selectable by a mode bit—the previous design supported only a 24-cycle dither). The frequency is dithered between F1 and F2, so that the frequency changes gradually from one to the other. These 24 or 48 cycles are always at the lower of F1 and F2. Once the frequency switch is finished, it takes 49 additional F2 mesh clocks (assuming a 3:1 bus) before the processor negates QREQ, signaling the end of the transition.

In the case of the full-to-quarter or quarter-to-full transition, the dithering described previously is between F1 and half frequency. After the 32-cycle pause at half frequency, a second dithering sequence between half frequency and F2 takes place. The rest of these transitions are the same as described previously.

The latency of the transition of the power tuning facility from QACK assertion to QREQ negation in cycles is:

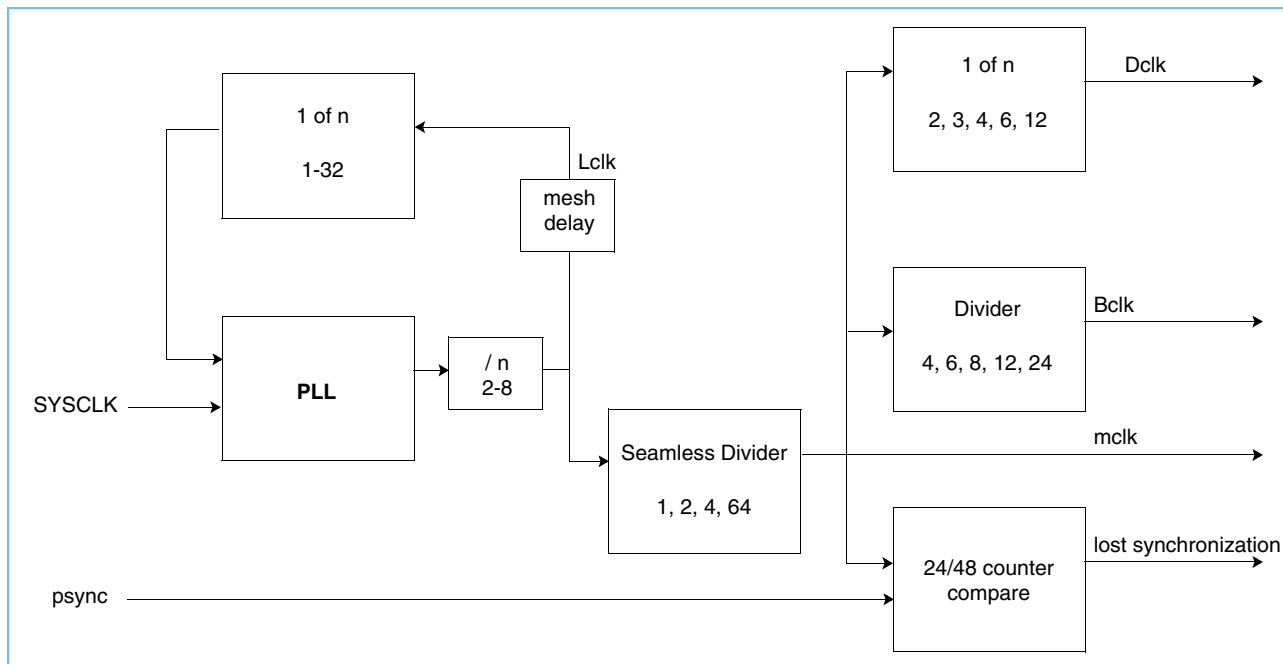
- 8 full + 20 F1 + 24 minimum (F1, F2) + 49 F2
- Plus an additional 24 minimum (F1, F2) when using a 48-cycle dither
- Plus an additional $(32 + 24) F_{\text{half}}$ when executing full-to-quarter or quarter-to-full

9.5 PLL Design

The PLL is designed to support the frequency scaling capability of the 970MP microprocessor. A single PLL drives the clock mesh, with the circuitry of the power tuning facility from PU0 controlling the mesh frequency. Both the processor clock and the bus clock are derived from the reference clock input to the chip in the 970MP design. For frequency scaling, it is assumed that the reference clock, SYSCLK, and the related synchronizing clock, *psync*, run at a constant frequency.

The PLL uses a fixed divider in the feedback path, but a variable, seamlessly switched divider in the forward path. The fixed feedback path allows the PLL to constantly run at a fixed frequency, avoiding the need to relock when switching frequencies. The processor clock (mclk) and bus clock (Bclk) frequencies can be changed seamlessly, while maintaining the ratio between these two clocks at a fixed value. *Figure 9-5* on page 190 shows this design. Note that the processor interface supports a double data rate bus. Therefore, the data rate clock (Dclk) is twice the Bclk frequency, and is constrained by the processor design to be no more than half the mclk frequency.

Figure 9-5. PLL Design



The PLL is designed to allow a feedback divider value ranging from 1 to 32, in series with an additional divide by 2 to 8 in the feedback path. The forward divider is also in series with the divide by 2 to 8. To generate **mclk** from the PLL output frequency, it has selectable values of 1, 2, and 4, plus a divide by 64 for use during Nap and Deep Nap modes. The forward divider can then generate the data rate clock from **mclk** with selectable values of 2, 3, 4, 6, and 12 (values of 8 and 24 are also available for debug, but are not supported on the processor interface bus, nor by the frequency scaling facility). Despite these many possible configurations, one constraint that limits the combinations of frequencies that can be used in the 970MP microprocessor is imposed by the *psync* counter.

The *psync* counter in the 970MP microprocessor continuously counts 24 **mclks** and then resets to zero, except when **Dclk** values of 4 and 12 are used. In these cases, the counter counts to 48. This *psync* counter is used to generate processor interconnect control signals that are synchronized with the North Bridge drivers and receivers, as mediated by the *psync* signal. Whenever a *psync* pulse is detected, the *psync* counter value is checked to be sure that synchronization is maintained. Because the *psync* pulse occurs once every 24 **SYSCLK** cycles, the **mclk** frequency is constrained to be a multiple of the **SYSCLK** frequency (an even multiple in the case of a **Dclk** divider of 4 or 12). The frequency scaling capability on the 970MP microprocessor further constrains the clock configuration values, because this *psync* counter constraint applies to the reduced-frequency, as well as the high-frequency, clock rates.

To meet the *psync* counter constraint at high, medium, and low frequencies, the only allowable divider values in the feedback path are multiples of four. With a feedback value of eight, for example, using a forward divider value of one yields the high-frequency **mclk** that is eight times the **SYSCLK**. Using a divider value of two then yields the medium-frequency **mclk** that is four times the **SYSCLK**. Using a divider value of four yields the low-frequency **mclk** that is two times the **SYSCLK**.

There are several constraints on frequency configurations for the 970MP microprocessor besides that imposed by the *psync* counter (see the *Power Management for the PowerPC 970FX RISC Microprocessor Application Note* for details).

9.6 Time-Base and Decrementer Registers

The Time-Base and Decrementer Registers run at a constant frequency, independent of changes to the processor and bus frequencies. The default operation of these timers is to run at 1/16 of the full processor frequency, even when the processor itself is running at a lower frequency. When *tben* is configured to clock these timers (HID0[19] equals '1'), the timers will run at the *tben* frequency. When the external clock input mode is used, the *tben* input frequency must not exceed the value specified in the *IBM PowerPC 970MP RISC Microprocessor Datasheet*.

Because the mesh clock frequency can be lowered to 1/64 of the full-speed, the time base and decrementer can be increased or decreased by more than one at a time. Therefore, testing that the decrementer has reached the value of zero in order to generate an internal interrupt is not sufficient. The logic detects that the counter has wrapped around. Additionally, the time resolution of the counters cannot exceed the mesh clock frequency.

9.7 I²C Bus Interface

The I²C bus interface operates at a constant speed independent of the current processor frequency.

Note: No I²C operations are supported during Deep Nap.

9.8 Frequency and Voltage Scaling

9.8.1 Frequency Scaling

Whenever an application requires less than the maximum performance available from the processor, reducing the processor clock frequency can reduce active power linearly. Furthermore, a reduction in frequency allows a reduction in voltage, resulting in an additional quadratic reduction in active power, plus a reduction in leakage.

Frequency scaling on the 970MP microprocessor involves changing the bus frequency along with the processor frequency, because of the high speed of the processor interconnect bus, and the constraint that the processor frequency be at least twice the bit rate of the bus. In order to support frequency scaling in a multiprocessor system, the North Bridge must be involved in initiating the sequence (see the *Power Management for the PowerPC 970FX RISC Microprocessor Application Note* for details).

9.8.1.1 Initiating a Frequency Change

Software initiates a frequency change by writing to the PCR. The value written to the PCR frequency field determines the target frequency being switched to. The values in the parameter fields must correspond to this new frequency. Similarly, if the voltage field is used, the voltage requested must correspond to the frequency requested. The North Bridge is responsible for changing the voltage *before* the frequency change when raising voltage. It is also responsible for changing the voltage *after* the frequency change when lowering the voltage.

IBM PowerPC 970MP RISC Microprocessor

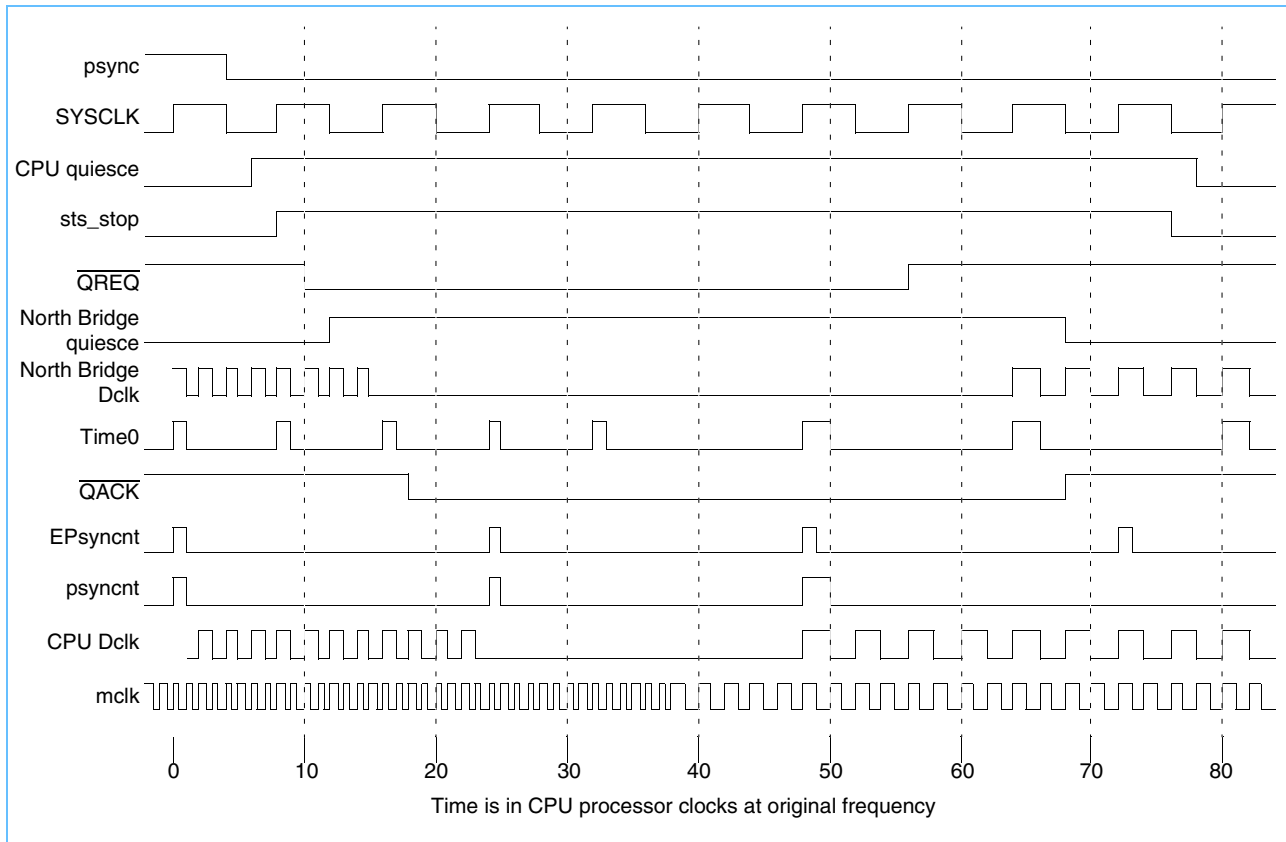
The QREQ and QACK signals have been overloaded to provide handshaking during the frequency change procedure. Therefore, these signals are not available for their normal use (handshaking for Nap mode) during the procedure. System hardware or software must enforce the negation of these signals at the beginning of the procedure. If a processor puts itself into Nap mode during the frequency change procedure, the processor blocks assertion of the QREQ signal for Nap signalling until after the frequency change is complete.

The waveforms in *Figure 9-6* on page 193 show the ordering of events on the CPU-to-North Bridge interface during a frequency change in which the clocks are slowed to half speed. The time shown at the bottom of the figure is in CPU processor clocks at the original frequency. However, this figure is intended to show the ordering of events, and not actual latencies between events. Latencies are discussed in *Section 9.8.5 Frequency and Voltage Scaling Latencies* on page 202.

The sequence in *Figure 9-6* on page 193 starts at the point after a CPU has sent the change request to the North Bridge, and the North Bridge has reflected that request to all the processors. Each CPU then completes any bus transactions in progress, and reach a quiescent state. The CPU quiesce signal shown in *Figure 9-6* indicates that the quiescent state is reached at cycle 6. Two cycles later, the CPU asserts its internal *sts_stop* signal. At this point, the core no longer has access to the L2 cache or bus. Two cycles later, the CPU asserts QREQ. During this time, the North Bridge has also been progressing toward a quiescent state. The North Bridge quiesce signal indicates that this state is reached at cycle 12, though it might occur before QREQ is asserted.

The combination of QREQ asserted and North Bridge quiescent causes the North Bridge to stop its bus clocks on a Time0 boundary, which occurs at cycle 16. Next, the North Bridge asserts QACK, shown at cycle 18. Once QACK is asserted, the CPU stops its bus clocks on the next internal *psync* boundary (*psyncnt*), shown at cycle 24. With its bus clocks stopped, the CPU changes the frequency of its processor (and therefore bus) clock, shown at cycle 38. Once the frequency change occurs, the CPU will start its bus clocks on the next *psync* boundary, shown at cycle 48. After starting its bus clocks, the CPU will negate QREQ, shown at cycle 56. The North Bridge then starts its bus clocks on a Time0 boundary (cycle 64), after which it negates QACK (cycle 68). Internal to the CPU, the negation of QACK leads to the negation of *sts_stop* (cycle 76). This enables allowing core access to the L2 cache and activity to proceed on the bus.

Figure 9-6. Frequency Scaling Event Ordering



IBM PowerPC 970MP RISC Microprocessor

9.8.1.2 Power Control Register

Software writes the PCR bits to indicate that a frequency change is wanted, and to pass the information corresponding to that frequency change to all the processors in the system. Writing to the PCR initiates the frequency change process, by generating a special bus transaction that is sent to the North Bridge and eventually reflected to all the processors. The address bits of this special transaction are copied from PCRH[22:31] and PCR[0:31] (see *Section 9.8.1.3 Power Control Register High (PCRH)* on page 196).

Note: The special bus transaction is generated when the PCR Register is written, so the Power Control Register High (PCRH) must be updated as needed before writing the PCR.

The PCR is implemented as a scan communications (SCOM) register; the odd-parity address for JTAG access is x'0AA0 0100'. The PCRH is also implemented as an SCOM register, at the same address as the PCR. The high-order bit in the register indicates which register is being written. The PCR high-order bit equals '1'; the PCRH high-order bit equals '0'. The 32-bit PCR and PCRH are written using Move To Special Purpose Register (**mtspr**) instructions that target the SCOM data (SCOMD) and SCOM control (SCOMC) special purpose registers (SPRs). Thus, the low-order 32 bits (bits 32:63) of the source register are moved to the target PCR or PCRH.

For example, before initiating a frequency change, set the following registers:

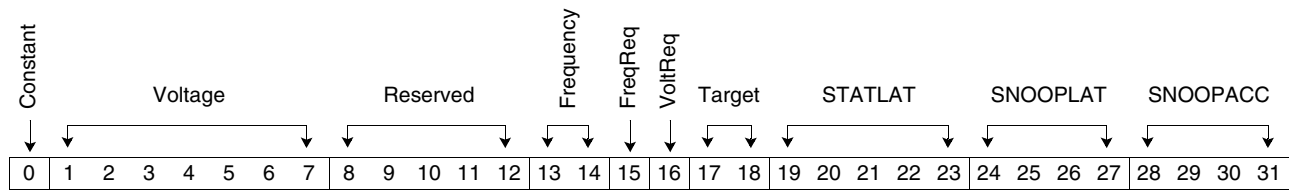
- Appropriate values in the low-order bits of gpr3 to indicate the required settings for the PCR (including bit 32 equals '1')
- Appropriate values for PCRH in gpr4 (including bit 32 equals '0')
- The SCOM address of these registers in gpr5 (to x'0000 0000 0AA0 0100')

The following sequence, where “gpr” stands for General Purpose Register, initiates a frequency change:

```
.set SCOMD 277          # SPRN for SCOMD
.set SCOMC 276          # SPRN for SCOMC
mtspr SCOMD, gpr4
isync
mtspr SCOMC, gpr5
isync
mtspr SCOMD, gpr3
isync
mtspr SCOMC, gpr5
```

Note: For the 970MP microprocessor, each frequency change should be preceded by a write to the PCR in which Gd contains all zeros. Not clearing the PCR will prevent further frequency scale commands from being issued by the bus even though the instruction sequence will complete within the processor.

Address x'0AA001'



GPR Bits	PCR Bits	Field Name	Description
0	32	Constant	Must be '1'.
1:7	33:39	Voltage	Voltage field.
8:12	40:44	Reserved	Spare field.
13:14	45:46	Frequency	Frequency field. 00 Full frequency 01 Half frequency 10 Quarter frequency 11 Illegal
15	47	FreqReq	Frequency request valid.
16	48	VoltReq	Voltage request valid.
17:18	49:50	Target	Target time.
19:23	51:55	STATLAT ¹	STATLAT is the number of bus beats between the last beat of the address/data (AD) packet and the first beat of the transfer-handshake (TH) packet.
24:27	56:59	SNOOPLAT ¹	SNOOPLAT is the number of bus beats between the last beat of a reflected command packet to the first beat of the individual snoop responses from each of the processors received at the North Bridge.
28:31	60:63	SNOOPACC ¹	SNOOPACC is the number of bus beats between the last beat of the individual snoop response sent from a processor to the first beat of the accumulated snoop response received from the North Bridge. Note: SNOOPACC is a 4-bit field. When coded with a value of 1 - 15, the actual value is x + 8. For example, a one in the SNOOPACC field is actually a nine. When a zero is coded in this field, the actual value is 24.

1. See Table 11-1 on page 281 for information about programmable delay parameters.

IBM PowerPC 970MP RISC Microprocessor

9.8.1.3 Power Control Register High (PCRH)

The Power Control Register High (PCRH) contains the high-order address field.

Address x'0AA001'



Bits	Field Name	Description
0	Constant	Must be '0'.
1:21	Reserved	Reserved.
22:31	High-Order Address	High-order address field.

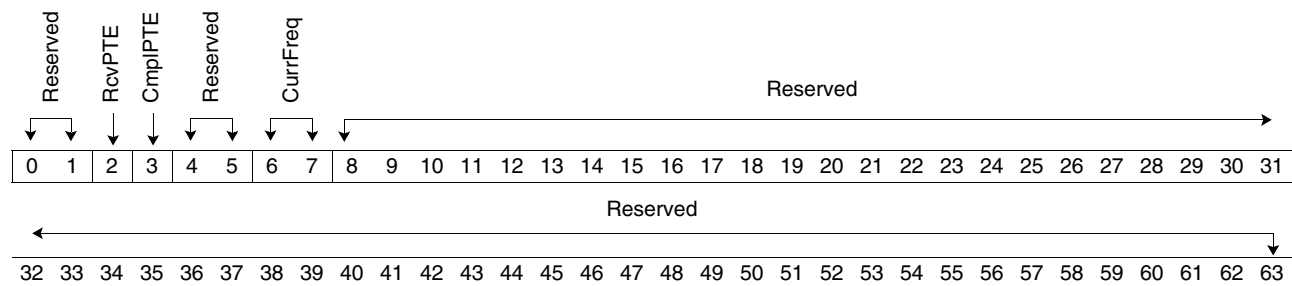
9.8.1.4 Power Status Register

The status of the power tuning facility is available in the Power Status Register (PSR). This register consists of read-only bits, indicating the current voltage (if supported by software) and the current frequency. This frequency value is valid when there is no frequency change in progress, as indicated when PSR[2] equals '0'.

When the processor receives the power adjustment special transaction reflected from the North Bridge, it sets PSR[2] to indicate that a frequency change is in progress. Shortly after the North Bridge has asserted QACK to start the frequency scale, the new frequency field is reflected in PSR[6:7]. Once the frequency scaling has completed PSR[3] is also set to '1'.

An SCOM read of the PSR once bit 2 and 3 are set will automatically clear both bits.

Address x'408001'



Bit	Field Name	Description
0:1	Reserved	Reserved.
2	RcvPTE	Power tuning command has been received.
3	CmplPTE	Power tuning command has completed.
4:5	Reserved	Reserved.
6:7	CurrFreq	Current frequency.
8:63	Reserved	Reserved.

IBM PowerPC 970MP RISC Microprocessor

9.8.2 Power Adjustment Bus Transaction

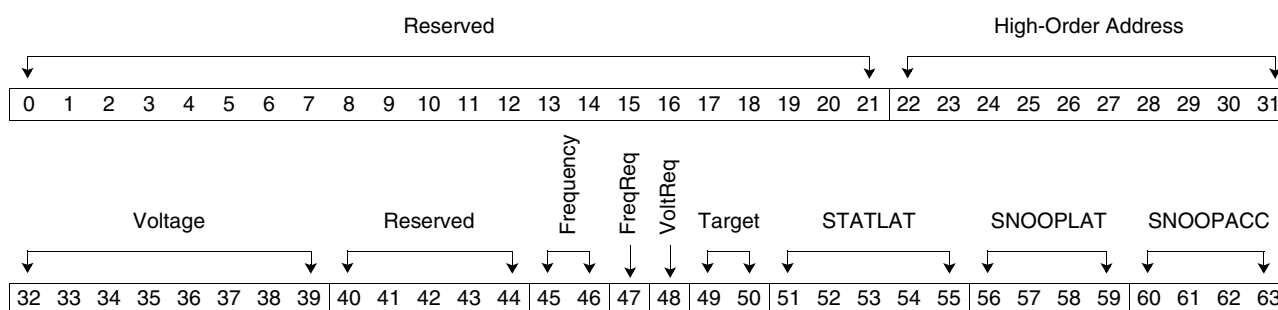
The processor sends a power adjustment transaction to the North Bridge to initiate the frequency and voltage scaling sequence in the system. This is a command-only transaction. It contains information that is encoded in a subset of the address bits to indicate the required target frequency, and the corresponding parameter information. *Table 9-9* shows the transaction type and related bus signals for this transaction.

Table 9-9. Power Adjustment Transaction

Bus Operation	Power Adjustment
Transaction type	0 0101 (x'05')
Address modifiers (WIMGRP) ¹	00 1000
Tag field	1 1011

1. W = write through, I = cache inhibited, M = memory coherent, G = guarded read, R = rerunning, P = pipelined snoop

The encoding of the address bits for this transaction is as follows:



Bits	Field Name	Description
0:21	Reserved	Not implemented.
22:31	High-Order Address	High-order address bits.
32:39	Voltage	Voltage field. The 970MP microprocessor does not use this field.
40:44	Reserved	Spare field.
45:46	Frequency	Frequency field. 00 Full frequency 01 Half frequency 10 Quarter frequency 11 Illegal
47	FreqReq	Frequency request valid.
48	VoltReq	Voltage request valid.
49:50	Target	Target time.

1. See *Table 11-1* on page 281 for information about programmable delay parameters.

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
51:55	STATLAT ¹	STATLAT is the number of bus beats between the last beat of the AD packet and the first beat of the TH packet.
56:59	SNOOPLAT ¹	SNOOPLAT is the number of bus beats between the last beat of a reflected command packet to the first beat of the individual snoop responses from each of the processors received at the North Bridge.
60:63	SNOOPACC ¹	SNOOPACC is the number of bus beats between the last beat of the individual snoop response sent from a processor to the first beat of the accumulated snoop response received from the North Bridge. Note: SNOOPACC is a 4-bit field. When coded with a value of 1 - 15, the actual value is $x + 8$. For example, a one in the SNOOPACC field is actually a nine. When a zero is coded in this field, the actual value is 24.
1. See Table 11-1 on page 281 for information about programmable delay parameters.		

These 42 low-order address bits are copied from the corresponding fields in the Power Control Register. Software can set bits 22 to 31 to any desired value to make the address fall within some desired range. Furthermore, if the voltage field is unused (voltage request valid is negated), bits 32 to 39 and the spare bits (40 to 44) can also be set to any desired value by software.

The North Bridge uses the frequency field to determine what new frequency is being requested. The frequency-request valid bit must be asserted if a frequency change is being requested. The presence of this bit allows the option of a voltage-change-only request. If the frequency request valid bit is negated, the North Bridge will not reflect this transaction to the processors. It is illegal to issue a frequency scale request to '11' or to the same frequency scale factor (that is, to issue a frequency scale command to full when it is already full). The North Bridge will not reflect these transactions to the processors.

Once the transaction is reflected to the processors, each processor responds as follows:

- If the frequency field indicates no change, the processor does nothing.
- If the frequency field indicates a change to the current frequency, or a change to a new frequency, then the processor must execute the frequency change procedure.

In addition to the four parameters passed in the power adjustment transaction, the processor interconnect also depends on the values of the programmable bit line and clock delays that are determined during the IAP at power-on. To support frequency scaling, this IAP must be run at the high-frequency, high-voltage setting for the processor. Then, the effect of running at lower frequencies is to widen the signal eye. However, the effect of lowering the core voltage while the I/O voltage remains constant is to increase all the bit and clock delays. Thus, once the IAP establishes the minimal bit skew and clock centering required for the interface to run at high frequency and high voltage, these same settings should also support the lower frequencies and voltage. Therefore, there is no facility for changing these delay values during a frequency or voltage change.

IBM PowerPC 970MP RISC Microprocessor

9.8.3 Clock Dithering

Input current to the processor can change significantly during transitions of the power tuning frequency. These current changes must be controlled to avoid over and under voltages that a high di/dt might cause because of the inductance in the power distribution network. A clock-dithering mechanism included in the power tuning facility enables gradually transitioning between frequencies.

The power tuning facility supports frequency scaling with a constant-frequency PLL that feeds multiple frequency dividers. The outputs of these dividers are fed to a frequency multiplexer, from which one divider output is selected as the processor mesh clock at any given time. Toggling this multiplexer-selection signal during a transition from frequency A to frequency B accomplishes clock dithering. Thus, most clocks are at frequency A at the beginning of the transition. Gradually, more and more frequency-B clocks are introduced in the dithering pattern.

Figure 9-7. Clock Dithering Block Diagram

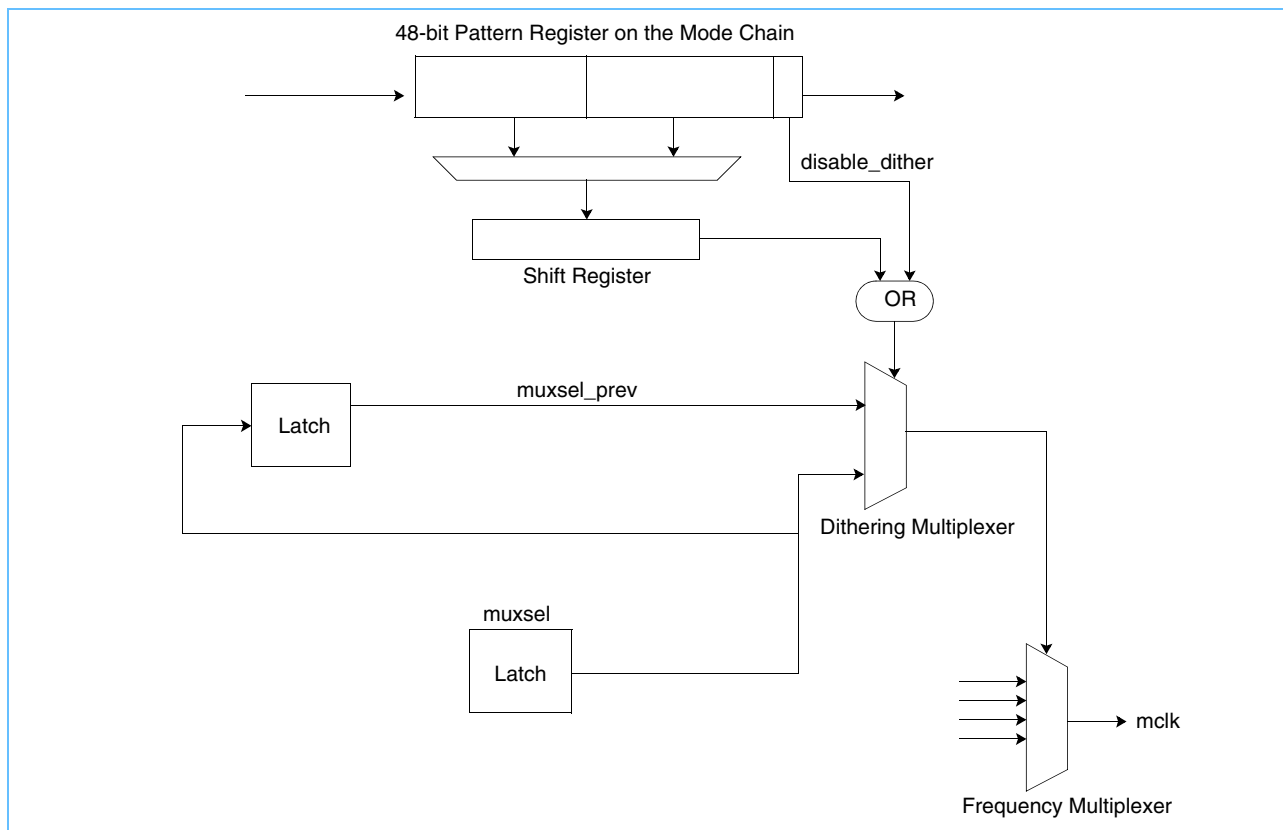


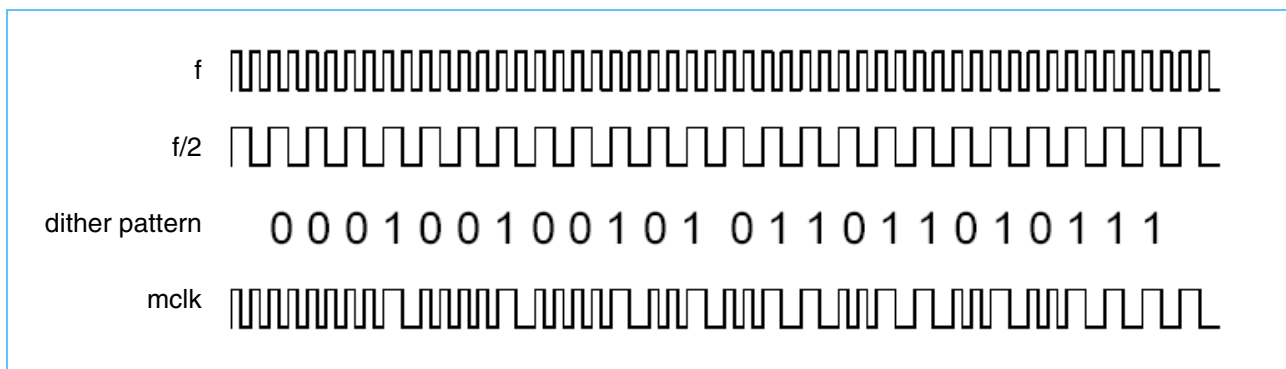
Figure 9-7 shows the components controlling the dithering of the clock. Two dithering patterns, selectable as either 24 or 48 bits in length, are provided in the mode ring. They support distinct dithering patterns for transitions between the high and medium frequencies and the transitions between the medium and low frequencies. When a frequency shift is initiated, the appropriate mode ring pattern is selected using a multiplexer for transfer to a 24-bit shift register. At the same time, the multiplexer select pattern for the previous frequency is saved in the *muxsel_prev* latch, while the new frequency is loaded into the *muxsel* latch.

Clock dithering involves a 2-level multiplexer selection process. The Shift Register is clocked at the lower of the previous and new frequencies. Starting on the rising edge of the $mclk/4$, it shifts the pattern one bit to the right every cycle. Then it applies the right-most bit to the dithering multiplexer to select a multiplexer-selection

pattern. That pattern is then applied to the frequency multiplexer to select the mesh clock frequency. A '1' bit in the shift pattern selects the new frequency; a '0' bit selects the old frequency. At the end of the shift pattern, a '1' bit is forced to continuously select the new frequency. A separate mode-ring bit can be used to disable clock dithering by forcing this control bit to always be a '1' through the OR circuit shown in *Figure 9-7*.

As an example of a shift pattern for achieving a gradual transition from high to medium frequency might be '1110 1011 0110 1010 0100 1000' (see *Figure 9-8* on page 201). These bits are shifted at the medium frequency. Each '1' corresponds to one cycle of medium frequency. Each '0' corresponds to two cycles of high frequency (because the Shift Register is clocked at medium frequency). Thus, reading the pattern from right to left, the pattern specifies six fast clocks, followed by one medium clock, followed by four fast clocks, followed by one medium clock, and so on, as indicated in the *Figure 9-8*.

Figure 9-8. Sample Shift Pattern



9.8.4 Voltage Scaling

To take the greatest advantage of frequency scaling, it is desirable to vary the voltage to match processing requirements. In operational modes, when the frequency is reduced, the voltage can also be reduced to realize a quadratic reduction in active power. When the voltage is changed with frequency, the voltage change must precede frequency increases, and must follow frequency decreases.

The processor supports the integration of voltage and frequency scaling as currently described. However, the software and system designers might choose to control the processor voltage independently of the power tuning facility. In that case, the software and system will be responsible for the sequencing and timing of voltage changes with respect to frequency changes (see the *Power Management for the PowerPC 970FX RISC Microprocessor Application Note* for details).

9.8.5 Frequency and Voltage Scaling Latencies

The sequence for raising the voltage and frequency has a latency from the time the operating system writes the configuration value in the PCR to the time when the status bit in the PSR indicates that the change is complete. That latency has the following components:

- Time to signal North Bridge
- Time to raise voltage
- Time to signal processors
- Time for North Bridge and processors to quiesce
- Time for North Bridge and processors to handshake
- Time for one *psync* (1:24) cycle
- Time to handshake and reset the status bit

The latency for lowering the frequency and voltage is similar. However, the voltage is lowered after the frequency, and processing does not need to wait for that to occur.

While the processor signals the frequency change to the North Bridge, and until the North Bridge reflects the power adjustment command to the processor, it proceeds normally. Once the processor begins to quiesce the bus, the processor core will no longer be able to access data and instructions from the L2 or bus interface. As long as the processor is able to execute with data and instructions in the L1 caches, it can continue to run. In the best case, the processor will only stall for about a cycle when the mesh clock frequency itself is switched. More specifically, the processor will be unable to respond to interrupts while the bus interface is in a quiescent state, unless the instructions and data needed to handle the interrupt are in the L1 caches before the frequency change. This means the interrupt response might be delayed because of a frequency switch (see the *IBM PowerPC 970MP RISC Microprocessor Datasheet* for latency values).

9.9 Reducing Clock Mesh Power

There are two power saving modes defined for the 970MP microprocessor, Nap and Doze. In Nap mode, the clocks to the core are turned off, while the timers, PLL, and part of the pervasive unit continue to operate. Doze mode is similar, except that snoop logic is also active. Doze mode is entered from Full Power (Full Run) mode by setting HID0[nap], and then the MSR[POW] bit. This also causes the QREQ signal to be asserted, requesting that the North Bridge put the bus in a quiescent state. When the North Bridge complies, it asserts QACK, causing the processor to transition into Nap mode. Whenever QACK is negated, the processor must return to Doze mode to process snoop transactions.

9.9.1 Power Saving in Deep Nap

When the processor is in Nap mode, the core is inactive, and clocks are gated at local clock buffers (LCBs). However, the clock distribution mesh itself continues to be clocked, dissipating significant power. To reduce the active power during Deep Nap mode, the processor clock is divided down to a very low frequency. The frequency is then be brought back up to its functional level as the processor transitions out of Deep Nap mode.

The frequency switching for Nap mode is completely under hardware control. When enabled, the frequency switch takes place after the clocks have been gated and subsequent to detecting that QACK has been asserted. Entering low-frequency, Deep Nap mode takes only one cycle longer than entering Nap mode without changing frequency. During low-frequency Nap mode, the bus clocks are disabled, and the bus signals are driven with the null transaction pattern.

As with normal Nap mode, negation of QACK or detection of an external (or decrementer, *hreset*, *sreset*, or machine check) interrupt causes the processor to leave low-frequency Nap mode. The time required to exit low-frequency Nap mode is longer than the time to exit normal Nap mode, because of the frequency change and corresponding synchronization required. Latencies for the transitions from Deep Nap to Doze and Full Run modes can be found in the *IBM PowerPC 970MP RISC Microprocessor Datasheet*.

HID0[deep nap] controls whether the clock frequency is reduced during Nap mode. When the bit is asserted, the processor will transition to Deep Nap mode immediately after entering Nap mode.

9.10 Additional Dynamic Power Management

The 970MP microprocessor implements dynamic power management—the gating of clocks to idle circuits while in an operational mode—in a number of functional units. For example, there are two levels of clock control for the VPU, a coarse level and a fine level. The coarse control is essentially a static form of clock gating control, making use of the vector processor available bit (MSR[VP]). When this bit is a zero, the latches in all VPU stages from issue to writeback are gated off. The fine level control is much more dynamic. It occurs on a stage-by-stage basis within each execution pipeline, starting with the latches following stage 2 of the Vector Register File (RF2). When this fine level of control is enabled, all clocks in all of the VPU stages from the register access to the write back are gated off at all times. The only exceptions are cycles during a stage that has active instructions.

Clock gating has been implemented in the VPU, IDU, STS, ISU, FXU, FPU and pervasive units. Because DPM has no negative impact on performance, it should always be enabled. For test purposes, DPM can be disabled as follows:

- For the VPU, IDU, and STS units, DPM is disabled by negating HID0[DPM].
- For the ISU, FXU, FPU, and pervasive units, setting bit 0 in the Dynamic Power-Management Options Register (x'000800') to a '1' disables DPM.

10. 970MP Performance Monitor

The 970MP microprocessor has a complex, speculative, out-of-order execution core coupled with an equally complex multilevel storage hierarchy. Users concerned with performance analysis and system optimization have access to performance monitoring features, which support a wide range of tasks which include:

- Profiling memory hierarchy behavior and tuning system algorithms to optimize scheduling, partitioning, and structuring for tasks and data
- Tuning applications for the target system
- Debugging, analyzing, and optimizing processor architecture features

The performance monitor facility provides information for a wide variety of activities and is part of the facilities that are collectively referred to as instrumentation facilities. Instrumentation facilities include matching/sampling, tracing, and thresholding.

Note: The 970MP performance monitor should only be used as a debug facility until characterization of its features and functions is complete.

10.1 Instrumentation Facilities Overview

The 970MP performance monitoring facility is an extension to that of earlier PowerPC processors. There are eight Performance Monitor Counter Registers (PMC1-8). They can count a variety of events, many of which are relevant to performance analysis. As before, the counters support user or supervisor and marked or unmarked filtering of events. A marked instruction is one that is eligible for sampling as determined by the instruction fetch unit (IFU) and instruction dispatch unit (IDU) instruction matching facilities.

The most-significant change introduced by the 970MP performance monitor is the concept of indirect events. A subset of the normally selected direct Performance Monitor Counter (PMC) events are multiplexed so that there is a larger number of total available events. Unlike event selection on previous PowerPC processors (which had only direct events), indirect events cannot be configured entirely independently (setting a multiplexer affects the indirect events on more than one PMC). Some indirect events can also be summed together by the hardware. This feature is most often used to sum the performance event counts of a functional unit pair (for example, floating-point unit 0 [FPU0] and floating-point unit 1 [FPU1]).

IBM PowerPC 970MP RISC Microprocessor

10.1.1 Performance Monitor Facilities

The instrumentation performance monitor (perfmon) on the 970MP microprocessor includes the following functions:

- Counts up to eight concurrent software selected events in individual 32-bit counters. The counting of events can be enabled by software under several conditions such as user (problem) or supervisor (privileged) state, and Run or Wait state.
- Generates a maskable exception when an event counter overflows (triggering).
- Freezes the contents of the event counters until a selected trigger occurs and then begin counting (triggering).
- Increments the event counters until a selected trigger occurs and then freezes counting (triggering).
- Monitors classes of instructions selected by the instruction matching facility.
- Randomly chooses an instruction for detailed monitoring (sampling).
- Counts start/stop event pairs that exceed a selected timeout value (thresholding).

10.1.2 Performance Monitor Event Selection

One event per counter can be selected for monitoring at a given time. The event to be monitored is selected by setting the appropriate value in the Monitor Mode Control Register (MMCR) bit field for that counter. The events counted might be the number of cycles that the event occurs or the number of occurrences of the event depending on the particular event selected.

10.1.3 Machine States and Enabling the Performance Monitor Counters

Performance monitor counting can be enabled or disabled under several machine states, which are selected using the counting control bit fields in the MMCRs and the state bits in other Special Purpose Registers (SPRs).

10.1.4 Trigger Events and Enabling the Performance Monitor Counters

Certain kinds of conditions and events, called trigger events, can be used to control performance monitor activities such as starting or stopping the counters and causing performance monitor exceptions. These scenarios are selected using the condition/event enable bits fields and the exception enable bits of the MMCRs in conjunction with control bits in other SPRs.

10.1.5 Performance Monitor Exceptions

Trigger events can cause performance monitor exceptions to occur based on the values of the exception enable bits in the MMCRs. An enabled exception might cause a performance monitor exception to occur if the exception is enabled in other SPRs.

10.1.6 Sampling

The 970MP microprocessor can be configured to sample instructions for detailed monitoring. The 970MP microprocessor instrumentation facilities support setting mask values for matching particular instructions or kinds of instructions that are then eligible to be sampled (that is, they are marked for sampling). The performance monitor includes events for counting marked instructions at each stage of the pipeline and in certain other situations. Instruction sampling is a useful facility for gathering both detailed and statistical information for particular instructions.

Note: Instruction marking is entirely separate from thread marking with the performance monitor mode bit in the Machine State Register (MSR[PMM]). The state of the MSR[PMM] bit is only relevant for event counting in order to determine when counters should be frozen (MMCR0[FCM1, FCM0] fields).

10.1.7 Thresholding

Unlike previous PowerPC processors, which implemented thresholding only on load instructions, the 970MP processing unit monitors the pipeline stage progression of sampled instructions and can detect when the stage-to-stage cycle count for a selected start/stop pair of pipeline stages exceeds a specified threshold value.

10.1.8 Trace Support Facilities

The 970MP microprocessor supports both the single step and the branch trace modes as defined by the PowerPC Architecture.

10.2 Instruction Sampling Facilities

10.2.1 Special Purpose Registers and Fields Associated with Instrumentation

The 970MP microprocessor instrumentation facilities and associated 970MP microprocessor components include several SPRs used for or associated with performance monitoring, matching, sampling, and tracing. Unless otherwise noted, the Special Purpose Registers described below and listed in *Table 10-1* on page 209 can be read in user (problem) and supervisor (privileged) state by using the Move From Special Purpose Register (**mf spr**) and written in supervisor state by using the Move To Special Purpose Register (**mt spr**) instructions. The MSR Register is read and written by the Move From Machine State Register (**mf msr**) and Move To Machine State Register (**mt msr**) instructions.

The 970MP microprocessor instrumentation facilities include the following Special Purpose Registers and register bit fields (also listed in *Table 10-1* on page 209):

- Performance Monitor Mode Control Registers (MMCRx)
These registers include both counting control and event select bit fields.
- Performance Monitor Counter Registers (PMCx)
These registers increment each time (or cycle, depending on the selected event) that an event occurs while the counter is enabled. These registers also have the control function for the counter overflow condition.
- Machine State Register [EE] (MSR[EE])
This register bit is used to enable or disable external interrupts. The performance monitor exception is considered an external interrupt.

IBM PowerPC 970MP RISC Microprocessor

- Machine State Register [PMM] (MSR[PMM])
This register bit is used to enable or disable performance monitor activity controlled by the process mark bit.
- Machine State Register [PR] (MSR[PR])
This register bit is used to establish user (problem) or privileged (supervisor) mode and the performance monitor counting activity controlled by this bit.
- Machine State Register [SE] (MSR[SE])
This register bit is used to enable or disable the trace exception after each instruction is completed.
- Machine State Register [BE] (MSR[BE])
This register bit is used to enable or disable the branch trace exception and after a branch instruction is completed.
- Hardware Implementation-Dependent Register0[13] (HID0[TG])
This register bit is used to determine the granularity the thresholder uses for counting cycles.
- Control Register[31] (CNTL[31])
This register bit is used to determine the Wait or Run state and the performance monitor activity controlled by this bit.
- Scan Communication Register x'240' [0:15] (SCOM x'240' [0:15])
These register bits are used to establish the timeout and resume delays used by the performance monitor to coordinate the matching and sampling facility.
- Scan Communication Register x'340' [11:12] (SCOM x'340' [11:12])
These register bits are used to establish the matching and sampling filter mode used by the matching and sampling facility to produce marked instructions that can be counted by the performance monitor.
- Instruction Match Content-Addressable Memory (CAM) Registers (IMC)
The IMC SPRs are used to access the IMC array that contains the mask values used for instruction matching. The Move To IMC (**mtimc**) and Move From IMC (**mfimc**) instructions can be executed only in supervisor mode.
- Time-Base Register [47, 51, 55, 63] (TB[47, 51, 55, 63])
These register bits are used to enable or disable the time-base events that can be used to enable or disable performance monitor counting.
- Sample Address Registers (SxAR)
The Sampled Instruction Address Register (SIAR) contains the address and the Sampled Data Address Register (SDAR) contains the data relating to a marked instruction. The registers can be read in supervisor (privileged) or user (problem) state, but are modified only by the hardware. The values written to these registers by the hardware depend on the processing state and on the kind of instruction that is being marked for sampling.
- Machine Status Save/Restore Register (SRR0, SRR1)
These registers are used to save machine status during exception handling. In addition, SRR1[33] is used to determine when the contents of the SIAR and SDAR Registers are synchronized, so that they refer to the same marked instruction.

Table 10-1. 970MP Performance Monitor and Trace-Related Special Purpose Registers

Register Name	SPR Address Bits ¹		Function
	5:9	0:4 ²	
MMCR0	'11000'	'n1011'	Performance Monitor Mode Control Register 0
MMCR1	'11000'	'n1110'	Performance Monitor Mode Control Register 1
MMCR4	'11000'	'n0010'	Performance Monitor Mode Control Register A
PMC1	'11000'	'n0011'	Performance Monitor Counter Register 1
PMC2	'11000'	'n0100'	Performance Monitor Counter Register 2
PMC3	'11000'	'n0101'	Performance Monitor Counter Register 3
PMC4	'11000'	'n0110'	Performance Monitor Counter Register 4
PMC5	'11000'	'n0111'	Performance Monitor Counter Register 5
PMC6	'11000'	'n1000'	Performance Monitor Counter Register 6
PMC7	'11000'	'n1001'	Performance Monitor Counter Register 7
PMC8	'11000'	'n1010'	Performance Monitor Counter Register 8
MSR[61]	Use mtmsr , mfmsr instructions (supervisor [privileged] mode only)		Machine State Register [Performance Monitor Mark]
MSR[48]			Machine State Register [External Interrupt]
MSR[49]			Machine State Register [User (Problem)/Supervisor (Privileged) State]
MSR[53]			Machine State Register [Single-Step Trace Enable]
MSR[54]			Machine State Register [Branch Trace Enable]
HID0[13]	'11111'	'10000'	Hardware Implementation-Dependent Register 0 [Threshold Granularity]
CTRL[31]	'00100'	'n1000'	Control Register [Run Bit]
SCOMC	Use mtscmc/d and mfscmc/d instructions		Scan Communication Control
SCOMD			Scan Communication Data
IMC	Use mtimc , mfimc instructions (supervisor mode write, user and supervisor mode read)		Instruction Match CAM Register
TBL [47,51,55,63]	'01000'	'n1100'	Time-base bits used for performance monitor time-base events
SIAR	'11000'	'n1100'	Sampled Instruction Address Register
SDAR	'11000'	'n1101'	Sampled Data Address Register
SRR1	'00000'	'n1011'	Machine Status Save/Restore Register 1

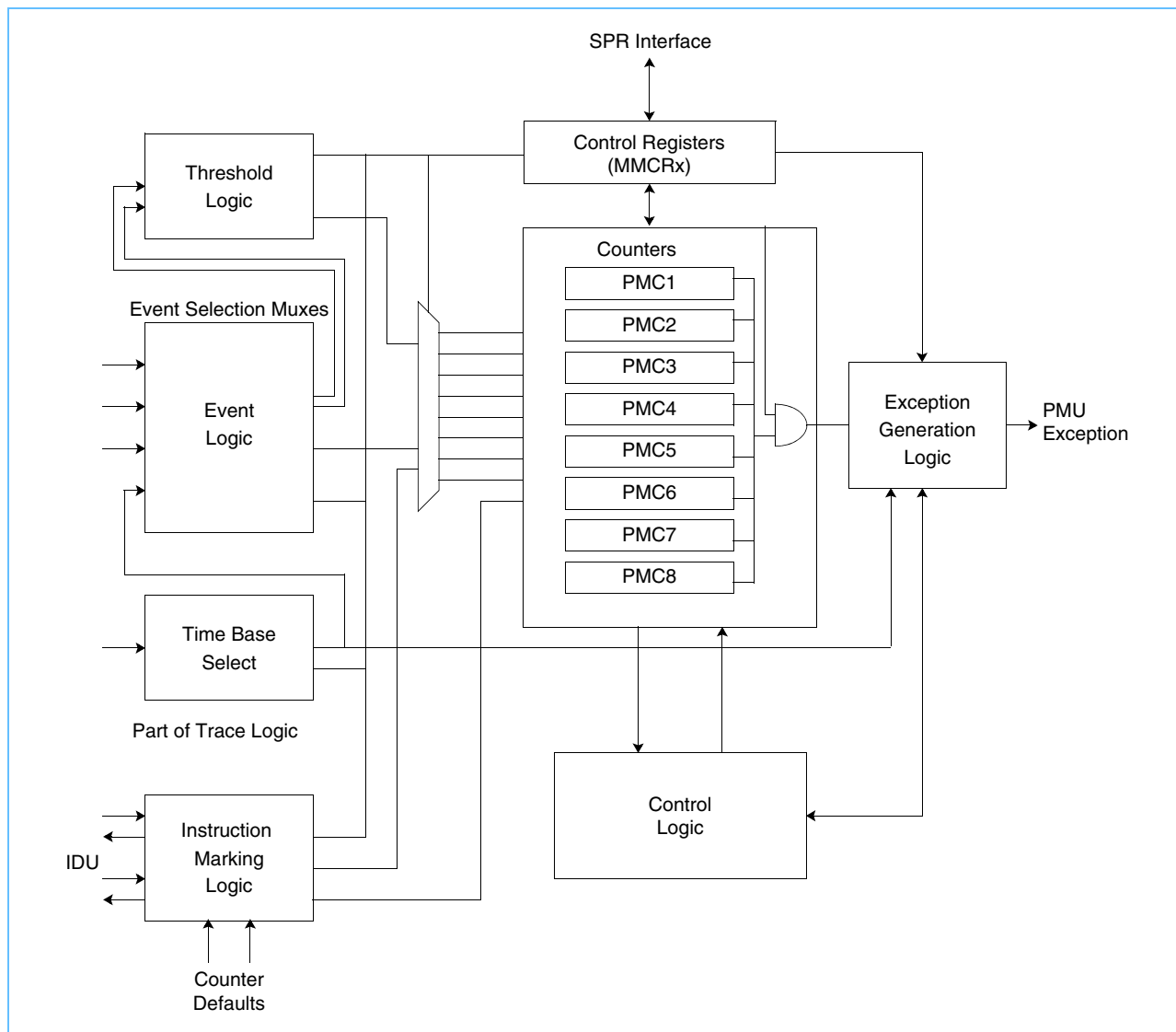
Note:

1. In a **mtspr/mfspr** instruction, the instruction SPR field of bits 11:15 hold SPR address bits 0:4 and bits 16:20 hold SPR field bits 5:9.
2. When n is set to '1', it indicates an SPR address value for a supervisor mode **mtspr** or **mfspr** instruction. When n is set to '0', it indicates an SPR address value for a user mode **mtspr** instructions. For **mfspr**, the instruction is supervisor mode if and only if SPR[0] is set to '1'.

10.3 Performance Monitor Components

A schematic overview of the components that make up the 970MP performance monitor is shown in *Figure 10-1*. These components and their use are described in the following sections.

Figure 10-1. Performance Monitor Architecture



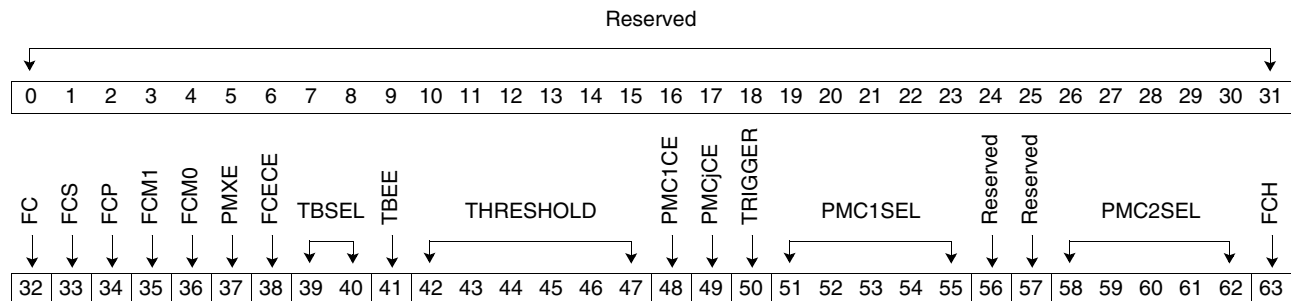
10.4 Performance Monitor Control Registers

The Performance Monitor Control Registers, MMCR0, MMCR1, and MMCRA, are used in conjunction with the MSR and other SPRs to set up the performance monitor enable states, exception conditions, threshold values, match criteria, and selection of the events counted in each of the Counter Registers, PMC1 - PMC8.

The MMCRx Register bit assignments are shown in *Section 10.4.1 Performance Monitor Control Register MMCR0* on page 211, *Section 10.4.2 Performance Monitor Control Register MMCR1* on page 214, and *Section 10.4.3 Performance Monitor Control Register MMCRA* on page 217. The MSR bits that relate to performance monitor functions are shown in *Table 10.4.5 Performance Monitor and Trace Related Bits in the Machine State Register (MSR)* on page 220.

For all of the Performance Monitor Control Register fields, it is always understood that the counter is incremented if that action is not prohibited by some other control condition. All of the MMCRx and PMCx Registers flush to zero unless otherwise noted in the following MMCRx and PMCx tables.

10.4.1 Performance Monitor Control Register MMCR0



Bits	Field Name	Description
0:31	—	Reserved.
32	FC	Freeze counters. 0 The PMCs are incremented. 1 The PMCs are not incremented. The processor sets this bit to '1' when an enabled condition or event occurs <i>and</i> the "freeze counters on enabled condition or event" bit is '1' (MMCR0[FCECE] = '1').
33	FCS	Freeze counters when in supervisor state. 0 The PMCs are incremented. 1 The PMCs are not incremented in supervisor state (MSR[PR] = '0').
34	FCP	Freeze counters when in user (problem) state. 0 The PMCs are incremented. 1 The PMCs are not incremented in user (problem) state (MSR[PR] = '1').
35	FCM1	Freeze counters when performance monitor mark bit (MSR[PMM]) is set to '1'. 0 The PMCs are incremented. 1 The PMCs are not incremented when the MSR mark bit is '1' (MSR[PMM] = '1').
36	FCM0	Freeze counters when performance monitor mark bit (MSR[PMM]) is set to '0'. 0 The PMCs are incremented. 1 The PMCs are not incremented when the MSR mark bit is '0' (MSR[PMM] = '0').

IBM PowerPC 970MP RISC Microprocessor

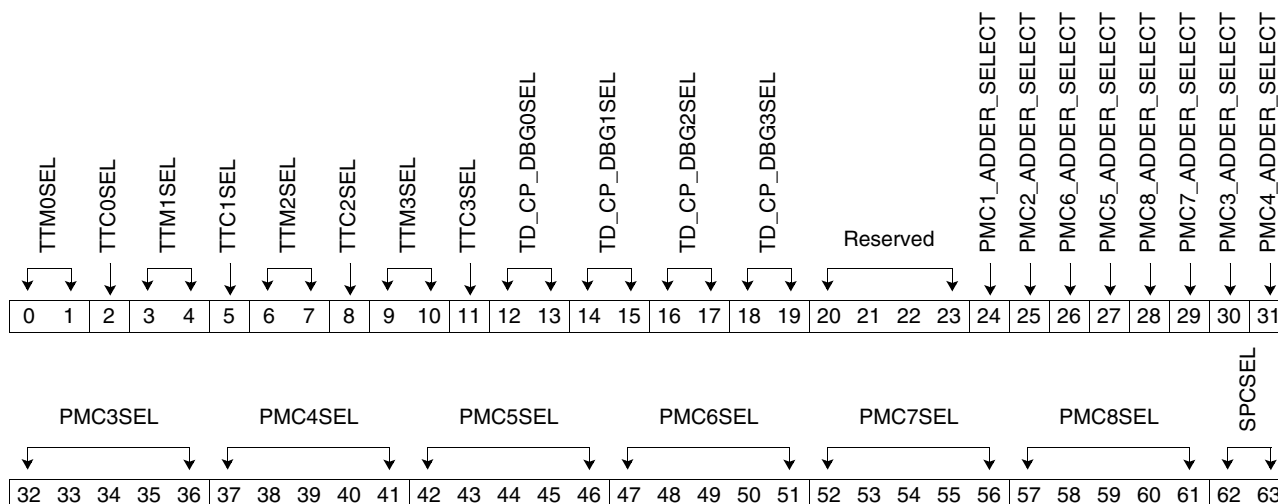
Bits	Field Name	Description
37	PMXE	Performance monitor exception enable. 0 Performance monitor exceptions are disabled. 1 Performance monitor exceptions are enabled until a performance monitor exception occurs, at which time the hardware disables the performance monitor exception (MMCR0[PXME] is set to '0'). For implementations that do not provide a performance monitor exception, software can set PXME to '1' and then poll the bit to determine whether an enabled condition or event has occurred.
38	FCECE	Freeze counters on enabled condition or event. 0 The PMCs are incremented. 1 The PMCs are incremented until detection of an enabled counter negative condition <i>or</i> detection of an enabled time-base transition event occurs <i>and</i> the trigger bit enables the detected event (MMCR0[TRIGGER] equals '0'). At that time the counters are frozen (MMCR0[FC] is set to '1') until the condition is reset by software. If the enabled condition or event occurs when MMCR0[TRIGGER] equals '1', then the FCECE bit is treated as if it were '0'.
39:40	TBSEL	Time-base selector. 00 Time-base bit 63 is selected. 01 Time-base bit 55 is selected. 10 Time-base bit 51 is selected. 11 Time-base bit 47 is selected. When the selected time base transitions from '0' to '1' <i>and</i> the time-base event is enabled (MMCR0[TBEE] equals '1') <i>and</i> the performance monitor exception is enabled, a performance monitor exception occurs and the performance monitor exception is disabled (MMCR0[PXME] is set to '0'). In multiprocessor systems with the Time-Base Registers synchronized among the processors, time-base transition events can be used to correlate the performance monitor data obtained by the several processors provided that software has specified the same TBSEL value for all of the processors in the system. The frequency of the time base is implementation dependent, and a system service routine should be invoked to obtain the frequency before a value for TBSEL is chosen.
41	TBEE	Time-base exception enable. 0 Disable time-base transition events. 1 Enable time-base transition events.
42:47	THRESHOLD	Threshold value. When a threshold event is selected, counting occurs only for those of the selected event occurrences whose duration in number of cycles exceeds the value in the THRESHOLD field.
48	PMC1CE	PMC1 count enable. This bit determines whether the counter negative condition because of a negative value in PMC1 is enabled. 0 Disable PMC1 counter negative condition. 1 Enable PMC1 counter negative condition.
49	PMCjCE	PMCj count enable (where j represents any counter from 2 to 8). This bit determines whether the counter negative condition because of a negative value in PMCj ($2 \leq j \leq 8$) is enabled. 0 Disable PMCj ($2 \leq j \leq 8$) counter negative condition. 1 Enable PMCj ($2 \leq j \leq 8$) counter negative condition.
50	TRIGGER	Trigger enable. 0 The PMCs are incremented. 1 PMC1 is incremented. The PMCjs ($2 \leq j \leq 8$) are not incremented until PMC1 is negative <i>or</i> an enabled condition or event occurs. At that time, the PMCj counters ($2 \leq j \leq 8$) resume counting and the trigger is disabled (MMCR0[TRIGGER] set equal to '0').
51:55	PMC1SEL	PMC1 event selector. The value in this bit field combined with MMCR1[0:31] determines which event will be counted by PMC1.
56	—	Reserved.
57	—	Reserved.

**IBM PowerPC 970MP RISC Microprocessor**

Bits	Field Name	Description
58:62	PMC2SEL	PMC2 event selector. The value in this bit field combined with MMCR1[0:31] determines which event will be counted by PMC2.
63	FCH	Freeze counters in hypervisor mode.

IBM PowerPC 970MP RISC Microprocessor

10.4.2 Performance Monitor Control Register MMCR1



Bits	Field Name	Description
0:1	TTM0SEL	FPU/ISU/IFU/VPU unit select. 00 FPU 01 Instruction sequencer unit (ISU) 10 IFU 11 Vector processing unit (VPU)
2	TTC0SEL	Reserved.
3:4	TTM1SEL	IDU/ISU/STS unit select. 00 IDU 01 Undefined 10 ISU 11 Storage subsystem (STS)
5	TTC1SEL	Reserved.
6:7	TTM2SEL	Reserved.
8	TTC2SEL	Reserved.
9:10	TTM3SEL	Load/store unit 1 (LSU1) select. 0x Lane 2 is LSU1 upper 1x Lane 2 is LSU1 lower x0 Lane 3 is LSU1 upper x1 Lane 3 is LSU1 lower
11	TTC3SEL	Reserved.
12:13	TD_CP_DBG0SEL	Byte lane 0 unit select. 00 Unit from TTM0 01 Unit from TTM1 10 LSU0, byte 0 11 LSU1, byte 0
14:15	TD_CP_DBG1SEL	Byte lane 1 unit select. 00 Unit from TTM0 01 Unit from TTM1 10 LSU0, byte 1 11 LSU1, byte 1

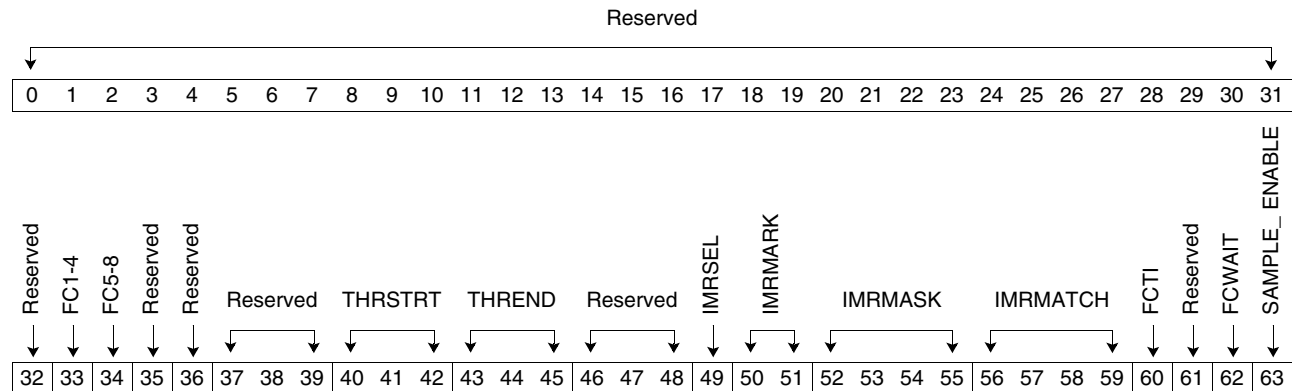
IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
16:17	TD_CP_DBG2SEL	Byte lane 2 unit select. 00 Unit from TTM0 01 Unit from TTM1 10 LSU0, byte 2 11 LSU1, byte 2 or byte 6 (controlled by TTM3SEL[0])
18:19	TD_CP_DBG3SEL	Byte lane 3 unit select. 00 Unit from TTM0 01 Unit from TTM1 10 LSU0, byte 3 11 LSU1, byte 3 or byte 7 (controlled by TTM3SEL[1])
20:23	—	Reserved.
24	PMC1_ADDER_SELECT	PMC1 event adder lane select. 0 Byte lane 0: Add 0 + 4 1 Byte lane 2: Add 0 + 4
25	PMC2_ADDER_SELECT	PMC2 event adder lane select. 0 Byte lane 0: Add 1 + 5 1 Byte lane 2: Add 1 + 5
26	PMC6_ADDER_SELECT	PMC6 event adder lane select. 0 Byte lane 0: Add 2 + 6 1 Byte lane 2: Add 2 + 6
27	PMC5_ADDER_SELECT	PMC5 event adder lane select. 0 Byte lane 0: Add 3 + 7 1 Byte lane 2: Add 3 + 7
28	PMC8_ADDER_SELECT	PMC8 event adder lane select. 0 Byte lane 1: Add 0 + 4 1 Byte lane 3: Add 0 + 4
29	PMC7_ADDER_SELECT	PMC7 event adder lane select. 0 Byte lane 1: Add 1 + 5 1 Byte lane 3: Add 1 + 5
30	PMC3_ADDER_SELECT	PMC3 event adder lane select. 0 Byte lane 1: Add 2 + 6 1 Byte lane 3: Add 2 + 6
31	PMC4_ADDER_SELECT	PMC4 event adder lane select. 0 Byte lane 1: Add 3 + 7 1 Byte lane 3: Add 3 + 7
32:36	PMC3SEL	PMC3 event selector. The value in this bit field combined with MMCR1[0:31] determines which event will be counted by PMC3.
37:41	PMC4SEL	PMC4 event selector. The value in this bit field combined with MMCR1[0:31] determines which event will be counted by PMC4.
42:46	PMC5SEL	PMC5 event selector. The value in this bit field combined with MMCR1[0:31] determines which event will be counted by PMC5.
47:51	PMC6SEL	PMC6 event selector. The value in this bit field combined with MMCR1[0:31] determines which event will be counted by PMC6.
52:56	PMC7SEL	PMC7 event selector. The value in this bit field combined with MMCR1[0:31] determines which event will be counted by PMC7.
57:61	PMC8SEL	PMC8 event selector. The value in this bit field combined with MMCR1[0:31] determines which event will be counted by PMC8.

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
62:63	SPCSEL	Speculative count event selector. 00 Reserved 01 Event A1x 10 Event A2x 11 Event A3x See <i>Table 10-6</i> on page 233 for definitions of the events.

10.4.3 Performance Monitor Control Register MMCRA

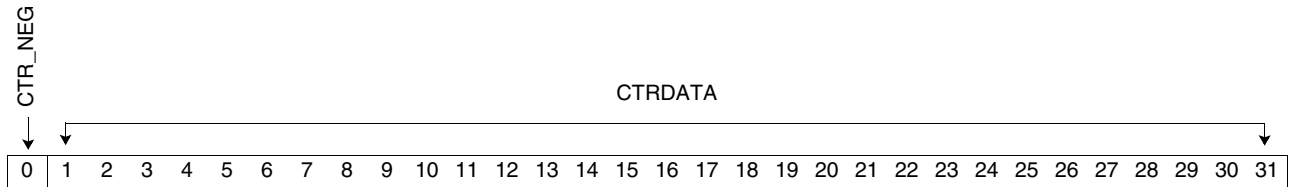


Bits	Field Name	Description
0:31	—	Reserved.
32	—	Reserved.
33	FC1-4	Freeze counters 1 - 4. 0 PMC1 - 4 are incremented. 1 PMC1 - 4 are not incremented.
34	FC5-8	Freeze counters 5 - 8. 0 PMC5 - 8 are incremented. 1 PMC5 - 8 are not incremented.
35	—	Reserved.
36	—	Reserved.
37:39	—	Reserved.
40:42	THRSTRT	Threshold start event.
43:45	THREND	Threshold end event.
46:48	—	Reserved.
49	IMRSEL	Instruction mark (IMR) select. IMR select interacts with IMR mark to determine stage 1 eligibility as described in <i>Section 10.11 IDU Instruction Sampling Facility</i> on page 254. 0 Stage 1 eligible instructions are determined through predecode bits from the IFU combined with the IMRMATCH and IMRMASK fields as described in <i>Section 10.11</i> on page 254. This is useful if the IMR mark equals '00'. 1 The instruction mark bit (IMR bit) from the IFU IMC match array is used to determine Stage 1 eligibility.
50:51	IMRMARK	IMR Mark. Chooses the mark mode for which instructions are Stage 2 eligible. 00 All Stage 1 eligible internal operations (IOPs). 01 Only Stage 1 eligible IOPs that resulted from microcode expansion. 10 Only one IOP per eligible PowerPC instruction. 11 First IOP that goes to the LSU for every eligible PowerPC load/store (ld/st) instruction.
52:55	IMRMASK	IMR Mask. A mask ANDed with the predecode bits before using the IMRMATCH field.

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
56:59	IMRMATCH	IMR Match. The value that the result of the IMRMASK ANDed with the predecode bits must match to be Stage 2 eligible. All 4 bits of the result must match the IMRMATCH exactly. To match ALL IOPs (that is, the match will always succeed) set IMRSEL equals '0', IMRMASK equals '0000', and IMRMATCH equals '0000'.
60	FCTI	Freeze Counters. 0 The PMCs are incremented. 1 The PMCs are not incremented.
61	—	Reserved
62	FCWAIT	Freeze Counters in Wait State (implies that CNTL[31] equals '0'). 0 The PMCs are incremented. 1 The PMCs (except those counting cycles) are not incremented when CNTL[31] equals '0'.
63	SAMPLE_ENABLE	0 Sampling is disabled. 1 Sampling is enabled.

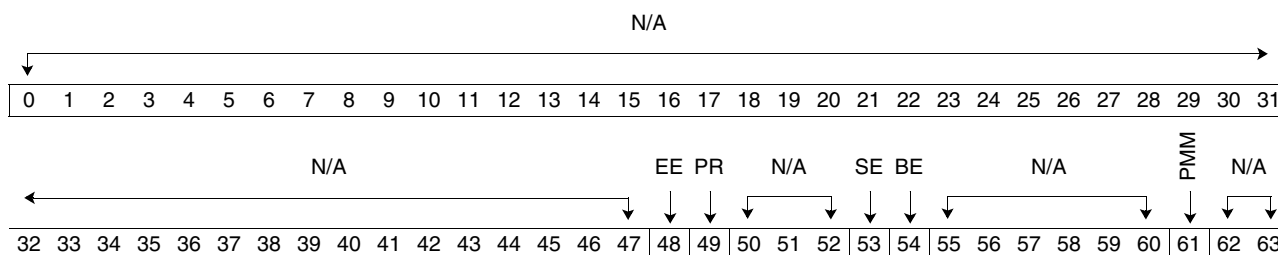
10.4.4 Performance Monitor Count Registers PMC1 - 8



Bits	Field Name	Description
0	CTR_NEG	Counter negative bit.
1:31	CTRDATA	Count data.

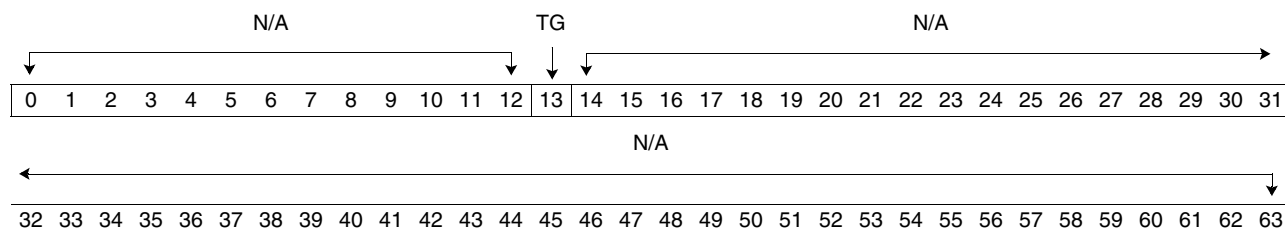
IBM PowerPC 970MP RISC Microprocessor

10.4.5 Performance Monitor and Trace Related Bits in the Machine State Register (MSR)



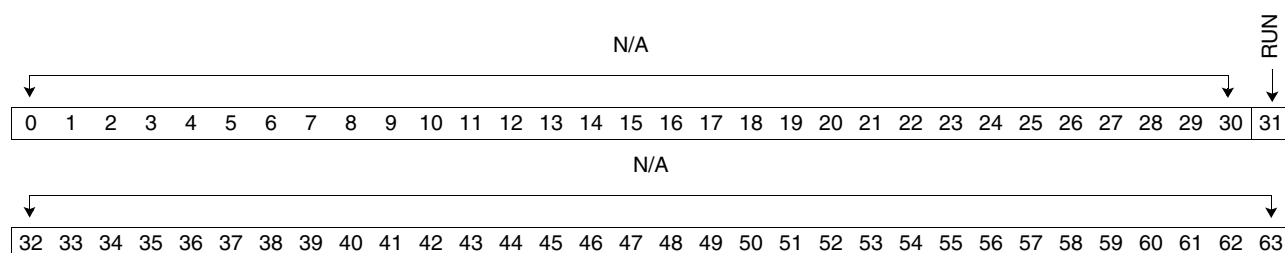
Bits	Field Name	Description
0:47	N/A	Not applicable.
48	EE	External interrupt enable. 0 The processor is disabled for external, decremter, and performance monitor exceptions. 1 The processor is enabled for external, decremter, and performance monitor exceptions.
49	PR	Problem (user) state. 0 The processor is privileged to execute any instruction. 1 The processor can execute only non-privileged instructions.
50:52	N/A	Not applicable.
53	SE	Single step trace enable. 0 The processor does not generate a trace exception after instruction completion. 1 The processor generates a trace exception after successfully completing the execution of the next instruction unless that instruction is an Return from Exception Doubleword (rfd), which is never traced.
54	BE	Branch trace enable. 0 The processor does not generate a trace exception after branch instruction completion. 1 The processor generates a trace exception after successfully completing the execution of a branch instruction whether the branch is taken.
55:60	N/A	Not applicable.
61	PMM	Performance monitor mode enable. 0 The currently executing process is not marked. 1 The currently executing process is marked. This bit is used to mark a process for the performance monitor. Several performance monitor MMCR0 control bits can then be set to enable counting based on the value of the PMM bit. When an exception occurs, this bit is saved, set to '0' for the duration of the exception processing, and then restored when the rfd instruction is executed. If this bit is changed with an mtmsr or Move to Machine State Register Doubleword (mtmsrd) instruction, the change is not guaranteed to have taken effect until after a subsequent context-synchronizing instruction has completed execution.
62:63	N/A	Not applicable.

10.4.6 Performance Monitor Related Bits in Hardware Implementation-Dependent Register 0 (HID0)



Bits	Field Name	Description
0:12	N/A	Not applicable.
13	TG	Performance monitor threshold granularity. 0 The thresholder counts every processor cycle. 1 The thresholder counts every 32 processor cycles.
14:63	N/A	Not applicable.

10.4.7 Performance Monitor Related Bits in the Control Register (CTRL)

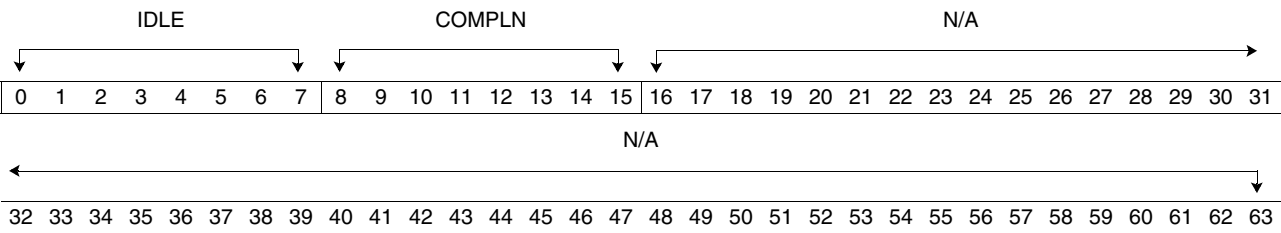


Bits	Field Name	Description
0:30	N/A	Not applicable.
31	RUN	Wait state bit.
32:63	N/A	Not applicable.



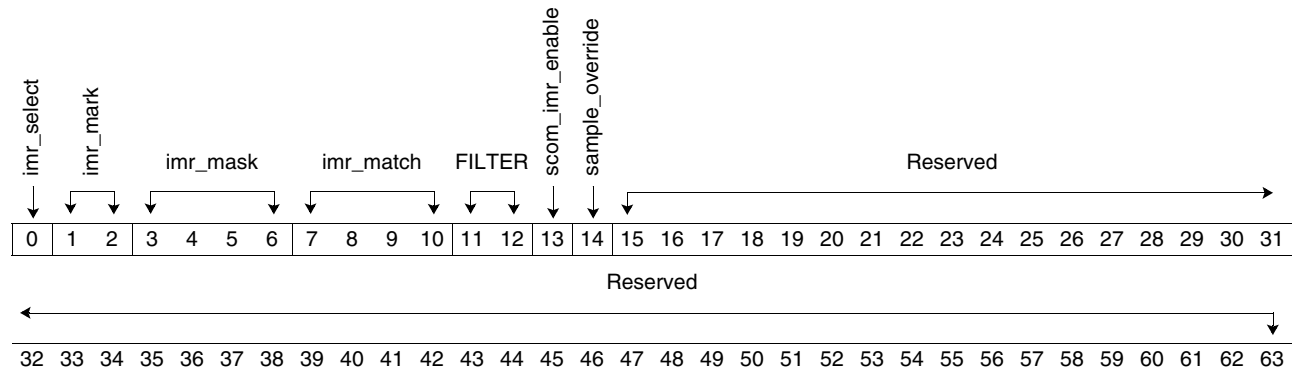
IBM PowerPC 970MP RISC Microprocessor

10.4.8 Performance Monitor Related Bits in the SCOM0240, 1240 Register (SCOM x'240')

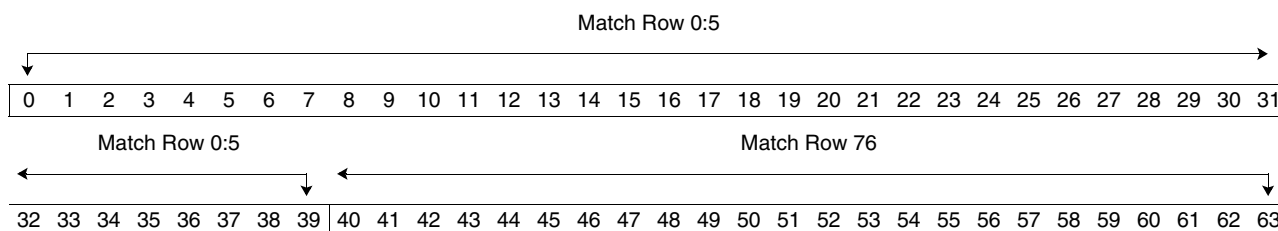


Bits	Field Name	Description
0:7	IDLE	Sampling logic idle delay.
8:15	COMPLN	Sampling logic completion delay.
16:63	N/A	Not applicable.

10.4.9 Performance Monitor Related Bits in the SCOM0360,1360 Register (SCOM x'360')



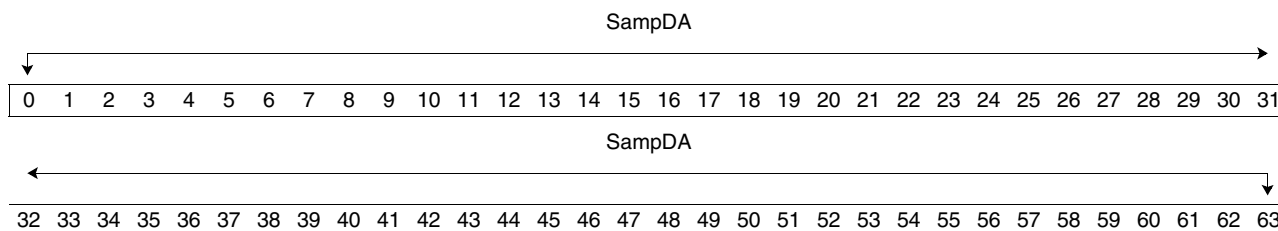
Bits	Field Name	Description								
0	imr_select	Same as MMCRA[imr_sel].								
1:2	imr_mark	Same as MMCRA[imr_mark] and overrides MMCRA if bit 13 equals '1'.								
3:6	imr_mask	Same as MMCRA[imr_mask] and overrides MMCRA if bit 13 equals '1'.								
7:10	imr_match	Same as MMCRA[imr_match] and overrides MMCRA if bit 13 equals '1'.								
11:12	FILTER	IMR filter random/all and first/all. These two bits form a 2-step filtering operation on the eligible bits associated with the instructions in the group. Bit 11 first determines whether instruction eligibility bits pass the first filter step based on either a random pass/nopass (bit 11 equals '1') choice or an all pass (bit 11 equals '0') choice for each instruction. Bit 12 determines how microcoded instructions are sampled (and has no effect on non-microcoded instructions): <table><tr><td>00</td><td>No filtering (OR).</td></tr><tr><td>01</td><td>No filtering (AND).</td></tr><tr><td>10</td><td>Use Good_Address mode of sampling microcode expansions.</td></tr><tr><td>11</td><td>Use More_Hits mode of sampling microcode expansions.</td></tr></table> In Good_Address mode, there is at most one IOP in any microcode expansion that is eligible for sampling. This is (a) the first load/store IOP if there are any load/store IOPs in the expansion, or (b) the first IOP in the final group of the expansion. If the random filter suppresses marking this IOP, then no IOP will be marked for the microcode expansion. In More_Hits mode, multiple IOPs in a microcode expansion are eligible for sampling. These are (a) the first load/store IOP in any group, or (b) the first IOP of the final group. If the random filter suppresses marking the first of these IOPs, a subsequent one might still be sampled. (However, at most one will be marked in a single microcode expansion.)	00	No filtering (OR).	01	No filtering (AND).	10	Use Good_Address mode of sampling microcode expansions.	11	Use More_Hits mode of sampling microcode expansions.
00	No filtering (OR).									
01	No filtering (AND).									
10	Use Good_Address mode of sampling microcode expansions.									
11	Use More_Hits mode of sampling microcode expansions.									
13	scom_imr_enable	<table><tr><td>0</td><td>Performance monitor fields are used for mark, mask, match.</td></tr><tr><td>1</td><td>SCOM fields are used for mark, mask, match.</td></tr></table>	0	Performance monitor fields are used for mark, mask, match.	1	SCOM fields are used for mark, mask, match.				
0	Performance monitor fields are used for mark, mask, match.									
1	SCOM fields are used for mark, mask, match.									
14	sample_override	<table><tr><td>0</td><td>Performance monitor "ok_to_sample" indication is used.</td></tr><tr><td>1</td><td>Overrides performance monitor "ok_to_sample" indication.</td></tr></table>	0	Performance monitor "ok_to_sample" indication is used.	1	Overrides performance monitor "ok_to_sample" indication.				
0	Performance monitor "ok_to_sample" indication is used.									
1	Overrides performance monitor "ok_to_sample" indication.									
15:63	—	Reserved.								

IBM PowerPC 970MP RISC Microprocessor
10.4.10 Performance Monitor Related Bits in the IMC Array (IMC)


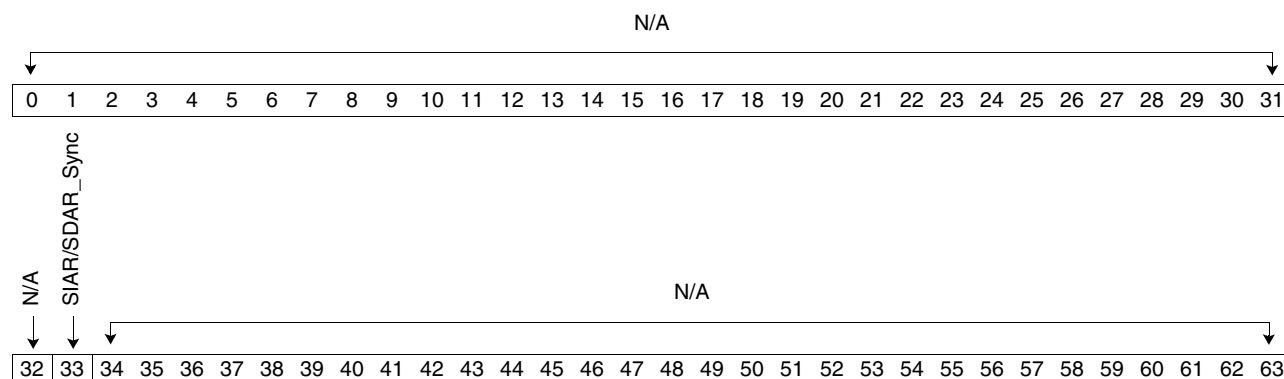
Bits	Field Name	Bit Description
0:39	Match Row 0	Opcode/extended opcode match.
0:39	Match Row 1	Opcode/extended opcode match.
0:39	Match Row 2	Opcode/extended opcode match.
0:39	Match Row 3	Opcode/extended opcode match.
0:39	Match Row 4	Opcode/extended opcode match.
0:39	Match Row 5	Opcode/extended opcode match.
0:63	Match Row 76	Full instruction match.

10.4.11 Performance Monitor Related Bits in the Sampled Instruction Address Register (SIAR)

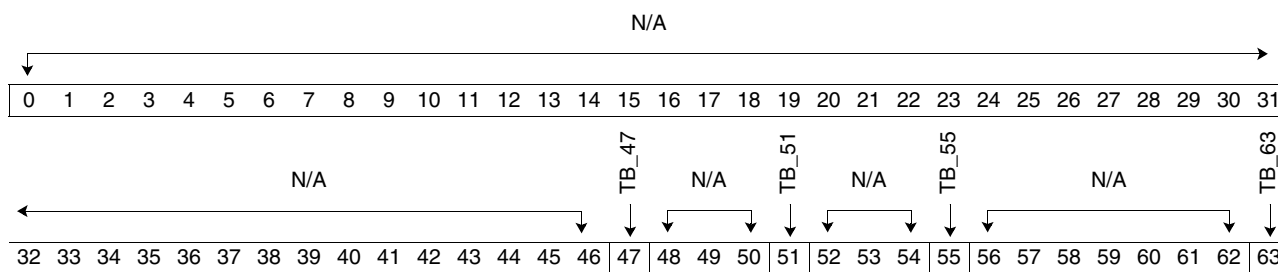

Bits	Field Name	Description
0:63	SamplA	Sampled instruction address.

10.4.12 Performance Monitor Related Bits in the Sampled Data Address Register (SDAR)


Bits	Field Name	Description
0:63	SampDA	Sampled data address.

**10.4.13 Performance Monitor Related Bits in the SRR1 (SRR1)**

Bits	Field Name	Description
0:32	N/A	Not applicable.
33	SIAR/SDAR_Sync	SIAR and SDAR contents synchronized.
34:63	N/A	Not applicable.

IBM PowerPC 970MP RISC Microprocessor
10.4.14 Performance Monitor Related Bits in the Time-Base Register (TB)


Bits	Field Name	Description
0:46	N/A	Not applicable.
47	TB_47	Time-Base Register bit 47.
48:50	N/A	Not applicable.
51	TB_51	Time-Base Register bit 51.
52:54	N/A	Not applicable.
55	TB_55	Time-Base Register bit 55.
56:62	N/A	Not applicable.
63	TB_63	Time-Base Register bit 63.

10.5 Performance Monitor Event Selection

Event signals are routed from the functional units through the processor performance monitor buses. These signals are multiplexed and divided into byte lanes 0 - 3. A smaller number of events are routed directly to the performance monitor unit (PMU); these events are referred to as direct events. Each PMC can be configured to count a subset of the direct events or one of two possible byte lanes. Counters 1, 2, 5, and 6 can be configured to count events on byte lane 0 or 2, counters 3, 4, 7, and 8 can be configured to count events on byte lane 1 or 3. The selection of event source (direct, byte lane) is controlled by the PMCxSEL field in MMCR1 (where x is the PMC number). *Figure 10-2* shows this selection. *Table 10-2* shows how the PMCxSEL field is used to select which events are monitored.

Performance monitor events fall into three categories:

- Direct: All the information is hardwired to the PMU.
- Bus: All the information is routed over the hierarchical event bus.
- Combined: Some information comes from the event bus; the PMU does additional processing on it.

Figure 10-2. Event Selection

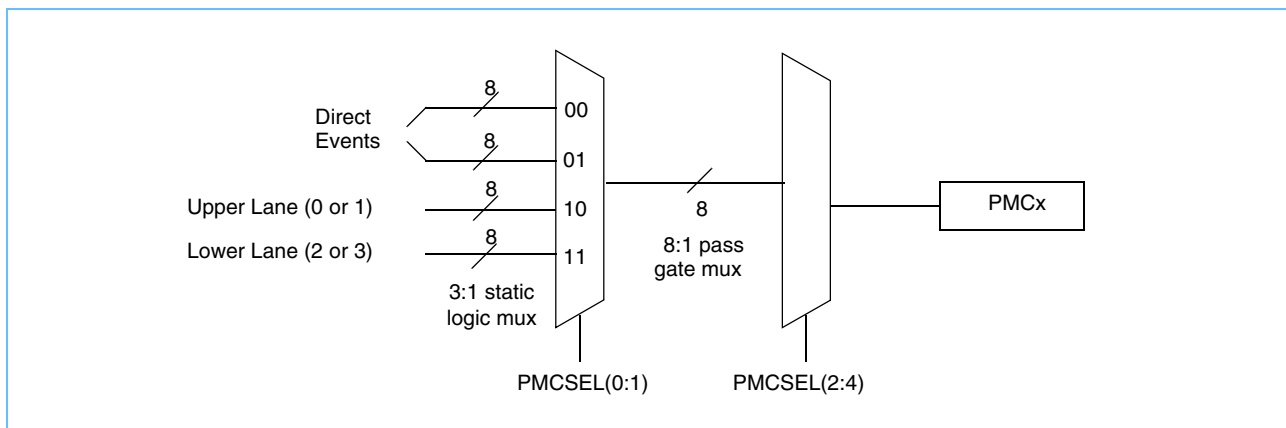


Table 10-2. Performance Monitor Internal Multiplexer PMCxSEL[0:4] Bit Values

PMCSEL[0:1]	PMCSEL[2:4]	Counted Event
00	000-111	Direct Events.
10	000	None. When count_en is '0', turn off counter.
10	111	Cycles.
01	000-111	Direct Events.
10	000-111	Select smaller byte lane.
11	000-111	Select larger byte lane.

10.5.1 Direct Events

As shown in *Table 10-2* on page 227, direct events are selected with PMCxSEL[0:1] set to '0x'. When PMCxSEL[0:1] equals '10' and PMCxSEL[2:4] equals '111', the counter is configured to count cycles. When PMCxSEL[0:1] equals '10' and PMCxSEL[2:4] is '000', the counter is off (counts nothing). The direct events that can be counted are shown in *Table 10-5* on page 230.

Some direct events, such as events that add two other events or interpret the memory source encodes for data or instruction fetches, also require data from the performance monitor events. Although they are listed in *Table 10-5 Direct Events*, they rely on a meaningful configuration of the performance monitor event selections to produce meaningful results.

10.5.1.1 Combined Events

Each PMC can add similar events to produce a single, combined count. For example, each load store unit provides a data cache miss event, which can be added to produce the total data cache miss count. The added events are considered direct events, but they rely on the performance monitor bus being configured properly to produce meaningful results. Because each PMC can receive event signals from two byte lanes on the performance monitor bus, the added events can be configured to add events on one of the two byte lanes. Events cannot be added from different byte lanes. The PMCx_ADDER_SELECT fields in MMCR1 control which byte lanes are used.

10.5.1.2 Source-Encoded Events

Source-encoded events (direct event 7 [PMCxSEL equals '00111'] for data and event 6 [SEL equals '00110'] for instructions) are combined events that count events from a specific source as shown in *Table 10-3* and *Table 10-4* on page 229.

Note: Intervention event sources are only meaningful on multiprocessor systems.

Table 10-3. Event Data Source Encodings

Encoding (0:3)	Event Source
0000	L2 cache
0001	Memory
0100	Shared Intervention (another L2 cache)
0101	Modified Intervention (another L2 cache)
All Others	Reserved

To count data source-encoded events, the performance monitor event bus must be configured as follows:

1. Route LSU1 byte 3 data to the PMU (the "L1 reload data source" LSU1 indirect event) by setting the TD_CP_DBG3SEL field in MMCR1 to '11'.
2. Select the direct event that decodes the required data source. To count L1 data reloads from the L2, for example, PMC1, direct event 7 (the PMC1SEL field in MMCR0 set to '00111') should be used.

Table 10-4. Event Instruction Source Encodings

Encoding (0:3)	Event Source
1001	I-cache
1010	Prefetch buffer
0000	L2 cache
0001	Memory
1111 or 1011	No instructions on bus

To count instruction source-encoded events, the performance monitor event bus should be similarly configured:

1. Route IFU byte 2 data to the PMU (the "iL1 cache data source" IFU indirect event) by setting the TD_CP_DBG2SEL field in MMCR1 to '00' and TTM0SEL to '10'.
2. Select the direct event that decodes the required data source. To count L1 instruction reloads from memory, for example, PMC3, direct event 6 (the PMC3SEL field in MMCR0 set to '00110') should be used.

10.5.1.3 Instruction Counts

Two types of instruction and IOP counting are available with the 970MP performance monitor:

- Direct event 1 (SEL equals '00001') on PMC1, PMC4, PMC6, PMC7, and PMC8 counts instructions according to the IMRMARK field of the MMCRA Register:
 - 00 All stage 1 eligible IOPs
 - 01 Stage 1 eligible IOPs from microcode expansion
 - 10 One IOP per eligible PowerPC instruction
 - 11 First IOP to LSU per eligible PowerPC load/store instruction
- Direct event 9 (SEL equals '01001') on PMC1 - PMC8 always counts PowerPC instructions independent of the IMRMARK field of the MMCRA Register (see Table 10-5 on page 230).



Table 10-5. Direct Events (Page 1 of 2)

SEL(0:4)	PMC1	PMC2	PMC3	PMC4	PMC5	PMC6	PMC7	PMC8
00 000 plus	Add 0 + 4	Add 1 + 5	Add 2 + 6	Add 3 + 7	Add 3 + 7	Add 2 + 6	Add 1 + 5	Add 0 + 4
MMCR1[24:31] = '0', '1'	MMCR1[24] = '0' byte lane 0	MMCR1[25] = '0' byte lane 0	MMCR1[30] = '0' byte lane 1	MMCR1[31] = '0' byte lane 1	MMCR1[27] = '0' byte lane 0	MMCR1[26] = '0' byte lane 0	MMCR1[29] = '0' byte lane 1	MMCR1[28] = '0' byte lane 1
	MMCR1[24] = '1' byte lane 2	MMCR1[25] = '1' byte lane 2	MMCR1[30] = '1' byte lane 3	MMCR1[31] = '1' byte lane 3	MMCR1[27] = '1' byte lane 2	MMCR1[26] = '1' byte lane 2	MMCR1[29] = '1' byte lane 3	MMCR1[28] = '1' byte lane 3
00 001	number of instructions complete	work held	stop completion	number of instructions complete	dispatch_success	number of instructions complete	number of instructions complete	number of instructions complete
00 010	marked group dispatch	LSU empty (load miss queue [LMQ] and store reorder queue [SRQ] empty)	LSU empty (LMQ and SRQ empty)	Fixed-point unit 0 (FXU0) idle and FXU1 busy	FXU0 idle and FXU1 idle	FXU0 busy and FXU1 busy	FXU0 busy and FXU1 idle	external interrupt
00 011	marked store complete	threshold timeout event	marked store with interrupt complete	SRQ empty	one or more PowerPC instructions completed	marked store sent to STS	group completed	group dispatch reject
00 100	global completion table (GCT) empty	group dispatch	cycles in supervisor mode	marked group complete	group marked in IDU	FXU marked instruction finish	FPU marked instruction finish	LSU marked instruction finish
00 101	run_cycles; that is, # cycles when CNTL[31] = '1'	branch unit (BRU) marked instruction finish	VPU marked instruction finish	condition register unit (CRU) marked instruction finish	marked group complete time out	marked group issued	marked instruction finish any unit	time base event
00 110	Instruction source encode 0000	Instruction source encode 0001	Instruction source encode 0010	Instruction source encode 0011	Instruction source encode 0100	Instruction source encode 0101	Instruction source encode 0110	Instruction source encode 0111
00 111	Data source encode 0000	Data source encode 0001	Data source encode 0010	Data source encode 0011	Data source encode 0100	Data source encode 0101	Data source encode 0110	Data source encode 0111
01 000	Counter OFF	Counter OFF	Counter OFF	Counter OFF	Counter OFF	Counter OFF	Counter OFF	Counter OFF
01 001	number of instructions complete	number of instructions complete	number of instructions complete	number of instructions complete	number of instructions complete	number of instructions complete	number of instructions complete	number of instructions complete
01 010	Overflow from counter 8	Overflow from counter 1	Overflow from counter 2	Overflow from counter 3	Overflow from counter 4	Overflow from counter 5	Overflow from counter 6	Overflow from counter 7
01 011	Reserved	GCT empty by SRQ full	Reserved	Reserved	—/A1a/A2a/A3a (*1) (See Table 10-6 on page 233)	Reserved	—/A1b/A2b/A3b (*1) (See Table 10-6 on page 233)	Reserved



Table 10-5. Direct Events (Page 2 of 2)

SEL(0:4)	PMC1	PMC2	PMC3	PMC4	PMC5	PMC6	PMC7	PMC8
00 000 plus	Add 0 + 4	Add 1 + 5	Add 2 + 6	Add 3 + 7	Add 3 + 7	Add 2 + 6	Add 1 + 5	Add 0 + 4
01 100	Reserved	Reserved	Reserved	Reserved	—/A1c/A2c/—(*1) (See Table 10-6 on page 233)	Reserved	—/A1d/A2d/—(*1) (See Table 10-6 on page 233)	Reserved
01 101	Instruction source decode 1000	Instruction source encode 1001	Instruction source encode 1010	Instruction source encode 1011	Instruction source encode 1100	Instruction source encode 1101	Instruction source encode 1110	Instruction source encode 1111
01 110	Byte 3 decode 1000	Data source encode 1001	Data source encode 1010	Data source encode 1011	Data source encode 1100	Data source encode 1101	Data source encode 1110	Data source encode 1111
01 111	Cycles	Cycles	Cycles	Cycles	Cycles	Cycles	Cycles	Cycles

10.5.2 Over 32-Bit Count

The 970MP PMU can chain together multiple 32-bit PMCs to create up to a 256-bit wide PMC Register when used in conjunction with overflow counting. This is useful for performance measurement on high clock-rate machines. The maximum count value depends on the following settings:

- $PMCN$ can count over 32-bits when $PMCN+1SEL(0:4)$ (where n is 1 - 7) is set to '01010'.
- PMC8 can count over 32-bits when PMC1SEL(0:4) is set to '01010'.
- When $PMCN+1$ uses this overflow counting function, $PMCN$ is prohibited from asserting an exception signal when a negative condition occurs ($PMC1CE(PMCjCE)$ equals '1' and $PMCN[0]$ is '1').

10.5.2.1 Examples of Over Bit Count

Example 1

When PMC1 is set to '00100' (GCT empty) and PMC2 is set to '01010' (overflow function), then PMC2 works as the upper 32 bits of PMC1. In this case, the overflow exception is only asserted by PMC2 (never by PMC1) when $PMCjCE$ equals '1' (don't care $PMC1CE$) and $PMC2[0]$ is '1'.

Example 2

When PMC8 is set to '00001' (number of instructions complete) and PMC1 are set to '01010' (overflow function), then PMC1 functions as the upper 32 bits of PMC8. In this case, the overflow exception is only asserted by PMC1 (never by PMC8) when $PMC1CE$ equals '1' (don't care $PMCjCE$) and $PMC1[0]$ is '1'.

Example 3

When PMC1 is set to '00100' (GCT empty) and PMC2, PMC3, and PMC4 is set to '01010' (overflow function), then PMC4, PMC3, and PMC2 function as the upper 96 bits of PMC1. In this case, the overflow exception is only asserted by PMC4 (never by PMC1, PMC2, or PMC3) when $PMCjCE$ equals '1' (don't care $PMC1CE$) and $PMC4[0]$ is '1'.

10.5.3 Speculative Count

PMC5 and PMC7 support the speculative count function with a backup register. This is enabled when $MMCR1[62:63]$ is set to '01', '10', or '11' and a speculative event is selected ($PMC[5,7]SEL$ equals '01011' or '01100'). The PMC starts counting speculatively whenever a next-to-complete (NTC) group completion stops (or GCT empty happens). The PMC then stores the counts to itself and its backup register if the last finished event matches what the PMC initially set up. If there is no match, the PMC restores the old count value from the backup register. This allows the PMU to establish a cycles per instruction (CPI) breakdown for various categories (CPI contribution because of an instruction-cache [I-cache] miss, data cache [D-cache] miss, LSU, FXU, FPU, and so on).

A negative condition exception only occurs when the count value is not speculative and a negative condition occurs. (When $PMC1CE[PMCjCE]$ is set to '1' and the backup register's negative bit is '1'.)

Table 10-6 on page 233 lists the speculative count events.

Table 10-6. Speculative Count Events

PMC Number		SEL(0:4)	MMCR1 Condition		Count Events	See Note
			Bit 62	Bit 63		
	5, 7	01011	0	0	Reserved	
A1a	5	01011	0	1	Completion stall by LSU instruction	
A2a	5	01011	1	0	Completion stall by FXU instruction	
A3a	5	01011	1	1	Completion stall by D-cache miss	
A1b	7	01011	0	1	Completion stall by FPU instruction	
A2b	7	01011	1	0	Completion stall by FXU long instruction	
A3b	7	01011	1	1	Completion stall by reject	
	5, 7	01100	0	0	Reserved	
A1c	5	01100	0	1	Completion stall by FPU long instruction	
A2c	5	01100	1	0	GCT empty by I-cache miss	1
A1d	7	01100	0	1	Completion stall by reject (ERAT miss)	
A2d	7	01100	1	0	GCT empty by branch miss predict	1
	5, 7	01100	1	1	Reserved	
1. This count event also requires MMCR1 bits. They should be set as follows: Bit 1 = '1' and bits 0, 16, and 17 = '0' or Bits 3 and 17 = '1' and bits 4 and 16 = '0'						

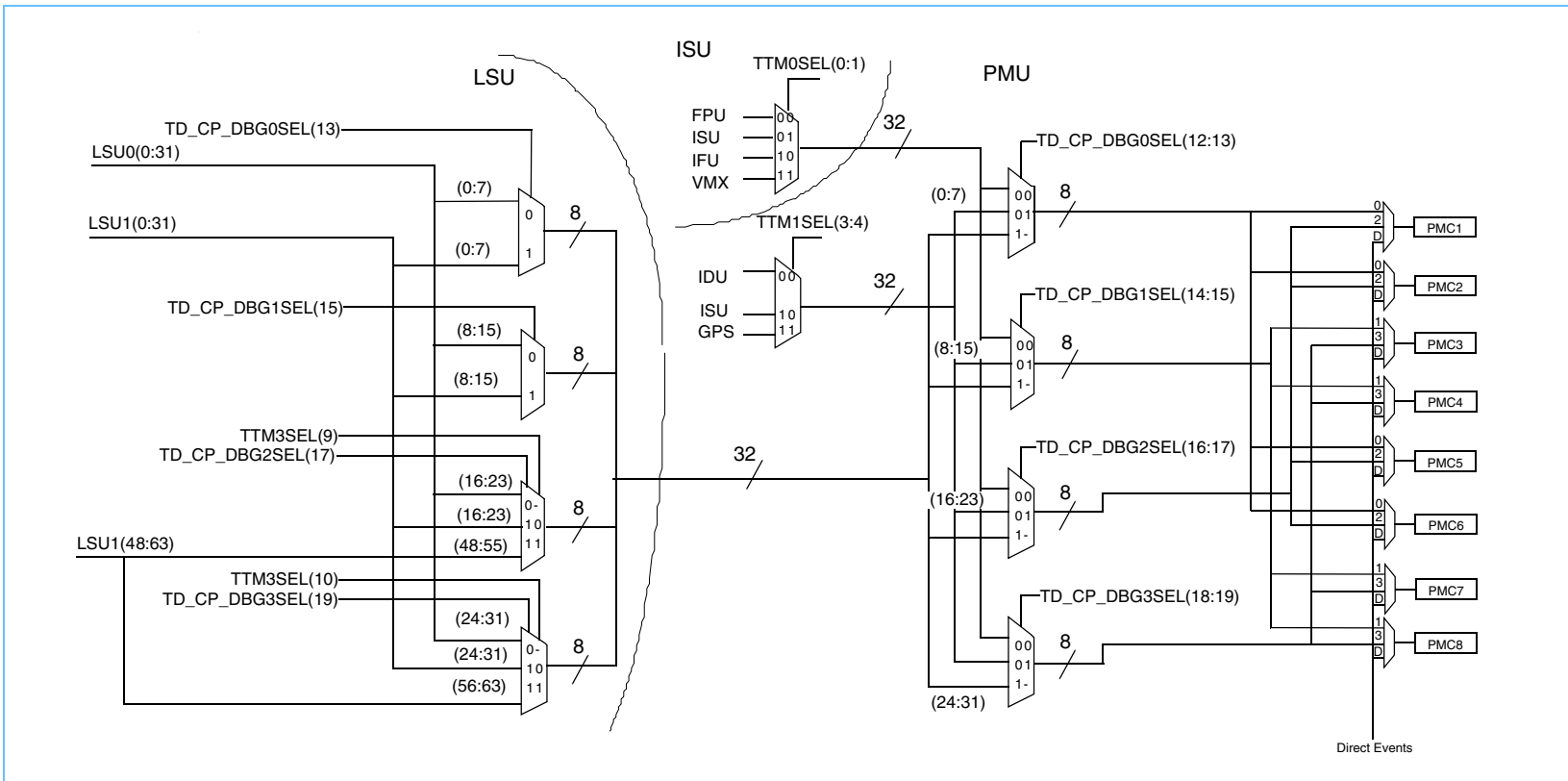
Speculative events are also able to count in the over 32-bit count mode. In this case, the overflow value is carried to the upper PMC only when the counting value is not speculative. For this reason, the upper PMC does not require a backup register to copy and restore the count value.

10.6 Configuring the Performance Monitor Bus

The 970MP performance monitor bus is configured through a series of hierarchical multiplexers, as shown in *Figure 10-3* on page 234. This diagram also shows that all unit event buses are 32 bits, except for the LSU1 that sources an extra 16 bits, denoted as LSU1[48:63]. The LSU0 and LSU1 event buses are multiplexed into a single 32-bit LSU event bus, using the multiplexers shown at the left of *Figure 10-3*. Basically, the multiplexers, on a byte basis, select either the LSU0 or the LSU1 events. The extra LSU1 events are selected using MMCR1[9:10], the TTM3SEL select signals.



Figure 10-3. 970MP Performance Monitor Bus Configuration



The rest of the first level of selection is controlled by the TTM0 and TTM1 multiplexers. These control from which unit the non-LSU event signals are selected. The ISU can be selected by more than one multiplexer (TTM0 and TTM1). The TTM multiplexers are controlled by the TTMxSEL fields in MMCR1. MMCR1[0:1] control TTM0 and MMCR1[3:4] control TTM1.

The three 32-bit outputs of the LSU, TTM0, and TTM1 multiplexers are sent to the TM_DEBUG multiplexers, which are controlled by the 2-bit TD_CP_DBGxSEL fields in the MMCR1; bits 12:13 control TM_DEBUG0, bits 14:15 control TM_DEBUG1, bits 16:17 control TM_DEBUG2, and bits 18:19 control TM_DEBUG3.

After the performance monitor bus is configured, individual events can be selected, as described at the beginning of this section. *Table 10-7* shows the events available through the performance monitor bus and the TTM and TM_DEBUG multiplexer used to select them.

Table 10-7. Performance Monitor Bus Assignments (Page 1 of 8)

	Bits [0:31]	Byte Lane	Bits [0:7]	Event Description
FPU: TTM0 = '00', TD_CP_DBGxSEL = '00'				
	0	0	0	FPU0 divide
	1	0	1	FPU0 mult-add
	2	0	2	FPU0 square root
	3	0	3	FPU0 add, mult, sub, compare, fsel
	4	0	4	FPU1 divide
	5	0	5	FPU1 mult-add
	6	0	6	FPU1 square root
	7	0	7	FPU1 add, mult, sub, compare, fsel
	8	1	0	FPU0 move, estimate
	9	1	1	FPU0 round, convert
	10	1	2	FPU0 estimate
	11	1	3	FPU0 finished and produced a result
	12	1	4	FPU1 move, estimate
	13	1	5	FPU1 round, convert
	14	1	6	FPU1 estimate
	15	1	7	FPU1 finished and produced a result

IBM PowerPC 970MP RISC Microprocessor
Table 10-7. Performance Monitor Bus Assignments (Page 2 of 8)

	Bits [0:31]	Byte Lane	Bits [0:7]	Event Description
	16	2	0	FPU0 denormalized operand
	17	2	1	FPU0 stall 3
	18	2	2	FPU0 store
	19	2	3	FPU0 single precision
	20	2	4	FPU1 denormalized operand
	21	2	5	FPU1 stall 3
	22	2	6	FPU1 store
	23	2	7	FPU1 single precision
	24	3	0	Floating-Point Status and Control Register (FPSCR)
	25	3	1	FPU0 multiply
	26	3	2	FPU0 compare
	27	3	3	FPU0 select
	28	3	4	FPU1 multiply
	29	3	5	FPU1 compare
	30	3	6	FPU1 select
	31	3	7	Floating-point stall store
IFU: TTM0 = '10', TD_CP_DBGxSEL = '00'				
	0:15	0:1	0:7	Nothing
	16:19	2	0:3	L1 I-cache data source
	20	2	4	Valid instruction available
	21	2	5	Cycles IFU is held by pipeline hold
	22	2	6	Instruction prefetch installed in prefetch buffer
	23	2	7	L2 prefetch request
	24	3	0	I-ERAT write
	25	3	1	Branch execution issue valid
	26	3	2	Branch miss predict because of Condition Register (CR) value
	27	3	3	Branch miss predict because of because of a target address predict
	28	3	4	Cycles L1 I-cache write active
	29:31	3	5:7	Nothing

Table 10-7. Performance Monitor Bus Assignments (Page 3 of 8)

	Bits [0:31]	Byte Lane	Bits [0:7]	Event Description
IDU: TTM1 = '00', TD_CP_DBGxSEL = '01'				
	0	0	0	Instruction queue has three slots full
	1	0	1	Instruction queue has one slot full
	2	0	2	Instruction queue has six slots full
	3	0	3	Instruction queue has zero slots full
	4	0	4	Instruction queue has four slots full
	5	0	5	Instruction queue has two slots full
	6	0	6	Instruction queue has seven slots full
	7	0	7	Instruction queue has eight slots full
	8	1	0	Instruction queue has five slots full
	9:15	1	1:7	Instruction queue full
	16:31	2:3	0:7	Nothing
LSU0: (See Figure 10-3 970MP Performance Monitor Bus Configuration on page 234) TD_CP_DBGxSEL = '1x'				
	0	0	0	Instruction translation lookaside buffer (TLB) miss
	1	0	1	Instruction segment lookaside buffer (SLB) miss
	2	0	2	Data ERAT (D-ERAT) miss side 0
	3	0	3	Snoop TLB Invalidate Entry (tlbie)
	4	0	4	Data TLB miss
	5	0	5	Data SLB miss
	6	0	6	D-ERAT miss side 1
	7	0	7	Tablewalk duration
	8	1	0	Marked flush unaligned load side 0
	9	1	1	Marked flush unaligned store side 0
	10	1	2	Marked flush from load reorder queue (LRQ) store-hit-load (SHL), load-hit-load (LHL) side 0
	11	1	3	Marked flush SRQ load-hit-store (LHS) side 0
	12	1	4	Marked flush unaligned load side 1
	13	1	5	Marked flush unaligned store side 1
	14	1	6	Marked flush from LRQ store-hit-load (shl), load-hit-load (lhl) side 1
	15	1	7	Marked flush SRQ LHS side 1

IBM PowerPC 970MP RISC Microprocessor

Table 10-7. Performance Monitor Bus Assignments (Page 4 of 8)

	Bits [0:31]	Byte Lane	Bits [0:7]	Event Description
	16	2	0	Marked L1 D-cache load miss side 0
	17	2	1	Store conditional (stcx) failed
	18	2	2	Marked IMR reload
	19	2	3	Marked L1 D-cache store miss
	20	2	4	Marked L1 D-cache load miss side 1
	21	2	5	stcx passed
	22	2	6	Marked stcx fail
	23	2	7	Load and reserve indexed (larx) executed 0
	24	3	0	Floating-point load side 0
	25	3	1	L1 cache prefetch request
	26	3	2	Out of streams
	27	3	3	L2 cache prefetch
	28	3	4	Floating-point load side 1
	29	3	5	VPU type L2 prefetch
	30	3	6	Data stream touch (dst) stream start
	31	3	7	New stream allocated
LSU1: (See Figure 10-3 970MP Performance Monitor Bus Configuration on page 234) TD_CP_DBGxSEL = '1x'				
	0	0	0	Flush unaligned load side 0
	1	0	1	Flush unaligned store side 0
	2	0	2	Flush from LRQ SHL, LHL side 0
	3	0	3	Flush SRQ LHS side 0
	4	0	4	Flush unaligned load side 1
	5	0	5	Flush unaligned store side 1
	6	0	6	Flush from LRQ SHL, LHL side 1
	7	0	7	Flush SRQ LHS side 1
	8	1	0	L1 D-cache load side 0
	9	1	1	L1 D-cache store side 0
	10	1	2	L1 D-cache load miss side 0
	11	1	3	L1 D-cache store miss
	12	1	4	L1 D-cache load side 1
	13	1	5	L1 D-cache store side 1
	14	1	6	L1 D-cache load miss side 1
	15	1	7	L1 D-cache entries invalidated from L2

Table 10-7. Performance Monitor Bus Assignments (Page 5 of 8)

	Bits [0:31]	Byte Lane	Bits [0:7]	Event Description
	16	2	0	SRQ store forwarding side 0
	17	2	1	SRQ slot 0 valid
	18	2	2	LRQ slot 0 valid
	19	2	3	LSU0 reject
	20	2	4	SRQ store forwarding side 1
	21	2	5	SRQ slot 0 allocated
	22	2	6	LRQ slot 0 allocated
	23	2	7	LSU1 reject
	24:27	3	0:3	L1 cache reload data source
	28	3	4	L1 cache reload data valid
	29	3	5	LMQ slot 0 valid
	30	3	6	LMQ slot 0 allocated
	31	3	7	LMQ full
	32:47	0:1	0:7	Nothing
	48	2	0	SRQ reject 0 - load hit store
	49	2	1	LMQ reject 0 - LMQ full or missed data coming
	50	2	2	LSU0 reject - reload critical data forward (CDF) or tag update collision
	51	2	3	LSU0 reject - ERAT miss
	52	2	4	SRQ reject 1 - load hit store
	53	2	5	LMQ reject 1 - LMQ full or missed data coming
	54	2	6	LSU1 reject - reload CDF or tag update collision
	55	2	7	LSU1 reject - ERAT miss
	56:58	3	0:3	L1 cache reload data source
	60	3	4	Marked L1 cache reload data source valid
	61	3	5	LMQ load-hit-reload merge
	62	3	6	Marked SRQ valid
	63	3	7	Nothing

IBM PowerPC 970MP RISC Microprocessor

Table 10-7. Performance Monitor Bus Assignments (Page 6 of 8)

	Bits [0:31]	Byte Lane	Bits [0:7]	Event Description
ISU: TTM0 = '01', TD_CP_DBGxSEL = '00' TTM1 = '10', TD_CP_DBGxSEL = '01'				
	0	0	0	Completion table full
	1	0	1	Floating-Point Register (FPR) mapper full
	2	0	2	Integer Exception Register (XER) mapper full
	3	0	3	FPU0 issue queue full
	4	0	4	CR mapper full
	5	0	5	Branch (BR) issue queue full
	6	0	6	Link Register/ Counter Register (LR/CTR) mapper full
	7	0	7	FPU1 issue queue full
	8	1	0	FXU0/LSU0 issue queue full
	9	1	1	CR issue queue full
	10	1	2	LRQ full
	11	1	3	SRQ full
	12	1	4	FXU1/LSU1 issue queue full
	13	1	5	Flush originated by LSU
	14	1	6	Flush originated by branch miss predict
	15	1	7	Flush (includes LSU, branch miss predict)
	16:18	2	0:2	Instructions dispatched count
	19	2	3	Dispatch valid
	20	2	4	Dispatch reject
	21	2	5	Nothing
	22	2	6	Group experienced a branch redirect
	23	2	7	Group experienced a branch miss predict
	24	3	0	Nothing
	25	3	1	Dispatch blocked by scoreboard
	26	3	2	FXU0 produced a result
	27	3	3	Duration of the external interrupt enable in the Machine State Register (MSR[EE] = '0')
	28	3	4	Nothing
	29	3	5	General Purpose Register (GPR) mapper full
	30	3	6	FXU1 produced a result
	31	3	7	MSR(EE) equals '0' and interrupt pending

Table 10-7. Performance Monitor Bus Assignments (Page 7 of 8)

	Bits [0:31]	Byte Lane	Bits [0:7]	Event Description
Vector: TTM0 = '11', TD_CP_DBGxSEL = '01'				
	0	0	0	Arithmetic logic unit (ALU) issue queue full
	1	0	1	Vector permute (VPERM) issue queue full
	2	0	2	ALU issue marked instruction
	3	0	3	VPERM issue marked instruction
	4	0	4	Saturation zero to one
	5	0	5	VPU mapper full
	6	0	6	Store issue marked instruction
	7	0	7	Nothing
8:15 below selected by OVMA.USADec2.CHICKEN1.IO.L2(7) = '0' (default)				
	8	1	0	Finish with IMR
	9	1	1	Generic forward
	10	1	2	Vector ALU issue count
	11	1	3	Denormalized traps
	12	1	4	Saturation bit set
	13	1	5	Finish contention cycle
	14	1	6	Nothing
	15	1	7	Nothing
16:19 below selected by OVMP.RPRPM.MODE_PMON_MISC.IO.L2 = '0' (default)				
	16	2	0	Instruction finish with IMR
	17	2	1	Forwarding occurred from VPERM or ALU or load
	17	2	2	Issue valid
	19	2	3	Saturation count for valid instruction
STS: TTM1 = '11', TD_CP_DBGxSEL = '01'				
	0	0	0	L2 cache access collision with L2 prefetch (Data Stream Touch [DST])
	1	0	1	L2 cache access collision with L2 prefetch (non-DST)
	2	0	2	L2 cache access for store
	3	0	3	L2 cache miss on store access (recent [R], shared [S], or invalid [I])
	4	0	4	L2 cache miss; bus response is modified intervention
	5	0	5	L2 cache miss; bus response is shared intervention
	6	0	6	I = '1' store operation (before gathering)
	7	0	7	I = '1' store operation completed on bus

IBM PowerPC 970MP RISC Microprocessor
Table 10-7. Performance Monitor Bus Assignments (Page 8 of 8)

	Bits [0:31]	Byte Lane	Bits [0:7]	Event Description
	8	1	0	I = '1' load operation completed on bus
	9	1	1	Cacheable store operation (before gathering)
	10	1	2	Master bus transactions completed
	11	1	3	Master bus transactions retried
	12	1	4	Master L2 cache store transaction on bus was retried
	13	1	5	Master L2 cache read transaction on bus was retried
	14	1	6	Master SYNC operation completed
	15	1	7	Master SYNC operation retried
	16	2	0	Load or store dispatch retries because of castout (CO) conflicts
	17	2	1	Load or store dispatch retries because of snoop conflicts
	17	2	2	Load or store dispatch retries
	19	2	3	All read/claim state machines busy
	20	2	4	All CO state machines busy
	21	2	5	All snoop state machines busy
	22	2	6	Cacheable store queue full
	23	2	7	I = '1' store queue full
	24	3	0	Snoop (external)
	25	3	1	Snoop state machine dispatched
	26	3	2	Snoop retried due to any conflict
	27	3	3	Snoop retried because all snoop state machines busy
	28	3	4	Snoop caused cache transition from modified (M) to exclusive (E) or shared (S)
	29	3	5	Snoop caused cache transition from E to S
	30	3	6	Snoop caused cache transition from E or S to recent (R) or invalid (I)
	31	3	7	Snoop caused cache transition from M to I

10.7 Enabling the Performance Monitor Counters

10.7.1 Machine States

The eight counters in the 970MP performance monitor can be enabled to count events as a result of a number of machine state conditions and triggering events. The machine state conditions and triggering events are enabled by the settings of the MMCR0, MMCR1, and MMCRA Register control fields, combined with the values of the performance monitor-related bits in the MSR and other SPRs. While machine states and triggering events are closely related in their effect on performance monitor behavior, it is easier to understand them if the two are first considered separately, as described in this section for the machine states and in *Section 10.7.2* on page 244 for the triggering events.

The term machine state condition as it is used here includes:

- Supervisor versus user (problem) state (MSR[PR], MMCR0[FCS, FCP])
- Marked versus unmarked process state (MSR[PMM], MMCR0[FCM1, FCM0])
- Conditional versus unconditional counting state (MMCR0[FC], MMCRA[FC1:4], FC[5:8], CTRL[31], MMCRA[FCWAIT])
- Wait state versus non-wait state (CTRL[31], MMCRA[FCWAIT])

By combining the state of the machine with the events selected for counting, many different aspects of performance can be obtained for a given program.

For example, a programmer might want to gather statistics on only a particular process. This can be done by doing the following steps:

1. Set the appropriate bits in MMCR0 that enable counting only for a marked process.
2. Before each run of the selected process begins, set the performance monitor mode bit in the Machine State Register (MSR[PMM]) to the value that marks that process.
3. After each run of the selected process ends, set the performance monitor mode bit to the value that unmarks that process.

Another example follows:

The performance monitor can be set up to count only when the machine is in the supervisor state by ensuring that the MMCR0 bits that specify counting are enabled only when the machine is in the supervisor (privileged) state and are disabled when the machine is in the user (problem) state.

Table 10-8 on page 244 illustrates several different counting scenarios.

IBM PowerPC 970MP RISC Microprocessor
Table 10-8. Examples of Event Counter Enabling States

Counting State	MMCR0[Bit] = Value	MSR[Bit] = Value
Disable all counting	FC = '1'	Does not count for all values of PR, PMM
Enable all counting	FC = '0'	Counts ¹ for all values of PR, PMM
Enable counting in supervisor state only	FCP = '1', FCS = '0'	Counts when PR = '0'
Disable counting in supervisor state only	FCS = '1', FCP = '0'	Counts when PR = '1'
Enable counting in user (problem) state only	FCS = '1', FCP = '0'	Counts when PR = '1'
Disable counting in user (problem) state only	FCS = '0', FCP = '1'	Counts when PR = '0'
Enable counting for marked processes only	FCM0 = '1', FCM1 = '0'	Counts when PMM = '1'
Disable counting for marked processes only	FCM0 = '0', FCM1 = '1'	Does not count when PMM = '1'
Enable counting for unmarked processes only	FCM0 = '0', FCM1 = '1'	Counts when PMM = '0'
Disable counting for unmarked processes only	FCM0 = '1', FCM1 = '0'	Does not count when PMM = '0'
Enable counting for marked processes in supervisor state only	FCP = '1', FCS = '0', FCM0 = '1', FCM1 = '0'	Counts when PR = '0' and PMM = '1'
Enable counting for unmarked processes in supervisor state only	FCP = '1', FCS = '0', FCM0 = '0', FCM1 = '1'	Counts when PR = '0' and PMM = '0'
Enable counting for marked processes in user (problem) state only	FCP = '0', FCS = '1', FCM0 = '1', FCM1 = '0'	Counts when PR = '1' and PMM = '1'
Enable counting for unmarked processes in user (problem) state only	FCP = '0', FCS = '1', FCM0 = '0', FCM1 = '1'	Counts when PR = '1' and PMM = '0'

Note:

1. All enables are based on whether the other MMCRx and/or MSR bits permit this action.

10.7.2 Trigger Events

Machine states that determine counter activity have been presented in *Section 10.7.1* on page 243. Several examples of states and their corresponding counter behaviors were shown in order to illustrate some of the more common uses. In addition to counting enable/disable for various machine states, there are certain kinds of performance monitor conditions and events that can affect performance monitor counting activity. The occurrence of these conditions and events, which together are called trigger events, combined with the controls that enable the trigger events, can be used together with machine states to further control performance monitor activity.

The term trigger event as it is used for the 970MP performance monitor includes the following conditions:

- The time-base transition event can occur when a selected time-base bit changes from '0' to '1'. The time-base event setup uses the following fields: TB_REG[47, 51, 55, 63], HID0[13], MMCR0[TBSEL]. The time-base event enable uses the following field: MMCR0[TBEE]. The possibility of side effects when an enabled time-base event occurs uses the following fields: MMCR0[FCECE, TRIGGER].
- The counter negative condition for PMC1 can occur when the value in PMC1 is negative. The PMC1 counter negative condition setup uses the following field: PMC1[0]. The PMC1 counter negative condition enable uses the following field: MMCR0[PMC1CE]. The possibility of side effects when the PMC1 counter negative condition occurs uses the following fields: MMCR0[FCECE, TRIGGER].
- The counter negative condition for PMCj ($2 \leq j \leq 8$) occurs when the value in any PMCj is negative. The PMCj counter negative condition setup uses the following field: PMCj[0]. The PMCj counter negative con-

dition enable uses the following field: MMCR0[PMCjCE]. The possibility of side effects when the PMCj counter negative condition occurs uses the following fields: MMCR0[FCECE, TRIGGER].

Note: The three kinds of trigger events can occur independently of each other and independently of whether the condition or event is enabled. For example, a counter can go negative regardless of whether the counter negative condition for that counter is enabled. However, the side effects of that counter going negative will be seen only if the counter goes negative, the counter negative condition for that counter is enabled, and the side effects are also enabled.

By combining the trigger events and their respective enables with the time-related values obtained in the counters, performance profiles of different kinds of events can be obtained for a given program.

10.7.2.1 Time-Base Transition Events

Time-base events occur when the selected time-base bit (TB_REG[47, 51, 55, 63], HID0[13], MMCR0[TBSEL]) changes value from '0' to '1'. If the time-base transition event is enabled (MMCR0[TBEE]), then any performance monitor action that is started by the occurrence of a trigger event (MMCR0[TRIGGER]) will be initiated. Any performance monitor action that is stopped by the occurrence of a trigger event (MMCR0[FCECE]) will be terminated. In multiprocessor systems with the Time-Base Registers synchronized among the processors, time-base transition events can be used to correlate the performance monitor data obtained by the several processors provided that software has specified the same TBSEL value for all of the processors in the system.

The frequency of the time base is implementation dependent, and a system service routine should be invoked to obtain the frequency before a value for TBSEL is chosen.

10.7.2.2 PMC1 Counter Negative Condition Events

The PMC1 counter negative condition occurs when PMC1[0] equals '1', which can occur either through counting from '0', counting from a positive value greater than '0', or through setting the PMC1[0] bit to '1' in an interrupt or service routine. If the PMC1 negative count condition is enabled (MMCR0[PMC1CE]), any performance monitor action that is started by the occurrence of a trigger event (MMCR0[TRIGGER]) will be initiated, and any performance monitor action that is stopped by the occurrence of a trigger event (MMCR0[FCECE]) will be terminated when PMC1[0] becomes negative.

For example, if the PMC1 negative count condition is to be used to start the other PMCj counters after a designated number of cycles has elapsed, the set up would be as follows:

1. PMC1 is set to the value (x'8000 0000' - <number of cycles>).
2. PMC1SEL is set up to count cycles.
3. MMCR0[PMC1CE] is set to enable the PMC1 negative counter condition.
4. TRIGGER is enabled.

In this case, it is not necessary to enable the PMC1 counter negative condition because the TRIGGER uses either PMC1 negative or an enabled trigger event to start the enabled PMCjs counting.

10.7.2.3 *PMCj* ($2 \leq j \leq 8$) Counter Negative Condition Events

The *PMCj* ($2 \leq j \leq 8$) counter negative condition event occurs when *PMCj*[0] equals '1' ($2 \leq j \leq 8$), which can occur through counting from '0', counting from a positive value greater than '0', or through setting the *PMCj*[0] ($2 \leq j \leq 8$) bit to '1' in an interrupt or service routine. If the *PMCj* ($2 \leq j \leq 8$) counter negative condition event is enabled (*MMCR0*[*PMCjCE*]), any performance monitor action that is started by the occurrence of a trigger event (*MMCR0*[*TRIGGER*]) will be initiated, and any performance monitor action that is stopped by the occurrence of a trigger event (*MMCR0*[*FCECE*]) will be terminated, when any of the *PMCj* ($2 \leq j \leq 8$) counters become negative.

10.7.3 Method for Enabling or Disabling Performance Monitor Counting

This section describes the fundamental mechanism that should be used to place the selected values into the Performance Monitor Registers and other SPRs to initiate and terminate counting.

Once all of the control and event selection choices have been made, there are 32-bit and 64-bit values that must be placed into each of the registers associated with performance monitor counting. These values are placed in the registers with the **mtspr** instruction, which can be executed only in supervisor mode.

Note: If the Performance Monitor Counter Register values are changed while the performance monitor is enabled for counting, then the resulting performance monitor state is undefined.

The basic steps for enabling the performance monitor counting activity are as follows:

1. Enter supervisor mode.
2. Execute a synchronizing instruction.
3. Execute all **mtspr** instructions that place values to enable counting into the performance monitor and other Special Purpose Registers except for *MMCR0*.
4. Execute all **mtspr** instructions to initialize the performance monitor counters to the appropriate values.
5. Execute the **mtspr** instructions that place the value to enable counting into *MMCR0*.
6. Execute a synchronizing instruction.
7. Exit supervisor mode.
8. Start the program for which counting is to be done.

When the program being counted completes, the following steps are used to disable performance monitor counting:

1. Enter supervisor mode.
2. Execute a synchronizing instruction.
3. Execute the **mtspr** instructions that places the value to disable counting into *MMCR0*.
4. Execute all **mfspir** instructions to read the values from the performance monitor counters.
5. Execute a synchronizing instruction.
6. Exit supervisor mode.

The performance monitor counters contain either the number of times the selected event has occurred or the number of cycles during which the monitored event occurred after the performance monitor was enabled for counting.

Note: In either case, any counted events that occur after the performance monitor counting is enabled in supervisor mode, but before the program under study is entered, will be included in the overall count value. In the same way, any counter events that occur after the program under study is exited, but before the performance monitor counting is disabled, will also be included in the overall count value.

10.8 Performance Monitor Exceptions

The three trigger events described in *Section 10.7.2* beginning on page 244 can cause a performance monitor exception to occur and the subsequent performance monitor exception to be generated if the following sequence of actions occurs:

1. The trigger event occurs.
2. The trigger event is enabled.
3. The performance monitor exception is enabled.
4. External interrupts are enabled.

Note: This is the highest priority interrupt.

A performance monitor exception can be disabled for a given trigger event by disabling that trigger event (MMCR0[TBEE, PMC1CE, PMCjCE]). Performance monitor exceptions can be disabled for all of the trigger events collectively by disabling the performance monitor exception (MMCR0[PMXE]). The performance monitor exception, which is classified as an external interrupt, can be disabled either by disabling the performance monitor exception (MMCR0[PMXE]) or by disabling the external interrupts (MSR[EE]).

When an enabled condition or event occurs and a performance monitor exception is taken, the performance monitor exception is disabled by the hardware so that the SIAR and SDAR will contain the address and data information for an instruction that was executing at or around the time of the exception. Because the contents of the SIAR and SDAR can be altered if and only if MMCR0[PMXE] equals '1', the contents of those registers can change only if software re-enables the performance monitor exception. If such a re-enable is done and multiple performance monitor exceptions occur before the performance monitor exception is taken, then the exception reflects the most recently occurring such exception. Data from the previous exceptions are lost.

If a performance monitor exception is pending and the value of MSR[EE] is changed from '0' to '1', then the performance monitor exception will occur before the next instruction is executed provided no higher priority exception exists. The occurrence of the performance monitor exception cancels the performance monitor exception.

In summary, the following registers are set when a performance monitor exception occurs:

- SRR0[0:63] is set to the effective address of the instruction that the processor would have attempted to execute next if no interrupt conditions were present.
- SRR1[33] is set to '1' if the contents of the SDAR and the SIAR are associated with the same instruction.
- Other SRR0 and SRR1 bits are set as described in *Chapter 4 Exceptions*.
- SIAR is set to the effective address of the marked instruction, where the marked instruction is an instruction that was executing, possibly out-of-order, at or around the time that the performance monitor exception occurred. The contents of the SIAR can be altered by the processor if and only if MMCR0[PMEE] is set to '1'. Thus, after a performance monitor exception occurs, the contents of the SIAR is not altered by the processor until software sets MMCR0[PMEE] to '1'. After software sets MMCR0[PMEE] to '1', the contents of SIAR is undefined until the next performance monitor exception occurs.

IBM PowerPC 970MP RISC Microprocessor

- SDAR is set to the effective address of the storage operand of an instruction that was executing, possibly out-of-order, at or around the time that the performance monitor exception occurred. This storage operand is called the marked data and can be, but need not be, the storage operand (if any) of the marked instruction. If the performance monitor causes a performance monitor exception, the SRR1 indicates whether the marked data is in fact the storage operand of the marked instruction. The contents of the SDAR can be altered by the processor if and only if MMCR0[PMEE] is set to '1'. Thus, after a performance monitor exception occurs, the contents of the SDAR is not altered by the processor until software sets MMCR0[PMEE] to '1'. After software sets MMCR0[PMEE] to '1', the contents of SDAR are undefined until the next performance monitor exception occurs.
- MSR is set the same as for other external interrupts.

10.9 Instruction Matching and Sampling

The 970MP performance monitor provides a facility for the detailed analysis of instruction flow by sampling particular instructions or classes of instructions. Instructions must pass through three stages of eligibility to be marked for sampling. The contents of the SIAR/SDAR reflect the marked instruction that is currently executing.

10.9.1 Stage 1 Eligibility

There are two instruction marking facilities for stage 1:

- **IFU** - Uses the IMC array to either set or clear the mark (imr) bit associated with any matching instruction. This imr bit, along with the branch (B), first (F), split (S), and last (L) predecode bits, are retained in the L1 instruction cache along with each instruction. All instructions with the imr bit set are eligible for stage 2 of marking.
- **IDU** - Uses the BFSL predecode bits (set independent of any instruction matching in the IMC array) along with the imr_match and imr_mask fields in the MMCRA. All instructions with predecode bits (BFSL) matching imr_match when ANDed with imr_mask are eligible for stage 2 of marking.

Which facility is used depends on the MMCRA imr_select field. If imr_select equals '0', the IDU facility will be used. Otherwise, if imr_select equals '1', the IFU facility will be used.

10.9.2 Stage 2 Eligibility

Any eligible instructions from stage 1 are further filtered by the imr_mask field in the MMCRA:

- '00': All IOPs
- '01': Only IOPs resulting from microcode expansion
- '10': Only one IOP per PowerPC instruction
- '11': First IOP to go to the LSU for every PowerPC load/store instruction

10.9.3 Stage 3 Eligibility

Any eligible instructions from stage 2 are marked if the internal *ok_to_sample* performance monitor signal is asserted. This results in at most one marked instruction in the pipeline at a time. Another eligible instruction will be marked after a marked instruction completes or the previous marking cycle has timed out (set by the SCOM x'240' IDLE field).

10.10 IFU Instruction Matching Facility

The PowerPC instruction matching by opcode or extended opcode is performed by the IFU Instruction Matching facility implemented in hardware through the use of an instruction match CAM (IMC) array. When a PowerPC instruction is fetched from memory, the IFU instruction matching facility compares the instruction with the opcode/extended opcode mask values in each of the IMC array rows. If a PowerPC instruction matches one or more IMC array row masks, the IFU predecode bits associated with the marked instruction are set based on the value of the IMC function bit in each of the matched IMC array rows. The IMC function bits and descriptions of the subsequent processing for the PowerPC instruction matched for each function bit are as follows:

- Force only (fo) forces the IDU to place the PowerPC instruction in a group by itself by setting the predecode bits for the instruction as 'first' and 'last' in the group. The IMC fo function bit is used for hardware debug and workaround. It is only accessible by using scan.
- Instruction marking (imr), which is used for performance monitor instruction marking, causes the IDU to recognize that the instruction is IFU-eligible for marking. The IMC imr function bit is used by the performance monitor for marked instruction events and threshold event counts. It is accessible to the user by using the supervisor mode **mtimc/mfimc** instruction.

In addition to the IFU predecode bits associated with the IMC function bits, other IFU predecode bits—based on the instruction type—are bundled with each instruction in the IFU instruction cache.

Note: As long as an instruction resides in the level 1 instruction cache, its match bit will remain unchanged. If the match condition for an instruction changes, then the Level 1 instruction cache should be flushed to ensure proper setting of the match bits for all instructions.

10.10.1 Overview of the IFU Instruction Matching Facility

Each processor core includes an IFU instruction matching facility, implemented as the IMC array, which is used to maintain the kinds of IFU instruction matching requests and the mask values used for each IFU matching request. The method of reserving an IMC array row differs depending on whether the row is being requested for use by the hardware patch/debug facility (using scan only) or for the performance monitor marked instruction or threshold event facility (using the supervisor **mtimc** and supervisor or user **mfimc** instructions). All IMC array rows required for hardware patch/debug operations (fo reservations) are reserved during the 970MP power-on reset (POR) scan sequence. Any IMC array rows that have not been fo-reserved during system initial program load (IPL) can be requested by an executing program for use by the performance monitor.

Before an executing program requests an IMC array row, the program must determine which IMC array rows are available to software; that is, which rows are not fo-reserved. This information is available by reading the IMC Special Purpose Register (SPR) with the **mfimc** instruction, which can be executed only in supervisor mode. An executing program can request any IMC array rows that are not fo-reserved by writing the IMC SPR with the **mtimc** instruction, which can be executed only in supervisor mode.

IBM PowerPC 970MP RISC Microprocessor
10.10.2 IMC Array

The IMC array, which is contained in the IFU, consists of eight row entries as follows:

- Six rows (0 through 5) support a 17-bit partial instruction match of the opcode field in instruction bits (0:5) and extended opcode fields in instruction bits (21:31).
- Two rows (6 and 7) combined support a full 32-bit instruction match of all the instruction fields in instruction bits (0:31).

Each IMC array row includes fields for:

- The imr function bit (bit 60, called the Mark bit) that determines the predecode tag value sent to the IDU. The imr bit is programmable through the Instruction Match facility.
- The two mask values that together encode the instruction match criteria (called v0 and v1).
- The machine configuration this match applies to (PR, FP Available, VPU Available). Note that the PR bit in the MSR determines the Privileged state if a '0' and the Problem state if a '1'.
- Optional replacement field for the BFSL (predecode) bits to replace for a matched instruction (SPR cannot access these fields).

The programmer's model view of the IMC array is shown in *Table 10-9* and *Table 10-10*.

Table 10-9. Partial Match Rows in the IMC Array

0:16	17	18	19	20	21:37	38	39	40	41	42:52	53:58	59	60	61:63
v0(17)	N/A	MSR [PR]	MSR [FP]	MSR [VPU]	v1(17)	N/A	MSR [PR]	MSR [FP]	MSR [VPU]	N/A	BFSL Replacement	Replace	Mark	Index
														0
														1
														2
														3
														4
														5

Table 10-10. Complete Match Rows in the IMC Array

0:31	32	33	34	35	36:52	53:58	59	60	61:63
v0 (Row 6) / v1 (Row 7)	N/A	MSR [PR]	MSR [FP]	MSR [VPU]	N/A	BFSL Replacement	Replace	Mark	Index
									6
									7

The following are some rules and general facts about the IMC array features and use:

- The IMC array row with full 32-bit v0 and v1 masks can be used to match the opcode, the extended opcode, and all other fields for any PowerPC instruction.
- The IMC array rows with 17-bit mask values cannot be used to match branch instructions.
- The 17-bit v0 and v1 mask values can be used to match only those bits of a PowerPC instruction (but not branch instructions) that represent the opcode and extended opcode bit fields of that instruction and cannot be used to match any other instruction bit fields such as the register fields, shift fields, mask fields, reserved fields, immediate fields, or the rc bit.
- The bits of v0 and v1 that correspond to any fields other than the opcode and extended opcode bit fields of instruction bits (21:31) should be set to the 'don't care' v0 and v1 value as is explained in *Section 10.10.5 The v0 and v1 Mask Criteria* on page 253.
- The 17-bit v0 and v1 bits (0:5) and v0 and v1 bits (6:16) correspond to instruction bits (0:5) and instruction bits (21:31).
- The 3 mode bits (PR, FP, VP) correspond to MSR[PR], MSR[FP], and MSR[VP].

10.10.3 Reading the IMC SPR with the mfimc Instruction

The IMC SPR is read in order to determine which IMC array rows are fo-reserved. The image of the IMC SPR that is obtained by executing the **mfimc** instruction (which can be executed in supervisor and user mode) is referred to as the **patch map**. The programmer's model view of the patch map is shown in *Figure 10-4*.

Figure 10-4. Patch Map

Patch Map Bit Number (IMC Array Row Address)								
0:55	56 (0)	57 (1)	58 (2)	59 (3)	60 (4)	61 (5)	62 (6)	63 (N/A)
Reserved ¹								Reserved

1. The designation 'reserved' is used both to indicate bits in the patch map that are not used for this implementation, as well as to identify the fields that are not accessible when using the **mfimc** instruction.

A patch map status bit value of '1' indicates that the associated IMC array row is fo-reserved and cannot be altered by the executing program. If the status bit value is '1', an **mtimc** to the associated IMC array row is treated as a no-op.

The facts summarized below emphasize what information is and is not available from the patch map:

- The only information that the patch map provides about the IMC array rows is whether a row is fo-reserved.
- The patch map provides no information about reservations made by the performance monitor IMR facility.
- An IMC array row that is reserved by the performance monitor IMR facility will not show a patch map status bit of '1' if the patch map is read with the **mfimc** instruction.
- There is no **mfimc**-like instruction that will help an executing program determine what v0 and v1 values were used when a performance monitor IMR facility request was made.
- All information about IMC array use must be maintained by the executing program.
- If an IMR reservation is made for an available IMC array row and the v0, v1 mask used for the IMR reservation is the same as a v0,v1 mask that is already being used for an fo reservation, the instruction selected by the v0, v1 mask will be correctly processed for both the imr request and the fo request.

IBM PowerPC 970MP RISC Microprocessor

As an example of the value returned by the **mfimc** instruction under a particular IMC array use scenario, assume that the following IMC array row is reserved:

- The IMC array row at address 5 is fo-reserved,

A **mfimc** instruction executed for that IMC array would return the patch map shown in *Table 10-11*.

Note: The bits for IMC array rows 1 and 4 are not set in the patch map.

Table 10-11. IMC SPR Patch Map Sample Results

Patch Map Bit Number (IMC Array Row Address)										
0		55	56 (0)	57 (1)	58 (2)	59 (3)	60 (4)	61 (5)	62 (6)	63(N/A)
Reserved			1	0	0	0	0	1	1	Reserved

10.10.4 Writing the IMC SPR With the mtime Instruction

After an executing program determines which IMC array rows are fo-reserved (by doing a **mfimc** and seeing which patch map bits are set to '1'), the program should initialize an IMC_reservation data structure, which it should then use to track all fo/imr-reservations.

The program can make an IMR reservation of any available IMC array row through use of the **mtime** instruction (which can be executed only in supervisor mode). The **mtime** instruction is used to select the IMC array row, provide the v0 and v1 mask values, and set the imr bit. After a performance monitor IMR request is successfully completed, the requesting program should update its IMC_reservation data structure to record the reservation and the v0, v1 mask values.

A performance monitor IMR request remains in effect on a processor until it is:

- Canceled by an **mtime** instruction that sets the imr bit to '0',
- Changed by an **mtime** instruction that changes the IMC array row fields v0, v1 and writes the imr bit to '1',
- Cleared or replaced by a system reboot, or
- Overwritten by the service processor using scan or SCOM.

When an existing performance monitor IMR request is changed or canceled by a subsequent **mtime** instruction, the executing program must update its IMC reservation data structure and invalidate the I-cache in order to reset the match bits set by a previous IMR reservation. Otherwise, stale instruction marks from the previous IMC setup might make the performance measurements unreliable, meaning old marks might still be encountered and new marks might not always occur depending on the state of the I-cache.

The programmer's model of the IMC SPR for the **mtime** instruction differs slightly for requests of the 32-bit and the 17-bit instruction match IMC array rows. The IMC array rows for 17-bit matches, which are at IMC array row addresses 0 - 5, are written with a single **mtime** instruction. It specifies the IMC array row address, the 17-bit opcode and extended opcode instruction mask values for v0 and v1, the machine mode mask values for v0 and v1, and it sets the imr bit to '1'. The image of the IMC SPR that is used when executing the **mtime** instruction for IMC array row addresses 0 - 5 is shown in *Table 10-12 on page 253*.

Table 10-12. IMC SPR for a 17-Bit Match

IMC Row Bit Numbers (IMC Array Row Fields)								
0:16	17	18:20	21:37	38	39:41	42:59	60	61:63
$v0_{[0-5,21-31]}$	Reserved	$V0_{PR,FP,VP}$	$v1_{[0-5,21-31]}$	Reserved	$v1_{PR,FP,VP}$	Reserved1	IMR1	IMC Row Address
1. The designation 'reserved' is used both to indicate bits in the IMC SPR that are not used for this implementation as well as to identify the fields that are not accessible when using the mtimc instruction.								

The IMC array row for 32-bit matches, which is at IMC array row address 6 and 7, is written with two **mtimc** instructions. Each specifies an IMC array row address, a 32-bit instruction mask value, and an imr bit value as follows:

- The first **mtimc** instruction sets the IMC SPR bits $(61:63) = 6_{10} = 110_2$, the IMC SPR bits $(0:31) = v0_{instruction[0-31]}$, the IMC SPR bits $(33:35) = v0_{PR,FP,VP}$ and the IMC SPR bit $(60) = '0'$.
- The second **mtimc** instruction sets the IMC SPR bits $(61:63) = 7_{10} = 111_2$, the IMC SPR bits $(0:31) = v1_{instruction[0-31]}$, the IMC SPR bits $(33:35) = v1_{PR,FP,VP}$ and the IMC SPR bit $(58) = '1'$.

The image of the IMC SPR that is used when executing the first **mtimc** instruction (for IMC array row address 6) is shown in Table 10-11 on page 252 and the image of the IMC SPR that is used when executing the second **mtimc** instruction (for IMC array row address 7) is shown in Table 10-13.

Table 10-13. IMC SPR Used when Writing the Second **mtimc** Instruction for a 32-Bit Match

IMC Row Bit Numbers (IMC Array Row Fields)							
0:31	32	33:35	36:59	60 imr	61:63 IMC Row Address		
v1 _[0-31]	Reserved	v1 _{PR,FP,VP}	Reserved	1	1	1	1

10.10.5 The v0 and v1 Mask Criteria

The mask criteria used for matching the values in the instruction bit fields and the machine state are based on the values in the v0 and v1 fields. Each pair of bits, $v0(n)$ and $v1(n)$; where n is 0–16, or n is 0–31, or n equals PR, FP; is interpreted as an encoded 4-value criterion. It determines how the corresponding instruction bit (m) is to be matched, where either m is 0–31 for a full instruction match or m is 0–5, 21–31 for an opcode/extended opcode match. Bit correspondences between v0, v1 bit numbers and instruction numbers depend on whether v0, v1 is a 17-bit or a 32-bit mask and are as follows:

- 17-bit mask opcode bits:
 $v0, v1(0:5)$ corresponds to instruction bits[0:5]
- 17-bit mask extended opcode bits:
 $v0, v1(6:16)$ corresponds to instruction bits[21:31]
- 32-bit mask full instruction bits:
 $v0, v1(0:31)$ corresponds to instruction bits[0:31]

The bit match criteria established by the four values of $v0(n)$, $v1(n)$ are shown in Table 10-14 on page 254.

Table 10-14. Encoding Bits v0 and v1 of the IMC Array Mask

v0 Value	v1 Value	Meaning
0	0	Never match (disable all)
0	1	Match a one (1)
1	0	Match a zero (0)
1	1	Always match (don't care)

10.10.6 Instruction Matching Examples

17-bit match using only instruction opcode (bits 0:5)

Load Doubleword: opcode = 58, extended opcode = N/A (don't care); PR=0; FP=1, VP=0/1

v0 = 0b0001011111111111 1101

v1 = 0b1110101111111111 1011

17 bit match using instruction opcode and extended opcode

Load Word and Zero Indexed: opcode = 31, extended opcode = 23; PR=1; FP=0/1, VP=0/1

v0 = 0b10000011111010001 1011

v1 = 0b01111100000101110 1111

10.11 IDU Instruction Sampling Facility

Another level of sampling activity—performed in the IDU—includes further processing for the imr and fo reservations made using the IFU Instruction Matching facility. Because of the way an instruction is processed through the two IDU selection stages, it is possible that the IDU instruction sampling processing can override previous IFU IMR marking.

The IDU instruction sampling facility that produces marked instructions for the instruction pipeline consists of two independent selection stages. In each of the two selection stages, the selection criteria for that stage determines which instructions pass out of that IDU selection stage as eligible to be marked. Because an instruction passes through each of the two IDU selection stages after it is processed for IFU IMR marking, it is possible that the IDU instruction sampling eligibility criteria can override previous IFU IMR marking. It is also possible that the IDU stage 2 processing can override IDU stage 1 processing if the criteria for each of the two selection stages are not set up correctly.

An eligible instruction that is marked by the IDU is referred to as a marked (sampled) instruction.

10.11.1 Overview of the IDU Instruction Sampling Facility

The IDU instruction sampling facility uses the IFU imr and predecode bits from the instruction cache, together with PMU/SCOM data and control fields, to determine which instructions are eligible to be marked (sampled) and when an instruction can actually be marked (sampled).

Operation of the IDU instruction sampling facility to determine which instructions are eligible for marking occurs continuously when performance monitor mode sampling is enabled (MMCR4[63] equals '1'). The choice of which instructions are eligible to be marked is based on the values of the IFU imr and predecode bits combined with the values of the select, mask, match, mark, and filter fields. The IDU continuously design-

nates instructions as eligible to be marked based on the above fields, but the IDU only marks instructions when sampling is enabled and the performance monitor signals the IDU that it is *ok_to_sample*. Only one marked group flows through the instruction pipeline at a time.

IDU processing of an instruction based on fo IMC function bit is independent of IDU processing based on the IFU imr function bit, so a given instruction might be processed by the IDU for any or all of the fo and imr functions.

The IDU eligibility stages continuously produce eligible instructions. The *sample_enable* field combined with the performance monitor signal *ok_to_sample* control final marking as outlined below. The information in parenthesis corresponds to the annotations on the left in *Figure 10-5* on page 259.

- The *imr_select* field (MMCRA[49]) determines the method used for stage 1 instruction eligibility (eligibility stage 1 - method).
- Depending on the *imr_select* field value, the *imr_mask* field (MMCRA[52:55]) and the *imr_match* field (MMCRA[56:59]) can be used to determine the type of the stage 1 eligible instructions that pass through to stage 2 (eligibility stage 1 - type).
- The *imr_mark* field (MMCRA[50:51]) determines what type of stage 1 eligible instructions are to be considered for stage 2 eligibility (eligibility stage 2 - type).
- The *imr_filter* field (SCOM x'34' [11:12]) determines which and how many of the stage 2 eligible instructions actually become marked instructions in the pipeline (eligibility stage 2 - method).
- The performance monitor signal *ok_to_sample* determines whether marking is blocked or might resume (mark/no mark stage).

The IFU matching and the IDU instruction sampling activities occur continuously regardless of whether instructions are being marked by the IDU. The IDU can mark an instruction only when the performance monitor signals that marking is enabled. After an instruction is marked by the IDU, the performance monitor disables marking until either a marked instruction completes, the instruction is a store that is sent to the STS, or the performance monitor completion delay timer indicates that the marked instruction has been flushed.

Note: As long as an instruction resides in the Level 1 instruction cache, its match bit will remain unchanged. If the match condition for an instruction changes, the Level 1 instruction cache should be flushed to ensure proper setting of the match bits for all instructions.

10.11.2 Stage 1 Eligibility

The IDU uses the IFU predecode bits for branch, first, split, and last (shown in *Table 10-15* on page 256) and the *imr* bit stored with an instruction in the instruction cache to establish eligibility for marking.

Note: As long as an instruction resides in the Level 1 instruction cache, its *imr* match bit will remain unchanged. If the match condition for an instruction changes, the Level 1 instruction cache should be flushed to ensure proper setting of the *imr* match bits for all instructions.

The method used to choose IDU stage 1 eligible instructions is based on the value of the *imr_select* field (MMCRA[49]). Depending on the value of the *imr_select* field, a second decision point might be required to choose the type of stage 1 eligible instructions using the *imr_mask* field (MMCRA[52:55]) and the *imr_match* field (MMCRA[56:59]). These two scenarios, based on the value of the *imr_select* field value, are as follows:

- *imr_select* equals '1':
The IFU *imr* bit is used to determine stage 1 eligibility. All instructions with the IFU *imr* bit set are passed through to stage 2 as eligible for marking.

IBM PowerPC 970MP RISC Microprocessor

- `imr_select` equals '0':
The IFU BFSL predecode bits are used to determine stage 1 eligibility.

If `imr_select` equals '1', so that the IFU `imr` bit determines stage 1 eligibility, it is possible to choose values for `imr_mark` (IDU eligibility stage 2 - type) that will cancel the IMR eligibility created in stage 1.

If `imr_select` equals '0' and the IFU BFSL predecode bits are used to determine stage 1 eligibility, there are two further stages of processing to establish the type of instructions that are eligible:

1. The BFSL predecode bits for the instruction are ANDed with the `imr_mask` field to produce a 4-bit intermediate result, and
2. The 4-bit intermediate result is compared with the `imr_match` field. All instructions with an exact 4-bit match between the intermediate result and the `imr_match` field are passed through to stage 2 as eligible for marking.

To match all instructions, and thus pass all instructions through to stage 2 as eligible for marking, set the following values for the stage 1 method/type decision variables:

- `imr_select`: '0'
- `imr_mask`: '0000'
- `imr_match`: '0000'

The IFU BFSL predecode will be used, the mask will result in all zeros for the intermediate result, and the match will always succeed. The eligibility method chosen at stage 1 can determine what kind of instruction is counted for the performance monitor count event called "number of instructions completed," depending on how the eligibility criteria is set up in stage 2.

Table 10-15. IFU BSFL Predecode Bit Definitions (Page 1 of 2)

B(ranch)	S(plit)	F(irst)	L(ast)	Classification	Description
0	0	0	0	Simple	One IOP formed from one instruction with restrictions.
0	0	0	1	Simple-Last	IOP formed will be the last in the resultant dispatch group.
0	0	1	0	Simple-First	IOP formed will be the first in the resultant dispatch group.
0	0	1	1	Simple-Only	IOP formed will be the only in the resultant dispatch group.
0	1	0	0	Split	Two IOPs formed from one instruction; both must be in the same dispatch group.
0	1	0	1	Split-Last	Two IOPs formed from one instruction; the second IOP must be the last IOP in the resultant group.
0	1	1	0	Split-First	Two IOPs formed from one instruction; the first IOP must be the first IOP in the resultant dispatch group.
0	1	1	1	Split-Only	Two IOPs formed from one instruction; no other IOPs are present in the resultant dispatch group.
1	0	0	0	Branch - Conditional	IOP formed from a conditional branch instruction.
1	0	0	1	Branch - Unconditional	IOP formed from an unconditional branch instruction.
1	0	1	0	Illegal Opcode	Not a valid instruction.
1	0	1	1	Reserved	
1	1	0	0	Microcode - Hard	A nonprogrammable microcode sequence must be generated.
1. Field mask used to identify the Condition Register (CR) fields to be updated by the mtcrf instructions.					

Table 10-15. IFU BSFL Predecode Bit Definitions (Page 2 of 2)

B(branch)	S(split)	F(first)	L(ast)	Classification	Description
1	1	0	1	Microcode - Soft	A system software programmable microcode sequence must be generated.
1	1	1	0	Microcode - Conditional (otherwise: Split-Last)	A nonprogrammable microcode sequence must be generated if certain conditions are not met.
1	1	1	1	Microcode - Conditional (otherwise: Split-Only)	A non-programmable microcode sequence must be generated if certain conditions are not met (for example, the FXM ¹ field is not singular).
1. Field mask used to identify the Condition Register (CR) fields to be updated by the mtcrf instructions.					

10.11.3 Stage 2 Eligibility

The `imr_mark` field (`MMCR[50:51]`) value and then the `imr_filter` field (`SCOM x'340'[11:12]`) are used by the IDU to establish stage 2 eligibility for marking. The first decision point for IDU stage 2 eligibility is the type of stage 1 eligible instructions that can be stage 2 eligible; this determines final stage 2 eligibility. The second decision point for IDU stage 2 eligibility is the method of passing the eligible information to the mark/no mark stage; this determines what eligible instructions actually get marked.

The type of stage 1 eligible instructions that will be stage 2 eligible is based on the value of the `imr_mark` field (`MMCR[50:51]`). The `imr_mark` field value determines stage 2 eligibility of instructions as follows:

00	All stage 1 eligible IOPs are stage 2 eligible for marking.
01	Only stage 1 eligible IOPs that resulted from microcode expansion are stage 2 eligible for marking.
10	Only one IOP per stage 1 eligible PowerPC instruction is stage 2 eligible for marking (use this mode to make all PowerPC instructions eligible).
11	For every stage 1 eligible PowerPC instruction, the first IOP that goes to the LSU is stage 2 eligible for marking (use this mode to make all load/store instructions eligible).

The stage 1 eligibility method combined with the stage 2 eligibility type determines what kind of instruction is counted for the performance monitor count event called "number of instructions completed" as shown in *Section 10.11.6 Examples of Instruction Sampling Scenarios* on page 261.

After the type of stage 2 eligible instructions is established, the method of passing the stage 2 eligibility information to the mark/no mark stage is determined in two steps using the `imf_filter[11]` bit value and then using the `imr_filter[12]` bit value, which have the following functions:

`imr_filter[11:12]`

0x	No filtering
1x	Randomly sample eligible IOPs

IBM PowerPC 970MP RISC Microprocessor

Bit 12 selects one of two behaviors in sampling from microcode expansions (and has no effect on sampling from non-microcode groups):

- | | |
|----|--|
| 10 | Use the Good_Address mode of sampling microcode expansions |
| 11 | Use the More_Hits mode of sampling microcode expansions |

In Good_Address mode, there is at most one IOP in any microcode expansion that is eligible for sampling. This is either the first load/store IOP if there are any load/store IOPs in the expansion, or the first IOP in the final group of the expansion. If the random filter suppresses marking this IOP, then no IOP will be marked for the microcode expansion.

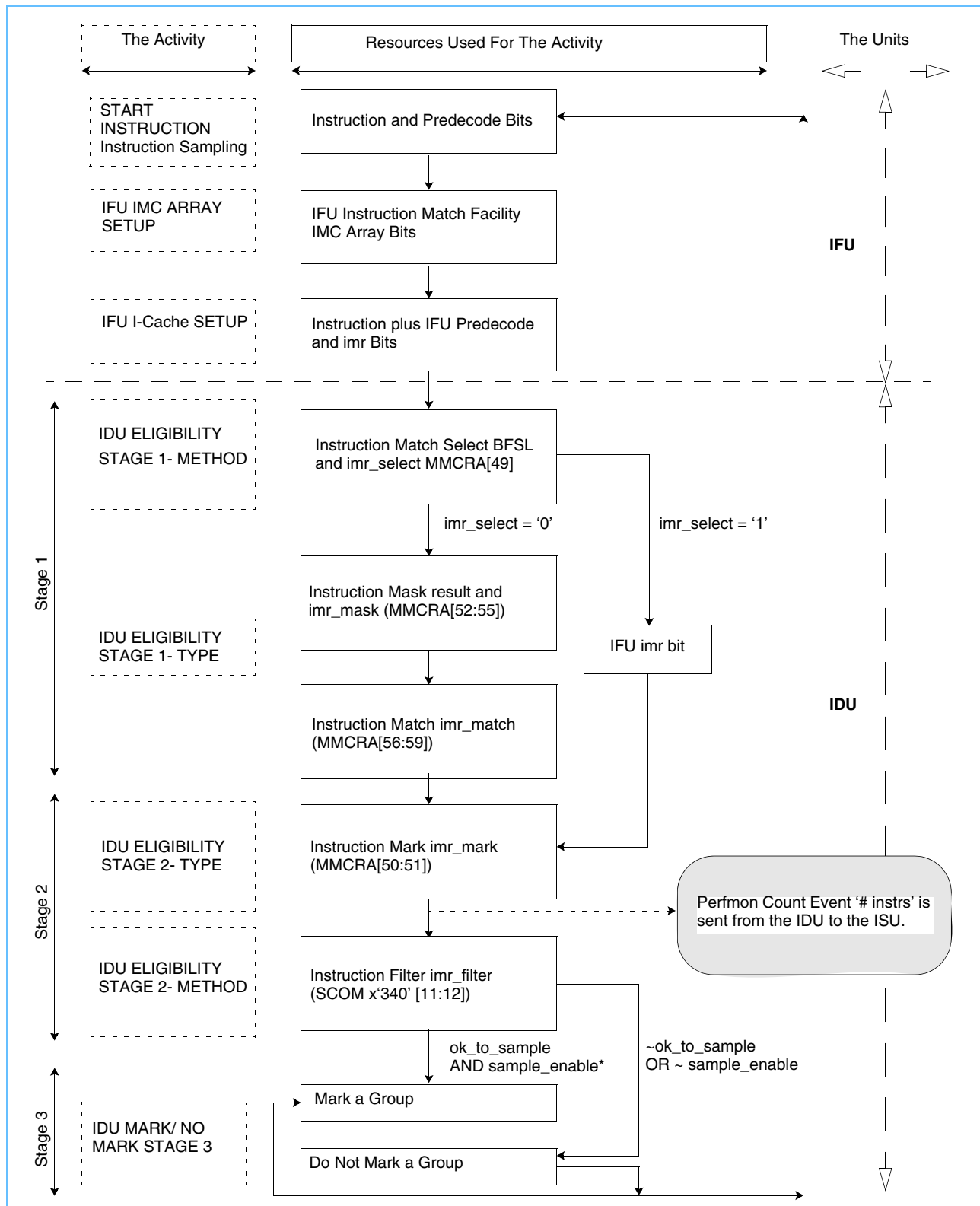
In More_Hits mode, multiple IOPs in a microcode expansion are eligible for sampling: the first load/store IOP in any group, or the first IOP of the final group. If the random filter suppresses marking the first of these IOPs, a subsequent one might still be sampled. (However, at most one will be marked in a single microcode expansion.)

The suggested mode for `imr_filter[11:12]` is '10'.

10.11.4 Stage 3 Mark/No Mark

The `sample_enable` field (`MMCRA[63]`) value and the `ok_to_sample` signal state are used by the IDU to complete the mark/no mark stage for instruction sampling. If marking is disabled with the `sample_enable` bit (`MMCRA[63]` equals '0'), then no group is marked by the IDU regardless of stage 2 eligibility. If marking is enabled with the `sample_enable` bit (`MMCRA[63]` equals '1'), then marking depends on the state of the handshake protocol between the IDU and the performance monitor signal `ok_to_sample`. The `ok_to_sample` signal is sent by the performance monitor to the IDU when the performance monitor determines that the previous marking cycle has completed successfully or has timed out. The handshake and synchronization mechanism are explained in *Section 10.11.5 Complete Masking, Matching, and Marking Cycle* on page 260.

Figure 10-5. IFU and IDU Instruction Sampling Flow



10.11.5 Complete Masking, Matching, and Marking Cycle

Instruction marking is set up by initializing the MMCRx[imr_xxx] fields, the SCOMyy[imr_filter] fields, and possibly the SCOMxx[xx_delay] for the kind of marking to be done. *Section 10.11.7 Enabling and Disabling Marking* on page 264 describes the procedure for initializing these registers plus any other registers to support performance monitor counted events. There are several examples at the end of this section that show the values of the imr_xxx fields for common marking and counting scenarios.

Once the instruction marking cycle is set up and enabled, instructions are continuously processed for eligibility as is described in *Section 10.11.2 Stage 1 Eligibility* on page 255, and in *Section 10.11.3 Stage 2 Eligibility* on page 257. The actual marking of a group described in *Section 10.11.4 Stage 3 Mark/No Mark* on page 258. The steps that occur from when the performance monitor *ok_to_sample* signal initiates marking until the next *ok_to_sample* signal initiates the next marking cycle are described in this section.

The overall timing of the marking cycle is driven by two performance monitor timers called the completion_delay counter and the idle_delay counter. The completion_delay counter is first initialized at the start of a marking cycle from the value in the sampling logic completion delay field (SCOM x'240' [COMPLN]). The idle_delay counter is initialized at the end of a marking cycle from the value in DELAY field (SCOM x'240'[DELAY]).

The purpose of the completion_delay counter is to predict the situation that the marked instruction has been flushed from the instruction pipeline before it can complete. The purpose of the idle_delay counter is to introduce a period of time between the end of a marking cycle (through either instruction completion or flush) and the start of the next marking cycle. The completion_delay and the idle_delay values must be greater than zero whenever matching or marking is enabled. Otherwise, the processor activity will be undefined.

A marking cycle begins when the performance monitor asserts the *ok_to_sample* signal for one cycle. The assertion of this signal (assuming that sample_enable equals '1') causes the IDU to mark the next group that enters stage 3 if it has passed all the stage 2 eligibility tests. After a group is marked in the IDU (this is a performance monitor count event), the IDU continues to process instructions for eligibility but does not mark another group until the next *ok_to_sample* signal is received from the PMU.

When the signal indicating that the group is marked in the IDU is sent from the IDU to the performance monitor (this is also a performance monitor count event), the performance monitor begins decrementing the completion_delay counter by one each cycle that the signal group_completed is asserted. If group_completed is not asserted, the completion_delay counter is not decremented.

This use of the completion_delay counter is intended to model a marked_group_flushed situation. The underlying assumption of this model and the default value of COMPLN equals 20 is as follows: if no marked group event occurs in any functional unit during a full wrap of the 20-entry completion buffer, then the marked group has been flushed. The completion_delay value must be greater than zero whenever instruction sampling is enabled or processor activity will be undefined.

If the completion_delay counter does not time out between when the signal indicating that the group is marked in the IDU is received by the performance monitor and when the next marked event signal is received by the performance monitor, the completion_delay timer is reset to the value in COMPLN. It resumes decrementing for each cycle that group_completed is asserted. At each stage of the marking cycle, the completion_delay counter is initialized, counts down whenever the signal group_completed is asserted, and either times out or is re-initialized when the next marked event occurs. Thus, at each stage of the marking cycle, the marked group is allowed the COMPLN number of cycles while groups are completing from the completion buffer before the marked group is considered flushed.

The sequence of events for a complete marking cycle, where a marked group moves through all of the instruction pipeline stages without being flushed, is as follows:

1. The performance monitor signal *ok_to_sample* is asserted for one cycle.
2. A group is marked when the IDU transfers the group to the ISU (performance monitor count event direct5[4]). The completion_delay counter is initialized to the value COMPLN and begins to decrement on each cycle that group_completed is asserted.
3. A marked group is dispatched (performance monitor count event direct1[2]). The completion_delay counter is reinitialized to the value COMPLN and begins to decrement on each cycle that group_completed is asserted.
4. A marked group is issued (performance monitor count event direct6[5]). The completion_delay counter is reinitialized to the value COMPLN and begins to decrement on each cycle that group_completed is asserted.
5. A marked group finishes (FPU: performance monitor count event direct7[4], FXU: performance monitor count event direct6[4], CRU: performance monitor count event direct4[5], BRU: performance monitor count event direct2[5], LSU: performance monitor count event direct8[4], any unit: performance monitor count event direct7[5]). The completion_delay counter is reinitialized to the value COMPLN and begins to decrement on each cycle that group_completed is asserted.
6. A marked group completes (performance monitor count event direct4[4]).
7. The idle_delay counter is initialized to the value DELAY and is then decremented to '0'.
8. The *ok_to_sample* signal is asserted for one cycle and the marking cycle begins again at step 2.

When the completion_delay counter times out (performance monitor count event direct5[5]), the performance monitor enters the marking cycle state where the idle_delay counter is set to the value DELAY and is then decremented to '0' (step 7 above). When the idle_delay counter times out, the performance monitor asserts the *ok_to_sample* signal for one cycle, and the marking cycle begins again.

The default idle_delay value is set to four. The idle_delay value must be greater than zero whenever matching/markings is enabled or processor activity will be undefined.

If a marked store is sent to the STS (performance monitor count event direct6[3]), the event called "marked store sent to STS" stops the marking cycle described previously and causes the performance monitor to enter the idle state. The performance monitor stays in the idle state until one of the signals "sampled store complete" (performance monitor count event direct1[3]) or "sampled store complete with intervention" (performance monitor count event direct3[3]) is received. At that time, the performance monitor resumes the marking cycle at marking cycle step 7 above.

When sampling_enable is set to zero, the performance monitor enters the idle state of the marking cycle until sampling is enabled again.

10.11.6 Examples of Instruction Sampling Scenarios

Follow the procedure in *Section 10.11.7 Enabling and Disabling Marking* on page 264 to place the values for instruction sampling into the appropriate Special Purpose Register fields. *Section 10.4 Performance Monitor Control Registers* on page 211 describes the performance monitor related registers and fields. To count marked events, the appropriate PMCxSEL fields should also be set up and the enable counting bit must be set to the enabled value. In each of the examples below, only the values of the Instruction Sampling Register fields are shown.

IBM PowerPC 970MP RISC Microprocessor

Example 1: Set up the instruction sampling facility to count PowerPC instructions as the performance monitor count event called "number instructions completed."

The values given here are those that set up the instruction sampling stages so that the performance monitor count event called "number instructions completed" will be the count for PowerPC instructions completed. If sampling is enabled, instructions will be randomly sampled. This is the recommended setting. The required field values are as follows:

imr_mark	'10'	Only one IOP per PowerPC instruction is stage 2 eligible for marking.
imr_select	'0'	
imr_mask	'0000'	
imr_match	'0000'	
imr_filter[11:12]	'10'	If sampling is enabled, randomly sample.

Example 2: Set up the instruction sampling facility to count IOP instructions as the performance monitor count event called "number instructions completed."

The values given here are those that set up the instruction sampling stages so that the performance monitor count event called "number instructions completed" will be the count for IOP instructions completed. The required field values are as follows:

imr_mark	'00'	All stage 1 eligible IOPs are stage 2 eligible for marking.
imr_select	'0'	
imr_mask	'0000'	
imr_match	'0000'	

Example 3: Set up the instruction sampling facility to count load/store instructions as the performance monitor count event called "number instructions completed."

The values given here are those that set up the instruction sampling stages so that the performance monitor count event called "number instructions completed" will be the count for load/store instructions completed. The required field values are as follows:

imr_mark	'11'	For every PowerPC load/store instruction, the first IOP that goes to the LSU is stage 2 eligible for marking.
imr_select	'0'	
imr_mask	'0000'	
imr_match	'0000'	

Example 4: Set up the instruction sampling facility to match or mask all PowerPC instructions that are BFSL-Split, and then sample eligible instructions that get through the random filter.

The values given here are those that set up the instruction sampling stages so that the performance monitor count event called "number instructions completed" will be the count for PowerPC BFSL-Split instructions completed. The required field values are as follows:

imr_mark	'00'	All stage 1 eligible IOPs are stage 2 eligible for marking.
imr_select	'0'	
imr_mask	'0100'	
imr_match	'0100'	
imr_filter[11:12]	'10'	If sampling is enabled, randomly sample.

Example 5: Set up the instruction sampling facility to match or mask all PowerPC instructions that are BFSL-Hard microcoded, and then sample all eligible instructions.

The values given here are those that set up the instruction sampling stages so that the performance monitor count event called "number instructions completed" will be the count for PowerPC BSFL-Hard microcoded instructions completed. The required field values are as follows:

imr_mark	'00'	All stage 1 eligible IOPs are stage 2 eligible for marking.
imr_select	'0'	
imr_mask	'1100'	
imr_match	'1100'	
imr_filter[11:12]	'00'	Pass all stage 1 eligible bits in the group.

Example 6: Set up the instruction sampling facility to IFU IMR match or mask all PowerPC add instructions, and then sample all eligible instructions.

The values given here are those that set up the instruction sampling stages so that the performance monitor count event called "number instructions completed" will be the count for IMC-marked PowerPC instructions completed. The required field values are as follows:

imr_mark	'10'	
imr_select	'1'	
imr_mask	'0000'	
imr_match	'0000'	
imr_filter[11:12]	'00'	If sampling is enabled, randomly sample.

10.11.7 Enabling and Disabling Marking

The processor comes out of reset with instruction marking disabled (MMCR0[63] equals '0') and with all of the MMCR0 and SCOM[jmr_xxx] fields set to zero. To set up the performance monitor for marking, follow these steps:

1. Enter supervisor mode.
2. Execute a synchronizing instruction or wait for any previous activity to complete.
3. Execute all **mtspr** instructions that place values to set up for marking (and counting if that is to be done) into the Performance Monitor Registers *except* for the counting enable in MMCR0 and the sample_enable in MMCR0.
4. Wait for all previous **mtspr** instructions to complete, and then execute the **mtspr** instruction to enable counting in MMCR0.
5. Wait for the previous **mtspr** instruction to complete, and then execute the **mtspr** instruction that enables marking in MMCR0.
6. Execute a synchronizing instruction or wait for the last **mtspr** instruction to complete.
7. Exit supervisor mode.
8. Start the program for which marking (and counting) is to be done.

Note: Any marked or counted events that occur after the performance monitor counting is enabled in supervisor mode, but before the program under study is entered, will be included in the overall mark or count activity. In the same way, any counter events that occur after the program under study is exited, but before the performance monitor marking or counting is disabled, will also be included in the overall mark/count activity.

When the program being marked or counted completes, the following steps disable performance monitor marking or counting:

1. Enter supervisor mode.
2. Wait for the previous **mtspr** instructions to complete, and then execute the **mtspr** instruction that disables marking in MMCR0.
3. Wait for the previous **mtspr** instruction to complete, and then execute the **mtspr** instruction to disable counting in MMCR0.
4. Execute a synchronizing instruction or wait for the last **mtspr** instruction to complete.
5. Wait for the counting operations that are in flight to complete and then execute the **mfspir** instructions to read the values from the performance monitor counters.
6. Execute a synchronizing instruction or wait for the last **mfspir** instruction to complete.
7. Exit supervisor mode.

Notes:

If marking is disabled while a marked instruction is still active, the performance monitor will finish that marking operation in the typical way. The marking state machine for the performance monitor will then go to idle until marking is again enabled.

Instruction marking is disabled while in Single Step or Branch trace mode.

10.12 SIAR and SDAR Registers

The Sampled Instruction Address Register (SIAR) and the Sampled Data Address Register (SDAR) are used, respectively, to save the effective address of a sampled instruction and the effective address of a storage operand for a sampled instruction when the processor is in either trace-marking mode or instruction-sampling mode. The terms “sampled” and “marked” are used interchangeably.

The processor is in instruction-sampling mode whenever MMCRA[63] equals ‘1’ and MSR[SE] and MSR[BE] equal ‘0’. The processor is in a well-defined trace-marking mode whenever MMCRA[63] equals ‘0’ and either MSR[SE] equals ‘1’ or MSR[BE] equals ‘1’, or MSR[SE] and MSR[BE] equal ‘1’.

Note: If instruction sampling is not disabled during trace-marking by setting MMCRA[63] to ‘0’, results are undefined.

The contents of the SIAR and SDAR depend on the marking modes that the processor is in, as explained in the following sections.

10.12.1 Instruction Sampling

When the processor is not in trace-marking mode and instruction-sampling mode is enabled, instruction-sampling mode is active regardless of whether the performance monitor is enabled for any counting activity. In instruction-sampling mode, the performance monitor interacts with the IDU to initiate the instruction-sampling cycle. It then monitors the progress of the sampled instruction as it moves through the instruction pipeline. Each instruction-sampling cycle ends either when the marked instruction completes or when the performance monitor determines that the marked instruction has been flushed from the pipeline.

10.12.1.1 Performance Monitor Exceptions

Performance monitor exceptions occur when a performance monitor counter becomes negative and the counter negative exception is enabled, or when a time-base event occurs and the time-base exception is enabled. When a performance monitor exception occurs, SIAR and SDAR have the following values:

- The SIAR contains the effective address of the last sampled instruction.
- The SDAR is set to the effective address of the storage operand of the last sampled instruction issued to the LSU.
- The effective address of the storage operand contained in the SDAR might be, but need not be, associated with the SIAR instruction as explained previously.
- If single step or branch trace (SE/BE) tracing is inactive, the contents of the SIAR and the SDAR are frozen when a performance monitor exception is raised, at which time the hardware sets MMCR0[PMXE] to ‘0’ (locking the SIAR and SDAR).
- If SE/BE tracing is active, the contents of the SIAR, the SDAR, and SRR1[33] as used by the performance monitor exception facility are undefined and can change even when performance monitor exceptions are disabled (MMCR0[PMXE] equals ‘0’).
- If SE/BE tracing is inactive, the contents of SIAR and SDAR remain frozen until software sets MMCR0[PMXE] to ‘1’. The contents of SIAR and SDAR can be altered by the processor if and only if MMCR0[PMXE] equals ‘1’ provided SE/BE tracing is inactive.
- If the performance monitor exception is enabled and MSR[EE] equals ‘1’, the performance monitor exception condition causes a performance monitor exception to be taken and the value of SRR1[33] indicates whether the contents of the SIAR and SDAR refer to the same instruction.

IBM PowerPC 970MP RISC Microprocessor

- If SRR1[33] equals '1', and SE/BE tracing is inactive, and there was only one sampled instruction in the machine, the SDAR and SIAR contents are associated with the same instruction.
- If SE/BE tracing is inactive, when SRR1[33] equals '0' it indicates either that the SIAR and SDAR contents are not associated with the same instruction or that the SIAR instruction had no storage operand.
- After software sets MMCR0[PMXE] to '1' and if SE/BE tracing is inactive, the contents of SIAR and SDAR are undefined with respect to performance monitor exception processing until the next performance monitor exception occurs.
- After software sets MMCR0[PMXE] to '1' and if SE/BE tracing is inactive, the contents of SIAR and SDAR are again available for use by the performance monitor instruction-sampling facility as described previously.

10.12.2 Single Step and Branch Trace Marking Mode

When the processor is in SE or BE trace mode, instruction-sampling activity on the performance monitor is disabled, but all other performance monitor activities (except sampling) can be active. In particular, performance monitor count events for marked instructions will still be processed if counting is enabled. The marked events counted when trace mode is active will be for trace-marked events, not for sampled events.

In BE mode, the performance monitor count event called "number of instructions completed" is not accurate. It is possible for software to calculate the number of instructions in a basic block by capturing the starting address of the basic block from the SRR0 value of the previous branch and using it together with the ending address of the basic block from the SIAR value.

10.12.2.1 Single Step Trace Mode

If MSR[SE] equals '1', the processor is in single step trace mode. Every instruction is trace-marked. The processor is forced into single instruction mode regardless of the value of the IFU predecode bits. An instruction is trace-marked when it is transferred from the IDU to the ISU and SRR1[33] is reset to zero. If the PowerPC instruction spans multiple groups, the first load/store IOP or the first IOP in the last group is the one marked.

The SIAR is updated at dispatch to contain the address for the trace-marked instruction. The SDAR is updated by the LSU if the PowerPC instruction includes one or more load/store operation. If the SDAR is updated by the LSU, it contains the address of the storage operand for the first load/store IOP. If the SDAR is updated by the LSU, the LSU also causes SRR1[33] to be set to '1' to indicate that the contents of the SIAR and the SDAR are associated with the same trace-marked instruction.

When the trace-marked instruction completes, the processor generates a trace exception. During trace exception processing, the SIAR value is that for the trace-marked instruction that has just successfully completed. During trace exception processing, the SDAR value—if it was updated for this instruction—is that of the first load/store operation of the trace-marked instruction that has just successfully completed. A global flush is performed after the return from trace exception processing.

In the interval between the **rfid** for the trace exception processing for the last completed instruction and the dispatch of the next instruction, the SIAR and SDAR contents represent the last instruction completed and not the instruction that is moving through the IFU to the IDU for dispatch. The next instruction that will be dispatched after the global flush is trace-marked when it is transferred from the IDU to the ISU, SRR1[33] is reset to zero ('0'), and the SIAR value is set for that next marked instruction. The single instruction trace-marking cycle continues as described previously.

10.12.2.2 Branch Trace Mode

If MSR[BE] equals '1', the processor is in branch trace mode. Every branch instruction is trace-marked. A branch instruction is trace-marked when it is transferred from the IDU to the ISU.

The SIAR is updated at dispatch to contain the address for the trace-marked branch instruction. The SDAR contents are undefined. When the trace-marked branch instruction completes, the processor generates a trace exception. During trace exception processing, the SIAR value represents the trace-marked branch instruction that has just completed successfully. A global flush is performed after the return from trace exception processing.

In the interval between the **rfd** for the trace exception processing for the last completed branch instruction and the dispatch of the next branch instruction, the SIAR contents represent the last branch instruction completed. The next branch instruction decoded by the IDU after the global flush is trace-marked when it is transferred from the IDU to the ISU and the SIAR value is set for that next marked branch instruction. The branch trace-marking cycle continues as described previously.

10.12.3 Comparison to Previous PowerPC Processors

According to the PowerPC Architecture, the SIAR contains the effective address of an instruction that was executing *around* the time of a performance monitor exception. The PowerPC 604 and POWER3 processors only updated the Sampled Instruction Registers for sampled instructions (although their sampling method is different). The RS64 processors accomplished this by updating the SIAR and SDAR with the address of the last instruction to complete when a performance monitor exception occurs.

On previous processors that had relatively short pipelines and few instructions in flight, the sampled instruction was at most 20 or so instructions away from the instruction that caused the exception. On the 970MP microprocessor, with the potential for over a hundred instructions in flight, that distance grows. The theoretical maximum is once every 200 - 250 instructions, while the likely distance is 50 - 75 instructions.

Performance profiling tools that use performance monitor events to determine when the SIAR and SDAR are read (for example, read SIAR every 100 L1 D-cache misses) can profile based on any performance monitor event. To assure accuracy, however, only *sampled* events should be profiled. These are a subset of all events that are caused by sampled instructions. The SIAR is set by a sampled instruction, so you can be fairly sure that when an exception caused by a sampled event (a counter overflowing for example), the SIAR is pointing to the *exact* instruction that caused it. In this case, the 970MP microprocessor is more accurate than previous processors. If you profile on non-sampled events, you cannot be sure that the exception was caused by the instruction (group actually) pointed to by the SIAR. The offending instruction was executing *around* the sampled instruction, depending on the event, probably within 50 - 100 instructions.

10.13 Thresholding

Thresholding can be used to obtain counts of the number of marked instructions for which the execution time between a designated start/end pipeline event pair exceeds a specified threshold value. Only one marked instruction is active in the 970MP processing unit pipeline at a time, and only one threshold value can be used for comparison with the selected start/end event pair. The start and end events that can be used for thresholding are shown in *Figure 10-6 Performance Monitor Threshold Logic* on page 268. The values of the respective threshold start/end bit fields in MMCRA[THRSTRT,THREND] are shown in *Table 10-16 Start and End Event Select Bits and the Performance Monitor Threshold Logic* on page 269. The threshold value is specified in the MMCR1[THRESHOLD] field. The threshold value can be further scaled by HID0[13]. If

IBM PowerPC 970MP RISC Microprocessor

HID0[13] equals '0', it causes the thresholder to count every processor cycle. If HID0[13] equals '1', it causes the thresholder to count every 32 processor cycles. For a marked instruction moving through the 970MP processing unit pipeline, the events that can be used for threshold start/end measurement occur in the following order: marked in IDU, dispatch, issue, finish, complete.

Once a pair of start/end threshold events is selected and the start event occurs, the threshold facility begins decrementing from the threshold value and continues to decrement until either the decrementer times out or the end event occurs. If the decrementer times out and if the threshold logic event is selected for counting, the threshold logic event counter is incremented. Both the thresholder time out and the occurrence of the end event cause the threshold decrementer to be reset to the threshold value. The thresholder begins decrementing when the next start event occurs.

Threshold start/end pairs must be selected in a manner that represents a reasonable scenario. For example, a start event that is the same as the end event will not provide useful threshold event count information regardless of the threshold value selected. A start event that occurs later in the pipeline than the end event will not give a useful measure of the transit time of a marked instruction through the pipeline. The results of unreasonable threshold start/end event selections might produce undefined results.

Figure 10-6. Performance Monitor Threshold Logic

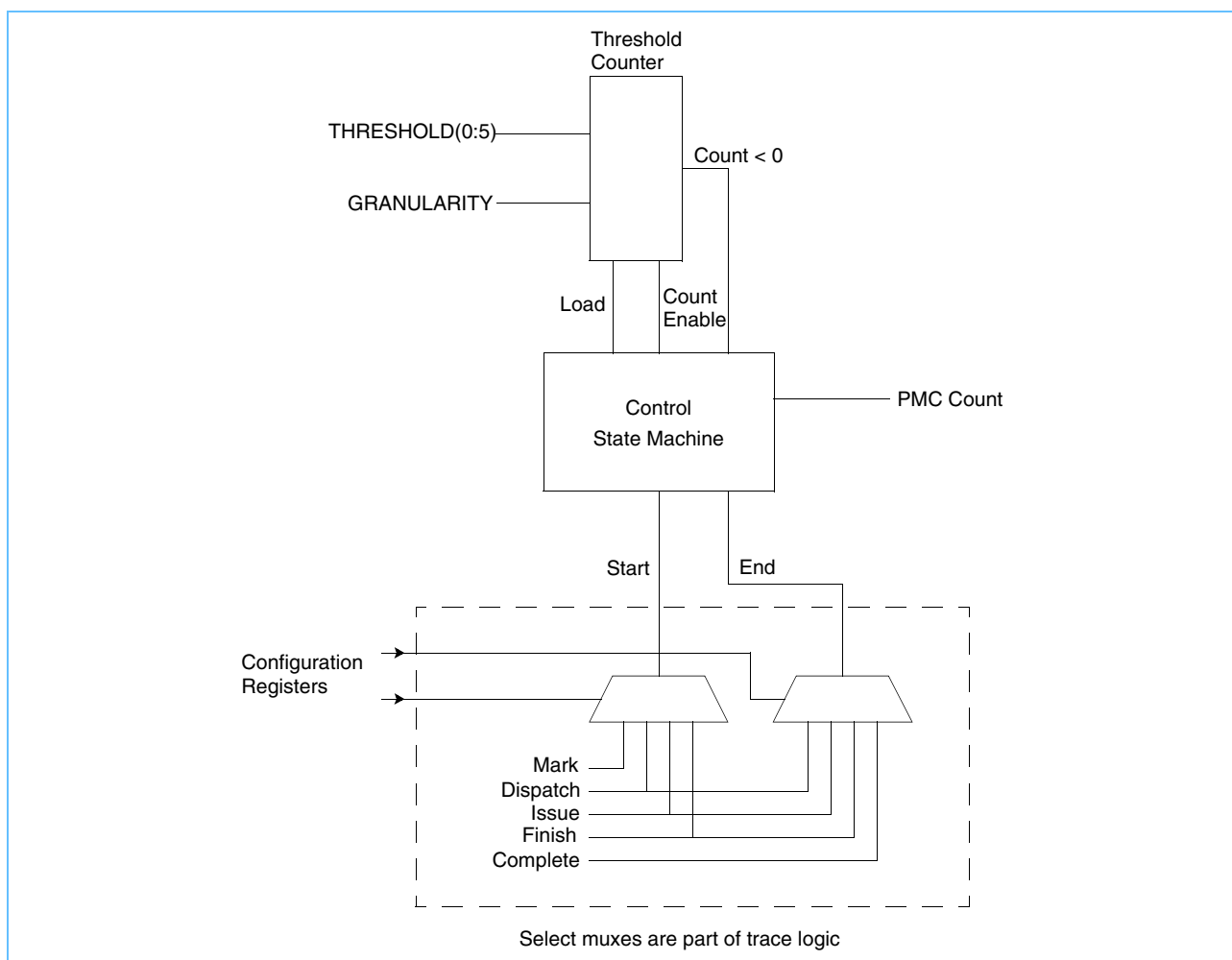


Table 10-16. Start and End Event Select Bits and the Performance Monitor Threshold Logic

MMCR1 [THRSTRT] Value	Threshold Start Event Selected	MMCR1 [THREND] Value	Threshold End Event Selected
000	No start event	000	No end event
001	Group marked in IDU	001	No end event
010	Marked group dispatched	010	Marked group dispatched
011	Marked group issued	011	Marked group issued
100	Marked group finish ¹	100	Marked group finish
101	Marked group complete ¹	101	Marked group complete
110	No start event	110	No end event
111	No start event	111	No end event

1. An instruction that has finished but not completed has gone all the way through the pipeline, and the renamed registers have been updated with new values. However, it is still sitting in the completion queue. When an instruction completes, the architected registers are updated with the values from the renamed registers.

IBM PowerPC 970MP RISC Microprocessor

10.14 Detailed Event Information

Table 10-17. Detailed Event Descriptions (Page 1 of 8)

Event Description	Detailed Description
CLB has x where x is 0 to 8	The cache line buffer (CLB) is a 4-instruction wide by 8-instruction deep buffer between the fetch unit and the dispatch unit. This signal indicates how many entries, each of which is 4-instructions wide, are occupied at any given time.
Valid instructions available, but IFU held by BIQ or IDU	This signal is asserted each time either the IDU is full or the branch instruction queue (BIQ) is full.
Branch execution issue valid	This signal is asserted each time the ISU issues a branch instruction.
Branch miss predict because of CR value	This signal is asserted when the branch execution unit detects a branch mispredict because the CR value is the opposite of the predicted value. This signal is asserted after a branch issue event and results in a branch redirect flush if not overridden by a flush of an older instruction.
Branch miss predict because of target address prediction	This signal is asserted each time the branch execution unit detects an incorrect target address prediction. This signal is asserted after a valid branch execution unit issue and causes a branch mispredict flush unless a flush is detected from an older instruction.
CR mapper full	The ISU sends a signal indicating that the CR mapper cannot accept any more groups. Dispatch is stopped. Note: This condition indicates that a pool of the mapper is full but the entire mapper might not be.
Tablewalk duration	This signal is asserted every cycle when a tablewalk is active. While a tablewalk is active, any request attempting to access the TLB is rejected and retried.
L1 D-cache entries invalidated from L2	A D-cache invalidated was received from the L2 because a line in L2 was castout.
Out of streams	A new prefetch stream was detected, but no more stream entries were available.
D-SLB miss	An SLB miss for a data request occurred. When there is a miss in the SLB, the operating system must reload the buffer with the information needed for a hit so that the transaction can proceed. Therefore, an SLB miss causes an interrupt (trap) to indicate to the operating system that it needs to resolve the problem.
D-TLB miss	A TLB miss for a data request occurred. Requests that miss the TLB can be retried until the instruction is in the next-to-complete group (unless HID4 is set to allow speculative tablewalks). This can result in multiple TLB misses for the same instruction.
Duration MSR[EE] equals '0'	The ISU sends the MSR[EE] bit to the PMU. It is up to the performance monitor to count the cycles while this bit is '0'.
MSR[EE] equals '0' and interrupt pending	The ISU sends the MSR[EE] bit and a signal indicating that an interrupt is pending to the PMU. It is up to the performance monitor to count the cycles while MSR[EE] equals '0' and the interrupt is pending.
FPR mapper full	The ISU sends a signal indicating that the FPR mapper cannot accept any more groups. Dispatch is stopped. Note: This condition indicates that a pool of mappers is full but the entire mapper might not be.
FPU0 add, mult, sub, compare, fsel	This signal is active for one cycle when FPU0 is executing an add, mult, sub, compare, or fsel kind of instruction. The instruction could be fadd* , fmul* , fsub* , fcmp* , or fsel .
FPU0 denormalized operand	This signal is active for one cycle when one of the operands is denormalized.
FPU0 divide	This signal is active for one cycle at the end of the microcode executed when FPU0 is executing a divide instruction. The instruction could be fddiv , fdivs , fdiv. , or fdivs .
FPU0 estimate	This signal is active for one cycle when FPU0 is executing one of the estimate instructions. The instruction could be fres* or frsqrt* where xyz* means xyz or xyz .
FPU0 finished and produced a result	This signal only indicates finish, not completion.
FPU0 mult-add	This signal is active for one cycle when FPU0 is executing a multiply-add kind of instruction. The instruction could be fmadd* , fnmadd* , fmsub* , or fnmsub* where xyz* means xyz , xyzs , xyz. , xyzs .

Table 10-17. Detailed Event Descriptions (Page 2 of 8)

Event Description	Detailed Description
FPU0 move, estimate	This signal is active for one cycle when FPU0 is executing a move kind of instruction or one of the estimate instructions. The instruction could be fmr* , fneg* , fabs* , fnabs* , fres* , or frsqrite* where xyz* means xyz or xyz .
FPU0 FPSCR	This signal is active for one cycle when FPU0 is executing an FPSCR move-related instruction. The instruction could be mtfsfi* , mtfsb0* , mtfsb1* , mffs* , mtfsf* , or mcrsf* where xyz* means xyz , xyzs , xyz. , or xyzs..
FPU0 round, convert	This signal is active for one cycle when FPU0 is executing frsp or a convert kind of instruction. The instruction could be frsp* , fcfid* , or fcti* where xyz* means xyz , xyzs , xyz. , or xyzs..
FPU0 square root	This signal is active for one cycle at the end of the microcode executed when FPU0 is executing a square root instruction. The instruction could be fsqrt* where xyz* means xyz , xyzs , xyz. , xyzs..
FPU0 issue queue full	The issue queue for FPU 0 cannot accept any more instructions. Issue is stopped.
FPU0 single precision	This signal is active for one cycle when FPU0 is executing a single-precision instruction.
FPU0 stall 3	This signal indicates that FPU0 has generated a stall in pipe 3 because of overflow, underflow, massive cancel, convert to integer (sometimes), or convert from integer (always). This signal is active during the entire duration of the stall.
FPU0 store	This signal is active for one cycle when FPU0 is executing a store instruction.
FPU1 add, mult, sub, compare, fsel	This signal is active for one cycle when FPU1 is executing an add, mult, sub, compare, or fsel kind of instruction. The instruction could be fadd* , fmul* , fsub* , fcmp** , or fsel where xyz* means xyz , xyzs , xyz. , xyzs. and xyz** means xyzu and xyzo .
FPU1 denormalized operand	This signal is active for one cycle when one of the operands is denormalized.
FPU1 divide	This signal is active for one cycle at the end of the microcode executed when FPU1 is executing a divide instruction. The instruction could be fddiv , fdivs , fddiv. , or fdivs.
FPU1 estimate	This signal is active for one cycle when FPU1 is executing one of the estimate instructions. The instruction could be fres* or frsqrite* where xyz* means xyz or xyz .
FPU1 finished and produced a result	This signal only indicates finish, not completion.
FPU1 mult-add	This signal is active for one cycle when FPU1 is executing a multiply-add kind of instruction. The instruction could be fmadd* , fnmadd* , fmsub* , or fnmsub* where xyz* means xyz , xyzs , xyz. , and xyzs..
FPU1 move, estimate	This signal is active for one cycle when FPU1 is executing a move kind of instruction or one of the estimate instructions. The instruction could be fmr* , fneg* , fabs* , fnabs* , fres* , or frsqrite* where xyz* means xyz or xyz .
FPU1 round, convert	This signal is active for one cycle when FPU1 is executing frsp or convert kind of instruction. The instruction could be frsp* , fcfid* , or fcti* where xyz* means xyz , xyzs , xyz. , xyzs..
FPU1 square root	This signal is active for one cycle at the end of the microcode executed when FPU1 is executing a square root instruction. The instruction could be fsqrt* where xyz* means xyz , xyzs , xyz. , xyzs..
FPU1 issue queue full	The issue queue for FPU 1 cannot accept any more instructions. Issue is stopped.
FPU1 single precision	This signal is active for one cycle when FPU1 is executing a single-precision instruction.
FPU1 stall 3	This signal indicates that FPU1 has generated a stall in pipe 3 because of overflow, underflow, massive cancel, convert to integer (sometimes), or convert from integer (always). This signal is active during the entire duration of the stall.
FPU1 store	This signal is active for one cycle when FPU1 is executing a store instruction.
FXU0/LSU0 issue queue full	The issue queue for FXU/LSU unit 0 cannot accept any more instructions. Issue is stopped.
FXU1/LSU1 issue queue full	The issue queue for FXU/LSU 1 cannot accept any more instructions. Issue is stopped.
FXU0 produced a result	The FXU0 finished an instruction and produced a result.

IBM PowerPC 970MP RISC Microprocessor

Table 10-17. Detailed Event Descriptions (Page 3 of 8)

Event Description	Detailed Description
FXU1 produced a result	The FXU1 finished an instruction and produced a result.
GPR mapper full	The ISU sends a signal indicating that the GPR mapper cannot accept any more groups. Dispatch is stopped. Note: This condition indicates that a pool of mapper is full but the entire mapper might not be.
Dispatch blocked by scoreboard	The ISU sends a signal indicating that dispatch is blocked by the scoreboard.
Dispatch reject	Dispatch successful equals <i>dispatch_valid</i> and one cycle later <i>~dispatch_reject</i> .
Dispatch valid	The ISU sends <i>dispatch_valid</i> and <i>dispatch_reject</i> signals to the PMU. It is up to the performance monitor to look at these signals to count the number of dispatch groups.
Instruction prefetch installed in prefetch buffer	This signal is asserted when a prefetch buffer entry (line) is allocated but the request is not a demand fetch.
Instruction prefetch request	Asserted when a non-canceled prefetch is made to the CIU.
Translation written to I-ERAT	This signal is asserted each time the I-ERAT is written. This indicates that an ERAT miss has been serviced. ERAT misses will initiate a sequence resulting in the ERAT being written. ERAT misses that are later ignored will not be counted unless the ERAT is written before the instruction stream is changed. This should be a fairly accurate count of ERAT missed (best available).
Instructions dispatched count	The ISU sends the number of instructions dispatched.
Valid instruction available	Asserted each cycle when the IFU sends at least one instruction to the IDU.
I-SLB miss	An SLB miss for an instruction fetch has occurred.
I-TLB miss	A TLB miss for an Instruction fetch has occurred.
L1 reload data source valid	The data source information is valid.
L1 prefetches	A request to prefetch data into the L1 was made.
L2 Prefetch	A request to prefetch data into the L2 was made.
larx executed side 0	An larx (lwarx or ldarx) was executed on side 0 (there is no corresponding unit 1 event because larx instructions can only execute on unit 0).
L1 D-cache load miss side 0	A load, executing on unit 0, missed the D-cache.
L1 D-cache store side 1	A store executed on unit 1.
L1 D-cache load miss side 1	A load, executing on unit 1, missed the D-cache.
L1 D-cache load side 0	A load executed on unit 0.
LR/CTR mapper full	The ISU sends a signal indicating that the LR/CTR mapper cannot accept any more groups. Dispatch is stopped. Note: This condition indicates that a pool of the mapper is full but the entire mapper might not be.
LMQ full	The LMQ was full.
LMQ LHR merge	A D-cache miss occurred for the same real cache line address as an earlier request already in the load miss queue and was merged into the LMQ entry.
LMQ slot 0 allocated	The first entry in the LMQ was allocated.
LMQ slot 0 valid	This signal is asserted every cycle when the first entry in the LMQ is valid. The LMQ has eight entries that are allocated on a FIFO basis.
LRQ full	The ISU sends this signal when the LRQ is full.
LRQ slot 0 allocated	LRQ slot zero was allocated.
LRQ slot 0 valid	This signal is asserted every cycle that slot zero of the store request queue is valid. The SRQ is 32 entries long and is allocated on a round-robin basis.

Table 10-17. Detailed Event Descriptions (Page 4 of 8)

Event Description	Detailed Description
SRQ full	The ISU sends this signal when the SRQ is full.
SRQ slot 0 allocated	SRQ slot zero was allocated.
SRQ slot 0 valid	This signal is asserted every cycle that slot zero of the store request queue is valid. The SRQ is 32 entries long and is allocated round-robin.
SRQ sync duration	This signal is asserted every cycle when a sync is in the SRQ.
LSU busy side 0	LSU 0 is busy rejecting instructions.
D-ERAT miss side 0	A data request (load or store) from LSU 0 missed the ERAT. Requests that miss the D-ERAT are rejected and retried until the request hits in the ERAT. This can result in multiple ERAT misses for the same instruction.
Flush from LRQ SHL, LHL side 0	A load was flushed by unit 1 because a a younger load executed before an older store executed and they had overlapping data. Alternatively, two loads executed out-of-order, they had byte overlap, and there was a snoop in between to an overlapped byte.
Flush SRQ LHS side 0	A store was flushed because a younger load hits an older store that is already in the SRQ or in the same group.
Flush unaligned load side 0	A load was flushed from unit 1 because it was unaligned (crossed a 64-byte boundary, or a 32-byte boundary if it missed the L1).
Flush unaligned store side 0	A store was flushed from unit 1 because it was unaligned.
Floating point load side 0	A floating-point load was executed from LSU unit 0.
SRQ store forwarding side 0	Data from a store instruction was forwarded to a load on unit 0.
LSU busy side 1	LSU 0 is busy rejecting instructions.
D-ERAT miss side 1	A data request (load or store) from LSU 1 missed the ERAT. Requests that miss the D-ERAT are rejected and retried until the request hits in the ERAT. This can result in multiple ERAT misses for the same instruction.
Flush from LRQ SHL, LHL side 1	A load was flushed by unit 1 because a a younger load executed before an older store executed and they had overlapping data. Alternatively, two loads executed out-of-order, they had byte overlap, and there was a snoop in between to an overlapped byte.
Flush SRQ LHS side 1	A store was flushed because younger load hits an older store that is already in the SRQ or in the same group.
Flush unaligned load side 1	A load was flushed from unit 1 because it was unaligned (crossed a 64-byte boundary, or a 32-byte boundary if it missed the L1).
Flush unaligned store side 1	A store was flushed from unit 1 because it was unaligned (crossed a 4KB boundary).
Floating point load side 1	A floating-point load was executed from LSU 1.
Marked IMR reload	A Data L1 Cache reload occurred because of a marked load.
Marked L1 reload data source valid	The source information is valid and is for a marked load.
Marked L1 D-cache load miss side 0	A marked load, executing on unit 0, missed the D-cache.
Marked L1 D-cache load miss side 1	A marked load, executing on unit 1, missed the D-cache.
Marked SRQ valid	This signal is asserted every cycle when a marked request is resident in the store request queue.
Marked flush from LRQ SHL, LHL side 0	A marked load was flushed by unit 0 because a a younger load executed before an older store executed and they had overlapping data. Alternatively, two loads executed out-of-order, they have byte overlap, and there was a snoop in between to an overlapped byte.
Marked flush SRQ LHS side 0	A marked store was flushed because a younger load hits an older store that is already in the SRQ or in the same group.
Marked flush unaligned store side 0	A marked store was flushed from unit 0 because it was unaligned.

IBM PowerPC 970MP RISC Microprocessor

Table 10-17. Detailed Event Descriptions (Page 5 of 8)

Event Description	Detailed Description
Marked flush unaligned load side 0	A marked load was flushed from unit 0 because it was unaligned (crossed a 64-byte boundary, or a 32-byte boundary if it missed the L1).
LSU side 0 finished IMR	LSU unit 0 finished a marked instruction.
Marked flush from LRQ SHL, LHL side 1	A marked load was flushed by unit 1 because a a younger load executed before an older store executed and they had overlapping data. Alternatively, two loads executed out-of-order, they had byte overlap, and there was a snoop in between to an overlapped byte.
Marked flush SRQ LHS side 1	A marked load was flushed by unit 1 because a a younger load executed before an older store executed and they had overlapping data. Alternatively, two loads executed out-of-order, they had byte overlap, and there was a snoop in between to an overlapped byte.
Marked flush unaligned load side 1	A marked load was flushed from unit 1 because it was unaligned (crossed a 64-byte boundary, or a 32-byte boundary if it missed the L1).
Marked flush unaligned store side 1	A marked store was flushed from unit 1 because it was unaligned (crossed a 4KB page boundary).
LSU side 1 finished IMR	LSU unit 1 finished a marked instruction.
Marked L1 D-cache store miss	A marked store missed the D-cache.
Marked stcx fail	A marked stcx (stwcx or stdcx) failed.
Snoop tlbie	A tlbie was snooped from another processor.
L1 D-cache store miss	A store missed the D-cache.
L1 D-cache store miss	A store missed the D-cache.
L1 D-cache store side 0	A store executed on Unit 0.
L1 D-cache load side 1	A load executed on Unit 1.
stcx failed	An stcx (stwcx or stdcx) failed.
stcx passed	An stcx (stwcx or stdcx) instruction was successful.
XER mapper full	The ISU sends a signal indicating that the XER mapper cannot accept any more groups. Dispatch is stopped. Note: This condition indicates that a pool of the mapper is full but the entire mapper might not be.
No instructions fetched	No instructions were fetched this cycle (because of IFU hold, redirect, or I-cache miss).
One or more PowerPC instruction completed	A group containing at least one PowerPC instruction completed. For microcoded instructions that span multiple groups, this will only occur once.
BR issue queue full	The ISU sends a signal indicating that the issue queue that feeds the IFU BR unit cannot accept any more groups (the queue is full of groups).
CR issue queue full	The ISU sends a signal indicating that the issue queue that feeds the IFU CR unit cannot accept any more groups (the queue is full of groups).
Processor cycles	Processor cycles.
Data loaded from L2	DL1 was reloaded from the local L2 because of a demand load.
Data loaded from memory	DL1 was reloaded from memory because of a demand load.
New stream allocated	A new prefetch stream was allocated.
External interrupts	An external interrupt occurred.
FPU executed add	This signal is active for one cycle when FPU0 is executing an add, mult, sub, compare, or fsel kind of instruction. The instruction could be fadd* , fmul* , fsub* , fcmp** , or fsel where xyz* means xyz , xyzs , xyz. , xyzs . and xyz** means xyzu and xyzo . Combined Unit 0 + Unit 1.

Table 10-17. Detailed Event Descriptions (Page 6 of 8)

Event Description	Detailed Description
FPU received denormalized data	This signal is active for one cycle when one of the operands is denormalized. Combined Unit 0 + Unit 1.
FPU executed FDIV instruction	This signal is active for one cycle at the end of the microcode executed when FPU0 is executing a divide instruction. The instruction could be fdiv , fdivs , fdiv , fdivs . Combined Unit 0 + Unit 1.
FPU executed FEST instruction	This signal is active for one cycle when executing one of the estimate instructions. The instruction could be fres* or frsqrt* where xyz* means xyz or xyz . Combined Unit 0 + Unit 1.
FPU produced a result	FPU finished and produced a result. This only indicates finish, not completion. Combined Unit 0 + Unit 1.
FPU executed multiply-add instruction	This signal is active for one cycle when FPU0 is executing a multiply-add kind of instruction. The instruction could be fmadd* , fnmadd* , fmsub* , or fnmsub* where xyz* means xyz , xyzs , xyz , xyzs . Combined Unit 0 + Unit 1.
FPU executing FMOV or FEST instructions	This signal is active for one cycle when executing a move kind of instruction or one of the estimate instructions. The instruction could be fmr* , fneg* , fabs* , fnabs* , fres* , or frsqrt* where xyz* means xyz or xyz . Combined Unit 0 + Unit 1.
FPU executed FRSP or FCONV instructions	This signal is active for one cycle when executing frsp or a convert kind of instruction. The instruction could be frsp* , fcfid* , fcti* where xyz* means xyz , xyzs , xyz , xyzs . Combined Unit 0 + Unit 1.
FPU executed FSQRT instruction	This signal is active for one cycle at the end of the microcode executed when FPU0 is executing a square root instruction. The instruction could be fsqrt* where xyz* means xyz , xyzs , xyz , xyzs . Combined Unit 0 + Unit 1.
Cycles FPU issue queue full	Cycles when one or both FPU issue queues are full.
FPU executed single-precision instruction	FPU is executing a single-precision instruction. Combined Unit 0 + Unit 1.
FPU stalled in pipe 3	FPU has generated a stall in pipe 3 because of overflow, underflow, massive cancel, convert to integer (sometimes), or convert from integer (always). This signal is active during the entire duration of the stall. Combined Unit 0 + Unit 1.
FPU executed store instruction	FPU is executing a store instruction. Combined Unit 0 + Unit 1.
Cycles FXLS queue is full	Cycles when one or both FXU/LSU issue queues are full.
FXU busy	FXU0 and FXU1 are both busy.
FXU produced a result	The fixed-point unit (Unit 0 + Unit 1) finished a marked instruction. Instructions that finish might not necessarily complete.
FXU idle	FXU0 and FXU1 are both busy.
FXU0 busy FXU1 idle	FXU0 is busy while FXU1 is idle.
FXU1 busy FXU0 idle	FXU0 is idle while FXU1 is busy.
Cycles GCT empty	The global completion table is completely empty.
Completion table full	The ISU sends a signal indicating that the GCT is full.
Group completed	A group completed. Microcoded instructions that span multiple groups will generate this event once per group.
Group dispatches	A group was dispatched.
Group dispatch rejected	A group that previously attempted dispatch was rejected.
Group dispatch success	Number of groups successfully dispatched (not rejected).
Group marked in IDU	A group was sampled (marked).
Instructions completed	Number of eligible instructions that completed.

IBM PowerPC 970MP RISC Microprocessor

Table 10-17. Detailed Event Descriptions (Page 7 of 8)

Event Description	Detailed Description
Instructions fetched from L1	An instruction fetch group was fetched from L1. Fetch groups can contain up to eight instructions.
Instructions fetched from L2	An instruction fetch group was fetched from L2. Fetch groups can contain up to eight instructions.
Instructions fetched from memory	An instruction fetch group was fetched from memory. Fetch groups can contain up to eight instructions.
Instructions fetched from prefetch	An instruction fetch group was fetched from the prefetch buffer. Fetch groups can contain up to eight instructions.
Cycles is L1 write active	This signal is asserted each cycle a cache write is active.
larx executed	An larx (lwarx or ldarx) was executed. This is the combined count from LSU0 + LSU1, but these instructions only execute on LSU0.
L1 D-cache load misses	Total DL1 load references that miss the DL1.
L1 D-cache load references	Total DL1 load references.
LSU busy	LSU (Unit 0 + Unit 1) is busy rejecting instructions.
D-ERAT misses	Total D-ERAT misses (Unit 0 + Unit 1). Requests that miss the D-ERAT are rejected and retried until the request hits in the ERAT. This can result in multiple ERAT misses for the same instruction.
LRQ flushes	A load was flushed because a younger load executed before an older store executed and they had overlapping data. Alternatively, two loads executed out-of-order, they had byte overlap, and there was a snoop in between to an overlapped byte.
SRQ flushes	A store was flushed because a younger load hits an older store that is already in the SRQ or in the same group.
LRQ unaligned load flushes	A load was flushed because it was unaligned (crossed a 64-byte boundary, or a 32-byte boundary if it missed the L1).
SRQ unaligned store flushes	A store was flushed because it was unaligned.
LSU executed floating-point load instruction	
Cycles LMQ and SRQ empty	Cycles when both the LMQ and SRQ are empty (LSU is idle).
Cycles SRQ empty	The store request queue is empty.
SRQ store forwarding side 1	Data from a store instruction was forwarded to a load on Unit 1.
Marked instruction BRU processing finished	The branch unit finished a marked instruction. Instructions that finish might not necessarily complete.
Marked instruction CRU processing finished	The condition register unit finished a marked instruction. Instructions that finish might not necessarily complete.
Marked data loaded from L2	DL1 was reloaded with modified (M) data from the L2 of a chip on this MCM because of a marked load.
Marked data loaded from memory	DL1 was reloaded with modified (M) data from the L2 of another MCM because of a marked load.
Marked instruction FPU processing finished	One of the floating-point units finished a marked instruction. Instructions that finish might not necessarily complete.
Marked instruction FXU processing finished	One of the fixed-point units finished a marked instruction. Instructions that finish might not necessarily complete.
Marked group completed	A group containing a sampled instruction completed. Microcoded instructions that span multiple groups will generate this event once per group.
Marked group dispatched	A group containing a sampled instruction was dispatched.

Table 10-17. Detailed Event Descriptions (Page 8 of 8)

Event Description	Detailed Description
Marked group issued	A sampled instruction was issued.
Marked group completion timeout	The sampling timeout expired indicating that the previously sampled instruction is no longer in the processor.
Marked instruction finished	One of the execution units finished a marked instruction. Instructions that finish might not necessarily complete.
Marked L1 D-cache load misses	
Marked instruction LSU processing finished	One of the load/store units finished a marked instruction. Instructions that finish might not necessarily complete.
Marked LRQ flushes	A marked load was flushed because a younger load executed before an older store executed and they had overlapping data. Alternatively, two loads executed out-of-order, they have byte overlap, and there was a snoop in between to an overlapped byte.
Marked SRQ flushes	A marked store was flushed because a younger load hits an older store that is already in the SRQ or in the same group.
Marked unaligned load flushes	A marked load was flushed because it was unaligned (crossed a 64-byte boundary, or a 32-byte boundary if it missed the L1).
Marked unaligned store flushes	A marked store was flushed because it was unaligned.
Marked store instruction completed	A sampled store has completed (data home).
Marked store completed with intervention	A marked store previously sent to the memory subsystem completed (data home) after requiring intervention.
Marked store sent to storage subsystem	A sampled store has been sent to the memory subsystem.
Run cycles	Processor cycles gated by the run latch.
L1 D-cache store references	Total DL1 store references.
Completion stopped	The RAS unit has signaled completion to stop.
Time-base bit transition	Occurs when the selected time-base bit (as specified in MMCR0[TBSEL]) transitions from '0' to '1'.
Threshold timeout	The threshold timer expired.
Work held	The RAS unit has signaled completion to stop and there are groups waiting to complete.



11. System Design

11.1 I²C Interface

I²C (Interconnect for Integrated Circuits) is a standard bus developed by Philips Electronics.¹ The I²C Slave in the 970MP converts data sent across an I²C bus into native JTAG commands. The I²C slave can be used as a test access port (TAP) controller that interfaces with the Access macro or with other IEEE 1149.1 compatible devices in order to read, write, and scan registers within a chip.

11.2 Bus Initialization, Configuration, Power Management, and Test

11.2.1 Bus Initialization

The bus devices use a physical layer initialization sequence to initialize the bus. A specific pattern is sent across the bus, which initializes the processor interface in slave devices. This sequence is described in *Section 11.3.1 Initialization at Power-On Reset* on page 289.

11.2.2 Configurable Parameters

The processor interconnect defines multiple configurable parameters for efficient operation of the bus. The values that can be programmed into these parameter registers are technology and implementation-dependent. During the initialization process at system start-up, the power-management unit identifies the system configuration and programs the individual devices attached to the bus (that is, the North Bridge and the processors) with the appropriate values using the I²C device interfaces. All values are in bus beats. For parameters that cross the processor interface, the values are from the final locally clocked flip-flop or latch output to the first locally clocked flip-flop or latch input, after deskewing has taken place through the processor interconnect.

Figure 11-1 shows the configurable timing parameters, COMPACE and STATLAT. COMPACE is the minimum number of bus beats between command packets issued from the processor. STATLAT is the number of bus beats between the last beat of the address/data (AD) packet and the first beat of the transfer-handshake (TH) packet.

1. I²C standard (IIC) for a serial bus. For more information see: <http://www-us2.semiconductors.philips.com/i2c/>.

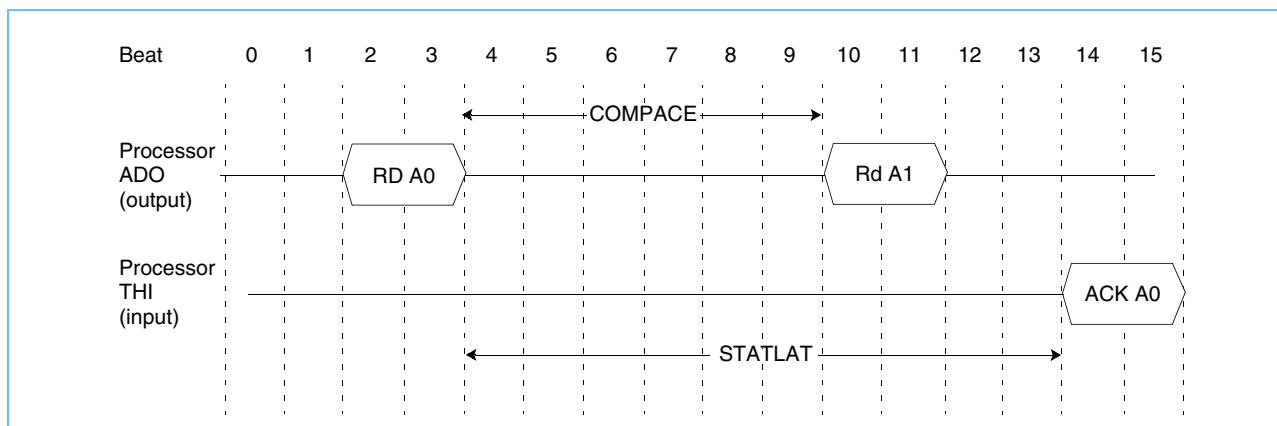
IBM PowerPC 970MP RISC Microprocessor
Figure 11-1. Configurable Timing Parameters


Figure 11-2 shows the North Bridge configurable timing parameters, SNOOPWIN, SNOOPLAT, and PAAMWIN. SNOOPWIN is the minimum number of idle bus beats between reflected command packets. PAAMWIN is the minimum number of bus beats between command packets reflected from the North Bridge to the processors when there is an address collision (shown as A0 in Figure 11-2). SNOOPLAT is the number of bus beats between the last beat of a reflected command packet to the first beat of the individual snoop responses from each of the processors received at the North Bridge. SNOOPLAT includes the time of flight across the interface and any switch devices interposed between the North Bridge and a processor.

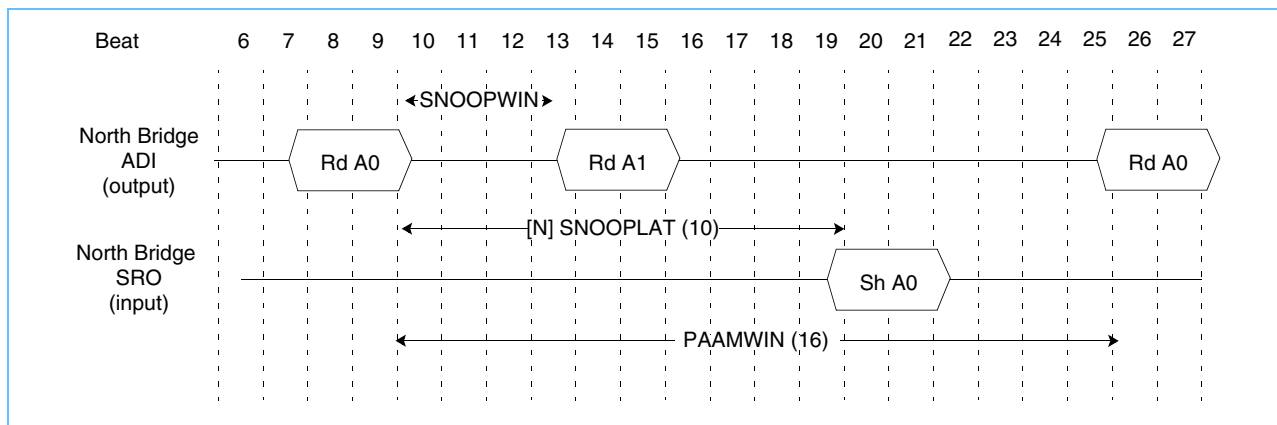
Figure 11-2. North Bridge Configurable Timing Parameters


Figure 11-3 shows the processor configurable timing parameters, SNOOPLAT and SNOOPACC. SNOOPLAT is defined above. SNOOPACC is the number of bus beats between the last beat of the individual snoop response sent from a processor to the first beat of the accumulated snoop response received from the North Bridge. SNOOPACC includes the time of flight across the interface and any switch devices interposed between a processor and the North Bridge.

Figure 11-3. Processor Configurable Timing Parameters

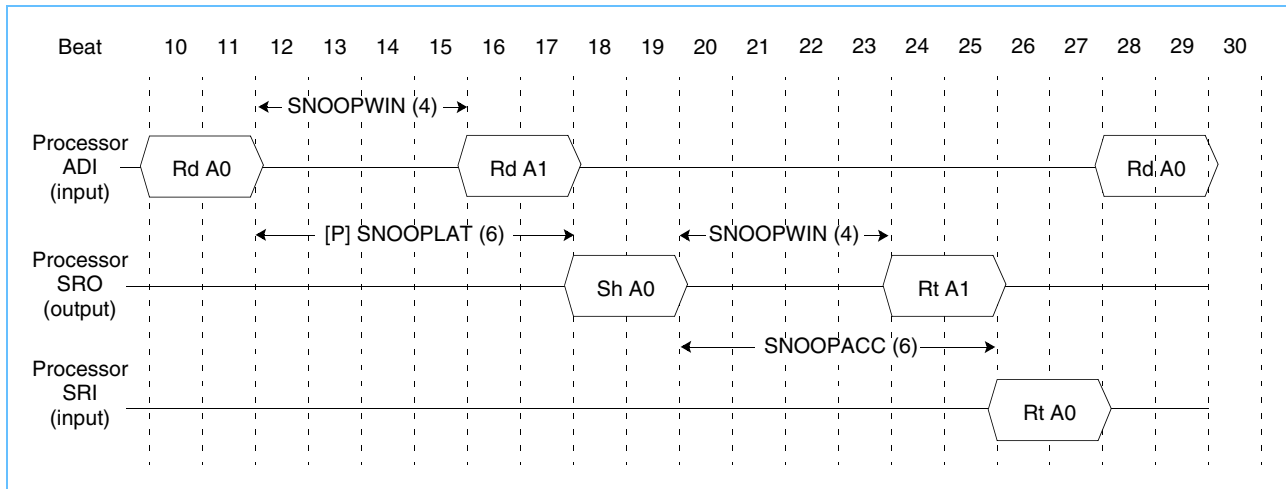


Table 11-1. Programmable Delay Parameters

Parameter	Processor	North Bridge	Range in Bus Beats			Description
			Minimum	Typical	Maximum	
COMPACT	Y	N	2	—	14	Command pipeline delay. See <i>Section 8.2.1.4 Command Pacing</i> on page 151.
STATLAT	Y	N	4	—	30	Transfer-handshake response latency. See <i>Section 8.2.3 Transfer-Handshake Packets</i> on page 155 and <i>Section 8.4.2 Memory Read Transactions (General)</i> on page 164.
STATLAT	N	Y	—	22	—	
SNOOPWIN	N	Y	—	4	—	Snoop window pacing. See <i>Section 8.3.1 Snoop-Response Bus Implementation</i> on page 159 and <i>Section 8.4.2 Memory Read Transactions (General)</i> on page 164.
SNOOPLAT	N	Y	—	25	—	North Bridge snoop latency. See <i>Section 8.3.1 Snoop-Response Bus Implementation</i> on page 159.
SNOOPLAT	Y	N	6	6	12	Processor snoop latency. See <i>Section 8.3.1 Snoop-Response Bus Implementation</i> on page 159.
SNOOPACC	Y	N	9	—	24	North Bridge snoop accumulation delay. See <i>Section 8.3.1 Snoop-Response Bus Implementation</i> on page 159 and <i>Section 8.4.2.3 Read with Intent to Modify Burst Transaction</i> on page 166.

IBM PowerPC 970MP RISC Microprocessor
11.2.3 Configuration Interface

An I²C interface is used by the power-management unit to configure the processor interconnect bus parameters. The interface complies with the *I²C Bus Specification*. *Table 11-2* lists the I²C interface signals.

Table 11-3 lists the I²C registers, which are described in this section.

The I²C interface consists of two bidirectional signals, $\overline{\text{I2CCK}}$ and $\overline{\text{I2CDT}}$. Both signals use open-drain drivers that require external pull-up resistors. Multiple devices can be connected to the same signals. The PROCID[0:1] inputs are used to address a specific processor.

Table 11-2. I²C Interface Signals

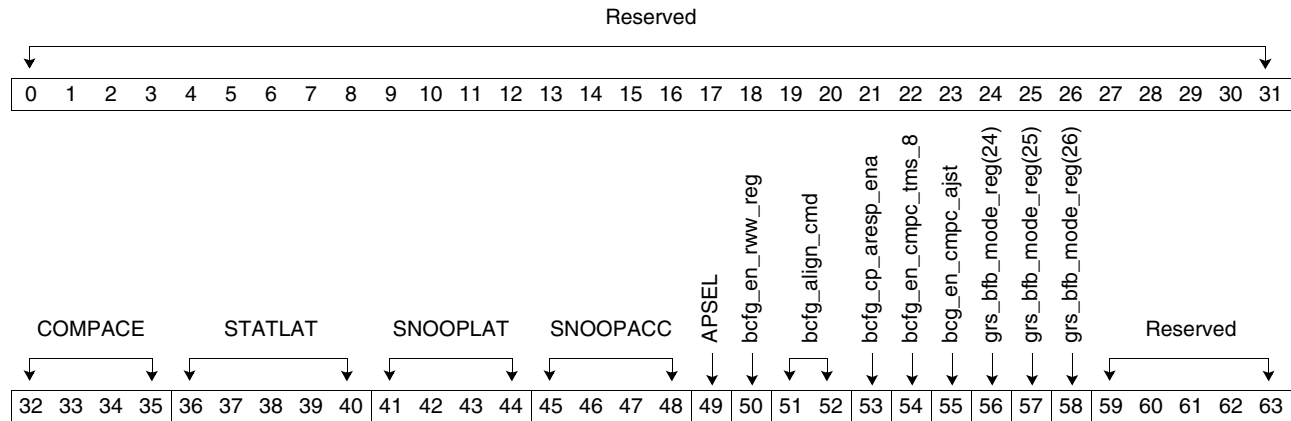
Signal	Polarity	Name
$\overline{\text{I2CCK}}$	Active Low	I ² C interface clock input
$\overline{\text{I2CDT}}$	Active Low	I ² C interface data input/output
PROCID[0:1]	Active High	Processor identification input

Table 11-3. I²C Registers Used by the 970MP Processor Interconnect

Name	Mode	Address	Description	See Page
PI Status Register	Read	x'084001'	Processor Interconnect Status Register	351
PI Mode Register 0	Read/Write	x'083000'	Processor Interconnect Mode Registers 0	347
PI Mode Register 1	Read/Write	x'083100'	Processor Interconnect Mode Registers 1	348
PI Mode Register 2	Read/Write	x'083200'	Processor Interconnect Mode Registers 2	349
PI Mode Register 3	Read/Write	x'083300'	Processor Interconnect Mode Registers 3	350
BUSCONF	Read/Write	x'0A8000'	Processor Configurable Timing Delay Parameter Register	283

11.2.3.1 Processor Configurable Timing Delay Parameter Register (BUSCONF)

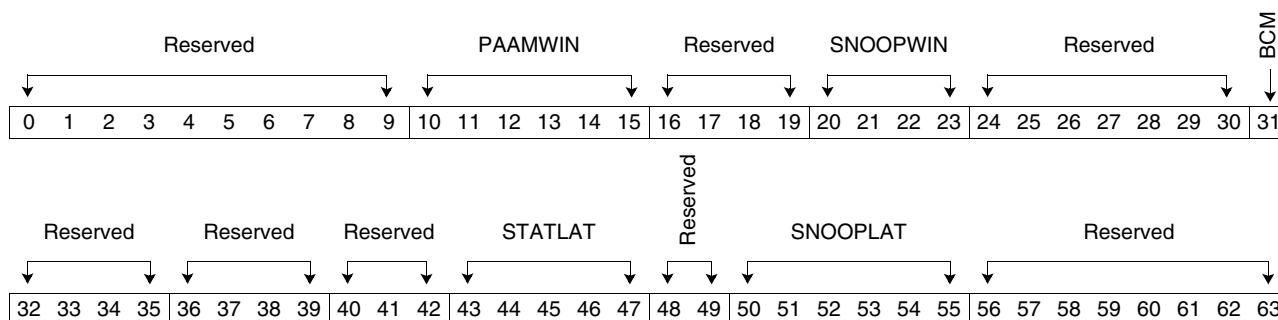
SCOM Address x'0A8000'



Bits	Field Name	Description
0:31	—	Reserved.
32:35	COMPACE	Command pipeline delay.
36:40	STATLAT	Transfer-handshake response latency.
41:44	SNOOPLAT	Processor snoop latency.
45:48	SNOOPACC	Snoop accumulation delay.
49	APSEL	Bus encode disable. ('1' = disable)
50	bcfg_en_rww_reg	Enable commands during data writes. ('0' = enable)
51:52	bcfg_align_cmd	Align command. 00 Command out on even or odd. 01 Command out on even. 10 Command out on odd.
53	bcfg_cp_aresp_ena	Disables wait for cresp for castouts and pushes. ('1' = disable)
54	bcfg_en_cmpc_tms_8	Enables a longer wait period before "back-off."
55	bcg_en_cmpc_ajst	Enables bus "back-off" of sending command.
56	grs_bfb_mode_reg(24)	Sets bus mode to no encode with single error correct and some double error detect.
57	grs_bfb_mode_reg(25)	Sets bus mode to no encode with single error correct and double error detect.
58	grs_bfb_mode_reg(26)	Power tuning engine disable.
59:63	—	Reserved.

IBM PowerPC 970MP RISC Microprocessor
11.2.3.2 North Bridge Configurable Timing Delay Parameter Register

SCOM Address x'0A8000'



Bits	Field Name	Description
0:9	—	Reserved.
10:15	PAAMWIN	Minimum number of bus beats between command packets reflected from the North Bridge to the processors when there is an address collision.
16:19	—	Reserved.
20:23	SNOOPWIN	Snoop window pacing.
24:30	—	Reserved.
31	BCM	Bus encode disable. 1 Disabled.
32:35	—	Reserved.
36:39	—	Reserved.
40:42	—	Reserved.
43:47	STATLAT	Transfer-handshake response latency.
48:49	—	Reserved.
50:55	SNOOPLAT	North Bridge snoop latency.
56:63	—	Reserved.

11.2.4 Power Management

The processor interconnect participates in the system power management through two asynchronous control signals called quiescent request (QREQ) and quiescent acknowledgment (QACK). QREQ is a processor output signal that is asynchronously sampled by the local clock of the North Bridge. QACK is a North Bridge output signal that is asynchronously sampled by the local clock of the processor and other bus masters.

Figure 11-4 on page 286 shows the sequence of steps for the processor to enter Doze or Nap mode.

Figure 11-5 on page 287 shows the sequence of complementary steps taken by the North Bridge in response to the assertion or negation of QREQ by a processor. In Doze mode, the processor must be capable of snooping all reflected command packets from the North Bridge. In Nap mode, the processor is not required to snoop transactions, although it must be capable of returning to Doze mode for the purpose of snooping if QACK is negated.

In the normal (or preferred) sequence of events, the processor and North Bridge observe a 4-phase handshake for QREQ and QACK. The processor first asserts QREQ after the processor has quiesced, the snoopers are idle, and all outstanding processor interconnect bus transactions have completed. The processor then waits for the North Bridge to assert QACK. While the processor is waiting for the assertion of QACK, it is in an intermediate mode called Doze. Once the North Bridge asserts QACK, the processor enters Nap mode. To exit Nap mode, the processor negates QREQ and then waits for the North Bridge to negate QACK before returning to the Run state.

There are a few scenarios in which the 4-phase handshake is preempted.

1. While in Doze mode, the North Bridge reflects command packet snooping. The action taken by the processor is to negate QREQ while snooping the reflected command packet and while staying in Doze mode.
2. While in Doze mode, the processor receives an interrupt. The action taken by the processor is to negate QREQ and return to the Run state.
3. While in Nap mode, the North Bridge negates QACK while the processor has QREQ asserted. The processor must then return to Doze mode within 64 bus clocks so that it can return to snooping reflected command packets from the North Bridge.

As shown in *Figure 11-5* on page 287, the North Bridge normally negates QACK when QREQ is negated by any of the attached processors. However, it might also negate QACK if there is bus activity from any of the other attached bus devices that can be a bus master.

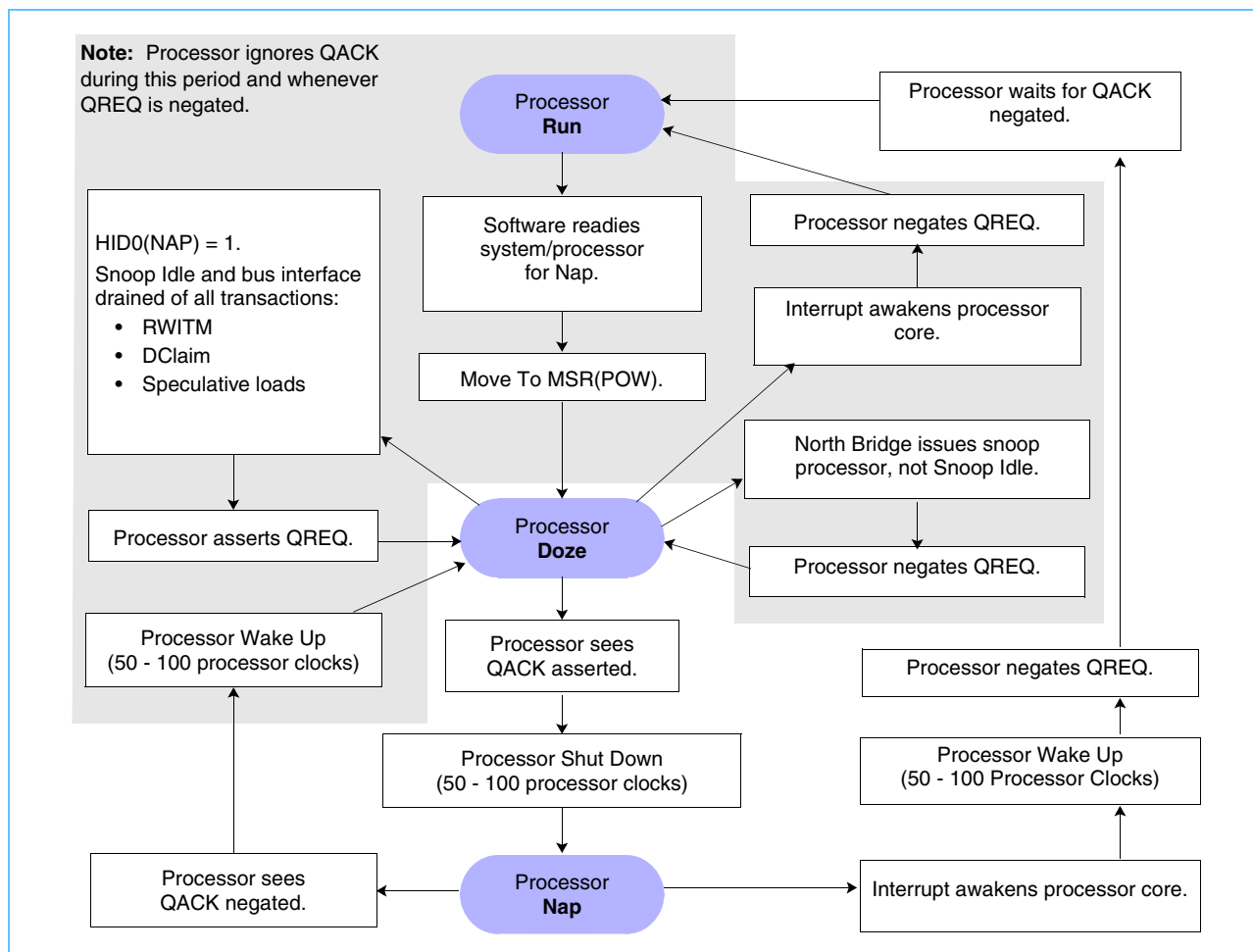
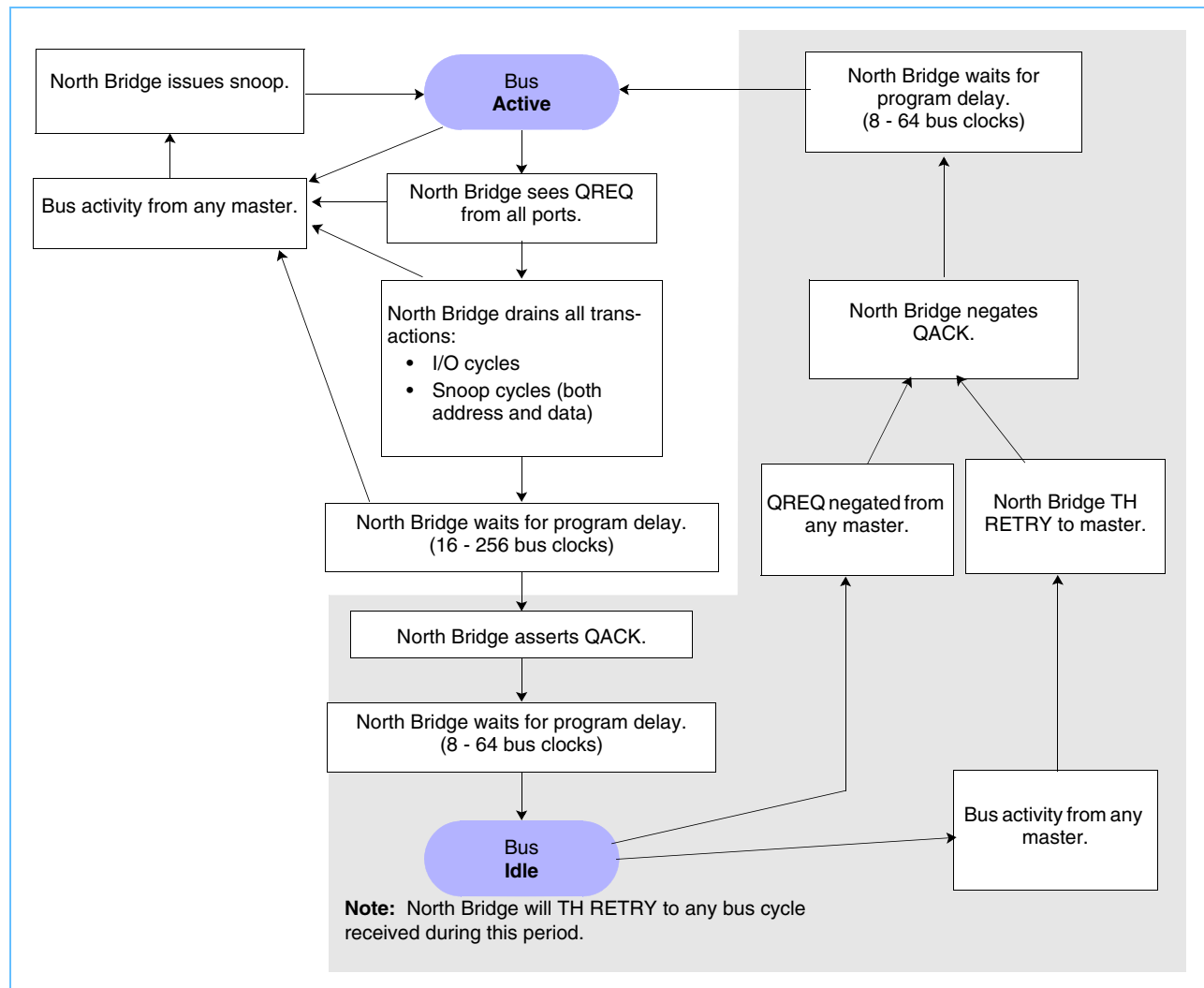
IBM PowerPC 970MP RISC Microprocessor
Figure 11-4. Processor QREQ and QACK Signalling


Figure 11-5. North Bridge QREQ and QACK Signalling



11.2.5 Reliability, Availability, and Serviceability (RAS) Requirement

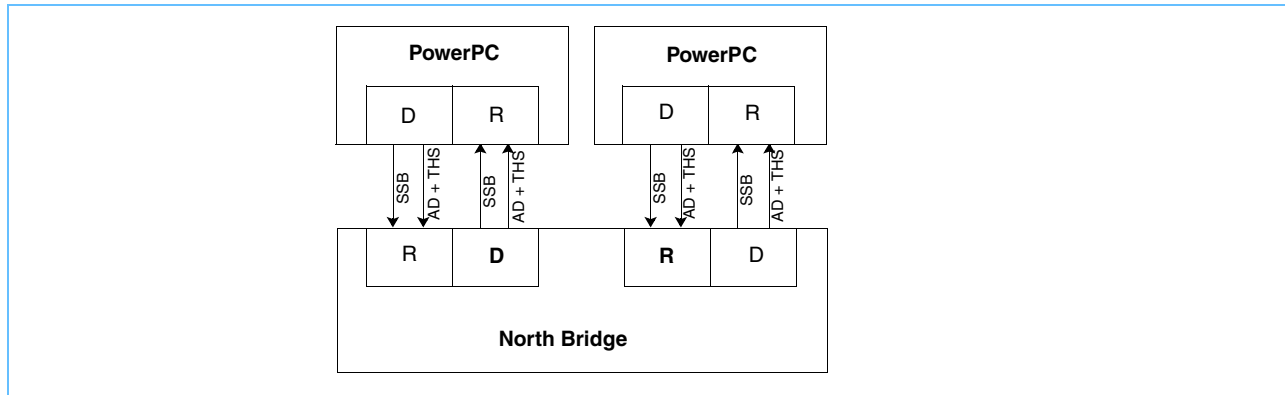
All devices attached to the processor interconnect bus must implement three registers that capture errors and assist in the isolation of failures. RAS circuitry ensures that a processor interconnect implementation that targets a particular processor will remain stable across various target processors.

A Fault Isolation Register (FIR), a Fault Isolation Capture Register (FICR), and a Fault Isolation Mask Register (FIMR) must be implemented. The FIR captures all failures that occur. The register is not frozen on the first error but continues to accumulate all detected errors. The FICR is used to log the first detected error. This register is a masked copy of the FIR (output of the FIR is masked with the FIMR) and is frozen once the first error is detected. The actual errors that are logged and the error reporting mechanism are system-dependent (see *Chapter 4 Exceptions* for more information).

11.3 Processor Interconnect Electrical Interface

The processor interconnect uses high-speed, source-synchronous buses (SSBs) to transfer data between the PowerPC and North Bridge chips, and to support the cache-coherency snooping protocols for multiprocessor configurations. The SSBs are unidirectional point-to-point connections between a drive side (D) and a receive side (R). SSBs are put into pairs to form a bidirectional channel between a PowerPC and a North Bridge chip as shown in *Figure 11-6*.

Figure 11-6. Bus Diagram of a Dual-Processor 970MP Processor Interconnect-Based System



Source-synchronous bus (SSB) data is transferred on every bus clock edge; that is, double the data rate (DDR) of the bus-clock frequency. There are 50 signal lines per SSB. Two lines are used for the differential bus clock lines, 44 signal lines are used to communicate 36 bits of logical data, and four signal lines are used for the differential snoop-response bus. The 36 data bits consist of 35 bits of the address/data (AD) channel and a single bit for the transfer-handshake bus (TH).

The SSBs achieve high-speed operation using low-cost packaging solutions by exploiting four features:

1. **Source-synchronous signalling.** The differential bus clocks are bundled with the single-ended data signals.
2. **Far-end (parallel) termination.** The single-ended data signals use parallel termination at the far end of the signal line to absorb signal reflections and maintain a quasi-constant current loading for each data signal line.
3. **Balanced coding.** The application of balanced coding to the SSB maintains a quasi-constant current loading across the entire SSB interface. Within the SSB, there is no net current flow across the power planes. This dramatically reduces noise problems due to power-supply rail collapse (that is, di/dt noise) and current voltage offsets between the chips.
4. **Point-to-point unidirectional signalling.** Restricting the signal fan-out to a single point and keeping the signal flow unidirectional mitigates problems associated with high-frequency signal attenuation.

11.3.1 Initialization at Power-On Reset

The receive-side circuitry for the SSBs inside a processor interconnect system might require initialization at power-on to deskew data signal lines, align the bus clocks, and synchronize the receive-side FIFO queues to the local clock domains of the ASICs and processors. Within a processor interconnect system, there is the concept of time zero, which is globally established across all the chips. In the processor interconnect, time zero is derived from the phase synchronization (*psync*) and global system clock (SYSCLK) signals (see *Section 11.3.2 Target Cycle* on page 289).

The purpose of the initial alignment pattern (IAP) is to establish the settings for the delay lines of the per bit deskew circuitry and optimize the positioning of the sampling clocks on the receive side. During IAP, each drive side transmits a bit pattern sequence across each SSB. This pattern is repeated by the drive side for as many bit times (for example, 500,000) as needed by the initialization sequential circuitry on the receive side. The I²C interface controls for how long the pattern is repeated. Upon IAP completion, the receive-side reports its status through a 10-bit PI Status Register, which is accessible from the I²C interface. An all-zero result stored in PI Status Register indicates that the IAP completed without error. A non-zero pattern indicates that there was an error. *Section PI Status Register* on page 351 describes the bit fields and their meaning.

The sequence of the power-on reset steps is:

1. Stabilize and lock the clocks to the globally distributed SYSCLK.
2. The drive side of each SSB begins transmission of the test patterns for receive-side calibration and optimization. This step is initiated from the I²C interface by a sequence that is system dependent.

Wait for a completion signal from each receive side SSB that has completed the IAP. The completion signal is registered and can be accessed from the I²C interface. The location of the register and how it is accessed through the I²C interface is implementation dependent. The transmission of the test patterns is terminated once the completion signal is detected. The results of the initialization can be read out from the I²C interface, and the bus is ready for general system use.

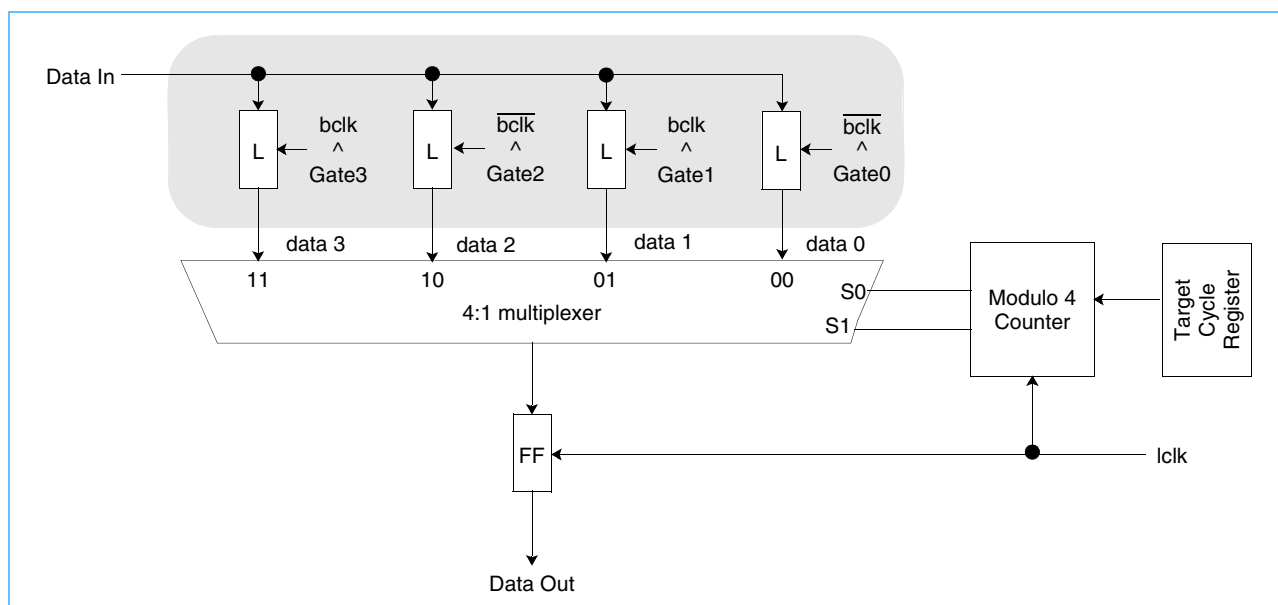
11.3.2 Target Cycle

The flight time of a data signal from the drive side to the receive side of an SSB can extend beyond the period of a single bus clock. The principles of the processor interface allow data and clocks signals to take multiple bus clock cycles to travel from one side to the other.

Furthermore, each receive side can be programmed to transfer SSB data across the time-domain boundary on the same target beat relative to time zero, which is the globally synchronized time domain for all of the processors in a processor interconnect system.

This synchronization can be accomplished using a FIFO-type circuit such as the one in *Figure 11-7*. The four gate signals (Gate0 through Gate3) are derived from the incoming bus clock (bclk) of the SSB. These signals are half the frequency of the bus clock, have a 50% duty cycle, and are 90 degrees out of phase from each other (see *Figure 11-8* on page 290). During the IAP, the gate signals are shifted one bit at a time until the '1' in the IAP pattern is aligned into the rightmost latch (data 0) and the '0' is captured in the leftmost latch (data 3). This alignment procedure occurs in the shaded box of *Figure 11-7*.

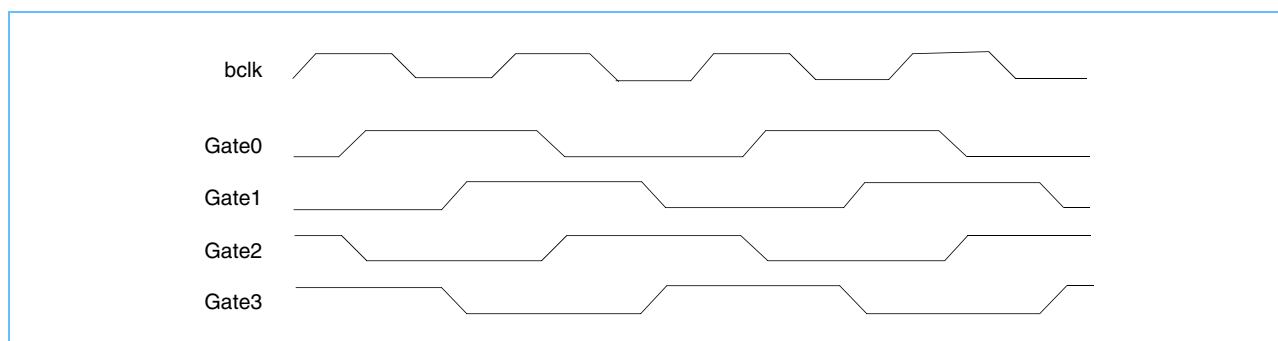
Figure 11-7. Receive-Side FIFO Circuit



The 4:1 multiplexer and the modulo 4 counter are used to cross the time domain from that of the SSB bus clock to the local clock (Lclk) of the chip. A static timing analysis determines the worst case aggregate latency from the drive side through the receive side up to the input point of the data out flip-flop (see *Figure 11-7*).

The combination of the 4:1 multiplexer and modulo 4 counter establishes four possible target cycles for transferring data between the two time domains. The duration of each target cycle equals one bit time. Depending on the results of the worst case analysis, it might be determined that SSB input data cannot be clocked into the data out flip-flop in the same target cycle that it arrives. This will occur if the timing violates the set-up and hold time requirements of the data out flip-flop. In this case, one of the other three target cycles is selected. For example, the following cycle would allow the shortest safe latency, but later cycles would provide larger set-up times. The target cycle is programmed through the I²C interface by loading a 2-bit value into the Target Cycle Register, which in turn initializes the modulo 4 counter relative to time zero.

Figure 11-8. Timing Diagram Showing Relationship Between Bclk and the Four Gate Signals



11.4 Processor Interconnect Bus Error Detection and Correction

11.4.1 Error Detection for Balanced Encoding

The processor interconnect bus protocol defines an encoding of each beat of information on the inbound address/data (ADI) and outbound address/data (ADO) bus, so that exactly half the signals carry a '1' bit and half the signals carry a '0' bit on each beat. This is done by converting the 36 bits of information on each bus to the 44-bit pattern that is transferred, in a scheme called balanced coding. This balanced coding scheme implicitly provides parity checking of the bus signals, in that an unequal number of ones and zeros in any beat indicates an error. This balanced coding bus mode is selected by setting the BUSCONF bit 49 to '0' (see *Section 11.2.3.1 Processor Configurable Timing Delay Parameter Register (BUSCONF)* on page 283).

11.4.2 Error Detection for Alternative Encodings

The 970MP design supports three unencoded bus modes, in which bits 0:35 of the ADI and ADO bus carry the address and data information, while bits 36:43 carry checking information. Bits 49, 56, and 57 determine the unencoded bus modes as follows:

- 100 This mode, described in *Section 11.4.2.1 Single-Error and Double-Error Detection*, does not provide single error correction. It only provides single and partial double bit detection.
- 110 This mode is the same as mode '100' except that single bit errors are corrected.
- 101 This mode is described in *Section 11.4.2.2 Single-Error Correct, Double-Error Detection*.
- 111 This mode is undefined.

11.4.2.1 Single-Error and Double-Error Detection

The first of these unencoded bus modes implements the following 10-input parity functions to generate the eight check bits:

```

b36 = P( b0, b6, b7, b29, b30, b31, b32, b33, b34, b35 )
b37 = P( b0, b1, b6, b23, b24, b25, b26, b27, b28, b35 )
b38 = P( b0, b1, b2, b18, b19, b20, b21, b22, b28, b34 )
b39 = P( b1, b2, b3, b14, b15, b16, b17, b22, b27, b33 )
b40 = P( b2, b3, b4, b11, b12, b13, b17, b21, b26, b32 )
b41 = P( b3, b4, b5, b9, b10, b13, b16, b20, b25, b31 )
b42 = P( b4, b5, b7, b8, b10, b12, b15, b19, b24, b30 )
b43 = P( b5, b6, b7, b8, b9, b11, b14, b18, b23, b29 )

```

where P(0 to 7) computes even parity over its input signals.

In the receiver, the error syndrome is computed by exclusive ORing the received and generated check bits. A syndrome of x'00' results when the received and generated check bits match, indicating that no error occurred. A non-zero syndrome indicates that an error occurred. This check bit implementation will detect any single-bit or double-bit error over the 44-bit pattern. This first unencoded bus mode is selected by setting BUSCONF bits 49, 56, and 57 to '100'.

Note: Single-bit errors that occur using this bus mode yield syndromes that allow the failing bit to be identified. However, some double-bit errors yield those same single-bit error syndromes. For this reason, this mode can be used to detect all single-bit and double-bit errors, but cannot be safely used to correct single-bit errors.

11.4.2.2 Single-Error Correct, Double-Error Detection

The second unencoded bus mode implements the following even parity functions to generate the eight check bits:

```

b36 = P( b23, b24, b25, b26, b27, b28, b29, b30, b31, b32, b33, b34, b35 )
b37 = P( b9, b10, b11, b12, b13, b14, b15, b16, b17, b18, b19, b20, b21, b22 )
b38 = P( b3, b4, b5, b6, b7, b8, b18, b19, b20, b21, b22, b33, b34, b35 )
b39 = P( b2, b5, b6, b7, b8, b15, b16, b17, b22, b29, b30, b31, b32 )
b40 = P( b1, b3, b4, b8, b12, b13, b14, b17, b21, b26, b27, b28, b32, b35 )
b41 = P( b0, b1, b2, b4, b7, b10, b11, b14, b20, b24, b25, b28, b31 )
b42 = P( b0, b1, b3, b6, b9, b11, b13, b16, b19, b23, b25, b27, b30, b34 )
b43 = P( b0, b2, b5, b9, b10, b12, b15, b18, b23, b24, b26, b29, b33 )

```

In the receiver, the error syndrome is computed by exclusive ORing the received and generated check bits. A syndrome of x'00' results when the received and generated check bits match, indicating that no error occurred. A non-zero syndrome indicates that an error occurred. *Table 11-4* on page 293 lists the syndromes from all single-bit errors, along with which failing bit causes that syndrome.

All non-zero syndromes that are not listed in *Table 11-4* indicate double-bit errors.

This check bit implementation can be used to correct any single-bit error and to detect any double-bit error over the 44-bit pattern. This second unencoded bus mode is selected by setting BUSCONF bits 49, 56, and 57 to '101'.

Table 11-4. Bit Error Position Identifier

Syndrome	Failing Bit
x'07'	0
x'0E'	1
x'15'	2
x'2A'	3
x'2C'	4
x'31'	5
x'32'	6
x'34'	7
x'38'	8
x'43'	9
x'45'	10
x'46'	11
x'49'	12
x'4A'	13
x'4C'	14
x'51'	15
x'52'	16
x'58'	17
x'61'	18
x'62'	19
x'64'	20
x'68'	21

Syndrome	Failing Bit
x'70'	22
x'83'	23
x'85'	24
x'86'	25
x'89'	26
x'8A'	27
x'8C'	28
x'91'	29
x'92'	30
x'94'	31
x'98'	32
x'A1'	33
x'A2'	34
x'A8'	35
x'80'	36
x'40'	37
x'20'	38
x'10'	39
x'08'	40
x'04'	41
x'02'	42
x'01'	43

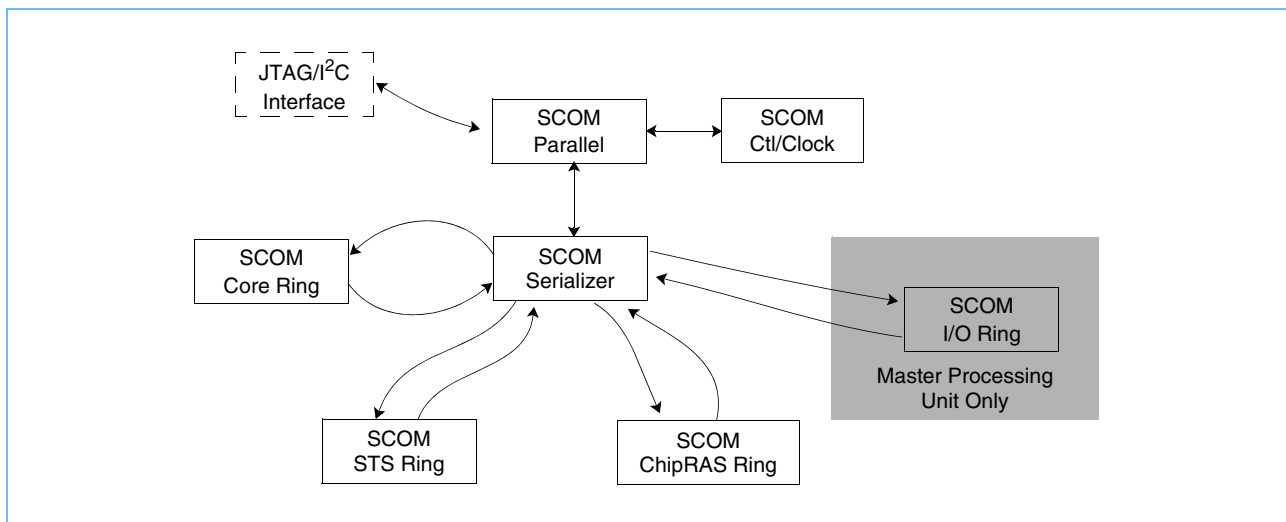


12. SCOM Interface and Registers

Scan communication (SCOM) is used to access vital chip debug and diagnostic facilities while the chip is running without stopping clocks. It is implemented as address and data serial rings running through the chip to limit the wiring. Each facility has a unique address on this ring, which is used to address it.

On the 970MP, all the SCOM facilities are per processor units except for the I/O SCOM facilities that are only available on the master processing unit (see *Figure 12-1*). The serial ring is split into four independent rings running in the four clock domains, so that a clock stop in the one domain will not break the SCOM. A small number of facilities that control the SCOM configuration and the chip clocks are addressed directly without using the serial ring.

Figure 12-1. Processor Unit SCOM Topology



12.1 Processor Core SCOM SPR Access

Each processor (core) has two special purpose registers (SPRs) used to access the SCOM interface: SCOMC and SCOMD. SCOMC and SCOMD are both 64-bit read/write SPRs and are used for SCOM Control and SCOM Data respectively. The interface is implemented as a direct connection to the parallel-to-serial converter, which handles the arbitration between the core and service processor.

12.1.1 Operating System Protocol to Access SCOM SPRs

In the 970MP, SCOMC and SCOMD are complete operations. They do not require a software protocol in order to function properly except to disable external (asynchronous) interrupts. Software must check the error bits after performing an SCOMC to ensure that the command successfully completed. *Table 12-1* on page 296 outlines a general software protocol for using these registers.

Table 12-1. Operating System Code to Access SCOM

For SCOM READ	For SCOM WRITE
set MSR[EE] = '0' MTSCOMC MFSCOMD MFSCOMC if Error = '1', branch to SCOM error routine set MSR[EE] = '1'	set MSR[EE] = '0' MTSCOMD MTSCOMC MFSCOMC if Error = '1' branch to SCOM error routine set MSR[EE] = '1'

Asynchronous interrupts must be disabled during these blocks. Otherwise, an interrupt could arrive and make the SCOM port busy. If that occurs between the MF and MT instructions that cause the reads and writes, then the SCOM interface out of the core could malfunction (or at least not perform as software intended).

12.1.2 SCOMD Format

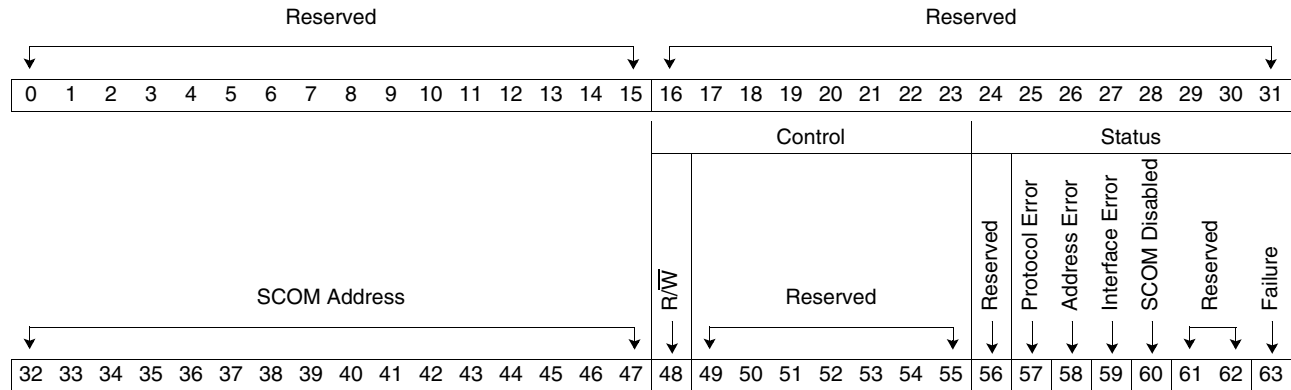
The SCOMD is a 64-bit register. The interpretation of the contents of this register is determined by the SCOMC Status and Control bits. It is the source for outgoing data *during* an SCOM write access (SCOMC[RW] = '1' when MTSCOMC is issued). It is the destination for incoming data *after* an SCOMC read access (SCOMC[RW] = '0' after MTSCOMC completes).

I

12.1.3 SCOMC Format

The SCOMC is divided into four 16-bit fields, as shown *Figure 12-2*.

Figure 12-2. SCOMC SPR Format



SCOMC Bits	Type	Usage	Description
0:31	Unused	—	Reserved
32:47	Write-Only	Address	SCOM Address (0:15)
48	Write-Only	Control	SCOM Read/Write Request Bit 0 Write request 1 Read request
49:55	Unused	Control	Reserved
56	Read-Only	Status	Reserved
57	Read-Only	Status	SCOM Protocol Error
58	Read-Only	Status	SCOM Address Error
59	Read-Only	Status	SCOM Interface Error
60	Read-Only	Status	SCOMC disabled by service processor
61	Read-Only	Status	Reserved (Zero)
62	Read-Only	Status	Reserved (Zero)
63	Read-Only	Status	Failure (SCOMC disabled or Interface Error [formerly Busy])

The reserved fields should be written to zeros by the software on an MTSCOMC and return zeros on an MFSCOMC. The Address and Control fields are undefined while Failure equals '1'.

All SCOMC Status bits will be cleared by the hardware upon an MTSCOMC with the exception of Failure, which is set to indicate to the operating system that the SPR SCOM access is active. Additional status bits will be set depending on the status of the SCOM operation:

- **Protocol Error:** The SCOM hardware has violated a basic protocol, such as giving a grant when not asked or returning a data packet when expecting an address packet. This error bit is *not* cleared on the next MTSCOMC.

Note: This bit will probably cause a checkstop to occur and sets a corresponding bit in the Fault Isolation Register (FIR).

IBM PowerPC 970MP RISC Microprocessor

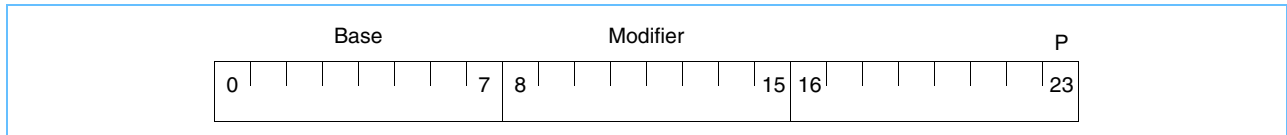
This bit indicates a problem has occurred in the SCOM hardware, and that this interface can no longer be trusted.

- **Address Error:** The SCOM address was not recognized by an SCOM satellite. This indicates that the write did not happen or that the read returned no data (depending on the R/W bit). This error bit is cleared on the next MTSCOMC. This indicates a probable software error.
- **Interface Error:** If the SCOM logic in the arbiter detects an error condition, such as a timeout on the SCOM interface, or if the core hang recovery engages while SCOMC is active, it sends a SCOM reset (screset). This causes the SCOMC operation to be killed and the logic to record an error. The error bit is cleared on the next MTSCOMC and is recoverable (the command must be retried).
- **SCOMC Disabled:** The service processor has the ability to disable a core from becoming an SCOM master, causing the core to treat MTSCOMC as a NOP. MFSCOMC will set this bit, along with Failure and Interface Error to ensure the software realizes this condition.
- **Failure:** Summary indicating if there were any errors since the last MTSCOMC. Formerly the “Busy” bit, which indicated if the SCOMC interface was in use.

12.2 SCOM Address Allocation

Scan communications support a 23-bit address; the 24th bit is a parity bit. This is the address that would be sent to SCOM through the JTAG/I²C port. The internal SCOM bus, the part that is serialized, needs no more than 16 bits of addressing. So, to simplify the logic, addresses sent internally are truncated to 16 bits. Thus, bits 0 through 15 are sent to the parallel SCOM controller, and bits 16 through 22 must be zero for chip SCOM addresses.

Figure 12-3. Format of an SCOM Address



For the bus interface unit (BIU), each unit's SCOM serial address and data are daisy chained together. For example, the BIU SCOM address and SCOM data out connect to the SCOM address and SCOM data in of the L2 cache. The SCOM address and SCOM data out of the L2 cache connect to the BIU SCOM address and SCOM data in. The BIU SCOM address and SCOM data out connect to the BIU SCOM address and SCOM data in.

Within the BIU, bit 0 of the serial SCOM address is the start bit, and bit 18 is the stop bit. Bit 17 is for SCOM read or write operations ('1' equals read; '0' equals write). Bits 1 through 8 are the SCOM base address for the BIU SCOM registers. Bits 9 through 16 are the actual SCOM address for each register. For the internal serial SCOM data bus, each SCOM register in the BIU is 32 bits wide. Bit 32 of the SCOM data bus is the stop bit.

Figure 12-4. Format of an SCOM Address within the BIU

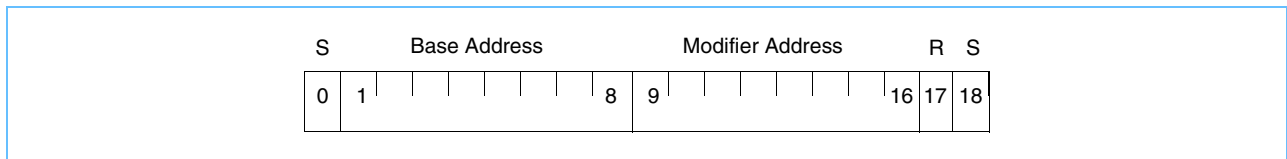
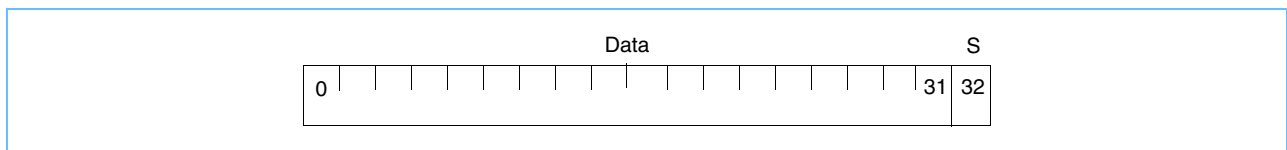


Figure 12-5. Format of an SCOM Data Bus



IBM PowerPC 970MP RISC Microprocessor

Table 12-2 shows the base address decodes. All data accesses to the SCOM bus are 64 bits wide. The domain column in Table 12-2 lists the following information for each SCOM register access:

- which clock domain in the chip must be on (use SCOM 800000)
- which SCOM ring must be on (use SCOM 60000). The SCOM ring for the ChipRAS domain is always enabled.

Table 12-2. SCOM Base Addresses

Base SCOM Address (0:7)	Functional Unit	Macro	Domain
x'01'	Core (Debug Unit)	td_cp_dbg	Core
x'02'	Core (RAS Unit)	td_cp_ras	Core
x'03'	Core (Fault Isolation Register [FIR] Unit)	td_cp_fir	Core
x'04'	L2 Slice	v_cerrs	STS/BIU
x'08'	I/O	z_ei_scom	I/O
x'0A'	BIU Controller	t_gusras_reg	STS/BIU
x'40'	Power-On Reset (POR)	tc_por	ChipRAS
x'50'	Global Controls (Free Running)	ts_glob	ChipRAS
x'60'	SCOM Mode/Status (Free Running)	t_pscm_cntl	Always available
x'8[0:4]'	Clock Controls (Free Running)	tc_ccintf	Always available

Table 12-3 lists the modifier address decodes.

Table 12-3. SCOM Modifier Addresses (Page 1 of 3)

Modifier SCOM Address (9:16)	Register	Domain	See Page
x'021001'	CoreRAS Control (Pulsed) Register	Core	305
x'021100'	CoreRAS Mode Register	Core	307
x'021200'	CoreRAS Status Register	Core	311
x'021301'	Core Hang-Recovery Control Register	Core	314
x'021400'	Core Power Down and Idle Status Register	Core	317
x'022001'	Service Processor Special Attention (SP-ATTN) Register	Core	318
x'022100'	Service Processor And-Mask Register	Core	318
x'022200'	Service Processor Or-Mask Register	Core	318
x'022601'	Asynchronous Machine-Check Source Register	Core	319
x'022700'	Asynchronous And-Mask Register	Core	319
x'022800'	Asynchronous Or-Mask Register	Core	319
x'023000'	Instruction Address Breakpoint Register	Core	320
x'023101'	Hardware Implementation Dependent Register 0 (HID0)	Core	321
x'023201'	Hardware Implementation Dependent Register 1 (HID1)	Core	322
x'023300'	Instruction Match CAM (IMC) Register	Core	323
x'023401'	Patch Map (IMC Write Control Register)	Core	324
x'023500'	Hypervisor Decrementer	Core	325

Table 12-3. SCOM Modifier Addresses (Page 2 of 3)

Modifier SCOM Address (9:16)	Register	Domain	See Page
x'023600'	Time Base Register	Core	326
x'024001'	Performance Monitor Sampling Control Register	Core	327
x'030001'	Core Fault Isolation Register	Core	328
x'031000'	Core And-Mask Register	Core	328
x'032000'	Core Or-Mask Register	Core	328
x'030400'	Core Fault Isolation Mask Register	Core	332
x'031401'	Core And-Mask Register	Core	332
x'032401'	Core Or-Mask Register	Core	332
x'030800'	Core Checkstop Enable Registers	Core	333
x'030901'	Core Machine-Check Enable Register	Core	334
x'036001'	Instruction Mark Configuration Register	Core	335
x'040000'	L2 Fault Isolation Register	STS/BIU	337
x'041001'	L2 Fault Isolation And-Mask Register	STS/BIU	337
x'042001'	L2 Fault Isolation Or-Mask Register	STS/BIU	337
x'040801'	L2 Fault Isolation Checkstop Register	STS/BIU	337
x'040401'	L2 Error Mask Register	STS/BIU	337
x'041400'	L2 Error And-Mask Register	STS/BIU	337
x'042400'	L2 Error Or-Mask Register	STS/BIU	337
x'040801'	L2 Checkstop Enable	STS/BIU	337
x'043000'	BIU Mode Register	STS/BIU	345
x'083000'	PI Mode Register 0	I/O	347
x'083101'	PI Mode Register 1	I/O	348
x'083201'	PI Mode Register 2	I/O	349
x'083300'	PI Mode Register 3	I/O	350
x'084001'	PI Status Register	I/O	351
x'085000'	PI Command Register	I/O	355
x'086000'	Driver Initial Alignment Pattern (IAP) Register	I/O	356
x'086101'	Receiver IAP Register	I/O	356
x'0A0001'	BIU Fault Isolation Register/And-Mask/Or-Mask	STS/BIU	340
x'0A0400'	BIU Error Mask/And-Mask/Or-Mask	STS/BIU	342
x'0A0800'	BIU Checkstop Enable	STS/BIU	343
x'0A8000'	Processor Configurable Timing Delay Parameter Register (BUSCONF)	STS/BIU	283
x'0A9000'	BIU Status Register	STS/BIU	344
x'400000'	Power-On Reset Status Register	ChipRAS	357
x'400101'	Power-On Reset Continue Register	ChipRAS	359
x'400201'	Power-On Reset I2C/JTAG Arbitration Register	ChipRAS	360

IBM PowerPC 970MP RISC Microprocessor
Table 12-3. SCOM Modifier Addresses (Page 3 of 3)

Modifier SCOM Address (9:16)	Register	Domain	See Page
x'400801'	Power-Management Control	ChipRAS	361
x'401400'	Power-On Reset Sequence Register 0	ChipRAS	363
x'402400'	Power-On Reset Sequence Register 1	ChipRAS	364
x'404400'	Power-On Reset Sequence Register 2	ChipRAS	365
x'408001'	Power Tuning Status Register	ChipRAS	366
x'500001'	Global Fault Isolation for Checkstop Conditions (Global FIR)	ChipRAS	367
x'500400'	Error Enable Mask	ChipRAS	368
x'500601'	Mode Register for Fault Isolation Registers	ChipRAS	369
x'500700'	Debug Mode Register	ChipRAS	370
x'503001'	Hang Pulse Generation	ChipRAS	372
x'503100'	Early Hang Pulse Generation	ChipRAS	373
x'504101'	Chip ID Register	ChipRAS	374
x'600001'	SCOM Mode Register	Always available	375
x'600100'	SCOM Controller Error Register	Always available	377
x'600200'	Electronic Chip ID	Always available	379
x'600400'	Clock Ratio Register (N:1 Phase Hold Control)	Always available	380
x'800000'	Clock Command Register	Always available	381
x'800003'	Status Register	Always available	383
x'800006'	Phase Synchronization Control Register	Always available	385
x'800009'	Clock Command Control Register	Always available	386
x'80000A'	Energy Star Register	Always available	391
x'80000C'	Status Register Mask	Always available	393
x'80000F'	I/O Control Register	Always available	394
x'820004'	ABIST Status Register	Always available	395
x'840002'	LBIST Options Register	Always available	396
x'840008'	LBIST Channel Length Register	Always available	398
x'84000B'	LBIST Test Length Register	Always available	399
x'84000D'	Clock Ramping Configuration Register	Always available	400

12.2.1 Register Description Conventions

The descriptions of the SCOM registers use the following terms:

Name	Refers to the instantiated latch name that will show up in the SCAN_DEF. The facility name enclosed in brackets is the data-out pin that can be referenced in the all event trace (AET) waveform file format.								
Reserved	Indicates that the latch might be implemented. The customer should write zeros and expect unknown data.								
Not Implemented	Indicates that the latch is not implemented (N/I). The customer should write zeros and expect zeros.								
Type	SCOM request type: <table> <tr> <td>RW</td><td>Read/Write</td></tr> <tr> <td>RO</td><td>Read Only. Write requests to a read-only SCOM register are treated as NOPs, and the data is thrown away. A good response is returned.</td></tr> <tr> <td>WO</td><td>Write Only. Read requests to a write-only SCOM register will result in an error condition.</td></tr> <tr> <td>RWor</td><td>Read/Write OR. The write operation is a special type that will OR into the specified bits in the register. An associated write-only AND mask is provided to enable clearing bits in this type of SCOM register.</td></tr> </table>	RW	Read/Write	RO	Read Only. Write requests to a read-only SCOM register are treated as NOPs, and the data is thrown away. A good response is returned.	WO	Write Only. Read requests to a write-only SCOM register will result in an error condition.	RWor	Read/Write OR. The write operation is a special type that will OR into the specified bits in the register. An associated write-only AND mask is provided to enable clearing bits in this type of SCOM register.
RW	Read/Write								
RO	Read Only. Write requests to a read-only SCOM register are treated as NOPs, and the data is thrown away. A good response is returned.								
WO	Write Only. Read requests to a write-only SCOM register will result in an error condition.								
RWor	Read/Write OR. The write operation is a special type that will OR into the specified bits in the register. An associated write-only AND mask is provided to enable clearing bits in this type of SCOM register.								

12.2.2 SCOM Error Handling

If an error occurs while servicing an SCOM command, the ANY_SCATTN bit in the Access Status Register is raised. The service processor unit (SPU) should issue an SCOM reset by using the Instruction Register (IR) operation code x'1B'. Then, the SPU should read the SCOM Controller Error Register (x'600100') to see what the fault was. To clear ANY_SCATTN, first write all zeros to the SCOM Controller Error Register. Then write all zeros to the JTAG SCOM Status Register (x'000080'). Finally write all zeros to the JTAG Access Status Register (x'000002').

The following error indications in bits 0 through 23 of the SCOM Controller Error Register might be due to programming errors:

x'010000'	Invalid address. Does not match any known address ranges.
x'001041'	The address was decoded as a serial type and sent onto the SCOM serial ring, but no SCOM satellite accepted ownership of the address. This is probably due to the use of an invalid address.
x'001040'	The address was decoded as a serial type read, but no data was returned. This is probably due to an invalid read request being sent on the serial SCOM bus. This error, for example, will happen if the customer attempts to read a write-only register.

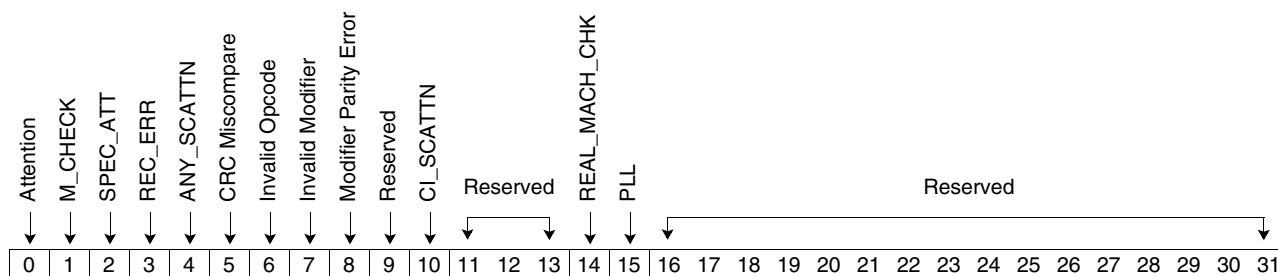
Bits 24 through 27 of the SCOM Controller Error Register contain the failing SCOM address.

IBM PowerPC 970MP RISC Microprocessor

12.2.3 Access Status Register

The Access Status Register contains bits that indicate error states that might occur during instruction or data scanning. This register is flushed to all zeros during POR.

Modifier Address x'000002'



Bits	Field Names	Description
0	Attention	(Master) The source for this bit is ATTENT_DR.
1	M_CHECK	Input to Access from the chip logic. Note: This is the system checkstop.
2	SPEC_ATT	SPEC_ATT input to Access from the chip logic.
3	REC_ERR	REC_ERR input to Access from the chip logic.
4	ANY_SCATTN	The OR of all SCOM attentions.
5	CRC Mismatch	If set, a scan data check mismatch was found on a scan-in operation.
6	Invalid Opcode	The IR opcode is not supported.
7	Invalid Modifier	The instruction does not support the received modifier.
8	Modifier Parity Error	Odd parity is required across bits 8 - 31 of the Instruction Register (the modifier address), except as noted in the Instruction Register description.
9	Reserved	Reserved.
10	CI_SCATTN	CI_SCATTN attention from the clock tree SCOM logic located in the ChipRAS clock control macro, ccintf.
11:13	Reserved	Reserved.
14	REAL_MACH_CHK	Machine check attention from the PowerPC 970 core.
15	PLL	PLL has lost lock.
16:31	Reserved	Reserved (not writable).

12.3 Core Pervasive SCOM Register Definitions

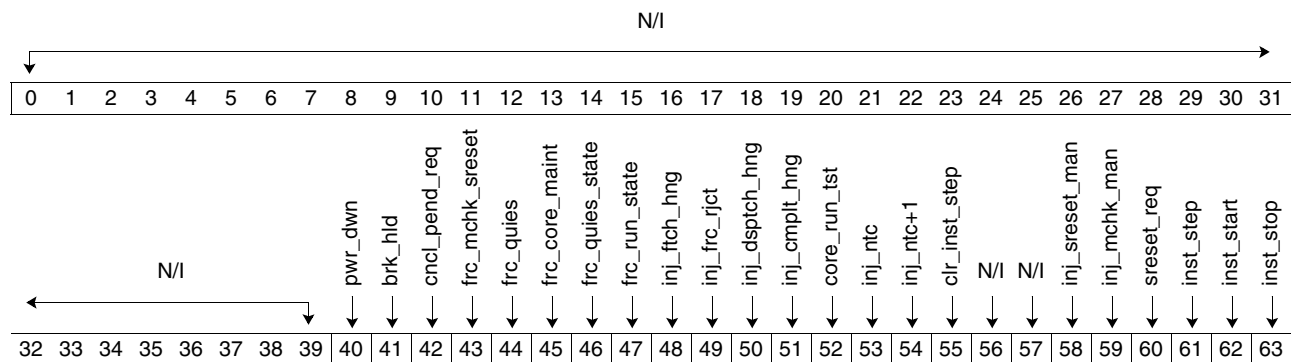
12.3.1 Processor CoreRAS Facilities (x'02[1:4]XXX')

CoreRAS Control (Pulsed) Register

Address x'021001'

Type WO

Reset N/A



Bits	Field Name	Description
0:39	N/I	Not implemented.
40	pwr_dwn	Enter core Power Down mode. This works without setting the Power Management bit of the MSR Register (MSR[POW]) (see bits 5:6 of the <i>CoreRAS Mode Register</i> on page 307).
41	brk_hld	Break hold on IFU/LSU status after core hang detect due to external source (see bits 59:63 of the <i>CoreRAS Status Register</i> on page 311).
42	cncl_pend_req	Cancel pending requests (quiesce, soft reset, machine check, core step). Go ahead and accept pending mode changes as well.
43	frc_mchk_sreset	Force the next machine check or soft reset (and all interrupts until then) to be marked nonrecoverable.
44	frc_quies	Force the core to quiesce manually (RAS LOGIC OVERRIDE).
45	frc_core_maint	Force core maintenance mode (RAS LOGIC OVERRIDE).
46	frc_quies_state	Force the quiesce state machine to the Quiesce state (RAS LOGIC OVERRIDE).
47	frc_run_state	Force the quiesce state machine to the Run state (RAS LOGIC OVERRIDE).
48	inj_ftch_hng	Inject fetch hang to test hang-recovery logic.
49	inj_frc_rjct	Inject force reject hang to test hang-recovery logic.
50	inj_dsptch_hng	Inject dispatch hang to test hang-recovery logic.
51	inj_cmplt_hng	Inject completion hang to test hang-recovery logic.
52	core_run_tst	Core running test. Use to see if the core is running (clears bit 15 of the CoreRAS Status Register until a group completes). Note: Do not do this if in Maintenance Single Step mode (bit 54 of the CoreRAS Status Register must equal '0').

IBM PowerPC 970MP RISC Microprocessor

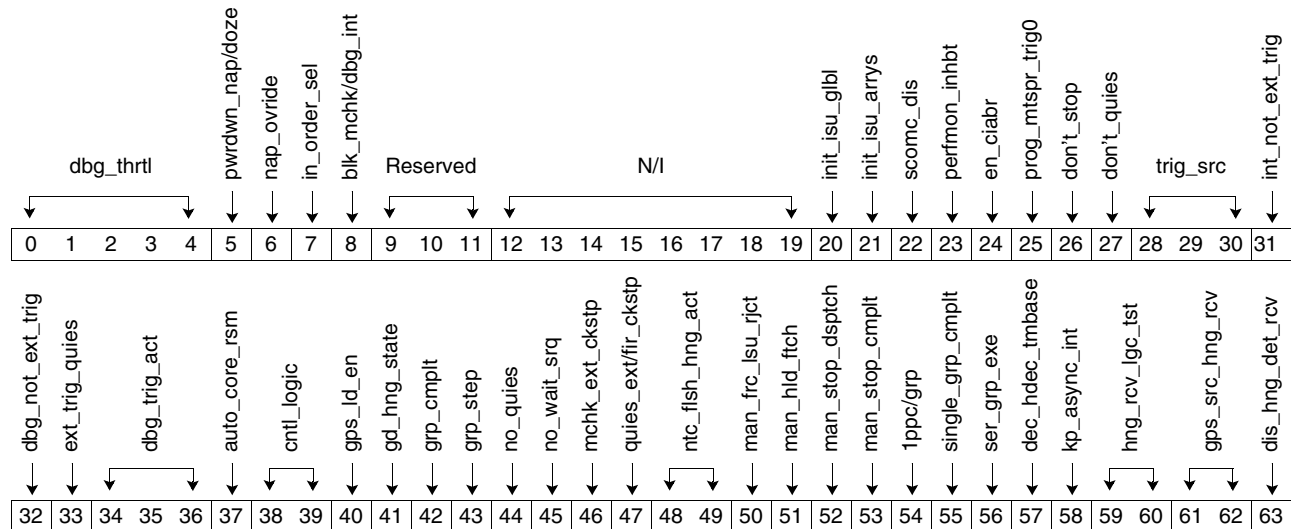
Bits	Field Name	Description
53	inj_ntc	Inject next to complete (NTC) for all instructions in the machine. Flush manually without the before and after waits. Note: Do not attempt if the processor is running.
54	inj_ntc+1	Inject next to complete plus one (NTC + 1) in all but the oldest instruction. Flush manually without the before and after waits. Note: Do not attempt if the processor is running.
55	clr_inst_step	Clear instruction stop due to checkstop. Also clears hang detection, hang history, and miscellaneous status latches.
56	N/I	Not implemented.
57	N/I	Not implemented.
58	inj_sreset_man	Inject <i>sreset</i> manually (no auto quiesce). The ISU can OR x'0100' with another interrupt vector if they occur simultaneously. Note: Do not attempt if the processor is running.
59	inj_mchk_man	Inject machine check manually (no auto quiesce). The ISU can OR x'0200' with another interrupt vector if they occur simultaneously. Note: Do not attempt if the processor is running.
Bit 15 of the CoreRAS Status Register will be cleared for bits 60:62 until a group completes (that is, until positive acknowledgment is received that the operation was successful).		
60	sreset_req	SRESET request. This causes the core to first quiesce, and then vector to x'0100' and start instructions. If quiesce is unsuccessful, a soft reset will not occur. A special attention will be sent to the service processor indicating a timeout on a quiesce request (assuming hang pulses are activated). After the request, check that bit 15 of the CoreRAS Status Register equals '1' to ensure that the soft reset was successful (core has started).
61	inst_step	Instruction step (core step). Notes: - The core must be in Maintenance mode (quiesced) (see bit 12 of the <i>CoreRAS Status Register</i> on page 311). - Single Group Completion mode is active, which means a core flush and refetch occurs between each step. This allows the next instruction address (NIA) to be changed (via scan) between steps if wanted. - Bit 43 of the CoreRAS Register determines the behavior. It is <i>not</i> determined by bit 54 of the CoreRAS Mode Register or bit 0 of Hardware Implementation Dependent Register 0 (HID0). 0 PowerPC instruction step (default). Completes more than one group if the PowerPC instruction is a microcoded, multi-group sequence. 1 Group step (for debug). Completes a single group or microcode group sequence. After the request, check that bit 15 of the CoreRAS Status Register equals '1' to ensure that step was successful (core has started).
62	inst_start	Instruction start (core resume). Notes: - Core must be in Maintenance mode (quiesced) (see bit 12 of the <i>CoreRAS Status Register</i> on page 311). - After the request, check that bit 15 of the CoreRAS Status Register equals '1' to ensure that start was successful (core has started).
63	inst_stop	Instruction stop (core stop). Note: Causes core quiesce, and leaves core in Maintenance mode.

CoreRAS Mode Register

Address x'021100'

Type RW

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0:4	dbg_thrtl	Debug throttle modes. Periodically alter instruction flow based on Core Debug Throttle Control. 00000 None. 01xxx Stop fetch (both IFU and LSU prefetch). 0x1xx Stop dispatch. 0xx1x Stop completion. 0xxx1 Force LSU reject (stops issue from ISU to LSU). 11xxx One PowerPC per group (can quiesce depending on bit 44). 1x1xx Single group completion (will quiesce to change). 1xx1x Serialized group issue (will quiesce to change). 1xxx1 Serialized group dispatch (does <i>not</i> quiesce to change).
5	pwrdown_nap/doze	Power down Nap/Doze mode enable override. (When set, ignores bits 8:9 of the Hardware Implementation Dependent Register 0 [HID0]) (see bit 6 for mode select). Note: To enter Power Down, software must either set MSR[POW] or use CoreRAS Control Register[40].
6	nap_ovride	Nap mode override selector. 0 Force Nap mode when bit 5 is set. 1 Force Doze mode when bit 5 is set.
7	in_order_sel	In-order issue select. 0 Serialized Group Execution, when selected, will serialize at dispatch. This is the most powerful form of serialization, and only allows one group in flight in the machine at a time. This includes branch and Condition Register (CR)-logical instructions. It also prevents groups from sharing ISU resources such as mappers and renames. 1 Serialized Group Execution, when selected, will serialize at issue. This does not include branches and CR-logical instructions and does not completely serialize the ISU.
8	blk_mchk/dbg_int	Blocks machine-check interrupt or debug interrupt injection based on debug triggers when one is already set in the Asynchronous Machine-Check Source Accumulation Register. This will prevent multiple triggers from causing a checkstop with machine-check enable (ME) equal to '0'. It allows the interrupt handler to ignore spurious triggers until it has a chance to clear the source register.

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
9:11	Reserved	Spare.
12:19	N/I	Not implemented.
20	init_isu_glbl	Initialize ISU global completion table.
21	init_isu_arrys	Initialize ISU SBM, SBX, and SFM arrays.
22	scomc_dis	SCOMC disable. This is a debug switch used to disable the core SCOM master accessible through the core SPR bus. RAS logic treats an MTSCOMC instruction in the core as a NOP.
23	perfmon_inhbt	Performance monitor inhibit (debug switch only; interrupts and multiplexer selects).
24	en_ciabr	Enable CIABR (bit 22 in Hardware Implementation Dependent Register 0 [HID0] override active).
25	prog_mtspr_trig0	Programmable MTSPR TRIG0 mode. 0 SPR 976 mtspr_data is <i>not</i> used to form TRIG1 and TRIG2. 1 SPR 976 mtspr_data(63) causes TRIG1, and mtspr_data(62) causes TRIG2. Note: All zeros in SPR 976 mtspr_data(0:63) always cause TRIG0.
26	don't_stop	Do not stop fetch, dispatch, and completion on checkstop. Set this bit when the chip is set to clock stop on a checkstop. Setting this bit prevents core scan rings from being corrupted when a checkstop occurs, because it takes several cycles to clock stop after a checkstop occurs.
27	don't_quies	Do not attempt to quiesce, and then machine check or soft reset, during core hang recovery. Note: This will cause a checkstop if the first two attempts are unsuccessful.

RAS trigger sources to debug logic

28:30	trig_src	000 No internal trigger selected. 001 External trigger causes internal trigger. 010 Decrementer interrupt causes internal trigger. 011 External interrupt causes internal trigger. 100 Machine-check interrupt causes internal trigger. 101 Problem-state hang detect causes internal trigger. 110 Bad-state hang detect causes internal trigger. 111 Quiesce causes internal trigger.
31	int_not_ext_trig	Internal trigger does <i>not</i> cause an external trigger.
32	dbg_not_ext_trig	Core DBG trigger does <i>not</i> cause an external trigger.

RAS activities based on debug trigger

33	ext_trig_quies	External trigger causes a core quiesce. Bit 37 determines behavior after the quiesce.
34:36	dbg_trig_act	Core DBG trigger causes: 000 No action selected. 001 Core quiesce (bit 37 determines behavior after the quiesce). 010 NTC flush (see bits 46:48 of the <i>Core Hang-Recovery Control Register</i> on page 314). 011 NTC + 1 flush (see bits 46:48 of the <i>Core Hang-Recovery Control Register</i> on page 314). 100 Machine-check interrupt (see bit 8 for mode to prevent checkstop). 101 Debug interrupt. 110 Enter Reduced Execution mode for a specified number of group completions (see bits 46:48 of the <i>Core Hang-Recovery Control Register</i> on page 314). 111 Pause (quiesce and hold prefetch) until core idle.
37	auto_core_rsm	Auto core resume on core quiesce due to debug trigger. When debug or an external trigger causes quiesce: 0 Quiesce, set Special ATTN, and enter Core Maintenance mode. 1 Auto-start core immediately after ISU completes the quiesce, which implicitly causes a flush. Special ATTN is not set, and the core resumes normal instruction execution.

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
RAS control logic modes		
38:39	cntl_logic	00 No hang-pulse induced random periods. 10 Enter Reduced Execution mode for a random period after every hang pulse (see bits 46:48 of the <i>Core Hang-Recovery Control Register</i> on page 314). 01 Quiesce for a random period after every hang pulse. 11 Undefined.
40	gps_ld_en	STS load mode enable. Throttle back core execution when the STS detects a memory subsystem hang.
41	gd_hng_state	Return to the Good Hang state immediately after meeting the completion criteria. Do <i>not</i> wait on hang pulse to leave Reduced Throughput mode.
42	grp_cmplt	Use any group completed (instead of PowerPC) to debug logic (trace trigger) and to indicate progress for the hang detection logic. 0 PowerPC group completed. 1 Any group completed (including intermediate multi-group sequences).
43	grp_step	Group step instead of PowerPC instruction step. Causes the instruction step <i>not</i> to enter a PowerPC mode automatically during Maintenance mode.
44	no_quies	Do not quiesce when changing a PowerPC mode.
45	no_wait_srq	Do not wait for the store reorder queue (SRQ) to be empty before auto-restarting after a quiesce due to a mode change, debug trigger, or when in single_group_completion mode.
46	mchk_ext_ckstp	Machine check on an external checkstop, required for logical partitioning (LPAR).
47	quies_ext/fir_ckstp	Quiesce on external or FIR checkstop (default is hold completion). Note: Causes Special ATTN to the service processor.
48:49	ntc_fish_hng_act	Action to perform for NTC flush hang recovery when the LSU is not safe (that is, the load miss queue [LMQ] is not empty). 00 Do not attempt flush. This action continues to hold completion, dispatch, and force reject at the same time. It eventually transitions to the next Hang-Recovery state if the LSU is not safe. 01 Stop dispatch and completion (sets the Service Processor Special Attention (SP-ATTN) Register). 10 Checkstop. 11 Do flush anyway. Caution: For debug only. Use the Unsafe NTC + 1 Mode (bit 17 of the Core Hang-Recovery Control Register) for NTC + 1 flushes.
Core toolbox -- hooks for the service processor		
50	man_frc_lsu_rjct	Manually force LSU reject.
51	man_hld_ftch	Manually hold fetch (both IFU and LSU).
52	man_stop_dsptch	Manually stop dispatch.
53	man_stop_cmplt	Manually stop completion.
Core toolbox -- behavior mode controls		
Note: The core <i>must</i> be quiesced before changing these fields. A corresponding HID0 change invokes the state machine to perform an automatic quiesce, mode change, and resume.		
54	1ppc/grp	One PowerPC per Group mode. When set, ORed with HID0[0]. This means that no more than one PowerPC instruction will be placed in a group. Some instructions are expanded into a multiple-group, microcoded sequence.
55	single_grp_cmplt	Single Group Completion mode. When set, ORed with HID0[1], which causes an automatic group step. This means that only one group (or microcoded group sequence) is allowed to complete at a time. A core quiesce (involving an instruction flush and refetch) occurs between the completion of each group. Subsequent groups of instructions will not be allowed to complete, but <i>will</i> be allowed to execute at the same time unless mode (bit 56) equals '1'.

IBM PowerPC 970MP RISC Microprocessor

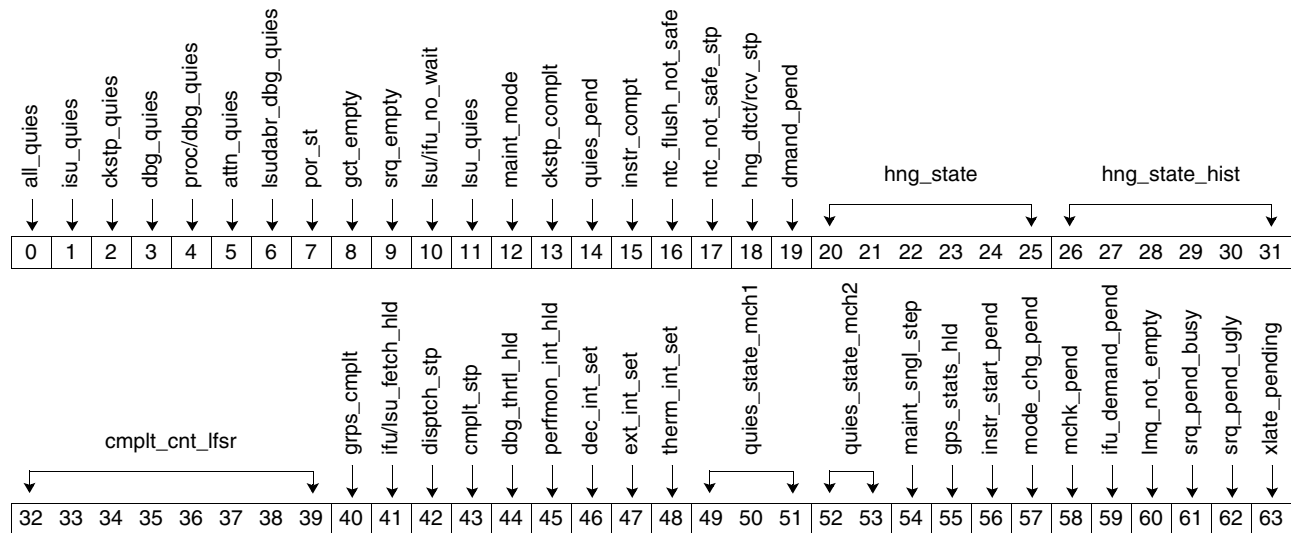
Bits	Field Name	Description
56	ser_grp_exe	Serialized Group Execution mode (in-order dispatch OR issue of groups). • ORed with HID0[3] when inorder_issue_select equals '0'. • ORed with HID0[16] when inorder_issue_select equals '1'. When set, this means that a group must be completed before the next one can be issued or dispatched, effectively serializing execution of groups in the processor. Subsequent groups of instructions are not allowed to execute at the same time (see inorder_issue_select, bit 7 of this register, for more details).
Core Maintenance Modes		
57	dec_hdec_tmbase	Run decremter (DEC), hypervisor decremter (HDEC), and time base while instruction stepping (take ISU quiesced into account during maint_mode). This also allows DEC and HDEC interrupts, and only these interrupts, while instruction stepping. Note: DEC, HDEC, and time-base stopping only takes effect if bit 18 of HID0 equals '0'.
58	kp_async_int	Keep asynchronous interrupts during quiesce or instruction stepping. Asynchronous interrupts include decremter, external, external machine-check, and performance monitor interrupts. If kept, the interrupts present only after maintenance activity completes.
59:60	hng_rcv_lgc_tst	Hang-Recovery Logic Test modes. 00 None 01 Allow Problem Hang Recovery to break test hang. 10 Allow Bad Hang Recovery to break test hang. 11 Allow Machine-Check Hang Recovery to break test hang.
61:62	gps_src_hng_rcv	STS Source Core Hang-Recovery modes. 00 None (only attempt hang recovery if nothing is pending to the STS). 01 Special ATTN if ambiguous source or if either IFU or LSU indicates STS pending transactions. 10 Attempt core hang recovery if ambiguous source; there are possibly STS pending transactions. 11 Always attempt a core hang recovery, even if ambiguous source or either IFU or LSU indicates STS pending transactions. Note: To cause Special ATTN for core-source hang detect (no STS pending), use bit 55 of the Core Hang-Recovery Control Register.
63	dis_hng_det_rcv	Disable hang detection and recovery based on hang pulse.

CoreRAS Status Register

Address x'021200'

Type RO

Reset All zeros except bits 7, 12, 20, and 26.



Bits	Field Name	Description
0	all_quies	Entire core is quiesced (that is, status[1] is equivalent to an AND of status[8:10]).
1	isu_quies	The ISU is quiesced for the reasons listed below.
2	ckstp_quies	A checkstop caused the quiesce request.
3	dbg_quies	Debug logic made the quiesce request to the ISU.
4	proc/dbg_quies	The ISU completed quiesce as requested by the service processor or debug logic.
5	attn_quies	An ATTN instruction quiesced the ISU.
6	lsudabr_dbg_quies	The LSU DABR Debug mode quiesced the ISU. Note: CIABR will <i>not</i> set this bit, but bit 2 of the Service Processor Special Attention (SP-ATTN) Register will.
7	por_st	POR state (scan flush indicator). Forces stop dispatch. When set, indicates a quiesce <i>must</i> be performed before starting the core.
8	gct_empty	The ISU global completion table (GCT) is empty (quiesce indication).
9	srq_empty	The ISU SRQ is empty (quiesce indication).
10	lsu/ifu_no_wait	The LSU and IFU are <i>not</i> waiting for an STS transaction (NOR of the <i>not-held</i> but masked status sourcing bits [59:63]).
11	lsu_quies	The LSU is quiesced. - The LMQ is idle (there are no outstanding load misses, including speculative loads). - The SRQ has no stores past completion. The service processor should poll this bit to verify that both the STS and core are quiesced.
1. "Ugly" operations are unsafe instructions that are in flight, so that the machine state is not clean.		

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description														
12	maint_mode	The core is in Maintenance mode; the ISU is quiesced or is being stepped. - Enter and stay in Maintenance mode when ISU quiesces (including during instruction stepping). - Leave Maintenance mode when core resume, soft reset, or external machine check occurs (see bit 59 of the <i>CoreRAS Mode Register</i> on page 307).														
13	ckstp_complt	A checkstop stops completion.														
14	quies_pend	A quiesce request is pending.														
15	instr_complt	An instruction (or group) completed since the last maintenance operation (that is, an instruction step, start, or soft reset). For a core running test, first use bit 55 of the CoreRAS Control (Pulsed) Register [x'021001']).														
16	ntc_flush_not_safe	An NTC flush is not safe and caused a checkstop (core HANG).														
17	ntc_not_safe_stp	An NTC is not safe and caused a Stop (ATTN).														
18	hng_dtct/rcv_stp	A hang detect/recovery caused a Stop (ATTN) on a Problem, Bad, or STS hang detect as programmed.														
19	dmand_pend	A Demand Instruction fetch has been pending since the last hang pulse.														
20:25	hng_state	Hang state <table><tr><th>Bit</th><th>State</th></tr><tr><td>20</td><td>Good</td></tr><tr><td>21</td><td>Problem</td></tr><tr><td>22</td><td>Bad</td></tr><tr><td>23</td><td>Quiesce/Machine Check</td></tr><tr><td>24</td><td>Wait</td></tr><tr><td>25</td><td>Fail</td></tr></table>	Bit	State	20	Good	21	Problem	22	Bad	23	Quiesce/Machine Check	24	Wait	25	Fail
Bit	State															
20	Good															
21	Problem															
22	Bad															
23	Quiesce/Machine Check															
24	Wait															
25	Fail															
26:31	hng_state_hist	Hang state history <table><tr><th>Bit</th><th>State</th></tr><tr><td>26</td><td>Good</td></tr><tr><td>27</td><td>Problem</td></tr><tr><td>28</td><td>Bad</td></tr><tr><td>29</td><td>Quiesce/Machine Check</td></tr><tr><td>30</td><td>Wait</td></tr><tr><td>31</td><td>Fail</td></tr></table>	Bit	State	26	Good	27	Problem	28	Bad	29	Quiesce/Machine Check	30	Wait	31	Fail
Bit	State															
26	Good															
27	Problem															
28	Bad															
29	Quiesce/Machine Check															
30	Wait															
31	Fail															
32:39	cmplt_cnt_lfsr	An 8-bit completion count LFSR. Matched against the Completion Count Limit in the Core Hang-Recovery Control Register (see page 314).														
40	grps_complt	Set to one, once the programmed number of groups complete after Hang state. This bit clears on return to the Good state.														
41	ifu/lsu_fetch_hld	The CoreRAS logic is holding Fetch on both the IFU and LSU.														
42	disptch_stp	The CoreRAS logic has stopped Dispatch.														
43	cmplt_stp	The CoreRAS logic has stopped Completion.														
44	dbg_thrtl_hld	Work (instruction completion) is being held due debug throttle hooks to CoreRAS.														
45	perfmon_int_hld	Performance monitor interrupt is being held pending completion of a core maintenance operation.														
46	dec_int_set	Decrementer interrupt is set.														
47	ext_int_set	External interrupt is set.														
48	therm_int_set	Thermal interrupt is set.														
49:51	quies_state_mch1	Quiesce state machine <table><tr><th>Bit</th><th>State</th></tr><tr><td>49</td><td>Quiesced</td></tr><tr><td>50</td><td>Running</td></tr><tr><td>51</td><td>Asynchronous interrupt</td></tr></table>	Bit	State	49	Quiesced	50	Running	51	Asynchronous interrupt						
Bit	State															
49	Quiesced															
50	Running															
51	Asynchronous interrupt															

1. "Ugly" operations are unsafe instructions that are in flight, so that the machine state is not clean.

IBM PowerPC 970MP RISC Microprocessor

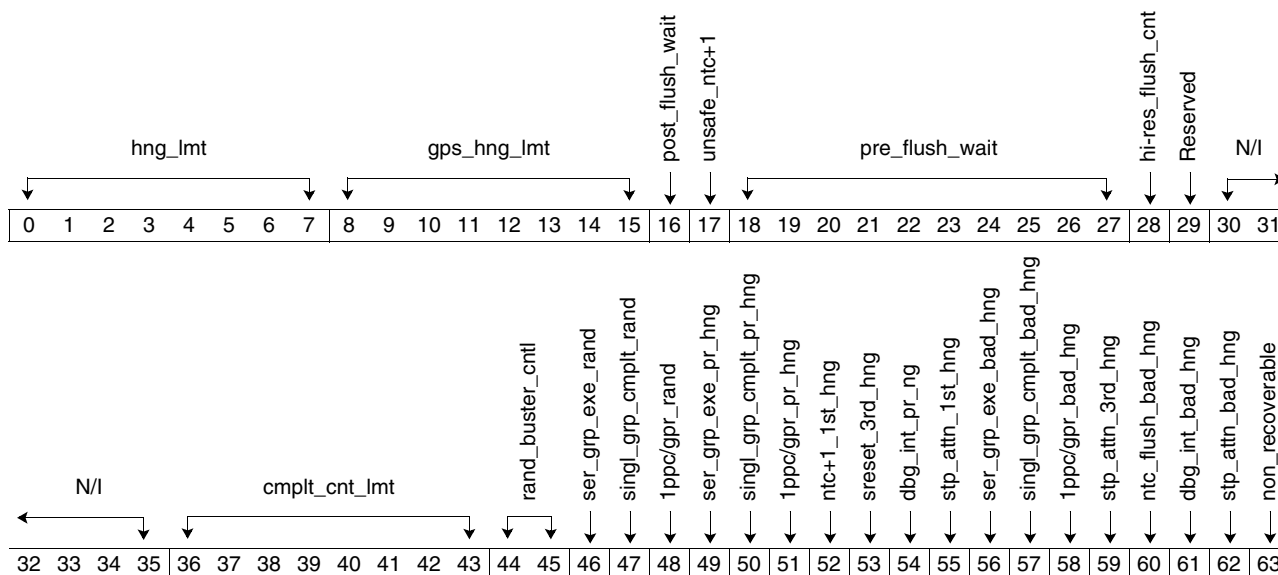
Bits	Field Name	Description						
52:53	quies_state_mch2	Quiesce state machine <table><tr><th>Bit</th><th>State</th></tr><tr><td>52</td><td>Change mode</td></tr><tr><td>53</td><td>Pause</td></tr></table>	Bit	State	52	Change mode	53	Pause
Bit	State							
52	Change mode							
53	Pause							
54	maint_sngl_step	Maintenance Single Stepping mode is in effect. The core is in Serial Dispatch mode (and automatically in a PowerPC mode if bit 43 of the CoreRAS Mode Register equals '0').						
55	gps_stats_hld	The STS status (bits 59:63 of this register) is being held due to a core-STS hang detect for debug/fault isolation.						
56	instr_start_pend	Instruction start (core resume) pending, or an instruction (or group) step pending.						
57	mode_chg_pend	Mode change pending.						
58	mchk_pend	Machine check pending.						
Note: Bits 59 and 60:63 are maskable by scan latches (TDR.IFU_STAT_DIS and TDR.LSU_STAT_DIS). Normally, they follow the masked IFU/LSU signals. However, they are held when a core-STS hang detect occurs to aid in debug/fault isolation (indicated by bit 55). Use bit 41 of the CoreRAS Control (Pulsed) Register to break the hold after an STS hang detect occurs. The hold is also broken during hang recovery as soon as a single group completes. However, it can take more than one group complete to deem hang recovery successful. In this case, if the processor is still hung, these status indicators follow the current status.								
59	ifu_demand_pend	The IFU has outstanding demand instruction fetches to the STS.						
60	lmq_not_empty	The LSU has outstanding loads to the STS (this also indicates that it is <i>not</i> safe to NTC flush).						
61	srq_pend_busy	The LSU has entries in the SRQ being held due to an STS busy on a store port.						
62	srq_pend_ugly	The LSU has outstanding ugly ¹ operations to the STS waiting for response. That is, an instruction cache block invalidate (ICBI), store word conditional (STWCX), or SYNC instruction is outstanding to the STS.						
63	xlate_pending	The LSU has an outstanding instruction or data-side table walk to the STS.						
1. "Ugly" operations are unsafe instructions that are in flight, so that the machine state is not clean.								

IBM PowerPC 970MP RISC Microprocessor
Core Hang-Recovery Control Register

Address x'021301'

Type RW

Reset 0:7 Reset to x'9F' = 3 hang pulses
 8:15 Reset to x'2D' = 100 hang pulses
 16:17 Reset to 00
 18:27 Reset to x'1FF' = 1 × 255 default
 36:43 Reset to x'FE' = 255 (maximum)
 44:63 Reset to all zeros during POR, except bits 52 and 60.



Bits	Field Name	Description
0:7	hng_lmt	Core hang limit (8-bit LFSR match value). This is the number of hang pulses without a PowerPC instruction (or group) completion used to detect a core hang. It is used when the <i>lsu_safe</i> signal indicates there are no outstanding load/store (LD/ST) instructions to the STS. This should be set to a minimum of 3 (LFSR code value for 3 is x'9F').
8:15	gps_hng_lmt	Core-STs hang limit (8-bit LFSR match value). This is the number of hang pulses without a PowerPC instruction (or group) completion used to detect a core-STs hang. It is used when <i>lsu_safe</i> indicates outstanding load/store (LD/ST) instructions to the STS. This must be set to a value greater than the core hang limit indicated in bits 0:7 of this register.
16	post_flush_wait	After flush wait time. After flush control. 0 Wait 255 cycles after attempting hang-recovery flush. 1 Use the before flush wait time, defined in bits 18:27, as the after flush wait time as well.
17	unsafe_ntc+1	Unsafe NTC + 1 mode. 0 Do not attempt NTC + 1 flushes if not safe. 1 Attempt NTC + 1 even if the LSU indicates not safe (the LMQ is not empty).

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
18:27	pre_flush_wait	Before flush wait time. Wait time ($x \times 255$ cycles) before hang-recovery flush (10-bit LFSR match value). Dispatch and completion (and LSU force reject for NTC) are stopped during this wait <i>before</i> attempting flush. 1FF $1 \times 255 = 255$ cycles (minimum wait). 3FE $1022 \times 255 = 260610$ cycles (maximum wait). Note: 3FF (count of zero) is invalid and will cause undefined results.
28	hi-res_flush_cnt	High-resolution flush counter (before and after wait times are single cycle instead of having a 255 multiplier).
29	Reserved	Spare.
30:35	N/I	Not implemented.
36:43	cmplt_cnt_lmt	Completion count limit (8-bit LFSR match value). This is the number of group completions to wait before returning to Good Hang state after a hang recovery is attempted.
Random pulse hang buster controls		
44:45	rand_buster_cntl	Random buster controls (use before and after wait times specified above). 00 Random pulse has no effect (disabled). 01 Random pulse injects NTC + 1 flush. 10 Random pulse injects NTC flush. Caution: A random NTC flush does not work in all cases. It should only be attempted in the bring-up lab. 11 Random pulse stalls instruction dispatch and completion with no flush (only waits for the before flush time). Note: Bit 17 of this register and bits 48:49 of the CoreRAS Mode Register determine the behavior of these flushes when LSU indicates not safe. The recommended setting is to not flush when LSU is not safe if any of these Random Busters are enabled.
Degraded mode during random period (either random pulse or hang pulse induced)		
Notes:		
- Bit 38 of the CoreRAS Mode Register must be set in order to enter these random periods. - These bits are also set until the programmed number of completions (above) have occurred after the flush on trigger. - inorder_issue_select (bit 7 of the CoreRAS Mode Register) affects the meaning of bits 46 and 49.		
46	ser_grp_exe_rand	Serialized Group Execution mode during random period (or after flush on trigger).
47	singl_grp_cmplt_rand	Single Group Completion mode during random period (or after flush on trigger).
48	1ppc/gpr_rand	One PowerPC per Group mode during random period (or after flush on trigger).
Degraded mode during first-level (problem) hang recovery (hang pulse based only)		
49	ser_grp_exe_pr_hng	Serialized Group Execution mode during problem hang recovery.
50	singl_grp_cmplt_pr_hng	Single Group Completion mode during problem hang recovery.
51	1ppc/gpr_pr_hng	One PowerPC per Group mode during problem hang recovery.
First-level (problem) hang state recovery actions (hang pulse based only)		
52	ntc+1_1st_hng	Attempt NTC + 1 flush for first hang detect (Problem Hang state). Default to '1'.
53	sreset_3rd_hng	Perform SRESET instead of machine check (MCHK) for third-level hang recovery, unless disabled by bit 27 of the CoreRAS Mode Register.
54	dbg_int_pr_ng	Cause maintenance (debug) interrupt to be taken after problem hang recovery.
55	stp_attn_1st_hng	Stop completion and cause special attention on first-level hang detect (sets the Service Processor Special Attention [SP-ATTN] Register).

IBM PowerPC 970MP RISC Microprocessor

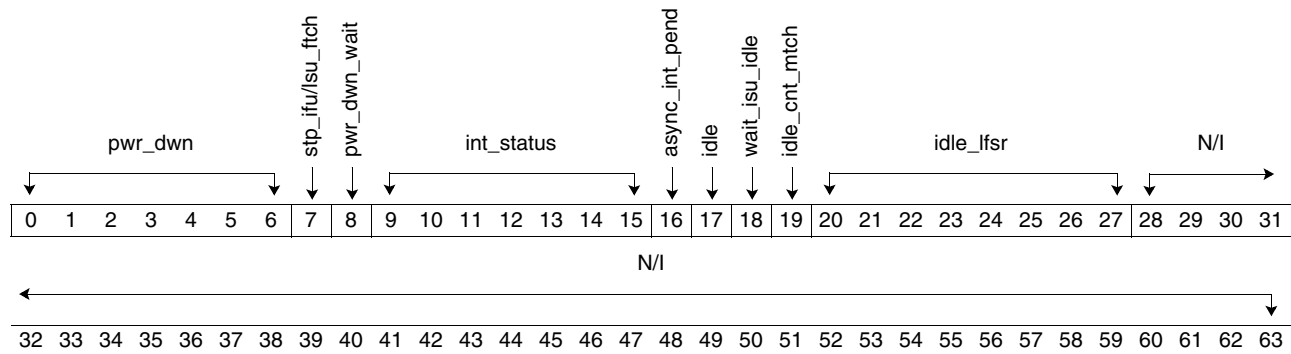
Bits	Field Name	Description
Degraded mode during second-level (bad) hang recovery		
56	ser_grp_exe_bad_hng	Serialized Group Execution mode during bad hang recovery.
57	singl_grp_cmplt_bad_hng	Single Group Completion mode during bad hang recovery.
58	1ppc/gpr_bad_hng	One PowerPC per Group mode during bad hang recovery.
Second-level (bad) Hang state recovery actions		
59	stp_attn_3rd_hng	Stop completion and cause special attention on third-level hang (sets the Service Processor Special Attention (SP-ATTN) Register). Note: Should also set bit 27 of the CoreRAS Mode Register.
60	ntc_flush_bad_hng	Attempt NTC flush on a bad hang. Defaults to '1'.
61	dbg_int_bad_hng	Causes a maintenance (debug) interrupt to be taken after a bad hang recovery.
62	stp_attn_bad_hng	Stop completion and cause a special attention on a bad hang (sets the Service Processor Special Attention (SP-ATTN) Register).
Nonrecoverable machine check for hang-recovery control		
63	non_recoverable	Make all hang-recovery MCHKs or SRESETs nonrecoverable (only has an effect if the quiesce was successful).

Core Power Down and Idle Status Register

Address x'021400'

Type RO

Reset Bits 0, 7, and 17 are initialized to '1'.



Bits	Field Name	Description
0:6	pwr_dwn	Power Down State Machine (0:6).
7	stp_ifu/lsu_ftch	IFU fetch and LSU prefetch stopped.
8	pwr_dwn_wait	Power down wait timer.
9:15	int_status	Interrupt status. <u>Bit</u> 9 Thermal interrupt active. 10 External interrupt active. 11 External machine check active or held, or machine check pending. 12 External <i>sreset</i> active or held, or <i>sreset</i> pending. 13 Performance monitor interrupt held. 14 Decrementer interrupt active. 15 Reserved.
16	async_int_pend	Asynchronous interrupt pending in the ISU.
17	idle	Core idle indication.
18	wait_isu_idle	Wait for idle from ISU.
19	idle_cnt_mtch	Idle count match.
20:27	idle_lfsr	Idle LFSR value.
28:63	N/I	Not implemented.

IBM PowerPC 970MP RISC Microprocessor
Service Processor Special Attention (SP-ATTN) Register
Service Processor And-Mask Register
Service Processor Or-Mask Register

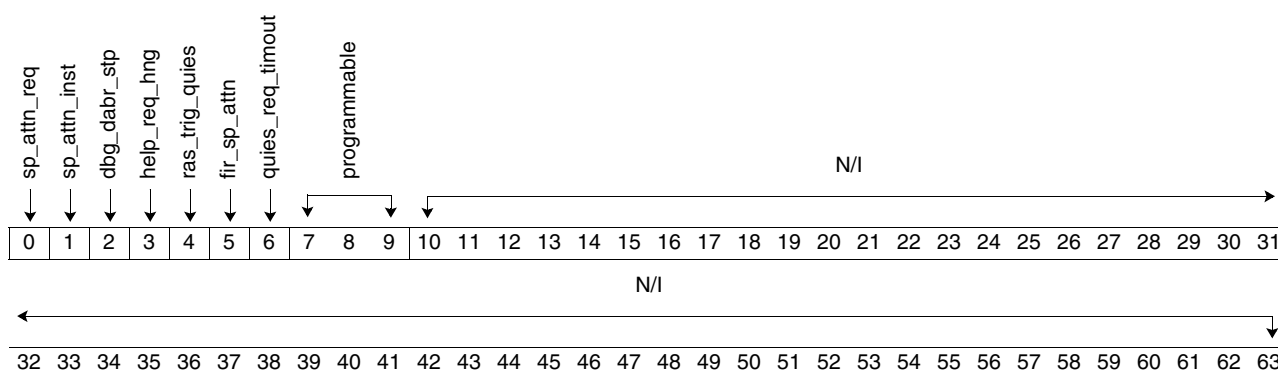
Any bit set in this register will drive a special attention to the service processor. The processor core can use this register to implement a mailbox function to communicate with the service processor.

Note: This is an accumulation register, and each source must be cleared to deassert the special attention. The AND mask should be used to clear bits in the register. The OR-mask should be used by software to set new bits in order to prevent an attention from another source from being missed.

Address x'022001'
 x'022100' (AND)
 x'022200' (OR)

Type RW
 WO (AND)
 WO (OR)

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0	sp_attn_req	Core-to-service-processor special-attention request.
1	sp_attn_inst	SP-ATTN instruction. The ISU is quiesced for maintenance due to a software breakpoint.
2	dbg_dabr_stp	A debug DABR soft stop (or ISU CIABR) caused the core quiesce.
3	help_req_hng	Core hang detected results from a CoreRAS help request to the service processor (see <i>CoreRAS Status Register</i> on page 311). Bit 41 of the CoreRAS Mode Register and bits 55 and 62 of the Core Hang-Recovery Control Register determine the mode. As a precaution, core dispatch and completion are stopped while this bit is set.
4	ras_trig_quies	RAS trigger caused core quiesce.
5	fir_sp_attn	Core FIR-induced Special ATTN.
6	quies_req_timeout	Time out on quiesce request (or pending change needing a quiesce). Two hang pulses have occurred and the request is still pending.
7:9	programmable	Programmable by the software or operating system.
10:63	N/I	Not implemented.

Asynchronous Machine-Check Source Register

Asynchronous And-Mask Register

Asynchronous Or-Mask Register

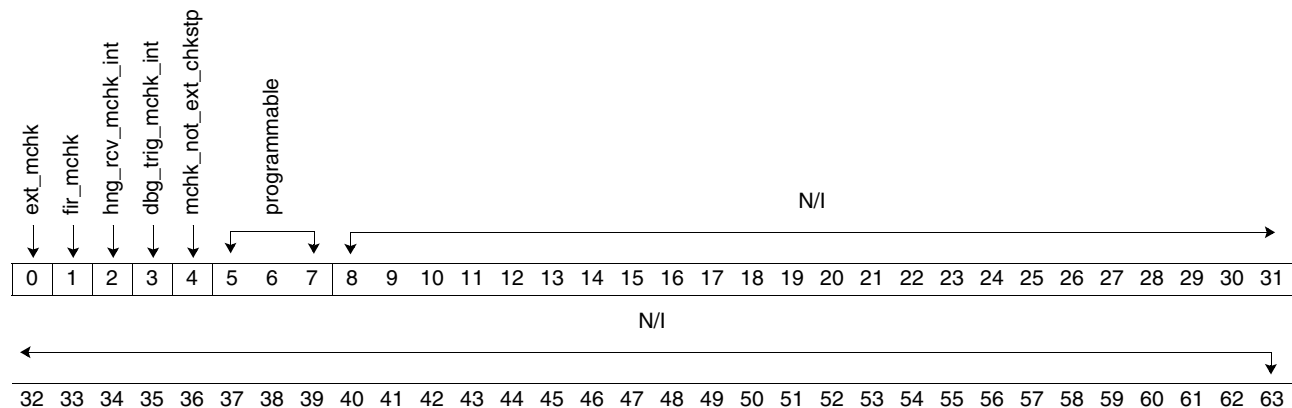
This register indicates the source of all asynchronous machine-check events. When any source event occurs, quiesce, then vector to x'0200', and restart instruction execution. If the quiesce is unsuccessful, a machine check will *not* occur. A special attention will be sent to the service processor indicating a timeout on the quiesce request (assuming hang pulses are activated). The service processor can then check bit 58 of the CoreRAS Status Register to see if the machine check was accepted or is still pending.

Note: This is a history accumulation register and must be cleared after each interrupt to absolutely determine the new source. The AND mask should be used to clear bits in the register. Software should use the OR mask to set new bits to prevent an interrupt from another source from being missed.

Address x'022601'
 x'022700' (AND)
 x'022800' (OR)

Type RW
 WO (AND)
 WO (OR)

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0	ext_mchk	External machine check from the chip C4 pin.
1	fir_mchk	FIR induced machine check (see <i>Section 12.3.4 Processor Core FIR Facilities (x'03[0:5]XXX')</i> on page 328).
2	hng_rcv_mchk_int	Hang-recovery machine-check interrupt attempt.
3	dbg_trig_mchk_int	Debug-logic trigger machine-check interrupt.
4	mchk_not_ext_chkstp	Machine check instead of external checkstop (see the <i>CoreRAS Mode Register</i> on page 307). Intended for the LPAR.
5:7	programmable	Programmable by software or the operating system.
8:63	N/I	Not implemented



IBM PowerPC 970MP RISC Microprocessor

12.3.2 Processor Core SPR SCOM Access (x'023XXX')

Instruction Address Breakpoint Register

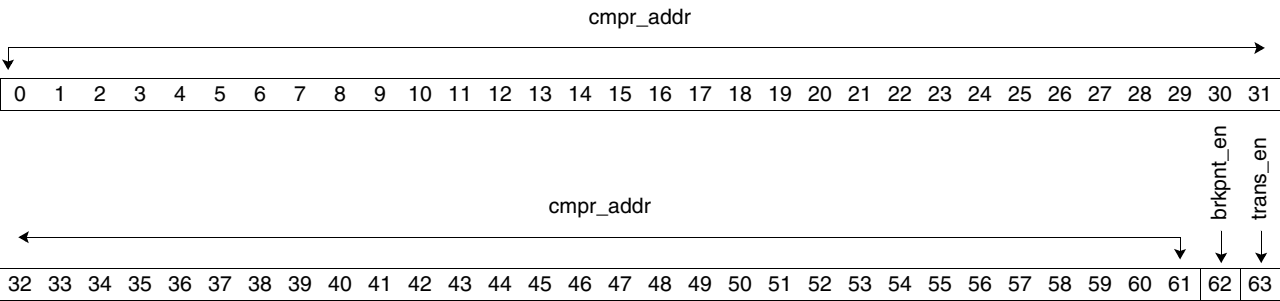
The Instruction Address Breakpoint Register (IABR) supports the address-breakpoint instruction. Writing this SCOM register *only* sets latches in the IFU, which can also be set by scan (GCP.PIFU.IFBT.IABR.L2[0:63]).

An IABR match occurs on the fetch of any instruction, even a speculative instruction. By default, the IABR matches if the address in the current fetch group is equal to or *after* the current IFAR. There can be multiple IABR matches for a single instruction before it is actually executed (or completed).

An additional mode bit accessible through scan only forces exact matches (GCP.PIFU.IFBC.ASSSCANON-LYNEW[7]).

Note: This register uses the IFU FETCH address, not the current instruction address (CIA) that is executing.

Address x'023000'
Type WO
Reset Reset to all zeros during POR.



Bits	Field Name	Description
0:61	cmpr_addr	Word address to be compared.
62	brkpt_en	Breakpoint enabled. An address match causes a trigger to the debug logic.
63	trans_en	Translation enabled. An IABR match is signaled only if MSR[IR] matches this bit.



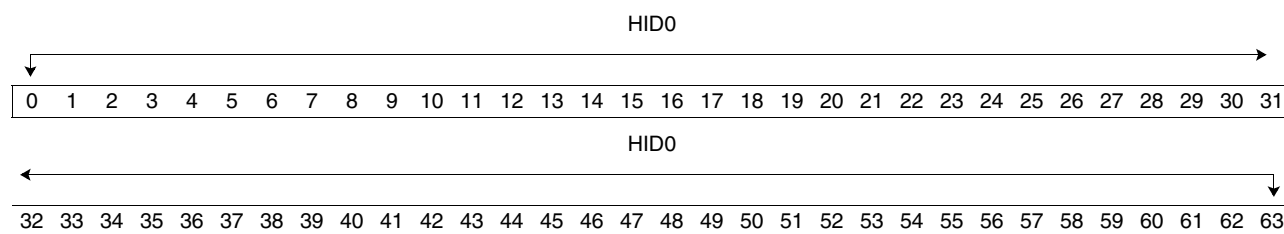
IBM PowerPC 970MP RISC Microprocessor

Hardware Implementation Dependent Register 0 (HID0)

Address x'023101'

Type RW

Reset Reset to all zeros during POR.



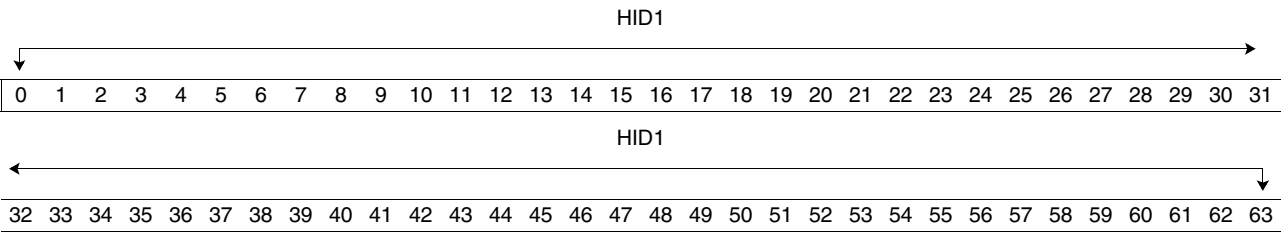
Bits	Field Name	Description
0:63	HID0	See the HID0 SPR definition in <i>Section 2.1.2.2 HID Registers (HID0, HID1, HID4, and HID5)</i> on page 54.



IBM PowerPC 970MP RISC Microprocessor

Hardware Implementation Dependent Register 1 (HID1)

Address x'023201'
Type RW
Reset Reset to all zeros during POR.



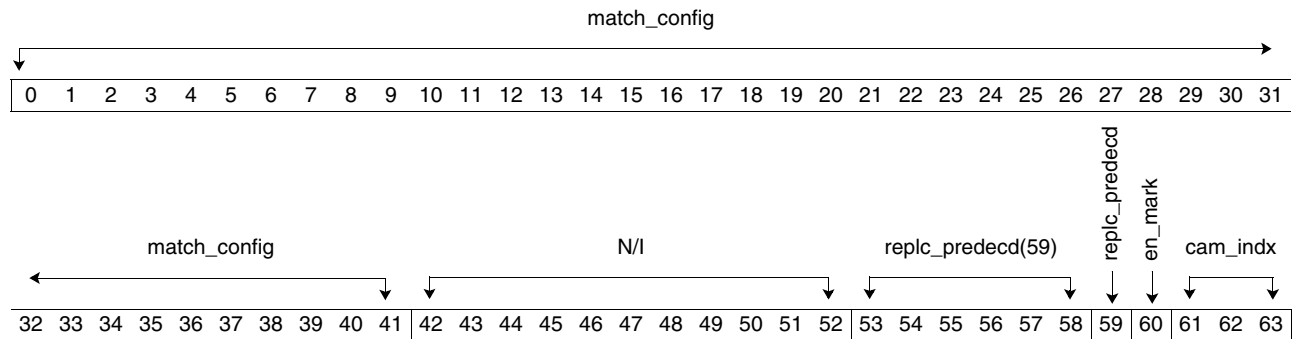
Bits	Field Name	Description
0:63	HID1	See the HID1 SPR definition in <i>Section 2.1.2.2 HID Registers (HID0, HID1, HID4, and HID5)</i> on page 54.

Instruction Match CAM (IMC) Register

Address x'023300'

Type WO

Reset N/A



Bits	Field Name	Description
0:41	match_config	Match configuration bits. See bits 61:63 of the current register for definitions of the v0 and v1 bits. v0,v1 00 No match (disable entire entry). v0,v1 01 Match a '1'. v0,v1 10 Match a '0'. v0,v1 11 Match always (zeros and ones are irrelevant).
42:52	N/I	Not implemented.
53:58	replc_predec(59)	Replacement predecode bits 0:5 (used only when bit 59 is set to '1'). Note: SPR cannot access this field.
59	replc_predec	Replace predecode bits 0:5 with bits 53:58 of this register. Note: SPR cannot access this field.
60	en_mark	Enable mark (for power controller (SPU) sampling or enabling hardware workarounds). Note: Setting this bit is incompatible with Single-Step Trace Enable (MSR[SE]) mode. It will cause bits 35, 36, and 42 of Save/Restore Register 1 (SRR1), as well as the Sampled Instruction Address Register (SIAR) and the Sampled Data Address Register (SDAR), to be undefined.
61:63	cam_indx	Content addressable memory (CAM) index (entry select). 000:101 IMC[0:20] v0 IMC[21:41] v1 Partial match (for entries 0-5) Instruction bits 0:5, 21:31 Problem (PR), floating point (FP) available, Vector/SIMD Multimedia eXtension (VMX) Available mode 110 IMC[0:35] v0 IMC[36:41] '000000' Full match (v0 for entry 6) Instruction bits 0:31 PR, FP, VMX mode 111 IMC[0:35] v1 IMC[36:41] '000000' Full match (entry 7 = v1 for entry 6) Instruction bits 0:31 PR, FP, VMX mode



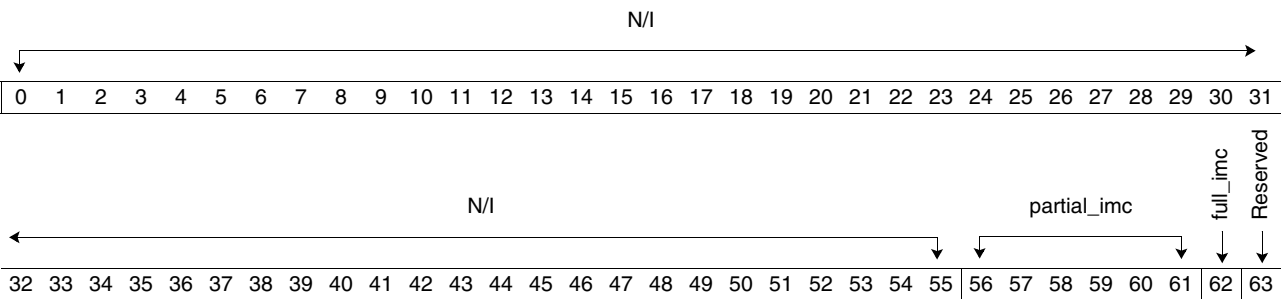
IBM PowerPC 970MP RISC Microprocessor

Patch Map (IMC Write Control Register)

Address x'023401'

Type RW

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0:55	N/I	Not implemented
56:61	partial_imc	Partial match IMC entry (0:5) is being used for debug. Do not allow SPR write to corresponding IMC entry.
62	full_imc	Full match IMC entries (6 AND 7) are used for debug. Do not allow SPR write to corresponding IMC entries.
63	Reserved	Reserved.



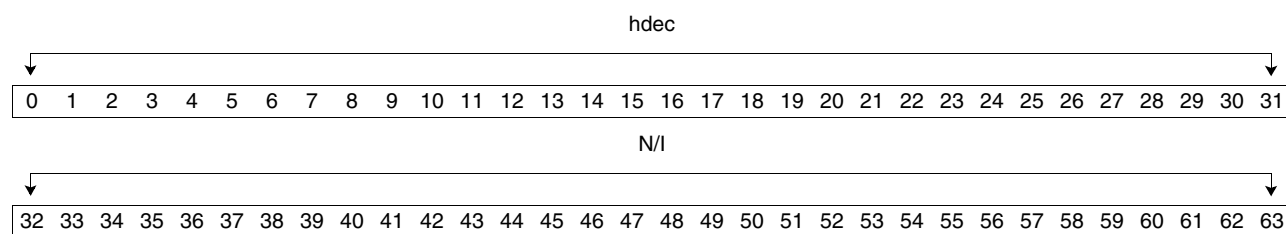
IBM PowerPC 970MP RISC Microprocessor

Hypervisor Decrementer

Address x'023500'

Type	RO
------	----

Reset	Reset to '7FFFFFFF' during POR.
-------	---------------------------------



Bits	Field Name	Description
0:32	hdec	Current HDEC value. Continuously runs with 8:1 divided core clocks.
32:63	N/I	Not implemented.



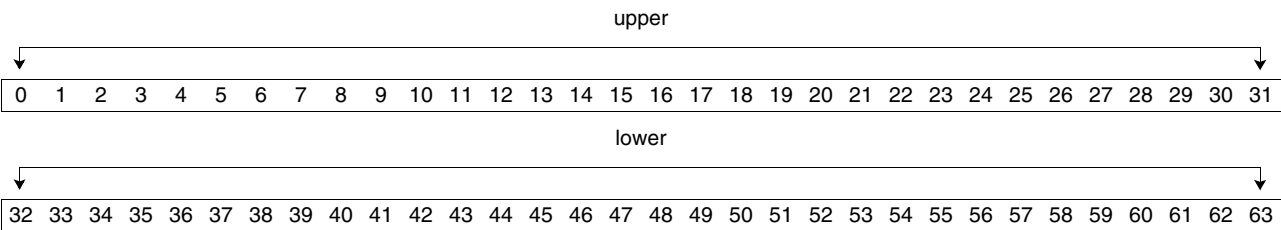
IBM PowerPC 970MP RISC Microprocessor

Time Base Register

Address x'023600'

Type RO

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0:32	upper	Upper time base. Runs with 8:1 divided core clocks or off an external nonuniform memory access (NUMA) oscillator.
32:63	lower	Lower time base. Runs with 8:1 divided core clocks or off an external NUMA oscillator.

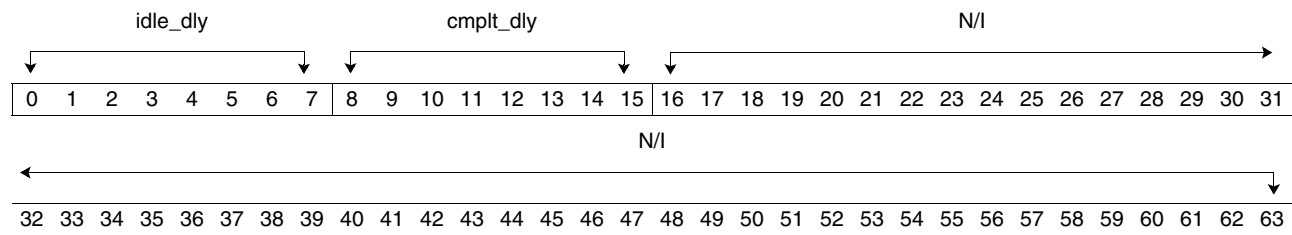
12.3.3 Processor Core Performance Monitor Sampling Control (x'02400X')

Performance Monitor Sampling Control Register

Address x'024001'

Type RW

Reset Reset to x'0414' during POR.



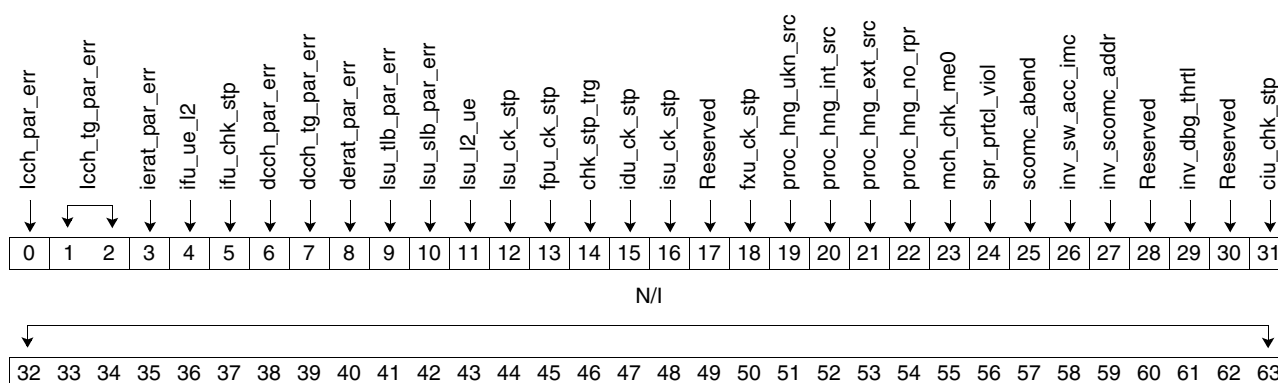
Bits	Field Name	Description
0:7	idle_dly	Idle delay.
8:15	cmplt_dly	Completion delay.
16:63	N/I	Not implemented.

IBM PowerPC 970MP RISC Microprocessor
12.3.4 Processor Core FIR Facilities (x'03[0:5]XXX')
Core Fault Isolation Register
Core And-Mask Register
Core Or-Mask Register

Address x'030001'
 x'031000' (AND)
 x'032000' (OR)

Type RW
 WO (And)
 WO (OR)

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0	lch_par_err	I-cache parity error. Treated as an L1 cache miss. Instructions fetched from L2 or the prefetch buffer, and first instruction bypassed to decode. Note: Even with a hard failure in the I-cache, forward progress can be made because instructions are bypassed from the L2 to decode. Because the I-cache can report multiple errors for a soft fail, the operating system will be called to flush the I-cache. (Service Element Threshold)
1:2	lch_tg_par_err	I-cache tag parity error (array 0, array 1). Treated as an L1 cache miss. Instructions fetched from L2 or the prefetch buffer, and first instruction bypassed to decode. Note: Even with a hard failure in the I-cache, forward progress can be made because instructions are bypassed from the L2 to decode. Because the I-cache can report multiple errors for a soft fail, the operating system will be called to flush the I-cache. (Service Processor Threshold)

Note: For processor hang detects, reference bits 26:31 of the CoreRAS Status Register to determine the extent of hang recovery that was required. In addition, for a processor hang with either an external or unknown source, the specific LSU and IFU status is available in bits 59:63 of the CoreRAS Status Register. Write to bits 55 and 41 respectively of the CoreRAS Control Register when clearing these processor hang FIR bits to clear the recovery and status information (if the hang was successfully recovered) for future diagnostics. Check bits 61:63 of the CoreRAS Mode Register to see what types of hang recovery are enabled (these settings affect how the Check-stop Enable mask should be set for these bits).

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
3	ierat_par_err	I-ERAT parity error. Treated as an I-ERAT miss. I-ERAT is reloaded from the TLB. Hardware does a flash invalidate of the I-ERAT. The hardware can tolerate one hard failure, but with slightly degraded performance because the error causes I-ERAT selection to go to the other processing unit where there will be a good I-ERAT. Note: A stuck fault on side 0 will hang the machine. (Service Processor Threshold)
4	ifu_ue_l2	The IFU fetched an uncorrected error (UE) from the L2. Corrupted data is discarded (not written into the I-cache). A speculative fetch that is not needed is discarded and instruction processing continues. An instruction that is required by the sequential execution model (SEM) causes a machine check interrupt. - SRR0 Address of corrupted instruction - SRR1 MSR bits and a bit indicating an instruction UE (PASSED ERROR)
5	ifu_chk_stp	The IFU detected a checkstop condition. Processor execution is halted. A system checkstop is raised that will halt the rest of the processors in the complex.
6	dcch_par_err	D-cache parity error. If the load was speculative and is eventually cancelled, instruction processing continues on the correct path (no interrupt is generated). If the load is required by the SEM, a machine check interrupt is generated reporting the error. - Common D-cache error recovery - SRR1 Bit indicating a LD/ST error - DSISR Bit indicating L1 data array parity (Machine Check Software Threshold)
7	dcch_tg_par_err	D-cache tag parity error. If the load was speculative and is eventually cancelled, instruction processing continues on the correct path (no interrupt is generated). If the load is required by the SEM, a machine check interrupt is generated reporting the error. - Common D-cache error recovery - SRR1 Bit indicating a LD/ST error - DSISR Bit indicating L1 address tag parity error
8	derat_par_err	D-ERAT parity error. If the access was speculative and is eventually cancelled, instruction processing continues on the correct path (no interrupt is generated). If the access is required by the SEM, a machine check interrupt is generated reporting the error. - Common D-cache error recovery - SRR1 Bit indicating a LD/ST error - DSISR Bit indicating D-ERAT parity (Machine Check Software Threshold)
9	lsu_tlb_par_err	LSU TLB parity error. If the access was speculative and is eventually cancelled, instruction processing continues on the correct path (no interrupt is generated), but the DERAT might be corrupted. If the access is required by the SEM, a machine check interrupt is generated reporting the error. - Common D-cache error recovery - SRR1 Bit indicating a LD/ST error - DSISR Bit indicating a TLB parity error (Machine Check Software Threshold)

Note: For processor hang detects, reference bits 26:31 of the CoreRAS Status Register to determine the extent of hang recovery that was required. In addition, for a processor hang with either an external or unknown source, the specific LSU and IFU status is available in bits 59:63 of the CoreRAS Status Register. Write to bits 55 and 41 respectively of the CoreRAS Control Register when clearing these processor hang FIR bits to clear the recovery and status information (if the hang was successfully recovered) for future diagnostics. Check bits 61:63 of the CoreRAS Mode Register to see what types of hang recovery are enabled (these settings affect how the Check-stop Enable mask should be set for these bits).

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
10	lsu_slb_par_err	LSU SLB parity error. If the access was speculative and is eventually cancelled, instruction processing continues on the correct path (no interrupt is generated), but the DERAT might be corrupted. If the access is required by the SEM, a machine check interrupt is generated reporting the error. • Common D-cache error recovery • SRR1 Bit indicating a LD/ST error • DSISR Bit indicating an SLB parity error If a multiple hit occurs, this bit is also set, because it cannot be determined whether there was a data parity error (because all the data is ORed together). (Machine Check Software Threshold)
11	lsu_l2_ue	LSU fetched an L2 UE. If the instruction is required by the SEM, a machine check interrupt is generated reporting the error. • SRR0 Address of the instruction to be executed • SRR1 Bit indicating a LD/ST error • DSISR Bit indicating the type of UE The I-ERAT might be corrupted. The TLB might be corrupted. (PASSED ERROR)
12	lsu_ck_stp	LSU detected checkstop condition. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.
13	fpu_ck_stp	FPU 0 or FPU 1 detected a checkstop condition. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.
14	chk_stp_trg	Checkstop on trigger (debug only). Programmable at the chip level. The trigger can be that processor execution is halted, a system checkstop, or even a clock stop for debug.
15	idu_ck_stp	IDU detected a checkstop condition. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.
16	isu_ck_stp	ISU detected a checkstop condition. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.
17	Reserved	Reserved (connected to additional ISU output).
18	fxu_ck_stp	FXU 0 or FXU 1 detected a checkstop condition. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.
19	proc_hng_ukn_src	Processor hang detected with an unknown source (could be either internal or external). The source is not known for sure to be internal or external to the core because the LSU indication of outstanding transactions to the STS is not stable. A processor hang recovery will be invoked in an attempt to clear the hang condition only if recovery on unsure source has been enabled.
20	proc_hng_int_src	Processor hang detected that was due to an internal source. Source is known to be internal to the core because neither the IFU or LSU have outstanding demand requests to the STS. Processor hang recovery has been invoked in an attempt to clear the hang condition only if recovery on the internal source has been enabled.

Note: For processor hang detects, reference bits 26:31 of the CoreRAS Status Register to determine the extent of hang recovery that was required. In addition, for a processor hang with either an external or unknown source, the specific LSU and IFU status is available in bits 59:63 of the CoreRAS Status Register. Write to bits 55 and 41 respectively of the CoreRAS Control Register when clearing these processor hang FIR bits to clear the recovery and status information (if the hang was successfully recovered) for future diagnostics. Check bits 61:63 of the CoreRAS Mode Register to see what types of hang recovery are enabled (these settings affect how the Checkstop Enable mask should be set for these bits).

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
21	proc_hng_ext_src	Processor hang detected that was due to an external source. Source is external to the core because the LSU or IFU indicates that a demand request to the STS is outstanding. Processor hang recovery has been invoked in an attempt to clear the hang condition only if recovery on the external source has been enabled. No field replaceable unit (FRU) call is possible; diagnostics should call for the next level of support.
22	proc_hng_no_rpr	Processor hung beyond repair. Core hang recovery failed. The processor is not making forward progress after all attempts at hang recovery. A system checkstop is raised, which will halt the rest of the processors in the complex. No FRU call is possible; diagnostics should call for the next level of support.
23	mch_chk_me0	Machine check and MSR[ME] equals '0'. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex. This is a latent checkstop. In other words, another error in the system caused a machine check. If this error is on by itself, diagnostics should call out the run time abstraction software (RTAS) or System Licensed Internal Code (SLIC).
24	spr_prctl_viol	Core SPR bus has violated protocol or SCOMC arbiter error. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex. This is a software error. Diagnostics should call out the RTAS or SLIC.
25	scomc_abend	SCOMC ABEND. Either an SCOM reset was issued through JTAG or a core hang recovery was activated while an SCOMC request was active. This error is recoverable.
26	inv_sw_acc_imc	Invalid software access to IMC. The software, probably the performance monitor code, tried to write an IMC entry that was being used for debug (force only) according the patch map. The state of the machine is not altered.
27	inv_scomc_addr	Invalid SCOMC address. The address used by SCOMC was not accepted by any of the SCOM satellites. This error is recoverable. This is a software error. Diagnostics should call out the RTAS or SLIC.
28	Reserved	Spare. System checkstop.
29	inv_dbg_thrtl	Service processor attempted an invalid debug throttle setting. Command is ignored.
30	Reserved	Spare. System checkstop.
31	ciu_chk_stp	CIU detected checkstop condition. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.
32:63	N/I	Not implemented.

Note: For processor hang detects, reference bits 26:31 of the CoreRAS Status Register to determine the extent of hang recovery that was required. In addition, for a processor hang with either an external or unknown source, the specific LSU and IFU status is available in bits 59:63 of the CoreRAS Status Register. Write to bits 55 and 41 respectively of the CoreRAS Control Register when clearing these processor hang FIR bits to clear the recovery and status information (if the hang was successfully recovered) for future diagnostics. Check bits 61:63 of the CoreRAS Mode Register to see what types of hang recovery are enabled (these settings affect how the Checkstop Enable mask should be set for these bits).



IBM PowerPC 970MP RISC Microprocessor

Core Fault Isolation Mask Register

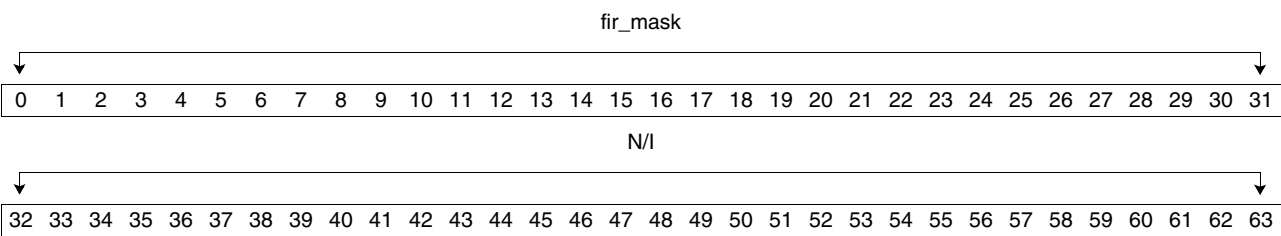
Core And-Mask Register

Core Or-Mask Register

Address x'030400'
 x'031401' (AND)
 x'032401' (OR)

Type RW
 WO (AND)
 WO (OR)

Reset Reset to all ones during POR.



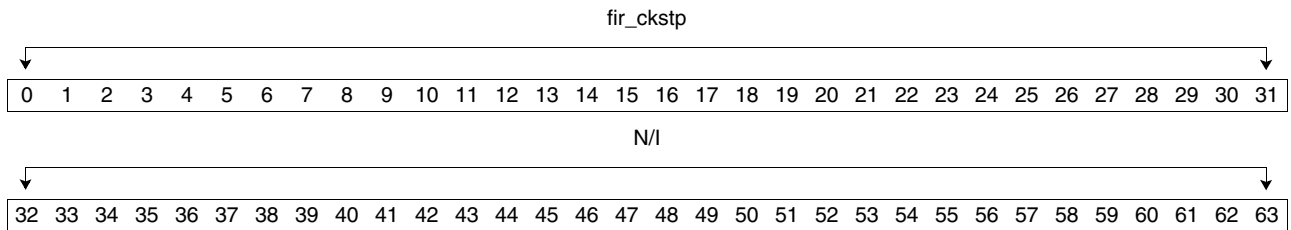
Bits	Field Name	Description
0:31	fir_mask	Mask 1 FIR bit masked off.
32:63	N/I	Not implemented.

Core Checkstop Enable Registers

Address x'030800'

Type RW

Reset Reset to all zeros during POR.



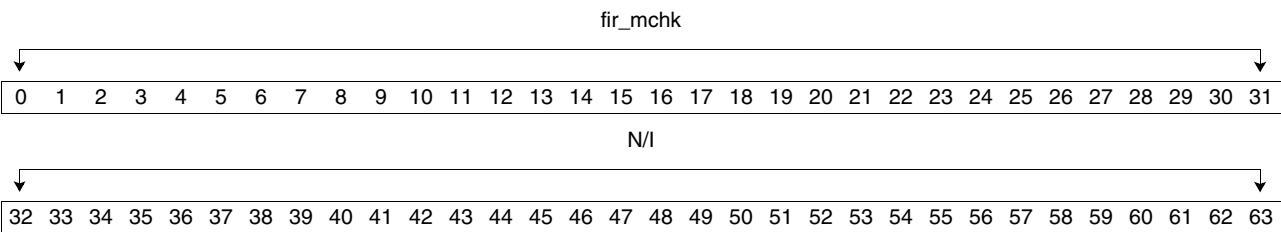
Bits	Field Name	Description
0:31	fir_ckstp	Causes a checkstop to occur when the corresponding FIR bit is set.
32:63	N/I	Not implemented.



IBM PowerPC 970MP RISC Microprocessor

Core Machine-Check Enable Register

Address x'030901'
Type RW
Reset Reset to all zeros during POR.



Bits	Field Name	Description
0:31	fir_mchk	Causes a machine check to occur when the corresponding FIR bit is set.
32:63	N/I	Not implemented.

12.3.5 Instruction Mark Configuration (x'03600X')

Instruction Mark Configuration Register

Address x'036001'

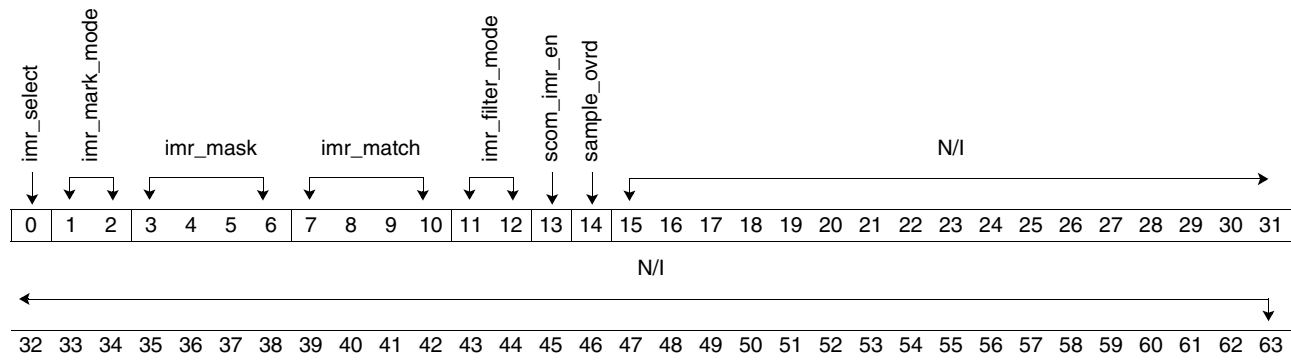
Type RW

Reset Reset to nonmarking mode during POR.

imr_mask '1111'

imr_select '1'

All other bits are zero at POR.



Bits	Field Name	Description
0	imr_select	Determine stage-1 eligibility from: 0 Predecode match (uses imr_match and imr_mask, bits 3:10 below). 1 Instruction match (IMR bit from IMC match).
1:2	imr_mark_mode	Chooses which instructions are stage-2 eligible for marking based on: 00 All stage-1, eligible internal operations (IOPs). 01 Only stage-1, eligible IOPs that resulted from microcode expansion. 10 Only one IOP per PowerPC instruction regardless of stage-1 eligibility. 11 First IOP that goes to the LSU for every PowerPC LD/ST instruction regardless of stage-1 eligibility.
3:6	imr_mask	Value ANDed with predecode before performing the predecode match.
7:10	imr_match	Value used to perform an exact compare against the masked 4-bit predecode field, used in determining stage 1 eligibility. To predecode, match <i>all</i> IOPs; set imr_mask to '0000' and imr_match to '0000'.
11:12	imr_filter_mode	Picks which stage 2, eligible IOPs are sampled as a marked group in the machine. 00 Sample all stage-2 eligible IOPs. 01 Sample only the first stage-2 eligible IOP in each group. 10 Randomly sample from all stage-2 eligible IOPs. 11 Sample only the first randomly picked stage-2 eligible IOP in each group. Note: Used for both Performance Monitor and Debug modes.
13	scom_imr_en	SCOM IMR enable. Use the first eleven bits of this register instead of performance monitor selects from the Monitor Mode Control Register 2 (MMCR2).

Usage Note: To use the IMC facility to set marks for debug triggering based on instruction match, set this register to "80060000 00000000" after configuring the Instruction Match CAM (IMC) Register (x'023300').

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
14	sample_ovrd	Sample override. Overrides the active performance monitor “ <i>ok_to_sample</i> ” indication. Enables the above marking modes for use in debug.
15:63	N/I	Not implemented.

Usage Note: To use the IMC facility to set marks for debug triggering based on instruction match, set this register to “80060000 00000000” after configuring the Instruction Match CAM (IMC) Register (x'023300').

12.4 Storage Subsystem SCOM Register Definition

12.4.1 L2 SCOM Register Definition

L2 Fault Isolation Register

L2 Fault Isolation And-Mask Register

L2 Fault Isolation Or-Mask Register

L2 Fault Isolation Checkstop Register

L2 Error Mask Register

L2 Error And-Mask Register

L2 Error Or-Mask Register

L2 Checkstop Enable

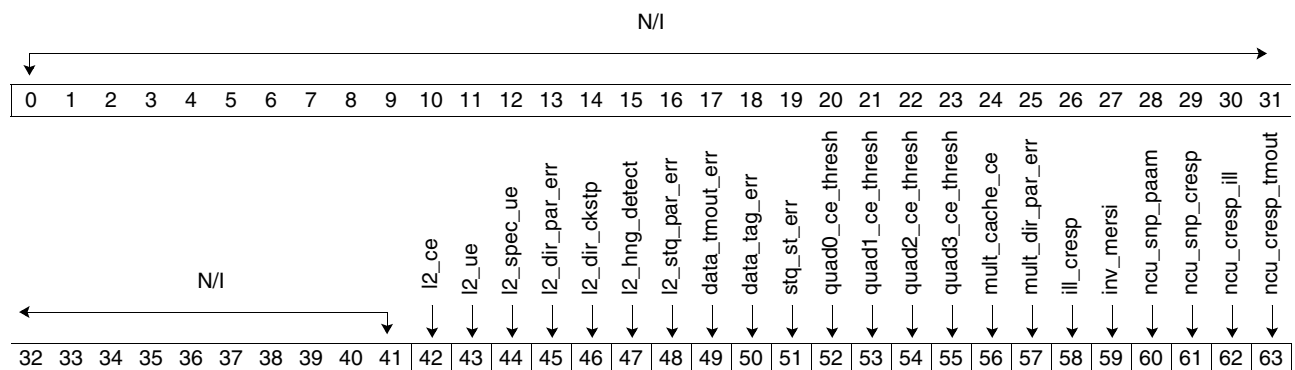
FIR Address x'040000'
 x'041001' (AND)
 x'042001' (OR)

Error Mask Address x'040401'
 x'041400' (AND)
 x'042400' (OR)

Checkstop Enable x'040801'
 Address

Type RW (FIR)
 WO (And)
 WO (OR)
 WR (Checkstop)

Reset Reset to all zeros during POR.



Bits	Field Name	Description	Default Mask Settings	
			Error Mask	Checkstop Enable
0:41	N/I	Not implemented	0	0

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description	Default Mask Settings	
			Error Mask	Checkstop Enable
42	l2_ce	L2-cache correctable data error (CE). - L2 CE on processor request (instruction or data). Data is corrected before forwarding to the processor. Corrected data is written back to the L2 cache. - L2 CE on castout (flush, read with intent to modify [RWITM], or victim of a replacement). Data is corrected before forwarding to the BIU. The threshold equals 32.	0	0
43	l2_ue	L2-cache uncorrectable error (UE). - A UE is detected in the L2-cache response to a processor load request. The UE response is sent to the requesting core. - A store hits in the L2 line that contains a UE. The store is merged into the line (timing does not permit discarding the store). Write a UE error checking and correction (ECC) to distinguish between a passed error from another source and a local UE that has been altered. - Uncorrectable error detected in the L2 cache in response to a cast out operation. A special UE (SUE) is sent to memory.	0	1
44	l2_spec_ue	L2-cache special UE. Same as bit 43, cache UE (passed error).	1	0
45	l2_dir_par_err	L2-directory parity error. The failing directory is refreshed with the contents of the other directory. The threshold equals 32.	0	0
46	l2_dir_ckstp	L2-directory checkstop. L2-directory parity error is detected on both halves of the directory. System checkstop.	0	1
47	l2_hng_detect	L2 hang detect. No response to memory read request. The same action is taken regardless of whether some other component acknowledged the request and then never returned the data, or no component on the fabric acknowledged the request. System checkstop.	0	1
48	l2_stq_par_err	L2 store-queue parity error. Any of the four store queues has a parity error. System checkstop.	0	1
49	data_tmout_err	Read/claim (RC) machine data timeout error. This is an internal control error. System checkstop.	0	1
50	data_tag_err	Non-cacheable control or RC machine data tag error. Either the non-cacheable control (ncctl) or the RC final state machine (rcfsm0-3) detected a valid data tag match when they were not expecting data. This is an internal control error. System checkstop.	0	1
51	stq_st_err	Store-queue store error. - A store command was interleaved between the first and second store-queue write/store (stqw_seq_err). or - A store-queue request was detected when the 4-entry store queue was full (stq_overflow_err).	0	1
52	quad0_ce_thresh	Quadrant0 CE threshold. Multiple cache CEs have been detected during a hang pulse duration.	0	0
53	quad1_ce_thresh	Quadrant1 CE threshold. Multiple cache CEs have been detected during a hang pulse duration.	0	0

IBM PowerPC 970MP RISC Microprocessor

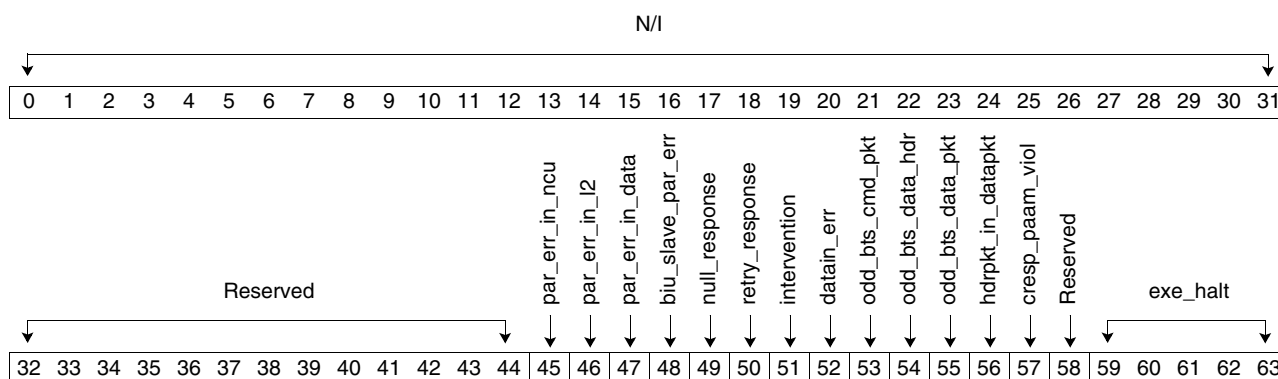
Bits	Field Name	Description	Default Mask Settings	
			Error Mask	Checkstop Enable
54	quad2_ce_thresh	Quadrant2 CE threshold. Multiple cache CEs have been detected during a hang pulse duration.	0	0
55	quad3_ce_thresh	Quadrant3 CE threshold. Multiple cache CEs have been detected during a hang pulse duration.	0	0
56	mult_cache_ce	Multiple cache CEs. Multiple cache CEs have been detected during a hang pulse duration from more than one quadrant.	0	0
57	mult_dir_par_err	Multiple directory parity errors. Multiple directory parity errors were detected within a period of two hang pulses. This indicates that the array error is not an intermittent fault. Because the BIU cannot make forward progress with a stuck fault in the directory, the system will be checkstopped.	0	1
58	ill_cresp	Illegal CRESP seen in receive state machine (RCFSM). The RC machine detected an illegal snoop, a combined response (CRESP) operation.	0	1
59	inv_mersi	Invalid MERSI detected. The RC machine detected or wrote an invalid modified/exclusive/reserved/shared/invalid (MERSI) cache-coherency protocol state in the directories.	0	1
60	ncu_snp_paam	NCU snoop PAAM error checkstop. This error occurs when the NCU snoop detects another ICBI or translation lookaside buffer invalidate entry (TLBI) request while the previous request is still active. That is, CRESP has not returned yet. The illegal previous adjacent address match (PAAM) request will be lost because the snoop does not register it; hence, there is <i>no</i> snoop response.	0	1
61	ncu_snp_cresp	NCU snoop CRESP error checkstop. This error occurs when an NCU snoop detects illegal values on CRESP(0:3): 0000 Good xxx1 Retry other Illegal The errors are registered, and the snoop hangs while waiting for a good or retry response. A secondary PAAM error can occur as a result. A debug switch, <code>cfg_snp_cresp_ckstp_en</code> , is provided to turn error detection off. In this case, the snoop will hang, and the result is undermined.	0	1
62	ncu_cresp_ill	NCU CRESP illegal error. This error occurs when the NCU receives a <code>cresp_valid</code> from the BIU with a CRESP(0:3) value that is illegal based on the transaction sent.	0	1
63	ncu_cresp_tmout	NCU CRESP timeout error. This error occurs when any of the state machines sourcing a transaction (store, load, or a micro-operation) to the bus time out while waiting for a valid and good CRESP. The timeout occurs upon the second hang pulse seen while waiting for the good CRESP.	0	1

IBM PowerPC 970MP RISC Microprocessor
12.4.2 BIU SCOM Register Definition
BIU Fault Isolation Register/And-Mask/Or-Mask

Address x'0A0001'

Type RW (FIR)
 WO (AND)
 WO (OR)

Reset Reset to all zeros during POR.



Bits	Field Name	Description	Error Mask	Checkstop Enable
0:31	N/I	Not implemented.	—	—
32:44	Reserved	Spare.	—	—
45	par_err_in_ncu	Parity error for input from NCU. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
46	par_err_in_l2	Parity error for input from L2. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
47	par_err_in_data	Parity error for input from encoded data. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
48	biu_slave_par_err	A BIU slave reports a parity error using a transaction response. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
49	null_response	A transaction response returns null. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
50	retry_response	A bus slave returns a transaction response retry on a data packet. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
51	intervention	A bus slave returns an intervention without shared or modified, or returns shared, modified, or intervention on a castout. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
52	datain_err	An error occurs on bus input data when encode is disabled. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1



IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description	Error Mask	Checkstop Enable
53	odd_bts_cmd_pkt	The command packet is an odd number of beats. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
54	odd_bts_data_hdr	The data header packet is an odd number of beats. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
55	odd_bts_data_pkt	The data packet is an odd number of beats. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
56	hdrpkt_in_datapkt	A header packet appears inside a data packet. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
57	cresp_paam_viol	CRESP PAAM window violation. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1
58	Reserved	Spare.	—	—
59:63	exe_halt	Tag associated with transaction response in reported errors. Processor execution is halted. A system checkstop is raised, which will halt the rest of the processors in the complex.	0	1



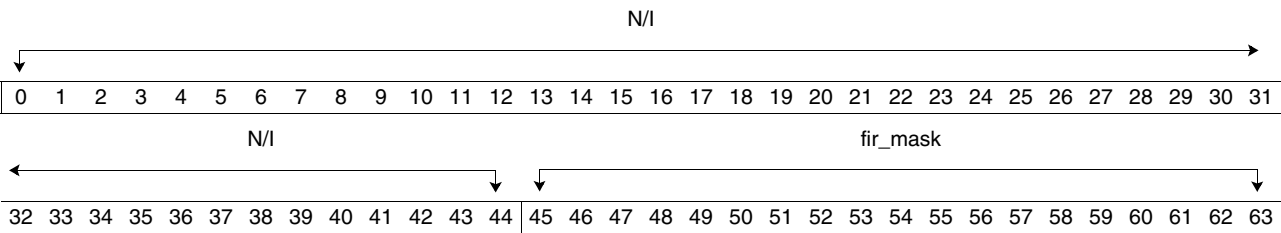
IBM PowerPC 970MP RISC Microprocessor

BIU Error Mask/And-Mask/Or-Mask

Address x'0A0400'

Type RW
 WO (AND)
 WO (OR)

Reset Reset to ones during POR.



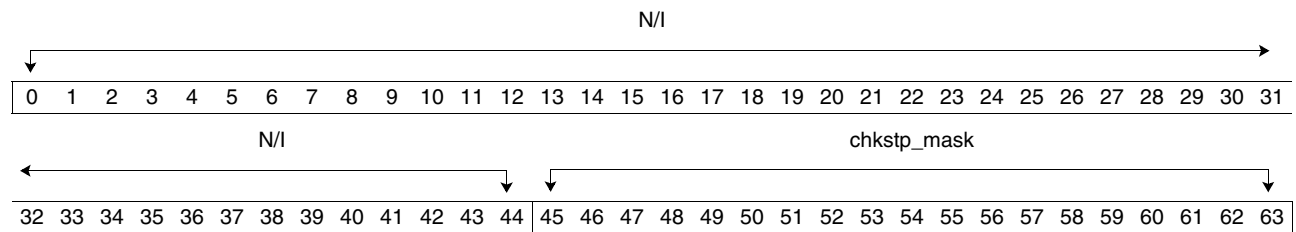
Bits	Field Name	Description
0:44	N/I	Not implemented.
45:63	fir_mask	Mask. 1 FIR bit is masked off.

BIU Checkstop Enable

Address x'0A0800'

Type RW

Reset Reset to zeros during POR.



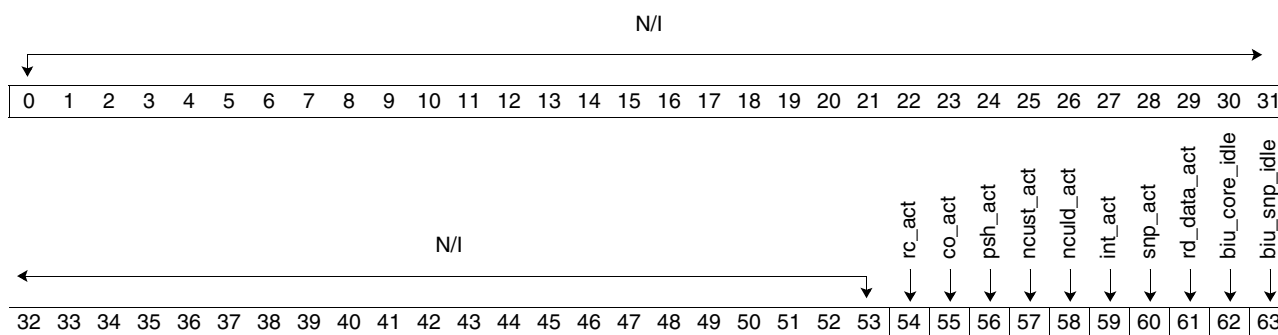
Bits	Field Name	Description
0:44	N/I	Not implemented.
45:63	chkstp_mask	Mask. 1 Checkstop.

IBM PowerPC 970MP RISC Microprocessor
BIU Status Register

Address x'0A9000'

Type RW

Reset Reset to all zeros during POR.



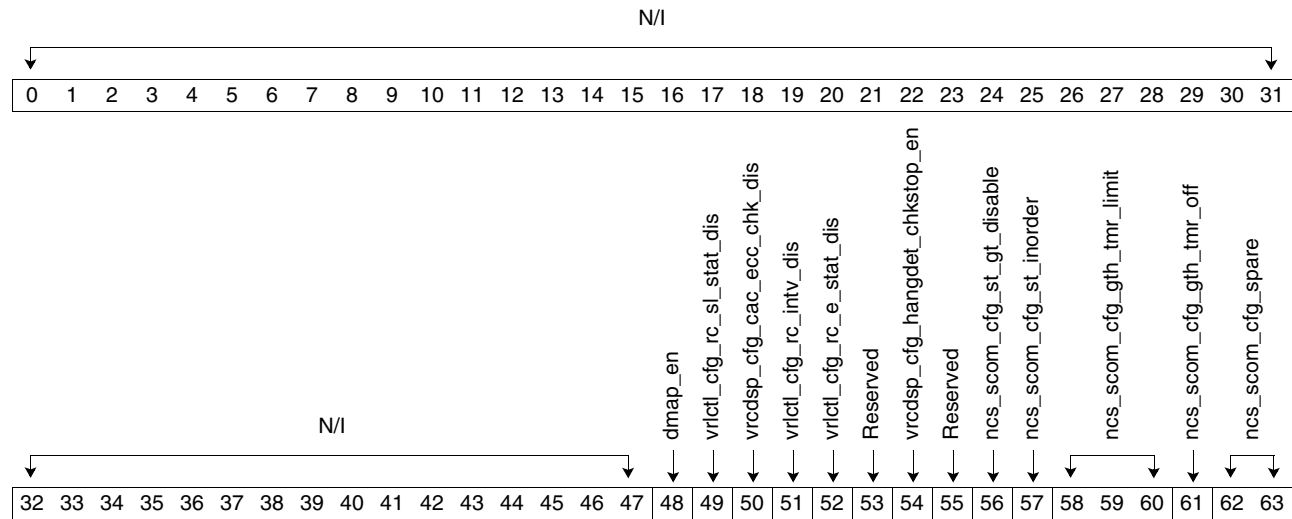
Bits	Field Name	Description
0:53	N/I	Not implemented.
54	rc_act	RC is the active operation (bfb_status_decode[0:3] equals '0000' or '1000').
55	co_act	Cast Out (CO) is the active operation (bfb_status_decode[0:3] equals '0001' or '1001').
56	psh_act	Cache push on snoop response (PSH) is the active operation (bfb_status_decode[0:3] equals '0010' or '1010').
57	ncust_act	Noncacheable unit store (NCUST) is the active operation (bfb_status_decode[0:3] equals '0011' or '1011').
58	nculd_act	Noncacheable unit load (NCULD) is the active operation (bfb_status_decode[0:3] equals '0100' or '1100').
59	int_act	INT is the active operation (bfb_status_decode[0:3] equals '0101' or '1101').
60	snp_act	The snoop command is the active operation (bfb_status_decode[0:3] equals '0110' or '1110').
61	rd_data_act	Read data is the active operation (bfb_status_decode[0:3] equals ['0111' to '1110']).
62	biu_core_idle	The BIU core is idle.
63	biu_snp_idle	The BIU snoop is idle.

BIU Mode Register

Address x'043000'

Type RW

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0:31	N/I	Not implemented.
32:47	N/I	Not implemented.
48	dmap_en	When set to '1', puts the L2 cache into Direct Mapped mode so that victim selection is based on a straight decode of address bits 42:44.
49	vrlctl_cfg_rc_sl_stat_dis	v_rcfsm disables going to the Shared Invalid (SI) state. Instead, it goes to Shared (S).
50	vrcdsp_cfg_cac_ecc_chk_dis	Disables viewing of any error correction code check (ECCCK) error detection.
51	vrlctl_cfg_rc_intv_dis	v_rctag: 1 Forces the intervention bit off.
52	vrlctl_cfg_rc_e_stat_dis	v_rcfsm: 1 Forces all Exclusive (E) directory states to Shared (S). 0 (Default) Allows E state.
53	Reserved	Spare.
54	vrcdsp_cfg_hangdet_chkstop_en	Enables a livelock warning signal to be sent to the L2 slice macro, where it can cause a checkstop.
55	Reserved	Spare.
56	ncs_scom_cfg_st_gt_disable	NCU store gather control disable.
57	ncs_scom_cfg_st_inorder	Force guarded access bit to '0'. Stores go out to the bus in order. Hence, one instruction has to complete with no retry before the other instruction can go out.

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
58:60	ncs_scom_cfg_gth_tmr_limit	These bits set the value for the timeout counter for the gather logic. The timeout counter is programmable for gather fine tuning. Setting these bits to zeros, in effect, discourages gathering although it does not totally disable it. Its value is (^bit0 & ^bit1 & bit2 & 0 & 0). The timeout is set as default '11000' pclk count when the latches are set to '000'. Its range is (0:56 pclks, step by 4).
61	ncs_scom_cfg_gth_tmr_off	Turn off gather timer. No timeout for gather.
62:63	ncs_scom_cfg_spare	Spare.

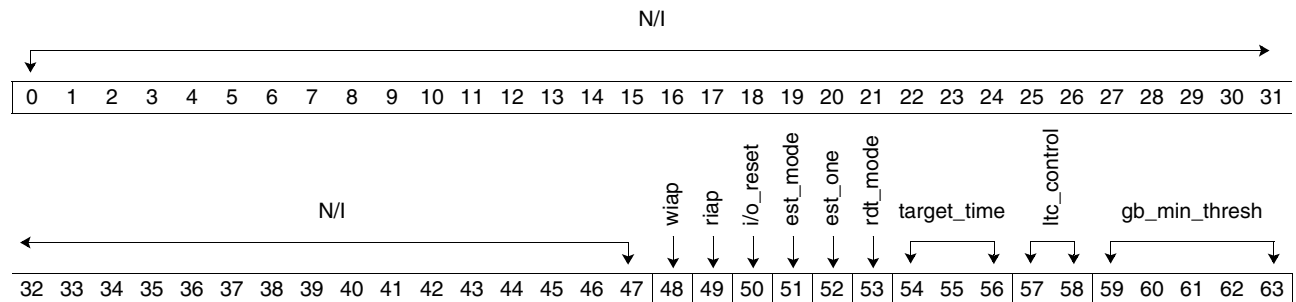
12.4.3 Processor Interconnect Registers

PI Mode Register 0

Address x'083000'

Type R/W

Reset Reset to all zeros during POR.



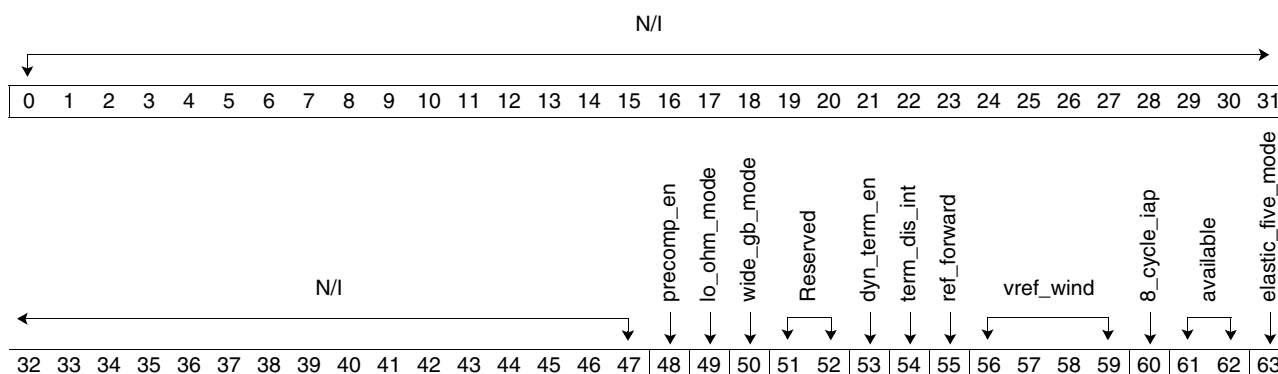
Bits	Field Name	Description
0:47	N/I	Not implemented.
48	wiap	Starts the IAP or self test on the driver.
49	riap	Starts the IAP or self test on the receiver.
50	i/o_reset	Resets the receiver, and readies for the IAP.
51	est_mode	Selects the shorts/open test.
52	est_one	Selects the polarity of the shorts test. 0 Creates a walking ones pattern 1 Creates a walking zeros pattern
53	rdt_mode	Selects random data test (RDT) mode.
54:56	target_time	Sets the target time.
57:58	ltc_control	Learned target cycle (LTC). 00 Minimum 01 Minimum + 1 10 Minimum + 2 11 Don't use LTC
59:63	gb_min_thresh	Guardband minimum threshold.

IBM PowerPC 970MP RISC Microprocessor
PI Mode Register 1

Address x'083101'

Type R/W

Reset Reset to all zeros during POR.



Bits	Field Name	Description															
0:47	N/I	Not implemented.															
48	precomp_en	Enables Precompensation mode at the drivers. Note: Bit 48 and bit 49 work together to determine how precompensation works. <table><tr><td>Bit 48</td><td>Bit 49</td><td></td></tr><tr><td>0</td><td>0</td><td>Precompensation is not enabled; high impedance is selected.</td></tr><tr><td>1</td><td>0</td><td>Precompensation is enabled; high impedance is selected. Precompensation can improve intersymbol interference.</td></tr><tr><td>0</td><td>1</td><td>Precompensation is not enabled; low impedance is selected.</td></tr><tr><td>1</td><td>1</td><td>Precompensation is enabled; low impedance is selected. Although precompensation is enabled, it has no effect when the drivers are in low-impedance mode.</td></tr></table>	Bit 48	Bit 49		0	0	Precompensation is not enabled; high impedance is selected.	1	0	Precompensation is enabled; high impedance is selected. Precompensation can improve intersymbol interference.	0	1	Precompensation is not enabled; low impedance is selected.	1	1	Precompensation is enabled; low impedance is selected. Although precompensation is enabled, it has no effect when the drivers are in low-impedance mode.
Bit 48	Bit 49																
0	0	Precompensation is not enabled; high impedance is selected.															
1	0	Precompensation is enabled; high impedance is selected. Precompensation can improve intersymbol interference.															
0	1	Precompensation is not enabled; low impedance is selected.															
1	1	Precompensation is enabled; low impedance is selected. Although precompensation is enabled, it has no effect when the drivers are in low-impedance mode.															
49	lo_ohm_mode	Enables low Ω mode at the drivers.															
50	wide_gb_mode	Defines the meaning of the double guardband fail calculation. <table><tr><td>0</td><td>Enables a narrow mode where a double guardband fail can occur if the bit fails a setup and a hold test at any time.</td></tr><tr><td>1</td><td>Enables a wide mode where a double guardband fail only occurs if the bit fails a setup and a hold for the same data beat.</td></tr></table>	0	Enables a narrow mode where a double guardband fail can occur if the bit fails a setup and a hold test at any time.	1	Enables a wide mode where a double guardband fail only occurs if the bit fails a setup and a hold for the same data beat.											
0	Enables a narrow mode where a double guardband fail can occur if the bit fails a setup and a hold test at any time.																
1	Enables a wide mode where a double guardband fail only occurs if the bit fails a setup and a hold for the same data beat.																
51:52	Reserved	Not used but available.															
53	dyn_term_en	Switches between clamp and termination on.															
54	term_dis_int	Disables termination on the receivers.															
55	ref_forward	Apply a reference voltage (V _{REF}) to the receivers. Calculated from the swing in the clock receiver.															
56:59	vref_wind	Signed number used to offset V _{REF} .															
60	8_cycle_iap	Selects IAP pattern length (4 or 8). <table><tr><td>0</td><td>4</td></tr><tr><td>1</td><td>8</td></tr></table>	0	4	1	8											
0	4																
1	8																
61:62	available	Not used but available.															
63	elastic_five_mode	This special mode of the PI is not supported.															

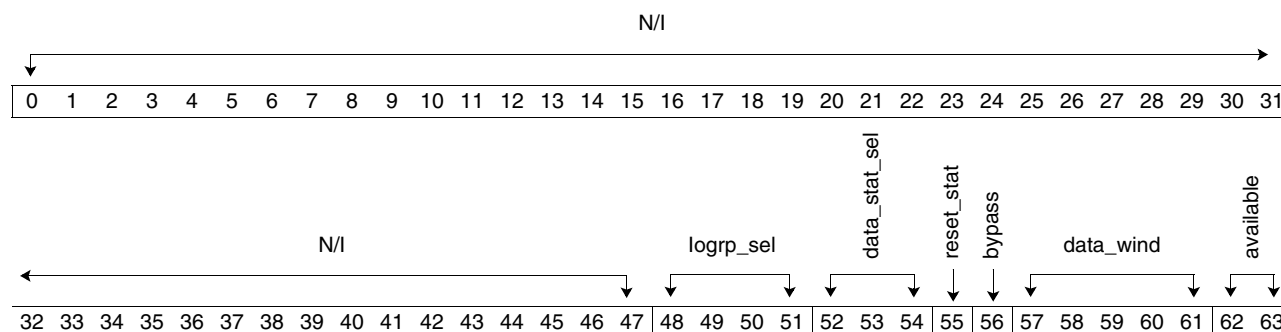
PI Mode Register 2

Address x'083201'

Type R/W

Note: Do not write during functional mode.

Reset Reset to all zeros during POR.



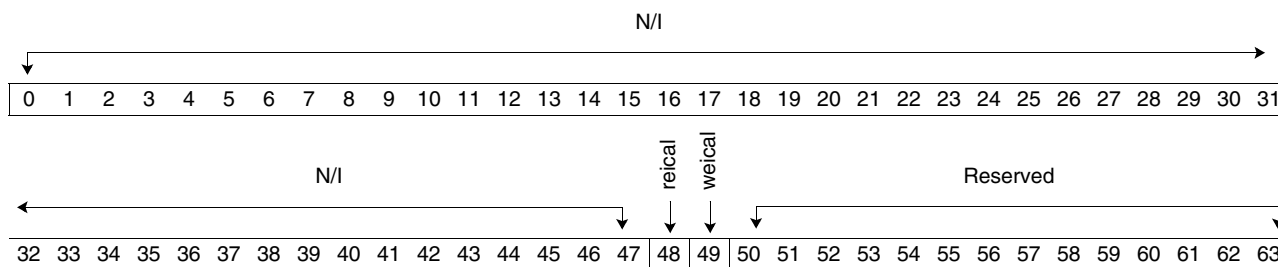
Bits	Field Name	Description
0:47	N/I	Not implemented.
48:51	logrp_sel	Selects the clock group for the status register.
52:54	data_stat_sel	Selects information to be retrieved for the status register.
55	reset_stat	Resets status information (stat[0:8] in the PI Status Register). Also begins a learn target cycle command.
56	bypass	Puts the receiver in bypass mode (no bit alignment and target cycle). For the second generation of PI (PI2), the PI bypass is not implemented the same way as for the first generation of PI (PI1).
57:61	data_wind	Add or subtract a delay from each bit. (Dissimilar to PI1).
62:63	available	Not used but available.

IBM PowerPC 970MP RISC Microprocessor
PI Mode Register 3

Address x'083300'

Type WO

Reset Reset to all zeros during POR.



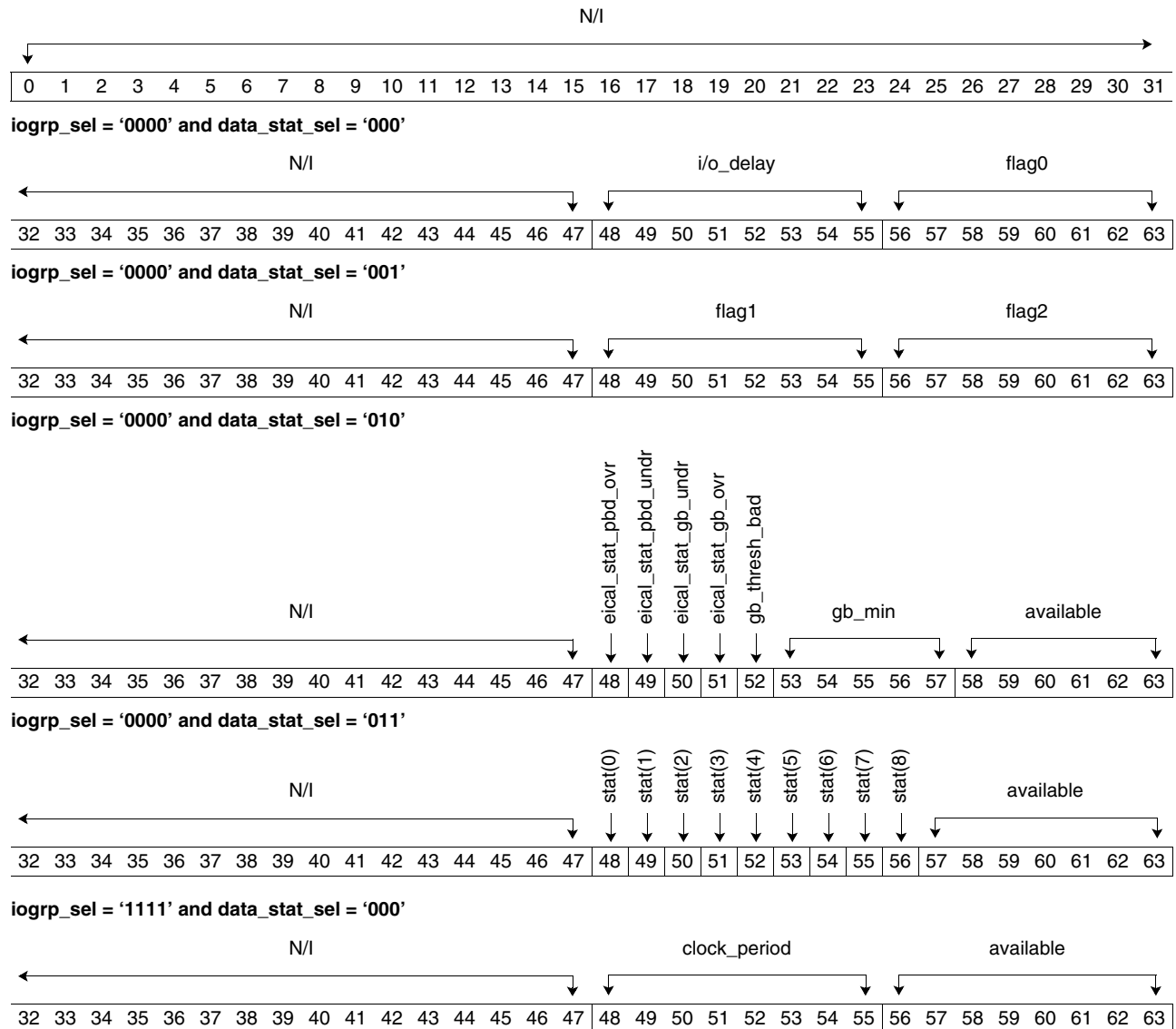
Bits	Field Name	Description
0:47	N/I	Not implemented.
48	reical	Enables processor interface recalibration (EICAL) for the receivers.
49	weical	Enables EICAL for the drivers.
48:63	Reserved	Reserved; not implemented.

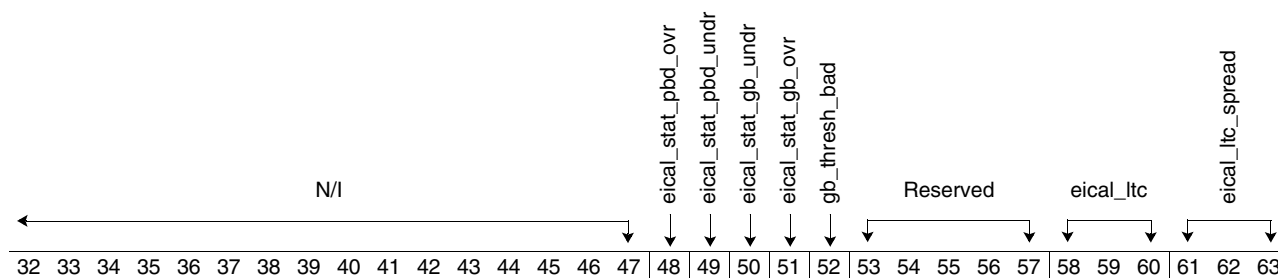
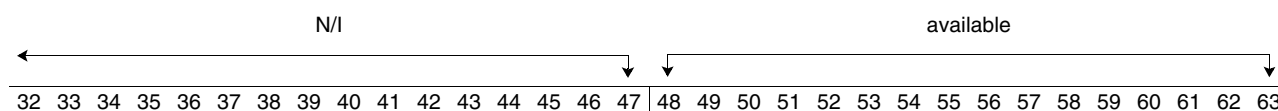
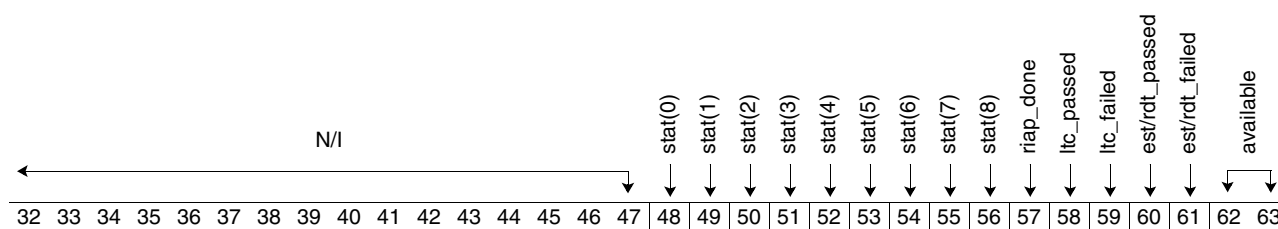
PI Status Register

Address x'084001'

Type RO

Reset Reset to all zeros during POR.



IBM PowerPC 970MP RISC Microprocessor
iogrp_sel = '1111' and data_stat_sel = '001'

iogrp_sel = '1111' and data_stat_sel = '011'

iogrp_sel = '1111' and data_stat_sel = '111'


Bits	Field Name	Description
If iogrp_sel = '0000' and data_stat_sel = '000' (Set in Mode Register 2)		
0:47	N/I	Not implemented.
48:55	i/o_delay	Number of delay elements in clock path.
56:63	flag0	Number of delay elements for flag 0.
If iogrp_sel = '0000' and data_stat_sel = '001'		
0:47	N/I	Not implemented.
48:55	flag1	Number of delay elements for flag 1.
56:63	flag2	Number of delay elements for flag 2.
If iogrp_sel = '0000' and data_stat_sel = '010'		
0:47	N/I	Not implemented.
48	eical_stat_pbd_ovr	Overflow of per-bit-deskew counter during IAP or EICAL.
49	eical_stat_pbd_undr	Underflow of per-bit-deskew counter during IAP or EICAL.
50	eical_stat_gb_undr	Guardband underflow during IAP or EICAL.
51	eical_stat_gb_ovr	Guardband overflow during IAP or EICAL.
52	gb_thresh_bad	Minimum guardband less than minimum required guardband.
53:57	gb_min	Minimum guardband value.
58:63	available	Not used but available.

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
If iogrp_sel = '0000' and data_stat_sel = '011'		
0:47	N/I	Not implemented.
48	stat(0)	Clock period too large to be measured with delay elements available.
49	stat(1)	IAP pattern '1' could not be found.
50	stat(2)	Right edge of data eye not found.
51	stat(3)	PBD maximum reached in one or more channels.
52	stat(4)	Failed to find flag 0.
53	stat(5)	Failed to find flag 1.
54	stat(6)	Failed to find flag 2.
55	stat(7)	Final clock delay does not find IAP pattern.
56	stat(8)	Negative I/O clock delay.
57:63	available	Not used but available.
If iogrp_sel = '1111' and data_stat_sel = '000'		
0:47	N/I	Not implemented.
48:55	clock_period	Clock period expressed in delay elements.
56:63	available	Not used but available.
If iogrp_sel = '1111' and data_stat_sel = '001'		
0:47	N/I	Not implemented.
48	eical_stat_pbd_ovr	Overflow of per-bit-deskew counter during IAP or EICAL.
49	eical_stat_pbd_undr	Underflow of per-bit-deskew counter during IAP or EICAL.
50	eical_stat_gb_undr	Guardband underflow during IAP or EICAL.
51	eical_stat_gb_ovr	Guardband overflow during IAP or EICAL.
52	gb_thresh_bad	Minimum guardband less than minimum required guardband.
53:57	Reserved	Not used but available.
58:60	eical_ltc	Learned target cycle value.
61:63	eical_ltc_spread	Difference in LTC between clock groups.
If iogrp_sel = '1111' and data_stat_sel = '011'		
0:47	N/I	Not implemented.
48:63	available	Not used but available.
If iogrp_sel = '1111' and data_stat_sel = '111'		
0:47	N/I	Not implemented.
48	stat(0)	Clock period too large to be measured with delay elements available.
49	stat(1)	IAP pattern '1' could not be found.
50	stat(2)	Right edge of data eye not found.
51	stat(3)	PBD maximum reached in one or more channels.
52	stat(4)	Failed to find flag 0.
53	stat(5)	Failed to find flag 1.

IBM PowerPC 970MP RISC Microprocessor

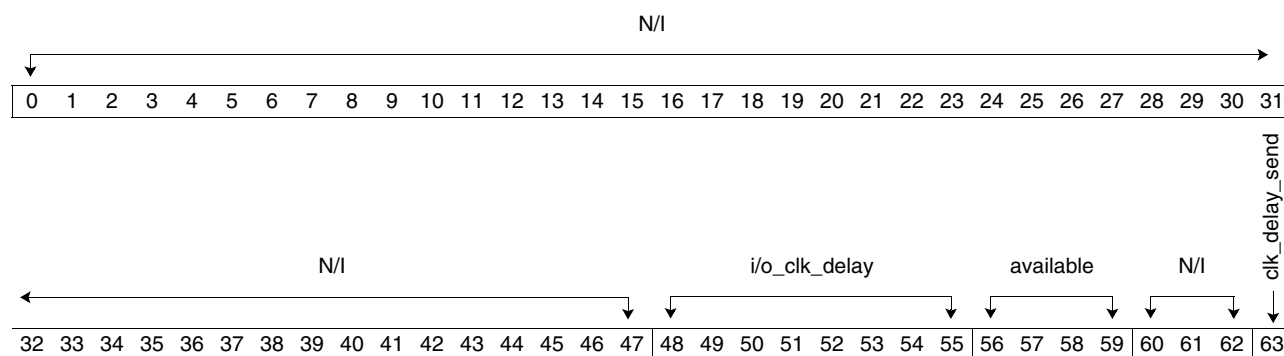
Bits	Field Name	Description
54	stat(6)	Failed to find flag 2.
55	stat(7)	Final clock delay does not find IAP pattern.
56	stat(8)	Negative I/O clock delay.
57	riap_done	Receiver IAP completed.
58	ltc_passed	Learned target cycle passed.
59	ltc_failed	Learned target cycle failed.
60	est/rdt_passed	EST or RDT passed.
61	est/rdt_failed	EST or RDT failed.
62:63	available	Not used but available.

PI Command Register

Address x'085000'

Type WO

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0:47	N/I	Not implemented.
48:55	i/o_clk_delay	Value of I/O clock delay to be loaded.
56:59	available	Not used but available.
60:62	N/I	Not implemented.
63	clk_delay_send	Set this bit to send the I/O clock delay to the I/O control macro.

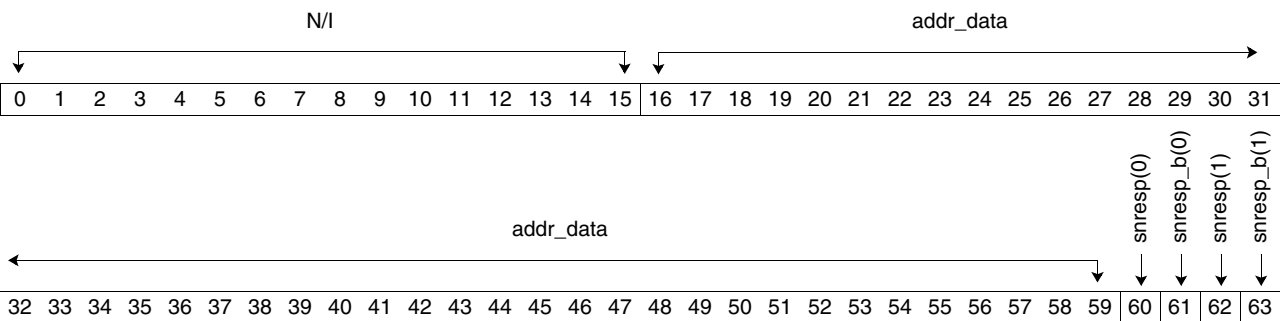
IBM PowerPC 970MP RISC Microprocessor

Driver IAP Register
Receiver IAP Register

Address x'086000' (Driver)
 x'086101' (Receiver)

Type R/W

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0:15	N/I	Not implemented.
16:59	addr_data	Mask for adin/out(0:43).
60	snresp(0)	Mask for srin/out(0).
61	snresp_b(0)	Mask for srin/out_b(0).
62	snresp(1)	Mask for srin/out(1).
63	snresp_b(1)	Mask for srin/out_b(1).

Note: To take effect, requires est_one set (PI Mode Register 0, bit 52).

12.5 Chip Pervasive SCOM Register Definition

12.5.1 Power-On Reset Registers (x'40XXXX')

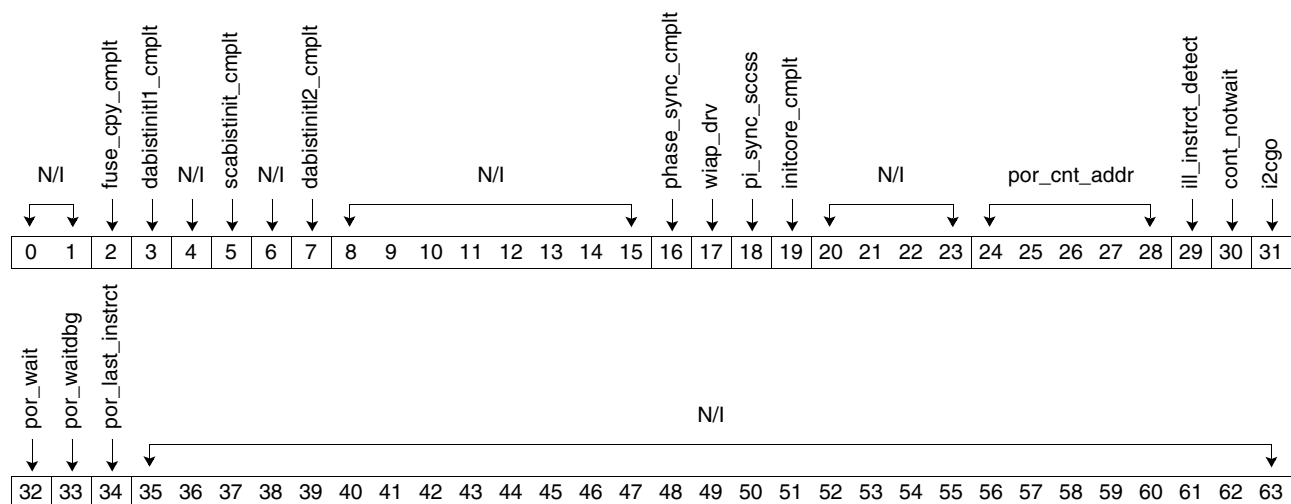
Power-On Reset Status Register

Address x'400000'

Type RO/WO

Clear entries are marked with an asterisk (*).

Reset Reset to all zeros during POR (inactive).



Bits	Field Name	Clear Entry	Description
0:1	N/I		Not implemented.
2	fuse_cpy_cmplt	*	Status is active after fuse copy completion.
3	dabistinit1_cmplt	*	Status is active after DABISTINITL1 completion.
4	N/I		Not implemented.
5	scabistinit_cmplt	*	Status is active after SCABISTINIT completion.
6	N/I		Not implemented.
7	dabistinit2_cmplt	*	Status is active after DABISTINITL2 completion.
8:15	N/I		Not implemented.
16	phase_sync_cmplt	*	Status is active after phase synchronization completion.
17	wiap_drv		Status is active while the writer initial alignment pattern (WIAP) is driven by POR.
18	pi_sync_sccss	*	Status is active after successful processor interface synchronization.
19	initcore_cmplt	*	Status is active after INITCORE completion.
20:23	N/I		Not implemented.
24:28	por_cnt_addr		Contains the current POR program counter address.

IBM PowerPC 970MP RISC Microprocessor

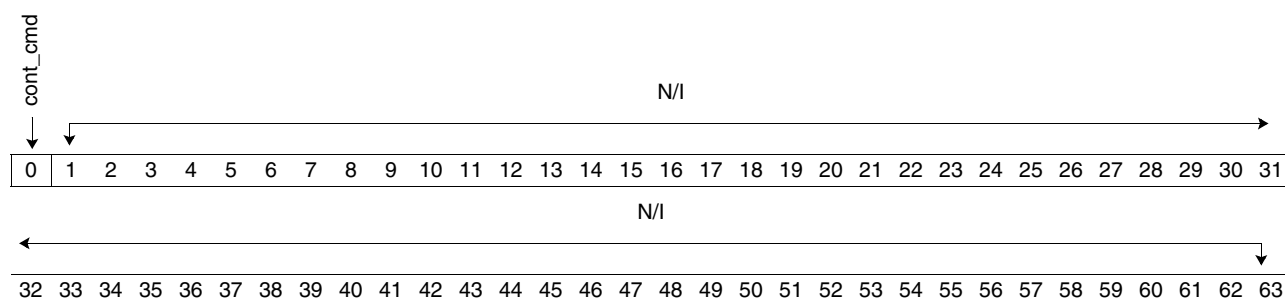
Bits	Field Name	Clear Entry	Description
29	ill_instrct_detect	*	Status is active after an illegal instruction is detected.
30	cont_notwait	*	Status is active if a continue was received while not in the Wait state.
31	i2cgo		Status is active after an I2CGO command until the next CONT command.
32	por_wait		Status is active when the POR state machine is in the Wait state.
33	por_waitdbg		Status is active when in the POR state machine is in the Waitdbg state.
34	por_last_instrct		Status is active when the POR state machine has reached the last instruction.
35:63	N/I		Not implemented.

Power-On Reset Continue Register

Address x'400101'

Type WO

Reset Reset to inactive during POR.



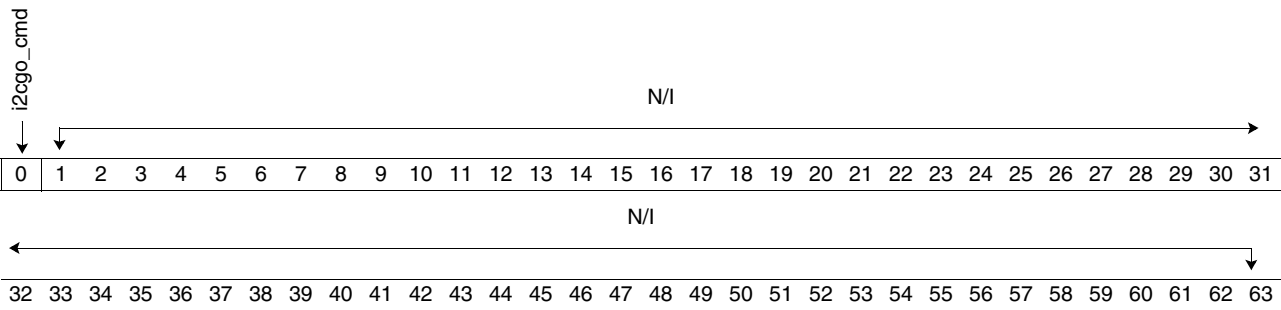
Bits	Field Name	Description
0	cont_cmd	Command bit. Sends a continue command to the POR state machine. This command also invalidates the <i>i2cgo</i> pin the next time the JTAG state machine enters the Test-Mode-Reset state or the Run-Idle state.
1:63	N/I	Not implemented.



IBM PowerPC 970MP RISC Microprocessor

Power-On Reset I²C/JTAG Arbitration Register

Address x'400201'
Type WO
Reset Reset to inactive during POR.



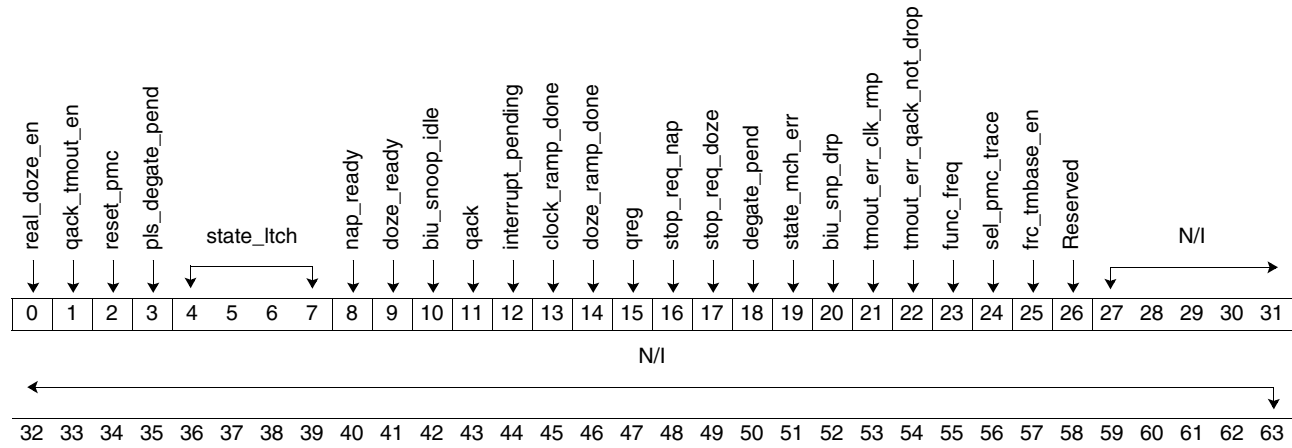
Bits	Field Name	Description
0	i2cgo_cmd	Command bit. Asserts and holds the <i>i2cgo</i> pin when the JTAG state machine enters the Test-Mode-Reset or Run-Idle state.
1:63	N/I	Not implemented.

Power-Management Control

Address x'400801'

Type Bits 0:3 and 24:26: RW
Bits 4:22: RO

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0	real_doze_en	Real Doze mode enable.
1	qack_tmout_en	Advance on quiescent acknowledgment (QACK) drop timeout enable.
2	reset_pmc	Reset power management control state machine.
3	pls_degate_pend	Pulse degate pending.
PMC Status Bits		
4:7	state_latch	State latches.
8	nap_ready	Nap_ready received.
9	doze_ready	Doze_ready received.
10	biu_snoop_idle	biu_snoop_idle received.
11	qack	QACK received.
12	interrupt_pending	interrupt_pending received.
13	clock_ramp_done	clock_ramp_done received.
14	doze_ramp_done	doze_ramp_done received.
15	qreg	QREG sent.
16	stop_req_nap	stop_req_nap sent.
17	stop_req_doze	stop_req_doze sent.
18	degate_pend	degate_pending sent.

IBM PowerPC 970MP RISC Microprocessor

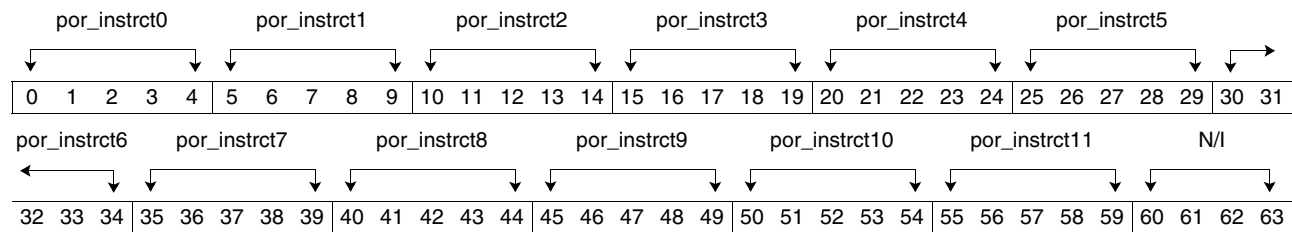
Bits	Field Name	Description
PMC Error Bits		
19	state_mch_err	State-machine error (undefined state).
20	biu_snp_drp	BIU Snoop Idle dropped while QACK active.
21	tmout_err_clk_rmp	Timeout error on clock ramp done.
22	tmout_err_qack_not_drop	Timeout error on QACK not dropped.
Miscellaneous		
23	func_freq	If set, allows f/64 as the functional frequency. In addition, if this bit is set, it sets the write margin in dynamic arrays to the maximum for the f/2 and f/4 frequencies.
24	sel_pmc_trace	Select PMC tracing.
25	frt_tmbase_en	Force time-base enable.
26	Reserved	Reserved.
27:63	N/I	Not implemented.

Power-On Reset Sequence Register 0

Address x'401400'

Type WO

Reset Initialized during POR (see the *Power-On Reset Specification* for more information).

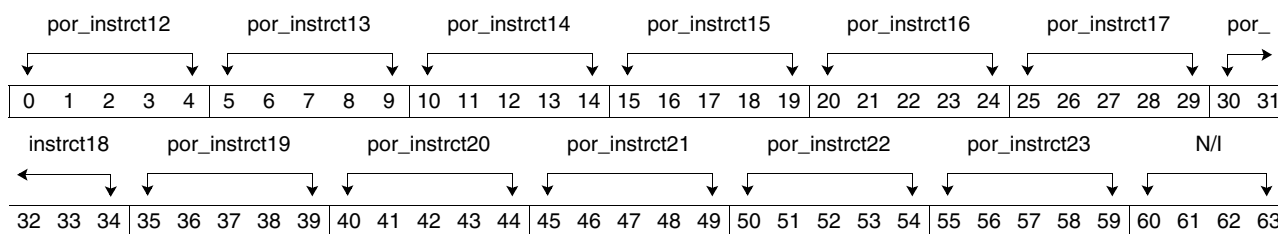


Bits	Field Name	Description
0:4	por_instrct0	Data: POR instruction 0 (first instruction).
5:9	por_instrct1	Data: POR instruction 1.
10:14	por_instrct2	Data: POR instruction 2.
15:19	por_instrct3	Data: POR instruction 3.
20:24	por_instrct4	Data: POR instruction 4.
25:29	por_instrct5	Data: POR instruction 5.
30:34	por_instrct6	Data: POR instruction 6.
35:39	por_instrct7	Data: POR instruction 7.
40:44	por_instrct8	Data: POR instruction 8.
45:49	por_instrct9	Data: POR instruction 9.
50:54	por_instrct10	Data: POR instruction 10.
55:59	por_instrct11	Data: POR instruction 11.
60:63	N/I	Not implemented.

IBM PowerPC 970MP RISC Microprocessor
Power-On Reset Sequence Register 1

Address x'402400'

Type WO

Reset Initialized during POR (see the *Power-On Reset Specification* for more information).


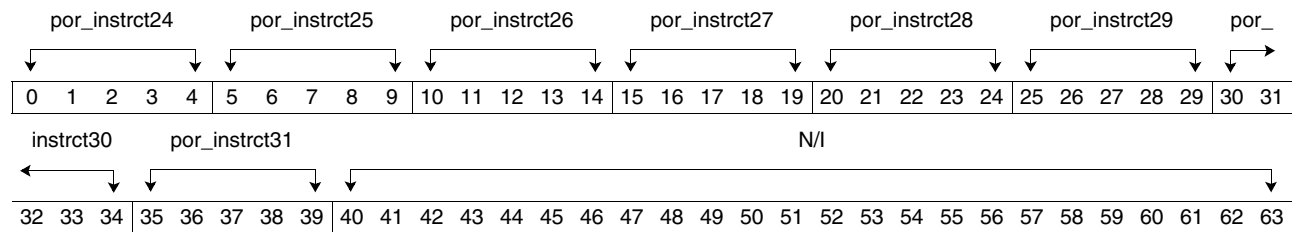
Bits	Field Name	Description
0:4	por_instrct12	Data: POR instruction 12.
5:9	por_instrct13	Data: POR instruction 13.
10:14	por_instrct14	Data: POR instruction 14.
15:19	por_instrct15	Data: POR instruction 15.
20:24	por_instrct16	Data: POR instruction 16.
25:29	por_instrct17	Data: POR instruction 17.
30:34	por_instrct18	Data: POR instruction 18.
35:39	por_instrct19	Data: POR instruction 19.
40:44	por_instrct20	Data: POR instruction 20.
45:49	por_instrct21	Data: POR instruction 21.
50:54	por_instrct22	Data: POR instruction 22.
55:59	por_instrct23	Data: POR instruction 23.
60:63	N/I	Not implemented

Power-On Reset Sequence Register 2

Address x'404400'

Type WO

Reset Initialized during POR (see the *Power-On Reset Specification* for more information).



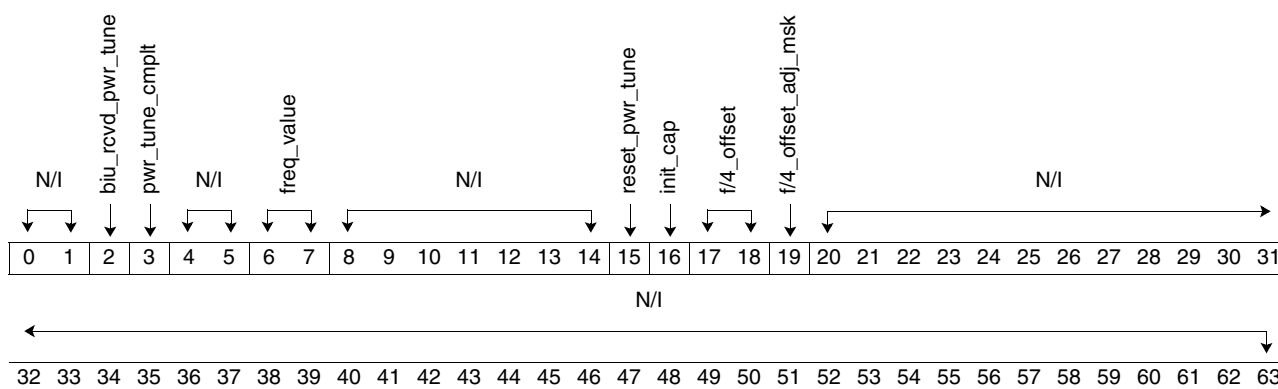
Bits	Field Name	Description
0:4	por_instrct24	Data: POR instruction 24.
5:9	por_instrct25	Data: POR instruction 25.
10:14	por_instrct26	Data: POR instruction 26.
15:19	por_instrct27	Data: POR instruction 27.
20:24	por_instrct28	Data: POR instruction 28.
25:29	por_instrct29	Data: POR instruction 29.
30:34	por_instrct30	Data: POR instruction 30.
35:39	por_instrct31	Data: POR instruction 31 (last instruction).
40:63	N/I	Not implemented.

IBM PowerPC 970MP RISC Microprocessor
Power Tuning Status Register

Address x'408001'

Type RW: bits 2:3, 7:8,16:19
 WO: bit 15

Reset Initialized to all zeros during POR.



Bits	Field Name	Description
0:1	N/I	Not implemented.
2	biu_rcvd_pwr_tune	Turned on when the BIU has received a power tuning command from the North Bridge. Reset after a read to this register, and after power tuning command completion.
3	pwr_tune_cmplt	Turned on after completion of a power tuning command. Reset after a read to this register, and after power tuning command completion.
4:5	N/I	Not implemented.
6:7	freq_value	Current/requested frequency value.
8:14	N/I	Not implemented.
15	reset_pwr_tune	Reset the power tuning machine inside ChipRAS. The BIU received the normal power tuning complete acknowledgment. No error is reported.
16	init_cap	Initialize clock alignment procedure (CAP). Set and then unset to start CAP.
17:18	f/4_offset	Read: Computed the low-speed (f/4) frequency offset to SYSClk. Write: Value to add to the f/4-frequency offset counter. Then toggle bit 19 to execute.
19	f/4_offset_adj_msk	Mask the f/4 offset counter adjust command. Set only to measure offset.
20:63	N/I	Not implemented.

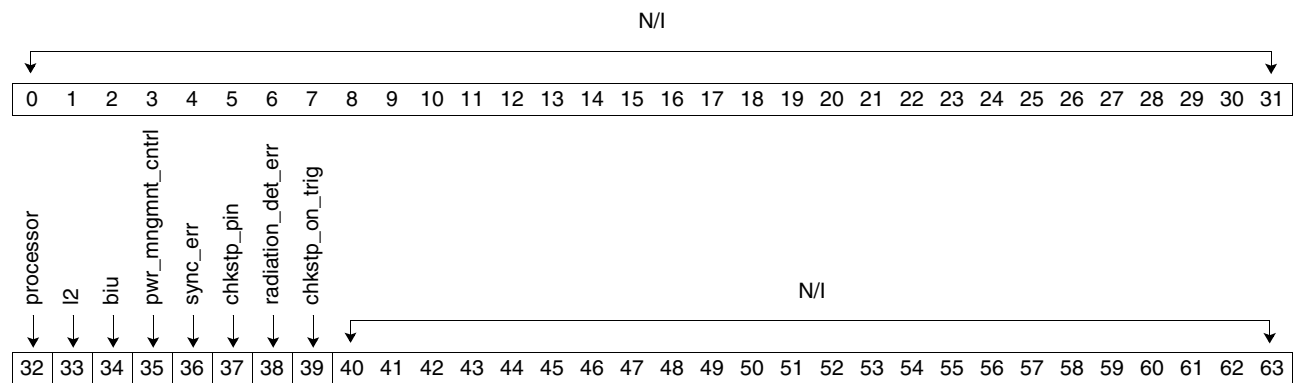
12.5.2 Chip Free-Running Clock Section Control/Status (x'50[0:4]XXX')

Global Fault Isolation for Checkstop Conditions (Global FIR)

Address x'500001'

Type RO

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0:31	N/I	Not implemented.
32	processor	Processor core.
33	l2	L2.
34	biu	BIU.
35	pwr_mngmnt_cntrl	Power-management control.
36	sync_err	Synchronization error in the free-running clock controls macro, tc_ccintf.
37	chkstp_pin	Checkstop pin.
38	radiation_det_err	Radiation detection error.
39	chkstp_on_trig	Checkstop on trigger.
40:63	N/I	Not implemented.



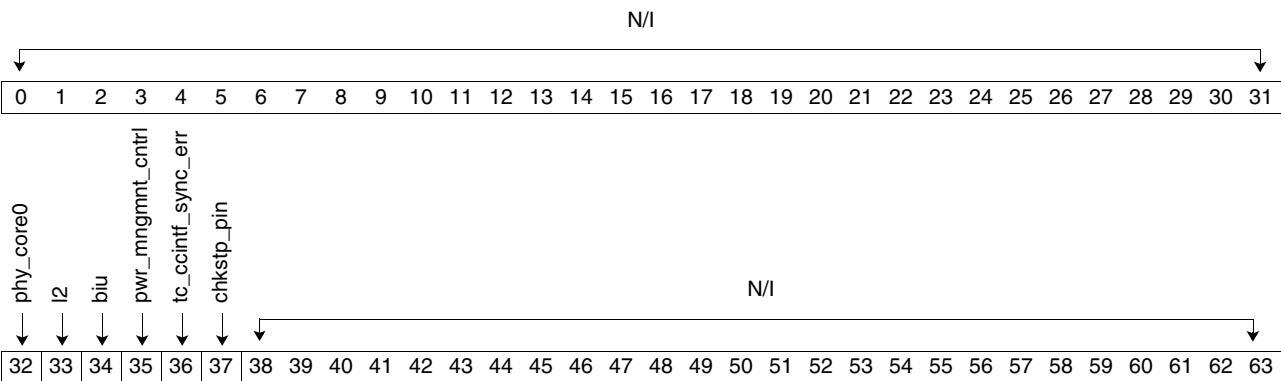
IBM PowerPC 970MP RISC Microprocessor

Error Enable Mask

Address x'500400'

Type RW

Reset Reset to all zeros during POR.



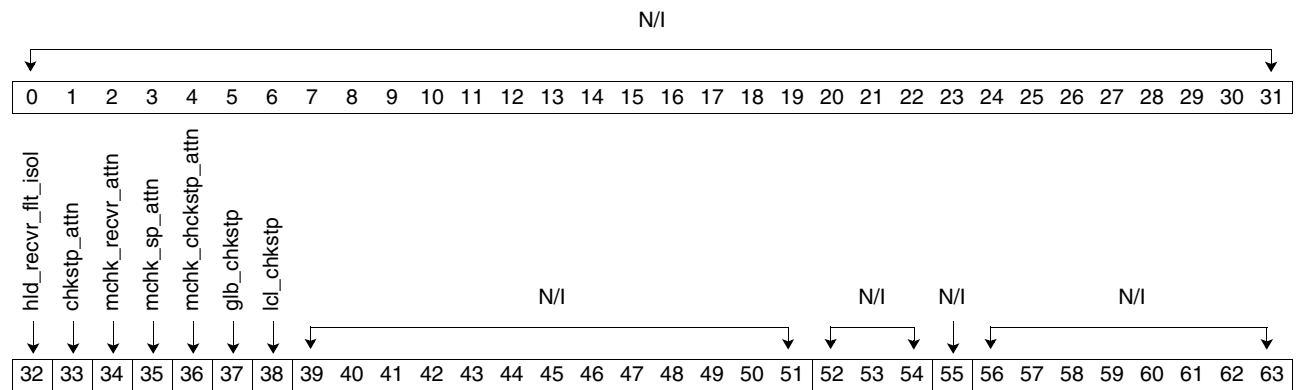
Bits	Field Name	Description
0:31	N/I	Not implemented.
32	phy_core0	Physical core0.
33	l2	L2.
34	biu	BIU.
35	pwr_mngmnt_cntrl	Power-management control.
36	tc_ccintf_sync_err	Synchronization error in TC_CCINTF.
37	chkstp_pin	Checkstop pin.
38:63	N/I	Not implemented.

Mode Register for Fault Isolation Registers

Address x'500601'

Type RW

Reset Reset to all zeros during POR.



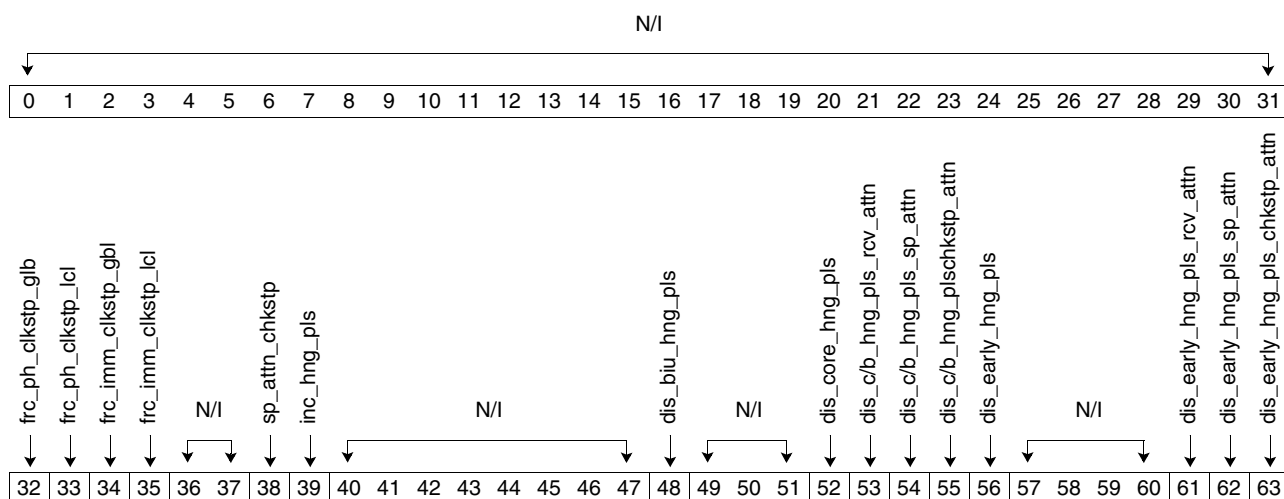
Bits	Field Name	Description
0:31	N/I	Not implemented.
32	hld_recvr_flt_isol	Hold recoverable-fault isolation when error is detected. Behaves like the checkstop FIR.
33	chkstp_attn	Checkstop indications from the checkstop pin, or checkstop trigger will set the checkstop attention.
34	mchk_recvr_attn	Processor machine check will set recoverable attention.
35	mchk_sp_attn	Processor machine check will set special attention.
36	mchk_chkstp_attn	Processor machine check will set checkstop attention.
37	glb_chkstp	Global checkstop signal will signal checkstop to processor cores.
38	lcl_chkstp	Local checkstop signal will signal checkstop to processor cores.
39:63	N/I	Not implemented.

IBM PowerPC 970MP RISC Microprocessor
Debug Mode Register

Address x'500700'

Type RW

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0:31	N/I	Not implemented.
32	frc_ph_clkstp_glb	Force a phase-aligned clock stop when the global checkstop signal is active.
33	frc_ph_clkstp_lcl	Force a phase-aligned clock stop when the local checkstop signal is active.
34	frc_imm_clkstp_gbl	Force an immediate clock stop when the global checkstop signal is active.
35	frc_imm_clkstp_lcl	Force an immediate clock stop when the local checkstop signal is active.
36:37	N/I	Not implemented.
38	sp_attn_chkstp	Core special attention will cause a checkstop.
39	inc_hng_pls	Increase hang pulse rate by 100 times.
40:47	N/I	Not implemented.
48	dis_biu_hng_pls	Disable BIU hang pulses.
49:51	N/I	Not implemented.
52	dis_core_hng_pls	Disable core hang pulses.
53	dis_c/b_hng_pls_rcv_attn	Disable core and BIU hang pulses when recoverable attention is set.
54	dis_c/b_hng_pls_sp_attn	Disable core and BIU hang pulses when special attention is set.
55	dis_c/b_hng_pls_chkstp_attn	Disable core and BIU hang pulses when checkstop attention is set.
56	dis_early_hng_pls	Disable early hang pulse to core.
57:60	N/I	Not implemented.

**IBM PowerPC 970MP RISC Microprocessor**

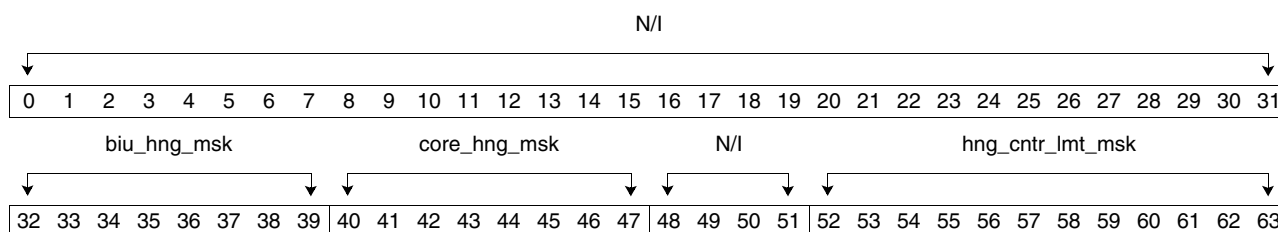
Bits	Field Name	Description
61	dis_early_hng_pls_rcv_attn	Disable early hang pulses when recoverable attention is set.
62	dis_early_hng_pls_sp_attn	Disable early hang pulses when special attention is set.
63	dis_early_hng_pls_chkstp_attn	Disable early hang pulses when checkstop attention is set.

IBM PowerPC 970MP RISC Microprocessor
Hang Pulse Generation

Address x'503001'

Type RW

Reset Reset to all zeros during POR.



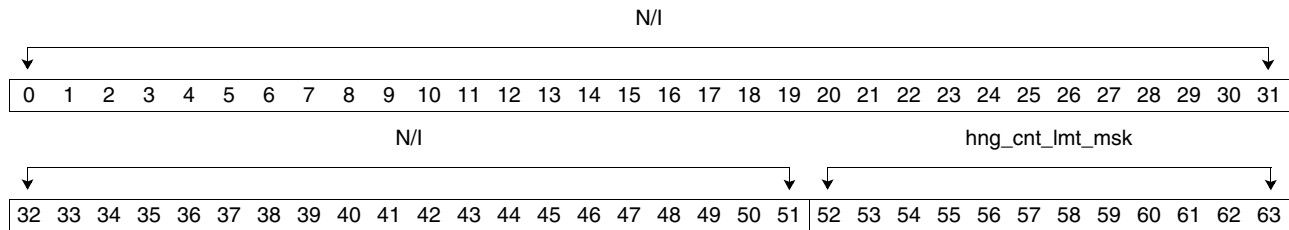
Bits	Field Name	Description
0:31	N/I	Not implemented.
32:39	biu_hng_msk	BIU hang mask. x'01' Hang pulse generated every <i>Counter x 2</i> . x'02' Hang pulse generated every <i>Counter x 4</i> . x'04' Hang pulse generated every <i>Counter x 8</i> . x'08' Hang pulse generated every <i>Counter x 16</i> . x'10' Hang pulse generated every <i>Counter x 32</i> . x'20' Hang pulse generated every <i>Counter x 64</i> . x'40' Hang pulse generated every <i>Counter x 128</i> . x'80' Hang pulse generated every <i>Counter x 256</i> .
40:47	core_hng_msk	Core hang mask. x'01' Hang pulse generated every <i>Counter x 2</i> . x'02' Hang pulse generated every <i>Counter x 4</i> . x'04' Hang pulse generated every <i>Counter x 8</i> . x'08' Hang pulse generated every <i>Counter x 16</i> . x'10' Hang pulse generated every <i>Counter x 32</i> . x'20' Hang pulse generated every <i>Counter x 64</i> . x'40' Hang pulse generated every <i>Counter x 128</i> . x'80' Hang pulse generated every <i>Counter x 256</i> .
48:51	N/I	Not implemented.
52:63	hng_cntr_lmt_msk	Hang counter limit mask (12-bit LFSR). x'4FC' 100 cycles x'CF0' 200 cycles x'DAE' 300 cycles x'E89' 400 cycles x'D1A' 500 cycles x'0A3' 600 cycles x'F8C' 700 cycles x'FAA' 800 cycles x'8F3' 900 cycles x'6C8' 1000 cycles x'5D1' 2000 cycles x'668' 3000 cycles x'D3E' 4000 cycles

Early Hang Pulse Generation

Address x'503100'

Type RW

Reset Reset to all zeros during POR.



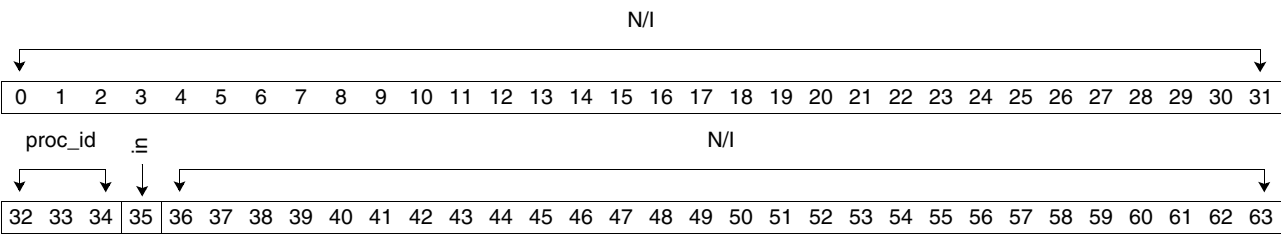
Bits	Field Name	Description
0:51	N/I	Not implemented.
52:63	hng_cnt_lmt_msk	Hang counter limit mask (12-bit LFSR). x'6A8' 20 cycles x'BA0' 50 cycles x'4FC' 100 cycles x'CF0' 200 cycles x'DAE' 300 cycles x'E89' 400 cycles x'D1A' 500 cycles x'0A3' 600 cycles x'F8C' 700 cycles x'FAA' 800 cycles x'8F3' 900 cycles x'6C8' 1000 cycles x'5D1' 2000 cycles x'668' 3000 cycles x'D3E' 4000 cycles



IBM PowerPC 970MP RISC Microprocessor

Chip ID Register

Address x'504101'
Type RW (Bit 35 is WO)
Reset Reset to zeros during POR.



Bits	Field Name	Description
0:31	N/I	Not implemented.
32:34	proc_id	Processor ID (0:2).
35	in	Capture values from processor ID (PID) primary C4 inputs. (This is a write only bit.)
36:63	N/I	Not implemented. Returns zeros.

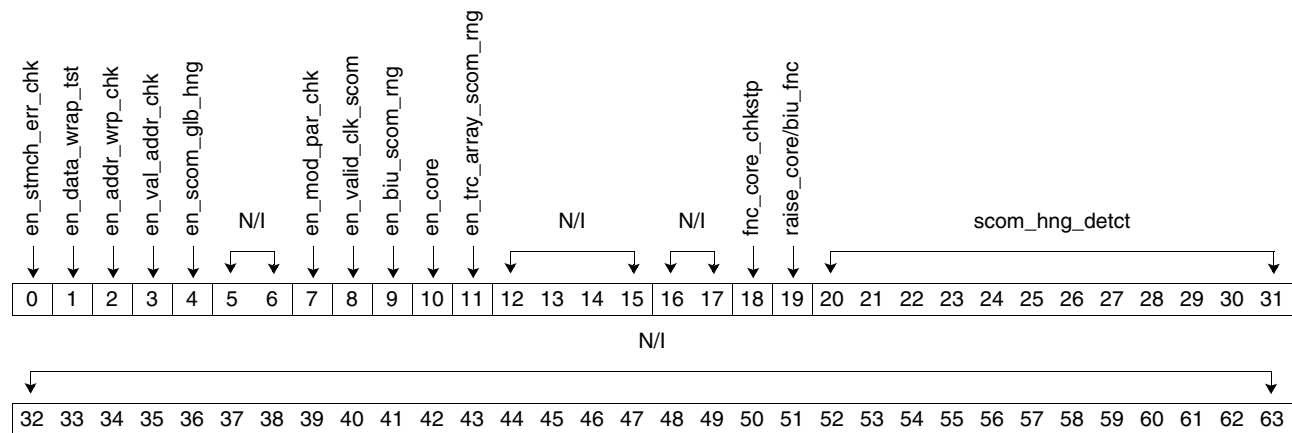
12.5.3 Chip Parallel SCOM Control (x'6XXXXX')

SCOM Mode Register

Address x'600001'

Type R/W

Reset Set to x'0000 0FFE 0000 0000' during POR or SCRESET.



Bits	Field Name	Description
0	en_stmch_err_chk	Enable state-machine error checking.
1	en_data_wrap_tst	Enable data wrap test.
2	en_addr_wrp_chk	Enable address wrap check.
3	en_val_addr_chk	Enable valid address checking.
4	en_scom_glb_hng	Enable SCOM global hang checking.
5:6	N/I	Not implemented.
7	en_mod_par_chk	Enable modifier parity checking.
8	en_valid_clk_scom	Enable valid clock SCOM address checking.
9	en_biu_scom_rng	Enable STS/BIU SCOM ring. Set to '0' before scanning BIU.
10	en_core	Enable core SCOM ring. Enables SCOM ring and functional fences. Set to '0' before scanning core.
11	en_trc_array_scom_rng	Enable I/O SCOM ring. Set to '0' before scanning I/O.
12:17	N/I	Not implemented.
18	fnc_core_chkstp	Fence cores on checkstop. Note: Bits 37:38 of the Mode Register for Fault Isolation Registers controls the core checkstop.
19	raise_core/biu_fnc	Raise core and BIU fences on a clock stop. Note: A clock stop is seen by the free-running logic several cycles before the clocks are actually stopped. By default, software should set this bit to '1'. However, it must be set to '0' before cycle stepping for debug.

IBM PowerPC 970MP RISC Microprocessor

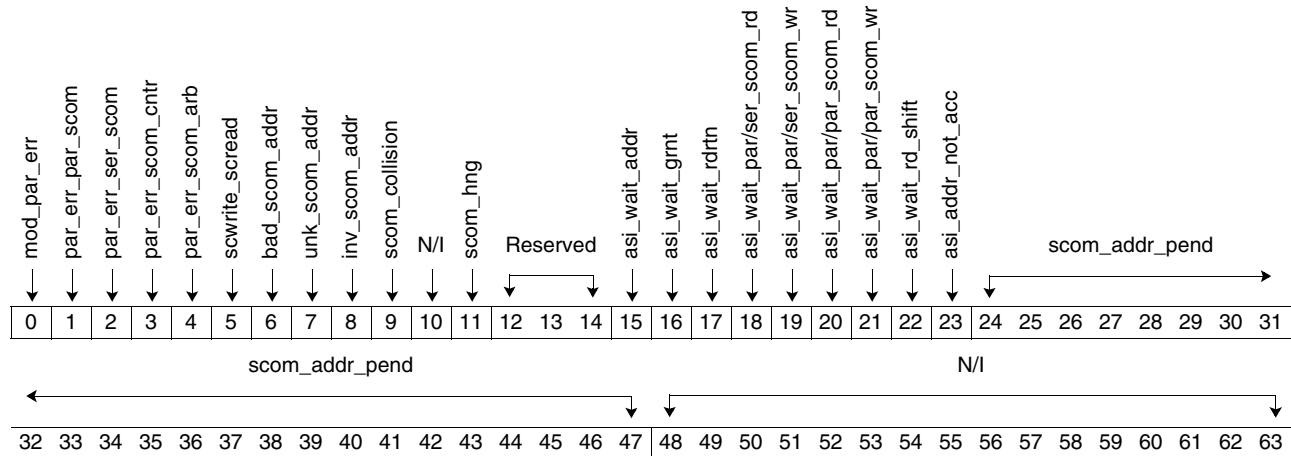
Bits	Field Name	Description
20:31	scom_hng_detct	When the LFSR counter matches the value in the register before the SCOM operation completes, a SCOM hang has been detected.
		Processor LFSR
		Clock Counter
		Cycles: Values:
		500 x'D1A'
		1000 x'6C8'
		2000 x'5D1'
		3000 x'668'
		4095 x'FFE'
32:63	N/I	Not implemented.

SCOM Controller Error Register

Address x'600100'

Type RO (Write zeros to clear after SCRESET).

Reset Reset to all zeros during POR.



Bits	Field Name	Description
0	mod_par_err	Modifier parity error.
1	par_err_par_scom	Parity error in the parallel SCOM state machine.
2	par_err_ser_scom	Parity error in the serial SCOM state machine.
3	par_err_scom_cntr	Parity error in the main SCOM controller state machine.
4	par_err_scom_arb	Parity error in the SCOM arbiter state machine.
5	scwrite_scread	The SCWRITE pipe latch and SCREAD pipe latch were both on.
6	bad_scom_addr	Bad address in the SCOM ring.
7	unk_scom_addr	The SCOM address was not recognized.
8	inv_scom_addr	The SCOM address for the clock command was invalid.
9	scom_collision	An SCOM collision in the core.
10	N/I	Not implemented.
11	scom_hng	SCOM hang. The active source identifier (ASI) in bits 15:23 indicates what was pending.
12:14	Reserved	Reserved.
15	asi_wait_addr	(ASI) Waiting for the address to return.
16	asi_wait_grnt	(ASI) Waiting for a grant from the arbiter.
17	asi_wait_rdrtn	(ASI) Waiting for read data to return.
18	asi_wait_par/ser_scom_rd	(ASI) The parallel-to-serial state machine is waiting for SCOM Read to drop.
19	asi_wait_par/ser_scom_wr	(ASI) The parallel-to-serial state machine is waiting for SCOM Write to drop.
20	asi_wait_par/par_scom_rd	(ASI) The parallel-to-parallel state machine is waiting for SCOM Read to drop.

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
21	asi_wait_par/par_scom_wr	(ASI) The parallel-to-parallel state machine is waiting for SCOM Write to drop.
22	asi_wait_rd_shift	(ASI) Waiting for the read shifter to empty.
23	asi_addr_not_acc	(ASI) The address was returned and not accepted.
24:47	scom_addr_pend	The SCOM address that was pending at the time of the error.
48:63	N/I	Not implemented.



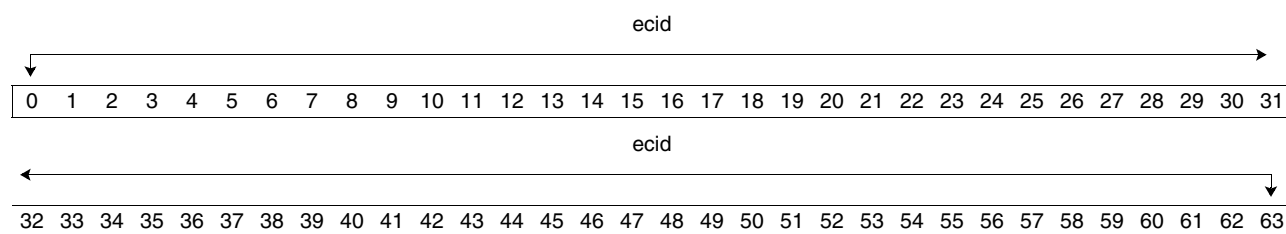
IBM PowerPC 970MP RISC Microprocessor

Electronic Chip ID

Address x'600200'

Type RO

Reset Set to fuse values when the fuse ring is scanned.

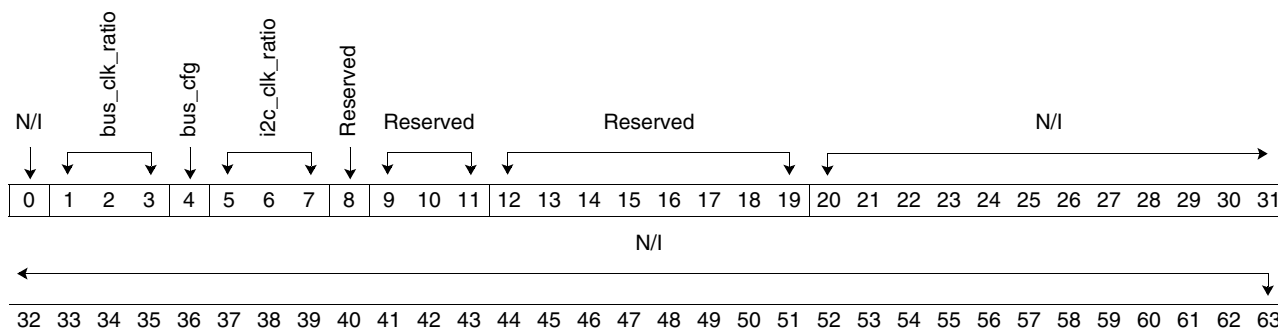


Bits	Field Name	Description
0:63	ecid	Electronic chip ID (ECID).

IBM PowerPC 970MP RISC Microprocessor
Clock Ratio Register (N:1 Phase Hold Control)

Address x'600400'

Type RW

Reset ratio: Reset to all zeros during POR.
 iap_encode: Reset to '1' during POR.
 count1us_factor: Reset to all ones during POR.


Bits	Field Name	Description
0	N/I	Not implemented.
1:3	bus_clk_ratio	Bus clock ratio (for N:1 phase_hold generation). 000 2:1 001 3:1 010 4:1 011 6:1 100 8:1 101 12:1 110 24:1 111 1:1
4	bus_cfg	Writing a '1' to this bit triggers the bus_cfg read operation, which captures values from the bus_cfg primary C4 inputs. Reads return '0'.
5:7	i2c_clk_ratio	I ² C clock ratio. This counter is asynchronous to the mod48 counter and scales with the processing unit frequency. 000 16:1 (full speed), 8:1 (half speed), 4:1 (quarter speed) 001 Not supported 010 8:1 (full speed), 4:1 (half speed), 2:1 (quarter speed) 011 Not supported 100 Not supported 101 Not supported 110 Not supported 111 1:1
8:19	Reserved	Reserved.
20:63	N/I	Not implemented.

12.5.4 Chip Clock/Scan Control (x'8[0:4]XXXX')

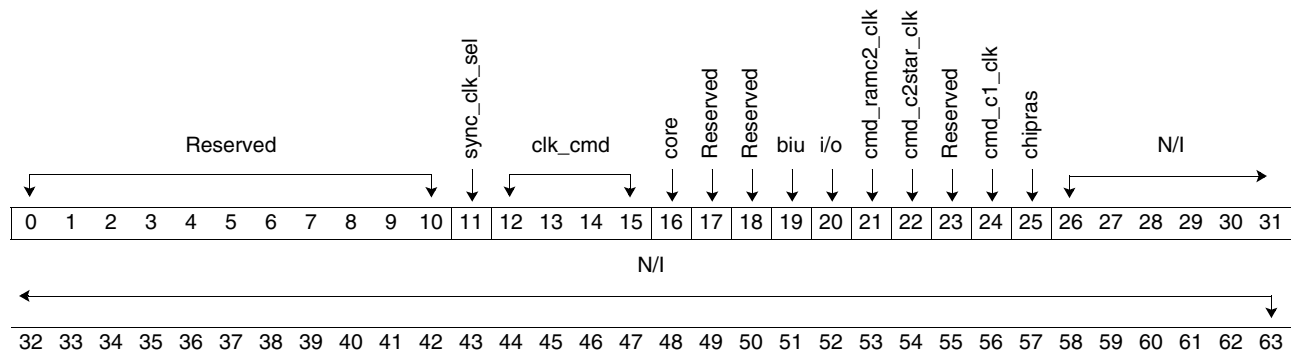
Clock Command Register

See Figure 12-6 Common Clock Commands on page 382.

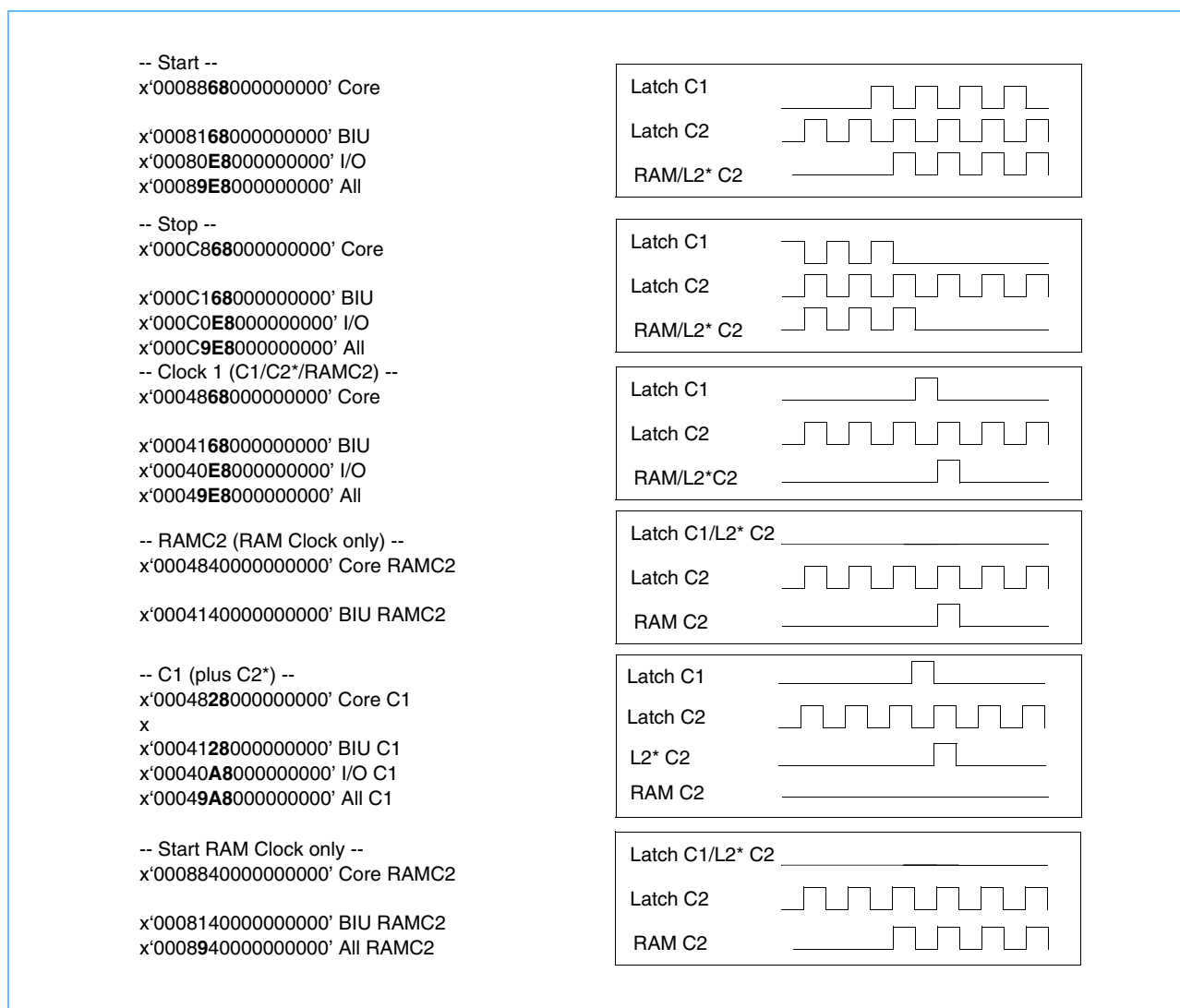
Address x'800000'

Type R/W

Reset Set to all zeros during POR.



Bits	Field Name	Description
0:10	Reserved	Unused.
11	sync_clk_sel	SYNC clock select. Must be programmed to logic level zero ['0'].
12:15	clk_cmd	Clock command x'4' Pulse selected clocks x'8' Start selected clocks x'C' Stop selected clocks Note: Bits 14:15 are not implemented.
Domain Select		
16	core	Domain select core.
17:18	Reserved	Unused.
19	biu	Domain select STS/BIU.
20	i/o	Domain select I/O.
Array Clock Commands (applies to Start, Pulse, and Stop)		
21	cmd_ramc2_clk	Apply command to RAMC2 clock signal (not valid for I/O).
22	cmd_c2star_clk	Apply command to C2star clock signals.
23	Reserved	Unused.
Latch Clock Commands (applies to Start or Pulse, and Stop)		
24	cmd_c1_clk	Apply command to C1 clock signal.
Free-Running Clock Section		
25	chipras	Domain select: ChipRAS.
26:63	N/I	Not implemented.

IBM PowerPC 970MP RISC Microprocessor
Figure 12-6. Common Clock Commands


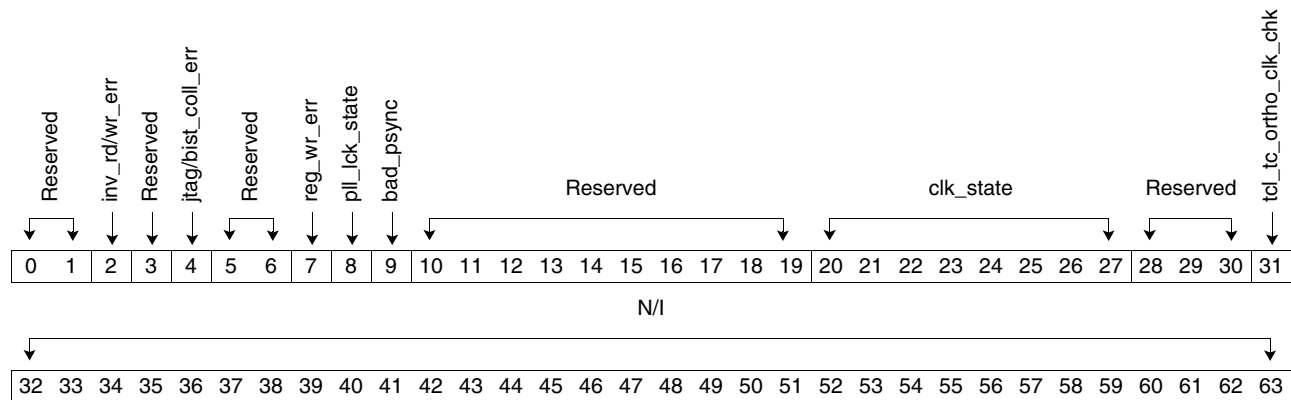
Status Register

Note: Bits 0:12 of the Status Register can be masked or blocked by setting bits 0:12 of the Status Register Mask.

Address x'800003'

Type R/W: bits 2, 4, 7:9, 31. RO: bits 20:21, 23:24

Reset Set to x'0000 0040 0000 0000' during POR.



Bits	Field Name	Description
0:1	Reserved	Reserved.
2	inv_rd/wr_err	Invalid read/write address error. An attempt was made to read or write to a clock interface address that does not exist. Raises SCATTN unless blocked.
3	Reserved	Reserved.
4	jtag/bist_coll_err	JTAG/BIST collision error. JTAG attempted to initiate a write to a valid register while the event processor or array built-in self test (ABIST) was in progress. Raises SCATTN unless blocked.
5:6	Reserved	Reserved.
7	reg_wr_err	Register write error. Either of the following actions raises SCATTN unless it is blocked: - A write to the Clock Command Register (address x'800009') while not all of the core0 or STS clocks are off. - A write to the I/O Control Register (address x'80000F') while the I/O clocks are running.
8	pll_lck_state	pll_lck state. If '1', PLL unlock is detected. Raises SCATTN unless blocked.
9	bad_psync	Bad <i>psync</i> ¹ detected. The Clock Controls mod48 counter does not match the Power Tuning mod48 counter.
10:19	Reserved	Reserved.

1. A signal provided by the North Bridge, which is active for one rising edge of SYSCLK every 24 SYSCLK cycles.

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description												
20:27	clk_state	Clock state bits (CLK_STATE[0:7]). These correspond one-to-one with the domains defined in bits 16 to 20 of the Clock Command Register. A value of '1' indicates that the C! clocks are running. These bits cannot be written. <u>Bits</u> <table><tr><td>20</td><td>Core</td></tr><tr><td>21</td><td>Pervasive</td></tr><tr><td>22</td><td>Reserved</td></tr><tr><td>23</td><td>STS</td></tr><tr><td>24</td><td>I/O</td></tr><tr><td>25:27</td><td>Reserved</td></tr></table>	20	Core	21	Pervasive	22	Reserved	23	STS	24	I/O	25:27	Reserved
20	Core													
21	Pervasive													
22	Reserved													
23	STS													
24	I/O													
25:27	Reserved													
28:30	Reserved	Reserved.												
31	tcl_tc_ortho_clk_chk	Indicates that an attempt was made to scan a domain while the clocks were not stopped.												
32:63	N/I	Not implemented.												

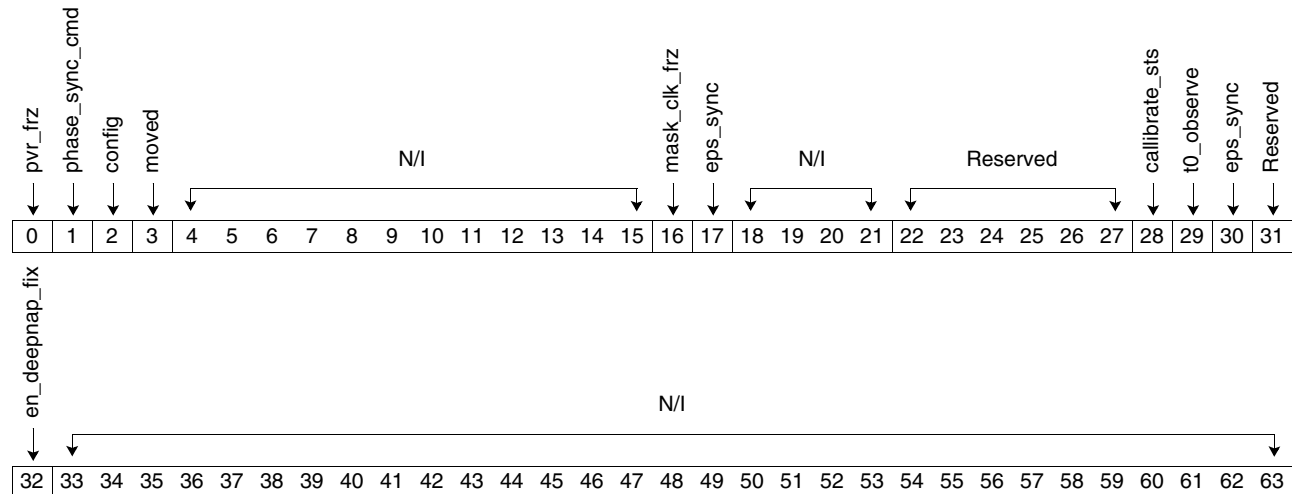
1. A signal provided by the North Bridge, which is active for one rising edge of SYSCLK every 24 SYSCLK cycles.

Phase Synchronization Control Register

Address x'800006'

Type R/W: 0:2, 16:17
WO: 22:27

Reset Set to all zeros during POR.



Bits	Field Name	Description
0	pvr_frz	Processor Version Register (PVR) freeze. If set, the PVR latches in the fixed-point unit (FXU) can be scanned to an arbitrary value.
1	phase_sync_cmd	Command bit. Run phase synchronization.
2	config	Configuration bit. Do not checkstop on bad_sync.
3	moved	Moved to bit 9 of the Status Register.
4:15	N/I	Not implemented.
16	mask_clk_frz	Mask clock freeze. When set to '1', the clock-freeze signal from the free-running global controls macro, ts_glob, is ignored.
17	eps_sync	Event processing sequencer (EPS) synchronization. If set, starts the EPS engine processing synchronously to the EPS engine of the other processing unit.
18:21	N/I	Not implemented.
22:27	Reserved	Reserved (all zeros).
28	calibrate_sts	STS calibrate. Used during test to generate a test-only (TO) initialization pulse to the STS.
29	t0_observe	T0 observation. If set, toggles the quiescent request (QREQ) pin on every T0 pulse.
30:31	Reserved	Reserved(all zeros).
32	en_deepnap_fix	Enable deep nap delaying to prevent clock race condition at certain bus ratios.
33:63	N/I	Not implemented.

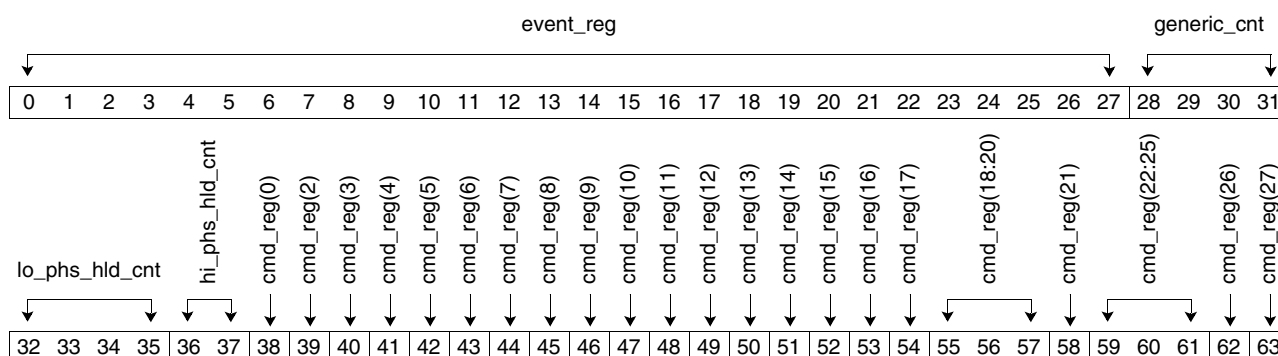
IBM PowerPC 970MP RISC Microprocessor
Clock Command Control Register

This SCOM register controls the Event Processing Sequencer (EPS) used to generate clock patterns for LBIST, ABIST, POR, and debugging.

Address x'800009'

Type R/W (Writing this register while the core or BIU is running will raise an attention.)

Reset Set to x'0000000 0314001A' during POR.



Bits	Field Name	Description
0:27	event_reg	Event register (four fields of 7 control bits per field). Bits 21:27 First event field. 14:20 Second event field. 7:13 Third event field. 0:6 Fourth event field. Field bit definitions: Bits 0, 7, 14, 21: Run system C1 clock. 1, 8, 15, 22: Run scan SC1 clock. 2, 9, 16, 23: Run RAM_C2 clock. 3, 10, 17, 24: Run C2STAR clock. 4, 11, 18, 25: Ignored. 5, 12, 19, 26: Enable clock-rate divide counter. This slows down the application of clock events. 6, 13, 20, 27: Enable generic counter, specifying the number of times to loop on the event field.
28:31	generic_cnt	Generic counter (0:3). Programmed count value: down counter.
32:35	lo_phs_hld_cnt	Lower-order phase hold counter (mod48) for event processing. Valid values are 0 to 11. Note: This counter cannot be used while the PHASESYNC POR instruction is active.

Notes:

- The system C2 clock is always running in functional mode.
- The domain selects (bits 38:42) only control the scope of C1, RAM_C2, and C2STAR.
- Bit 52 initiates this sequence, which performs the events in order. Bits 55:57 control looping on these events. The SCOM LBIST Test Length Register must be used to configure the TEST LENGTH count.
- If scan SC1 is the *only* clock selected for an event, then the CHANNEL LENGTH count is used for that event. The SCOM LBIST Channel Length Register must be used to configure the CHANNEL LENGTH count. If SC1 is selected in addition to either a SYSTEM or RAM clock, then only a single clock is given.

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
36:37	hi_phs_hld_cnt	Upper-order phase hold counter for event processing. Programmed count value: up counters 0 to 3. Notes: - This counter cannot be used while the PHASESYNC POR instruction is active. - This counter counts the carry out of the lower-order phase hold counter (for event processing).
38	cmd_reg(0)	CORE 0 select.
39	cmd_reg(2)	BIU select.
40	cmd_reg(3)	I/O select.
41	cmd_reg(4)	Free-running select.
42	cmd_reg(5)	Domain select for hard stop. Free-running domain.
43	cmd_reg(6)	Global array inhibit. This signal gets ORed with <i>ABIST_EN</i> and <i>Local_Latch</i> OR <i>ESP_RAMSCAN</i> and NOT <i>Local_Lt</i> OR LSSD/GSD Scan-Active. If the scan 0 has not occurred through the scan-ABIST section, the arrays will be enabled functionally when the global array inhibit is inactive. Set during POR. Need to clear by writing.
44	cmd_reg(7)	SCAN 0 command (similar to the FLUSH 0 Access Command). Scan in '0' to all the selected domains specified by Cmd_reg(0:3) <i>except</i> the FUSE and NOT_BIST (timing chain) rings.
45	cmd_reg(8)	FULL SCAN 0 command. Used in conjunction with the SCAN 0 Command. Causes the FUSE and NOT_BIST (timing chain) to also scan to '0'. POR sets it to '1' (clean NOT_BIST chain).
46	cmd_reg(9)	LBIST. When set to run LBIST, causes LBIST ring-selects and causes the Multiple Input Signature Register (MISR) connections to be established. The LBIST logic is fenced out in the free-running domain. This bit has no effect on the EPS engine. Bit 11 in the I/O Control Register (x'80000F') is automatically set to fence the I ² C. It is not reset when the LBIST bit is reset.
47	cmd_reg(10)	CAM BIST. Set for testing content-addressable memory (CAM) arrays with scanned ABIST.
48	cmd_reg(11)	Central scanned ABIST core. Set to test core arrays covered by scanned ABIST.
49	cmd_reg(12)	Central scanned ABIST not core. Set to test noncore arrays covered by scanned ABIST.
50	cmd_reg(13)	Decentral ABIST. Set to test arrays covered by the following dedicated ABIST engines: L2 cache, L2 directory, L2 least recently used (LRU), IFU cache, IFU directory, IFU branch history table (BHT), L1 cache data (L1D).
51	cmd_reg(14)	Decentral ABIST RAM_SELECT. Zeros for the first half of the arrays; ones for the second half.
52	cmd_reg(15)	Event processing command. Initiates a clock event process (see <i>Figure 12-7</i> on page 390 for more information).
53	cmd_reg(16)	Phase load control(0). Load the phase counter with the programmed value at event processor state 2.
54	cmd_reg(17)	Phase load control(1). Load the phase counter with the programmed value with Event Complete. Normally, for LBIST, this bit would be set. Not programming this bit makes it possible to change the RUN_CLOCK pulse to PHASE_HOLD alignment.
Notes: - The system C2 clock is always running in functional mode. - The domain selects (bits 38:42) only control the scope of C1, RAM_C2, and C2STAR. - Bit 52 initiates this sequence, which performs the events in order. Bits 55:57 control looping on these events. The SCOM LBIST Test Length Register must be used to configure the TEST LENGTH count. - If scan SC1 is the <i>only</i> clock selected for an event, then the CHANNEL LENGTH count is used for that event. The SCOM LBIST Channel Length Register must be used to configure the CHANNEL LENGTH count. If SC1 is selected in addition to either a SYSTEM or RAM clock, then only a single clock is given.		

IBM PowerPC 970MP RISC Microprocessor

Bits	Field Name	Description
55:57	cmd_reg(18:20)	Run_N Command control. Causes the event processor to loop back to different event fields. This provides the capability of an initialization setup. 000 Loop on all events. Branches back to first event. 100 Loop on fourth event. 101 Branch back to second event. 110 Branch back to third event.
58	cmd_reg(21)	Event processing override. Provides the means to interrupt an event process that is in progress.
59:61	cmd_reg(22:25)	Domain select for hard stop. <u>SCOM bits</u> <u>cmd_reg bit</u> 59 22 Core active high. 23 Reserved. 60 24 STS active high. 61 25 I/O active high.
62	cmd_reg(26)	I/O mesh clock select. The processor interface runs off an <i>input</i> clock called the IO_CLOCK. For LBIST and scanning of the I/O domain, this bit should be set first. POR sets this bit to '1'.
63	cmd_reg(27)	LBIST ac mode. Causes 8:1 logic clocks to turn off.

Notes:

- The system C2 clock is always running in functional mode.
- The domain selects (bits 38:42) only control the scope of C1, RAM_C2, and C2STAR.
- Bit 52 initiates this sequence, which performs the events in order. Bits 55:57 control looping on these events. The SCOM LBIST Test Length Register must be used to configure the TEST LENGTH count.
- If scan SC1 is the *only* clock selected for an event, then the CHANNEL LENGTH count is used for that event. The SCOM LBIST Channel Length Register must be used to configure the CHANNEL LENGTH count. If SC1 is selected in addition to either a SYSTEM or RAM clock, then only a single clock is given.

Table 12-4. EPS Engine Description

```

alias eps_start      = rch9<52>           // set to start
alias event_code[1:4] = { rch9<0:6>, rch9<7:13>, rch9<14:20>, rch9<21:27> } // micro-code
alias cnt_gen        = rch9<28:31>        // generic counter
alias cnt_scan       = rch8<0:15>         // scan counter
alias slow_clk       = rcha<0:5>          // sleep between clock
alias scan_speed     = rch2<28:31>        // sc1 speed
alias loop_type      = rch9<55:57>        // where to loop after last instr
alias test_len       = rchb<0:19>         // # of loops
alias load_phase_imm = rch9<53>           // load phase prior to EPS run
alias load_phase     = rch9<54>           // load phase after each instr.
alias mod48val       = rch9<36:37>rch9<32:35> // phase value to load
alias eps_running    = rch4<0>            // running
alias eps_complete   = rch4<1>            // completed
alias run_continuous = rch2<22>           // loops for ever
alias eps_break      = rch9<58>           // stop EPS machine

waitfor (eps_start == 1);

eps_running=1; eps_complete=0;
pc=4;
if(load_phase_imm) mod48 = mod48val;

loop1:
event      = event_code[pc];
c1_stop    = event[0];
sc1_stop   = event[1];
ramc2_stop = event[2];
c2star_stop = event[3];
killdynclk = event[4];
useslowclks = event[5];

if( c1_stop == 1 AND sc1_stop == 0 AND event[6] == 1) cnt = cnt_gen;
else if(c1_stop == 0 AND sc1_stop == 1) cnt = cnt_scan;
else cnt=0;

loop2:
if(useslowclks) stop c1,sc1,ramc2,c2star clocks for slow_clk cycles;
start clks for 1 cycle depending on c1_stop,sc1_stop,ramc2_stop,c2star_stop,killdynclk;
if(sc1_stop) stop c1,sc1,ramc2,c2star clocks for scan_speed cycles;

cnt--; if (cnt >= 0) goto loop2;

pc--; if(pc==0)
switch (loop_type)
  case '000': pc=4; // loop to 1st event
  case '100': pc=1; // loop to 4th event
  case '101': pc=3; // loop to 2nd event
  case '110': pc=2; // loop to 3rd event
test_len--;

if(load_phase) mod48 = mod48val;
if((test_len>=0 OR continuous_run) AND NOT eps_break) goto loop1;
eps_running=0; eps_complete=1;

```

IBM PowerPC 970MP RISC Microprocessor

Figure 12-7. Example of LBIST Commands using the EPS Engine

```

-- Scan / Wait / Clock 1 (All C1 Clocks) --
x'840002' x'0000 0005 0000 0000' Scan Clock Speed = 5
x'600400' x'7000 0000 0000 0000' N:1 Bus Ratios set to 1:1
x'80000A' x'7800 000E 0000 0000' slow clock=15
x'84000B' x'vvvv v000 0000 0000' Test Length Value: 2**20 max loops
x'800009' x'B819 2040 0392 0802' LBIST Clock Event

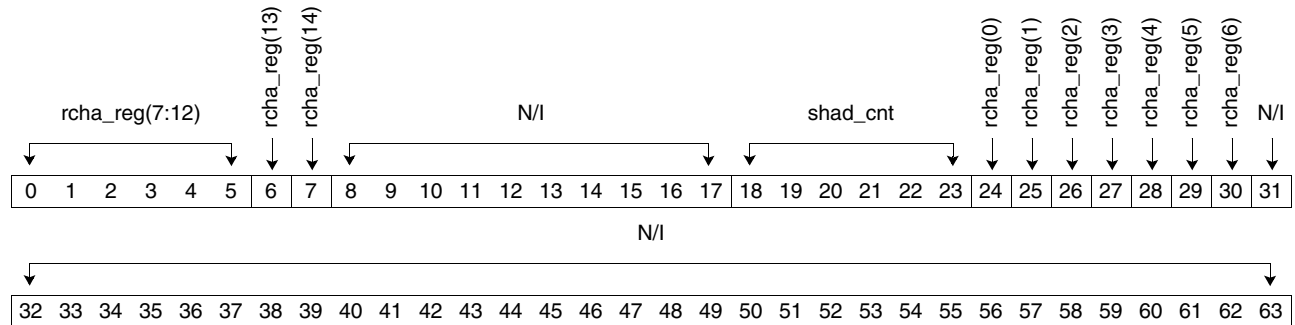
-- Scan / Wait / Clock 2 (First C1 clock is only 1:1) --
x'840002' x'0000 0005 0000 0000' Scan Clock Speed = 5
x'600400' x'0000 0000 0000 0000' N:1 Bus Ratios set to 2:1
x'80000A' x'7800 000E 0000 0000' slow clock=15
x'84000B' x'vvvv v000 0000 0000' Test Length Value: 2**20 max loops
x'800009' x'BA19 2041 0392 0A03' LBIST Clock Event (generic counter=1)

-- Scan / Wait / Clock 2 (N:1 Clocks occur with first 1:1 C1) --
x'840002' x'0000 0005 0000 0000' Scan Clock Speed = 5
x'600400' x'0000 0000 0000 0000' N:1 Bus Ratios set to 2:1
x'80000A' x'7800 000E 0000 0000' slow clock=15
x'84000B' x'vvvv v000 0000 0000' Test Length Value: 2**20 max loops
x'800009' x'BA19 2041 1392 0A03' LBIST Clock Event (generic counter=1)

```

Energy Star Register

Address	x'80000A'
Type	R/W
Reset	Set to x'xx00 0x20 0000 0000' during POR (values marked as x depend on POR sequence).



Bits	Field Name	Description
0:5	rcha_reg(7:12)	Clock rate divide counter (0 to 5). Programmed count value: down counter. Note: Moved here from the Clock Command Control Register due to space limitation.
6	rcha_reg(13)	Enable debug logic on. The initial value is '0'.
7	rcha_reg(14)	Fuse clock control. The initial value is '0'.
8:17	N/I	Not implemented.
18:23	shad_cnt	Shadow counter (<i>Read-Only</i>). The master counter values at the time of the clock freeze on an immediate stop are in the shadow counter. This shadow counter value can be read and then loaded into the master mod48 counter to restart the clocks using Run-N in the event processor exactly where they were stopped.
24	rcha_reg(0)	Enter Energy Star mode.
25	rcha_reg(1)	Exit Energy Star mode.
26	rcha_reg(2)	Configuration shadow stop enable. Allow starting and pulsing clocks based on the saved value for the master phase hold counter. Note: In order for this mode to work, this bit must be written while both the STS and I/O clocks are running. From the POR state, the following sequence must be programmed: 1. x'80_00_00' x'0008_1800_0000_0000' Start <i>both</i> STS and I/O clocks. 2. x'80_00_0A' x'0000_0020_0000_0000' Enable shadow stop. 3. x'80_00_00' x'000C_1800_0000_0000' Stop <i>either</i> STS or I/O clocks. 4. x'80_00_09' x'0000_0000_0F38_1002' Scan0. 5. x'80_00_09' x'0000_0000_0000_0000' Clear array inhibit and receiver inhibit. 6. x'80_00_00' x'0004_D800_0000_0000' Pulse clocks (repeat as needed). ... 7. x'80_00_00' x'0008_D800_0000_0000' Run clocks. Clocks resume at the next, subsequent phase_hold alignment after the last pulse clock.
27	rcha_reg(3)	<i>Unconditional immediate exit</i> from Energy Star mode.
28	rcha_reg(4)	If '0', stops ABIST clocks in core and VMX.
29	rcha_reg(5)	If '0', stops ABIST clocks in STS.

IBM PowerPC 970MP RISC Microprocessor

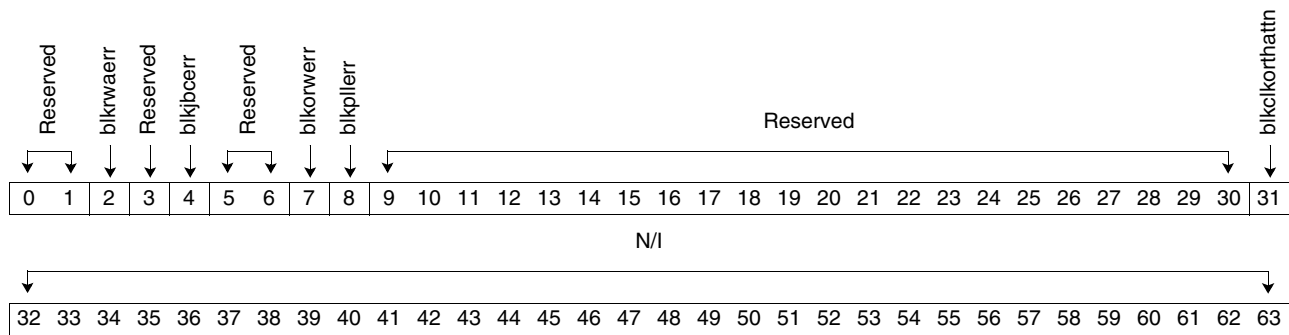
Bits	Field Name	Description
30	rcha_reg(6)	If '0', stops ABIST clocks in scanned ABIST machines. In addition, if '0' this bit prevents scanning through the local clock (lclk) domain.
31	N/I	Not implemented.
32:63	N/I	Not implemented.

Status Register Mask

Address x'80000C'

Type R/W

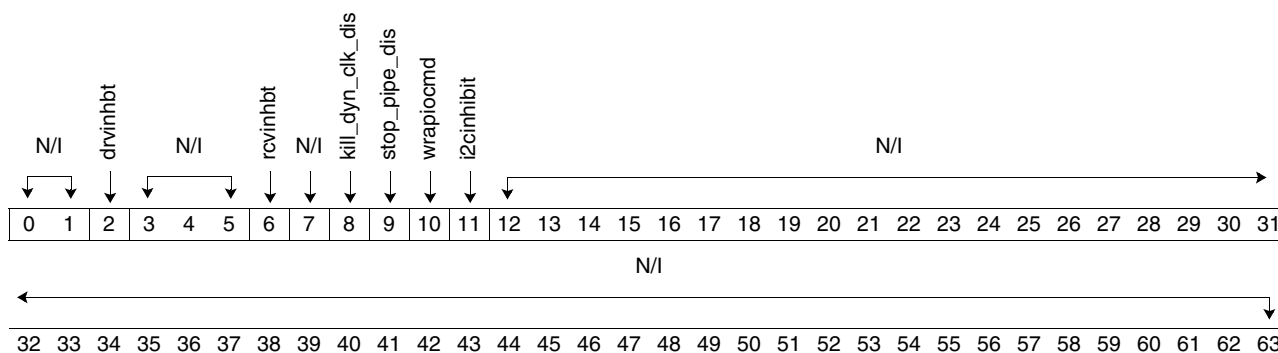
Reset Set to x'0080 0000 0000 0000' during POR.



Bits	Field Name	Description
0:1	Reserved	Reserved.
2	blkwaerr	Block invalid read/write address error.
3	Reserved	Reserved.
4	blkjbcerr	Block JTAG/BIST collision error.
5:6	Reserved	Reserved.
7	blkorwerr	Block Options Register write error.
8	blkplerr	Block PLL lock error.
9:30	Reserved	Reserved.
31	blkclkorthattn	Block clock orthogonality check from generating attention.
32:63	N/I	Not implemented.

IBM PowerPC 970MP RISC Microprocessor
I/O Control Register
Address x'80000F'

Type R/W (Writing this register while I/O is running will raise an attention.)

Reset Set to x'2200 0000 0000 0000' during POR.


Bits	Field Name	Description
Note: The tristate control bits(0:6) determine whether drivers are placed in a high-impedance state. A bit value of '1' places the appropriate drivers in the high-impedance state. These bits should be set before executing LBIST, and are reset to all ones during POR.		
0:1	N/I	Not implemented.
2	drvinhbt	Drivers inhibit.
3:5	N/I	Not implemented.
6	rcvinhbt	Receivers inhibit.
7	N/I	Not implemented.
8	kill_dyn_clk_dis	This bit is asserted to stop the kill_dyn_clk (that is, it asserts the <i>kill_dyn_clk</i> signal). Note: This bit <i>must</i> be zero at all times.
9	stop_pipe_dis	This bit disables the global <i>stop_pipe</i> signal, which is used to enable the stop_ctl to dynamic logic. The <i>stop_pipe</i> signal is asynchronous. Note: This control turns on the clocks to the dynamic, nonscan L1 latches in order to initialize them after SCAN 0 initialization. This bit must be zero at all other times.
10	wrapiocmd	Prevents driver inhibit during the wrap I/O test.
11	i2cinhibit	I ² C inhibit. If set, the I ² C outputs to the JTAG macro are fenced. This bit is set when the LBIST bit, bit 46 in the Clock Command Control Register, is set (not reset when resetting LBIST bit).
12:63	N/I	Not implemented.

ABIST Status Register

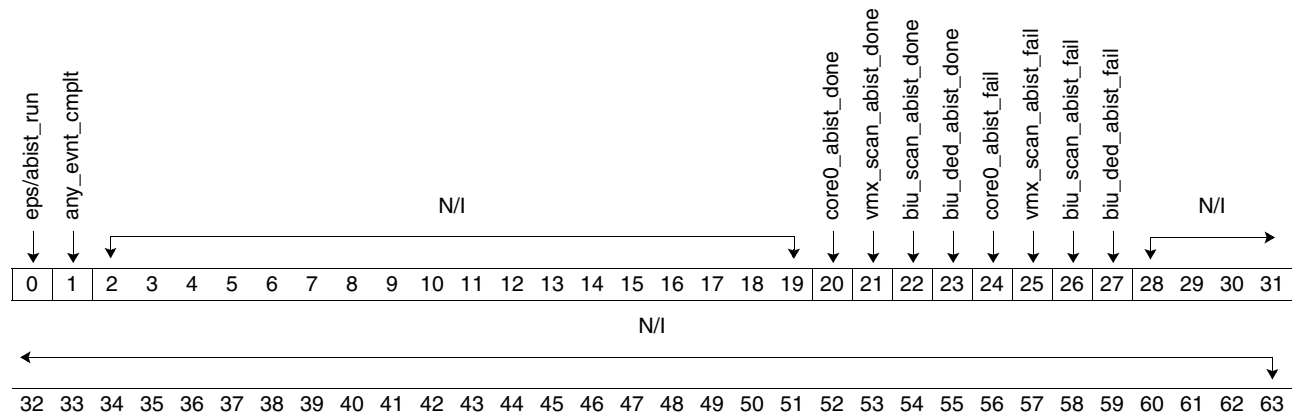
Address x'820004'

Type R/W

Bits 21:26 are unspecified after an LBIST SCAN0.SCAN and should be reinitialized before an ABIST. ABIST fail bits are only valid if the corresponding ABIST done bit is asserted.

Bits in this register are only updated during an EPS controlled operation. Turn on ccintf_local_psav_dis if using ABIST with functional clocks.

Reset Set to x'0000 0000 0000 0000' during POR.



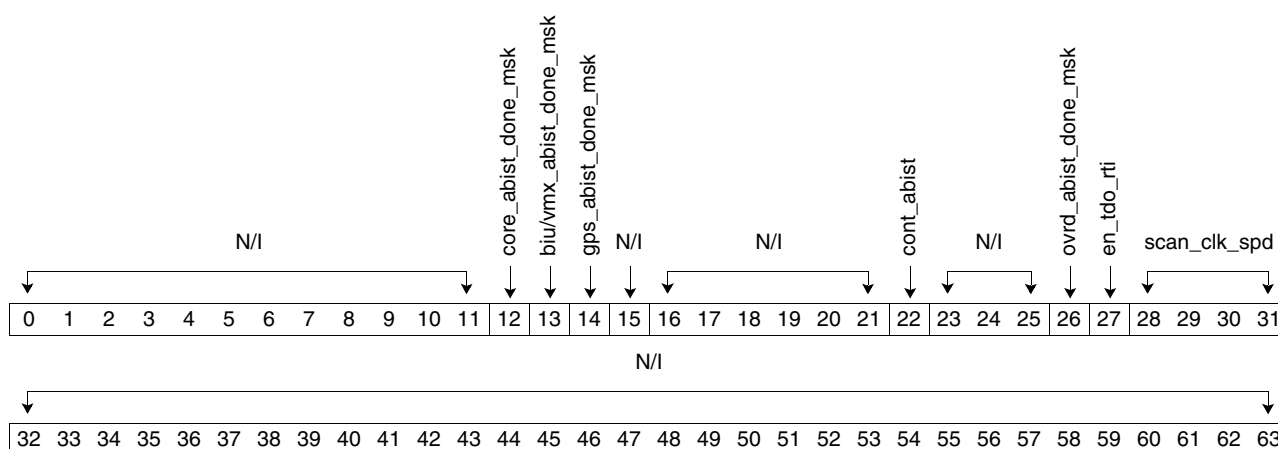
Bits	Field Name	Description
0	eps/abist_run	EPS running, or a decentral ABIST in progress. This is a read-only bit.
1	any_evnt_cmplt	Any event complete. Cleared by running an event. Set and held after completion of event processing. Cleared when bit 52 of the Clock Command Control Register is zero.
2:19	N/I	Not implemented.
20	core0_abist_done	Physical core0 (without VMX) ABIST done.
21	vmx_scan_abist_done	VMX scanned; ABIST done.
22	biu_scan_abist_done	BIU scanned; ABIST done.
23	biu_ded_abist_done	BIU dedicated; ABIST done.
24	core0_abist_fail	Core ABIST Fail. The ABIST Fail is held after being set. To clear this bit with an SCOM write to this register, the abist_out must be cleared first.
25	vmx_scan_abist_fail	VMX scanned; ABIST fail. The ABIST Fail is held after being set. To clear this bit with an SCOM write to this register, abist_out must be cleared first.
26	biu_scan_abist_fail	BIU scanned; ABIST fail. The ABIST Fail is held after being set. To clear this bit with an SCOM write to this register, abist_out must be cleared first.
27	biu_ded_abist_fail	BIU dedicated; ABIST fail. The ABIST Fail is held after being set. To clear this bit with an SCOM write to this register, abist_out must be cleared first.
28:63	N/I	Not implemented.

IBM PowerPC 970MP RISC Microprocessor
LBIST Options Register

Address x'840002'

Type R/W

Reset Set to x'000E 0005 0000 0000' during POR.



Bits	Field Name	Description
0:11	N/I	Not implemented.
12	core_abist_done_msk	Physical core ABIST done mask. Mask ABIST done from the core. (ABIST pass is also masked as a result of the masking.)
13	biu/vmx_abist_done_msk	BIU + VMX scanned ABIST done mask.
14	gps_abist_done_msk	STS ABIST done mask. Mask ABIST done from STS. (ABIST pass is also masked as a result of the masking.)
15:21	N/I	Not implemented.
22	cont_abist	Continual LBIST. Set in order to override the LBIST Test Length Register. Useful for power and noise measurements. Continual LBIST is stopped with a write to the eps_override bit (bit 58) of the <i>Clock Command Register</i> (x'800009').
23:25	N/I	Not implemented.
26	ovrd_abist_done_msk	Override ABIST done mask. The ABIST pass bit is gated by not <i>abist_done</i>. If <i>abist_done</i> is not asserted, then the ABIST pass status cannot be determined. However, by setting this override bit, you can observe the ABIST pass status. Notes: - The <i>abist_fail</i> is '0' until <i>abist_done</i> is set. Once <i>abist_done</i> is set, then <i>abist_fail</i> can be set. - This override effects all three domains: core, scanned ABIST not core, and STS. Programming Note: The <i>gps_scan_abist</i> requires x'FFFF' + 1 number of tester loops to complete. Each tester loop is 131 times the Scan_Clock_Rate number of mesh clocks. Thus, only x'4FFFF' tester loops are run. Therefore, the <i>abist_done</i> status bit will not be set, and the Override ABIST DONE Mask should be set in order to validate the ABIST_FAIL status. The <i>abist_done</i> signal from the GPS_SCAN_ABIST engine (HTBC RLM) should be checked.

IBM PowerPC 970MP RISC Microprocessor

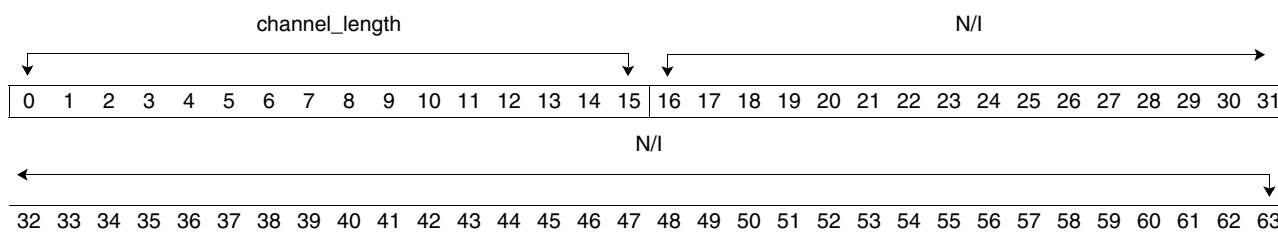
Bits	Field Name	Description
27	en_tdo_rti	Enable TDO during <i>RUN_TEST_IDLE</i> (for ABIST real time fail observation). The bit-fail mapping and array diagnostics for debugging ABIST fails on the tester often need to observe the ABIST fail on a real-time basis. Because the ABIST fail information is sent to <i>ACCESS</i> for status, it is conveniently routed to TDO, which is typically a burn-in pin as well. A special <i>TDO_ENABLE</i> is required to observe this signal. Note: The normal <i>IEEE JTAG Specification</i> has the TDO in a high-impedance state during run-test-idle. This state is then used to enable the <i>TDO_ENABLE</i> during ABIST operations when bit 27 is asserted.
28:31	scan_clk_spd	Scan clock speed. Set to the divide-by value to limit scan clock frequency during scan events on the event processor. This accommodates relaxed timing on the scan paths. Typical applications are flush0, LBIST, and scanned ABIST. The scan clock speed has no effect on system scan-ring access using the 0F access command, which occurs with TCK frequency when in the <i>SHIFT_DR</i> state. Bit 31 is the LSb. - The nominal value is '0101' (that is, 1/6th of the system clock). - A value of '0000' is supported, and can be used for increased simulation throughput. - The value x'0001' is invalid and must not be used.
32:63	N/I	Not implemented.

IBM PowerPC 970MP RISC Microprocessor
LBIST Channel Length Register

Address x'840008'

Type R/W

Reset Set to x'0800 0000 0000 0000' during POR.



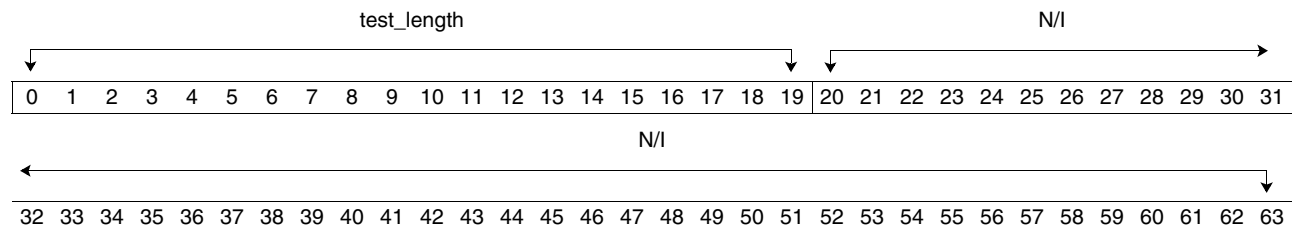
Bits	Field Name	Description
0:15	channel_length	Channel length. Contains the number of cycles <i>minus one</i> the event processor is to loop on a particular event that has the SCAN_CTL bit asserted in the microcoded instructions. Exception: When RAMSTOP_CTL is set coincident with SCAN_CTL, only one scan clock pulse and one ramc2 clock pulse are generated together after a delay of the SCAN_SPEED value from the beginning of the event.
16:63	N/I	Not implemented.

LBIST Test Length Register

Address x'84000B'

Type R/W

Reset Set to x'4000 0000 0000 0000' during POR.



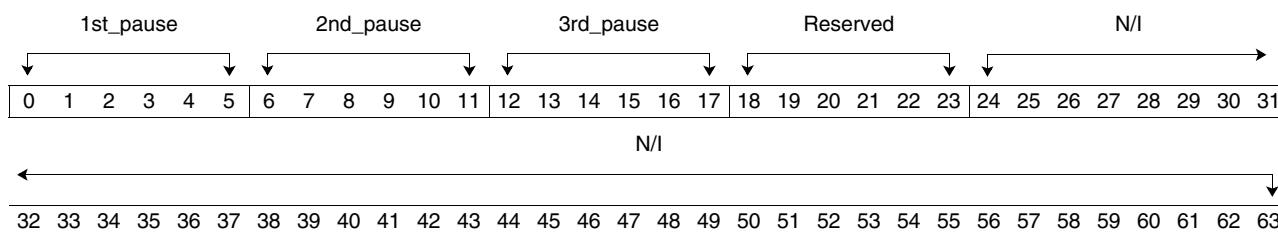
Bits	Field Name	Description
0:19	test_length	Test length. Contains the number of cycles <i>minus one</i> that the event processor is to loop on the micro-coded instructions. For LBIST, it would be the number of pseudo-random test pattern generator (PRPG) loads and system clock loops to run. If loaded to all zeros, then <i>one</i> PRPG load and system clock cycle is run. MISR CLEAR has no overriding effect on the test length. Example: To loop four times, program 3 into this register ('0000000000000000000011'). Note: Do not attempt less than two loops in this register while doing a RUN_N that is using the last event due to logic implementation (pipelining).
20:63	N/I	Not implemented.

IBM PowerPC 970MP RISC Microprocessor
Clock Ramping Configuration Register

Address x'84000D'

Type R/W

Reset Set to x'2D75 E300 0000 0000' during POR.



Bits	Field Name	Description
0:5	1st_pause	First pause in ramping up or down. 5 cycles at f, 2 cycles at f/2, 1 cycle at f/4
6:11	2nd_pause	Second pause in ramping up or down. 21 cycles at f, 10 cycles at f/2, 5 cycles at f/4
12:17	3rd_pause	Third pause in ramping up or down. 26 cycles at f, 13 cycles at f/2, 6 cycles at f/4
18:23	Reserved	Reserved.
24:63	N/I	Not implemented.

13. Vector Processing Unit

The Vector/SIMD technology, referred to in this document as the vector processing unit (VPU), provides a software model that accelerates the performance of various software applications and runs on reduced instruction set computing (RISC) microprocessors. This is a short vector parallel architecture that extends the instruction set architecture (ISA) of the PowerPC Architecture. It is based on separate vector/SIMD¹-style execution units that have high data parallelism. This parallelism allows it to perform on multiple data elements in a single instruction. The term vector refers to the spatial parallel processing of short, fixed-length, one-dimensional matrices performed by an execution unit. It should not be confused with the temporal parallel (pipelined) processing of long, variable-length vectors performed by classical vector machines. High degrees of parallelism are achievable with simple in-order instruction dispatch and low-instruction bandwidth. However, the ISA is designed so as not to impede additional parallelism through superscalar dispatch to multiple execution units or multithreaded execution unit pipelines.

13.1 970MP Vector and SIMD Multimedia Overview

The VPU expands the current PowerPC Architecture through the addition of a 128-bit vector execution unit, which operates concurrently with the existing scalar integer and floating-point units. This new engine provides for highly parallel operations, allowing for the simultaneous execution of up to four 32-bit floating operations or sixteen 8-bit fixed-point operations in one instruction. All VPU data paths and execution units are 128 bits wide and are fully pipelined.

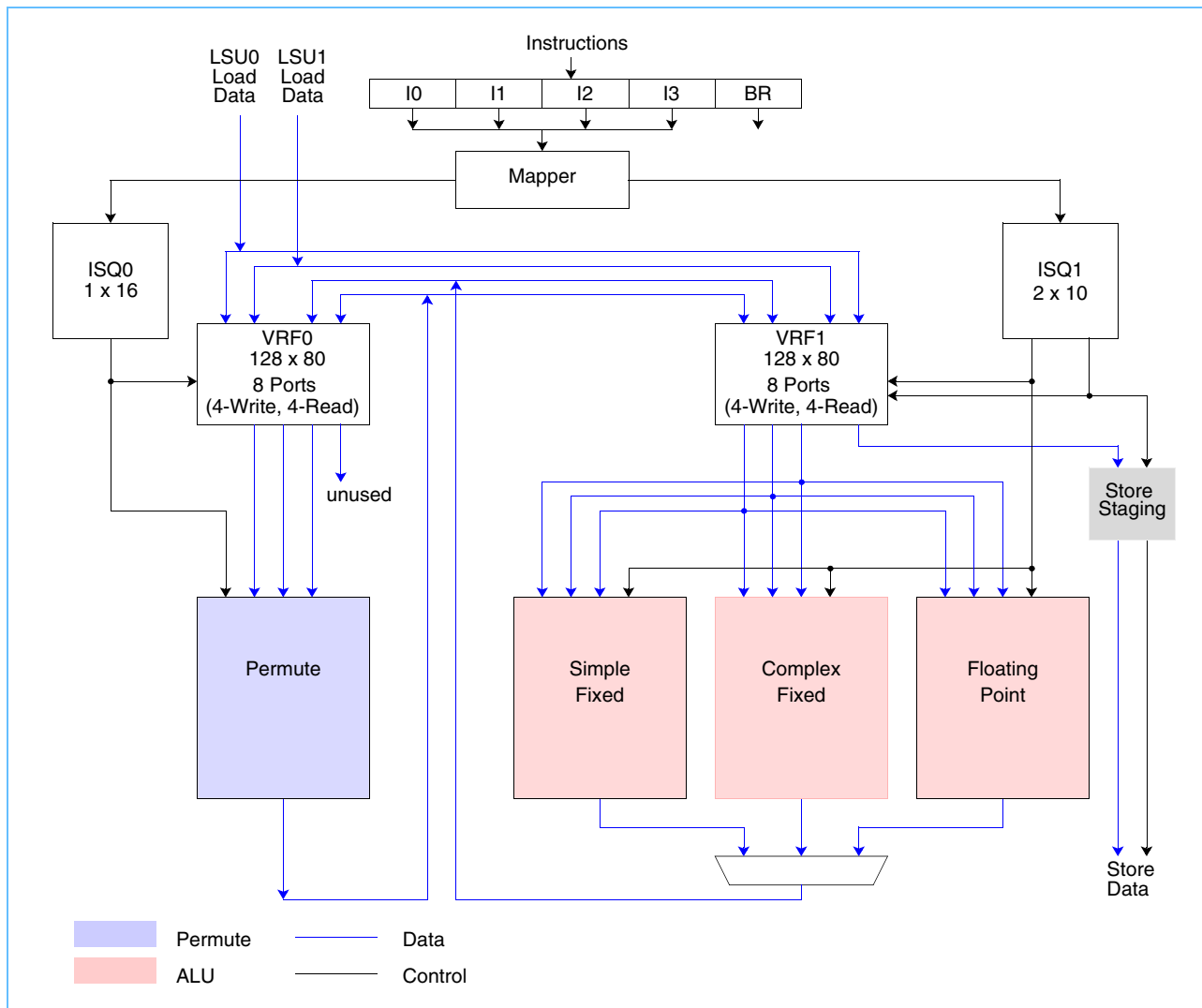
13.1.1 VPU Implementation

The 970MP microprocessor implements many aspects of a preferred implementation as described in the *PowerPC Microprocessor Family: Vector/SIMD Multimedia Extension Technology Programming Environments Manual*. The key features of a preferred implementation include:

- All data paths and execution units are 128 bits wide.
- There are two independent VPU sub-units, one for all arithmetic logic unit (ALU) instructions and one for permute operations.

The VPU is divided into two dispatchable units: vector ALU and vector permute. The vector ALU unit is further subdivided into a vector floating-point unit, a vector simple-fixed unit, and a vector complex-fixed unit. The vector ALU and permute units receive predecoded instructions from the issue queue in the instruction sequencer unit for the VPU (ISV). Vector instructions are issued to the appropriate vector unit when all of the source operands are available. Vector loads, stores, and data stream touch (DST) instructions are executed in the load/store unit (LSU) pipes. There are two copies of the Vector Register files; one provides operands for the vector permute unit, and one provides operands for the vector ALU. *Figure 13-1* on page 402 provides a high-level view of the instruction sequencer unit (ISU) interaction with the VPUs.

1. Single instruction stream, multiple data streams

IBM PowerPC 970MP RISC Microprocessor
Figure 13-1. VPU Block Diagram

13.1.2 Vector ALU

Conceptually, the vector unit ALU is capable of operating on three source vectors and producing a single result vector on each instruction. The ALU is an SIMD-style arithmetic unit, where an instruction performs the same operation on all the data elements that comprise each vector. The ALU is partitioned into four separate ALUs for 32-bit integers and for single-precision floating-point operands. For 16-bit integers, the ALU is partitioned into 8 ALUs, and for 8-bit integers it is partitioned into 16 separate ALUs. No arithmetic is performed on elements larger than 32 bits. The largest adder in the vector ALU is 32 bits wide, and the largest multiplier array is 24 bits wide for the single-precision floating-point mantissa.

13.2 Vector Registers

13.2.1 VRSAVE Register

This 32-bit register is maintained and managed by software only. Each bit in the VRSAVE Register corresponds to a Vector Register and indicates whether the corresponding register is currently being used by the executing process. Therefore, the operating system needs to save and restore only those Vector Registers (VRs) when an exception occurs. The register is handled as a renamed register within the General Purpose Register (GPR) file in the 970MP microprocessor.

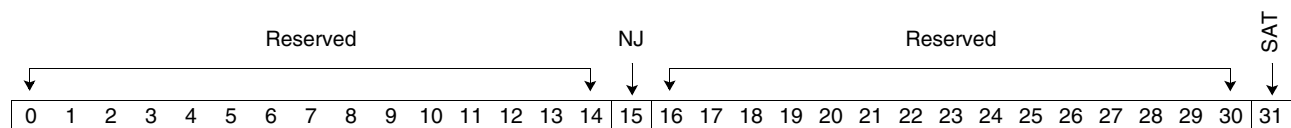
Note: If this approach is taken, it must be applied rigorously. If a program fails to indicate that a given vector register is in use, software errors can occur that will be difficult to detect and correct because they are timing-dependent. Some operating systems save and restore VRSAVE only for programs that also use other vector registers.

The VRSAVE Register can be accessed only by the Move From Special Purpose Register (**mf spr**) or Move To Special Purpose Register (**mt spr**) instructions. The **mf spr** instruction copies VRSAVE to the low-order 32 bits of a GPR; the **mt spr** instruction copies the low-order 32 bits of a GPR to VRSAVE.

13.2.2 Vector Status and Control Register (VSCR)

The Vector Status and Control Register is a special 32-bit register (not an SPR) that is read and written in a manner similar to the Floating-Point Status and Control Register (FPSCR) in the scalar floating-point unit. Two special instructions, Move From Vector Status and Control Register (**mf vscr**) and Move To Vector Status and Control Register (**mt vscr**), are provided to move the VSCR from and to a Vector Register. When moved to or from a Vector Register, the 32-bit VSCR is right justified in the 128-bit Vector Register. When moved to a Vector Register, the upper 96 bits (0:95) of the Vector Register are cleared (set to zeros).

Figure 13-2. VSCR Format



The VSCR has two defined bits, the non-Java mode (NJ) bit (VSCR[15]) and the saturation (SAT) bit (VSCR[31]). The remaining bits are reserved. VSCR bit settings are shown in *Table 13-1* on page 404.

IBM PowerPC 970MP RISC Microprocessor

The VSCR bits, after being moved to a Vector Register, are shown in *Figure 13-3*.

Figure 13-3. VSCR Moved to a Vector Register

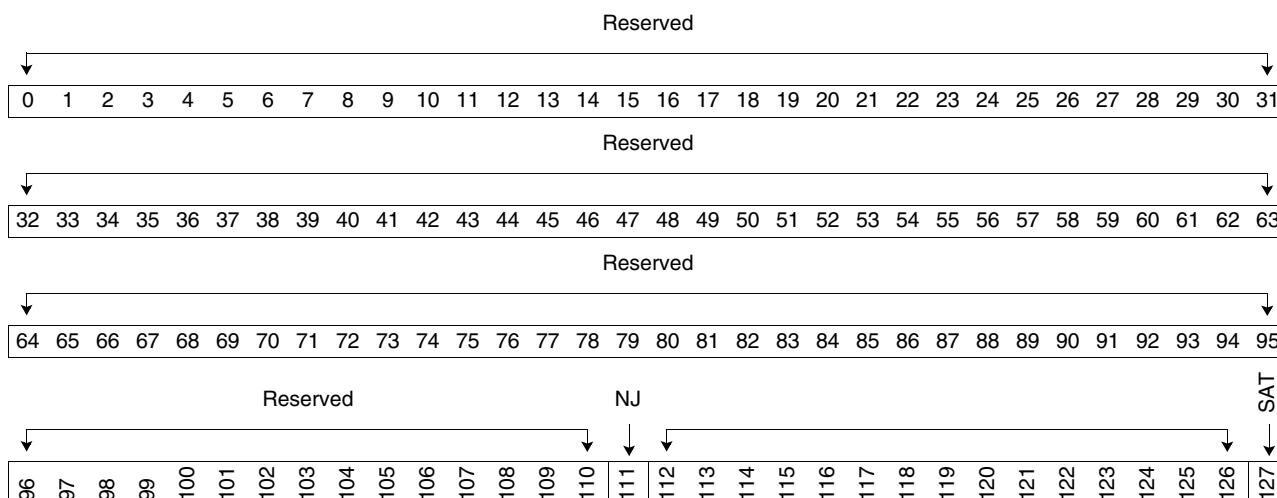


Table 13-1. VSCR Field Descriptions

Bits	Field Name	Description
0:14	—	Reserved.
15	NJ	Non-Java. A mode control bit that determines whether VPU floating-point operations are performed in a mode that is compliant with Java, IEEE, and C9X or in a possibly faster noncompliant mode. 0 The Java-IEEE-C9X-compliant mode is selected. Denormalized values are handled as specified by the Java, IEEE, and C9X standard. 1 The non-Java/non-IEEE-compliant mode is selected. If an element in a source Vector Register contains a denormalized value, the value '0' is used instead. If an instruction causes an underflow exception, the corresponding element in the target Vector Register (VR) is cleared to '0'. In both cases, the '0' has the same sign as the denormalized or underflowing value.
16:30	—	Reserved.
31	SAT	Saturation. A sticky status bit indicating that some field in a saturating instruction became saturated since the last time SAT was cleared. In other words, when SAT is set to '1', it remains set to '1' until it is cleared to '0' by a mtvscr instruction. 0 Indicates no saturation has occurred since this bit was last cleared by a mtvscr instruction. 1 The vector saturate instruction implicitly sets when saturation has occurred on the results of one of the vector instructions having saturate in its name. (See <i>Table 13-4</i> on page 409.)

The **mtvscr** instruction is context synchronizing. This implies that all vector instructions logically preceding an **mtvscr** in the program flow will execute in the architectural context (NJ mode) that existed before completion of the **mtvscr**. It also implies that all instructions logically following the **mtvscr** will execute in the new context (NJ mode) established by the **mtvscr**.

After an **mtvscr** instruction executes, the result in the target Vector Register is architecturally precise. It reflects all updates to the SAT bit that could have been made by vector instructions logically preceding it in the program flow. Further, it will not reflect any SAT updates that can be made to it by vector instructions logically following it in the program flow. Reading the VSCR can be much slower than typical vector instructions, and therefore care must be taken in reading it to avoid performance problems.

13.3 Effects on Existing PowerPC Facilities

13.3.1 Control Flow

Vector instructions can be freely intermixed with existing PowerPC instructions to form a complete program. Vector instructions provide a vector compare and select mechanism to implement conditional execution as the preferred mechanism to control data flow in VPU programs. Vector compare instructions can update the Condition Register, thus providing the communication from the vector execution units to the PowerPC branch instructions necessary to modify program flow based on vector data.

13.3.1.1 Condition Register

The Condition Register (CR) is affected by the VPU architecture. The CR is a 32-bit register, divided into eight 4-bit fields, CR0-CR7 (see *Figure 13-4*), that reflect the results of certain arithmetic operations and provide a mechanism for testing and branching. For the VPU ISA, the CR6 field can optionally be used. If the record bit (Rc) of a vector instruction field is set in a vector compare instruction, then the CR6 field is updated according to *Table 13-2*.

Figure 13-4. Condition Register (CR)

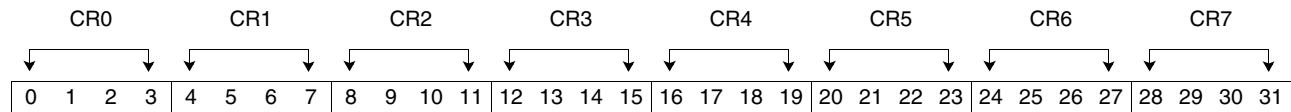


Table 13-2. CR6 Field Bit Settings for Vector Compare Instructions

Bits	Description
Vector Compare	
0	1 Comparison successful for all fields 0 Comparison failed for at least one field
1	Always zero
2	1 Comparison failed for all fields 0 Comparison successful for at least one field
3	Always zero
Vector Compare Bounds (vcmpbfp)	
0	Always zero
1	Always zero
2	1 All values within bounds 0 Not all values within bounds
3	Always zero

The Rc bit should be used sparingly. As for other PowerPC instructions, in some implementations, instructions with the Rc bit set to '1' could have a longer latency or be more disruptive to the instruction pipeline flow than instructions with the Rc bit set to '0'. Therefore, techniques of accumulating results and testing infrequently are advised.

IBM PowerPC 970MP RISC Microprocessor
13.3.1.2 Machine State Register

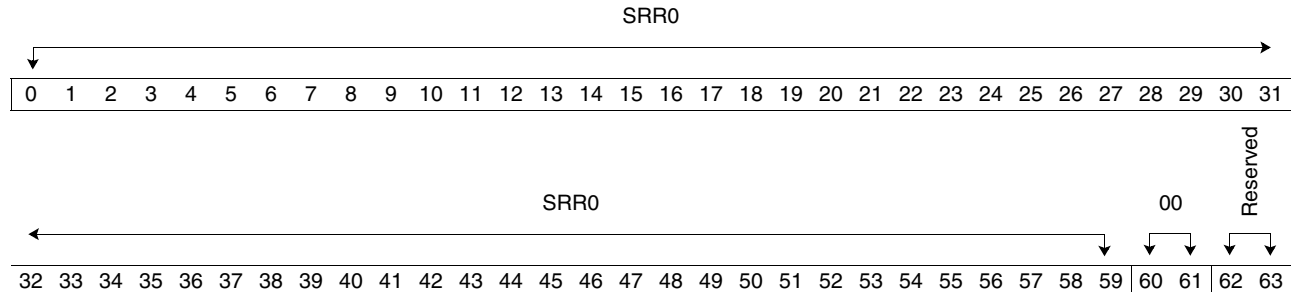
Certain bits in the Machine State Register (MSR) affect instructions in the vector data stream. MSR[VP] indicates whether the vector processor is available. *Table 13-3* defines the VP, PR, and DR bits.

Table 13-3. MSR Bit Settings Affecting the VPU

Bits	Field Name	Description
38	VP	VP available. 0 The processor prevents execution of all vector instructions, including loads, stores, and moves. If such execution is attempted, a VPU unavailable exception is raised. 1 The processor can execute all vector instructions. Note: The VRSAVE Register is not protected by MSR[VP]. The data streaming family of instructions (dst, dstt, dstst, dststt, dss, and dssall) are not affected by the MSR[VP].
49	PR	Problem (user) state. 0 The processor is privileged to execute any instruction. 1 The processor can only execute non-privileged instructions. Note: Care should be taken if data-stream prefetching is used in a supervisor (privileged) state (MSR[PR] is set to '0'). For each existing data stream, prefetching is enabled if (a) MSR[DR] is set to '1' and (b) MSR[PR] has the value it had when the dst or dstst instruction that specified the data stream was executed. Otherwise, prefetching for the data stream is suspended.
59	DR	Data address translation. 0 Data address translation is disabled. If data stream touch (dst) and data stream touch for store (dstst) instructions are executed when DR is set to '0', the results are boundedly undefined. 1 Data address translation is enabled. Data stream touch (dst) and data stream touch for store (dstst) instructions are supported when DR is set to '1'.

13.3.1.3 Machine Status Save/Restore Registers (SRR0, SRR1)

SRR0 holds the effective address (EA) for the instruction that caused the VPU unavailable exception, and SRR1 holds the machine state status as described in *Chapter 4 Exceptions*.



13.4 Exceptions

There are three exceptions that can result from the execution of a vector instruction:

- VPU unavailable exception
- VPU assist exception
- Data storage exception

13.4.1 VPU Unavailable Exception

This interrupt is described in *Section 13.3.1.2 Machine State Register* on page 406.

13.4.2 VPU Assist Exception

The VPU assist exception happens when operating in Java mode and either the input operands or the result of an operation are denormalized. After this exception, execution resumes at offset x'0000 0000 0000 1700'. See *Section 4.5.18 VPU Assist Exception* on page 118 for more information.

13.4.3 Data Storage Exception

Load Vector Indexed and Store Vector Indexed instructions transfer quadword vectors between memory and Vector Registers. Load Vector Element Indexed and Store Vector Element Indexed instructions transfer byte, halfword, and word scalar elements between memory and Vector Registers. All vector loads and vector stores use the index ($rA10 + rB$) addressing mode to specify the target memory address. No update forms are provided. A Load Vector Element Indexed instruction transfers a scalar data element from memory into the destination Vector Register, leaving other elements in the vector with boundedly-undefined values. A Store Vector Element Indexed transfers a scalar data element from the source Vector Register to memory leaving other elements in the quadword unchanged. No data alignment occurs; that is, all scalar data elements are transferred directly on their natural memory byte-lanes to or from the corresponding element in the Vector Register. Quadword memory accesses made by Load Vector Indexed and Store Vector Indexed are not guaranteed to be atomic. Direct-store segments (where T equals '1') are not supported. Any vector load or store that attempts to access a direct-store segment will cause a data storage exception (DSI).

13.5 Optional Instructions

The 970MP microprocessor implements all vector instructions as listed in the *PowerPC Microprocessor Family: Vector/SIMD Multimedia Extension Technology Programming Environments Manual*. See *Table 13-4 Supported Vector Instructions* on page 409.

13.5.1 Java Mode Instruction Handling Implementation

The 970MP VPU implementation handles certain instructions differently based on the Java mode setting in the VSCR. Java compliance does require compliance with certain aspects of the IEEE Standard including:

- Support of denorms as inputs and results (gradual underflow) for arithmetic operations
- Not a number (NaN) results for invalid operations
- NaNs compare unordered with respect to everything, so that the result of any comparison of any NaN to any data type is always false
- NaNs are handled the same way in both the Java or non-Java mode for the 970MP implementation.

For some instructions, denormalization produces the exact result without trapping. The 970MP implementation of the VPU handles most denormalization by trapping at interrupt vector x'0000 0000 0000 1700' (VPU assist interrupt).

13.5.2 Least Recently Used Instructions

The Vector/SIMD Architecture suggests that Load Vector Indexed LRU (**lvxl**) and Store Vector Indexed LRU (**stvxl**) are handled differently than the regular load/store instructions in that they leave cache entries in the least recently used (LRU) state instead of a most recently used (MRU) state. This supports efficient processing of data that is known to have little reuse and poor caching characteristics.

The 970MP microprocessor will treat **lvxl** and **stvxl** as a regular load and store with respect to the replacement algorithm. That is, the cache entry will be set as MRU.

13.5.3 Data Stream Instructions

DST instructions are broken up into two internal instructions (IOPs), one for the effective address and one for the prefetch information. They are marked serialized and are not executed until they are the next instruction to complete. Additional information can be found in *Section 3.5.3 Data Prefetch* on page 96.

13.6 Vector Instruction Set

Table 13-4 lists the supported vector instructions.

Table 13-4. Supported Vector Instructions (Page 1 of 7)

Number	Mnemonic	Operands	Execution Unit	Description
Load Vector Element Indexed				
1	lvebx	vD,rA,rB	LOAD	Load Vector Element Byte Indexed
2	lvehx	vD,rA,rB	LOAD	Load Vector Element Halfword Indexed
3	lviewx	vD,rA,rB	LOAD	Load Vector Element Word Indexed
4	lvx	vD,rA,rB	LOAD	Load Vector Indexed
5	lvxli	vD,rA,rB	LOAD	Load Vector Indexed LRU
Store Vector Element Indexed				
6	stvebx	vS,rA,rB	STORE	Store Vector Element Byte Indexed
7	stvehx	vS,rA,rB	STORE	Store Vector Element Halfword Indexed
8	stviewx	vS,rA,rB	STORE	Store Vector Element Word Indexed
9	stvx	vS,rA,rB	STORE	Store Vector Indexed
10	stvxli	vS,rA,rB	STORE	Store Vector Indexed LRU
Load Vector for Shift				
11	lvsl	vD,rA,rB	LOAD	Load Vector for Shift Left
12	lvslr	vD,rA,rB	LOAD	Load Vector for Shift Right
Move To and Move From Vector Status and Control Register				
13	mtvscr	vB	Simple	Move To Vector Status and Control Register
14	mfvsr	vD	Simple	Move From Vector Status and Control Register
Data Stream				
15	dst	rA,rB,tag	LSU	Data Stream Touch
16	dstt	rA,rB,tag	LSU	Data Stream Touch Transient (treated as dst)
17	dstst	rA,rB,tag	LSU	Data Stream Touch for Store (treated as dst)
18	dststt	rA,rB,tag	LSU	Data Stream Touch for Store Transient (treated as dst)
19	dss	tag	LSU	Data Stream Stop
20	dssall		LSU	Data Stream Stop All
Vector Add				
21	vaddubm	vD,vA,vB	Simple	Vector Add Unsigned Byte Modulo
22	vaddubs	vD,vA,vB	Simple	Vector Add Unsigned Byte Saturate
23	vaddsbs	vD,vA,vB	Simple	Vector Add Signed Byte Saturate
24	vadduhm	vD,vA,vB	Simple	Vector Add Unsigned Halfword Modulo
25	vadduhs	vD,vA,vB	Simple	Vector Add Unsigned Halfword Saturate
26	vaddshs	vD,vA,vB	Simple	Vector Add Signed Halfword Saturate
27	vadduwm	vD,vA,vB	Simple	Vector Add Unsigned Word Modulo

IBM PowerPC 970MP RISC Microprocessor

Table 13-4. Supported Vector Instructions (Page 2 of 7)

Number	Mnemonic	Operands	Execution Unit	Description
28	vadduws	vD,vA,vB	Simple	Vector Add Unsigned Word Saturate
29	vaddsws	vD,vA,vB	Simple	Vector Add Signed Word Saturate
30	vaddfp	vD,vA,vB	Float	Vector Add Float
Vector Add and Write Carry-Out				
31	vaddcuw	vD,vA,vB	Simple	Vector Add and Write Carry-Out Unsigned Word
Vector Subtract				
32	vsububm	vD,vA,vB	Simple	Vector Subtract Unsigned Byte Modulo
33	vsububs	vD,vA,vB	Simple	Vector Subtract Unsigned Byte Saturate
34	vsubsbs	vD,vA,vB	Simple	Vector Subtract Signed Byte Saturate
35	vsubuhm	vD,vA,vB	Simple	Vector Subtract Unsigned Halfword Modulo
36	vsubuhs	vD,vA,vB	Simple	Vector Subtract Unsigned Halfword Saturate
37	vsubshs	vD,vA,vB	Simple	Vector Subtract Signed Halfword Saturate
38	vsubuwm	vD,vA,vB	Simple	Vector Subtract Unsigned Word Modulo
39	vsubuws	vD,vA,vB	Simple	Vector Subtract Unsigned Word Saturate
40	vsubsws	vD,vA,vB	Simple	Vector Subtract Signed Word Saturate
41	vsubfp	vD,vA,vB	Float	Vector Subtract Float
Vector Subtract and Write Carry-Out				
42	vsubcuw	vD,vA,vB	Simple	Vector Subtract and Write Carry-Out Unsigned Word
Vector Multiply Odd Integer				
43	vmuloub	vD,vA,vB	Complex	Vector Multiply Odd Unsigned Byte
44	vmulosb	vD,vA,vB	Complex	Vector Multiply Odd Signed Byte
45	vmulouh	vD,vA,vB	Complex	Vector Multiply Odd Unsigned Halfword
46	vmulosh	vD,vA,vB	Complex	Vector Multiply Odd Signed Halfword
Vector Multiply Even Integer				
47	vmuleub	vD,vA,vB	Complex	Vector Multiply Even Unsigned Byte
48	vmulesb	vD,vA,vB	Complex	Vector Multiply Even Signed Byte
49	vmuleuh	vD,vA,vB	Complex	Vector Multiply Even Unsigned Halfword
50	vmulesh	vD,vA,vB	Complex	Vector Multiply Even Signed Halfword
Vector Multiply-Add				
51	vmhaddshs	vD,vA,vB,vC	Complex	Vector Multiply-High and Add Signed Halfword Saturate
52	vmhraddshs	vD,vA,vB,vC	Complex	Vector Multiply-High Round and Add Signed Halfword Saturate
53	vmladduhm	vD,vA,vB,vC	Complex	Vector Multiply-Low and Add Unsigned Halfword Modulo
54	vmaddfp	vD,vA,vC,vB	Float	Vector Multiply-Add Float

Table 13-4. Supported Vector Instructions (Page 3 of 7)

Number	Mnemonic	Operands	Execution Unit	Description
Vector Multiply-Sum Integer				
55	vmsumubm	vD,vA,vB,vC	Complex	Vector Multiply-Sum Unsigned Byte Modulo
56	vmsummbm	vD,vA,vB,vC	Complex	Vector Multiply-Sum Mixed-Sign Byte Modulo
57	vmsumuhm	vD,vA,vB,vC	Complex	Vector Multiply-Sum Unsigned Halfword Modulo
58	vmsumuhs	vD,vA,vB,vC	Complex	Vector Multiply-Sum Unsigned Halfword Saturate
59	vmsumshm	vD,vA,vB,vC	Complex	Vector Multiply-Sum Signed Halfword Modulo
60	vmsumshs	vD,vA,vB,vC	Complex	Vector Multiply-Sum Signed Halfword Saturate
Vector Sum Across Signed Integer Saturate				
61	vsumsws	vD,vA,vB	Complex	Vector Sum Across Signed Word Saturate
Vector Sum Across Partial (1/2) Signed Integer Saturate				
62	vsum2sws	vD,vA,vB	Complex	Vector Sum Across Partial (1/2) Signed Word Saturate
Vector Sum Across Partial (1/4) Integer Saturate				
63	vsum4ubs	vD,vA,vB	Complex	Vector Sum Across Partial (1/4) Unsigned Byte Saturate
64	vsum4sbs	vD,vA,vB	Complex	Vector Sum Across Partial (1/4) Signed Byte Saturate
65	vsum4shs	vD,vA,vB	Complex	Vector Sum Across Partial (1/4) Signed Halfword Saturate
Vector Average Integer				
66	vavgub	vD,vA,vB	Simple	Vector Average Unsigned Byte
67	vavgub	vD,vA,vB	Simple	Vector Average Signed Byte
68	vavguh	vD,vA,vB	Simple	Vector Average Unsigned Halfword
69	vavgsh	vD,vA,vB	Simple	Vector Average Signed Halfword
70	vavguw	vD,vA,vB	Simple	Vector Average Unsigned Word
71	vavgsw	vD,vA,vB	Simple	Vector Average Signed Word
Vector Logical				
72	vand	vD,vA,vB	Simple	Vector Logical AND
73	vor	vD,vA,vB	Simple	Vector Logical OR
74	vxor	vD,vA,vB	Simple	Vector Logical XOR
75	vandc	vD,vA,vB	Simple	Vector Logical AND with Complement
76	vnor	vD,vA,vB	Simple	Vector Logical NOR
Vector Rotate Left Integer				
77	vrlb	vD,vA,vB	Simple	Vector Rotate Left Integer Byte
78	vrlh	vD,vA,vB	Simple	Vector Rotate Left Integer Halfword
79	vrlw	vD,vA,vB	Simple	Vector Rotate Left Integer Word

IBM PowerPC 970MP RISC Microprocessor

Table 13-4. Supported Vector Instructions (Page 4 of 7)

Number	Mnemonic	Operands	Execution Unit	Description
Vector Shift Left Integer				
80	vslb	vD, vA, vB	Simple	Vector Shift Left Integer Byte
81	vslh	vD, vA, vB	Simple	Vector Shift Left Integer Halfword
82	vslw	vD, vA, vB	Simple	Vector Shift Left Integer Word
83	vsl	vD, vA, vB	Simple	Vector Shift Left
Vector Shift Right Integer				
84	vsrb	vD, vA, vB	Simple	Vector Shift Right Byte
85	vsrab	vD, vA, vB	Simple	Vector Shift Right Algebraic Byte
86	vsrh	vD, vA, vB	Simple	Vector Shift Right Halfword
87	vsrah	vD, vA, vB	Simple	Vector Shift Right Algebraic Halfword
88	vsrw	vD, vA, vB	Simple	Vector Shift Right Word
89	vsraw	vD, vA, vB	Simple	Vector Shift Right Algebraic Word
90	vsr	vD, vA, vB	Simple	Vector Shift Right
Vector Compare Greater-Than				
91	vcmpgtub[.]	vD, vA, vB	Simple	Vector Compare Greater-Than Unsigned Byte [Record]
92	vcmpgtsb[.]	vD, vA, vB	Simple	Vector Compare Greater-Than Signed Byte [Record]
93	vcmpgtuh[.]	vD, vA, vB	Simple	Vector Compare Greater-Than Unsigned Halfword [Record]
94	vcmpgtsh[.]	vD, vA, vB	Simple	Vector Compare Greater-Than Signed Halfword [Record]
95	vcmpgtuw[.]	vD, vA, vB	Simple	Vector Compare Greater-Than Unsigned Word [Record]
96	vcmpgtsw[.]	vD, vA, vB	Simple	Vector Compare Greater-Than Signed Word [Record]
97	vcmpgtfp[.]	vD, vA, vB	Simple	Vector Compare Greater-Than Float [Record]
Vector Compare Equal-To				
98	vcmpqub[.]	vD, vA, vB	Simple	Vector Compare Equal-To Unsigned Byte [Record]
99	vcmpquh[.]	vD, vA, vB	Simple	Vector Compare Equal-To Unsigned Halfword [Record]
100	vcmpquw[.]	vD, vA, vB	Simple	Vector Compare Equal-To Unsigned Word [Record]
101	vcmpqfp[.]	vD, vA, vB	Simple	Vector Compare Equal-To Float [Record]
Vector Compare Greater-Than-or-Equal-To				
102	vcmpgefp[.]	vD, vA, vB	Simple	Vector Compare Greater-Than-or-Equal-To Float [Record]
Vector Compare Bounds Float				
103	vcmpbfp[.]	vD, vA, vB	Simple	Vector Compare Bounds Float [Record]
Vector Conditional Select				
104	vsel	vD, vA, vB, vC	Simple	Vector Conditional Select
Vector Pack				
105	vpkuhum	vD, vA, vB	Permute	Vector Pack Unsigned Halfword Unsigned Modulo
106	vpkuhus	vD, vA, vB	Permute	Vector Pack Unsigned Halfword Unsigned Saturate
107	vpkshus	vD, vA, vB	Permute	Vector Pack Signed Halfword Unsigned Saturate

Table 13-4. Supported Vector Instructions (Page 5 of 7)

Number	Mnemonic	Operands	Execution Unit	Description
108	vpkshss	vD,vA,vB	Permute	Vector Pack Signed Halfword Signed Saturate
109	vpkuwum	vD,vA,vB	Permute	Vector Pack Unsigned Word Unsigned Modulo
110	vpkuwus	vD,vA,vB	Permute	Vector Pack Unsigned Word Unsigned Saturate
111	vpkswus	vD,vA,vB	Permute	Vector Pack Signed Word Unsigned Saturate
112	vpkswss	vD,vA,vB	Permute	Vector Pack Signed Word Signed Saturate
113	vpkpx	vD,vA,vB	Permute	Vector Pack Pixel32
Vector Unpack High				
114	vupkhsb	vD,vB	Permute	Vector Unpack High Signed Byte
115	vupkhsh	vD,vB	Permute	Vector Unpack High Signed Halfword
116	vupkhpX	vD,vB	Permute	Vector Unpack High Pixel16
Vector Unpack Low				
117	vupklb	vD,vB	Permute	Vector Unpack Low Signed Byte
118	vupklsh	vD,vB	Permute	Vector Unpack Low Signed Halfword
119	vupklpx	vD,vB	Permute	Vector Unpack Low Pixel16
Vector Merge High				
120	vmrghb	vD,vA,vB	Permute	Vector Merge High Byte
121	vmrghh	vD,vA,vB	Permute	Vector Merge High Halfword
122	vmrghw	vD,vA,vB	Permute	Vector Merge High Word
Vector Merge Low				
123	vmrglb	vD,vA,vB	Permute	Vector Merge Low Byte
124	vmrglh	vD,vA,vB	Permute	Vector Merge Low Halfword
125	vmrglw	vD,vA,vB	Permute	Vector Merge Low Word
Vector Splat				
126	vspltb	vD,vB,UIM	Permute	Vector Splat Byte
127	vsplth	vD,vB,UIM	Permute	Vector Splat Halfword
128	vspltw	vD,vB,UIM	Permute	Vector Splat Word
Vector Splat Immediate Signed Integer				
129	vspltisb	vD,SIM	Permute	Vector Splat Immediate Signed Byte
130	vspltish	vD,SIM	Permute	Vector Splat Immediate Signed Halfword
131	vspltisw	vD,SIM	Permute	Vector Splat Immediate Signed Word
Vector Permute				
132	vperm	vD,vA,vB,vC	Permute	Vector Permute
Vector Shift Left Double by Octet Immediate				
133	vsldoi	vD,vA,vB,SH	Permute	Vector Shift Left Double by Octet Immediate

IBM PowerPC 970MP RISC Microprocessor

Table 13-4. Supported Vector Instructions (Page 6 of 7)

Number	Mnemonic	Operands	Execution Unit	Description
Vector Shift by Octet				
134	vslo	vD,vA,vB	Permute	Vector Shift Left by Octet
135	vsro	vD,vA,vB	Permute	Vector Shift Right by Octet
Vector Maximum				
136	vmaxub	vD,vA,vB	Simple	Vector Maximum Unsigned Byte
137	vmaxsb	vD,vA,vB	Simple	Vector Maximum Signed Byte
138	vmaxuh	vD,vA,vB	Simple	Vector Maximum Unsigned Halfword
139	vmaxsh	vD,vA,vB	Simple	Vector Maximum Signed Halfword
140	vmaxuw	vD,vA,vB	Simple	Vector Maximum Unsigned Word
141	vmaxsw	vD,vA,vB	Simple	Vector Maximum Signed Word
142	vmaxfp	vD,vA,vB	Simple	Vector Maximum Float
Vector Minimum				
143	vminub	vD,vA,vB	Simple	Vector Minimum Unsigned Byte
144	vminsb	vD,vA,vB	Simple	Vector Minimum Signed Byte
145	vminuh	vD,vA,vB	Simple	Vector Minimum Unsigned Halfword
146	vminsh	vD,vA,vB	Simple	Vector Minimum Signed Halfword
147	vminuw	vD,vA,vB	Simple	Vector Minimum Unsigned Word
148	vminsw	vD,vA,vB	Simple	Vector Minimum Signed Word
149	vminfp	vD,vA,vB	Simple	Vector Minimum Float
Vector Estimate Float				
150	vrefp	vD,vB	Float	Vector Reciprocal Estimate Float
151	vrsqrtefp	vD,vB	Float	Vector Reciprocal Square Root Estimate Float
152	vlogefp	vD,vB	Float	Vector Log 2 Estimate Float
153	vexpftefp	vD,vB	Float	Vector 2 Raised to the Exponent Estimate Float
Vector Negative Multiply-Subtract Float				
154	vnmsubfp	vD,vA,vC,vB	Float	Vector Negative Multiply-Subtract Float
Vector Round to Floating-Point Integral Value				
155	vrfi	vD,vB	Float	Vector Round to Floating-Point Integer Nearest
156	vrfi	vD,vB	Float	Vector Round to Floating-Point Integer toward Zero
157	vrfi	vD,vB	Float	Vector Round to Floating-Point Integer toward Positive infinity
158	vrfi	vD,vB	Float	Vector Round to Floating-Point Integer toward Minus infinity
Vector Convert To Fixed-Point				
159	vctuxs	vD,vB,UIM	Float	Vector Convert to Unsigned Fixed-Point Word Saturate
160	vctxs	vD,vB,UIM	Float	Vector Convert to Signed Fixed-Point Word Saturate

*Table 13-4. Supported Vector Instructions (Page 7 of 7)*

Number	Mnemonic	Operands	Execution Unit	Description
Vector Convert From Fixed-point				
161	vcfux	vD,vB,UIM	Float	Vector Convert From Unsigned Fixed-Point Word
162	vcfsx	vD,vB,UIM	Float	Vector Convert From Signed Fixed-Point Word